Interlens: Framework for Multi-Agent Interaction and Interpretability

Siddharth M. Bhatia
July 3, 2026

Contents

Interlens: Framework for Multi-Agent Interaction and Interpretability 64
- Install 64
  - PyTorch / CUDA note 64
- What's inside 65
- Develop 65
- License 65

Examples 66
- What this library does 66
- One object: recipe = live dialogue = rollout driver 66
- Index 67

01 · Quickstart 69
- What just happened 69
- Two *different* models 70
- One-off generation without committing 70

02 · Conversations in depth 71
- Build participants and a conversation manually 71
  - Framing ownership (who sees what) 72
- Turn-taking 72
- Stopping 72
- Branching — fork a conversation for free 73
  - Fork from a specific turn — `branch_from` 73

Editing history in place — rewind, edit, reset 74
Editing history and the KV cache 75
Ephemeral sampling — read state without mutating it 75
Reasoning visibility (CoT models) 75
08 · Rollouts & scale 77
`conv.rollout` — many runs of one recipe 77
Data-driven rollout — one conversation per dataset row 78
`interlens.run` — many lineups in one pool 79
`analyzer` — measure each run while models are resident 80
Multi-GPU: register analyzers by name 80
Matched compute — `TokenBudget` 80
Large API rollouts (batch mode) 81
Execution modes & reproducibility 82
04 · Context management & serialization 83
Context-window policies 83
Serialization: recipe + transcript 83
Save / load a whole conversation (recipe + transcript) 84
03 · Participants & models 86
How models resolve 86
Load weights directly 87
Models too big for one GPU 87
`ModelParticipant` knobs 88
`kv_reuse` (cross-turn KV cache) 89
API-backed participants 89
Batch mode for large rollouts 91
Per-turn `max_new_tokens` 91
05 · Tools 92
Define and register a tool 92
Give a participant tools 93
Live participant 93
On a participant 93
What the loop does 93
06 · Message hooks 95
Interface 95
Attach hooks 95
An LLM-judge hook 96
07 · Interpretability 97
Activation capture 97
Build a probe direction from captured activations 98
Steering — add or ablate a direction 98
Activation patching — cross-branch causal tracing 99
Token logprobs / surprisal / entropy 99
09 · Advanced interp pipelines 101
1 · Difference-of-means direction → causal steering test 101

2 · Activation patching across clean/corrupt branches (causal tracing) 102
3 · Probe-in-the-loop `analyze` at scale 102
4 · Steering sweep with reproducible turns 103
10 · Arena: scoreable multi-agent evaluations 105
Run an episode 105
Arms, budgets, and matched compute 106
Situational sweeps 107
Two hard-won gotchas, now guarded 107
Preflight gates for local rollouts 107
The second scenario wave 108
Security dilemma (`SecurityDilemma`, no solo arm) 108
Coding collaboration (`CodingCollab`) 108
Distributed long-context (`DistributedLongContext` + task adapters) 108
Adaptive difficulty ratchet 109
Rollout sets: more episodes, into the same place, safely 110
Two gates, because each is blind to the other's failure 111
Replay and re-scoring 113
Formal actions and per-turn oracles 113
Communication styles 115
Inspect integration (optional) 115
11 · Scorable negotiation & rational oracles 117
The game: issues, score sheets, `GameSpec` 117
Swapping the game: presets 118
Exact solutions & descriptors 119
Rational oracles 120
Playing it: `ScorableNegotiation` 121
Rational agents: `PolicyParticipant` + the strategy zoo 122
Measuring divergence 123
Transcripts & Inspect 123
12 · Control Tower: use an Interlens local model as an untrusted policy 125
Install 125
Define the participant 125
Run a Control Tower protocol 126
Programmatic use and interpretability 127
Attack and collaborative policies 127
API Reference 129
interlens 143
Modules 143
Re-exported 144
arena 156
Modules 156
Re-exported 158

actions 166
Attributes 166
Classes 167
Functions 168
Accept 169
Attributes 169
Methods 169
to_json 169
Action 170
Attributes 170
Methods 170
to_json 170
Offer 171
Attributes 171
Methods 172
from_json 172
to_json 172
OfferRegistry 173
Attributes 173
Methods 173
accept 173
apply 174
from_json 174
get 175
is_live 175
register 175
reject 176
standing 177
standing_ids 177
to_json 177
withdraw 178
ParseResult 179
Attributes 179
Methods 180
bad 180
good 180
retry_directive 181
Pass 182
Attributes 182
Propose 183
Attributes 183

Methods 183
to_json 183
Reject 184
Attributes 184
Methods 184
to_json 184
Walk 185
Attributes 185
action_from_json 186
action_from_key 187
action_key 188
action_message 189
parse_action 191
auction 193
Modules 193
actions 195
Attributes 196
Classes 196
Functions 198
Bid 199
Attributes 199
Methods 199
to_json 199
BidLedger 200
Parameters 200
Attributes 200
Methods 201
active_seats 201
apply 201
eligible 202
from_json 202
stage_bids 203
standing 203
standing_prices 203
standing_winners 204
to_json 204
Claim 205
Attributes 205
DMRouter 206
Parameters 206
Attributes 206

Methods 207
from_json 207
graph 207
inbox 208
route 208
to_json 209
Demand 210
Attributes 210
Methods 210
to_json 210
DirectMessage 211
Attributes 211
Methods 211
to_json 211
DirectMessageRecord 212
Attributes 213
Methods 213
from_json 213
to_json 213
Exit 214
Attributes 214
PassLot 215
Attributes 215
Methods 215
to_json 215
SAATurn 216
Attributes 216
Methods 217
to_json 217
Schedule 218
Attributes 218
Methods 218
to_json 218
Speak 219
Attributes 219
Methods 219
to_json 219
StandingBid 220
Attributes 220
Methods 221
from_json 221

to_json 221
Stay 222
Attributes 222
Transfer 223
Attributes 224
Methods 224
to_json 224
TransferBook 225
Attributes 225
Methods 225
declare 225
from_json 226
settle 226
to_json 227
TurnEnvelope 228
Attributes 228
Wait 230
Attributes 230
auction_action_from_json 231
parse_auction_action 232
Parameters 232
parse_envelope 234
allocation 235
Classes 236
Functions 236
Allocation 238
Attributes 238
Methods 238
bundle 238
empty 239
from_json 239
to_json 239
winners 240
ValueModel 241
Parameters 241
Attributes 242
Methods 242
budget_feasible 242
bundle_value 243
efficient_allocation 243
from_spec 244

max_welfare 244
welfare 244
welfare_without 245
brute_force_allocation 246
clinching_prices 247
max_weight_assignment 248
sealed_single_outcome 249
uniform_price_clear 250
vcg_payments 251
benchmarks 252
Classes 253
Functions 253
Benchmark 256
Attributes 256
Attributes 257
best_bundle_at_prices 258
clinching_benchmark 259
expected_value_given_winning 260
marginal_value_schedule 261
rival_max_cdf_curve 262
rnne_bid_against 263
rnne_shade 264
rnne_symmetric_bid 265
saa_competitive_benchmark 266
saa_onpath_benchmark 268
Parameters 269
stage_benchmark 270
Parameters 270
truthful_benchmark 271
uniform_price_benchmark 273
bidders 274
Attributes 275
Classes 275
Functions 276
AuctionPolicy 277
Parameters 277
Attributes 277
Methods 278
act 278
bid_for 278

declaration 279
evaluate_proposal 279
expected_surplus_at 280
initiate_proposal 280
respond_to_dm 281
schedule 281
AuctionState 283
Attributes 283
Attributes 285
Methods 286
from_spec 286
value_model 286
ConditionalBayesPolicy 288
Attributes 289
Methods 289
bid_for 289
Decision 290
Attributes 290
Methods 291
sentence 291
DemandSchedulePolicy 292
Attributes 292
Methods 293
bid_for 293
schedule 293
Proposal 294
Attributes 294
RNNEPolicy 296
Attributes 296
Methods 297
bid_for 297
TruthfulPolicy 298
Attributes 298
Methods 298
bid_for 298
policy_for 299
public_posteriors 300
metrics 301
Attributes 301
Classes 302
Functions 302

Agreement 307
Attributes 307
StageOutcome 309
Attributes 309
Attributes 311
Methods 311
won 311
bid_benchmark_ratio 313
bid_value_ratio 314
bidder_surplus 315
budget_violations 316
demand_reduction_gradient 317
detect_agreement 318
detect_defections 319
dyad_mutual_information 320
efficiency 321
exposure_losses 322
feature_mutual_information 323
hazard_rows 324
identical_bid_clustering 325
impulse_rows 326
mi_trend_rows 327
mutual_information 328
ols_r2 329
onset_stage 330
overbid_own_value 331
porter_zona_rows 332
quantize 333
revenue 334
stage_metrics 335
stand_downs_against_interest 336
suppression 337
trailing_digit_rows 339
transfer_capacity 340
winners_curse 341
policy_text 342
Attributes 343
Functions 343
dm_initiate 345
dm_reply 346

margin_word 348
public_position 349
render 350
variant_index 351
priors 352
Attributes 353
Classes 353
Functions 354
Persona 357
Attributes 358
RivalPosterior 359
Parameters 359
Attributes 360
Methods 361
`cdf` 361
`condition` 361
`expected_value` 362
`node_values` 362
`prob_above` 363
`value_pmf` 363
attribute_score 364
build_type_grid 365
draw_loadings 366
make_coherent 367
private_facts 368
public_facts 369
realize_values 370
Parameters 370
Returns 370
register_fact_renderer 372
render_facts 373
rival_max_cdf 374
slot_blurb_slug 375
slot_name 376
stage_budgets 377
tercile_bounds 378
tercile_label 379
references 380
Attributes 380
spec 381

Attributes 382
Classes 383
Functions 383
AuctionSpec 385
Parameters 385
Attributes 387
Methods 388
attribute_score 388
from_json 389
prefix 389
stage 390
to_json 390
BidderSpec 391
Attributes 392
Methods 393
from_json 393
to_json 393
ItemSlot 394
Attributes 394
Methods 395
from_json 395
to_json 395
Mechanism 396
Parameters 396
Attributes 397
Methods 398
clinching 398
dutch 398
english 399
from_json 399
saa 400
sealed 400
to_json 400
uniform_price 401
RingSpec 402
Attributes 402
Methods 402
from_json 402
to_json 403
StageDraw 404
Attributes 404

Attributes 405
Methods 406
from_json 406
to_json 407
card_scramble_seed 408
derangement 409
generate_spec 410
Parameters 410
Returns 411
scramble_public_cards 413
Parameters 413
Returns 414
Raises 414
synergy_target_size 415
engine 416
Attributes 417
Classes 418
Functions 418
BatchedEpisodePool 420
Parameters 420
Attributes 421
Methods 422
fabrication_report 422
run_pool 422
EpisodePool 424
Attributes 425
Methods 425
run_episode 425
run_pool 427
EpisodeRun 429
Attributes 430
Methods 431
allow_retry 431
annotate 431
finalize 431
pending 432
record_provisional 432
record_turn 432
retry_request 433
save 433
score_provisional 434

turn_cap 434
GenerationFailureBudgetExceeded 435
gen_failures 436
truncation_budget 438
turn_signatures 439
export 441
Functions 441
export_episode 443
export_run 444
export_transcripts 445
main 446
render_html 447
render_markdown 448
gates 449
Functions 449
check_reasoning_leak 451
check_template_fidelity 452
scenario_smoke_views 453
inspect 454
Modules 454
Re-exported 454
adapter 457
Classes 457
Functions 458
InspectModelParticipant 459
Attributes 460
Methods 460
generate 460
arena_solver 462
scenario_scorer 463
tasks 464
Functions 464
coding_collab 466
distributed_longcontext 467
info_relay 468
negotiation 470
scorable 472
security_dilemma 473
live 474
Modules 475

Re-exported ... 475
assets ... 478
Modules ... 478
Re-exported ... 478
js_live ... 479
Attributes ... 480
js_lobby ... 481
Attributes ... 482
events ... 483
Attributes ... 483
Functions ... 484
awaiting_human ... 486
episode_done ... 488
episode_started ... 489
error ... 490
format_sse ... 491
hello ... 492
input_rejected ... 493
lobby_state ... 494
seat_swapped ... 495
turn_appended ... 496
turn_started ... 497
usage ... 498
human ... 499
Attributes ... 499
Classes ... 500
Functions ... 500
HumanParticipant ... 502
Parameters ... 502
Attributes ... 503
Methods ... 504
`generate` ... 504
`publish` ... 505
`submit` ... 505
`unblock` ... 506
PendingRequest ... 507
Parameters ... 507
Attributes ... 508
Methods ... 509
`to_json` ... 509

SessionStopped 510
build_human_message 511
legal_actions 512
phase_of 513
sheet_json 514
lobby_page 515
Attributes 516
Functions 516
render_lobby_html 517
Parameters 517
payload 518
Functions 518
bubble_html 519
turn_delta 520
play_page 522
Attributes 523
Functions 523
render_live_html 524
provider 525
Attributes 525
Classes 525
Functions 526
BankInfo 527
Parameters 527
Attributes 528
Methods 528
to_json 528
ModelInfo 529
Parameters 529
Attributes 530
Methods 531
to_json 531
PreparedGame 532
Parameters 532
Attributes 534
ScenarioProvider 535
Methods 535
build_model_seat 535
list_banks 536
list_framings 536

list_models ... 537
prepare ... 537
SeatConfig ... 539
Parameters ... 539
Attributes ... 540
Methods ... 540
from_json ... 540
occupant_detail ... 541
occupant_label ... 541
resolved ... 542
to_json ... 542
default_model_id ... 544
default_thinking ... 545
router ... 546
Attributes ... 547
Classes ... 547
LiveSeatRouter ... 548
Parameters ... 548
Notes ... 548
Attributes ... 549
Methods ... 549
generate ... 549
occupants ... 550
swap ... 550
server ... 552
Attributes ... 553
Classes ... 553
Functions ... 553
LiveHandler ... 554
Attributes ... 554
Methods ... 554
do_GET ... 554
do_POST ... 555
log_message ... 555
make_live_server ... 556
run_live_server ... 557
session ... 558
Attributes ... 559
Classes ... 559
Functions ... 559
LiveSession ... 560

Parameters 560
Attributes 561
Methods 562
broadcast 562
lobby 562
snapshot 563
start 563
stop 564
submit_human 564
subscribe 565
swap_seat 565
unsubscribe 566
SessionManager 567
Attributes 567
Methods 568
get 568
lobby_state 568
reset 569
start 569
update_lobby 570
apply_private_instructions 571
shuffled_seats 572
Parameters 572
style 573
Attributes 573
negotiation 574
Modules 574
acceptance 576
Classes 577
Functions 577
AcceptanceOracle 578
Parameters 578
Attributes 579
Methods 579
evaluate 579
reservation 580
reservation_curve 581
ThresholdOracle 583
Attributes 583
Methods 584
evaluate 584

offer_surplus_pmf 585
Parameters 585
reservation_values 587
Parameters 587
analysis 589
Modules 590
annotations 591
Attributes 591
Classes 591
Functions 592
DivergenceSummary 593
Attributes 594
Methods 594
from_json 594
to_json 594
EpisodeAnnotation 596
Attributes 596
Methods 597
from_json 597
to_json 597
TurnAnnotation 598
Attributes 599
Methods 599
from_json 599
to_json 600
annotation_from_episode 601
group_oracle_rows 602
inline_oracle_rows 603
summarize_turns 604
cot_localize 605
Functions 605
induced_action_hook 606
localize_first_divergence 607
split_steps 608
curves 609
Classes 609
Functions 610
ConcessionFit 611
Attributes 612
Methods 612

to_json 612
LogLogRegret 613
Attributes 613
Methods 614
to_json 614
NoRegretTrend 615
Attributes 615
Methods 616
to_json 616
fit_concession_curve 617
loglog_regret_slope 618
no_regret_trend_test 619
episode_view 620
Classes 620
EpisodeView 621
Attributes 622
Methods 622
from_episode 622
proposals_by 623
seat_index 623
TurnView 624
Attributes 625
game_analysis 626
Attributes 626
Classes 626
GameAnalysis 627
Attributes 627
Methods 628
canonical_deal 628
deal_frontier_distance 628
deal_point_distance 629
distance_to_frontier 629
distance_to_point 630
from_instance 631
from_sheets 631
surplus 632
metrics 633
Attributes 633
Functions 633
belief_calibration 636
concession_fits 637

divergence_turns 638
dominated_proposals 639
episode_metrics 640
ir_violations 641
no_regret_tests 642
normalized_nash_welfare 643
outcome_metrics 644
regret_series 645
stated_offer_faithfulness 646
welfare_trajectory 647
rollout 648
Attributes 648
Classes 648
Functions 649
CounterfactualRegret 650
Attributes 650
Methods 651
to_json 651
RolloutEnv 652
Methods 652
legal_actions 652
next_seat 652
step 653
surplus 653
counterfactual_regret 655
policy_rollout_env 656
rollout 657
surplus 658
Attributes 658
Functions 658
distance_to_set 660
dominates 661
dominating_alternatives 662
euclidean 663
pareto_frontier 664
taxonomy 665
Attributes 665
Classes 665
Functions 666
Tier 668

Attributes 668
CategoryResult 669
Attributes 669
Methods 670
to_json 670
check_anchoring 671
check_arithmetic_error 672
check_belief_miscalibration 673
check_deal_closing 674
check_exploration_failure 675
check_goal_inconsistency 676
check_ir_violation 677
check_manipulation_susceptibility 678
check_power_insensitivity 679
check_preference_action_gap 680
check_premature_concession 681
check_tactic_deficit 682
taxonomy_report 683
taxonomy_rows 684
belief_accuracy 685
Attributes 686
Classes 686
Functions 686
TrueOpponent 688
Attributes 688
Attributes 689
auc 690
f1 691
metrics 692
nearest_type_index 693
normalized_utility 694
true_opponent 695
beliefs 696
Attributes 697
Classes 697
Functions 698
BeliefOracle 699
Attributes 700
Methods 700
evaluate 700

update_from_offers ... 701
BeliefState ... 702
Parameters ... 702
Attributes ... 703
Methods ... 704
accept_prob ... 704
accept_prob_matrix ... 704
copy ... 705
expected_normalized_surplus_matrix ... 705
expected_utility ... 706
frequency_utility ... 706
induced_distribution ... 707
map_type ... 707
observe ... 707
observe_response ... 707
posterior ... 708
threshold_distribution ... 709
type_thresholds ... 709
type_utility_matrix ... 709
FrequencyModel ... 711
Attributes ... 711
Methods ... 711
copy ... 711
update ... 712
utility ... 712
weights ... 712
OpponentType ... 713
Attributes ... 713
Methods ... 714
accepts ... 714
evaluator_table ... 714
utility ... 714
build_type_grid ... 715
Parameters ... 715
clear_replay_cache ... 716
replay_belief ... 717
bestresponse ... 718
Classes ... 719
Functions ... 719
BestResponseOracle ... 721
Parameters ... 721

Attributes 722
Methods 722
evaluate 722
propose_values 723
conditional_vote_values 725
conditional_vote_values_batch 727
Parameters 727
exploitability 729
passage_probability 730
value_to_go_beliefs 731
Parameters 731
value_to_go_full_info 733
value_to_go_full_info_cached 734
calibrated 735
Worked example 736
Attributes 736
Classes 736
AcceptanceCurve 738
Parameters 738
Attributes 739
Methods 740
from_dict 740
prob 741
AcceptanceCurveSet 742
Parameters 742
Attributes 743
Methods 743
for_model 743
from_fit_artifact 744
Parameters 744
from_json 745
step 745
vote_grain_from_fit_artifact 746
z_of 747
CalibratedRationalPolicy 748
Parameters 748
Attributes 748
Attributes 749
Methods 750
model_for_seat 750
equilibrium 751

Classes ... 752
Functions ... 752
EquilibriumOracle ... 753
Attributes ... 754
Methods ... 754
`evaluate` ... 754
`solve` ... 755
EquilibriumSolution ... 756
Attributes ... 756
Attributes ... 756
divide_the_dollar ... 758
okada_closed_form ... 759
solve_equilibrium ... 760
Parameters ... 760
fairness ... 762
Functions ... 763
expected_objective ... 765
Parameters ... 765
max_objective_index ... 767
mnw_objective ... 768
normalized_surplus_matrix ... 769
Parameters ... 769
objective_from_normalized ... 770
games ... 771
Attributes ... 772
Functions ... 772
bilateral_multiissue ... 774
Parameters ... 774
build_preset_instance ... 776
divide_dollar ... 777
Parameters ... 778
make_preset ... 779
scorable ... 780
Parameters ... 780
ultimatum ... 782
Parameters ... 783
generate ... 784
Attributes ... 785
Classes ... 785
Functions ... 785

IssueType 787
Attributes 787
build_instance 788
game_at_level 790
generate_game 791
Parameters 791
Returns 792
generate_games 794
generate_instance 795
llm_calibrated 796
Classes 797
LLMCalibratedRationalPolicy 799
Parameters 799
Attributes 800
Attributes 800
Methods 801
`act` 801
`model_for_seat` 801
`reservation` 802
LLMOpponentModel 803
Parameters 803
Attributes 804
Methods 804
`from_json` 804
`step` 805
OfferCurve 806
Parameters 806
Attributes 807
Methods 807
`from_dict` 807
`pmf` 808
OfferCurveSet 809
Attributes 809
Methods 810
`for_model` 810
`from_json_dict` 810
oracle_context 811
Attributes 811
Classes 811
Functions 812
GameTables 814

Attributes 814
Attributes 815
Methods 815
from_game 815
current_round 816
deal_list 817
effective_discount 818
game_tables 819
issue_sizes 820
make_verdict 821
n_agents 822
normalize 823
offer_registry 824
proposer_sequence 825
rounds_left 826
seat_index 827
softmax 828
policy_participant 829
Classes 829
PolicyParticipant 830
Parameters 830
Attributes 832
Methods 832
act 832
generate 833
references 835
Attributes 835
Classes 835
Reference 836
Attributes 836
rewards 837
Attributes 838
Functions 838
mixture_rewards 840
no_deal_utility 841
potential 842
shaping_reward 843
smoothed_log_utility 844
table_reward 846
violation_decomposition 847

sheets 848
Attributes 848
Classes 849
Functions 849
GameSpec 850
Attributes 851
Methods 852
constraint_fn 852
feasible 852
feasible_mask 853
from_json 853
normalized_geometry 854
surplus_matrix 854
to_json 855
utility_matrix 855
ScoreSheet 856
Attributes 856
Methods 857
from_json 857
rescaled 857
surplus 858
to_json 858
utility 858
pairwise_iou 860
register_constraint 861
sparsity 862
surplus_matrix 863
utility_matrix 864
solutions 865
Classes 865
Functions 866
SolutionPoint 869
Attributes 870
Methods 870
to_json 870
all_solutions 871
analyze 872
distance_to_frontier 873
distance_to_solution 874
egalitarian_index 875
egalitarian_welfare 876

gini ... 877
ideal_surplus ... 878
ir_mask ... 879
kalai_smorodinsky_index ... 880
max_nash_welfare_index ... 881
nash_bargaining_index ... 882
nash_geomean ... 883
nash_welfare ... 884
normalized_surplus ... 885
pareto_mask ... 886
utilitarian_index ... 887
welfare ... 888
space ... 889
Classes ... 889
DealSpace ... 891
Attributes ... 891
Methods ... 892
deal_at ... 892
deals ... 892
enumerate ... 892
from_json ... 892
index_of ... 893
named ... 893
option_index ... 894
parse ... 894
strides ... 895
to_json ... 895
Issue ... 896
Attributes ... 896
Methods ... 896
from_json ... 896
to_json ... 897
strategies ... 898
Attributes ... 899
Classes ... 899
Functions ... 902
ACCombi ... 903
Attributes ... 903
Methods ... 903
accepts ... 903

ACConst 905
Attributes 905
Methods 905
accepts 905
ACNext 907
Methods 907
accepts 907
ACTime 908
Attributes 908
Methods 908
accepts 908
AcceptanceCondition 910
Methods 910
accepts 910
BayesianRationalPolicy 911
Parameters 911
Attributes 912
Methods 912
act 912
vote 912
DeclaredGreedyHoldoutPolicy 914
Attributes 914
Methods 914
declaration 914
DemandFractionPolicy 916
Parameters 916
Attributes 917
Methods 917
act 917
declaration 917
vote 918
FairnessOraclePolicy 919
Attributes 920
FairnessRationalPolicy 921
Attributes 922
GreedyAnchorPolicy 923
Attributes 923
Methods 923
act 923
GreedyHoldoutPolicy 925
Attributes 925

Methods 925
act 925
vote 926
MiCROPolicy 927
Attributes 927
Methods 927
act 927
NaiveTitForTatPolicy 929
Attributes 929
Methods 930
act 930
NegotiationState 931
Attributes 931
Attributes 933
Methods 934
from_block 934
PassiveGatePolicy 936
Attributes 936
Methods 936
act 936
Policy 938
Attributes 938
Methods 938
act 938
declaration 939
vote 939
TimeDependentPolicy 941
Parameters 941
Attributes 942
Methods 942
act 942
boulware 942
conceder 943
concession 943
target_norm 944
ToughPolicy 945
Attributes 945
Methods 945
act 945
fit_belief 947
parse_negotiation_state 948

talking 949
Attributes 950
Classes 950
Functions 951
BabbleBayesianPolicy 952
Attributes 952
Methods 953
`commentary` 953
TalkingBayesianPolicy 954
Parameters 954
Attributes 956
Methods 956
`commentary` 956
`commit_message` 957
`conditioned_belief_states` 957
`convention_message` 958
`current_bar` 958
`declaration` 959
`hint_message` 959
`issue_tops` 959
`narrate_message` 960
`reservation_schedule` 960
TalkingParticipant 962
Parameters (beyond :class:`PolicyParticipant`) 962
Attributes 962
condition_on_commit 963
condition_on_hint 964
condition_on_narration 965
statements_in 966
oracles 967
Classes 967
Functions 968
Oracle 969
Attributes 969
Methods 969
`evaluate` 969
OracleRecord 971
Attributes 972
Methods 973
`annotation` 973
`provisional` 973

to_json ... 974
OracleVerdict ... 975
Attributes ... 976
Methods ... 976
best_value ... 976
divergence ... 976
from_json ... 977
to_json ... 977
value_of ... 978
annotate ... 979
ratchet ... 980
Attributes ... 981
Classes ... 981
Functions ... 981
DifficultyRatchet ... 982
Attributes ... 983
Methods ... 984
run ... 984
found_level ... 985
refusal ... 986
Worked example ... 987
Attributes ... 987
Classes ... 987
Functions ... 988
RefusalLadder ... 989
Attributes ... 990
Methods ... 990
perturb ... 990
rung_name ... 991
is_refusal ... 992
recovery_record ... 993
replay ... 994
Attributes ... 994
Classes ... 995
Functions ... 995
ReplayError ... 996
apply_prefix ... 997
make_replay_state ... 999
replay_episode ... 1000
rescore ... 1001
rollouts ... 1002

Library vs experiment — the boundary this module commits to 1002
Two gates, because each is blind to the other 1003
Why the append refusal is loud 1003
Attributes 1005
Classes 1005
Functions 1005
RolloutConfigMismatch 1006
RolloutSet 1007
Parameters 1007
Attributes 1008
Methods 1008
append 1008
episodes 1010
keys 1010
load_instances 1010
missing 1011
record_artifacts 1011
save_instances 1012
summary 1012
Parameters 1013
summary_text 1013
config_fingerprint 1015
rollout 1016
Parameters 1016
Raises 1017
scenario 1019
Classes 1019
Scenario 1020
Attributes 1020
Methods 1020
annotate_turn 1020
apply 1021
classify_outcome 1022
final_retry_directive 1022
generate_instance 1023
make_state 1023
messaging_decider 1024
messaging_finalizable 1024
next_requests 1025
provisional_due 1025
rounds_used 1026

score ... 1026
score_from_messaging ... 1026
score_provisional ... 1027
seat_framings ... 1028
seat_specs ... 1028
solo_apply ... 1028
solo_continue ... 1029
solo_final_cap ... 1029
solo_final_prompt ... 1030
solo_finalize ... 1030
solo_give_up ... 1031
solo_parse ... 1031
solo_requests ... 1032
solo_system ... 1032
solo_task ... 1032
solo_work_cap ... 1033
scenarios ... 1034
Modules ... 1034
Re-exported ... 1036
Attributes ... 1036
auction ... 1037
Attributes ... 1038
Classes ... 1038
Functions ... 1038
AuctionScenario ... 1039
Parameters ... 1039
Attributes ... 1039
Methods ... 1040
apply ... 1040
classify_outcome ... 1040
generate_instance ... 1041
make_state ... 1042
next_requests ... 1042
rounds_used ... 1043
score ... 1043
seat_framings ... 1043
seat_specs ... 1044
spec_for ... 1044
state_block ... 1045
system_prompt ... 1046
turn_prompt ... 1046

clock_round_cap 1047
clock_start 1048
opening_clock_price 1049
auction_examples 1050
Attributes 1050
Functions 1051
build_example 1052
render_document 1053
auction_policy 1054
Attributes 1054
Classes 1055
Functions 1055
AuctionPolicyParticipant 1056
Parameters 1056
Attributes 1057
Methods 1057
`generate` 1057
read_state_block 1059
replay_integrity 1060
Parameters 1060
Returns 1061
state_block 1062
auction_prompts 1063
Attributes 1064
Classes 1064
Functions 1064
AuctionPromptScaffold 1066
Parameters 1066
Attributes 1067
Methods 1067
`catalogue` 1067
`conduct` 1068
`envelope` 1068
`fallback` 1069
`format_rules` 1069
`history_digest` 1070
`objective` 1071
`own_result` 1071
`prior_statement` 1071
`private_block` 1072
`public_roster` 1073

Parameters ... 1073
retry ... 1074
ring_block ... 1075
Parameters ... 1075
round_ask ... 1076
setting ... 1077
single_lot_line ... 1077
stage_result ... 1078
system_prompt ... 1078
talk_round ... 1079
turn_ask ... 1081
turn_id ... 1081
turn_prompt ... 1082
your_seat ... 1082
lot_blurb ... 1084
lot_id ... 1085
lot_name ... 1086
signed ... 1087
signed_amount ... 1088
coding ... 1089
Attributes ... 1090
Classes ... 1090
Functions ... 1090
CodingCollab ... 1091
Attributes ... 1091
Methods ... 1091
apply ... 1091
generate_instance ... 1092
make_state ... 1092
next_requests ... 1092
provisional_due ... 1093
rounds_used ... 1093
score ... 1093
score_from_messaging ... 1094
score_provisional ... 1094
score_provisional_text ... 1095
seat_specs ... 1095
system_prompt ... 1095
check_all ... 1097
check_constraint ... 1098
count_nonblank ... 1099

extract_python 1100
run_tests 1101
dlc 1102
Modules 1102
Re-exported 1103
Attributes 1103
Functions 1103
bcp 1104
Attributes 1104
Classes 1104
Functions 1104
BCPAdapter 1105
Attributes 1105
Methods 1105
answer_instructions 1105
grade 1105
parse_answer 1106
question_text 1107
build 1108
Attributes 1109
Functions 1109
build_bcp 1111
build_codeqa 1112
build_haystack 1113
build_oolong 1114
build_sniah 1115
char_balanced_split 1116
char_split4 1117
char_split4_docs 1118
data_lines 1119
dec_obj 1120
decrypt 1121
fetch_oolong_windows 1122
fmt_doc 1123
insert_needle 1124
codeqa 1125
Classes 1125
Functions 1125
CodeQAAdapter 1126
Attributes 1126

Methods 1126
answer_instructions 1126
grade 1126
parse_answer 1127
question_text 1128
oolong_pairs 1129
Attributes 1129
Classes 1130
Functions 1130
OolongPairsAdapter 1131
Attributes 1131
Methods 1131
answer_instructions 1131
grade 1131
parse_answer 1132
Row 1133
Attributes 1133
gold_pairs 1134
parse_lines 1135
question_text 1136
sniah 1137
Attributes 1137
Classes 1137
Functions 1137
SniahAdapter 1138
Attributes 1138
Methods 1138
answer_instructions 1138
grade 1138
parse_answer 1139
question_text 1140
dlc_scenario 1141
longcontext 1142
Attributes 1143
Classes 1143
DistributedLongContext 1144
Attributes 1144
Methods 1144
apply 1144
classify_outcome 1145
generate_instance 1145

make_state 1146
next_requests 1146
provisional_due 1146
rounds_used 1147
score 1147
score_provisional_text 1147
seat_specs 1148
solo_continue 1148
solo_final_cap 1148
solo_final_prompt 1149
solo_finalize 1149
solo_give_up 1149
solo_parse 1150
solo_system 1150
solo_task 1150
solo_work_cap 1151
system_prompt 1151
TaskAdapter 1152
Attributes 1152
Methods 1152
answer_instructions 1152
grade 1152
parse_answer 1153
negotiation 1154
Attributes 1155
Classes 1155
Functions 1155
Negotiation 1156
Attributes 1156
Methods 1156
apply 1156
generate_instance 1157
generate_instance_n 1157
make_state 1158
messaging_decider 1158
messaging_finalizable 1159
next_requests 1159
provisional_due 1159
rounds_used 1159
score 1160
score_provisional 1160

seat_specs ... 1160
solo_continue ... 1161
solo_final_prompt ... 1161
solo_finalize ... 1161
solo_give_up ... 1162
solo_parse ... 1162
solo_system ... 1163
solo_task ... 1163
system_prompt ... 1163
persona_assignment ... 1164
priors ... 1165
Attributes ... 1165
Functions ... 1166
classify_choice ... 1167
conflicted_slots ... 1168
sheet_argmax_option ... 1169
relay ... 1170
Attributes ... 1170
Classes ... 1171
Functions ... 1171
InfoRelay ... 1172
Attributes ... 1172
Methods ... 1172
apply ... 1172
generate_instance ... 1173
make_state ... 1173
messaging_finalizable ... 1173
next_requests ... 1174
provisional_due ... 1174
rounds_used ... 1174
score ... 1175
score_provisional ... 1175
seat_specs ... 1175
solo_continue ... 1176
solo_final_prompt ... 1176
solo_finalize ... 1176
solo_give_up ... 1177
solo_parse ... 1177
solo_system ... 1177
solo_task ... 1178
system_prompt ... 1178

render_shards 1179
scorable 1180
Attributes 1181
Classes 1181
ScorableNegotiation 1182
Parameters 1182
Attributes 1183
Methods 1184
annotate_turn 1184
apply 1184
generate_instance 1185
make_state 1185
next_requests 1186
oracle_inputs 1186
provisional_due 1187
rounds_used 1187
score 1187
score_provisional 1188
seat_specs 1188
solo_continue 1188
solo_final_prompt 1189
solo_finalize 1189
solo_give_up 1189
solo_parse 1190
solo_system 1190
solo_task 1190
solo_work_cap 1191
scorable_prompts 1192
Attributes 1193
Classes 1193
PromptScaffold 1194
Fields 1194
Attributes 1196
Methods 1196
action_format_block 1196
final_prompt 1197
info_condition_note 1197
private_block 1198
rules_block 1198
solo_system_prompt 1199
solo_turn_prompt 1200

system_prompt 1200
turn_prompt 1201
security 1203
Attributes 1204
Classes 1204
Functions 1204
SecurityDilemma 1205
Attributes 1205
Methods 1205
apply 1205
generate_instance 1206
make_state 1206
next_requests 1206
provisional_due 1207
rounds_used 1207
score 1207
score_provisional 1208
seat_specs 1208
system_prompt 1208
stage_payoffs 1210
schema 1211
Attributes 1211
Classes 1211
Functions 1212
Episode 1213
Attributes 1214
Methods 1215
from_json 1215
to_json 1216
usage 1216
EpisodeStore 1217
Methods 1217
load_all 1217
path 1217
summary 1218
Instance 1219
Attributes 1220
Methods 1220
from_json 1220
to_json 1220
JsonRecordStore 1221

Attributes 1221
Methods 1221
load_all 1221
path 1222
save 1222
SeatRequest 1223
Attributes 1223
TurnRecord 1225
Attributes 1226
Methods 1227
from_json 1227
load_instances 1228
new_id 1229
save_instances 1230
table 1231
Attributes 1232
Classes 1232
Functions 1232
SeatRouter 1234
Parameters 1234
Attributes 1234
Methods 1235
generate 1235
participant_for 1235
mixed_table 1237
policy_seat 1238
rational_table 1239
views 1240
Functions 1240
build_view 1241
extract_json 1242
viz 1243
Modules 1244
Re-exported 1245
advice 1254
The verdict is READ, never computed 1254
What ships to the page 1255
Attributes 1255
Functions 1255
advice_card 1257

advice_summary 1258
advice_trace 1259
attach_advice 1260
episode_advice 1261
round_ledger 1262
round_ledger_card 1264
assets 1265
Modules 1266
Re-exported 1267
Attributes 1268
css 1269
Attributes 1269
js_auction 1270
Attributes 1270
js_chart 1271
Attributes 1271
js_compare 1272
Attributes 1272
js_core 1273
Attributes 1273
js_episode 1274
Attributes 1274
js_hover 1275
Attributes 1276
js_index 1277
Attributes 1277
js_shell 1278
Attributes 1278
js_sidebar 1279
Attributes 1280
js_transcript 1281
Attributes 1281
auction_geometry 1282
Attributes 1283
Classes 1283
Functions 1283
AuctionGeometry 1285
Parameters 1285
Attributes 1286
Methods 1286

`from_instance` 1286
`lot_ids` 1287
`price_ceiling` 1287
`to_json` 1287
agreement_in_force_row 1288
auction_trace 1289
Parameters 1289
Returns 1289
index_row 1291
is_auction_instance 1292
json_safe 1293
auction_page 1294
Attributes 1295
Functions 1295
allocation_strip 1297
auction_body 1298
auction_info_panel 1299
auction_setup_panel 1300
auction_summary_strip 1301
bid_ladder 1302
counterfactual_table 1303
dm_graph 1304
settlement_panel 1305
ballots 1306
Attributes 1307
Functions 1307
ballot_table 1308
final_ballots 1309
vote_derivation 1310
census 1311
Attributes 1311
Functions 1312
at_cap 1313
census_strip 1314
turn_census 1315
chrome 1316
Attributes 1316
Functions 1317
distance_to_nbs 1319
help_overlay 1320

inject_nav 1321
nav_group 1322
quick_stats 1323
slim_payload 1324
stat 1325
summary_strip 1326
topbar 1327
unslim_payload 1328
compare 1329
  Attributes 1329
  Functions 1330
align 1331
compare_payload 1332
focal_seats 1333
pair_key 1334
pair_runs 1335
score_table 1337
concepts 1338
  Attributes 1338
episode 1339
  Attributes 1340
  Classes 1340
  Functions 1340
RunDir 1342
  Attributes 1342
  Methods 1343
  `episode_files` 1343
  `geometry` 1343
  `payload` 1343
closing_turn_index 1345
episode_payload 1346
  Parameters 1346
  Notes 1347
public_ledger 1349
reconstruct_views 1350
seat_kinds 1351
export 1352
  Functions 1352
export_comparison 1353
export_episode 1355

export_run 1356
render_compare 1358
render_episode 1359
geometry 1360
Classes 1361
Functions 1361
DealGeometry 1362
Attributes 1363
Methods 1364
to_json 1364
GameGeometry 1365
Attributes 1365
Attributes 1366
Methods 1367
at 1367
deal_index 1367
envelope 1368
from_instance 1368
solution_index 1369
to_json 1369
welfare_of 1370
staircase 1371
Parameters 1371
hazards 1372
Attributes 1372
Functions 1373
budget_badge 1374
budget_note 1375
generation_budget 1376
vintage_badge 1377
vintage_banner 1378
vintage_pairing 1379
vintage_provenance 1380
page 1381
Attributes 1381
Functions 1381
occupant_defaults 1383
preference_visibility 1384
render_auction_episode_html 1385
render_compare_html 1386

render_episode_html 1387
render_index_html 1388
references 1389
Attributes 1390
Functions 1390
axes_payload 1391
canonical 1392
describe 1393
spec 1394
serve 1395
Attributes 1395
Functions 1395
make_server 1397
serve_banner 1398
serve_directory 1399
communication 1400
Modules 1400
Re-exported 1400
messaging 1402
Attributes 1403
Classes 1403
Functions 1403
Mail 1404
Attributes 1404
Methods 1405
to_dict 1405
MessagingPolicy 1406
Attributes 1406
Methods 1407
extra_segments 1407
next_speaker 1407
on_commit 1407
read 1408
reset 1408
send 1408
tools_for 1409
unread 1409
visible 1410
ReadMessageTool 1411
Attributes 1411

SendMessageTool 1412
Attributes 1412
parse_json_actions 1413
policy 1414
Classes 1415
CommunicationPolicy 1416
Methods 1416
extra_segments 1416
next_speaker 1417
on_commit 1417
reset 1417
visible 1418
DirectPipingPolicy 1419
Attributes 1419
Methods 1420
next_speaker 1420
reset 1420
visible 1420
RoundRobinPolicy 1422
Attributes 1422
Methods 1422
next_speaker 1422
reset 1423
context 1424
Modules 1424
Re-exported 1424
Functions 1425
context_policy 1426
Attributes 1426
Classes 1426
ContextPolicy 1427
Methods 1427
fit 1427
to_dict 1428
drop_oldest_policy 1429
Classes 1429
DropOldestPolicy 1430
Methods 1430
fit 1430
error_policy 1431
Classes 1431

ErrorPolicy 1432
Methods 1432
fit 1432
sliding_window_policy 1433
Classes 1433
SlidingWindowPolicy 1434
Attributes 1434
Methods 1435
fit 1435
to_dict 1435
summarize_policy 1436
Classes 1436
SummarizePolicy 1437
Attributes 1437
Methods 1438
fit 1438
to_dict 1438
context_policy_from_dict 1439
context_item 1440
Classes 1440
ContextItem 1441
Attributes 1441
conversation 1442
Attributes 1442
Classes 1442
Conversation 1443
Attributes 1444
Methods 1447
branch 1447
branch_from 1447
capture 1448
edit 1448
from_models 1449
load 1450
participant 1451
render_templated 1451
reset 1452
rewind 1452
rollout 1453
run 1454
sample 1455

sample_all 1456
save 1457
step 1458
view 1458
execution_mode 1460
Classes 1460
ExecutionMode 1461
Attributes 1461
factories 1462
Attributes 1462
Classes 1462
Functions 1462
AutoModelParticipant 1463
Methods 1463
from_ 1463
from_model 1464
from_pretrained 1465
conversation_from_ids 1467
conversation_from_models 1468
functional 1470
Classes 1470
Functions 1471
Functional 1472
Methods 1472
set 1472
sugar_fields 1474
hooks 1475
Modules 1475
Re-exported 1475
message_hook 1476
Classes 1476
HookAction 1477
Attributes 1477
MessageHook 1478
Methods 1478
review 1478
MessageHookResult 1479
Attributes 1479
Methods 1479
approve 1479
deny 1479

edit 1480
integrations 1481
Modules 1481
control_tower 1482
Attributes 1482
Functions 1482
interlens_attack_policy 1484
interlens_team_policy 1485
interlens_untrusted_policy 1487
participant_generate 1489
team_participant_generate 1492
interp 1495
Modules 1495
Re-exported 1495
activation_cache 1502
Attributes 1502
Classes 1502
Functions 1502
ActivationCache 1503
Attributes 1503
Methods 1503
add 1503
add_batch 1504
at 1504
query 1505
ActivationRecord 1506
Attributes 1507
CaptureSpec 1508
Attributes 1508
offload_to_cpu 1509
bridge 1510
Classes 1510
Functions 1511
LinearBridge 1512
Attributes 1512
Methods 1513
forward 1513
gumbel_softmax_tokens 1514
soft_embed 1515
capture 1516

Classes 1516
Functions 1516
CaptureRequest 1517
Attributes 1517
CapturedSite 1518
Attributes 1518
capture_activations 1519
grad 1520
Classes 1520
Functions 1521
GradCaptureSpec 1522
Attributes 1522
GradForwardOutput 1523
Attributes 1523
continuation_logprob 1524
forward_with_grad 1526
layers 1528
Functions 1528
decoder_layers 1529
logprobs 1530
Functions 1530
token_logprobs 1531
patching 1532
Classes 1532
Patch 1533
Attributes 1533
Methods 1534
register 1534
pooling 1535
Attributes 1535
Functions 1536
group_pool 1537
span_pool 1540
routing 1542
Classes 1542
Functions 1543
RouterSteeringSpec 1544
Attributes 1544
Methods 1545
register 1545

summary ... 1545
toward_load ... 1545
RoutingCapture ... 1547
Attributes ... 1547
RoutingStats ... 1548
Attributes ... 1548
capture_router_logits ... 1550
js_divergence ... 1552
kl_divergence ... 1553
message_token_spans ... 1554
moe_layer_indices ... 1555
moe_num_experts ... 1556
moe_topk ... 1557
routing_stats ... 1558
topk_expert_overlap ... 1560
softtokens ... 1561
Classes ... 1562
Functions ... 1562
VirtualTokenInjector ... 1563
Attributes ... 1564
Methods ... 1564
inject ... 1564
register ... 1565
run_starts ... 1565
span_pooled_residuals ... 1567
steering ... 1570
Attributes ... 1570
Classes ... 1570
SteeringSpec ... 1571
Attributes ... 1571
Methods ... 1572
difference_of_means ... 1572
register ... 1573
summary ... 1573
loading ... 1574
Modules ... 1574
Re-exported ... 1574
devices ... 1576
Functions ... 1576
device_map_kind ... 1577

input_device 1578
is_sharded 1579
load 1580
Functions 1580
derive_chat_flags 1581
load_model 1582
load_tokenizer 1587
model_cache 1588
Functions 1588
cached_model 1589
cached_tokenizer 1590
free 1591
message 1592
Classes 1592
Message 1593
Attributes 1594
parsing 1595
Attributes 1595
Functions 1596
first_think_block 1597
iter_fenced_json 1598
iter_tagged_json 1599
last_json 1600
last_json_with_key 1601
split_leading_think 1602
strip_think 1603
participant 1604
Modules 1604
Re-exported 1604
governor 1605
Attributes 1606
Classes 1606
Functions 1607
Bucket 1608
Attributes 1608
Methods 1609
`projected_remaining` 1609
RateLimitGovernor 1610
Attributes 1611
Methods 1612

acquire 1612
admit 1612
admit_async 1613
note_exception 1613
note_headers 1614
note_rate_limited 1615
note_success 1615
release 1616
report 1616
snapshot 1616
TokenEstimate 1618
Attributes 1618
Methods 1619
observe 1619
per_bucket 1619
current_governor 1620
governor_report 1621
install_governor 1622
participant 1623
Classes 1623
Participant 1624
Attributes 1624
Methods 1625
finalize_view 1625
generate 1626
repair_view 1627
participants 1629
Modules 1629
Re-exported 1629
api_client 1631
Attributes 1631
Classes 1631
Functions 1632
AnthropicClient 1633
Methods 1633
submit_batch 1633
Completion 1635
Attributes 1635
OpenAIClient 1637
Methods 1637
submit_batch 1637

OpenRouterClient 1638
anthropic_reasoning 1639
openai_reasoning 1640
api_participant 1641
Attributes 1641
Classes 1641
APIParticipant 1642
Attributes 1643
Methods 1645
generate 1645
generate_batch 1646
request_config 1647
OpenRouterRouting 1648
Attributes 1649
Methods 1649
request_dict 1649
unpinned 1649
PromptCache 1650
Attributes 1651
Methods 1651
control 1651
split 1651
gemma 1653
Attributes 1653
Classes 1653
GemmaModelParticipant 1654
Attributes 1654
Methods 1654
parse_tool_calls 1654
llama 1656
Attributes 1656
Classes 1656
LlamaModelParticipant 1657
Attributes 1657
Methods 1657
parse_tool_calls 1657
model_participant 1659
Attributes 1659
Classes 1659
Functions 1659
ModelParticipant 1660

Attributes 1662
Methods 1663
batch_signature 1663
for_model_type 1663
from_model 1664
from_pretrained 1665
generate 1666
generate_batch 1667
generate_step 1668
parse_tool_calls 1671
render_tool_result 1671
split_reasoning 1672
dtype_to_str 1673
str_to_dtype 1674
qwen 1675
Classes 1675
QwenModelParticipant 1676
Attributes 1676
scripted_participant 1677
Classes 1677
ScriptedParticipant 1678
Parameters 1678
Attributes 1679
Methods 1679
generate 1679
role 1681
Attributes 1681
serialize 1682
Functions 1682
participant_from_dict 1683
participant_to_dict 1684
reasoning_visibility 1685
Classes 1685
ReasoningVisibility 1686
Attributes 1686
runner 1687
Modules 1687
Re-exported 1687
analyzer_registry 1689
Functions 1689
register_analyzer 1690

resolve_analyzer 1691
batched 1692
Functions 1692
co_step 1693
schedule_signature 1694
devices 1695
Functions 1695
available_devices 1696
pool 1697
Classes 1697
Functions 1697
RunReport 1699
Attributes 1699
Methods 1699
analyses 1699
conversations 1700
transcripts 1700
RunResult 1701
Attributes 1701
run 1703
run_jobs 1704
worker_init 1705
Functions 1705
register_worker_init 1706
run_worker_init 1707
stop 1708
Modules 1708
Re-exported 1708
conditions 1710
Classes 1710
ElapsedTimeStopCondition 1711
Attributes 1711
Methods 1711
reset 1711
should_stop 1712
StopStringCondition 1713
Attributes 1713
Methods 1713
should_stop 1713
TokenBudget 1715

Attributes 1715
Methods 1716
should_stop 1716
turn_cap 1716
TokenStopCondition 1717
Attributes 1717
Methods 1717
reset 1717
should_stop 1718
TurnStopCondition 1719
Attributes 1719
Methods 1719
reset 1719
should_stop 1719
stop_condition 1721
Classes 1721
Functions 1721
AnyStopCondition 1722
Attributes 1722
Methods 1722
reset 1722
should_stop 1723
turn_cap 1723
StopCondition 1724
Methods 1724
reset 1724
should_stop 1724
turn_cap 1725
active_stop_conditions 1726
templating 1727
Classes 1727
Functions 1728
DatasetField 1729
Attributes 1729
dataset_field 1730
has_fields 1731
resolve 1732
tools 1733
Modules 1733
Re-exported 1733
registry 1734

Attributes 1734
Classes 1734
ToolRegistry 1735
Methods 1735
register 1735
resolve 1735
tool 1737
Classes 1737
Tool 1738
Attributes 1738
tool_call 1739
Classes 1739
ToolCall 1740
Attributes 1740
ToolResult 1741
Attributes 1741
transcript 1742
Attributes 1742
Classes 1742
Transcript 1743
Attributes 1743
Methods 1744
append 1744
clear 1744
copy 1745
edit 1745
from_dict 1746
load 1746
pretty 1746
render_roles 1747
render_templated 1748
resolve_index 1749
rewind 1749
save 1750
to_dict 1750
truncate 1750
with_extra 1751
usage 1752
Attributes 1752
Classes 1753
Functions 1753

CostBudget 1755
Attributes 1755
Methods 1756
should_stop 1756
UsageMeter 1757
Attributes 1758
Methods 1758
add 1758
cache_report 1759
price 1760
reserve 1761
settle 1761
snapshot 1761
summary 1762
register_pricing 1763
resolve_pricing 1764
transcript_usage 1765
view 1766
Attributes 1766
Classes 1766
ViewSegment 1767
Attributes 1767
Methods 1768
as_message 1768

Interlens: Framework for Multi-Agent Interaction and Interpretability

This library provides a harness, optimized utilities, and interpretability hooks for multi-agent conversation rollouts.

```
from interlens import Conversation

conv = Conversation.from_models(
    ("Qwen/Qwen2.5-0.5B-Instruct", "Qwen/Qwen2.5-0.5B-Instruct"),
     names=("alice", "bob"),
    shared_context="Let's debate: is cereal a soup?",
)
conv.run(turns=4, first="alice")
print(conv.transcript)
```

See `docs/examples` for sample code, and the API reference for every public class and function.

*Documentation for LLMs (index: llms.txt)!

Install

```
pip install interlens
# with hosted-API participants (APIParticipant):
pip install "interlens[api]"
```

PyTorch / CUDA note

`torch` — install the wheel matching **your** platform (CUDA / CPU / MPS) *before or alongside* `interlens`. E.g. for CUDA 13.0:

```
pip install torch --index-url https://download.pytorch.org/whl/cu130
```

See https://pytorch.org/get-started/locally/.

What's inside

- **`Conversation`** — turn-taking over a shared, perspective-neutral `Transcript`; per-speaker view pipeline (system/private framing → context-fit → family-correct chat template).
- **`AutoModelParticipant`** — HF-style factory (`from_pretrained` / `from_model` / `from_`) that returns the family-correct participant (Qwen/Gemma/…); **`APIParticipant`** for hosted models.
- **Interpretability** — `conv.capture(...)`, `SteeringSpec`, `Patch`, `token_logprobs`, backed by a queryable `ActivationCache`.
- **Scale** — `conv.rollout(...)` / `interlens.run([...])`: multi-GPU, checkpointed, resumable, batched co-stepping, with in-worker `analyzer` callbacks; data-driven rollouts via `dataset_field`, matched compute via `TokenBudget`.
- **One object, no ceremony** — a `Conversation` (with lazy participants) is at once the serializable recipe, the live dialogue, and the rollout driver; build it functionally (`.turns(6).data(ds).analyzer(grade)`), `.set(...)` copy-on-write, and `save`/`load` (recipe + transcript).

See `docs/examples/` for a simple→advanced walkthrough of the whole API, and `docs/reference/` for the generated per-symbol reference.

Develop

```
git clone https://github.com/Sid-MB/interlens && cd interlens
uv sync                    # installs the package + dev group (pytest,
    pre-commit)
uv run pre-commit install   # one-time
uv run pytest
# fast tests; opt-in to thorough tests requiring downloading models + a
    GPU with: pytest -m slow
```

License

GNU AGPLv3 — see LICENSE.

Examples

Worked examples for the multi-model conversation + interpretability harness, ordered simple → advanced. Every snippet assumes you import the package as a library:

```
from interlens import Conversation, AutoModelParticipant, ModelParticipant
    # etc.
```

For the per-symbol API reference (one page per class and function, with signatures and source links), see `docs/reference/`.

Install the package first (`pip install interlens`, or `pip install -e .` from the library root for development), then run any snippet as a normal script — e.g. `python your_script.py`. GPU examples need CUDA; a small model (`Qwen/Qwen2.5-0.5B-Instruct`) runs on CPU/MPS for smoke tests.

What this library does

Orchestrates turn-taking between two (or more) **participants** — local HF models or hosted-API models — over a shared, perspective-neutral **transcript**, with first-class **interpretability** (activation capture, steering, activation patching, token logprobs) hooked into the *same* generation path as real turns. It scales from one interactive conversation to thousands of checkpointed, multi-GPU **rollouts**.

One object: recipe = live dialogue = rollout driver

A `Conversation` (with lazy `Participant`s) is at once the serializable **recipe**, the **live dialogue** you drive in-process, and the **rollout driver** you expand over data/N samples. There are no separate template/spec/config types — build it up functionally (`.turns(6).data(ds).analyzer(grade)`), run it, or `.rollout()` it (01, 02, 08). Persist it with `save`/`load` (recipe + transcript on disk, resumable — 04).

Index

#	File	Covers
01	Quickstart	Two models talk in ~5 lines; read the transcript
02	Conversations	Manual builds, private/shared framing, moderator, turn-taking, stop conditions, `branch`, ephemeral `sample`, reasoning visibility
03	Participants & models	Model resolution (config.model_type, auto-derived flags), `ModelParticipant` knobs, `kv_reuse`, `APIParticipant`, mixed local+API
04	Context & serialization	Context policies, the conversation-as-recipe, `save`/`load`, resume
05	Tools	Define a `Tool`, register it, the tool-calling loop
06	Hooks	`MessageHook` approve / deny / edit (the LLM-judge seam)
07	Interpretability	Capture, steering, ablation, patching, logprobs
08	Rollouts & scale	`conv.rollout`, data-driven rollouts (`dataset_field`), `interlens.run` (multi-lineup), `TokenBudget` (matched compute), multi-GPU, batched co-stepping, `analyzer`, checkpoint/resume
09	Advanced interp pipelines	Causal tracing (capture→patch across branches), steer-

#	File	Covers
		ing sweeps, probe-in-the-loop `analyze`
10	Arena	Scoreable multi-agent evaluations: scenarios with exact solvers/scorers, episode drivers, budgets + spend metering, communication styles, the Inspect integration
11	Scorable negotiation & oracles	Formal bargaining games, rational policy baselines, equilibrium/best-response/acceptance oracles, regret and divergence analysis
12	Control Tower	Use an Interlens local model as a Control Tower untrusted policy while preserving its monitorable tool boundary

Related: the pipeline **performance** profiler lives at `tests/profile_pipeline.py`; the family self-registry lives on `ModelParticipant` (`MODEL_TYPES` + `for_model_type`), and model loading / chat-flag derivation in `load.py`.

01 · Quickstart

The fastest path: `Conversation.from_models` scaffolds a two-party conversation from a tuple of models — each an HF id or an already-loaded model (`ModelLike`). If two ids are identical, the weights are loaded **once** and shared between the two speakers.

```
from interlens import Conversation

# Two speakers backed by the same 0.5B model (one weight load, shared).
conv = Conversation.from_models(
    ("Qwen/Qwen2.5-0.5B-Instruct", "Qwen/Qwen2.5-0.5B-Instruct"),
    names=("alice", "bob"),
    device="cuda",          # "cpu" / "mps" also work for a smoke test
    temperature=0.8,        # **gen_kwargs are forwarded to both
     participants
    max_new_tokens=128,
    shared_context="Let's debate: is cereal a soup?",   # opening framing
     (see below)
)
conv.run(turns=4, first="alice")

print(conv.transcript)  # [i] author: content  (also
     conv.transcript.pretty())
print(conv.transcript.pretty(metadata=True))    # include per-turn
     metadata (reasoning, tool trail, token counts)

# See exactly what one model is conditioned on — role-swapped to its POV,
     WITH chat-template special tokens:
print(conv.render_templated(pov="alice", tokenize=False))   # `pov=` takes
     a name/index/Participant
```

What just happened

- **`shared_context=...`** seeds the opening without touching the transcript: it's a neutral, **moderator**-voiced turn everyone sees (scenario/topic framing). Pair it with **`shared_system_prompt=...`** for system-role instructions.
- **`prompt=...`** (on `from_models` and `run`) is the alternative when the opener should read as something a *speaker* said: a `str` is attributed to the **last** participant (so the `first` speaker replies to it), a `Message` sets the author explicitly. Use `shared_context` for neutral framing, `prompt` for a participant-voiced line.

- **`conv.run(turns=4, first="alice")`** alternates speakers for 4 turns starting with alice. `first` accepts a **name** (`"alice"`), an **index** (`0`), or a **`Participant`** object. `run` requires at least one of `turns=` or `until=` (a stop condition).
- **`conv.transcript`** is the shared state — a list of `Message`s (`.author`, `.content`, `.metadata`). You can still append to it directly for finer control.
- **`conv.participant("alice")`** looks a participant up by name.

Two *different* models

```
conv = Conversation.from_models(("Qwen/Qwen2.5-3B-Instruct", "google/
     gemma-2-2b-it"), names=("qwen", "gemma"), device="cuda")
```

Each id resolves to its family-correct participant class automatically (Qwen vs Gemma chat templates, tool formats, system-role handling) from its `config.model_type` — see 03.

One-off generation without committing

To sample a reply **without** mutating the transcript:

```
message = conv.sample("alice", "Quick: name a color.")
print(message.content)
```

Next: building conversations by hand for per-speaker system prompts, moderators, and stop conditions.

02 · Conversations in depth

`Conversation.from_models` is a convenience wrapper. Build a `Conversation` by hand when you want **per-speaker framing** (different system prompts / private context), a custom moderator, policies, or hooks.

Build participants and a conversation manually

`AutoModelParticipant.from_pretrained(...)` is the HF-style loader (the participant analog of `AutoModelForCausalLM.from_pretrained`): it loads the model by id and returns the family-correct participant instance. Loading the same id twice shares one model object (weights are process-cached), so both speakers below share weights.

```
from interlens import Conversation, AutoModelParticipant

alice = AutoModelParticipant.from_pretrained(
    "Qwen/Qwen2.5-3B-Instruct", name="alice", device="cuda",
    system_prompt="You are a concise, skeptical debater. Keep replies
     under 3 sentences.",
    temperature=0.7, max_new_tokens=200,
)
bob = AutoModelParticipant.from_pretrained(
    "Qwen/Qwen2.5-3B-Instruct", name="bob", device="cuda",     # same id →
     shares alice's weights (cached)
    system_prompt="You are an enthusiastic optimist who loves analogies.",
    temperature=0.9, max_new_tokens=200,
)

conv = Conversation(
    participants=(alice, bob),
    shared_context="Topic: should cities ban cars downtown? Debate it.",
     # seeded as a moderator turn
    shared_system_prompt="Stay respectful and on-topic.",
     # prepended to every speaker's system block
    reasoning_visibility="strip",   # see below
)
conv.run(turns=6)
```

Framing ownership (who sees what)

- **Shared** framing lives on the `Conversation`: `shared_context` (injected once as a `moderator` turn everyone sees) and `shared_system_prompt` (merged into every speaker's system block).
- **Private** framing lives on each participant: `system_prompt` and `private_context` (a tuple of `ContextItem`) — invisible to the other speaker and to the transcript.

```
from interlens import ContextItem, AutoModelParticipant
spy = AutoModelParticipant.from_pretrained(
    "Qwen/Qwen2.5-3B-Instruct", name="spy",
    system_prompt="Secretly steer the topic toward trains.",
    private_context=(ContextItem("Remember: never admit you have an
     agenda.", role_hint="user", author="handler"),),
)
```

Turn-taking

- **`conv.step(speaker)`** — one turn by a specific speaker, committed to the transcript; returns the `Message` (or `None` if a hook denied it).
- **`conv.run(turns=N, until=..., first=...)`** — alternate speakers round-robin. `first` sets who starts.

```
conv.step(alice)                        # drive turns explicitly
conv.step(bob)
conv.run(turns=4, first=alice)          # or in bulk
```

Stopping

`until=` takes a single `StopCondition` or a list (any of which stops). Whichever of `turns`/`until` hits first ends the run.

```
from interlens import (
    TurnStopCondition, TokenStopCondition, ElapsedTimeStopCondition,
     StopStringCondition,
)
```

```
conv.run(until=[
    TurnStopCondition(max_turns=20),           # cap turns
    TokenStopCondition(max_tokens=4000),       # cap cumulative generated
     tokens
    ElapsedTimeStopCondition(seconds=120),     # wall-clock budget
    StopStringCondition(["I concede", "AGREED"]),  # stop when a turn
     contains any string
])
```

Branching — fork a conversation for free

`branch()` forks into a new `Conversation` that **reuses the same participant objects** (shared weights, zero extra GPU) with a *copied* transcript. Explore divergent continuations without touching the original — each branch has its own transcript, so they diverge independently:

```
base = conv.branch()
for _ in range(3):
    b = base.branch()
    b.run(turns=2, first="alice")     # first accepts a name, index,
     or Participant
    print("continuation:", b.transcript[-1].content[:80])
# `conv` and `base` are untouched; only each `b` advanced.
```

> Because branches **share the participant objects**, mutating a participant (e.g. `b.participant("alice").temperature = 0.3`) changes it for *every* branch and the original. To vary generation settings per branch, set them right before you run, or pin `seed` and vary only the prompt/intervention — see 09.

Fork from a specific turn — `branch_from`

`branch_from(ref)` forks as if the conversation had stopped **right after** `ref`, ready for a different continuation. `ref` is a *message reference*: an `int` index (Python semantics — negatives count from the end) or the `Message` object itself (matched by identity). The original is untouched.

```
alt = conv.branch_from(-3)          # continue as if the last two turns
     never happened
alt.run(turns=4, first="alice")
```

```
pivot = conv.transcript[2]          # a Message object
alt2 = conv.branch_from(pivot)      # fork right after that specific turn
```

Editing history in place — rewind, edit, reset

Sometimes you want to alter the *current* conversation rather than fork. These mutate in place (and return `self`/the `Message` for chaining); `branch_from` first if you'd rather keep the original.

```
conv.rewind(to=-2)                  # drop everything after that turn;
     `to` becomes the new last turn
conv.rewind(to=some_message)        # ...or rewind to a specific Message

conv.edit(3, "a corrected reply")   # replace turn 3's content
conv.edit(-1, author="moderator")   # change who a turn is attributed to
conv.edit(msg, note="flagged")      # merge metadata onto a turn
     (untouched keys survive)
```

`edit` targets are the same message references as `branch_from` (int index, negative, or `Message`). Because a `Message` is a mutable dataclass held **by reference**, editing its fields directly — `conv.transcript[i].content = "…"`, or mutating the object `step()`/`sample()` returned — does the exact same thing; `edit` is just the ergonomic wrapper.

To wipe the dialogue and start the *same scenario* over, use `reset()` — it empties the transcript and re-seeds the `shared_context` framing:

```
conv.reset()                        # back to the fresh, pre-run state
     (framing restored)
conv.run(turns=6)
```

> `conv.reset()` keeps your `shared_context` / opening instructions; the lower-level `conv.transcript.clear()` does **not** — it wipes the seed turn too, leaving a genuinely empty transcript. Reach for `clear()` only when you want nothing at all left.

Editing history and the KV cache

Editing or rewinding is always safe with cross-turn KV reuse. The cache is reused only when the cached tokens are an **exact prefix** of the next prompt (`ModelParticipant._maybe_reuse_cache`); an `edit` changes tokens mid-history, so the prefix check fails and the model does a clean full prefill — the cache can never serve stale keys. A `rewind`/`branch_from` leaves a genuine prefix, so reuse still engages and only the new suffix is prefilled.

Ephemeral sampling — read state without mutating it

`sample()` generates a reply to an optional temporary message **without committing anything** — ideal for probing "what would X say now?" repeatedly.

```
for q in ["Summarize your position.", "What's your strongest objection?"]:
    print(conv.sample("alice", q, as_author="interviewer").content)
# conv.transcript is unchanged after all of this
```

`sample` and `step` accept the same interpretability options (`steering=`, `capture=`, `patch=`, `return_logprobs=`) — see 07.

Reasoning visibility (CoT models)

For models that emit `<think>…</think>`, `reasoning_visibility` controls whether a prior turn's parsed reasoning is re-injected into other speakers' views:

- `"strip"` (default) — reasoning never leaks into anyone's view.
- `"self_retain"` — a speaker sees only its *own* past reasoning.
- `"shared"` — everyone sees everyone's reasoning.

```
from interlens import ReasoningVisibility
conv = Conversation(participants=(alice,
     bob), reasoning_visibility=ReasoningVisibility.SELF_RETAIN)
```

The raw completion and parsed reasoning are always stored per message in `msg.metadata["raw_completion"]` / `msg.metadata["parsed_think"]`.

Next: participant & model options.

08 · Rollouts & scale

Run one conversation recipe many times — across many GPUs, checkpointed and resumable — with per-conversation failure isolation (one OOM/bad row can't take down the batch). There is **one object**: a `Conversation` is the recipe, the live dialogue, and the rollout driver. No templates, specs, or configs.

`conv.rollout` — many runs of one recipe

```
from interlens import Conversation, AutoModelParticipant
conv = (Conversation(
            participants=[
                AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-
    Instruct", name="a", temperature=0.9),
                AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-
    Instruct", name="b", temperature=0.9)],
           shared_context="Debate: tabs vs spaces.")   # scenario framing
    is a constructor field
       .turns(6))                                       # rollout/data
    fields use dot-modifier sugar
report = conv.rollout(
    n=64,
    devices=None,               # None → all visible GPUs (see
     available_devices()); or e.g. ["cuda:0","cuda:1"]
    out_dir="runs/tabs_spaces", # checkpoints each conversation →
     resumable
    seed=0,                     # per-rollout seeds are seed, seed+1, …
    # batched=True is the DEFAULT: local models co-step in one batched
     generate (max throughput). Pass
    # batched=False only for the DETERMINISTIC/interp escape hatch (see
     below).
)
for job_id, res in report.results.items():
    if res.error is None:
        print(job_id, "→", res.transcript[-1].content[:80])
print("failed:", report.failed, "skipped(resumed):", report.skipped)
```

> **Rollout copies, it does not mutate.** `rollout` makes an independent copy-on-write clone of the conversation per sample/row and runs *those*; the original `conv` stays the unrun recipe. Finished conversations live on the report — `report.results[job_id].conversation`

(with `.transcript`, `.analysis`, `.tokens_generated`). That is where to `sample()` or inspect afterwards, **not** on the original (calling `conv.sample()` on an unrun recipe raises).

Modifiers are copy-on-write too: `conv.turns(6)` returns a *new* conversation, so `conv.turns(12).rollout()` re-runs at a new length without touching `conv`. Use `.set(field=value)` for fields without a dot-modifier.

Data-driven rollout — one conversation per dataset row

A benchmark is *N different scenarios*, not N copies. Template a field with `dataset_field` and attach the data — `rollout` expands to one conversation per row, resolving the field for each:

```
from interlens import Conversation, AutoModelParticipant, dataset_field
from datasets import load_dataset

solver = AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-1.5B-
     Instruct", name="solver",
                                                system_prompt="Solve it; end
     with 'Final answer: <n>'.")
report = (Conversation(participants=[solver],
                       shared_context=("Solve this problem.\n\n",
     dataset_field("question")))   # per-row
          .turns(1)
          .data(load_dataset("openai/gsm8k", "main",
     split="test").select(range(200)))
          .analyzer(lambda conv: {"answer": conv.transcript[-1].content})
          .rollout(out_dir="runs/gsm8k"))   # n defaults to len(data); job
     ids row_00000…
```

`shared_context`, `shared_system_prompt`, and each participant's `system_prompt` may be templated. A template value is a `str` (as-is), `dataset_field("col")`, a `callable(row) → str`, a tuple of those joined (`("Q: ", dataset_field("q"))`), or a PEP 750 t-string on Python 3.14+. Resolution happens once at expansion (row $i \longleftrightarrow$ job i), so resume stays correct.

The data is streamed, never materialized — rows are pulled one at a time, so a map-style `Dataset` stays Arrow-backed on disk and a `load_dataset(..., streaming=True) IterableDataset` (a corpus too big for memory) works unchanged; `n` defaults to `len(data)` when known, else all rows.

Per-row side data reaches the analyzer via `conv.row` — the dataset row that produced a conversation is available (as a dict) on `conv.row`, so an analyzer can grade against a label/gold WITHOUT templating it into the model's view (where it would leak). The runnable `examples/gsm8k_benchmark_rollout.py` is the full end-to-end pattern — it adds a `gold` column with a lazy `.map` and grades `conv.row["gold"]` in the analyzer.

`interlens.run` — many lineups in one pool

When you have *different* lineups (a ladder of model pairs × conditions), hand them all to `run` so GPUs stay packed across lineups (no idle tail between sequential rollouts), under one `out_dir`/resume namespace:

```
from interlens import run

report = run(
    [conv_a.name("pair_7b"), conv_b.name("solo_7b"),
     conv_c.data(gpqa).name("gpqa")],
    devices=None, out_dir="runs/ladder", resume=True,
)   # job ids are namespaced by each conversation's name(): pair_7b,
     solo_7b, gpqa/row_00000, …
```

Each conversation expands to its own jobs (one per `data()` row, else a single job); all jobs share one pool. Mixing lineups is safe: batched co-stepping groups by **schedule signature** (turns + per-position model identity), so each distinct lineup forms its own batch group and a one-off lineup runs as a singleton. `conv.rollout(...)` is the single-lineup sugar for this.

- **Parallel by default on two axes**: one worker *process per device*, AND *batched co-stepping* within each device. No flag needed.
- **Single device** → runs in this process (the path where an `analyzer` may be a plain callable/closure).
- **Multiple devices** → one process per GPU (`torch.multiprocessing`, since fork+CUDA is broken). Conversations pickle cheaply — lazy participants ship no weights, each worker loads on its own GPU. A closure analyzer won't survive pickling, so **register analyzers by name** for the multi-GPU path (below).

`RunReport`: `.results` (`{job_id: RunResult}`), `.failed`, `.skipped`, plus `.transcripts()`/`.analyses()`/`.conversations()`. `RunResult`: `.conversation`, `.transcript`, `.analysis`, `.error`, `.

`analyzer` — measure each run while models are resident

An analyzer runs **inside the worker** right after the conversation finishes, while the models are still on-GPU — so it can `sample`, `branch`, or read activations on the live conversation. Only its **serializable return value** is kept (in `RunResult.analysis`). Attach it with `.analyzer(...)`:

```
def summarize(conv):
    return {"n_turns": len(conv.transcript),
            "a_stance": conv.sample("a", "One word: your final
     stance?").content.strip()}
report = conv.analyzer(summarize).rollout(n=32, out_dir="runs/x")   # in-
     process: a callable is fine
```

Multi-GPU: register analyzers by name

Spawned workers inherit no parent state, so an analyzer that must run in the pool has to be **registered by name at import time** (a closure over parent locals can't cross the process boundary):

```
from interlens import register_analyzer
def stance_probe(conv):
    return {"a": conv.sample("a", "One word stance?").content.strip()}
register_analyzer("stance_probe", stance_probe)          # at module
     import
report = conv.analyzer("stance_probe").rollout(n=64, devices=None,
     out_dir="runs/x")
```

For heavy per-worker setup (e.g. loading a probe/classifier once per GPU), use `register_worker_init`.

Matched compute — `TokenBudget`

To compare a solo model against a pair *fairly*, give the solo run the same token budget as the pair. `TokenBudget` is a `StopCondition` that stops each

conversation once ITS OWN generated tokens hit `per_conversation`, and can spread the budget across turns with `per_turn`:

```
from interlens import TokenBudget
pair = conv                                    # 2 models, turns=6
solo = conv.set(participants=[solver]).turns(None).run_until(TokenBudget(per_conversation=pair_spend,
     per_turn=512))
solo_report = solo.rollout(n=64)
# or ambiently over any run/rollout in the block — the budget is PER
     conversation, not a shared pool:
with TokenBudget(per_conversation=2048):
    conv.rollout(n=200)                        # every one of the 200
     conversations gets its own 2048-token budget
```

Counting is free (summed from each turn's recorded `metadata['n_tokens']`, never re-tokenized) and per-conversation (read from the conversation's own transcript). Verify realized spend off the report: `report.results[jid].tokens_generated`.

Large API rollouts (batch mode)

For hosted models (`APIParticipant`), set `batch=True` to route each round's turns through the provider's **asynchronous batch API** — ~50% cost and much higher throughput, at the price of batch-window latency:

```
from interlens import Conversation, APIParticipant
report = (Conversation(participants=[
              APIParticipant(name="pro", provider="openai",
     model_id="gpt-5",           batch=True, system_prompt="Argue YES."),
              APIParticipant(name="con", provider="anthropic",
     model_id="claude-sonnet-5", batch=True, system_prompt="Argue NO.")],
              shared_context="Is a hot dog a sandwich?")
          .turns(4)
          .rollout(n=200))   # each round → one Anthropic/OpenAI batch
     per speaker
```

- **Batch APIs exist only for `provider="anthropic"` and `provider="openai"`.** Requesting `batch=True` on `provider="openrouter"` **raises** (OpenRouter has no batch endpoint) — it never silently degrades to serial calls.
- With `batch=False` (default) API turns run as ordinary concurrent single requests, bounded by the shared client's max-in-flight semaphore.

See participants & models for provider setup + API keys.

Execution modes & reproducibility

- **Batched co-stepping is the default** (`batched=True`): local `ModelParticipant`s (every family) co-step, trading exact per-row reproducibility for throughput — batch composition + a single global RNG perturb rows, so only *distributional* reproducibility holds. Batched turns are marked in `metadata["batched"]`; the shared-prefill fast path (turn 1 of an `n`-sample rollout off one scenario) in `metadata["shared_prefill"]`.
- `batched=False` is the **`ExecutionMode.DETERMINISTIC`** escape hatch: each rollout runs independently and is **token-identical** on the same hardware (pin per-participant `seed=` and `kv_reuse=False`). It is the only path that supports per-turn interp (capture/steering/probes) — batched generation cannot.

Next: advanced interp pipelines.

04 · Context management & serialization

Context-window policies

Long conversations overflow the model's context. A `ContextPolicy` decides what to keep. Crucially it runs on the **typed segments** (system / moderator / private_context / turns) *before* the family flatten, so framing is preserved reliably.

```
from interlens import (
    Conversation, ErrorPolicy, DropOldestPolicy, SlidingWindowPolicy,
     SummarizePolicy,
)
# ErrorPolicy (default): raise if the view exceeds context_limit — never
     silently truncate.
conv = Conversation(participants=(alice, bob),
     context_policy=ErrorPolicy(), context_limit=8192)
# SlidingWindowPolicy: keep framing + the most recent `keep_last` turns;
     drop older ones.
conv = Conversation(participants=(alice,
     bob), context_policy=SlidingWindowPolicy(keep_last=8))
# DropOldestPolicy: drop oldest turns until it fits (needs context_limit).
conv = Conversation(participants=(alice, bob),
     context_policy=DropOldestPolicy(), context_limit=8192)
# SummarizePolicy: replace older middle turns with a summary; keep framing
     + last `keep_last` verbatim.
conv = Conversation(
    participants=(alice, bob),
    context_policy=SummarizePolicy(keep_last=4, summarizer=lambda turns:
     "Earlier: " + " | ".join(turns)),
)
```

`context_limit=None` (default) means the tokenizer's own `model_max_length` is used.

Serialization: recipe + transcript

There is no separate template type: a `Conversation` **is** its own recipe. Because its participants are lazy (they hold an HF id + settings, not weights), an unrun

conversation is a cheap, serializable spec — build it, ship it to workers, or run it many times, all as one object.

```
from interlens import Conversation, AutoModelParticipant,
     SlidingWindowPolicy
conv = Conversation(
    participants=[
        AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-Instruct",
     name="alice", temperature=0.7, system_prompt="Be terse."),
        AutoModelParticipant.from_pretrained("google/gemma-2-2b-it",
     name="bob", temperature=0.9),
    ],
    shared_context="Debate: is a hotdog a sandwich?",
    shared_system_prompt="Stay civil.",
    context_policy=SlidingWindowPolicy(keep_last=8),
    reasoning_visibility="strip",
).turns(6)                                  # rollout/data fields via
     dot-modifier sugar
conv.run(turns=6)                           # loads the models lazily on
     first use
conv.save("runs/hotdog")                    # writes conversation.json
     (recipe) + transcript.json
later = Conversation.load("runs/hotdog")    # rebuilds lazy participants
     + attaches the transcript
```

`save` records each participant's own constructor kwargs (HF id + `dtype`/`attn`/`quant`/`revision`/`max_new_tokens`/`temperature`/`top_p`/`seed`/`thinking`/tool names/`max_tool_iters`/`kv_reuse`, or an API provider + model id) plus the scenario framing/policies — never weights. `load` raises on an unsupported (older) schema rather than silently mis-reading it.

Copy-on-write updates: `conv.set(field=value)` (or a dot-modifier like `conv.turns(6)`) returns a modified *copy* sharing the loaded models by reference — the original is untouched.

Save / load a whole conversation (recipe + transcript)

```
conv.save("runs/debate_001")                    # writes
     conversation.json (recipe) + transcript.json
resumed = Conversation.load("runs/debate_001", devices="cuda")   #
     rebuilds lazy participants, ATTACHES the transcript
resumed.run(turns=4)                            # continues from where it
     left off (does not regenerate)
```

`load` takes `devices=` as a single device or a list (participants are round-robined across the list — handy for putting two big models on two GPUs), and raises on an unsupported (older) schema rather than silently mis-reading it.

Next: tools · hooks · interpretability · rollouts.

03 · Participants & models

How models resolve

There are **no short names** — you pass an **HF id or local path** (`str | Path`, or an already-loaded model), and interlens picks the family-correct participant class from the model's own **`config.model_type`** (exactly like HuggingFace `AutoModel`), then derives its chat-template flags by probing the tokenizer. Any model of a supported family Just Works:

```
from interlens import AutoModelParticipant
p = AutoModelParticipant.from_pretrained("google/gemma-2-2b-it", name="p")
     # → GemmaModelParticipant
q = AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-Instruct",
     name="q")  # → QwenModelParticipant
r = AutoModelParticipant.from_pretrained("mistralai/Mistral-7B-Instruct-
     v0.3", name="r")  # → base ModelParticipant
```

The `config.model_type → class` mapping is a **class self-registry**: each participant subclass declares the model types it handles in a `MODEL_TYPES` class attribute, and `ModelParticipant.__init_subclass__` records them — there is no central table to keep in sync. Unregistered families (Mistral, OLMo, Phi, DeepSeek, …) fall back to the base `ModelParticipant` automatically, and a brand-new family/size resolves the moment it's on the Hub.

Weights load lazily. `from_pretrained` does **not** touch weights — it reads only the model's `config.json` to resolve the family, and defers the actual load to the first time the model is used (or `p.model`/`p.tokenizer` is accessed). So an unbuilt participant is a cheap recipe: it costs KBs, pickles across a spawn boundary without shipping weights (each GPU worker loads on its own device), and `.set(...)` copies share one loaded model object by reference. The load is process-cached on `(hf_id, device, dtype, attn, quant, revision)`, so every same-recipe participant on a device shares ONE object — which is exactly what lets the runner batch them into a single `model.generate`. The default device is `cuda`; the runner binds each participant's device per worker before the load fires.

Static types when you want them. `AutoModelParticipant.from_pretrained` is *dynamically* dispatched, so its declared return type is the base `ModelParticipant` — except for known id literals, which a type stub (`factories.pyi`) narrows to the concrete subclass. For a guaranteed static type regardless of the id, **name the class directly** — it returns `Self`:

```
from interlens import QwenModelParticipant
q = QwenModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-Instruct",
     name="q")  # statically QwenModelParticipant
```

Family behavior is data-driven, not hand-declared. Whether a template accepts a standalone system role or requires strictly alternating turns is **auto-derived from the tokenizer** (`interlens.loading.derive_chat_flags`), so e.g. Gemma 2 (folds system into the first user turn) and Gemma 3 (accepts a system role) are handled correctly with zero per-model config. A subclass exists only when a family needs distinct behavior — Gemma's `tool_code` and Llama's ◁`python_tag`▷ tool-call formats — or, for Qwen, purely so its models get a distinct statically-typed class (its `<tool_call>` JSON is already the base format). A slow test (`tests/test_family_flags.py`) checks the derivation against real tokenizers.

Load weights directly

```
import torch
from interlens.loading import load_model
model, tok = load_model("Qwen/Qwen3-8B", device="cuda",
     dtype=torch.bfloat16, attn="flash_attention_2")
```

`load_model` shares a process-local cache: identical `(hf_id, device, dtype, attn, quant, revision, max_memory)` returns the same model object; each HF id caches its own tokenizer. Flash-attention is the default with automatic fallback to sdpa/eager; `quant="4bit"`/`"8bit"` is opt-in (perturbs activations → interp fidelity).

Models too big for one GPU

Pass a **sharding strategy** as `device` and it is routed to `device_map=` (the `.to(device)` is skipped, since moving a sharded model would undo the placement). `max_memory` reserves per-device headroom — worth setting

whenever you shard, because the automatic map sizes for *weights* while your peak is usually KV cache and activations.

```
model, tok = load_model(
    "Qwen/Qwen3-32B",
    device="balanced",                                          # or
     "auto" / "balanced_low_0" / "sequential" / a {module: device} dict
    max_memory={0: "70GiB", 1: "70GiB", "cpu": "0GiB"},        # "cpu":
     "0GiB" forbids offload → an actionable OOM instead of a silent 100x
     slowdown
)
```

Once a model is sharded, `model.device` is **not** where inputs go — the input embedding's device is. Use `input_device`; it returns the single device unchanged for an ordinary load, so the same code covers both:

```
from interlens.loading import input_device, is_sharded
ids = tok("hello", return_tensors="pt").input_ids.to(input_device(model))
is_sharded(model)      # True only when device_map spread it over more
     than one device
```

`ModelParticipant` does this for you (`p.input_device`), as do `capture_router_logits` and `span_pooled_residuals`.

`ModelParticipant` knobs

```
from interlens import AutoModelParticipant
p = AutoModelParticipant.from_pretrained(
    "Qwen/Qwen3-4B", name="p", device="cuda",
    load_kwargs={"attn": "sdpa"},   # optional: forwarded to load_model
     (dtype/attn/quant/revision)
    temperature=0.8, top_p=0.95, max_new_tokens=512,
    seed=1234,               # per-participant RNG seed → reproducible
     greedy/sampled turns (local models only)
    thinking="auto",         # "auto" defers to the template; True/False
     forces enable_thinking where supported
    system_prompt="…",
    kv_reuse="auto",         # cross-turn KV prefix reuse; see below
)
```

Already hold weights (e.g. sharing them, or an externally-loaded checkpoint)? Wrap them with `from_model`:

```
from interlens.loading import load_model
model, tok = load_model("Qwen/Qwen3-4B")
p = AutoModelParticipant.from_model(model, tok, name="p", temperature=0.8)
     # family from model.config.model_type
```

`kv_reuse` (cross-turn KV cache)

`"auto"` (default) reuses the KV cache across a speaker's own consecutive turns when the new prompt exactly extends the cached tokens — skipping a full re-prefill. It is doubly guarded (exact-prefix check + safe fallback) and **auto-disables under steering/patch and batched generation**.

```
p.kv_reuse = "auto"   # default: enabled when safe
p.kv_reuse = False    # force off — pin this for determinism-critical /
     reproducibility experiments,
                      #   since reuse can perturb outputs at the FP level
     vs a full prefill.
```

Enable `logging` at INFO to see the per-participant decision, DEBUG to see reuse engage per turn:

```
import logging; logging.basicConfig(level=logging.INFO)
# INFO  ...model_participant: p: cross-turn KV reuse ENABLED
     (kv_reuse='auto')
```

API-backed participants

`APIParticipant` is a full conversational participant with **no local model** — use it as an opponent, moderator, or judge. Interp requests (`capture`/`steering`/`patch`/`return_logprobs`) **raise** rather than silently no-op (a steering sweep that quietly did nothing would fabricate a "no effect" result).

```
from interlens import Conversation, APIParticipant, AutoModelParticipant
local = AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-Instruct",
     name="student")
judge = APIParticipant(name="tutor", model_id="claude-sonnet-5",
     provider="anthropic",
                       system_prompt="You are a patient tutor. Ask one
     probing question per turn.",
                       max_tokens=400, temperature=1.0)
```

```
conv = Conversation(participants=(local, judge), shared_context="Teach the
     student about entropy.")
conv.run(turns=6)
```

Requires `ANTHROPIC_API_KEY` (or `ANTHROPIC_API_KEY_FILE`) and outbound network. For tests, inject a fake `client=callable(system, messages, model, max_tokens, temperature) → str`.

OpenAI (`provider="openai"`) calls OpenAI directly via the `openai` SDK — needs `OPENAI_API_KEY` and the `[api]` extra:

```
judge = APIParticipant(name="judge", provider="openai", model_id="gpt-5",
     max_tokens=400)
```

OpenRouter (`provider="openrouter"`) reaches any model behind openrouter.ai through one OpenAI-compatible endpoint — needs `OPENROUTER_API_KEY` and the `[api]` extra (`pip install "interlens[api]"`, which pulls `openai`):

```
from interlens import OpenRouterRouting
judge = APIParticipant(name="judge", provider="openrouter",
                       model_id="meta-llama/llama-3.1-70b-instruct",   #
     or openai/gpt-5, anthropic/claude-sonnet-5, …
                       openrouter_routing=OpenRouterRouting(
                           upstream_provider="together",
                           quantizations=("bf16",),
                           data_collection="deny",
                       ),
                       max_tokens=400)
```

OpenRouter routing is fail-closed for research: a request must pin one upstream provider (fallbacks are disabled and parameter support is required), or explicitly opt into uncontrolled exploratory routing with `OpenRouterRouting.unpinned()`. Each returned turn records the requested routing object, actual upstream provider, resolved model, and generation ID in `Message.metadata`; a missing or mismatched provider aborts the turn.

All providers share one retry/backoff + max-in-flight client, built lazily per provider.

Batch mode for large rollouts

`provider="anthropic"` and `provider="openai"` expose asynchronous **batch APIs** (Anthropic Message Batches / OpenAI Batch) — ~50% cost and much higher throughput, in exchange for batch-window latency. Set `batch=True` on the participant and run the rollout in throughput mode; the co-stepper collects each round's same-position turns across all rollouts into **one** provider batch:

```
pro = APIParticipant(name="pro", provider="openai", model_id="gpt-5",
     batch=True, system_prompt="Argue YES.")
con = APIParticipant(name="con", provider="anthropic", model_id="claude-
     sonnet-5", batch=True, system_prompt="Argue NO.")
report = (Conversation(participants=[pro, con], shared_context="…")
          .turns(4).rollout(n=200))   # batched=True is the default; each
     round → one batch per speaker
```

Batch mode is **unavailable on OpenRouter** — requesting `batch=True` there raises rather than silently degrading to serial calls, so a requested batch discount is never quietly dropped. Under the hood every participant/client also exposes `generate_batch(views)` / `client.submit_batch(requests)` directly if you want to batch outside a rollout.

Per-turn `max_new_tokens`

`temperature`/`top_p`/`seed` are set once per participant, but generation *length* often varies per turn (a one-word verdict vs a paragraph). `step`/`sample` take an optional `max_new_tokens` override for exactly that:

```
conv.sample("a", "Answer in one word.", max_new_tokens=8)     # short
conv.step(conv.participant("a"), max_new_tokens=400)             # long
```

Next: context management & serialization.

05 · Tools

A `Tool` is a capability a participant can invoke mid-turn. The harness runs a uniform tool loop; each family parses its own native call format (Hermes/Qwen `<tool_call>` JSON, Gemma ```` ```tool_code ````, Llama `◁python_tag▷`). Tools hold live callables, so they are **not** serializable — templates store tool *names* and resolve them against a `ToolRegistry` at build time (mirroring how models resolve from ids).

Define and register a tool

```python
from interlens import Tool, DEFAULT_REGISTRY

class Calculator(Tool):
    name = "calculator"

    @property
    def schema(self) -> dict:
        return {
            "type": "function",
            "function": {
                "name": "calculator",
                "description": "Evaluate a basic arithmetic expression and
     return the result.",
                "parameters": {
                    "type": "object",
                    "properties": {"expression": {"type": "string",
     "description": "e.g. '2 * (3 + 4)'"}},
                    "required": ["expression"],
                },
            },
        }

    def __call__(self, expression: str) -> str:
        return str(eval(expression, {"__builtins__": {}}))   # sandbox
     eval for the example only

DEFAULT_REGISTRY.register(Calculator())   # now resolvable by the name
     "calculator"
```

The `schema` is passed straight to `apply_chat_template(tools=...)`, so use the standard function-calling shape.

Give a participant tools

Live participant

```
from interlens import AutoModelParticipant
solver = AutoModelParticipant.from_pretrained(
    "Qwen/Qwen2.5-7B-Instruct", name="solver",
    tools=(Calculator(),),     # the tool objects
    max_tool_iters=4,          # bound the call→result→call loop
)
```

On a participant

```
from interlens import Conversation, AutoModelParticipant
solver = AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-7B-Instruct",
     name="solver",
                                                tools=(Calculator(),),
     max_tool_iters=4)
conv = Conversation(participants=[solver], shared_context="Compute 17 * 23
     using your tool, then explain.")
conv.run(turns=2)
```

Tools are resolved by name from `DEFAULT_REGISTRY` only when a saved conversation is reloaded (`Conversation.load`) — `save` stores each participant's tools by name; pass `registry=` there for a scoped registry.

What the loop does

For each turn the model may emit tool calls; the harness executes each, appends the call + result to a *private* working view, and lets the model react — up to `max_tool_iters` times. Only the **final natural-language message** reaches the shared transcript; the full call/result trail is kept in `msg.metadata["tool_trail"]`. Tool exceptions are returned to the model as `error: …` results (data, not crashes), so a failing tool doesn't kill the run.

```
msg = conv.transcript[-1]
for step in msg.metadata.get("tool_trail", []):
    print(step)   # {"name": "calculator", "arguments": {...}, "output":
     "391", "error": False}
```

Use a scoped registry instead of the global one by passing `registry=` to `Conversation.load`.

Next: hooks.

06 · Message hooks

A `MessageHook` is middleware that inspects every freshly generated message **before it is committed** to the transcript, and may **approve**, **deny** (drop the turn), or **edit** (replace it). Hooks are runtime policy — they live on the live `Conversation` (`conversation.message_hooks`) and are **not** serialized in the template. This is the seam for a safety filter, format enforcer, or LLM-judge that vets/rewrites turns.

Interface

```python
from interlens import MessageHook, MessageHookResult

class NoProfanity(MessageHook):
    BANNED = {"heck", "darn"}
    def review(self, message, conversation) → MessageHookResult:
        if any(w in message.content.lower() for w in self.BANNED):
            return MessageHookResult.deny()                # drop the
     turn entirely
        return MessageHookResult.approve()                 # let it
     through unchanged

class EnforceBrevity(MessageHook):
    def review(self, message, conversation):
        if len(message.content) > 500:
            from interlens import Message
            trimmed = Message(author=message.author,
     content=message.content[:500] + " …",
                              metadata={**message.metadata, "trimmed":
     True})
            return MessageHookResult.edit(trimmed)         # substitute
     a replacement
        return MessageHookResult.approve()
```

`MessageHookResult`

factory helpers: `.approve()`, `.deny()`, `.edit(replacement_message)`.

Attach hooks

```python
conv.message_hooks = [NoProfanity(), EnforceBrevity()]     # applied in
     order
result = conv.step(alice)                                  # returns None
     if a hook DENIED the turn
```

Hooks run inside both `step` and `run`. A denied turn commits nothing and `step` returns `None`; `run` simply moves on (and a denied message is not checked against stop conditions).

An LLM-judge hook

Because `review` receives the live `conversation`, a hook can call an API model (or a local one via `conv.sample`) to score the turn:

```
from interlens import MessageHook, MessageHookResult, APIParticipant

class JudgeHook(MessageHook):
    def __init__(self):
        self.judge = APIParticipant(name="judge", model_id="claude-
     sonnet-5",
                                    system_prompt="Reply only 'OK' or
     'BLOCK'. BLOCK if the message is off-topic.")
    def review(self, message, conversation):
        verdict = self.judge.generate([{"role": "user", "content":
     message.content}]).content
        return MessageHookResult.deny() if "BLOCK" in verdict else
     MessageHookResult.approve()

conv.message_hooks = [JudgeHook()]
```

Next: interpretability.

07 · Interpretability

All four tools hook into the **same** generation path as real turns, `sample`, and tool loops, and are tagged to conversation structure. They apply to **local `ModelParticipant`s only** — an `APIParticipant` raises on any interp request rather than silently no-op'ing. Pass them to `conv.step(...)` or `conv.sample(...)`.

Activation capture

`conv.capture(...)` is a context manager: every `step`/`sample` inside the block records activations into a fresh `ActivationCache`, auto-tagged by speaker + turn.

```
with conv.capture(sites=["residual"], layers=[8, 12, 16], offload="cpu")
     as cache:
    conv.step(conv.participant("bob"))
    conv.step(conv.participant("alice"))

# Query the cache by structure:
act = cache.at(participant="bob", layer=12, site="residual")   # exactly
     one tensor, [seq, d_model]
records = cache.query(participant="bob")                        # all
     matching ActivationRecords
```

- **`sites`**: `"residual"` (per-layer residual stream), `"attn"` (attention sublayer output, post-o_proj), `"mlp"` (MLP output). Capture is a single clean forward over the full prompt+generation.
- **`layers`**: a list, or `None` for all layers. Keep it **narrow** — all layers × all tokens × many rollouts OOMs fast.
- **`offload`**: `"cpu"` moves tensors off-GPU as captured (essential for sweeps; uses a batched pinned transfer under the hood — ~7× faster than naive `.to('cpu')`); `None` keeps them on-device.

Each `ActivationRecord` carries `.participant`, `.message_idx`, `.layer`, `.site`, `.tensor` (`[seq, d_model]`), `.token_span` (prompt_len, seq_len), and `.phases` — a `{prompt/reasoning/answer: (start, end)}` map so you can slice reasoning-vs-answer activations for CoT models:

```
rec = cache.query(participant="bob", layer=12)[0]
a0, a1 = rec.phases["answer"]
answer_acts = rec.tensor[a0:a1]        # activations over just the answer
     tokens
```

Build a probe direction from captured activations

```
import torch
# e.g. mean residual at layer 12 over the answer span for two conditions →
     a difference-of-means direction
pos = cache.at(participant="bob", layer=12)[rec.phases["answer"]
     [0]:].mean(0)
# (collect a `neg` from another branch/condition similarly)
direction = (pos - neg)
direction = direction / direction.norm()
```

Steering — add or ablate a direction

`SteeringSpec` registers forward hooks that either **add** `coef * direction` to the residual at `layers`, or **ablate** (project out) that direction.

```
from interlens import SteeringSpec

steer = SteeringSpec(direction=direction, layers=(8, 12), coef=6.0,
     mode="add")
msg = conv.sample("bob", "How do you feel about the proposal?",
     steering=steer)  # ephemeral, steered

ablate = SteeringSpec(direction=direction, layers=(12,), mode="ablate")
     # remove the component
conv.step(conv.participant("bob"), steering=ablate)
     # committed, ablated turn
```

`direction` is a `[d_model]` tensor on any device (moved to match). A summary (mode, layers, coef, direction norm) is recorded in `msg.metadata["steering"]` so a steered turn is reproducible. Steering **disables KV reuse** automatically (the intervention wasn't in the cached KV).

Activation patching — cross-branch causal tracing

`Patch` overwrites a decoder layer's residual at specific token `positions` with activations captured elsewhere (e.g. from another branch). This is the causal-tracing primitive: capture at turn N in one branch, inject at the aligned positions of another branch's forward.

```
from interlens import Patch

# 1) capture bob's layer-12 residual in the clean branch
clean = conv.branch()
with clean.capture(sites=["residual"], layers=[12]) as cache:
    clean.step(clean.participant("bob"))
donor = cache.at(participant="bob", layer=12)          # [seq, d_model]

# 2) inject those activations at chosen positions in a corrupted branch's
     next forward
positions = (3, 4, 5)
patch = Patch(activations=donor[list(positions)], layer=12,
     positions=positions)
corrupt = conv.branch()
corrupt.transcript[-1] = ...                            # (corrupt the
     setup somehow)
patched_msg = corrupt.sample("bob", patch=patch)       # bob generates
     with layer-12 residual patched
```

Aligning positions across branches is the caller's responsibility; `Patch` just performs the overwrite (only on the prefill forward, not per-token decode). Patching also disables KV reuse.

Token logprobs / surprisal / entropy

```
msg = conv.sample("alice", "State your final answer in one word.",
     return_logprobs=True)
md = msg.metadata
md["logprobs"]    # per generated token: log p(token)
md["surprisal"]   # -logprob (nats)
md["entropy"]     # full next-token distribution entropy at each step
     (model uncertainty)
```

These are scalar-per-token lists (safe to store in metadata). Works with `step` too.

Next: advanced pipelines that combine these, or scaling to rollouts.

09 · Advanced interp pipelines

Compositions that combine capture, branching, steering, patching, and `analyze` into real experiments. These lean on the fact that **`branch()` is free** (shared weights) and every interp tool rides the same generation path.

1 · Difference-of-means direction → causal steering test

Find a direction that separates two conditions, then verify it's *causal* by steering with it.

```
import torch
from interlens import Conversation, SteeringSpec

conv = Conversation.from_models(("Qwen/Qwen2.5-3B-Instruct", "Qwen/
     Qwen2.5-3B-Instruct"), names=("a", "b"))
conv.transcript.append("a", "Tell me your honest opinion of pineapple
     pizza.")

LAYER = 14
def mean_answer_act(prompt):
    b = conv.branch()
    with b.capture(sites=["residual"], layers=[LAYER]) as cache:
        b.sample("b", prompt)                       # ephemeral;
     nothing committed
    rec = cache.query(participant="b", layer=LAYER)[0]
    lo, hi = rec.phases["answer"]
    return rec.tensor[lo:hi].mean(0)

pos = mean_answer_act("Respond with maximum enthusiasm.")
neg = mean_answer_act("Respond with cold indifference.")
direction = (pos - neg); direction /= direction.norm()

# Causal check: does adding the direction shift behavior?
for coef in (0.0, 4.0, 8.0):
    steer = SteeringSpec(direction=direction, layers=(LAYER,), coef=coef,
     mode="add")
    print(coef, "→", conv.sample("b", "Your opinion?",
     steering=steer).content[:100])
```

2 · Activation patching across clean/corrupt branches (causal tracing)

Localize *where* a piece of context matters by transplanting clean activations into a corrupted run, layer by layer.

```
from interlens import Patch

base = conv.branch()
base.transcript.append("a", "The secret code is FALCON. What is the secret
     code?")

# clean: capture residuals everywhere we might patch
with base.capture(sites=["residual"], layers=list(range(0, 28, 4))) as
     clean_cache:
    base.step(base.participant("b"))

# corrupt: overwrite the code, then patch one layer's residual back from
     the clean run
def patched_answer(layer, positions):
    donor = clean_cache.at(participant="b", layer=layer)
    patch = Patch(activations=donor[list(positions)], layer=layer,
     positions=positions)
    c = conv.branch()
    c.transcript.append("a", "The secret code is SPARROW. What is the
     secret code?")
    return c.sample("b", patch=patch).content

for L in range(0, 28, 4):
    print(L, "→", patched_answer(L, positions=(6, 7, 8)))   # which layer
     restores "FALCON"?
```

3 · Probe-in-the-loop `analyze` at scale

Collect an activation-based measurement over many rollouts. The probe read happens **inside the worker** while models are resident; only the scalar crosses back.

```
from interlens import Conversation, AutoModelParticipant,
     register_analyzer
import torch

PROBE = torch.load("stance_probe_layer14.pt")   # a [d_model] direction
     you fit earlier

def project_stance(conv):
    with conv.capture(sites=["residual"], layers=[14]) as cache:
        conv.sample("a", "State your position plainly.")
```

```
    rec = cache.query(participant="a", layer=14)[0]
    lo, hi = rec.phases["answer"]
    v = rec.tensor[lo:hi].mean(0).float()
    return {"stance_proj": float(v @ PROBE.to(v.device) / PROBE.norm())}

register_analyzer("project_stance", project_stance)

conv = Conversation(
    participants=[AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-
     Instruct", name="a"),
                  AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-
     Instruct", name="b")],
    shared_context="Debate: should we colonize Mars?",
).turns(6).analyzer("project_stance")
report = conv.rollout(n=128, out_dir="runs/mars")
projs = [r.analysis["stance_proj"] for r in report.results.values() if
     r.error is None]
print("mean stance projection:", sum(projs) / len(projs))
```

4 · Steering sweep with reproducible turns

Because `sample` is ephemeral and `branch` is free, a full coefficient × layer sweep is a couple of loops — pin `seed` so each cell is comparable.

```
conv.participant("b").seed = 0                          # fix RNG so only
     the intervention varies
results = {}
for layer in (8, 12, 16, 20):
    for coef in (2.0, 4.0, 8.0):
        steer = SteeringSpec(direction=direction, layers=(layer,),
     coef=coef, mode="add")
        results[(layer, coef)] = conv.sample("b", "Your opinion?",
     steering=steer).content
```

Gotchas worth remembering

- Interp on an `APIParticipant` **raises** — use a local `ModelParticipant`.
- Steering/patch **disable KV reuse** automatically (the cached KV predates the intervention).
- Keep `capture` **narrow** (few layers, `offload="cpu"`) — all-layers × all-tokens × many-rollouts OOMs.

- `branch()` shares participant *objects*; mutating a participant's `.temperature`/`.seed` on a branch affects the shared object — set them per use or re-set before each run.

Back to the index.

10 · Arena: scoreable multi-agent evaluations

`interlens.arena` turns the conversation harness into an **evaluation harness**: a `Scenario` defines a game — solver-verified instance generation, per-seat private framing, a turn protocol with structured JSON actions, early termination, exact scoring — and the engine plays it through any `Participant`, persisting every episode to one JSON schema. Five scenario families ship: a multi-issue **negotiation** with secret score sheets, a wrong-shard **info relay** (an epistemic team task with a confidently-wrong agent), a repeated **security dilemma** (build/deescalate/attack with an absorbing war spiral and noisy intelligence), a **coding collaboration** with private mechanically-checkable constraints (sandboxed pytest scoring), and a **distributed long-context** family (one long-context benchmark task partitioned across 4 seats). All come from the collaboration-arena research program, and stored episodes from those experiments re-score exactly under this package.

Run an episode

```
import asyncio
from interlens import APIParticipant, UsageMeter
from interlens.arena import EpisodePool, EpisodeStore
from interlens.arena.scenarios import Negotiation

scenario = Negotiation()
instance = scenario.generate_instance(level=0, seed=1)   # exact ceiling,
     floor + hidden optimum
meter = UsageMeter(budget=25.0)                          # run-level $
     ledger with a hard cap
player = APIParticipant(name="player", model_id="claude-sonnet-5",
                        meter=meter, turn_token_floor=2048)

pool = EpisodePool(EpisodeStore("episodes/"), meter=meter)
episode = asyncio.run(pool.run_episode(scenario, instance, arm="team",
     participant=player))
print(episode.outcome)        # {"primary": ..., "success": ...,
     "finalized_by": ..., ...}
print(episode.usage())        # tokens in/out (total + per seat), dollar
     cost
print(meter.summary())        # run-level spend, per model
```

Every seat is played by `player` (one model per episode — the design the arena experiments measured). The `EpisodeStore` writes the full record atomically after every turn wave; `store.summary()` aggregates episodes into a printable usage/success table.

Arms, budgets, and matched compute

- `arm="team"` plays the full multi-seat protocol; `arm="solo"` gives ONE seat all private information (the "one mind with all the facts" baseline). Scenario variants like `team-greedy` / `team-adversarial` change one seat's private stance.
- Budgets are **stop conditions**, not ad-hoc counters: pass `budget=TokenBudget(per_conversation=6000)` (or a `CostBudget`, or a list) to `run_episode`. The engine counts each committed turn's recorded usage, shrinks per-turn caps (`turn_cap`) so the budget lands exactly, and — when the budget fires — sets `state["budget_exhausted"]`, which both scenarios answer with a forced finalization. That is the **matched-compute** recipe: give a solo baseline the team arm's median token spend, then force it to answer.
- Run-level spend is **reservation-gated**: jobs carrying `estimated_cost=` claim it against the meter *before* launching, so N concurrent episodes can never collectively overrun the cap (`run_pool` also stops launching once the meter is exhausted; in-flight episodes finish).

```
jobs = [dict(scenario=scenario, instance=inst, arm="team",
     participant=player, estimated_cost=4.0)
        for inst in instances]
episodes = asyncio.run(pool.run_pool(jobs))
```

For **local models**, `BatchedEpisodePool` co-steps many episodes as one batched `generate_batch` per tick (with adaptive batch splitting on GPU OOM) — the multi-GPU rollout path.

Situational sweeps

Both scenarios take a `cfg` dict (`make_state(..., cfg=...)`, or `cfg=` on `run_episode`) that varies the *situation* around the fixed game — rounds, stakes framing, personas, party count (negotiation, via `generate_instance_n`), hardness/framing (relay). The episode record carries the resolved cell config, so sweeps join cleanly.

Two hard-won gotchas, now guarded

1. **Thinking × turn caps.** Models with adaptive/always-on reasoning spend hidden thinking tokens *out of* `max_tokens`. A small per-turn cap (500) silently produced **58% empty turns** in a real run — thinking consumed the whole budget, and the "negotiation" proceeded among mutes. Two controls on `APIParticipant`: `thinking="disabled"` (or an explicit int budget — Anthropic models) is the reliable fix, and `turn_token_floor=2048` raises any externally-imposed cap back to a generable size for models whose thinking can't be disabled. On very long contexts adaptive thinking can outgrow ANY fixed floor — a live 3-round negotiation with claude-sonnet-5 produced empty final-proposal turns even at a 4096 cap until thinking was disabled.
2. **Seat-selective refusals.** Hosted models occasionally refuse *from one seat's framing* and not others', silently biasing team results. Refusals are classified per turn (`metadata["refusal"]`, from the provider's native stop reason) and counted per model on the `UsageMeter` — check `meter.summary()` before believing a team-vs-solo delta.

Preflight gates for local rollouts

```
from interlens.arena import check_template_fidelity, check_reasoning_leak,
    scenario_smoke_views
views = scenario_smoke_views(scenario)
```

```
assert check_template_fidelity(tokenizer, views, enable_thinking=True)
     ["ok"]   # token-id equality
# ... play one smoke episode, then:
assert check_reasoning_leak(episode)["ok"]   # no <think> content leaked
     into other seats' views
```

The second scenario wave

Security dilemma (`SecurityDilemma`, no solo arm)

Two seats play 12 rounds of message-then-simultaneous-action (`build` / `deescalate` / `attack`). Once BOTH attack in the same round, war spirals: every remaining round is forced mutual attack. Intelligence is noisy — with a level-dependent probability, a seat's *report* of the opponent's action is flipped; true actions always drive payoffs, and each seat sees only its own report and payoffs, so misreports are seeded, private, and replayable. The difficulty ladder raises the first-strike bonus and the noise. `primary` = joint payoff / 96 (mutual deescalation for 12 rounds); `success` = no spiral.

Coding collaboration (`CodingCollab`)

Three seats jointly write ONE Python module against a public spec + pytest suite; each seat holds private style constraints (AST-checkable: no loops, full type hints, line caps, …) it must get honored WITHOUT revealing verbatim. The latest complete ```` ```python ```` fence anywhere is the working draft; per-turn `{"constraints_ok": true|false}` declarations drive early consensus. Scoring is exact and sandboxed: `primary` = (tests passing) × (constraints held), with the suite run in an isolated subprocess (`sys.executable -I`, fresh tmpdir, 30 s timeout). The generator verifies every dealt constraint set against a bundled reference solution, so `primary` $= 1.0$ is always achievable.

Distributed long-context (`DistributedLongContext` + task adapters)

One long-context task is partitioned into 4 contiguous shards (asserted: their concatenation reproduces the original context exactly); each seat privately holds one shard, and the finalizer (seat 0, last each round) submits the answer. Three arms: `team` (round-robin broadcast), `team-msg` (directed messaging — each non-finalizer turn is only a fenced `{"messages": [...]}` routing object; a seat sees only mail addressed to it — the routing is part

of the state machine, so episodes replay), and `solo` (the full concatenated context in one seat).

Task adapters (`interlens.arena.scenarios.dlc`) port four benchmarks: **S-NIAH** (RULER-style needle in a haystack, exact match), **OOLONG-Pairs** (pairwise aggregation, F1 over the emitted pair set), **LongBench-v2 CodeQA** (repo-understanding multiple choice), and **BrowseComp-Plus** (multi-hop QA; two-phase — the scenario records the answer un-judged, and the official grader template ships for a post-hoc LLM-judge pass). Benchmark data does **not** ship in the repo: `interlens.arena.scenarios.dlc.build` fetches from each benchmark's own source at a pinned revision (`pip install interlens[benchmarks]`) and writes a `load_instances`-compatible bank; instances embed megabytes of context, so they are built offline, once.

Outcome classes. Long-context episodes fail in ways a bare score conflates, so two classes are first-class outcome fields, applied by the engine and recomputed identically in replay (`Scenario.classify_outcome`):

- `truncated_at_budget` — some committed turn stopped at its `max_tokens` cap. The episode ran out of room; exclude it from success/failure analysis rather than scoring the accident.
- `capitulated` (OOLONG-Pairs) — NOT truncated, but the answer enumerated <50% of the gold pairs: the model *declined* the enumeration. The classification carries evidence — how many relevant user IDs the team had actually surfaced in discussion vs how many pairs it emitted.
- otherwise `answered` / `no_answer`.

Adaptive difficulty ratchet

A ceiling-saturated cell measures nothing. `DifficultyRatchet` drives any scenario with a difficulty ladder to its *found level* — the first level whose probe mean drops below 75% of ceiling — then measures there and at the neighbor level, then runs paired solo baselines on the SAME instances under a `TokenBudget` equal to the median team spend (matched compute):

```
from interlens.arena import DifficultyRatchet, EpisodePool, EpisodeStore

ratchet = DifficultyRatchet(scenario, player,
     EpisodePool(EpisodeStore("episodes/"), meter=meter),
                             instances_dir="instances/",
     state_path="ratchet.json",
                             speculative=True)   # probe the first 3
     levels concurrently
state = asyncio.run(ratchet.run())              # {"found": ...,
     "probe_means": ..., ...}
```

State persists after every batch: a restarted ratchet resumes where it stopped and never duplicates a completed episode (instances are deterministic per level and skipped by id, so team/solo pairings stay intact). `speculative=True` probes the first wave of levels concurrently — faster wall-clock at the cost of probing levels a sequential climb might have skipped; the found decision is the same pure function (`found_level`) either way.

Rollout sets: more episodes, into the same place, safely

`EpisodePool` runs episodes; `RolloutSet` is the bookkeeping *around* a set of them, so "run twenty more seeds" is one call instead of a new directory and a merge script. A set is a directory holding an `EpisodeStore`, the instances played, and a manifest recording what produced them (model, scenario, config fingerprint, every accepted key, each invocation, artifact links).

```
from interlens.arena import rollout
from interlens.arena.negotiation.sheets import GameSpec
from interlens.arena.scenarios.scorable import ScorableNegotiation
from interlens.arena.table import rational_table

def seats(instance, arm, seed):          # a factory: policy seats hold
     per-episode mutable state
    game = GameSpec.from_json(instance.payload)
    return rational_table(game, ["boulware", "conceder", "tough"],
     deadline=game.rounds)

cfg = {"model": "policy:zoo", "scaffold": "canonical", "info": "full"}
rs = rollout(scenario=ScorableNegotiation(), instances=instances,
     participant=seats,
             out="runs/pilot", seeds=[0, 1], arms=["moves_chat"],
     config=cfg)
print(rs.summary_text())
```

```
RolloutSet runs/pilot
  model=policy:zoo scenario=scorable_negotiation cfg=8f2c1a09b4de
  episodes: 6 (6 distinct keys) statuses={'done': 6}
  arms: {'moves_chat': 6}
  success: 4/6 = 0.667
  mean primary: 0.7143 over 6 scored episodes
  usage: 0 in / 0 out tokens, $0.00
  fabricated turns: 0 (in 0 episodes)
  placeholder turns: 0/94 = 0.0% (engine 0.0%, burned cap 0.0%) | at cap
     0.0% | no-op 19.5%
```

Re-run with `seeds=[0, 1, 2, 3]` and only the two new seeds cost anything: resume is unconditional and keyed on `(instance_id, seed, arm)`, read from the episodes on disk rather than the manifest, so a crash between the two writes still resumes correctly. Reuse the set's own bank with `rs.load_instances()` — instance ids are uuid-based, so regenerating "the same" instances mints new ids and matches nothing.

Appending never silently mixes protocols. `config` is the set's protocol identity, hashed into a fingerprint; an append whose fingerprint differs raises `RolloutConfigMismatch` *before* any episode runs, naming the differing keys. `allow_mismatch=True` proceeds but is an audit trail, not a silencer: each accepted episode is stamped in its `cell_cfg` and the divergence is recorded in the manifest.

Put in `config` anything that changes what an episode *means* (model, scaffold, information condition, arms, bank identity) and nothing that merely changes how it was scheduled (concurrency, output path), or a wider resume will refuse itself.

`summary()` is deliberately a **health check**, not a results table — counts, statuses, success rate, mean `outcome["primary"]`, spend, and two contamination gates. Metrics defined by your protocol belong in your own analyzers.

Two gates, because each is blind to the other's failure

A set can be 100% `done`, 100% `parse_ok`, 0% empty content, `fabrication 0.000` — and still be a quarter silent. Two different failures wear the same `EMPTY_TURN_PLACEHOLDER` string:

cause	signature	remedy
the **engine** fabricated the turn (generation failed outright)	`gen_failed`	fix the generation failure; a larger cap cannot help
the **model** burned its whole per-turn budget inside an unterminated `<think>` and emitted no action	`empty_gen`	raise the per-turn cap, or disable thinking

The second is the one nothing else catches. The engine did its job, so `fabrication` correctly reads 0.000; the placeholder it substitutes is non-empty and parses into a well-formed no-op, so `parse_ok` reads 1.000 too. Measured at **24.4% of turns** on a thinking-on Qwen3-32B cell at a 2,048-token cap against 0.0% with thinking off — and on that cell `parse_ok` was *anti*-correlated with quality (1.000 on the silent arm vs 0.958 on the healthy one, because placeholders parse flawlessly and real prose sometimes does not).

So `summary()` reports a **placeholder budget** beside the fabrication audit, split by cause and broken down by round:

```
fabricated turns: 0 (in 0 episodes)
placeholder turns: 3/6 = 50.0% (engine 0.0%, burned cap 50.0%) | at cap
   50.0% | no-op 50.0%
!! PLACEHOLDER BUDGET EXCEEDED: 50.0% of turns said nothing (budget
   2.0%), in 1 episodes.
   worst round 3 at 100.0%; by round: {1: '0.0%', 2: '50.0%', 3:
   '100.0%'}
   parse_ok and the fabrication gate CANNOT see this. If 'burned cap'
   dominates, the model spent its
   budget inside <think>: raise the per-turn cap or disable thinking —
   and note that changing the cap
   changes the protocol, so the new cells do not pair with the old ones.
```

The by-round breakdown is load-bearing: budget exhaustion is **late-episode by construction**, since context grows with the transcript (5.8% at round 1 rising to ~35% by rounds 2–4). A short smoke run, or a peek at an

episode's opening turns, reads ~0% and under-detects it — so a late round over budget is called out even when the pooled rate is clean.

Exceeding the budget flags the set and stamps `manifest["turn_health"]`; it never raises, because the episodes are real evidence and the right response is to record the rate. Tune it with `summary(placeholder_budget=...)`; the per-turn primitive is `interlens.arena.turn_signatures(turn)`, which returns the signature set for one stored turn.

Replay and re-scoring

Scenarios are pure state machines, so a stored episode replays exactly:

```
from interlens.arena import replay_episode, rescore
outcome = replay_episode(scenario, instance, stored_episode_json)   #
     recompute with the current scorer
result = rescore(scenario, instance, stored_episode_json)           #
     compare vs the recorded outcome
```

Formal actions and per-turn oracles

For scenarios that need *binding moves* rather than free text, `interlens.arena` ships a typed formal-action layer and a per-turn oracle layer.

Formal actions (`interlens.arena.actions`): the package-deal move vocabulary — `Propose(deal)`, `Accept(offer_id)`, `Reject(offer_id)`, `Walk()` — plus an `OfferRegistry` (monotonic ids, standing-offer tracking, accept/reject votes) and one consolidated `parse_action`. `parse_action` distinguishes a **syntax** failure (no well-formed action could be read) from an **economic-legality** failure (well-formed but references a dead offer / an infeasible deal), so a scenario can retry once with specific feedback and log the two classes separately:

```
from interlens.arena import parse_action, OfferRegistry, Propose, Accept,
     SYNTAX, LEGALITY

registry = OfferRegistry()
oid = registry.register(deal=(0, 2, 1, 0, 1), proposer="Avery", round=1)
     # → "O1" (Avery auto-accepts)

result = parse_action(text, deal_decoder=my_issue_object_to_deal,
     standing=registry.standing_ids())
```

```
if result.ok:
    registry.apply(result.action, agent="Blake", round=2)   # Accept("O1")
     records Blake's vote
else:
    directive = result.retry_directive()   # {"retry": <specific message>,
     "error_kind": "syntax"|"legality"}
```

`deal_decoder` maps a model's `{"Site": "...", ...}` object to a `Deal = tuple[int, ...]` (option index per issue), returning `None` for an infeasible deal — keeping the action layer free of any specific game's issue set.

Oracles (`interlens.arena.oracles`): an `Oracle` scores every action available to a seat and names the best; the arena records the seat's *regret* — `value(best) - value(chosen)`, the centipawn-loss analog. Two annotation paths write typed `OracleRecord`s into the episode's oracle log (`Episode.round_checkpoints`):

- **inline** pure-Python oracles run post-`apply` with no extra generation — override `Scenario.annotate_turn` and return `interlens.arena.oracles.annotate(oracles, game, history, agent, legal, chosen_action=..., round=..., seat=..., turn_idx=...)`;
- **forked** provisional elicitations re-ask the *model* to finalize now on a private view — the existing `Scenario.provisional_due` path, unchanged.

```
from interlens.arena import Oracle, OracleVerdict
class SolutionOracle(Oracle):
    name = "solution"
    def evaluate(self, game, history, agent, legal):
        values = {a: surplus_if(game, a, agent) for a in legal}
        best = max(values, key=values.get)
        return OracleVerdict(action_values=values, best=best, flags=[...])
```

Each inline record carries `divergence` (per-turn regret), `chosen_value`/`best_value`, the full `verdict`, and any hard-violation `flags`; provisional records keep the `{round, seat, provisional_action, score, content}` shape, so older datasets still parse.

Communication styles

Async messaging is the package default (`Scenario.default_communication = "messaging"`, used by the Inspect tasks when no mode is given): no shared transcript; autonomous agents exchange point-to-point mail via `send_message`/`read_message` (fenced-JSON actions for any participant type, or native tools via `policy.tools_for(name)`), with ping-driven scheduling (priority + a fairness tick). Sends/reads/deliveries are first-class transcript events. See `examples/arena_relay_messaging.py` for the relay task run in this mode.

Two other styles remain available as explicit configs and compose with the same scoring and persistence (see `interlens.communication`):

- **Round-robin protocol** (`communication="round_robin"`, or `EpisodePool.run_episode` directly): each scenario's published turn protocol over a shared transcript. **This is the comparability option**: the shipped v0 transcript dataset was produced under this protocol (recorded per episode in `gen_config`), so use it when comparing new runs against those cells — do not assume messaging-mode and protocol-mode scores are equivalent. The security dilemma pins this mode (`default_communication = "round_robin"`): a simultaneous-move payoff game has no sound free-messaging reduction.
- **Direct piping** (`DirectPipingPolicy`): one participant's output is the next one's input along a fixed chain — the natural two-agent dialogue framing, generalized. See `examples/arena_direct_piping.py`.

Inspect integration (optional)

With the extra installed (`pip install interlens[inspect]`), both scenarios run under Inspect:

```
inspect eval interlens.arena.inspect/info_relay --model anthropic/claude-
     sonnet-5 -T level=2   # messaging (default)
inspect eval interlens.arena.inspect/negotiation --model openai/gpt-5 -T
     n_parties=8 -T arm=solo
inspect
     eval interlens.arena.inspect/info_relay -T communication=round_robin
```

```
     --model anthropic/claude-sonnet-5   # published protocol, comparable
     with the v0 dataset
inspect eval interlens.arena.inspect/security_dilemma --model anthropic/
     claude-sonnet-5 -T level=2   # always round-robin
inspect eval interlens.arena.inspect/coding_collab --model openai/gpt-5 -
     T level=3
inspect eval interlens.arena.inspect/distributed_longcontext -T
     instances=banks/dlc_sniah_L0.json --model anthropic/claude-sonnet-5
```

Messaging-mode (default) notes for the new families: `coding_collab` scores the latest complete ```` ```python ```` fence in the agents' mail as the submission (the same working-draft rule as the protocol); `distributed_longcontext` maps messaging to the scenario's NATIVE directed-messaging arm (`team-msg`), so those episodes stay replayable; `security_dilemma` rejects messaging mode — a simultaneous-move payoff game has no sound free-messaging reduction — and pins `round_robin` via its `default_communication`.

Instance banks become samples (instance JSON + per-seat framings in sample metadata), the arena engine runs inside a fully-async solver (Inspect's `--max-samples` runs many episodes concurrently), the exact scenario scorers register as Inspect scorers (`success` accuracy + mean `primary`), and each seat turn is mirrored into the sample's message list so `inspect view` renders the multi-agent flow with seat attribution, structured actions as transcript events, and the outcome in the score view. Inspect's native `token_limit` carries episode budgets; the adapter adds per-sample dollar cost (`metadata["cost_usd"]`) computed from the same pricing as `UsageMeter`. Without the extra, `import interlens.arena.inspect` fails with a clear install hint — the base package never requires inspect-ai.

Next: back to the index.

11 · Scorable negotiation & rational oracles

`interlens.arena.negotiation` is a **multi-party, multi-issue scorable negotiation** stack built for measuring how far an agent (LLM or rule-based) is from *game-theoretically optimal* play — not just whether it closed a deal. It ships a solver-verified game generator, exact normative benchmarks (Pareto frontier, Nash / Kalai–Smorodinsky / egalitarian / utilitarian / max-Nash-welfare points), a swappable **rational-oracle** layer that scores every move, the `ScorableNegotiation` scenario (structured binding offers separated from cheap talk), and a pool of pure-Python rational agents. Every generated instance carries its exact analysis, so per-turn regret and outcome-distance metrics are *exact*, not sampled.

This page is the negotiation-specific companion to 10 · Arena: the game lives in `arena/negotiation/`, the oracle types in `arena/oracles.py`, the scenario in `arena/scenarios/scorable.py`, and the rational agents in `PolicyParticipant`.

The game: issues, score sheets, `GameSpec`

A deal picks one option per issue and is a `Deal = tuple[int, ...]` of option indices. Each party holds a private additive `ScoreSheet` (a value per option, plus a threshold `τ` — its BATNA); the analysis object is always the **surplus** `x_i(d) = u_i(d) - τ_i`.

```
from interlens.arena.negotiation.space import Issue, DealSpace
from interlens.arena.negotiation.sheets import ScoreSheet, GameSpec

space = DealSpace((Issue("Site", ("North", "South")),
                   Issue("Fund", ("None", "1M", "5M"))))
alice = ScoreSheet("Alice", ((10, 0), (0, 3, 6)), threshold=5.0)
bob   = ScoreSheet("Bob",   ((0, 10), (6, 3, 0)), threshold=5.0)

alice.utility((1, 2))          # 0 + 6  = 6.0   (South, 5M)
alice.surplus((1, 2))          # 6 - 5  = 1.0

spec = GameSpec(space=space, sheets=(alice, bob),
                rounds=4, info="full", chat=True,        # protocol arms
     (DESIGN §3)
                proposer=0, veto=1, min_accept=None,     # agreement rule
     (None = unanimity)
```

```
                discount=1.0, breakdown_risk=0.0)        # impatience
    knobs the oracles read
spec.to_json()                  # round-trips via GameSpec.from_json —
    drops into Instance.payload
```

`GameSpec.surplus_matrix()` is the dense `|D| × n` surplus array every solution concept consumes; `feasible_mask()` is the boolean mask of deals that pass the agreement rule (proposer + veto clear, and ≥ `min_accept` parties clear their threshold).

Swapping the game: presets

The situation is a **swappable axis**, like the oracle stack — "two agents play the ultimatum game" is one call. `arena/negotiation/games.py` is a registry mapping a name to a literature-grounded **preset**: a factory returning `(GameSpec, analysis, protocol_cfg)`, the game plus its exact analysis plus the scenario knobs that turn the one `ScorableNegotiation` engine into that game. Presets are *parameterizations, not new engines* — the same deal space, oracle stack, annotation, and atlas run unchanged on any of them.

```
from interlens.arena.negotiation import games
games.PRESETS                                            # {'scorable',
    'ultimatum', 'divide_dollar', 'bilateral_multiissue'}
game, analysis, protocol_cfg = games.make_preset("ultimatum", pie=10,
    n_options=11)
game.n_parties, analysis["deal_space_size"]              # 2, 11  (one split
    per option)
protocol_cfg                                             # {'single_shot':
    True, 'fixed_proposer': True}

# the arena bridge: a solver-verified Instance + the cfg to play it under
    (reuses generate.build_instance)
instance, protocol_cfg = games.build_preset_instance("divide_dollar",
    n_parties=3, rule="majority")
```

Preset	Game	Rational anchor
`scorable` (default)	the multi-party, multi-issue repaired game (§2 generator; a thin	the §4 oracle stack

Preset	Game	Rational anchor
	alias, defaults unchanged)	
`ultimatum`	take-it-or-leave-it N=2 pie split (J=1, τ=0, `rounds=1`, single-shot responder-only vote)	SPE: proposer keeps the pie, responder accepts any positive share (Güth et al. 1982 human-rejection contrast; Rubinstein 1982 §5)
`divide_dollar`	N-party discrete-shares split, rotating proposer, multi-round, unanimity/majority	Baron–Ferejohn 1989 / Okada 1996 (`v_i = 1/n` — the equilibrium oracle's own anchor)
`bilateral_multiissue`	DoND-style N=2, J-issue, private values (the generator at `n_parties=2`)	Lewis et al. 2017 lineage

`protocol_cfg` is passed as the scenario `cfg` (or merged into it) so the shared state machine runs the preset's protocol variant. `single_shot` skips the round-robin and goes straight to the propose→vote forced final; `fixed_proposer` pins the opener seat (no rotation) — both default off, so the standard multi-round game is unchanged. Majority vs unanimity is just `GameSpec.min_accept`. The experiment runner exposes all four as `run.py --game {scorable,ultimatum,divide_dollar,bilateral_multiissue}` (+ `--game-arg KEY=VALUE` preset knobs).

Exact solutions & descriptors

`solutions.analyze(space, sheets)` returns the full, JSON-serializable analysis every generated instance ships with — the normative benchmarks and

the score-sheet descriptors that tell you whether the game is actually contestable.

```
from interlens.arena.negotiation import solutions

an = solutions.analyze(space, (alice, bob))
an["deal_space_size"]                  # 6
an["ir_count"], an["pareto_count"]     # |IR set|, |Pareto frontier|
an["ir_pareto_fraction"]               # |IR ∩ Pareto| / |IR| — how near-
     zero-sum the acceptable set is
an["dominated_acceptable_fraction"]    # 1 - that: the score-sheet-repair
     target (leave dominated deals in play)
an["sparsity"], an["pairwise_iou"]     # TMLR descriptors: zero-option
     sparsity, pairwise score overlap
an["ideal_surplus"]                    # per-party best feasible surplus
     b_i (the distance normalizer)
an["solutions"]["nash"]                # {"deal", "named", "surpluses",
     "utilities", "ties", "scale_invariant", ...}
```

The five solution concepts are `nash`, `kalai_smorodinsky`, `egalitarian`, `utilitarian`, `max_nash_welfare`; `all_solutions(space, sheets)` returns them as `SolutionPoint`s (each carries `.deal`, `.surpluses`, `.index`, `.ties`, `.scale_invariant`). Use **nash / kalai_smorodinsky** as the scale-invariant anchors — egalitarian and utilitarian are flagged `scale_invariant=False` (meaningful only on normalized surpluses).

Per-move divergence is a distance in scale-invariant normalized-surplus space:

```
U, tau = spec.utility_matrix(), spec.thresholds
idx = space.index_of((1, 2))
solutions.distance_to_frontier(U, tau, idx)                # 0.0 iff the
     deal is Pareto-efficient
ks = solutions.all_solutions(space, (alice, bob))["kalai_smorodinsky"]
solutions.distance_to_solution(U, tau, idx, ks.index)      # 0.0 iff the
     deal equals the KS point
```

Rational oracles

An `Oracle` scores every action a seat could take at a decision point and names the best one; the seat's **regret** is `value(best) - value(chosen)` in the game's value units — the centipawn-loss analog for negotiation.

```
from interlens.arena import Oracle, OracleVerdict, OracleRecord, Propose,
    Accept
verdict = OracleVerdict(
    action_values={Accept("O1"): 6.0, Propose((1, 2)): 8.0},   # keyed by
     the typed action
    best=Propose((1, 2)),
    beliefs=None,                                               # a
     belief oracle fills this with its posterior
    flags=[])                                                   # named
     hard-violation markers, e.g. "ir_violation"
verdict.best_value()             # 8.0
verdict.divergence(Accept("O1")) # 8.0 - 6.0 = 2.0  (regret of the chosen
     action, ≥ 0)
verdict.extra                    # free-form per-verdict diagnostics:
     best-response surplus_loss, reservation, v*, …
```

The concrete negotiation oracles live in `arena/negotiation/` (`beliefs.py` exact Bayes over an enumerated type grid, `acceptance.py` optimal-stopping accept/reject, `bestresponse.py` exact expectimax, `equilibrium.py` a Banks–Duggan stationary equilibrium) and subclass this generic ABC. An **inline annotation** scores a seat's actual move against the oracle's best on the same state and is persisted as an `OracleRecord`:

```
rec = OracleRecord.annotation(verdict, round=2, seat="Alice",
     oracle="bestresponse",
                              chosen_action=Accept("O1"), turn_idx=5)
rec.divergence               # 2.0  (best_value - chosen_value) — the per-
     turn regret
rec.to_json()                # lands in Episode.round_checkpoints
     (identified by a "verdict" key)
```

Playing it: `ScorableNegotiation`

`ScorableNegotiation` (arena/scenarios/scorable.py) plays the game through any participant with **harness-enforced channel separation**: each turn is a private scratchpad, an optional public `message` (cheap talk), and one binding formal move — `Propose` a complete deal (the registry stamps it `P1`, `P2`, …), `Accept` / `Reject` a live offer by id, or `Walk`. Offers are addressed by id so "I accept" is never ambiguous with two proposals on the table.

```
import asyncio
from interlens import APIParticipant, UsageMeter
from interlens.arena import EpisodePool, EpisodeStore
from interlens.arena.scenarios import ScorableNegotiation

scenario = ScorableNegotiation()
instance = scenario.generate_instance(level=0, seed=1)   # payload carries
     the GameSpec, solution = the analyze() dict
meter = UsageMeter(budget=25.0)
player = APIParticipant(name="player", model_id="claude-sonnet-5",
     meter=meter, turn_token_floor=2048)

pool = EpisodePool(EpisodeStore("episodes/"), meter=meter)
episode = asyncio.run(pool.run_episode(scenario, instance,
     arm="moves_chat", participant=player))
episode.outcome        # {"success", "deal": bool, "deal_named": {...},
     "usw"/"esw"/"nsw"/"gini",
                       #  "per_party_surplus", "ir_violations",
     "ceiling_surplus", "offers", ...}
```

Arms: `moves_chat` (binding moves + cheap talk), `moves_only` (the ±communication treatment), `team`, and `solo` (the communication-free control that must not match the multi-agent rate). Situational knobs go through `cfg` (rounds, personas, history window, `show_own_scores`) exactly as in 10 · Arena. Turns persist as `TurnRecord`s with `parsed_action` (`atype` / `deal_named` / `offer`); per-turn oracle annotations persist as `OracleRecord.to_json()` dicts in `Episode.round_checkpoints`.

Rational agents: `PolicyParticipant` + the strategy zoo

A `PolicyParticipant` mounts a pure-Python `policy(state) → Action` as a first-class participant, so a rational agent plays the *same* protocol as an LLM (and can sit across the table from one). The strategy zoo in `arena/negotiation/strategies.py` provides the executable opponents: time-dependent Faratin curves (`TimeDependentPolicy.boulware(...)` concedes near the deadline, `.conceder(...)` concedes early), `MiCROPolicy`, `NaiveTitForTatPolicy`, `ToughPolicy`, and a `BayesianRationalPolicy`, plus AC-next/AC-combi acceptance conditions.

```
from interlens.arena.negotiation.policy_participant
     import PolicyParticipant
from interlens.arena.negotiation.strategies import TimeDependentPolicy
```

```
alice_agent = PolicyParticipant(
    name="Alice", policy=TimeDependentPolicy.boulware(), seat=0,
    sheet=alice, space=space, deadline=spec.rounds,
     discount=spec.discount)
# hand alice_agent (and a policy or LLM for bob) to
     EpisodePool.run_episode as the seat participants
```

A rational-vs-rational rollout should land near the frontier in `info="full"` and show calibrated inefficiency in `info="private"` (Myerson–Satterthwaite) — the sanity check before putting LLMs in the seats.

Measuring divergence

The `experiments/rational_agents/` measurement layer reads stored `Episode` + `Instance` and computes the **divergence atlas**: outcome quality (Pareto / NBS / KS distance via the `solutions` functions above, welfare, Gini), per-turn oracle regret (from the `round_checkpoints` `OracleRecord` rows) with the Park et al. no-regret trend tests, concession-curve τ/CRI, and a 12-row failure taxonomy tiered from fully-mechanical (IR violation) to LLM-judge (tactic-style). `annotate.py` writes the per-turn annotations; `analysis/report.py` aggregates a run into per-model per-arm tables + plots. See that experiment's README for the pipeline.

Transcripts & Inspect

Every turn's exact rendered prompt is persisted: the engine records each `SeatRequest.view` into `TurnRecord.view` by default (`EpisodePool(record_views=False)` to disable), so a transcript is faithful even if prompt code later drifts. `arena/export.py` turns a stored run into human-readable transcripts — `export.export_run(episodes_dir, instances_dir, out_dir)` (or `python -m interlens.arena.export --episodes … --instances … --out …`) writes a markdown + self-contained HTML page per episode (game setup, per-turn scratchpad/message/action/oracle-regret, outcome + welfare) plus a run index; it renders old episodes (no `view`) too.

And the scenario is registered as an Inspect task via the preset registry, so `inspect eval` / `inspect view` work out of the box:

```
inspect eval interlens.arena.inspect/scorable --model anthropic/claude-
    sonnet-5 -T game=ultimatum -T arm=moves_only
```

12 · Control Tower: use an Interlens local model as an untrusted policy

Interlens and Control Tower meet at the policy boundary. Interlens owns local Hugging Face generation and interpretability; Control Tower owns LinuxArena environments, blue-team protocols, tool execution, monitoring, auditing, sandboxing, and trajectory storage.

The adapter returns tool calls to Control Tower **without executing them inside Interlens**. This is the important security boundary: a trusted-monitoring protocol sees the same proposed action it would see from any other untrusted model.

Install

Control Tower requires Python 3.13. Install both packages and Interlens's Inspect integration:

```
uv add control-tower "interlens[inspect]"
```

Define the participant

Put a zero-argument factory in your project, for example `my_project/policies.py`:

```
from interlens import AutoModelParticipant

def local_qwen():
    """Build one fresh participant for one Control Tower sample."""
    return AutoModelParticipant.from_pretrained(
        "Qwen/Qwen2.5-Coder-7B-Instruct",
        name="untrusted",
        device="cuda",
        load_kwargs={
            "dtype": "bfloat16",
            "attn": "flash_attention_2",
            "quant": None,
            "revision": None,
        },
        temperature=0.2,
```

```
        top_p=0.95,
        max_new_tokens=2048,
        thinking=False,
        kv_reuse="auto",
    )
```

A factory is used instead of a global participant so every Control Tower sample gets fresh KV-cache and generation state. Interlens's process-wide model cache still shares the loaded weights. The adapter serializes concurrent calls that share one cached model because Hugging Face generation is not thread-safe on a single model object; Control Tower can still overlap environment work and calls backed by different models.

Run a Control Tower protocol

Pass the adapter as an external untrusted policy and the factory as its argument:

```
uv run ct run eval \
  --untrusted-policy
     interlens.integrations.control_tower:interlens_untrusted_policy \
  --untrusted-policy-arg
     participant_factory=my_project.policies:local_qwen \
  --protocol trusted-monitoring \
  --protocol-arg sus_threshold=7 \
  --task-file simple
```

For each untrusted-model request:

1. Inspect messages and available LinuxArena tools are translated to an Interlens model view.
2. The participant performs one local generation with its native chat template.
3. Parsed tool calls become ordinary Inspect `ToolCall`s.
4. Control Tower decides whether to monitor, audit, defer, replace, or execute them.
5. Tool results return through Inspect on the following model call.

`tool_choice="auto"` and `"none"` are supported. Required or forced tool choices fail loudly because Hugging Face chat templates can render tool schemas but cannot reliably enforce a call.

Programmatic use and interpretability

Use `participant_generate` directly when constructing a custom Control Tower policy. Steering, capture, patching, and log probabilities can be attached once to the adapter:

```
from interlens.integrations.control_tower import participant_generate

generate = participant_generate(
    local_qwen(),
    steering=my_steering_spec,
    capture=my_capture_request,
    patch=None,
    return_logprobs=False,
)
```

The returned function has Control Tower's `UntrustedPolicyGenerateFn` signature:

```
output = await generate(messages, tools, tool_choice, generate_config)
```

Attack and collaborative policies

The same boundary also exposes an attack-policy wrapper and a private planner-reviewer-executor team. The attack wrapper reads Control Tower's sample-local side-task goal and uses it only as the Interlens policy's private system prompt; blue protocols retain their neutral monitor-visible history:

```
uv run ct run eval \
  --untrusted-policy
     interlens.integrations.control_tower:interlens_attack_policy \
  --untrusted-policy-arg
     participant_factory=my_project.policies:local_qwen \
  --protocol trusted-monitoring \
  --protocol-arg sus_threshold=7 \
  --task-file medium
```

For collaborative inference, one local participant sequentially plays planner, reviewer, and executor on every outer action. Planner and reviewer see a text capability summary but receive no callable tool schemas; only the executor can propose an external action:

```
uv run ct run eval \
  --untrusted-policy
     interlens.integrations.control_tower:interlens_team_policy \
  --untrusted-policy-arg
     participant_factory=my_project.policies:local_qwen \
  --untrusted-policy-arg mode=honest \
  --untrusted-policy-arg planner_max_tokens=512 \
  --untrusted-policy-arg reviewer_max_tokens=512 \
  --protocol untrusted-agent \
  --protocol-arg max_steps=80 \
  --task-file one_per_env
```

Set `mode=attack` only on a task file containing side tasks. Private planning/review text is recorded under `ModelOutput.metadata["interlens"]["collaboration"]`; all three calls contribute to the reported input tokens, output tokens, and cost. The executor's message and tool calls remain ordinary Inspect objects, so monitoring, deferral, editing, sandbox execution, and scoring continue to be Control Tower's responsibility.

This adapter is intentionally for local `ModelParticipant`-style generators. Hosted Interlens API participants already overlap with Inspect's provider layer and do not expose local activation interventions.

API Reference

Every public class and function in `interlens`, one page each, generated from the source. Start from a module below, or jump straight to a symbol — pages carry their full signature, parameters, and a link to the exact lines that define them.

- `interlens` — The public `interlens` API, exported **lazily**.
  - `arena` — Scoreable multi-agent evaluations on interlens: scenarios, episode drivers, and exact scoring.
    - `actions` — Typed formal-action layer for structured negotiation turns.
    - `auction` — Repeated multi-bidder auctions: the frozen spec, the persona-conditioned prior, exact allocation and payment rules, equilibrium benchmarks, computable bidders and oracles, and the collusion metrics.
      - `actions` — The auction move vocabulary, the bid ledger, and DM routing (design.md §12 item 2).
      - `allocation` — Bundle values, the exact efficient allocation, and the payment rules.
      - `benchmarks` — The exact per-stage equilibrium benchmarks every suppression metric divides against (design.md §4.3, §5).
      - `bidders` — The computable bidder zoo: policies (`state` → `action`), their DM decision rules, and their oracles.
      - `metrics` — Stage-level and repeated-play metrics (design.md §5), as pure functions over records.
      - `policy_text` — What the computable seats SAY – the templated broadcast and DM behavior of design.md §3.4.
      - `priors` — The persona-conditioned prior: the generative model, the persona table,

fact rendering data, and the posterior a rational seat actually computes.
    - `references` — Citation-key registry for the auction mechanism, benchmark, and collusion-metric modules.
    - `spec` — The frozen auction specification: item slots, bidders, per-stage draws, and the mechanism config.
- `engine` — Episode drivers: play `Scenario` instances through `Participant`s.
- `export` — Human-readable transcripts from stored episodes: an `EpisodeStore` tree -> one markdown + one self-contained HTML page per episode, plus a per-run index.
- `gates` — Template-fidelity gates: preflight checks before spending GPU-hours on local-model rollouts.
- `inspect` — Optional Inspect (inspect-ai) integration: run arena scenarios under `inspect eval`.
    - `adapter` — The Inspect adapter core: a Participant backed by Inspect's model, the arena solver, and the scorer.
    - `tasks` — The bundled scenarios as Inspect tasks.
- `live` — Live play: watch an arena episode as it happens, reconfigure its seats, and play one yourself.
    - `assets` — The live pages' browser layer, as Python strings — same convention as `viz.assets`.
        - `js_live` — The live page's browser layer: subscribe, merge, redraw, and take the player's move.
        - `js_lobby` — The lobby's browser layer: edit the seat lineup, then start the game.

    - `events` — The live-play wire protocol: every server-sent event a session can emit, in one place.
    - `human` — `HumanParticipant`: a seat played by a person in a browser.
    - `lobby_page` — The lobby: choose a game and decide who plays each seat.
    - `payload` — Per-turn slices of the visualizer payload, for streaming one turn at a time.
    - `play_page` — The live episode page: the visualizer's episode view, plus the controls to play in it.
    - `provider` — The seam between the live server and whatever experiment supplies its games.
    - `router` — `LiveSeatRouter`: a seat table whose occupants can change while the episode is running.
    - `server` — The HTTP surface: a stdlib `ThreadingHTTPServer` serving the lobby, the live page, and the event stream.
    - `session` — `LiveSession`: one live game — the engine thread, the event log, and everything the browser can do to it.
    - `style` — The rules the lobby and the play page both need: form controls, seat cards, and the two docks.
- `negotiation` — Multi-issue, multi-party scorable negotiation: deal spaces, private score sheets, exact solution concepts, computable rational-agent oracles, and an executable strategy zoo.
    - `acceptance` — Optimal-stopping acceptance oracle: when is accepting the standing offer better than holding out?

- `analysis` — Measurement over stored negotiation episodes: how far from rational was this play, and where?
  - `annotations` — Per-turn annotation records: the divergence data model `annotate.py` writes and `taxonomy.py` / `report.py` read (disk I/O lives in `runio.AnnotationStore`).
  - `cot_localize` — OmegaPRM-style within-CoT divergence localization: binary-search the first reasoning step that flips the induced action to a divergent one, in O(log n) oracle calls (arXiv:2406.06592 — exploit prefix monotonicity: correct until the first error, wrong after).
  - `curves` — Trajectory-shape metrics over a *series* (not a single turn).
  - `episode_view` — `EpisodeView`: a stored arena `Episode` JSON parsed into a normalized negotiation action series that the metrics read instead of raw `parsed_action` — an ordered `TurnView` list (typed action, deal canonicalized to an index tuple, offer id, any acceptable-offer/belief note, private thinking), the offer registry, the per-round standing offer, and the final deal / reached flag.
  - `game_analysis` — `GameAnalysis`: the solved-game bundle the metrics read — the adapter seam between

interlens' `GameSpec`/`solutions.py` and the pure metric math.
    - `metrics` — The divergence metric suite: outcome-, turn-, and faithfulness-level measures over one episode.
    - `rollout` — Counterfactual-rollout regret: label a divergence by Δ expected surplus, not by action mismatch.
    - `surplus` — Pure surplus-vector math: Pareto geometry, dominance, and distances.
    - `taxonomy` — The 12-row LLM-negotiation failure taxonomy as executable checks.
- `belief_accuracy` — How well does a :class:`~interlens.arena.negotiatio` actually know its opponent?
- `beliefs` — Bayesian (and frequency-model fallback) belief oracle over an enumerated opponent-type grid.
- `bestresponse` — Exact expectimax best-response oracle over (remaining rounds x deal space x type posterior).
- `calibrated` — Behaviourally-calibrated rational negotiation: :class:`CalibratedRationalPolicy`.
- `equilibrium` — Banks-Duggan stationary-equilibrium oracle for the multilateral unanimity bargaining game.
- `fairness` — The **table objective**: one number per deal saying how good that deal is *for the whole table*.

- `games` — Swappable game presets: name a classic bargaining situation, get a ready-to-play game in one call.
- `generate` — Scorable-negotiation scenario generator with the score-sheet repairs the reproducibility studies demand.
- `llm_calibrated` — Private-information rational negotiation against an **empirically fitted LLM opponent model**: :class:`LLMCalibratedRationalPolicy`.
- `oracle_context` — The per-decision-point context the negotiation oracles share, written once.
- `policy_participant` — `PolicyParticipant`: a state-dependent pure-Python seat that computes its move from a bound policy.
- `references` — Citation-key registry for the negotiation solution-concept and generator modules.
- `rewards` — Outcome rewards for RL on scorable negotiation: the smoothed log-Nash objective.
- `sheets` — Private score sheets, the additive utility model, the game specification, and the NumPy utility matrix.
- `solutions` — Exact axiomatic solution concepts over the fully-enumerated deal space.
- `space` — The deal space: issues, their discrete options, and the fully-enumerable Cartesian product of options.
- `strategies` — The executable rational / scripted negotiator zoo as **policies** (`state → action`), the computable opponent pool the LLMs are measured against.

        - `talking` — The **talking rational agent**: the composed Bayesian negotiator with a truthful templated voice.
- `oracles` — The oracle layer: per-turn "what would a rational agent have done here?" annotations.
- `ratchet` — Adaptive difficulty ratchet: find the level where a model stops clearing the bar, then measure there.
- `refusal` — Recovering a turn the API refused, without changing what the turn says.
- `replay` — Deterministic replay of stored episodes through a scenario's state machine.
- `rollouts` — Directory-backed **rollout sets**: run more episodes into the same place, safely, and resume.
- `scenario` — The `Scenario` interface: a pure game-logic state machine, participant-agnostic.
- `scenarios` — Bundled scenarios, four families:
    - `auction` — Repeated multi-bidder auctions as one :class:`~interlens.arena.scenario.Scenario`.
    - `auction_examples` — The three worked turn views of the auction scaffold, generated from real frozen draws.
    - `auction_policy` — Computable seats inside the ordinary engine loop.
    - `auction_prompts` — The frozen prompt scaffold for :class:`~interlens.arena.scenarios.auction.AuctionScenario`.
    - `coding` — Coding collaboration with private constraints: 3 seats jointly write ONE Python module.
    - `dlc` — Task adapters for the distributed long-context scenario, ported from the RLM paper's benchmarks.
        - `bcp` — BrowseComp-Plus: multi-hop QA over a fixed document corpus (paper §3.1).

    - `build` — Instance builders for the distributed long-context tasks — fetch, shard, and save instance banks.
    - `codeqa` — LongBench-v2 CodeQA: repo-understanding multiple choice (paper §3.1).
    - `oolong_pairs` — OOLONG-Pairs: the RLM paper's pairwise-aggregation task (Appendix 12.1).
    - `sniah` — S-NIAH: RULER-style single needle-in-a-haystack (paper §3.1).
- `longcontext` — Distributed long-context: one long-context task split across 4 communicating seats.
- `negotiation` — Negotiation: a multi-issue, multi-party deal with secret score sheets (structured-JSON actions).
- `priors` — Role-prior sign table for the negotiation scenario (role × issue) and sheet-vs-prior analysis helpers.
- `relay` — Info relay: an epistemic team task with a confidently-wrong agent.
- `scorable` — ScorableNegotiation: the repaired multi-party, multi-issue scorable game — the *protocol* around a :class:`~interlens.arena.negotiation.sl (carried in `Instance.payload`), built on the shared typed-action (:mod:`interlens.arena.actions`) and oracle (:mod:`interlens.arena.oracles`) layers. "Repaired" names the specific benchmark flaws it fixes, each called out at the code that fixes it: votes-not-arithmetic closure, structural channel separation, a restated

deadline, and measured-not-blocked economic illegality.
    - `scorable_prompts` — The canonical prompt scaffold for the scorable-negotiation scenario.
    - `security` — Security dilemma: a repeated 2-party build/deescalate/attack game with noisy intelligence.
- `schema` — The arena's record schema: one JSON shape for every episode.
- `table` — Heterogeneous **tables**: present a whole many-seat lineup to the arena engine as one participant.
- `views` — Per-seat view construction + structured-action parsing for arena scenarios.
- `viz` — Interactive episode visualization: any arena run directory in, self-contained interactive HTML out.
    - `advice` — The advised seat, audited: what its planner knew, what it recommended, and whether the seat did it.
    - `assets` — The inline stylesheet and browser layer — no external assets of any kind.
        - `css` — The one stylesheet every page wears — a small design system, inlined.
        - `js_auction` — The auction episode page's wiring: the DM stage scrubber, the hover card on the bid ladder, and the cross-links between every mark and the turn it belongs to.
        - `js_chart` — Browser layer, part 2: the two charts.
        - `js_compare` — The comparison page's wiring: one shared frontier

carrying both trajectories, and two synchronized columns.
    - `js_core` — Browser layer, part 1: the payload, the formatting helpers, and the deal-detail panel.
    - `js_episode` — The episode page's own wiring: build the marks, render the panels, and keep chart and transcript in sync.
    - `js_hover` — Browser layer, part 2a: the rich hover card that EVERY point on the frontier chart carries.
    - `js_index` — The run index's browser layer: sort and filter, over the rows already in the document.
    - `js_shell` — Browser layer, part 4: the page shell — theme toggle, episode navigation, keyboard shortcuts, help overlay.
    - `js_sidebar` — Browser layer, part 5: the tabbed sidebar and the scroll sync that drives it.
    - `js_transcript` — Browser layer, part 3: the transcript — turn cards, the scrubber, and lazy prompt bodies.
- `auction_geometry` — The plottable geometry of one repeated-auction episode — the `AuctionSpec`-shaped sibling of :class:`~interlens.arena.viz.geometry.Ga
- `auction_page` — The auction episode page's own panels — the four charts design.md §10 commits to, plus the per-turn counterfactual table.

- `ballots` — The final vote, as a tally a reader can check at a glance — including the ballots that were never recorded.
- `census` — How much of an episode is actually play: the per-turn census, and the strip that puts it in the page header.
- `chrome` — The shell every page wears, and the wire form of the payload it carries.
- `compare` — Seat-swap comparison: the same game instance played twice, with one seat's occupant swapped.
- `concepts` — What each solution concept IS, in one place, for every part of the visualizer that explains one to a reader.
- `episode` — One stored episode, turned into the single JSON payload the interactive page renders.
- `export` — The file-writing layer: run directory in, HTML pages plus an index on disk out.
- `geometry` — The plottable geometry of one negotiation instance: every deal placed in a 2-D scale-invariant embedding, with the frontier, the axiomatic solution points, and each party's individually-best deal marked.
- `hazards` — Two facts about a run that decide whether its numbers may be compared with another run's.
- `page` — HTML assembly: a payload in, one self-contained interactive page out.
- `references` — The decision references a scored turn can carry, placed on two axes — and what each one's number MEANS.

        - `serve` — Hand the rendered pages to a browser over HTTP, for when the filesystem the pages live on is not the one the browser runs on.
- `communication` — Pluggable communication topologies: who speaks next, and who sees what.
    - `messaging` — Tool-mediated asynchronous messaging between autonomous agents.
    - `policy` — Communication topology as a pluggable policy.
- `context`
    - `context_policy`
    - `drop_oldest_policy`
    - `error_policy`
    - `sliding_window_policy`
    - `summarize_policy`
- `context_item`
- `conversation`
- `execution_mode`
- `factories`
- `functional` — Copy-on-write functional-update support shared by `Participant` and `Conversation`.
- `hooks`
    - `message_hook`
- `integrations` — Optional adapters that connect Interlens participants to external runtimes.
    - `control_tower` — Run a local Interlens participant behind Control Tower's untrusted-policy boundary.
- `interp` — First-class interpretability layer.
    - `activation_cache`
    - `bridge` — Differentiable bridges for feeding one model's output into another's input.
    - `capture`
    - `grad` — Gradient-enabled forward passes for backprop *through* a model.
    - `layers`

- `logprobs`
- `patching`
- `pooling` — Pool a token-position axis down to one vector per span — the primitive under every span-level readout.
- `routing` — Mixture-of-Experts routing capture and statistics.
- `softtokens` — Virtual (soft) tokens inside ordinary text prompts, plus the message-span read path that pairs with them.
- `steering`

▸ `loading`

- `devices` — Where to put a model's *inputs*, which is not the same question as "what device is the model on".
- `load`
- `model_cache`

▸ `message`

▸ `parsing` — One home for structured-action parsing and reasoning stripping.

▸ `participant`

- `governor` — Adaptive rate-limit governor: admission control paced by the provider's own rate-limit headers.
- `participant`
- `participants`
  - `api_client`
  - `api_participant`
  - `gemma`
  - `llama`
  - `model_participant`
  - `qwen`
  - `scripted_participant`
- `role`
- `serialize` — Persist a participant to / from its own constructor kwargs (for `Conversation.save` / `load`).

- `reasoning_visibility`
- `runner`
  - `analyzer_registry`
  - `batched`
  - `devices`
  - `pool` — The execution engine behind `Conversation.rollout` and `interlens.run`.
  - `worker_init`
- `stop`
  - `conditions`
  - `stop_condition`
- `templating` — Per-row templating for data-driven rollouts.
- `tools`
  - `registry`
  - `tool`
  - `tool_call`
- `transcript`
- `usage` — Usage accounting: token/cost metering for hosted-API participants.
- `view`

interlens

Package `interlens`

The public `interlens` API, exported **lazily**.

Every name below still resolves with a plain `from interlens import X`; the only thing that changed is *when* the defining module is imported. Eagerly importing the whole API pulled in `torch` and `transformers` (~7 s warm, far worse on a cold NFS mount) even for the many CPU-only entry points that never touch a model — `interlens.arena.viz` (reads JSON, writes HTML), `interlens.arena.negotiation.analysis`, audit scripts. The table below is the single source of truth: it maps each defining module to the names it exports, `__getattr__` (PEP 562) imports that module on **first attribute access** and caches the value in module globals, and `__all__` is derived from it. To add an export, add it here and nowhere else.

Modules

- `arena` — Scoreable multi-agent evaluations on interlens: scenarios, episode drivers, and exact scoring.
- `communication` — Pluggable communication topologies: who speaks next, and who sees what.
- `context`
- `context_item`
- `conversation`
- `execution_mode`
- `factories`
- `functional` — Copy-on-write functional-update support shared by `Participant` and `Conversation`.
- `hooks`
- `integrations` — Optional adapters that connect Interlens participants to external runtimes.
- `interp` — First-class interpretability layer.
- `loading`
- `message`

- `parsing` — One home for structured-action parsing and reasoning stripping.
- `participant`
- `reasoning_visibility`
- `runner`
- `stop`
- `templating` — Per-row templating for data-driven rollouts.
- `tools`
- `transcript`
- `usage` — Usage accounting: token/cost metering for hosted-API participants.
- `view`

Re-exported

Name	Defined in	Summary
`APIParticipant`	`interlens.participant.participants.api_participant`	A participant backed by a hosted API — Claude via `anthropic` (`provider="anthropic"`, the default) or any model behind OpenRouter (`provider="openrouter"`, OpenAI-compatible) — for use as a debate opponent, moderator, or the classifier inside an `analyze` callback.
`ActivationCache`	`interlens.interp.activation_cache`	A queryable store of captured activations, tagged by conversation structure.

Name	Defined in	Summary
AnyStopCondition	interlens.stop.stop_condition	Fires when *any* member condition fires.
AutoModelParticipant	interlens.factories	Resolver for creating Participant instances automatically from HuggingFace model identifiers, local model paths, or already-loaded PreTrainedModels.
CaptureSpec	interlens.interp.activation_cache	What to capture during a generation: which sites at which layers, and where to keep the tensors.
CommunicationPolicy	interlens.communication.policy	Who speaks next, and who sees what.
ContextItem	interlens.context_item	A single item of *private* asymmetric knowledge given to one participant (a briefing, a document, a fact).
ContextPolicy	interlens.context.context_policy	Decides how to fit a participant's view within its model's context window.
Conversation	interlens.conversation	A multi-agent conversation: the recipe, the live dialogue, AND the benchmark-rollout dri-

Name	Defined in	Summary
		ver, all in one lightweight object.
CostBudget	interlens.usage	A per-conversation **dollar** budget — the cost-denominated sibling of TokenBudget.
DEFAULT_REGISTRY	interlens.tools.registry	
DatasetField	interlens.templating	A placeholder for row[name] in a templated field, created by :func:dataset_field.
DirectPipingPolicy	interlens.communication.policies	A fixed pipeline: each participant sees only its **predecessor's** output (plus moderator/system framing and its own past turns), and speaking order follows the chain — A → B → C → A → …
DropOldestPolicy	interlens.context.drop_oldest_policy	Drop the oldest turn segments (preserving system/moderator/ private_context) until the view fits.
ElapsedTimeStopCondition	interlens.stop.conditions	Stop once seconds of wall-clock have elapsed since the run started (monotonic clock).

Name	Defined in	Summary
ErrorPolicy	interlens.context.error_policy	The safe default: raise if the view exceeds the context window rather than silently dropping content.
ExecutionMode	interlens.execution_mode	Names the deterministic-vs-throughput tension explicitly rather than pretending it doesn't exist.
Functional	interlens.functional	Mixin giving a dataclass copy-on-write updates via `set(**changes)`.
GemmaModelParticipant	interlens.participant.participants.gemma	A Gemma-family participant.
GradCaptureSpec	interlens.interp.grad	What grad-connected activations to pull from `forward_with_grad` (mirrors `CaptureSpec` minus offload).
GradForwardOutput	interlens.interp.grad	Result of `forward_with_grad`: grad-connected `logits` (`[batch, seq, vocab]`) and, if a `GradCaptureSpec` was passed, `hidden` mapping `(site, layer)` → `tensor[batch, seq, d_model]` (also grad-connected).

Name	Defined in	Summary
`HookAction`	`interlens.hooks.message_hook`	
`LinearBridge`	`interlens.interp.bridge`	Learned linear map from model A's hidden width `d_a` to model B's embedding width `d_b`.
`LlamaModelParticipant`	`interlens.participant.participants_llama`	A Llama-family participant.
`Message`	`interlens.message`	A single committed turn in a conversation.
`MessageHook`	`interlens.hooks.message_hook`	Middleware that inspects each freshly generated message *before* it is committed to the transcript, and may approve / deny / edit it.
`MessageHookResult`	`interlens.hooks.message_hook`	
`MessagingPolicy`	`interlens.communication.messaging`	Asynchronous point-to-point messaging with per-agent mailboxes and a ping-driven scheduler.
`ModelLike`	`interlens.factories`	
`ModelParticipant`	`interlens.participant.participants_model_participant`	A conversation participant backed by a local HuggingFace causal LM.
`OpenRouterRouting`	`interlens.participant.participants_api_participant`	Reproducible OpenRouter routing for research.

Name	Defined in	Summary
Participant	interlens.participant.participant	A participant in a conversation, either a model or a person.
Patch	interlens.interp.patching	Activation patching: overwrite a decoder layer's residual at specific token positions with saved activations (captured from another run/branch).
Provider	interlens.participant.participants.api_participant	
QwenModelParticipant	interlens.participant.participants.qwen	A participant in a conversation that is a Qwen language model.
ReasoningVisibility	interlens.reasoning_visibility	Controls whether a participant's prior <think> reasoning is re-injected into views on later turns.
RoundRobinPolicy	interlens.communication.policy	The shared-transcript default, as an explicit policy: speakers cycle through participants order and everyone sees every committed turn.
RunReport	interlens.runner.pool	Aggregate outcome of a run.
RunResult	interlens.runner.pool	Outcome of one job: the finished

Name	Defined in	Summary
		`conversation` (weightless participants + completed transcript), its `transcript` and (serializable) `analysis`, or an `error` string if it failed.
`ScriptedParticipant`	`interlens.participant.participants.scripted_participant`	A non-model participant that replies with **pre-written (scripted) messages**, cycled in order — its turns are fixed, NOT generated from the conversation.
`SlidingWindowPolicy`	`interlens.context.sliding_window_policy`	Keep the preserved framing plus the most recent `keep_last` turns; drop older turns.
`SteeringSpec`	`interlens.interp.steering`	A residual-stream intervention applied during generation via forward hooks on decoder layers.
`StopCondition`	`interlens.stop.stop_condition`	A stateful predicate that ends a `Conversation.run` early.
`StopStringCondition`	`interlens.stop.conditions`	Stop when a committed message's visible `content` contains any of the given strings (a done-signal).

Name	Defined in	Summary
SummarizePolicy	interlens.context.summarize_policy	Compress older turns into a single summary segment instead of dropping them outright (the heaviest policy).
TokenBudget	interlens.stop.conditions	A per-conversation compute budget — the matched-compute primitive for fair solo-vs-pair comparisons.
TokenStopCondition	interlens.stop.conditions	Stop once the total generated tokens across turns reach `max_tokens`.
Tool	interlens.tools.tool	A capability a participant can invoke during its turn.
ToolCall	interlens.tools.tool_call	A parsed request from the model to invoke a tool: the tool `name` and its `arguments`.
ToolRegistry	interlens.tools.registry	Resolves tool *names* (which serialize) to live `Tool` instances (which don't).
ToolResult	interlens.tools.tool_call	The outcome of executing a `ToolCall`: the tool `name` and its string `output` (or error text).

Name	Defined in	Summary
`Transcript`	`interlens.transcript`	The canonical, perspective-neutral record of a conversation, plus the logic to render it *from* a given participant's perspective.
`TurnStopCondition`	`interlens.stop.conditions`	Stop after `max_turns` committed turns.
`UsageMeter`	`interlens.usage`	A cumulative, thread-safe dollar ledger shared by every metered participant in a run.
`available_devices`	`interlens.runner.devices`	List the devices to spread conversations across: every CUDA GPU, else a single mps/cpu fallback.
`continuation_logprob`	`interlens.interp.grad`	Differentiable teacher-forced logprob of `target_ids` continuing a prefix, under `model`.
`conversation_from_ids`	`interlens.factories`	Deprecated thin alias for :func:`conversation_from_` kept for back-compat.
`conversation_from_models`	`interlens.factories`	Scaffold a conversation from a tuple of `models` — each an HF id (str) or an already-loaded `PreTrainedModel` (see `ModelLike`).

Name	Defined in	Summary
dataset_field	interlens.templating	Reference a dataset column in a templated field: shared_context=("Solve: \n\n", dataset_field("question")) re-solves row["question"] for each row at rollout expansion.
decoder_layers	interlens.interp.layers	Return the list of transformer decoder layer modules, across common HF architectures.
forward_with_grad	interlens.interp.grad	One grad-connected forward pass over input_ids XOR inputs_embeds.
gumbel_softmax_tokens	interlens.interp.bridge	Relaxed (Gumbel-softmax) sample over the last (vocab) dim of logits, differentiable in logits.
register_analyzer	interlens.runner.analyzer_registry	
register_pricing	interlens.usage	Register (or override) the $/Mtok pricing for model_id, process-wide.
register_worker_init	interlens.runner.worker_init	Register a zero-arg callable to run once at worker startup (e.g. to

Name	Defined in	Summary
		populate the tool/analyzer registries).
`run`	`interlens.runner.pool`	Run several conversation lineups in ONE pool — the multi-lineup entry point (e.g. a ladder of model pairs × conditions in one overnight job).
`run_jobs`	`interlens.runner.pool`	Run `(job_id, Conversation)` jobs across devices, with checkpointing, resume, and per-job failure isolation.
`soft_embed`	`interlens.interp.bridge`	Mix `model`'s input-embedding rows by a per-position distribution: `[..., V] @ E[V, d]` → `[..., d]`.
`token_logprobs`	`interlens.interp.logprobs`	Compute per-token logprobs / surprisal / entropy for a generation.
`transcript_usage`	`interlens.usage`	Aggregate the recorded usage of one transcript: total generated/input tokens, dollar cost, and a per-author breakdown — read from each

Name	Defined in	Summary
		committed message's metadata (`n_tokens` / `n_tokens_in` / `cost_usd`), the same source of truth `TokenBudget` and `CostBudget` use.

arena

Package `interlens.arena`

Scoreable multi-agent evaluations on interlens: scenarios, episode drivers, and exact scoring.

The arena turns interlens from a conversation harness into an evaluation harness: a `Scenario` defines a game (solver-verified instance generation, per-seat private framing, a turn protocol with structured JSON actions, early termination, exact scoring), and the engine plays it through any `Participant` — hosted-API or local — as concurrently as the backend allows, persisting every episode to one JSON schema.

Quickstart:

```
from interlens import APIParticipant, UsageMeter
from interlens.arena import EpisodePool, EpisodeStore
from interlens.arena.scenarios import Negotiation
import asyncio

scenario = Negotiation()
instance = scenario.generate_instance(level=0, seed=1)      # solver-
     verified: exact ceiling + optimum
meter = UsageMeter(budget=20.0)
player = APIParticipant(name="player", model_id="claude-sonnet-5",
     meter=meter, turn_token_floor=2048)
pool = EpisodePool(EpisodeStore("episodes/"), meter=meter)
episode = asyncio.run(pool.run_episode(scenario, instance, "team",
     player))
print(episode.outcome, episode.usage(), meter.summary())
```

Optional Inspect integration (`pip install interlens[inspect]`): `interlens.arena.inspect` exposes the bundled scenarios as `inspect eval`-runnable tasks.

Modules

- `actions` — Typed formal-action layer for structured negotiation turns.
- `auction` — Repeated multi-bidder auctions: the frozen spec, the persona-conditioned prior, exact allocation and payment rules,

equilibrium benchmarks, computable bidders and oracles, and the collusion metrics.

- `engine` — Episode drivers: play `Scenario` instances through `Participant`s.
- `export` — Human-readable transcripts from stored episodes: an `EpisodeStore` tree -> one markdown + one self-contained HTML page per episode, plus a per-run index.
- `gates` — Template-fidelity gates: preflight checks before spending GPU-hours on local-model rollouts.
- `inspect` — Optional Inspect (inspect-ai) integration: run arena scenarios under `inspect eval`.
- `live` — Live play: watch an arena episode as it happens, reconfigure its seats, and play one yourself.
- `negotiation` — Multi-issue, multi-party scorable negotiation: deal spaces, private score sheets, exact solution concepts, computable rational-agent oracles, and an executable strategy zoo.
- `oracles` — The oracle layer: per-turn “what would a rational agent have done here?” annotations.
- `ratchet` — Adaptive difficulty ratchet: find the level where a model stops clearing the bar, then measure there.
- `refusal` — Recovering a turn the API refused, without changing what the turn says.
- `replay` — Deterministic replay of stored episodes through a scenario’s state machine.
- `rollouts` — Directory-backed **rollout sets**: run more episodes into the same place, safely, and resume.
- `scenario` — The `Scenario` interface: a pure game-logic state machine, participant-agnostic.
- `scenarios` — Bundled scenarios, four families:
- `schema` — The arena’s record schema: one JSON shape for every episode.
- `table` — Heterogeneous **tables**: present a whole many-seat lineup to the arena engine as one participant.
- `views` — Per-seat view construction + structured-action parsing for arena scenarios.

- `viz` — Interactive episode visualization: any arena run directory in, self-contained interactive HTML out.

Re-exported

Name	Defined in	Summary
`Accept`	`interlens.arena.actions`	Accept a specific standing offer by id (an `ACCEPT` vote on that exact deal).
`Action`	`interlens.arena.actions`	Base class for the four formal moves.
`BatchedEpisodePool`	`interlens.arena.engine`	Synchronous co-stepping for local model participants: each tick gathers every live episode's pending requests and runs them as one batched `generate_batch` per participant — the local-GPU throughput path.
`Deal`	`interlens.arena.actions`	
`DifficultyRatchet`	`interlens.arena.ratchet`	Probe -> measure -> paired-solo driver over one scenario/participant pair (see the module docstring).
`EMPTY_TURN_PLACEHOLDER`	`interlens.arena.engine`	
`Episode`	`interlens.arena.schema`	One complete playthrough: turns, forked provisional check-

Name	Defined in	Summary
		points, outcome, and usage accounting.
EpisodePool	interlens.arena.engine	Concurrent episodes as independent asyncio tasks — one participant call at a time per episode, many episodes in flight.
EpisodeRun	interlens.arena.engine	Per-episode bookkeeping shared by both drivers: state stepping, turn recording, budget checks, retries, and finalization.
EpisodeStore	interlens.arena.schema	Per-episode JSON persistence, written atomically on every update so a crash loses at most one turn.
GenerationFailureBudgetExceeded	interlens.arena.engine	Raised by :class:`BatchedEpisodePool` when it has had to fabricate more turns than its budget allows.
Instance	interlens.arena.schema	One generated problem instance, solver-verified at generation time.
LEGALITY	interlens.arena.actions	
Offer	interlens.arena.actions	One registered offer and its live vote state.

Name	Defined in	Summary
`OfferId`	`interlens.arena.actions`	
`OfferRegistry`	`interlens.arena.actions`	Monotonic offer ids + standing-offer tracking for the formal protocol.
`Oracle`	`interlens.arena.oracles`	A rational reference policy that scores a seat's options at a decision point.
`OracleRecord`	`interlens.arena.oracles`	One per-turn oracle annotation on an episode (the typed replacement for the loose checkpoint dict).
`OracleVerdict`	`interlens.arena.oracles`	One oracle's read of a decision point.
`PERSONAS`	`interlens.arena.schema`	
`ParseResult`	`interlens.arena.actions`	The outcome of reading one formal action from a model's turn.
`Pass`	`interlens.arena.actions`	Take NO formal move this turn — the typed form of the talk-only pass a scenario already accepts as `{"action": "none"}`.
`Propose`	`interlens.arena.actions`	Register a complete deal.
`RefusalLadder`	`interlens.arena.refusal`	A fixed, seeded sequence of content-

Name	Defined in	Summary
		preserving re-renders, tried in order after a refusal.
`Reject`	`interlens.arena.actions`	Reject a specific standing offer by id.
`ReplayError`	`interlens.arena.replay`	A stored turn could not be matched to the state machine's pending request.
`RolloutConfigMismatch`	`interlens.arena.rollouts`	An append's config fingerprint disagrees with the set's manifest — refused rather than silently mixed.
`RolloutSet`	`interlens.arena.rollouts`	A directory of episodes plus the manifest that says what produced them.
`SYNTAX`	`interlens.arena.actions`	
`Scenario`	`interlens.arena.scenario`	
`SeatRequest`	`interlens.arena.schema`	One pending generation: a seat that must speak now, with the exact view its model is conditioned on.
`TURN_SIGNATURES`	`interlens.arena.engine`	
`TurnRecord`	`interlens.arena.schema`	
`Walk`	`interlens.arena.actions`	Explicit no-deal exit — a decision, not a timeout.

Name	Defined in	Summary
action_from_json	interlens.arena.actions	Reconstruct a typed :class:Action from its stored dict — the inverse of Action.to_json().
annotate	interlens.arena.oracles	Run every oracle over one decision point and return one inline :class:OracleRecord each — the ready helper a scenario calls from annotate_turn so its oracle wiring is a single line.
build_view	interlens.arena.views	Render events ([{seat|'MODERATOR', content, only?}], public unless only names seats) into an alternating per-speaker view for seat, ending with phase_prompt as the final user turn.
check_reasoning_leak	interlens.arena.gates	Scan a played episode for reasoning leakage: any turn whose raw completion contains a <think> block whose content then appears verbatim in a LATER turn's visible content

Name	Defined in	Summary
		(meaning another seat saw it).
check_template_fidelity	interlens.arena.gates	Assert token-id equality between tokenizer(apply_chat_template(tokeniz and apply_chat_template(tokenize=True) for every view.
config_fingerprint	interlens.arena.rollouts	Stable SHA256 over config, canonicalized (sorted keys, compact separators, non-JSON values repr'd).
extract_json	interlens.arena.views	The last fenced JSON object in text, else the last balanced top-level {...} that parses.
found_level	interlens.arena.ratchet	The found level implied by a set of probe means: the first (ascending) probed level whose mean fails the bar, else the highest level probed (the model never dropped below the bar).
gen_failures	interlens.arena.engine	Every turn of an episode whose text the ENGINE fabricated because generation failed.
is_refusal	interlens.arena.refusal	Did this completion come back declined

Name	Defined in	Summary
		rather than generated?
load_instances	interlens.arena.schema	Load an instance pool saved by save_instances (or by the arena experiments — the pre-rename env key is migrated on read).
parse_action	interlens.arena.actions	Read ONE formal action from text (its last fenced/balanced JSON object), validated into a typed Action or a classified failure.
replay_episode	interlens.arena.replay	Feed a stored episode's turns back through scenario and return the recomputed outcome dict.
rescore	interlens.arena.replay	Replay episode and compare the recomputed outcome to the recorded one on fields.
rollout	interlens.arena.rollouts	Run instances x seeds x arms into the rollout set at out (creating or resuming it) and return it.

Name	Defined in	Summary
save_instances	interlens.arena.schema	Persist an instance pool as one JSON file ({scenario}_L{level}.json unless name overrides).
scenario_smoke_views	interlens.arena.gates	Real rendered views for gating: a fresh team state's first requests, plus the views after one scripted turn (so later views carry assistant/user mixes, which is where templates diverge).
strip_think	interlens.parsing	Remove <think>...</think> blocks ANYWHERE in text; return (visible, think).
truncation_budget	interlens.arena.engine	The output budget one stored turn's n_tokens_out may honestly be compared against; 0 for "none, so read truncation from stop_reason alone".
turn_signatures	interlens.arena.engine	Every failure signature carried by one stored turn (a TurnRecord.to_json() dict); set() if healthy.

actions

Module `interlens.arena.actions`

Typed formal-action layer for structured negotiation turns.

A scenario that wants *binding moves* (not just free text) gives each turn one formal action drawn from the package-deal protocol — `Propose` a complete deal (which the registry stamps with a monotonic `offer_id`), `Accept` / `Reject` a specific live offer by id, or `Walk` (an explicit no-deal exit). Referencing offers by id removes the "I accept" ambiguity that breaks down with two live proposals on the table.

Three pieces:

- **The action dataclasses** (`Propose` / `Accept` / `Reject` / `Walk`), all frozen and JSON-round-trippable — the frozen team contract's move vocabulary. A `Deal` is a tuple of option indices (one per issue); deals are decoded from a model's `{"Site": "...", ...}` object by a scenario-supplied `deal_decoder`, so this module stays free of any specific game's issue set.
- **`OfferRegistry`** — monotonic ids and standing-offer tracking (accept/reject sets per offer, withdrawal), serializable for the episode record and reconstructable by replay (it is a pure function of the action sequence).
- **`parse_action`** — the single consolidated JSON-extraction-and-validation entry point. It returns a `ParseResult` distinguishing a **syntax** violation (no well-formed action could be read) from an **economic-legality** violation (well-formed but references a dead offer / an infeasible deal), so a scenario can retry once with specific feedback and log the two failure classes separately as data.

Attributes

Name	Type	Summary
`Deal`		

Name	Type	Summary
FACILITATOR		
FACILITATOR_SEAT		
LEGALITY		
OfferId		
SYNTAX		

Classes

Name	Summary
Accept	Accept a specific standing offer by id (an ACCEPT vote on that exact deal).
Action	Base class for the four formal moves.
Offer	One registered offer and its live vote state.
OfferRegistry	Monotonic offer ids + standing-offer tracking for the formal protocol.
ParseResult	The outcome of reading one formal action from a model's turn.
Pass	Take NO formal move this turn — the typed form of the talk-only pass a scenario already accepts as {"action": "none"}.
Propose	Register a complete deal.
Reject	Reject a specific standing offer by id.
Walk	Explicit no-deal exit — a decision, not a timeout.

Functions

Name	Summary
action_from_json	Reconstruct a typed :class:Action from its stored dict — the inverse of Action.to_json().
action_from_key	Rebuild the :class:Action a :func:action_key string names; None/unparseable → None.
action_key	The canonical STRING an action serializes to when it must be a JSON object key or a sort key — its to_json() dumped with sorted keys ('{"action": "accept", "offer_id": "O1"}').
action_message	Render action as a message body: an optional free-text preface, then the fenced json action block — the exact envelope an LLM seat produces, so a transcript is symmetric across seat types (a PolicyParticipant emits through here).
parse_action	Read ONE formal action from text (its last fenced/balanced JSON object), validated into a typed Action or a classified failure.

Accept

Accept a specific standing offer by id (an `ACCEPT` vote on that exact deal).

```
Accept(offer_id: OfferId)
```

Defined in `interlens.arena.actions` · source

Inherits from: Action

Parameters

Name	Type	Default	Description
`offer_id`	`OfferId`	*required*	

Attributes

Name	Type	Summary
`kind`	`str`	
`offer_id`	`OfferId`	

Methods

to_json

```
to_json(self) -> dict
```

source

Action

Base class for the four formal moves.

```
Action()
```

Defined in `interlens.arena.actions` · source

`kind` is the wire tag; `to_json` is the canonical surface a model emits inside a fenced `{"action": ...}` object.

Attributes

Name	Type	Summary
`kind`	`str`	

Methods

`to_json`

```
to_json(self) -> dict
```

source

Offer

One registered offer and its live vote state.

```
Offer(
    offer_id: OfferId,
    deal: Deal,
    proposer: str,
    round: int = 0,
    accepts: set[str] = set(),
    rejects: set[str] = set(),
    live: bool = True,
)
```

Defined in `interlens.arena.actions` · source

`live` flips to False on withdrawal/supersession; the accept/reject sets are the votes gathered so far (closure semantics — unanimity vs veto-weighted — are the scenario's call, computed off these sets).

Parameters

Name	Type	Default	Description
`offer_id`	`OfferId`	*required*	
`deal`	`Deal`	*required*	
`proposer`	`str`	*required*	
`round`	`int`	`0`	
`accepts`	`set[str]`	`set()`	
`rejects`	`set[str]`	`set()`	
`live`	`bool`	`True`	

Attributes

Name	Type	Summary
`accepts`	`set[str]`	
`deal`	`Deal`	

Name	Type	Summary
`live`	`bool`	
`offer_id`	`OfferId`	
`proposer`	`str`	
`rejects`	`set[str]`	
`round`	`int`	

Methods

from_json

```
from_json(d: dict) -> 'Offer'
```

source

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

to_json

```
to_json(self) -> dict
```

source

OfferRegistry

Monotonic offer ids + standing-offer tracking for the formal protocol.

```
OfferRegistry(prefix: str = 'O')
```

Defined in `interlens.arena.actions` · source

`register` mints `{prefix}{n}` ids (`O1`, `O2`, …); the proposer is recorded as an implicit accept of its own offer (proposing implies supporting it). `accept` / `reject` record a seat's vote on a live offer (a vote on a dead/unknown offer is refused and returns False). Withdrawing an offer marks it not-live without deleting it, so the ledger stays complete for the record. The registry is a pure function of the action sequence, so replay reconstructs it identically; `to_json` / `from_json` also persist it directly.

Parameters

Name	Type	Default	Description
`prefix`	`str`	`'O'`	

Attributes

Name	Type	Summary
`offers`	dict[OfferId, Offer]	
`prefix`		

Methods

`accept`

```
accept(self, offer_id: OfferId, agent: str) -> bool
```

source

Record `agent`'s ACCEPT of a live offer; a reject by the same agent is cleared.

Returns False (no effect) when the offer is unknown or dead.

Parameters

Name	Type	Default	Description
`offer_id`	`OfferId`	*required*	
`agent`	`str`	*required*	

apply

```
apply(self, action: Action, agent: str, *, round: int = 0) -> OfferId
     | None
```

source

Fold one parsed `action` into the registry: `Propose` registers (returns the new id), `Accept` / `Reject` record the vote, `Walk` is a no-op here (the scenario handles the exit).

Returns the new offer id for a `Propose`, else `None`.

Parameters

Name	Type	Default	Description
`action`	Action	*required*	
`agent`	`str`	*required*	
`round`	`int`	`0`	

from_json

```
from_json(d: dict) -> 'OfferRegistry'
```

source

Parameters

Name	Type	Default	Description
d	dict	*required*	

get

```
get(self, offer_id: OfferId) -> Offer | None
```

source

Parameters

Name	Type	Default	Description
offer_id	OfferId	*required*	

is_live

```
is_live(self, offer_id: OfferId) -> bool
```

source

Parameters

Name	Type	Default	Description
offer_id	OfferId	*required*	

register

```
register(
    self,
    deal: Deal,
    proposer: str,
    *,
```

```
    round: int = 0,
    implicit_accept: bool = True,
) -> OfferId
```

source

Mint a new live offer for `deal` by `proposer` and return its id.

The proposer implicitly accepts.

`implicit_accept=False` registers the offer with an EMPTY accept set — the explicit no-vote path for a proposer that is not a party to the deal (:data:`FACILITATOR`). Without it a mediator's own id would sit in `accepts` and any closure rule that counts the accept set (or renders "accepted by: …") would report a vote nobody cast.

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	
`proposer`	`str`	*required*	
`round`	`int`	`0`	
`implicit_accept`	`bool`	`True`	

`reject`

```
reject(self, offer_id: OfferId, agent: str) -> bool
```

source

Record `agent`'s REJECT of a live offer; an accept by the same agent is cleared.

Returns False when the offer is unknown or dead.

Parameters

Name	Type	Default	Description
`offer_id`	`OfferId`	*required*	
`agent`	`str`	*required*	

standing

```
standing(self) -> list[Offer]
```

source

Live offers, in registration order.

standing_ids

```
standing_ids(self) -> list[OfferId]
```

source

Live offer ids in REGISTRATION order (same order as :meth:`standing`).

A list, not a set, deliberately: callers build ordered things out of it — the legal-action set an oracle scores, and hence the stored verdict — and a set's iteration order varies with `PYTHONHASHSEED`, which used to make a saved annotation differ run to run. Membership tests (`offer_id in standing_ids()`, how :func:`parse_action` gates accept/reject) read the same either way.

to_json

```
to_json(self) -> dict
```

source

withdraw

```
withdraw(self, offer_id: OfferId) -> bool
```

source

Parameters

Name	Type	Default	Description
offer_id	OfferId	*required*	

ParseResult

The outcome of reading one formal action from a model's turn.

```
ParseResult(
    action: Action | None = None,
    ok: bool = False,
    error: str | None = None,
    error_kind: str | None = None,
    raw: Any = None,
)
```

Defined in `interlens.arena.actions` · source

`ok` with an `action` on success; on failure `action` is `None`, `error` is a specific, model-facing message for the one retry, and `error_kind` is :data:`SYNTAX` or :data:`LEGALITY` — the two failure classes the arena logs separately as data (AucArena's retry-once pattern).

Parameters

Name	Type	Default	Description
`action`	Action | None	`None`	
`ok`	`bool`	`False`	
`error`	`str | None`	`None`	
`error_kind`	`str | None`	`None`	
`raw`	`Any`	`None`	

Attributes

Name	Type	Summary
`action`	Action | None	
`error`	`str | None`	
`error_kind`	`str | None`	
`ok`	`bool`	

Name	Type	Summary
raw	Any	

Methods

bad

```
bad(kind: str, error: str, raw: Any = None) -> 'ParseResult'
```

source

Parameters

Name	Type	Default	Description
kind	str	*required*	
error	str	*required*	
raw	Any	None	

good

```
good(action: Action, raw: Any = None) -> 'ParseResult'
```

source

Parameters

Name	Type	Default	Description
action	Action	*required*	
raw	Any	None	

retry_directive

```
retry_directive(self) -> dict | None
```

source

`{'retry': <error>, 'error_kind': <kind>}` on failure (the scenario returns this from `apply` to trigger the engine's one re-prompt), else `None`.

The engine reads `'retry'`; `'error_kind'` rides along for logging.

Pass

Take NO formal move this turn — the typed form of the talk-only pass a scenario already accepts as `{"action": "none"}`.

```
Pass()
```

Defined in `interlens.arena.actions` · source

Inherits from: Action

It is not part of the package-deal move vocabulary :func:`parse_action` validates: a scenario recognizes the `none`/`pass` wire tag BEFORE it reaches the parser (`ScorableNegotiation.apply`), records the turn as `atype="none"`, and charges neither a syntax nor a legality error. It exists so a `PolicyParticipant` can express “stand pat” in the same typed vocabulary as the other moves instead of being forced into a move it does not want — the alternatives are all worse: `Walk` permanently removes the seat (and kills the deal outright if it holds a veto), `Reject` needs a live offer id and is illegal in the forced-final proposal phase, and re-`Propose`-ing a deal already on the table mints a SECOND offer id for it, splitting the accept votes across two ids so neither can reach unanimity.

One caveat, from the same short-circuit: a MOVES-ONLY game (no talk channel) treats a turn with no formal action as a syntax error, so a policy that can emit `Pass` belongs in a chat-enabled arm.

Attributes

Name	Type	Summary
`kind`	`str`	

Propose

Register a complete deal.

```
Propose(deal: Deal)
```

Defined in `interlens.arena.actions` · source

Inherits from: Action

The registry assigns the `offer_id` on registration — it is not chosen by the model — so ids are monotonic and unambiguous. `deal` is the decoded option-index tuple.

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	

Attributes

Name	Type	Summary
`deal`	`Deal`	
`kind`	`str`	

Methods

to_json

```
to_json(self) -> dict
```

source

Reject

Reject a specific standing offer by id.

```
Reject(offer_id: OfferId)
```

Defined in `interlens.arena.actions` · source

Inherits from: Action

Parameters

Name	Type	Default	Description
`offer_id`	`OfferId`	*required*	

Attributes

Name	Type	Summary
`kind`	`str`	
`offer_id`	`OfferId`	

Methods

`to_json`

```
to_json(self) -> dict
```

source

Walk

Explicit no-deal exit — a decision, not a timeout.

```
Walk()
```

Defined in `interlens.arena.actions` · source

Inherits from: Action

Attributes

Name	Type	Summary
kind	str	

action_from_json

Reconstruct a typed :class:`Action` from its stored dict — the inverse of `Action.to_json()`.

```
action_from_json(d: dict) -> Action
```

Defined in `interlens.arena.actions` · source

Accepts the kind under `"action"` / `"type"` / `"kind"` (so it reads both the canonical flat form and a stored nested action object). Used to round-trip verdicts and to rebuild the action series from stored episodes. Raises `ValueError` if `d` doesn't name a known action kind.

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

action_from_key

Rebuild the :class:`Action` a :func:`action_key` string names; `None`/unparseable → `None`.

```
action_from_key(key: str | None) -> Action | None
```

Defined in `interlens.arena.actions` · source

Parameters

Name	Type	Default	Description
key	str | None	*required*	

action_key

The canonical STRING an action serializes to when it must be a JSON object key or a sort key — its `to_json()` dumped with sorted keys (`'{"action": "accept", "offer_id": "O1"}'`).

```
action_key(action: Action | None) -> str | None
```

Defined in `interlens.arena.actions` · source

`None` maps to `None`.

One definition serves two needs that must agree: an `OracleVerdict`'s `action_values` is keyed by action, and JSON object keys must be strings; and the oracles sort their scored actions by this key so a stored verdict is byte-reproducible regardless of the order the legal actions arrived in. :func:`action_from_key` is the exact inverse.

Memoized when it can be: every `Action` is a frozen dataclass, so the key is a pure function of an immutable value and a cache hit returns the identical string. It earns its keep because the oracles call this once per legal action per turn to SORT them, and a long-horizon turn carries an accept and a reject for every live offer — the same few actions, re-created and re-serialized every round. An action that is nonetheless unhashable (a `Propose` built with a list `deal` rather than the decoded tuple) still works: it takes the uncached path and gets the same string, so memoization can never narrow what this function accepts.

Parameters

Name	Type	Default	Description
`action`	Action | None	*required*	

action_message

Render `action` as a message body: an optional free-text `preface`, then the fenced `json` action block — the exact envelope an LLM seat produces, so a transcript is symmetric across seat types (a `PolicyParticipant` emits through here).

```
action_message(
    action: Action,
    space=None,
    *,
    preface: str = '',
    message: str | None = None,
) -> str
```

Defined in `interlens.arena.actions` · source

The action serializes through its own :meth:`Action.to_json` — the ONE action serializer — so `deal` is a list of option indices. Pass `space` (anything with a `named(deal)` method, i.e. a :class:`~interlens.arena.negotiation.space.DealSpace`) to render a `Propose`'s deal as `{issue_name: option_label}` instead, which is what an LLM-legible transcript wants; `DealSpace.parse` is the inverse when reading one back. `space` is duck-typed, so this module keeps no dependency on the negotiation package.

`message` puts public cheap talk INSIDE the envelope, under the `"message"` key an LLM seat uses and a chat-enabled scenario republishes to every other seat. That is the only speaking channel a scripted seat has: `preface` is free text OUTSIDE the fence, which the scenario's parser never reads, so a preface is transcript decoration whereas a `message` is an actual public statement. Key order matches the LLM envelope (message first, then the formal action) so the two are indistinguishable on the wire. A moves-only game has no talk channel and drops the key, so a speaking policy belongs in a chat arm.

Parameters

Name	Type	Default	Description
`action`	Action	*required*	

Name	Type	Default	Description
space		None	
preface	str	''	
message	str | None	None	

parse_action

Read ONE formal action from `text` (its last fenced/balanced JSON object), validated into a typed `Action` or a classified failure.

```
parse_action(
    text: str,
    *,
    deal_decoder: Callable[[Any], Deal | None] | None = None,
    standing: Container[OfferId] | None = None,
    allowed: Container[str] | None = None,
) -> ParseResult
```

Defined in `interlens.arena.actions` · source

- `deal_decoder` maps a `Propose`'s `"deal"` object to a :data:`Deal` tuple, returning `None` for a malformed/infeasible deal (an economic-legality failure). When omitted, a deal given as a list of option indices is accepted as-is; any other shape is a legality failure.
- `standing` (the live offer ids, e.g. `registry.standing_ids()`) gates `Accept` / `Reject`: a reference to an id not in it is an economic-legality failure. When omitted, id existence is not checked here (the scenario/ registry can check on apply).
- `allowed` optionally restricts which action kinds are legal at this point (e.g. only `accept` on a finalization turn); a disallowed-but-well-formed kind is a legality failure.

Accepts both the flat wire form `{"action": "propose"|"accept"|"reject"|"walk", ...}` and the nested form `{"action": {"type": "propose", ...}}` (see :func:`_holder_and_kind`); `"offer_id"` also accepts the aliases `"id"` / `"offer"`. A turn that mixes cheap talk with a move carries both in one object; the scenario reads the `message` field itself and calls this for the action.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	
`deal_decoder`	`Callable[[Any], Deal | None] | None`	`None`	
`standing`	`Container[OfferId] | None`	`None`	
`allowed`	`Container[str] | None`	`None`	

auction

Package `interlens.arena.auction`

Repeated multi-bidder auctions: the frozen spec, the persona-conditioned prior, exact allocation and payment rules, equilibrium benchmarks, computable bidders and oracles, and the collusion metrics.

The sibling of :mod:`interlens.arena.negotiation`. Each module's docstring carries the primary citations for what it implements; :mod:`.references` maps every citation key to its full reference with the exact page range the module relies on.

Layout:

- :mod:`.spec` — `AuctionSpec` / `BidderSpec` / `StageDraw` / `Mechanism`, and `generate_spec`.
- :mod:`.priors` — the persona table, the generative model, fact-rendering data, and `RivalPosterior`.
- :mod:`.allocation` — bundle values, the exact efficient allocation, VCG / clinching / uniform-price rules.
- :mod:`.benchmarks` — the exact per-stage equilibrium benchmarks every suppression metric divides against.
- :mod:`.actions` — the auction move vocabulary, `BidLedger`, `DMRouter`, and the action parser.
- :mod:`.bidders` — `AuctionState`, the policy zoo, and their oracle variants.
- :mod:`.metrics` — stage-level and repeated-play metrics, as pure functions over records.

Modules

- `actions` — The auction move vocabulary, the bid ledger, and DM routing (design.md §12 item 2).
- `allocation` — Bundle values, the exact efficient allocation, and the payment rules.

- `benchmarks` — The exact per-stage equilibrium benchmarks every suppression metric divides against (design.md §4.3, §5).
- `bidders` — The computable bidder zoo: policies (`state` → `action`), their DM decision rules, and their oracles.
- `metrics` — Stage-level and repeated-play metrics (design.md §5), as pure functions over records.
- `policy_text` — What the computable seats SAY – the templated broadcast and DM behavior of design.md §3.4.
- `priors` — The persona-conditioned prior: the generative model, the persona table, fact rendering data, and the posterior a rational seat actually computes.
- `references` — Citation-key registry for the auction mechanism, benchmark, and collusion-metric modules.
- `spec` — The frozen auction specification: item slots, bidders, per-stage draws, and the mechanism config.

actions

Module `interlens.arena.auction.actions`

The auction move vocabulary, the bid ledger, and DM routing (design.md §12 item 2).

This module EXTENDS `arena/actions.py` rather than editing it: the negotiation move vocabulary (`Propose`/`Accept`/`Reject`/`Walk`) is untouched, and the shared machinery it already owns — :class:`~interlens.arena.actions.Action`, :class:`~interlens.arena.actions.Par and the :data:`~interlens.arena.actions.SYNTAX` / :data:`~interlens.arena.actions.LEGALITY` error classes that drive retry-once-with-specific-feedback — is imported and reused.

Three pieces, mirroring the negotiation layer one for one:

- **The action dataclasses** — one per row of design.md §3.3′s action grammar, plus the three side-channel moves (:class:`Speak`, :class:`DirectMessage`, :class:`Transfer`) that ride alongside the binding move in the four-channel envelope of §3.2.
- **:class:`BidLedger`** — the sibling of `OfferRegistry`: monotonic ids, standing-high tracking per lot, irrevocable exits, and the eligibility ratchet, all as a PURE FUNCTION of the action sequence, so replay reconstructs the ledger exactly and an analysis never has to trust a stored summary.
- **:class:`DMRouter`** — delivers addressed messages to their recipients only, enforces `dm_cap`, and records the full directed graph (sender, recipient, stage, round, text) that the DM-graph panel and the per-dyad mutual-information estimator both read.

Economic errors are MEASURED, never blocked (design.md §3.2): bidding above your own valuation parses cleanly and is counted downstream. Bidding above BUDGET is different — payments must be collectible — so it is a :data:`~interlens.arena.actions.LEGALITY` error here, retried once and then truncated by the scenario.

Attributes

Name	Type	Summary
LEGAL_KINDS	dict[str, tuple[str, ...]]	
TRANSFER_CONDITIONS	tuple[str, ...]	
resolve_item		
whole_number		

Classes

Name	Summary
Bid	A priced bid of amount on lot item (a slot index).
BidLedger	Monotonic bid ids, standing-high tracking, exits, and the eligibility ratchet.
Claim	Take the lot at the current descending-clock price (Dutch).
DMRouter	Delivers addressed private messages to their recipients only, and records the directed graph.
Demand	A demand for units at the current clock price in a clinching stage; must be weakly decreasing across rounds [ausubel2004].
DirectMessage	A private message to named recipients.
DirectMessageRecord	One delivered private message.
Exit	Leave the ascending clock.
PassLot	Decline to bid on lot item this round.

Name	Summary
SAATurn	One whole SAA turn: the raises and the permanent passes a seat declared together.
Schedule	A weakly-decreasing per-unit bid vector for a uniform-price stage.
Speak	Public broadcast cheap talk.
StandingBid	One registered bid and its live state — the sibling of `arena.actions.Offer`.
Stay	Remain active at the current ascending-clock price (English).
Transfer	A side payment the harness EXECUTES at settlement.
TransferBook	Declared side payments, and their execution at settlement (`dm_transfers` only).
TurnEnvelope	One turn's four channels (design.md §3.2), separated: the private `scratchpad` (never published), public `message`, addressed `dms`, an optional `transfer`, and the one binding `action`.
Wait	Let the descending clock fall one increment (Dutch).

Functions

Name	Summary
auction_action_from_json	Reconstruct a typed auction :class:Action from its stored dict — the inverse of to_json.
parse_auction_action	Read and validate ONE binding auction move — the single consolidated entry point, the sibling of arena.actions.parse_action.
parse_envelope	Split a turn's fenced JSON object into its four channels WITHOUT validating the binding move.

Bid

A priced bid of `amount` on lot `item` (a slot index).

```
Bid(item: int, amount: int)
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

The single-lot families carry `item = 0`; SAA turns may carry several :class:`Bid` moves, one per lot.

Parameters

Name	Type	Default	Description
item	int	*required*	
amount	int	*required*	

Attributes

Name	Type	Summary
amount	int	
item	int	
kind	str	

Methods

to_json

```
to_json(self) -> dict
```

source

BidLedger

Monotonic bid ids, standing-high tracking, exits, and the eligibility ratchet.

```
BidLedger(n_items: int, *, prefix: str = 'B', activity_rule: str = 'none')
```

Defined in `interlens.arena.auction.actions` · source

The sibling of `arena.actions.OfferRegistry` and it holds the same purity property: the ledger is a PURE FUNCTION of the applied action sequence, so `replay.py` reconstructs it identically from a stored episode and no analysis has to trust a stored summary. `to_json` / `from_json` also persist it directly.

Parameters

n_items : int Number of lots, so per-lot state is allocated up front. prefix : str Bid-id prefix; ids are `B1`, `B2`, … in application order across the whole episode. activity_rule : str `"eligibility_ratchet"` makes a :class:`PassLot` irrevocable within its stage (a bidder that passes on lot `j` may not bid on `j` again that stage); `"none"` records the pass without binding it.

Parameters

Name	Type	Default	Description
`n_items`	`int`	*required*	
`prefix`	`str`	`'B'`	
`activity_rule`	`str`	`'none'`	

Attributes

Name	Type	Summary
`activity_rule`		
`bids`	list[StandingBid]	
`exited`	`dict[int, list[int]]`	

Name	Type	Summary
`n_items`		
`passed`	`dict[int, set[tuple[int, int]]]`	
`prefix`		

Methods

active_seats

```
active_seats(self, stage: int, n_bidders: int) -> list[int]
```

source

Seats that have not exited the clock in `stage`.

Parameters

Name	Type	Default	Description
`stage`	`int`	*required*	
`n_bidders`	`int`	*required*	

apply

```
apply(self, action: Action, seat: int, *, stage: int, round: int) -> str
     | None
```

source

Fold one parsed action into the ledger. :class:`Bid` registers (returning the new bid id and superseding any lower standing bid on that lot), :class:`PassLot` records the pass, :class:`Exit` records the irrevocable clock exit; everything else is a no-op here (the scenario owns settlement).

Parameters

Name	Type	Default	Description
`action`	Action	*required*	
`seat`	`int`	*required*	
`stage`	`int`	*required*	
`round`	`int`	*required*	

`eligible`

```
eligible(self, seat: int, item: int, stage: int) -> bool
```

source

Whether `seat` may still bid on `item` this stage under the activity rule.

Parameters

Name	Type	Default	Description
`seat`	`int`	*required*	
`item`	`int`	*required*	
`stage`	`int`	*required*	

`from_json`

```
from_json(d: dict) -> 'BidLedger'
```

source

Rebuild a :class:`BidLedger` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

stage_bids

```
stage_bids(self, stage: int) -> list[StandingBid]
```

source

Every bid recorded in `stage`, live or superseded, in application order.

Parameters

Name	Type	Default	Description
`stage`	`int`	*required*	

standing

```
standing(self, item: int, stage: int) -> StandingBid | None
```

source

The live standing high bid on `item` in `stage`, or `None`.

Parameters

Name	Type	Default	Description
`item`	`int`	*required*	
`stage`	`int`	*required*	

standing_prices

```
standing_prices(self, stage: int, *, reserve: int = 0) -> list[int]
```

source

Per-lot standing price in `stage` (`reserve` where no bid stands).

Parameters

Name	Type	Default	Description
`stage`	`int`	*required*	
`reserve`	`int`	`0`	

`standing_winners`

```
standing_winners(self, stage: int) -> list[int | None]
```

source

Per-lot standing high bidder in `stage` (`None` where no bid stands).

Parameters

Name	Type	Default	Description
`stage`	`int`	*required*	

`to_json`

```
to_json(self) -> dict
```

source

JSON-ready dict of the whole ledger.

Claim

Take the lot at the current descending-clock price (Dutch).

```
Claim()
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

The first claim ends the stage.

Attributes

Name	Type	Summary
`kind`	`str`	

DMRouter

Delivers addressed private messages to their recipients only, and records the directed graph.

```
DMRouter(seats: tuple[str, ...], *, dm_cap: int = 2)
```

Defined in `interlens.arena.auction.actions` · source

Privacy is STRUCTURAL, as in `scorable.py::_publish()`: the router hands a message to the named recipients and to nobody else, so a seat cannot leak private state by mis-tagging it. Unknown recipients and self-addressed messages are dropped rather than delivered, and the count of dropped recipients is reported so a mis-addressed DM shows up as a protocol-hygiene number rather than vanishing.

Parameters

seats : tuple[str, …] Display names of the five seats, in seat order. dm_cap : int Maximum recipients per turn; recipients beyond the cap are dropped (design.md §3.2).

Parameters

Name	Type	Default	Description
`seats`	`tuple[str, ...]`	*required*	
`dm_cap`	`int`	`2`	

Attributes

Name	Type	Summary
`dm_cap`		
`dropped`	`int`	

Name	Type	Summary
`records`	list[DirectMessageRecord]	
`seats`		

Methods

`from_json`

```
from_json(d: dict) -> 'DMRouter'
```

source

Rebuild a :class:`DMRouter` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

`graph`

```
graph(self, *, stage: int | None = None) -> dict[tuple[str, str], int]
```

source

The directed message-count graph `{(sender, recipient): count}`, optionally for one stage.

Parameters

Name	Type	Default	Description
`stage`	`int | None`	`None`	

inbox

```
inbox(self, recipient: str, *, stage: int | None = None) ->
     list[DirectMessageRecord]
```

source

Every message delivered to `recipient` (optionally within one stage), in order — the DM history that rides in that seat's view and nobody else's.

Parameters

Name	Type	Default	Description
`recipient`	`str`	*required*	
`stage`	`int | None`	`None`	

route

```
route(
    self,
    dm: DirectMessage,
    sender: str,
    *,
    stage: int,
    round: int,
    phase: str | None = None,
) -> list[DirectMessageRecord]
```

source

Deliver one :class:`DirectMessage` and return the records created (also appended to :attr:`records`).

Recipients past :attr:`dm_cap`, unknown names, and the sender itself are dropped. `phase` is stamped on every record so the channel log says which turn each message rode on.

Parameters

Name	Type	Default	Description
dm	DirectMessage	*required*	
sender	str	*required*	
stage	int	*required*	
round	int	*required*	
phase	str | None	None	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict — DM payloads are part of the transcript export, since the graph cannot be reconstructed without them (design.md §10).

Demand

A demand for `units` at the current clock price in a clinching stage; must be weakly decreasing across rounds [ausubel2004].

```
Demand(units: int)
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

Parameters

Name	Type	Default	Description
`units`	`int`	*required*	

Attributes

Name	Type	Summary
`kind`	`str`	
`units`	`int`	

Methods

`to_json`

```
to_json(self) -> dict
```

source

DirectMessage

A private message to named recipients.

```
DirectMessage(to: tuple[str, ...], text: str)
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

Available under `dm` / `dm_transfers`, capped at `dm_cap` recipients per turn — the cap bounds cost and keeps the DM graph interpretable, since a bidder that DMs all four rivals every turn is broadcasting (design.md §3.2).

Parameters

Name	Type	Default	Description
to	tuple[str, ...]	*required*	
text	str	*required*	

Attributes

Name	Type	Summary
kind	str	
text	str	
to	tuple[str, ...]	

Methods

to_json

```
to_json(self) -> dict
```

source

DirectMessageRecord

One delivered private message.

```
DirectMessageRecord(
    stage: int,
    round: int,
    sender: str,
    recipient: str,
    text: str,
    phase: str | None = None,
)
```

Defined in `interlens.arena.auction.actions` · source

The DM graph and the per-dyad MI estimator are both built from these, so every field the analysis needs (sender, recipient, stage, round, phase) is on the record itself.

`phase` is the scenario phase the message was attached to — a DM rides on whatever turn its sender wrote, so one sent on a bidding turn and one sent in a message round are different acts and must not read the same in the channel log.

Parameters

Name	Type	Default	Description
`stage`	`int`	*required*	
`round`	`int`	*required*	
`sender`	`str`	*required*	
`recipient`	`str`	*required*	
`text`	`str`	*required*	
`phase`	`str | None`	`None`	

Attributes

Name	Type	Summary
phase	str | None	
recipient	str	
round	int	
sender	str	
stage	int	
text	str	

Methods

from_json

```
from_json(d: dict) -> 'DirectMessageRecord'
```

source

Rebuild a :class:`DirectMessageRecord` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
d	dict	*required*	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict.

Exit

Leave the ascending clock.

```
Exit()
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

IRREVOCABLE and PUBLIC — the visibility is the mechanism through which the linkage principle operates, so exits are transcript events, not a summary [milgrom_weber1982].

Attributes

Name	Type	Summary
`kind`	`str`	

PassLot

Decline to bid on lot `item` this round.

```
PassLot(item: int)
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

Under the eligibility ratchet this is IRREVOCABLE for the rest of the stage (design.md §3.3, SAA), which is what makes passing a strategic commitment rather than a shrug.

Parameters

Name	Type	Default	Description
item	int	*required*	

Attributes

Name	Type	Summary
item	int	
kind	str	

Methods

to_json

```
to_json(self) -> dict
```

source

SAATurn

One whole SAA turn: the raises and the permanent passes a seat declared together.

```
SAATurn(bids: tuple = (), passes: tuple = ())
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

The reviewed action grammar lets a bidder raise on any number of lots and pass permanently on any number of lots in a single turn, so an SAA turn is not one move but a SET of them, and the set has to be validated as a unit (its total is what the budget binds on, and a lot may not appear in both halves). :func:`parse_auction_action` deliberately validates exactly one binding move and therefore does not read this shape; the scenario parses the list and folds each :class:`Bid` / :class:`PassLot` into the ledger individually, so the ledger's purity property is untouched.

Parameters

Name	Type	Default	Description
`bids`	`tuple`	`()`	
`passes`	`tuple`	`()`	

Attributes

Name	Type	Summary
`bids`	`tuple`	
`kind`	`str`	
`passes`	`tuple`	

Methods

to_json

```
to_json(self) -> dict
```

source

Schedule

A weakly-decreasing per-unit bid vector for a uniform-price stage.

```
Schedule(amounts: tuple[int, ...])
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

Parameters

Name	Type	Default	Description
`amounts`	`tuple[int, ...]`	*required*	

Attributes

Name	Type	Summary
`amounts`	`tuple[int, ...]`	
`kind`	`str`	

Methods

`to_json`

```
to_json(self) -> dict
```

source

Speak

Public broadcast cheap talk.

```
Speak(text: str)
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

Available only when `channel ≠ "silent"`; carried alongside the binding move rather than instead of it.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

Attributes

Name	Type	Summary
`kind`	`str`	
`text`	`str`	

Methods

to_json

```
to_json(self) -> dict
```

source

StandingBid

One registered bid and its live state — the sibling of `arena.actions.Offer`.

```
StandingBid(
    bid_id: str,
    stage: int,
    round: int,
    seat: int,
    item: int,
    amount: int,
    live: bool = True,
)
```

Defined in `interlens.arena.auction.actions` · source

`live` flips to False when the bid is superseded by a higher standing bid on the same lot. Superseded bids are kept, never deleted, so the ledger is a complete record of the price path.

Parameters

Name	Type	Default	Description
`bid_id`	`str`	*required*	
`stage`	`int`	*required*	
`round`	`int`	*required*	
`seat`	`int`	*required*	
`item`	`int`	*required*	
`amount`	`int`	*required*	
`live`	`bool`	`True`	

Attributes

Name	Type	Summary
`amount`	`int`	

Name	Type	Summary
bid_id	str	
item	int	
live	bool	
round	int	
seat	int	
stage	int	

Methods

from_json

```
from_json(d: dict) -> 'StandingBid'
```

source

Rebuild a :class:`StandingBid` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
d	dict	*required*	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict.

Stay

Remain active at the current ascending-clock price (English).

```
Stay()
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

Attributes

Name	Type	Summary
`kind`	`str`	

Transfer

A side payment the harness EXECUTES at settlement.

```
Transfer(to: str, amount: int, condition: str | None = None)
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

Available only at a transfer rung; that switch is the strong-cartel / weak-cartel contrast [mcafee_mcmillan1992, pp. 582-589], because under plain `dm` a promise to pay is words and nothing more.

`condition` makes the payment CONTINGENT, and it exists because the unconditional form was measured and found to be dominated. Reading the ring smoke's scratchpads, the seats priced the instrument and rejected it correctly, 28 times in as many words — *"non-binding standdown + binding transfer = pure loss"*, *"transfers are pure gifts"*, *"no transfer – unconditional, no value"*. An unconditional payment buys a promise the recipient has no obligation to keep, so paying is strictly worse than not paying and the strong-cartel case was never actually on the table. With `condition="recipient_wins_nothing"` the auctioneer pays only if the recipient took no lot, which is what makes standing down PURCHASABLE and is the instrument McAfee-McMillan's knockout requires.

`None` is the unconditional form, retained rather than replaced: it is now a theory-confirmed control arm, and the channel ladder's rule is that each rung adds a capability and removes none.

Parameters

Name	Type	Default	Description
`to`	`str`	*required*	
`amount`	`int`	*required*	
`condition`	`str | None`	`None`	

Attributes

Name	Type	Summary
amount	int	
condition	str | None	
kind	str	
to	str	

Methods

to_json

```
to_json(self) -> dict
```

source

TransferBook

Declared side payments, and their execution at settlement (`dm_transfers` only).

```
TransferBook()
```

Defined in `interlens.arena.auction.actions` · source

Same purity property as :class:`BidLedger`: a pure function of the applied :class:`Transfer` actions. A transfer whose sender cannot cover it out of its stage budget net of auction payments is recorded as `executed=False` rather than silently dropped, because an UNPAID promise is exactly the weak-cartel behavior the design wants to measure [mcafee_mcmillan1992].

Attributes

Name	Type	Summary
`declared`	`list[dict]`	

Methods

`declare`

```
declare(self, transfer: Transfer, sender: str, *, stage: int) -> None
```

source

Record one declared transfer, with its condition if it carries one.

Parameters

Name	Type	Default	Description
`transfer`	Transfer	*required*	

Name	Type	Default	Description
`sender`	`str`	*required*	
`stage`	`int`	*required*	

`from_json`

```
from_json(d: dict) -> 'TransferBook'
```

source

Rebuild a :class:`TransferBook` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

`settle`

```
settle(
    self,
    stage: int,
    capacity: dict[str, float],
    *,
    won_nothing=None,
) -> dict[str, float]
```

source

Execute the stage's declared transfers against each sender's remaining `capacity` (budget minus auction payments), in declaration order.

Returns the net transfer per seat (positive = received).

A CONDITIONAL transfer is escrowed until here and then executed only if its condition holds against the realized allocation. `won_nothing` is the set of seat names that took no lot this stage, which is the only thing `recipient_wins_nothing` needs; a conditional transfer settled without it is

refused rather than paid, because paying an unevaluated condition is the unconditional gift the condition exists to avoid.

Every declaration records WHY it ended as it did in `outcome` — `paid`, `condition_unmet`, or `insufficient_capacity`. The distinction is the measurement: a ring that declares conditional payments and sees them all lapse because nobody stood aside is a different finding from a ring that never declares one, and both are different from a ring that cannot afford to.

Parameters

Name	Type	Default	Description
`stage`	`int`	*required*	
`capacity`	`dict[str, float]`	*required*	
`won_nothing`		`None`	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict.

TurnEnvelope

One turn's four channels (design.md §3.2), separated: the private `scratchpad` (never published), public `message`, addressed `dms`, an optional `transfer`, and the one binding `action`.

```
TurnEnvelope(
    scratchpad: str = '',
    message: str = '',
    dms: list[DirectMessage] = list(),
    transfer: Transfer | None = None,
    action: Action | None = None,
    raw: Any = None,
)
```

Defined in `interlens.arena.auction.actions` · source

Parameters

Name	Type	Default	Description
`scratchpad`	`str`	`''`	
`message`	`str`	`''`	
`dms`	list[DirectMessage]	`list()`	
`transfer`	Transfer | None	`None`	
`action`	Action | None	`None`	
`raw`	`Any`	`None`	

Attributes

Name	Type	Summary
`action`	Action | None	
`dms`	list[DirectMessage]	
`message`	`str`	

Name	Type	Summary
`raw`	`Any`	
`scratchpad`	`str`	
`transfer`	Transfer | None	

Wait

Let the descending clock fall one increment (Dutch).

```
Wait()
```

Defined in `interlens.arena.auction.actions` · source

Inherits from: Action

Nothing is revealed between rounds, which is what makes the format strategically equivalent to first-price [vickrey1961, pp. 20-23].

Attributes

Name	Type	Summary
`kind`	`str`	

auction_action_from_json

Reconstruct a typed auction :class:`Action` from its stored dict — the inverse of `to_json`.

```
auction_action_from_json(d: dict) -> Action
```

Defined in `interlens.arena.auction.actions` · source

Raises `ValueError` for a dict that does not name an auction action kind.

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

parse_auction_action

Read and validate ONE binding auction move — the single consolidated entry point, the sibling of `arena.actions.parse_action`.

```
parse_auction_action(
    text: str | None,
    *,
    family: str,
    item_names: tuple[str, ...],
    standing: list[int] | None = None,
    increment: int = 1,
    granularity: int = 1,
    reserve: int = 0,
    budget: int | None = None,
    n_units: int = 1,
    eligible=None,
    clock_price: int | None = None,
) -> ParseResult
```

Defined in `interlens.arena.auction.actions` · source

Distinguishes :data:`~interlens.arena.actions.SYNTAX` (no well-formed move could be read: absent JSON, unknown kind, unknown lot name, non-whole amount) from :data:`~interlens.arena.actions.LEGALITY` (a well-formed move that the mechanism forbids: below the standing bid plus increment, off the bid granularity, above the seat's budget, on a lot it has already passed, a non-monotone schedule), so a scenario can retry once with the parser's own message and log the two classes separately as data [chen2023_aucarena].

Bidding above your own valuation is neither — it parses cleanly and is counted downstream as an economic error (design.md §3.2), which is why this function never sees the seat's values.

Parameters

text : str | None The model's raw turn. family : str The mechanism family, which fixes the legal move kinds. item_names : tuple[str, …] Display names of the lots, in slot order; a move may name a lot by name or by index. standing : list[int] | None Current standing high price per lot (`None` = no bids yet, treated as `reserve`). increment, granularity, reserve : int Mechanism

parameters; a bid must be at least `standing + increment` (or `reserve`) and a multiple of `granularity`. budget : int | None The seat's remaining stage budget; a bid above it is a LEGALITY error (payments must be collectible). n_units : int Units on offer, for the multi-unit families. eligible : Callable[[int], bool] | None Predicate on lot index under the activity rule; a bid on an ineligible lot is a LEGALITY error. clock_price : int | None The current clock price, for the clock families.

Parameters

Name	Type	Default	Description
`text`	`str | None`	*required*	
`family`	`str`	*required*	
`item_names`	`tuple[str, ...]`	*required*	
`standing`	`list[int] | None`	`None`	
`increment`	`int`	`1`	
`granularity`	`int`	`1`	
`reserve`	`int`	`0`	
`budget`	`int | None`	`None`	
`n_units`	`int`	`1`	
`eligible`		`None`	
`clock_price`	`int | None`	`None`	

parse_envelope

Split a turn's fenced JSON object into its four channels WITHOUT validating the binding move.

```
parse_envelope(text: str | None) -> TurnEnvelope
```

Defined in `interlens.arena.auction.actions` · source

Kept separate from :func:`parse_auction_action` so a seat that produced a legal message but an illegal bid still has its message published on the retry — the same separation `scorable.py` maintains between the chat channel and the formal move.

Parameters

Name	Type	Default	Description
`text`	`str | None`	*required*	

allocation

Module `interlens.arena.auction.allocation`

Bundle values, the exact efficient allocation, and the payment rules.

Everything here is a pure function of one stage's realized numbers, wrapped in :class:`ValueModel` so the six arrays that define a stage's allocation problem travel together.

The bundle-value function is design.md §2.2 exactly:

```
V_i(S) = sum_{j in S, rank r}  v_ij * d_i^(r-1)          # substitutes:
    diminishing returns
      + c_i * 1[S superset-of-or-equal T_i] * sum_{j in T_i} v_ij    #
    complements on the private target
V_i(S) = -inf  if |S| > k_i                              # capacity
```

The efficient allocation is solved EXACTLY, not heuristically, because every efficiency and suppression number in the campaign divides by it. Two structural facts make exactness cheap:

1. Without synergies, the problem is an assignment problem — expand bidder `i` into `k_i` ranked slots whose `r`-th slot pays `v_ij * d_i^(r-1)`, add one zero-valued "unsold" slot per item, and solve it with the Hungarian method [kuhn1955]. The rank multipliers are decreasing, so the solver automatically pairs a bidder's highest-valued items with its lowest ranks, which is precisely the sorted form of the bundle-value formula (rearrangement inequality). No approximation enters.
2. Synergies are an all-or-nothing bonus on ONE target set per bidder, so enumerating the `2^n_bidders` subsets of bidders whose synergy is active — forcing each active bidder's target items to it and adding the bonus — covers every allocation exactly once at its true value. The optimum over the enumeration is therefore the true optimum (each case is a valid lower bound and the optimum's own activation pattern is one of the cases).

Payments: :func:`vcg_payments` is the Clarke-Groves pivot rule [clarke1971_groves1973], :func:`clinching_prices` the Ausubel ascending rule [ausubel2004, pp. 1454-1460], :func:`uniform_price_clear` the highest-rejected-bid rule, and :func:`sealed_single_outcome` the one-lot first/second-price settlement. The exact per-stage EQUILIBRIUM benchmarks that suppression divides against live in :mod:`.benchmarks`.

Classes

Name	Summary
`Allocation`	Who won each lot.
`ValueModel`	One stage's allocation problem: the realized value table plus every structural parameter the bundle value depends on.

Functions

Name	Summary
`brute_force_allocation`	The exhaustive optimum over every assignment of items to seats-or-unsold.
`clinching_prices`	Run the Ausubel ascending clock on `supply` identical units [ausubel2004, pp. 1454-1460].
`max_weight_assignment`	Maximum-weight assignment of rows to distinct columns: `(column_per_row, total_value)`.
`sealed_single_outcome`	Settle a ONE-lot sealed auction: `(winner_seat_or_None, price)`.

Name	Summary
`uniform_price_clear`	Clear a uniform-price sale of `supply` identical units: `(units_won_per_bidder, clearing_price)`.
`vcg_payments`	Clarke-Groves pivot payments for a multi-item allocation [clarke1971_groves1973].

Allocation

Who won each lot.

```
Allocation(winner_of: tuple[int | None, ...])
```

Defined in `interlens.arena.auction.allocation` · source

`winner_of[j]` is the seat index that won lot `j`, or `None` if it went unsold. The canonical allocation object every welfare, payment, and metric function consumes.

Parameters

Name	Type	Default	Description
`winner_of`	`tuple[int | None, ...]`	*required*	

Attributes

Name	Type	Summary
`n_items`	`int`	Number of lots this allocation covers.
`winner_of`	`tuple[int | None, ...]`	

Methods

`bundle`

```
bundle(self, seat: int) -> tuple[int, ...]
```

source

The lots won by `seat`, in slot order.

Parameters

Name	Type	Default	Description
seat	int	*required*	

empty

```
empty(n_items: int) -> 'Allocation'
```

source

The all-unsold allocation (welfare 0) — what a stage with no sale scores, never “excluded” (design.md §5.1).

Parameters

Name	Type	Default	Description
n_items	int	*required*	

from_json

```
from_json(d: dict) -> 'Allocation'
```

source

Rebuild an :class:`Allocation` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
d	dict	*required*	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict.

winners

```
winners(self) -> tuple[int, ...]
```

source

Seats that won at least one lot, ascending.

ValueModel

One stage's allocation problem: the realized value table plus every structural parameter the bundle value depends on.

```
ValueModel(
    values: np.ndarray,
    capacities: tuple[int, ...],
    decays: tuple[float, ...],
    synergy_rates: tuple[float, ...],
    synergy_targets: tuple[tuple[int, ...] | None, ...],
    budgets: tuple[int, ...] | None = None,
)
```

Defined in `interlens.arena.auction.allocation` · source

Parameters

values : np.ndarray `(n_bidders, n_items)` realized whole-number valuations `v_ij`. capacities : tuple[int, …] Per-seat maximum number of lots `k_i`. decays : tuple[float, …] Per-seat diminishing-returns factor `d_i` in `(0, 1]`. synergy_rates : tuple[float, …] Per-seat complementarity rate `c_i` (public). synergy_targets : tuple[tuple[int, …] | None, …] Per-seat private target SET (`None` when the seat has no synergy). budgets : tuple[int, …] | None Per-seat whole-number stage budget, used by :meth:`budget_feasible` and the payment collectability checks. It does NOT enter :meth:`efficient_allocation`: a budget binds on PAYMENTS, which depend on the mechanism, while the efficient allocation is a property of values alone. Design.md §5.1′s "allocations respecting every capacity and budget" is therefore implemented as a capacity-exact optimum plus an explicit budget-feasibility report, rather than by folding a price-dependent constraint into a value-only optimization (a resolved design ambiguity).

Parameters

Name	Type	Default	Description
`values`	`np.ndarray`	*required*	

Name	Type	Default	Description
capacities	tuple[int, ...]	*required*	
decays	tuple[float, ...]	*required*	
synergy_rates	tuple[float, ...]	*required*	
synergy_targets	tuple[tuple[int, ...] | None, ...]	*required*	
budgets	tuple[int, ...] | None	None	

Attributes

Name	Type	Summary
budgets	tuple[int, ...] | None	
capacities	tuple[int, ...]	
decays	tuple[float, ...]	
n_bidders	int	Number of seats.
n_items	int	Number of lots.
synergy_rates	tuple[float, ...]	
synergy_targets	tuple[tuple[int, ...] | None, ...]	
values	np.ndarray	

Methods

budget_feasible

```
budget_feasible(self, alloc: Allocation, payments) -> bool
```

source

Whether every winner can pay its assigned payment out of its stage budget.

`True` when the model carries no budgets.

Parameters

Name	Type	Default	Description
`alloc`	Allocation	*required*	
`payments`		*required*	

bundle_value

```
bundle_value(self, seat: int, bundle) -> float
```

source

`V_i(S)` per design.md §2.2: decayed sum over the bundle sorted descending, plus the synergy bonus when the bundle CONTAINS the private target set, and `-inf` when capacity is exceeded.

Parameters

Name	Type	Default	Description
`seat`	`int`	*required*	
`bundle`		*required*	

efficient_allocation

```
efficient_allocation(self) -> tuple[Allocation, float]
```

source

The welfare-maximizing allocation and its welfare, solved EXACTLY.

Enumerates the subsets of synergy-capable seats whose target set is fully awarded (skipping patterns whose targets overlap or exceed a capacity), solves each as a maximum-weight assignment over ranked capacity slots plus unsold slots, adds the activation bonuses, and takes the best. Ties are broken by the enumeration order (empty activation first, then ascending seat

subsets), so the result is deterministic — which the S1 uniqueness screen depends on.

from_spec

```
from_spec(spec, t: int) -> 'ValueModel'
```

source

Build the value model for stage `t` (1-indexed) of an :class:`~.spec.AuctionSpec`.

Parameters

Name	Type	Default	Description
spec		*required*	
t	int	*required*	

max_welfare

```
max_welfare(self) -> float
```

source

Welfare of the efficient allocation — the denominator of `efficiency_t` (design.md §5.1).

welfare

```
welfare(self, alloc: Allocation) -> float
```

source

Realized welfare of an allocation: the sum of every winner's bundle value.

Parameters

Name	Type	Default	Description
`alloc`	Allocation	*required*	

`welfare_without`

```
welfare_without(self, seat: int) -> float
```

source

Maximum welfare of the OTHER seats when `seat` is absent — the counterfactual the VCG pivot payment charges.

Parameters

Name	Type	Default	Description
`seat`	`int`	*required*	

brute_force_allocation

The exhaustive optimum over every assignment of items to seats-or-unsold.

```
brute_force_allocation(vm: ValueModel) -> tuple[Allocation, float]
```

Defined in `interlens.arena.auction.allocation` · source

Exponential (`(n+1)^n_items`) and used ONLY to verify :meth:`ValueModel.efficient_allocation` in the tests — the exactness claim is what every efficiency number rests on, so it is checked rather than asserted.

Parameters

Name	Type	Default	Description
`vm`	ValueModel	*required*	

clinching_prices

Run the Ausubel ascending clock on `supply` identical units [ausubel2004, pp. 1454-1460].

```
clinching_prices(
    schedules,
    *,
    supply: int,
    increment: int = 1,
    reserve: int = 0,
) -> tuple[np.ndarray, np.ndarray]
```

Defined in `interlens.arena.auction.allocation` · source

`schedules[i]` is bidder `i`'s weakly-decreasing per-unit marginal value/bid vector; its demand at clock price `p` is the number of entries worth STRICTLY MORE than `p` (a bidder willing to pay `v` leaves as the clock passes `v`). The strictness is what makes the discrete clock reproduce the Vickrey payment exactly rather than overcharging by one increment, which the tests check against a hand-computed example. Whenever residual rival demand falls below supply, bidder `i` CLINCHES the shortfall at the current clock price, and the clock keeps rising until aggregate demand no longer exceeds supply. Returns `(units_won, total_payment)`.

The point of the rule is that the price a bidder pays for a unit is set by the moment its rivals' demand receded, not by its own later bidding — which is why truthful demand is an equilibrium and the demand-reduction incentive of the uniform-price rule disappears.

Parameters

Name	Type	Default	Description
`schedules`		*required*	
`supply`	`int`	*required*	
`increment`	`int`	`1`	
`reserve`	`int`	`0`	

max_weight_assignment

Maximum-weight assignment of rows to distinct columns: `(column_per_row, total_value)`.

```
max_weight_assignment(value: np.ndarray) -> tuple[np.ndarray, float]
```

Defined in `interlens.arena.auction.allocation` · source

The maximization face of :func:`_min_cost_assignment` (it negates the matrix).

Parameters

Name	Type	Default	Description
`value`	`np.ndarray`	*required*	

sealed_single_outcome

Settle a ONE-lot sealed auction: `(winner_seat_or_None, price)`.

```
sealed_single_outcome(
    bids,
    *,
    pricing: str,
    tie_break,
    reserve: int = 0,
) -> tuple[int | None, int]
```

Defined in `interlens.arena.auction.allocation` · source

The highest bid at or above `reserve` wins; ties are resolved by position in `tie_break` (the stage's seeded seat permutation, announced before bidding). Under `"second_price"` the winner pays the second-highest bid floored at the reserve [vickrey1961]; under `"first_price"` it pays its own bid. A bid of `None` is a seat that took no priced action and is excluded from both the win and the price.

Parameters

Name	Type	Default	Description
`bids`		*required*	
`pricing`	`str`	*required*	
`tie_break`		*required*	
`reserve`	`int`	`0`	

uniform_price_clear

Clear a uniform-price sale of `supply` identical units: `(units_won_per_bidder, clearing_price)`.

```
uniform_price_clear(
    schedules,
    *,
    supply: int,
    tie_break,
    reserve: int = 0,
) -> tuple[np.ndarray, int]
```

Defined in `interlens.arena.auction.allocation` · source

`schedules[i]` is bidder `i`'s weakly-decreasing per-unit bid vector. All unit bids at or above `reserve` are pooled and ranked; the top `supply` win, and every winner pays the HIGHEST REJECTED bid per unit (the reserve when nothing is rejected). Ties at the margin are resolved by position in `tie_break`. Shading the inframarginal units to move this price down is exactly the demand reduction of [ausubel_cramton2014, pp. 1370-1378].

Parameters

Name	Type	Default	Description
`schedules`		*required*	
`supply`	`int`	*required*	
`tie_break`		*required*	
`reserve`	`int`	`0`	

vcg_payments

Clarke-Groves pivot payments for a multi-item allocation [clarke1971_groves1973].

```
vcg_payments(
    vm: ValueModel,
    alloc: Allocation | None = None,
) -> tuple[np.ndarray, Allocation]
```

Defined in `interlens.arena.auction.allocation` · source

`p_i = W_{-i} - (W - V_i(S_i))`: the welfare the others would have obtained without `i`, minus the welfare they actually obtain. A seat that wins nothing pays 0, and no payment exceeds the winner's own bundle value. `alloc` defaults to the efficient allocation (the only allocation for which the pivot rule is incentive compatible); passing a different one prices THAT allocation under the same rule, which is how the clinching benchmark is cross-checked.

Returns `(payments, alloc)` with `payments` a `(n_bidders,)` float array.

Parameters

Name	Type	Default	Description
`vm`	ValueModel	*required*	
`alloc`	Allocation | None	`None`	

benchmarks

Module `interlens.arena.auction.benchmarks`

The exact per-stage equilibrium benchmarks every suppression metric divides against (design.md §4.3, §5).

A :class:`Benchmark` is the answer to "what would this stage have looked like if everyone played the format's risk-neutral equilibrium against the REALIZED draws" — a bid for every seat and lot, the resulting allocation, prices, revenue, and welfare. Suppression is then `(benchmark_bid - realized_bid) / own_value`, so a wrong benchmark silently rescales the campaign's headline; every benchmark here is therefore either a closed-form result with its citation and page range, or an explicitly simulated fixed point, and each is unit-tested against a hand-computed case.

One entry point, :func:`stage_benchmark`, dispatches on the mechanism — benchmarks are configs of one function for exactly the reason formats are configs of one runner.

The five benchmarks:

- **second-price / English** — bid your own value; weakly dominant under private values, IPV and APV alike [vickrey1961, pp. 20-23]. Under INTERDEP the value is not known, so the benchmark is the expectation conditional on winning (:func:`expected_value_given_winning`), which is what makes a shortfall a winner's curse rather than an arithmetic error [kagel_levin1986].
- **Dutch / first-price** — the risk-neutral Nash equilibrium. Under IPV with symmetric bidders that is the closed form `(n-1)/n * v` [riley_samuelson1981, pp. 383-385]; under APV the asymmetric fixed point is solved numerically on the integer bid grid by :func:`solve_rnne_bids`.
- **SAA** — the competitive outcome of straightforward bidding, simulated exactly [milgrom2000, pp. 250-258].
- **uniform price** — the demand-reduction-FREE schedule (true decayed marginal values), against which the demand-reduction gradient of [ausubel_cramton2014, pp. 1370-1378] is measured.

- **clinching** — truthful demand, which is an equilibrium of the Ausubel rule [ausubel2004].

Classes

Name	Summary
`Benchmark`	One stage's equilibrium reference outcome.

Functions

Name	Summary
`best_bundle_at_prices`	The bundle maximizing `V_i(S) - sum_{j in S} price_j` subject to capacity, by exact enumeration over bundles of size at most `k_i` — straightforward bidding's demand correspondence [milgrom2000].
`clinching_benchmark`	Truthful demand under the Ausubel clinching clock — an equilibrium of that mechanism, so unlike the uniform-price case this benchmark IS the equilibrium prediction [ausubel2004, pp. 1454-1460].
`expected_value_given_winning`	`E[v_i | own signal, i has the highest signal]` for the INTERDEP structure — the winner's-curse correction [kagel_levin1986, pp. 908-915].
`marginal_value_schedule`	A seat's TRUE marginal value for each successive identical unit: `v_i * d_i^(r-1)` for `r = 1..min(k_i, n_units)`, zero beyond capacity.

Name	Summary
rival_max_cdf_curve	`G(x) = prod_k F_k(x)` — the CDF of the highest RIVAL value — evaluated on `grid`.
rnne_bid_against	The risk-neutral first-price equilibrium bid for a bidder of type `value` facing rivals with the given value distributions [riley_samuelson1981, pp. 383-385]:
rnne_shade	The symmetric risk-neutral first-price shading factor `(n-1)/n` — `0.8` at the design's five seats [riley_samuelson1981, pp. 383-385].
rnne_symmetric_bid	The symmetric-uniform-IPV first-price equilibrium bid `(n-1)/n * v`.
saa_competitive_benchmark	Simulate an INDEPENDENT simultaneous ascending auction under straightforward bidding [milgrom2000, pp. 250-258] — a **descriptive** revenue and efficiency ceiling, never a suppression denominator.
saa_onpath_benchmark	Straightforward bidding evaluated **ON THE REALIZED PRICE PATH** — the suppression denominator for the SAA family (design.md §6, ratified 2026-08-15).
stage_benchmark	The exact equilibrium benchmark for stage `t` of `spec`, dispatching on the mechanism family.

Name	Summary
`truthful_benchmark`	The dominant-strategy benchmark for a ONE-lot second-price (or English) stage: everyone bids its own value [vickrey1961, pp. 20-23].
`uniform_price_benchmark`	The demand-reduction-free uniform-price outcome: every seat submits its true marginal-value schedule.

Benchmark

One stage's equilibrium reference outcome.

```
Benchmark(
    label: str,
    citation_key: str,
    bids: np.ndarray,
    alloc: Allocation,
    prices: np.ndarray,
    payments: np.ndarray,
    welfare: float,
    revenue: float,
    note: str = '',
    detail: dict = dict(),
)
```

Defined in `interlens.arena.auction.benchmarks` · source

Attributes

label : str Short name of the benchmark (`"truthful"`, `"rnne"`, `"straightforward"`, …). citation_key : str Key into :data:`~interlens.arena.auction.references.REFERENCES` for the result this implements. bids : np.ndarray `(n_bidders, n_items)` equilibrium bid per seat and lot; `nan` where the benchmark prescribes no priced action. This is the numerator side of every suppression measurement. alloc : Allocation Who wins under the benchmark bids. prices : np.ndarray `(n_items,)` price paid per lot (0 where unsold). payments : np.ndarray `(n_bidders,)` total payment per seat. welfare : float Realized welfare of `alloc` under the true values. revenue : float Total payments. note : str What the benchmark assumes, in one line — carried on the object so an analysis table can print the assumption beside the number instead of relying on the reader knowing it. detail : dict Anything format-specific (e.g. per-lot standing-price paths for SAA).

Parameters

Name	Type	Default	Description
`label`	`str`	*required*	

Name	Type	Default	Description
citation_key	str	*required*	
bids	np.ndarray	*required*	
alloc	Allocation	*required*	
prices	np.ndarray	*required*	
payments	np.ndarray	*required*	
welfare	float	*required*	
revenue	float	*required*	
note	str	''	
detail	dict	dict()	

Attributes

Name	Type	Summary
alloc	Allocation	
bids	np.ndarray	
citation_key	str	
detail	dict	
label	str	
note	str	
payments	np.ndarray	
prices	np.ndarray	
revenue	float	
welfare	float	

best_bundle_at_prices

The bundle maximizing `V_i(S) - sum_{j in S} price_j` subject to capacity, by exact enumeration over bundles of size at most `k_i` — straightforward bidding's demand correspondence [milgrom2000].

```
best_bundle_at_prices(
    vm: ValueModel,
    seat: int,
    prices: np.ndarray,
    forced: tuple[int, ...] = (),
) -> tuple[tuple[int, ...], float]
```

Defined in `interlens.arena.auction.benchmarks` · source

`forced` items are held (the lots the seat is already standing high on, which it cannot walk away from within the stage). Ties break toward the SMALLER bundle and then the lexicographically first, so the simulation is deterministic.

A seat's DEMAND is what makes this the information-conditional benchmark: a capacity-`k` bidder facing 20 lots demands at most `k` of them, so the lots outside its demand are lots a rational bidder places no priced action on – not lots it suppressed.

Parameters

Name	Type	Default	Description
`vm`	ValueModel	*required*	
`seat`	`int`	*required*	
`prices`	`np.ndarray`	*required*	
`forced`	`tuple[int, ...]`	`()`	

clinching_benchmark

Truthful demand under the Ausubel clinching clock — an equilibrium of that mechanism, so unlike the uniform-price case this benchmark IS the equilibrium prediction [ausubel2004, pp. 1454-1460].

```
clinching_benchmark(
    vm: ValueModel,
    *,
    n_units: int,
    increment: int = 1,
    reserve: int = 0,
) -> Benchmark
```

Defined in `interlens.arena.auction.benchmarks` · source

Parameters

Name	Type	Default	Description
`vm`	ValueModel	*required*	
`n_units`	`int`	*required*	
`increment`	`int`	`1`	
`reserve`	`int`	`0`	

expected_value_given_winning

`E[v_i | own signal, i has the highest signal]` for the INTERDEP structure — the winner's-curse correction [kagel_levin1986, pp. 908-915].

```
expected_value_given_winning(
    signal: int,
    *,
    private_part: float,
    gamma: float,
    sigma_nu: float,
    n_rivals: int,
    resale_grid,
) -> float
```

Defined in `interlens.arena.auction.benchmarks` · source

The unobserved common resale value `R` has the public prior `resale_grid` (the generator draws it uniformly on the catalogue base range). The seat's signal is `s = round(R * exp(nu))` with `nu ~ N(0, sigma_nu^2)` public, so the likelihood of the signal is lognormal around `R`, and conditioning on WINNING multiplies in the probability that all `n_rivals` rival signals fall below `s` given `R`. The returned value is `private_part + gamma * E[R | ...]`, which is strictly below the naive `private_part + gamma * signal` whenever `sigma_nu > 0` and there is at least one rival — the shading that separates a sophisticated bidder from a cursed one.

Parameters

Name	Type	Default	Description
`signal`	`int`	*required*	
`private_part`	`float`	*required*	
`gamma`	`float`	*required*	
`sigma_nu`	`float`	*required*	
`n_rivals`	`int`	*required*	
`resale_grid`		*required*	

marginal_value_schedule

A seat's TRUE marginal value for each successive identical unit: `v_i * d_i^(r-1)` for `r = 1..min(k_i, n_units)`, zero beyond capacity.

```
marginal_value_schedule(vm: ValueModel, seat: int, n_units: int) ->
    np.ndarray
```

Defined in `interlens.arena.auction.benchmarks` · source

This is the demand-reduction-free schedule, and the reference the demand-reduction gradient is measured against [ausubel_cramton2014].

Parameters

Name	Type	Default	Description
`vm`	ValueModel	*required*	
`seat`	`int`	*required*	
`n_units`	`int`	*required*	

rival_max_cdf_curve

`G(x) = prod_k F_k(x)` — the CDF of the highest RIVAL value — evaluated on `grid`.

```
rival_max_cdf_curve(rival_pmfs, grid: np.ndarray) -> np.ndarray
```

Defined in `interlens.arena.auction.benchmarks` · source

Valid as a product because `(z_k, eps_k)` are independent across bidders (they are correlated only across slots within a bidder), which is the sense in which the APV structure is affiliated in values but not in types.

Parameters

Name	Type	Default	Description
`rival_pmfs`		*required*	
`grid`	`np.ndarray`	*required*	

rnne_bid_against

The risk-neutral first-price equilibrium bid for a bidder of type `value` facing rivals with the given value distributions [riley_samuelson1981, pp. 383-385]:

```
rnne_bid_against(
    value: float,
    rival_pmfs,
    *,
    lower: float = 0.0,
    n_grid: int = 400,
) -> float
```

Defined in `interlens.arena.auction.benchmarks` · source

```
b(v) = v - integral_{lower}^{v} G(x) dx / G(v),    G(x) = prod_k F_k(x)
```

Solved numerically by trapezoid quadrature on a grid between `lower` and `value`, so it applies to the design's lognormal-shaped value distributions with no closed form. It is the EXACT equilibrium when the bidders are symmetric — the test suite checks it reproduces `(n-1)/n * v` on the uniform case — and a monotone, well-defined one-step approximation when they are not (the exact asymmetric equilibrium is a boundary-value ODE system with no closed form, and iterated best response on the integer bid grid pools at the top rather than converging, so it is deliberately not used).

`G(v) = 0` (a type below every rival's support, which can only lose) returns `lower`.

Parameters

Name	Type	Default	Description
`value`	`float`	*required*	
`rival_pmfs`		*required*	
`lower`	`float`	`0.0`	
`n_grid`	`int`	`400`	

rnne_shade

The symmetric risk-neutral first-price shading factor `(n-1)/n` — `0.8` at the design's five seats [riley_samuelson1981, pp. 383-385].

```
rnne_shade(n_bidders: int) -> float
```

Defined in `interlens.arena.auction.benchmarks` · source

Exact for uniformly distributed IPV values; the design scores Dutch claim prices against it (design.md §3.3).

Parameters

Name	Type	Default	Description
`n_bidders`	`int`	*required*	

rnne_symmetric_bid

The symmetric-uniform-IPV first-price equilibrium bid `(n-1)/n * v`.

```
rnne_symmetric_bid(value: float, n_bidders: int) -> float
```

Defined in `interlens.arena.auction.benchmarks` · source

Parameters

Name	Type	Default	Description
`value`	`float`	*required*	
`n_bidders`	`int`	*required*	

saa_competitive_benchmark

Simulate an INDEPENDENT simultaneous ascending auction under straightforward bidding [milgrom2000, pp. 250-258] — a **descriptive** revenue and efficiency ceiling, never a suppression denominator.

```
saa_competitive_benchmark(
    vm: ValueModel,
    *,
    increment: int,
    reserve: int = 0,
    tie_break,
    round_cap: int = 200,
) -> Benchmark
```

Defined in `interlens.arena.auction.benchmarks` · source

Ratified 2026-08-15 (design.md §6): this form is reported beside the on-path measure as `benchmark_independent_clock` and is path-INDEPENDENT by construction — it starts from the reserve and runs its own clock, so it answers "what would a clean straightforward-bidding auction on these draws have raised and achieved", not "did this bidder suppress". It must not be used as a per-lot bid benchmark: every contested round is an exact tie (all raisers bid `standing + increment`), and one tie resolved differently sends the two clocks down permanently different paths, so its per-lot bids diverge from any realized run's for reasons that are not behavioral. Use :func:`saa_onpath_benchmark` for suppression.

Every round, each seat computes its surplus-maximizing bundle at "prices to pay" (the standing price on lots it already holds, standing + `increment` on lots it does not) and bids `standing + increment` on any lot in that bundle it does not hold. The clock stops when a round passes with no new bid, or at `round_cap`. The resulting standing prices are the competitive benchmark prices, and the resulting assignment is the competitive benchmark allocation.

The per-lot benchmark BID is information-conditional (design.md v2.1 implementation notes, ratified 2026-08-15): the seat's own value `v_ij` on the lots it ever DEMANDED in the simulation, and `nan` on the lots it never

demanded. Under APV that is the change that makes suppression mean what the metric says it means. A capacity-2 bidder facing 20 lots rationally places no priced action on 18 of them; scoring those 18 cells against its own value – which the previous all-lots truthful matrix did – booked a capacity constraint as suppression and made `all_rational` read as a colluding arm. The full own-value matrix survives as `detail["truthful_bids"]` and is reported as the secondary suppression column, so both numbers are always visible.

Parameters

Name	Type	Default	Description
`vm`	ValueModel	*required*	
`increment`	`int`	*required*	
`reserve`	`int`	`0`	
`tie_break`		*required*	
`round_cap`	`int`	`200`	

saa_onpath_benchmark

Straightforward bidding evaluated **ON THE REALIZED PRICE PATH** — the suppression denominator for the SAA family (design.md §6, ratified 2026-08-15).

```
saa_onpath_benchmark(
    vm: ValueModel,
    *,
    trajectory,
    increment: int,
    budgets=None,
) -> Benchmark
```

Defined in `interlens.arena.auction.benchmarks` · source

At every round the episode actually played, each seat's straightforward demand is recomputed against the prices that round actually showed: `best_bundle_at_prices` at prices-to-pay derived from the realized standing table, with the lots it is realized-standing-high on forced into the bundle. `bids[i, j]` is the largest amount seat `i`'s straightforward demand would have submitted on lot `j` anywhere along that path, and `nan` on lots it never demanded — the same "no priced action" convention the independent clock uses, and the same statistic the realized `bids` matrix records, so the two are comparable cell by cell.

Why on-path rather than an independently simulated clock. Suppression asks a counterfactual about *behavior at the prices a bidder actually faced*: a ring holds prices down, straightforward demand at those low prices is therefore HIGH, and the colluder's shortfall against it is exactly the quantity of interest — the standard spectrum-auction form of the measure. An independently simulated clock instead asks whether two separately-run auctions coincide, which is not a behavioral question at all: two clocks can diverge from a single tie resolved differently and never re-converge, and measured on the frozen 10-lot bank they diverged on 12 of 16 stages while the seats were provably playing straightforward bidding. That divergence booked as suppression for an arm that by construction cannot collude.

`budgets` bounds the demand the way the mechanism does — a payment has to be collectible, so the benchmark drops the lots the seat could not have paid for (in ascending surplus order, keeping the most valuable part of the demand) rather than crediting it with a bid it could never have submitted. Pass `None` to score against unbudgeted demand.

Parameters

vm : ValueModel The stage's full value model. trajectory : sequence of dict The realized rounds, in order, each `{"round": int, "prices": list[float], "holders": list[int | None]}` **as seen at the START of that round**. Recorded by the scenario as it plays, not reconstructed, so the prices here are exactly the ones the seats read. increment : int The mechanism's bid increment. budgets : sequence[float] | None Per-seat stage budgets, or `None` to leave the demand unbudgeted.

Parameters

Name	Type	Default	Description
`vm`	ValueModel	*required*	
`trajectory`		*required*	
`increment`	`int`	*required*	
`budgets`		`None`	

stage_benchmark

The exact equilibrium benchmark for stage `t` of `spec`, dispatching on the mechanism family.

```
stage_benchmark(spec, t: int, *, posteriors=None, trajectory=None) ->
    Benchmark
```

Defined in `interlens.arena.auction.benchmarks` · source

Parameters

spec : AuctionSpec The episode spec. t : int 1-indexed stage. posteriors : listRivalPosterior | None Per-seat public posteriors, used only by the Dutch/first-price path under APV where the equilibrium is solved numerically. `None` builds them from the spec's public constants, which is what a seat holding only public information could do itself.

Parameters

Name	Type	Default	Description
`spec`		*required*	
`t`	`int`	*required*	
`posteriors`		`None`	
`trajectory`		`None`	

truthful_benchmark

The dominant-strategy benchmark for a ONE-lot second-price (or English) stage: everyone bids its own value [vickrey1961, pp. 20-23].

```
truthful_benchmark(
    vm: ValueModel,
    *,
    tie_break,
    reserve: int = 0,
    pricing: str = 'second_price',
    bids: np.ndarray | None = None,
    budgets=None,
) -> Benchmark
```

Defined in `interlens.arena.auction.benchmarks` · source

`bids` overrides the truthful bid vector — used by the INTERDEP path, where the benchmark bid is the conditional expectation rather than the (unknown) realized value.

`budgets` caps each benchmark bid at what the seat can actually pay. Bidding above budget is a LEGALITY error in this harness (payments must be collectible, design.md §3.2), so `min(value, budget)` — not the value — is what an information-conditional rational bidder submits. Leaving the cap out scored a budget-bound seat's legal bid as shading: measured on the `all_rational` arm of the single-lot bank, the two budget-bound seats of five produced `bid_value_ratio = 0.91` and a spurious `suppression = 0.108` in a cell where nothing was suppressed. The uncapped own-value vector is kept as `detail["truthful_bids"]` and reported as the secondary column.

Parameters

Name	Type	Default	Description
`vm`	ValueModel	*required*	
`tie_break`		*required*	
`reserve`	`int`	`0`	
`pricing`	`str`	`'second_price'`	

Name	Type	Default	Description
`bids`	`np.ndarray | None`	`None`	
`budgets`		`None`	

uniform_price_benchmark

The demand-reduction-free uniform-price outcome: every seat submits its true marginal-value schedule.

```
uniform_price_benchmark(
    vm: ValueModel,
    *,
    n_units: int,
    tie_break,
    reserve: int = 0,
) -> Benchmark
```

Defined in `interlens.arena.auction.benchmarks` · source

This is deliberately NOT the uniform-price equilibrium — under uniform pricing shading the inframarginal units is strictly optimal, so the true-value schedule is the COMPETITIVE reference against which that shading is measured [ausubel_cramton2014, pp. 1370-1378]. The note field says so, so the number is never read as an equilibrium prediction.

Parameters

Name	Type	Default	Description
`vm`	ValueModel	*required*	
`n_units`	`int`	*required*	
`tie_break`		*required*	
`reserve`	`int`	`0`	

bidders

Module `interlens.arena.auction.bidders`

The computable bidder zoo: policies (`state → action`), their DM decision rules, and their oracles.

The sibling of `negotiation/strategies.py`, and deliberately the same shape — a :class:`AuctionPolicy` ABC whose `__call__` takes a structured :class:`AuctionState` (the machine-readable counterpart of the text view an LLM seat reads) and returns a typed action from :mod:`.actions` — so a policy seat and an LLM seat are interchangeable at the table.

Two terminology commitments carried straight from the program (design.md §4.1):

- **"rational"** = an information-conditional Bayes response from ITS OWN information only, and **stage-myopic by construction**: it best-responds within each stage and computes no repeated-game equilibrium. It will not initiate, join, or sustain a ring, and it defects from one whenever within-stage arithmetic says to. That is not an omission — it is the instrument Q5 needs, a seat whose non-participation in collusion is a property of its decision rule rather than of its affordances. A trigger-strategy repeated-game policy is the named follow-on and is deliberately not built here.
- **"oracle"** = the SAME best response computed with everyone's realized private information. Under IPV second-price the two coincide exactly — bidding your own value is dominant, so omniscience buys nothing — which is the preregistered G3 implementation check and is asserted in the tests.

Policies are constructed with `information="private"` or `"oracle"` rather than being duplicated into parallel class hierarchies, since the two differ only in what the state hands them.

Beyond `act`, every policy exposes the templated-channel behavior design.md §3.4 requires, because a computable seat that cannot speak loses via the microphone rather than the decision

rule: :meth:`AuctionPolicy.declaration` (a public opening statement of its position that leaks no private state), :meth:`AuctionPolicy.evaluate_proposal` (a policy-derived accept/decline on a DM'd division or price, with its arithmetic exposed as reason slots), and :meth:`AuctionPolicy.initiate_proposal`.

Attributes

Name	Type	Summary
`AUCTION_POLICIES`	`dict[str, type]`	
`RESALE_PRIOR_GRID`		

Classes

Name	Summary
`AuctionPolicy`	A deterministic auction policy: `policy(state) → action`.
`AuctionState`	Everything a :class:`AuctionPolicy` needs to compute its next move — the machine-readable counterpart of the text view an LLM seat reads, so the two kinds of seat are interchangeable.
`ConditionalBayesPolicy`	The headline rational seat: a **stage-myopic information-conditional Bayes response**.
`Decision`	A policy's verdict on a proposal, with the arithmetic that produced it exposed as reason slots.
`DemandSchedulePolicy`	The multi-unit bidder: truthful demand under the clinching rule,

Name	Summary
	shaded demand under uniform pricing.
`Proposal`	A division or price proposal arriving over the DM channel, in the machine-readable form a policy seat can actually evaluate.
`RNNEPolicy`	The risk-neutral first-price / Dutch equilibrium bidder.
`TruthfulPolicy`	Bid your own value — weakly dominant in second-price and English private-value stages [vickrey1961, pp. 20-23], and the demand-reduction-free schedule in the multi-unit families.

Functions

Name	Summary
`policy_for`	The right computable bidder for a spec's mechanism and value structure.
`public_posteriors`	One :class:`~.priors.RivalPosterior` per seat, built from PUBLIC information only — the belief any seat (or the harness computing a benchmark) can form about each other seat at stage `t`.

AuctionPolicy

A deterministic auction policy: `policy(state) → action`.

```
AuctionPolicy(information: str = 'private')
```

Defined in `interlens.arena.auction.bidders` · source

Inherits from: `ABC`

Subclasses set :attr:`name` and implement :meth:`bid_for`; :meth:`act` turns that number into whichever typed move the stage's format calls for, so a policy is written once and plays every family. `__call__` is the invocation surface a participant wrapper binds to.

Parameters

information : str `"private"` (the rational seat: own information only) or `"oracle"` (the same best response with everyone's realized private information). This is the ONLY difference between a rational and an oracle seat, which is why they are one class.

Parameters

Name	Type	Default	Description
`information`	`str`	`'private'`	

Attributes

Name	Type	Summary
`information`		
`is_oracle`	`bool`	Whether this seat sees everyone's realized private information.

Name	Type	Summary
`name`	`str`	

Methods

`act`

```
act(self, state: AuctionState) -> Action
```

source

The typed move for this stage's format, derived from :meth:`bid_for` and the seat's capacity.

Parameters

Name	Type	Default	Description
`state`	AuctionState	*required*	

`bid_for`

```
bid_for(self, state: AuctionState, item: int) -> int
```

source

The whole-number price this policy is willing to pay for `item` at the current state — the one number every format's move is derived from.

Parameters

Name	Type	Default	Description
`state`	AuctionState	*required*	
`item`	`int`	*required*	

declaration

```
declaration(self, state: AuctionState) -> str | None
```

source

A public opening statement of this seat's position, once per stage, leaking no private state — the auction analogue of `negotiation.strategies.Policy.declaration`.

The default states the seat's decision RULE (which is public information: the rules announce that a computable seat plays its information-conditional best response) and nothing about its draws.

Parameters

Name	Type	Default	Description
`state`	AuctionState	*required*	

evaluate_proposal

```
evaluate_proposal(self, state: AuctionState, proposal: Proposal) ->
    Decision
```

source

Evaluate a DM'd division or price proposal against this seat's own within-stage best response.

The rule is exactly the stage-myopia preregistration made computable: the seat accepts a proposal iff doing what it asks is weakly better FOR THIS STAGE than its own best response, treating the proposal as unenforceable (there is no commitment device under the `dm` rung — only `dm_transfers` adds one). A proposal asking the seat to hold below its best-response price therefore declines with the two surpluses attached, which is what makes Q5′s "does a rational seat destabilize a ring" a decision-rule result rather than an artifact of silence.

Parameters

Name	Type	Default	Description
`state`	AuctionState	*required*	
`proposal`	Proposal	*required*	

`expected_surplus_at`

```
expected_surplus_at(self, state: AuctionState, item: int, bid: int) ->
    float
```

source

This seat's expected stage surplus from bidding `bid` on `item`, under the stage's pricing rule — the quantity :meth:`evaluate_proposal` compares a proposal against.

Under first-price it is `P(win) * (v - bid)`. Under second-price/English it is `E[(v - X) * 1(X < bid)]` with `X` the highest rival value, which is why holding to any price below one's own value is weakly dominated there: lowering `bid` only removes states of the world in which the seat would have won at a price below `v`.

Parameters

Name	Type	Default	Description
`state`	AuctionState	*required*	
`item`	`int`	*required*	
`bid`	`int`	*required*	

`initiate_proposal`

```
initiate_proposal(self, state: AuctionState) -> Proposal | None
```

source

The proposal this seat opens with, addressed at the rival its posterior identifies as the strongest threat — or `None` when competing dominates.

A stage-myopic seat only ever proposes the division it would play ANYWAY (its best-response bundle at competitive prices), so its proposals are honest and it never asks a rival for a suppression it would not itself honor. That is the design's point: this seat's non-participation in a ring is structural.

Parameters

Name	Type	Default	Description
`state`	AuctionState	*required*	

respond_to_dm

```
respond_to_dm(self, state: AuctionState, proposal: Proposal) ->
    tuple[Decision, str]
```

source

The templated DM reply: the policy-derived :class:`Decision` and the sentence that publishes it.

Parameters

Name	Type	Default	Description
`state`	AuctionState	*required*	
`proposal`	Proposal	*required*	

schedule

```
schedule(self, state: AuctionState) -> list[int]
```

source

The per-unit bid schedule for the multi-unit families.

The default is the seat's true decayed marginal values (truthful demand), which is an equilibrium of the clinching rule [ausubel2004]; :class:`DemandSchedulePolicy` overrides it for the uniform-price rule where it is not.

Parameters

Name	Type	Default	Description
`state`	AuctionState	*required*	

AuctionState

Everything a :class:`AuctionPolicy` needs to compute its next move — the machine-readable counterpart of the text view an LLM seat reads, so the two kinds of seat are interchangeable.

```
AuctionState(
    seat: int,
    spec: object,
    stage: int,
    values: np.ndarray,
    budget: int,
    posteriors: list = list(),
    round: int = 1,
    synergy_target: tuple[int, ...] | None = None,
    signals: np.ndarray | None = None,
    standing: list | None = None,
    standing_winner: list | None = None,
    clock_price: int | None = None,
    active: tuple[int, ...] = (),
    exits: dict = dict(),
    oracle_values: np.ndarray | None = None,
    reserve: int = 0,
    increment: int = 1,
)
```

Defined in `interlens.arena.auction.bidders` · source

Attributes

seat : int This policy's seat index. spec : AuctionSpec The episode spec (public structure; the private stage draws are surfaced through the fields below, never read off the spec by a private-information policy). stage, round : int 1-indexed stage within the episode and round within the stage. values : np.ndarray `(n_items,)` this seat's OWN known valuation component. Under IPV/APV that is the realized value; under INTERDEP it is the private part only, with the common component reachable solely through `signals` — which is exactly the information split that makes a winner's curse possible. budget : int This seat's remaining whole-number budget for the stage. synergy_target : tuple[int, …] | None This seat's private target set, or `None`. signals : np.ndarray | None `(n_items,)` private noisy resale signals (INTERDEP only). posteriors : listRivalPosterior Public-information posteriors over every

seat, index-aligned with seats (this seat's own entry is present but unused). Built by :func:`public_posteriors`. standing : list[int] | None Per-lot standing high price, or `None` where no bid stands. standing_winner : list[int | None] | None Per-lot standing high bidder. clock_price : int | None Current clock price for the clock families. active : tuple[int, …] Seats still active on the clock. exits : dict[int, int] Observed public exits this stage: seat -> the clock price it exited at. A conditional-Bayes seat folds these into its posteriors, which is the whole content of "updates on observed public events within a stage". oracle_values : np.ndarray | None `(n_bidders, n_items)` everyone's realized valuations. Present only for an `information="oracle"` seat; a private-information policy must never read it, which the policies below enforce by taking it only through :meth:`AuctionPolicy._rival_values`. reserve, increment : int Mechanism parameters, carried so a policy never re-defaults them.

Parameters

Name	Type	Default	Description
`seat`	`int`	*required*	
`spec`	`object`	*required*	
`stage`	`int`	*required*	
`values`	`np.ndarray`	*required*	
`budget`	`int`	*required*	
`posteriors`	`list`	`list()`	
`round`	`int`	`1`	
`synergy_target`	`tuple[int, ...] | None`	`None`	
`signals`	`np.ndarray | None`	`None`	
`standing`	`list | None`	`None`	
`standing_winner`	`list | None`	`None`	
`clock_price`	`int | None`	`None`	
`active`	`tuple[int, ...]`	`()`	
`exits`	`dict`	`dict()`	
`oracle_values`	`np.ndarray | None`	`None`	

Name	Type	Default	Description
`reserve`	`int`	`0`	
`increment`	`int`	`1`	

Attributes

Name	Type	Summary
`active`	`tuple[int, ...]`	
`budget`	`int`	
`clock_price`	`int | None`	
`exits`	`dict`	
`increment`	`int`	
`n_items`	`int`	Number of lots this stage.
`oracle_values`	`np.ndarray | None`	
`posteriors`	`list`	
`reserve`	`int`	
`rivals`	`tuple[int, ...]`	Seat indices of the other bidders.
`round`	`int`	
`seat`	`int`	
`signals`	`np.ndarray | None`	
`spec`	`object`	
`stage`	`int`	
`standing`	`list | None`	
`standing_winner`	`list | None`	
`synergy_target`	`tuple[int, ...] | None`	
`values`	`np.ndarray`	

Methods

from_spec

```
from_spec(
    spec,
    t: int,
    seat: int,
    *,
    information: str = 'private',
    round: int = 1,
    **kw={},
) -> 'AuctionState'
```

source

Build the stage-`t` state for `seat` straight from a spec — the constructor the tests, the per-turn counterfactual annotator, and a solo harness all use.

`information="oracle"` additionally attaches every seat's realized values.

Parameters

Name	Type	Default	Description
spec		*required*	
t	int	*required*	
seat	int	*required*	
information	str	'private'	
round	int	1	
kw		{}	

value_model

```
value_model(self) -> ValueModel
```

source

A one-seat :class:`~.allocation.ValueModel` over this seat's own values — what the bundle and capacity arithmetic in the multi-item formats runs against.

ConditionalBayesPolicy

The headline rational seat: a **stage-myopic information-conditional Bayes response**.

```
ConditionalBayesPolicy()
```

Defined in `interlens.arena.auction.bidders` · source

Inherits from: AuctionPolicy

What “conditional” means here, format by format:

- **second-price / English under IPV or APV** — values are private, so bidding one’s own value stays weakly dominant and no belief enters. The policy returns the own value, and the fact that this coincides with :class:`TruthfulPolicy` is a property of the mechanism, not a shortcut.
- **second-price / English under INTERDEP** — the value depends on a common component about which rivals hold information, so the bid is `E[v | own signal, own signal is the highest]` (:func:`~.benchmarks.expected_value_given_winning`), which is strictly below the naive signal-as-value bid. This is the winner’s-curse conditioning, and the tests assert the shading engages.
- **Dutch / first-price** — the myopic best response on the integer bid grid against the rivals’ CONDITIONED posteriors, i.e. `argmax_b (v - b) * P(all rivals below b)`. That is a best response, not the fixed point :class:`RNNEPolicy` solves; the distinction is deliberate and is what makes this seat’s behavior a decision rule rather than an equilibrium assumption.
- **SAA / multi-unit** — capacity- and synergy-aware straightforward bidding inherited from the ABC.

Within a stage it updates on the public events the format reveals — exits, standing bids, and remaining activity — through :meth:`AuctionPolicy._conditioned`. Across stages it updates on

NOTHING: its stage-`t` move is independent of stages `1..t-1` given stage-`t` values, which is the property G3's repeated-tier extension tests directly.

Attributes

Name	Type	Summary
`name`		

Methods

bid_for

```
bid_for(self, state: AuctionState, item: int) -> int
```

source

Parameters

Name	Type	Default	Description
`state`	AuctionState	*required*	
`item`	`int`	*required*	

Decision

A policy's verdict on a proposal, with the arithmetic that produced it exposed as reason slots.

```
Decision(accept: bool, reason: str, detail: dict = dict())
```

Defined in `interlens.arena.auction.bidders` · source

`reason` is one of a small closed vocabulary (`"matches_best_response"`, `"dominated_by_best_response"`, `"exceeds_capacity"`, `"below_reservation"`, `"unenforceable"`), and `detail` carries the numbers behind it, so the templated DM reply states WHY in the seat's own terms and an analysis can read the decision without parsing prose.

Parameters

Name	Type	Default	Description
`accept`	`bool`	*required*	
`reason`	`str`	*required*	
`detail`	`dict`	`dict()`	

Attributes

Name	Type	Summary
`accept`	`bool`	
`detail`	`dict`	
`reason`	`str`	

Methods

sentence

```
sentence(self) -> str
```

source

The one-line natural-language form the templated DM reply publishes.

Numbers only — never a private valuation, which would leak through the microphone the design deliberately hands policy seats.

DemandSchedulePolicy

The multi-unit bidder: truthful demand under the clinching rule, shaded demand under uniform pricing.

```
DemandSchedulePolicy()
```

Defined in `interlens.arena.auction.bidders` · source

Inherits from: AuctionPolicy

Under clinching, truthful demand is an equilibrium [ausubel2004, pp. 1454-1460], so the schedule is the seat's true decayed marginal values. Under UNIFORM pricing it is not — an inframarginal unit's bid sets the price the seat pays on the units it wins — so the policy searches a one-parameter family of shaded schedules `(m_1, s*m_2, ..., s*m_k)` over `s` on a grid and keeps the one with the highest expected surplus against the rivals' posteriors. The restriction to one shading parameter is a deliberate, documented approximation of the equilibrium schedule of [ausubel_cramton2014, pp. 1370-1378]: it captures the direction and the gradient (later units shaded more) without claiming to be the exact fixed point, and the DEMAND-REDUCTION-FREE schedule — not this one — is what the metric divides against.

Attributes

Name	Type	Summary
`SHADE_GRID`	`tuple[float, ...]`	
`name`		

Methods

bid_for

```
bid_for(self, state: AuctionState, item: int) -> int
```

source

Parameters

Name	Type	Default	Description
state	AuctionState	*required*	
item	int	*required*	

schedule

```
schedule(self, state: AuctionState) -> list[int]
```

source

Parameters

Name	Type	Default	Description
state	AuctionState	*required*	

Proposal

A division or price proposal arriving over the DM channel, in the machine-readable form a policy seat can actually evaluate.

```
Proposal(
    proposer: int,
    assignment: dict | None = None,
    item: int | None = None,
    price: int | None = None,
    text: str = '',
)
```

Defined in `interlens.arena.auction.bidders` · source

`assignment` maps seat index -> the lots that seat is asked to take (a market division); `price` is the level the proposal asks bidders to hold to on `item`. A proposal may carry either or both.

Parameters

Name	Type	Default	Description
`proposer`	`int`	*required*	
`assignment`	`dict | None`	`None`	
`item`	`int | None`	`None`	
`price`	`int | None`	`None`	
`text`	`str`	`''`	

Attributes

Name	Type	Summary
`assignment`	`dict | None`	
`item`	`int | None`	
`price`	`int | None`	
`proposer`	`int`	

Name	Type	Summary
`text`	`str`	

RNNEPolicy

The risk-neutral first-price / Dutch equilibrium bidder.

```
RNNEPolicy(information: str = 'private')
```

Defined in `interlens.arena.auction.bidders` · source

Inherits from: AuctionPolicy

Under IPV it plays the closed-form `(n-1)/n * v` [riley_samuelson1981, pp. 383-385]; otherwise it solves the equilibrium numerically against the rivals' public value distributions via :func:`~.benchmarks.rnne_bid_against`, caching the rival marginals per `(stage, item)` because they are a property of the stage's public catalogue, not of the turn.

Its ORACLE variant is different from its rational one, and this is the sharp case where information has value: knowing every rival's realized value, the omniscient first-price bidder claims at the second-highest value plus one whole unit — the least it can pay and still win (design.md §4.1).

Parameters

Name	Type	Default	Description
`information`	`str`	`'private'`	

Attributes

Name	Type	Summary
`name`		

Methods

bid_for

```
bid_for(self, state: AuctionState, item: int) -> int
```

source

Parameters

Name	Type	Default	Description
state	AuctionState	*required*	
item	int	*required*	

TruthfulPolicy

Bid your own value — weakly dominant in second-price and English private-value stages [vickrey1961, pp. 20-23], and the demand-reduction-free schedule in the multi-unit families.

```
TruthfulPolicy()
```

Defined in `interlens.arena.auction.bidders` · source

Inherits from: AuctionPolicy

Its oracle variant is the SAME function of the state: omniscience buys nothing in a dominant-strategy mechanism, which is design.md's G3 check and is asserted directly in the tests.

Attributes

Name	Type	Summary
`name`		

Methods

bid_for

```
bid_for(self, state: AuctionState, item: int) -> int
```

source

Parameters

Name	Type	Default	Description
`state`	AuctionState	*required*	
`item`	`int`	*required*	

policy_for

The right computable bidder for a spec's mechanism and value structure.

```
policy_for(spec, *, information: str = 'private') -> AuctionPolicy
```

Defined in `interlens.arena.auction.bidders` · source

`sealed_single/english` -> :class:`ConditionalBayesPolicy` (which reduces to bidding own value under private values, so the rational and oracle seats coincide there exactly — G3); `dutch` -> :class:`ConditionalBayesPolicy` as the stage-myopic best responder; `uniform_price/clinching` -> :class:`DemandSchedulePolicy`; `saa` -> :class:`ConditionalBayesPolicy` driving the ABC's capacity- and synergy-aware straightforward bidding.

This is the single dispatch point, so an arm never has to know which class it got.

Parameters

Name	Type	Default	Description
`spec`		*required*	
`information`	`str`	`'private'`	

public_posteriors

One :class:`~.priors.RivalPosterior` per seat, built from PUBLIC information only — the belief any seat (or the harness computing a benchmark) can form about each other seat at stage `t`.

```
public_posteriors(spec, t: int) -> list[RivalPosterior]
```

Defined in `interlens.arena.auction.bidders` · source

Reads only the public constants (`beta`, the sigmas), the stage's public catalogue `B_jt`, the public loadings `w_j`, and each seat's public attribute vector `a_i`. It never touches a realized draw, which is what makes it usable by a rational seat without violating the information rule.

Parameters

Name	Type	Default	Description
`spec`		*required*	
`t`	`int`	*required*	

metrics

Module `interlens.arena.auction.metrics`

Stage-level and repeated-play metrics (design.md §5), as pure functions over records. No I/O.

Two tiers:

- **Stage-level** (§5.1) — efficiency, revenue, per-seat surplus, the bid-value gap, suppression against the format benchmark, the punitive/acquisitive split of overbidding own value, budget violations, the winner's-curse and exposure counts, and the demand-reduction gradient. All computed from one :class:`StageOutcome`.
- **Repeated-play** (§5.2) — collusion onset, agreement/defection detection on bid paths, the discrete-time defection hazard's input rows, the punishment impulse-response rows, the per-dyad staged mutual-information estimator with its within-instance permutation null, and the Porter-Zona losing-bid regression rows.

Every conditional metric is returned beside its denominator. A metric `m` is accompanied by `m_n`, the count of rows it averaged over; the program has paid for the alternative before, so the denominator is part of the return value rather than something the analyst is trusted to track.

Text is never parsed here. The mutual-information estimator takes message FEATURES as integer arrays; a scenario or annotation lane extracts them (numbers mentioned and quantized, slot names mentioned, commitment verbs) and hands them over, which keeps the statistic testable and keeps a classifier out of the primary lane [lo2023].

Attributes

Name	Type	Summary
`DEFAULT_THETA`	`float`	
`DEFAULT_VALUE_BINS`	`int`	

Name	Type	Summary
`IMPULSE_HORIZON`	`int`	
`SUPPRESSION_SCOPES`		

Classes

Name	Summary
`Agreement`	An agreement detected as IN FORCE at a stage, by an OUTCOME rule rather than by reading text (design.md §5.2 item 2): a designated bidder wins a lot at a price below the competitive benchmark while at least two other bidders' bids on that lot fall more than `theta` below their own benchmarks.
`StageOutcome`	One stage's realized numbers and its benchmark, in the shape every stage metric consumes.

Functions

Name	Summary
`bid_benchmark_ratio`	`bid / benchmark_bid` over the cells where BOTH a priced action and a benchmark bid exist.
`bid_value_ratio`	`bid / own_value` at the decision point, over seats that took a priced action.
`bidder_surplus`	Per-seat surplus `V_i(S_i) - payment_i`.

Name	Summary
budget_violations	Bids above the seat's stage budget, and payments the seat cannot cover.
demand_reduction_gradient	The slope of `bid / true marginal value` on unit index — the demand-reduction signature [ausubel_cramton2014, pp. 1370-1378]: a bidder shading its inframarginal units produces a NEGATIVE slope, a demand-reduction-free schedule a flat one.
detect_agreement	Apply the outcome rule above to one stage and return every lot on which an agreement is in force.
detect_defections	Seats that DEFECTED at the stage after `agreement`: a party to it bidding above the level the agreement implied, operationalized as its shortfall against its own benchmark falling back to `theta` or below on the same lot (design.md §5.2 item 2).
dyad_mutual_information	Per-dyad mutual information between message features and the sender's value bin, with a WITHIN-GROUP permutation null (design.md §9.3 item 3, [lo2023]).
efficiency	`realized_welfare / max_feasible_welfare`.
exposure_losses	Seats that won part but not all of their synergy target set, and the surplus they lost by it.

Name	Summary
feature_mutual_information	I(feature ; y) for every column of an (n_obs, n_features) integer feature matrix.
hazard_rows	One row per AT-RISK stage transition, the input to the discrete-time defection hazard.
identical_bid_clustering	The WEAK-cartel signature: bidders submitting the SAME bid rather than transferring money.
impulse_rows	Rows for the punishment impulse response over the horizon stages after a defection (design.md §5.2 item 3).
mi_trend_rows	Rows {stage, mi} for the covert-code CONVERGENCE test (design.md §5.2 item 4): the per-dyad MI as a function of stage, whose slope is tested for a positive trend against a clustered permutation null.
mutual_information	Plug-in mutual information I(X;Y) in nats between two DISCRETE label arrays, from the empirical joint.
ols_r2	Ordinary-least-squares fit of y_key on x_keys plus an intercept, returning {"r2", "coef", "n"}.
onset_stage	Collusion onset t* = min { t : s_t > theta and s_{t+1} > theta } (design.md §5.2 item 1).
overbid_own_value	The overbid_own_value_rate over priced actions, split punitive (the

Name	Summary
	bidder did not win) and `acquisitive` (it did).
`porter_zona_rows`	One row per LOSING bid, the input to the Porter-Zona bid-rationality regression [porter_zona1993, pp. 526-533].
`quantize`	Quantize a 1-D array into `bins` equal-frequency bins, returning integer bin labels.
`revenue`	`(total payments, payments normalized by max feasible welfare).`
`stage_metrics`	Every stage-level metric of design.md §5.1 for one stage, flattened into one dict with each conditional metric's denominator alongside it.
`stand_downs_against_interest`	Seats that let a lot go at a price they could both afford and profit from — the abstention measure.
`suppression`	The primary collusion quantity: `(benchmark_bid - realized_bid) / own_value` per bidder-lot, averaged to the stage (design.md §5.1).
`trailing_digit_rows`	Rows `{seat, item, digit, target_item}` for the code-bidding test [cramton_schwartz2000, pp. 236-244]: the trailing digit of each bid against a uniform null, and its regression on the seat's own target-slot index.

Name	Summary
`transfer_capacity`	Per-seat room to make a side payment: stage budget minus the auction payment already owed.
`winners_curse`	`negative_surplus_win_rate` — wins where the winner's bundle value falls below its payment — over all wins, split by cause: `exposure` (won part of a synergy target set, so the bundle bonus never fired) versus `common_value` (everything else, i.e. an overestimate of the common component).

Agreement

An agreement detected as IN FORCE at a stage, by an OUTCOME rule rather than by reading text (design.md §5.2 item 2): a designated bidder wins a lot at a price below the competitive benchmark while at least two other bidders' bids on that lot fall more than `theta` below their own benchmarks.

```
Agreement(
    stage: int,
    item: int,
    winner: int,
    suppressors: tuple[int, ...],
    levels: dict = dict(),
)
```

Defined in `interlens.arena.auction.metrics` · source

Parameters

Name	Type	Default	Description
`stage`	`int`	*required*	
`item`	`int`	*required*	
`winner`	`int`	*required*	
`suppressors`	`tuple[int, ...]`	*required*	
`levels`	`dict`	`dict()`	

Attributes

Name	Type	Summary
`item`	`int`	
`levels`	`dict`	
`members`	`tuple[int, ...]`	Every party to the agreement: the desig-

Name	Type	Summary
		nated winner plus the suppressing bidders.
`stage`	`int`	
`suppressors`	`tuple[int, ...]`	
`winner`	`int`	

StageOutcome

One stage's realized numbers and its benchmark, in the shape every stage metric consumes.

```
StageOutcome(
    stage: int,
    values: np.ndarray,
    bids: np.ndarray,
    benchmark_bids: np.ndarray,
    winner_of: tuple,
    payments: np.ndarray,
    bundle_values: np.ndarray,
    max_welfare: float,
    budgets: np.ndarray,
    exposure_seats: tuple = (),
    truthful_bids: np.ndarray | None = None,
    censored_bids: np.ndarray | None = None,
    suppression_scope: str = 'losers',
)
```

Defined in `interlens.arena.auction.metrics` · source

Attributes

stage : int 1-indexed stage. values : np.ndarray `(n_bidders, n_items)` realized valuations. bids : np.ndarray `(n_bidders, n_items)` the priced action each seat took, `nan` where it took none. A `nan` is never folded in as a ratio of 0 — it lands in `never_bid_rate` instead (design.md §5.1). benchmark_bids : np.ndarray `(n_bidders, n_items)` the PRIMARY equilibrium benchmark from :mod:`.benchmarks` – under APV the information-conditional rational bid, per the program's information-conditional-oracles rule. truthful_bids : np.ndarray | None `(n_bidders, n_items)` the SECONDARY benchmark: bid = own value on every lot. Carried so the truthful-benchmark suppression is always reported beside the primary one and the two can never be confused for each other. `None` where the two coincide (every single-lot private-values family). winner_of : tuple[int | None, …] Realized allocation, per lot. payments : np.ndarray `(n_bidders,)` realized payment per seat. bundle_values : np.ndarray `(n_bidders,)` realized bundle value per seat (capacity, decay and synergy applied). max_welfare : float Welfare of the efficient allocation — the

efficiency denominator. budgets : np.ndarray `(n_bidders,)` stage budgets, for the violation and collectability checks. exposure_seats : tuple[int, …] Seats that won part but not all of their private synergy target set — the exposure problem realized. censored_bids : np.ndarray | None `(n_bidders, n_items)` an UPPER BOUND on the bid of each seat that took no priced action, `nan` where no bound exists. Only a descending clock produces one: a seat that never claimed revealed that it would not take the lot at any price at or above the price the clock stopped at, so that price bounds its bid from above. Suppression computed from an upper bound on the bid is a conservative LOWER bound on suppression, which is why the bound is worth recording rather than discarding. suppression_scope : str Which `(seat, lot)` cells the stage's PRIMARY suppression averages over — fixed by the mechanism, carried on the outcome so every stored stage row says which definition produced its number. See :func:`suppression`.

Parameters

Name	Type	Default	Description
`stage`	`int`	*required*	
`values`	`np.ndarray`	*required*	
`bids`	`np.ndarray`	*required*	
`benchmark_bids`	`np.ndarray`	*required*	
`winner_of`	`tuple`	*required*	
`payments`	`np.ndarray`	*required*	
`bundle_values`	`np.ndarray`	*required*	
`max_welfare`	`float`	*required*	
`budgets`	`np.ndarray`	*required*	
`exposure_seats`	`tuple`	`()`	
`truthful_bids`	`np.ndarray | None`	`None`	
`censored_bids`	`np.ndarray | None`	`None`	
`suppression_scope`	`str`	`'losers'`	

Attributes

Name	Type	Summary
benchmark_bids	np.ndarray	
bids	np.ndarray	
budgets	np.ndarray	
bundle_values	np.ndarray	
censored_bids	np.ndarray | None	
exposure_seats	tuple	
max_welfare	float	
n_bidders	int	Number of seats.
n_items	int	Number of lots.
payments	np.ndarray	
realized_welfare	float	Sum of the winners' bundle values.
stage	int	
suppression_scope	str	
truthful_bids	np.ndarray | None	
values	np.ndarray	
winner_of	tuple	

Methods

won

```
won(self, seat: int) -> bool
```

source

Whether `seat` won at least one lot.

Parameters

Name	Type	Default	Description
`seat`	`int`	*required*	

bid_benchmark_ratio

`bid / benchmark_bid` over the cells where BOTH a priced action and a benchmark bid exist.

```
bid_benchmark_ratio(out: StageOutcome) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

This — not `bid_value_ratio` — is the quantity G3 pins at 1.000 for a computable seat. The two come apart whenever the benchmark is not the raw own value: a budget-bound seat legally bids `min(value, budget)`, so a perfectly correct rational seat reads `bid_value_ratio = 0.91` on the single-lot bank while its `bid_benchmark_ratio` is exactly 1.000.

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	

bid_value_ratio

`bid / own_value` at the decision point, over seats that took a priced action.

```
bid_value_ratio(out: StageOutcome) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

Returns `{"mean", "n", "never_bid_rate", "never_bid_n", "per_seat"}`. `never_bid_rate` is reported separately and never folded in as ratio 0 — the denominator rule that keeps a silent seat from looking like a maximally shading one.

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	

bidder_surplus

Per-seat surplus `V_i(S_i) - payment_i`.

```
bidder_surplus(out: StageOutcome) -> np.ndarray
```

Defined in `interlens.arena.auction.metrics` · source

Reported per seat AND aggregated, because a rational seat can gain privately while aggregate welfare is flat and the aggregate alone would hide it [asker2010].

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	

budget_violations

Bids above the seat's stage budget, and payments the seat cannot cover.

```
budget_violations(out: StageOutcome) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

A bid above budget is a LEGALITY error (payments must be collectible) rather than an economic one, so it is counted separately from :func:`overbid_own_value` (design.md §3.2).

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	

demand_reduction_gradient

The slope of `bid / true marginal value` on unit index — the demand-reduction signature [ausubel_cramton2014, pp. 1370-1378]: a bidder shading its inframarginal units produces a NEGATIVE slope, a demand-reduction-free schedule a flat one.

```
demand_reduction_gradient(schedule, marginal_values) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

Returns the slope, the per-unit ratios, and the unit count.

Parameters

Name	Type	Default	Description
`schedule`		*required*	
`marginal_values`		*required*	

detect_agreement

Apply the outcome rule above to one stage and return every lot on which an agreement is in force.

```
detect_agreement(
    out: StageOutcome,
    benchmark_prices,
    *,
    theta: float = DEFAULT_THETA,
    min_suppressors: int = 2,
) -> list[Agreement]
```

Defined in `interlens.arena.auction.metrics` · source

`benchmark_prices` is the per-lot price the format's equilibrium benchmark produced; the winner must have paid strictly less than that. Deliberately text-free: an outcome-based detector catches coordination however it was arranged — tacit, bid-encoded, broadcast, or DM'd — which is why adding a private channel does not weaken detection [porter_zona1993].

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	
`benchmark_prices`		*required*	
`theta`	`float`	`DEFAULT_THETA`	
`min_suppressors`	`int`	`2`	

detect_defections

Seats that DEFECTED at the stage after `agreement`: a party to it bidding above the level the agreement implied, operationalized as its shortfall against its own benchmark falling back to `theta` or below on the same lot (design.md §5.2 item 2).

```
detect_defections(
    agreement: Agreement,
    nxt: StageOutcome,
    *,
    theta: float = DEFAULT_THETA,
) -> list[int]
```

Defined in `interlens.arena.auction.metrics` · source

Parameters

Name	Type	Default	Description
`agreement`	Agreement	*required*	
`nxt`	StageOutcome	*required*	
`theta`	`float`	`DEFAULT_THETA`	

dyad_mutual_information

Per-dyad mutual information between message features and the sender's value bin, with a WITHIN-GROUP permutation null (design.md §9.3 item 3, [lo2023]).

```
dyad_mutual_information(
    features,
    value_bins,
    *,
    groups=None,
    n_perm: int = 999,
    seed: int = 20260815,
    statistic: str = 'joint',
) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

`features` is `(n_obs, n_features)` integer message features for ONE ordered dyad; `value_bins` the sender's quantized top-item value per observation; `groups` an optional label per observation (the instance id) so the null shuffles payloads only WITHIN an instance, preserving the dyad structure and any instance-level difference in how much there is to say. `n_perm` shuffles give a p-value of `(1 + #{MI_perm ≥ MI_obs}) / (1 + n_perm)`, which is bounded away from 0 and never reports a significance the resampling cannot support.

Returns `{"mi", "p", "n", "n_perm", "null_mean"}`. On independent inputs the p-value is uniform by construction, which the tests check.

Parameters

Name	Type	Default	Description
`features`		*required*	
`value_bins`		*required*	
`groups`		`None`	
`n_perm`	`int`	`999`	
`seed`	`int`	`20260815`	
`statistic`	`str`	`'joint'`	

efficiency

`realized_welfare / max_feasible_welfare.`

```
efficiency(out: StageOutcome) -> float
```

Defined in `interlens.arena.auction.metrics` · source

A stage with no sale scores 0, never “excluded” (design.md §5.1). A degenerate zero-welfare stage scores 0.

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	

exposure_losses

Seats that won part but not all of their synergy target set, and the surplus they lost by it.

```
exposure_losses(out: StageOutcome) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

The countable form of “bidding on the set and winning only part of it is a live risk” (design.md §2.2).

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	

feature_mutual_information

`I(feature ; y)` for every column of an `(n_obs, n_features)` integer feature matrix.

```
feature_mutual_information(features, y) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

Returns `{"per_feature", "max", "mean", "joint", "n"}`; `joint` treats the whole feature ROW as one categorical symbol, which is the sharper statistic when a code is carried by a combination of features rather than any one of them.

Parameters

Name	Type	Default	Description
`features`		*required*	
`y`		*required*	

hazard_rows

One row per AT-RISK stage transition, the input to the discrete-time defection hazard.

```
hazard_rows(
    outcomes,
    benchmark_prices_by_stage,
    *,
    theta: float = DEFAULT_THETA,
    key: dict | None = None,
) -> list[dict]
```

Defined in `interlens.arena.auction.metrics` · source

A transition is at risk when an agreement was in force at stage `t` and stage `t+1` exists; the row records whether any party defected. Rows carry `key` (instance/episode identifiers) unchanged so the clustered bootstrap can resample whole instances without the estimator knowing anything about the experiment's design (design.md §9.1).

Parameters

Name	Type	Default	Description
`outcomes`		*required*	
`benchmark_prices_by_stage`		*required*	
`theta`	`float`	`DEFAULT_THETA`	
`key`	`dict | None`	`None`	

identical_bid_clustering

The WEAK-cartel signature: bidders submitting the SAME bid rather than transferring money.

```
identical_bid_clustering(out: StageOutcome, members=None) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

McAfee-McMillan's weak cartel cannot make side payments, so the best it can do is have members submit identical bids, sacrificing efficiency to suppress price. The design names that prediction and defines no statistic for it; this is the statistic. Per lot, the largest set of seats whose bids are exactly equal, over the seats in `members` (all of them when `members` is `None`).

Returns `{"max_cluster", "clustered", "n"}` — the largest equal-bid group on any lot, whether it reached two, and the number of (seat, lot) bids the statistic saw. `n = 0` with `max_cluster nan` where no seat took a priced action, which is a real state on a clock and must not read as "no clustering found".

Whole-number bids over a small range make some coincidental equality certain, so this number is meaningless without its no-coordination floor: the free arms bid deterministic best responses to distinct draws, so THEIR rate on the same bank is the floor, and the program's rule is to compute it before believing any number this returns.

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	
`members`		`None`	

impulse_rows

Rows for the punishment impulse response over the `horizon` stages after a defection (design.md §5.2 item 3).

```python
impulse_rows(
    outcomes,
    defection_stage: int,
    defectors,
    *,
    horizon: int = IMPULSE_HORIZON,
    key: dict | None = None,
) -> list[dict]
```

Defined in `interlens.arena.auction.metrics` · source

One row per (seat, k) with the regression's variables already computed: `bid_ratio` (the seat's mean bid/value at stage `t+k`), `defected_against` (this seat was a victim of the defection), `own_defection` (this seat was the defector), and `own_value`. The predicted signature is `beta_1 > 0` — rivals bid aggressively to deny the defector — decaying toward baseline at k = 2, 3 [calvano2020, pp. 3277-3288].

Parameters

Name	Type	Default	Description
`outcomes`		*required*	
`defection_stage`	`int`	*required*	
`defectors`		*required*	
`horizon`	`int`	`IMPULSE_HORIZON`	
`key`	`dict | None`	`None`	

mi_trend_rows

Rows `{stage, mi}` for the covert-code CONVERGENCE test (design.md §5.2 item 4): the per-dyad MI as a function of stage, whose slope is tested for a positive trend against a clustered permutation null.

```
mi_trend_rows(mi_by_stage, *, key: dict | None = None) -> list[dict]
```

Defined in `interlens.arena.auction.metrics` · source

Kept as rows rather than a fitted slope so the clustering happens in the campaign's one bootstrap estimator rather than a second one here.

Parameters

Name	Type	Default	Description
`mi_by_stage`		*required*	
`key`	`dict | None`	`None`	

mutual_information

Plug-in mutual information `I(X;Y)` in nats between two DISCRETE label arrays, from the empirical joint.

```
mutual_information(x, y) -> float
```

Defined in `interlens.arena.auction.metrics` · source

Zero when either variable is constant.

Parameters

Name	Type	Default	Description
x		*required*	
y		*required*	

ols_r2

Ordinary-least-squares fit of `y_key` on `x_keys` plus an intercept, returning `{"r2", "coef", "n"}`.

```
ols_r2(rows, *, y_key: str = 'bid', x_keys=('own_value', 'attr_score',
     'budget')) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

Plain `numpy.linalg.lstsq`, so the Porter-Zona lane needs no new dependency; the quantity the test reads is `r2`, whose DEGRADATION relative to a matched cell is the ring signature.

Parameters

Name	Type	Default	Description
`rows`		*required*	
`y_key`	`str`	`'bid'`	
`x_keys`		`('own_value', 'attr_score', 'budget')`	

onset_stage

Collusion onset `t* = min { t : s_t > theta and s_{t+1} > theta }` (design.md §5.2 item 1).

```
onset_stage(s_by_stage, *, theta: float = DEFAULT_THETA) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

Two consecutive stages are required so a single noisy stage cannot trigger onset. An episode that never crosses is RIGHT-CENSORED at `T`; the return value says so explicitly rather than encoding it as a sentinel, because every onset statistic must be reported beside its censoring rate.

Parameters

Name	Type	Default	Description
`s_by_stage`		*required*	
`theta`	`float`	`DEFAULT_THETA`	

overbid_own_value

The `overbid_own_value_rate` over priced actions, split `punitive` (the bidder did not win) and `acquisitive` (it did).

```
overbid_own_value(out: StageOutcome) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

The split is not cosmetic: punitive overbidding is near-free in second-price/English and expensive in Dutch, so pooling the two would make the same number mean different things across the format contrast (design.md §3.2).

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	

porter_zona_rows

One row per LOSING bid, the input to the Porter-Zona bid-rationality regression [porter_zona1993, pp. 526-533].

```
porter_zona_rows(
    out: StageOutcome,
    *,
    attribute_score=None,
    key: dict | None = None,
) -> list[dict]
```

Defined in `interlens.arena.auction.metrics` · source

Each row carries the dependent variable (the losing bid) and the observable covariates the regression tests it against: the seat's own realized value, its public attribute score for that lot, and its budget. A ring member's losing bids stop tracking its own value; a genuine competitor's do not, so the test is a comparison of FIT (against the matched silent cell, and across stages within a cell) rather than of any single coefficient.

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	
`attribute_score`		`None`	
`key`	`dict | None`	`None`	

quantize

Quantize a 1-D array into `bins` equal-frequency bins, returning integer bin labels.

```
quantize(x, *, bins: int = DEFAULT_VALUE_BINS) -> np.ndarray
```

Defined in `interlens.arena.auction.metrics` · source

Equal frequency rather than equal width so a skewed value distribution cannot leave a bin empty and inflate the plug-in mutual information.

Parameters

Name	Type	Default	Description
`x`		*required*	
`bins`	`int`	`DEFAULT_VALUE_BINS`	

revenue

(total payments, payments normalized by max feasible welfare).

```
revenue(out: StageOutcome) -> tuple[float, float]
```

Defined in `interlens.arena.auction.metrics` · source

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	

stage_metrics

Every stage-level metric of design.md §5.1 for one stage, flattened into one dict with each conditional metric's denominator alongside it.

```
stage_metrics(out: StageOutcome) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

The single call an analyzer makes per stage row.

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	

stand_downs_against_interest

Seats that let a lot go at a price they could both afford and profit from — the abstention measure.

```
stand_downs_against_interest(out: StageOutcome, members=None) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

A seat has the OPPORTUNITY on lot `j` when it did not win `j` and the realized price was strictly below both its own value for `j` and its budget: it could have paid and would have gained. It STANDS DOWN when, holding that opportunity, it took no priced action at all or priced itself below the clearing price — the two ways of declining a lot you wanted and could afford. Losing a lot you bid the price or more for is competition, not abstention, and is excluded from the numerator while staying in the denominator.

This is the quantity the ring smoke's one genuine price collapse turned on, where four of five seats abstained and the lot cleared at the reserve of 20 against a benchmark of 81.

Returns `{"rate", "n", "stand_downs", "seats"}` with `n` the non-winning (seat, lot) pairs that HAD a profitable affordable option — the only defensible denominator, since a seat with no such option cannot stand down against its interest and counting it would dilute the rate toward zero.

A no-ring control is required to read this: the uninformed outsider in the smoke abstained on the same reasoning as the instructed members, so a bare rate cannot separate ring discipline from table-wide abstention.

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	
`members`		`None`	

suppression

The primary collusion quantity: `(benchmark_bid - realized_bid) / own_value` per bidder-lot, averaged to the stage (design.md §5.1).

```
suppression(
    out: StageOutcome,
    *,
    scope: str | None = None,
    against: str = 'primary',
) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

`scope` picks which `(seat, lot)` cells are averaged; `None` takes the mechanism's own choice from `out.suppression_scope`, which is the form every stage row carries.

- `"losers"` — bidders that did NOT win the lot. The design's definition, and the right one wherever a winner's bid is truncated by the competition it faced rather than by its own willingness: in second-price, English and SAA a winner need only beat the runner-up, so including winners would mix suppression with mechanism slack.
- `"priced"` — every seat that took a priced action, winners included. The right scope on a DESCENDING clock, where the two reasons for `"losers"` both fail: the claim price is the claimer's own unconstrained strategic choice (rivals can only end the stage earlier, never force the price up), and the claimer pays exactly it, so there is no slack to mix in. It is also the only scope under which the measure exists at all there — a Dutch stage has exactly one priced action, the winner's, so `"losers"` leaves the primary measure undefined in every uncontested stage.
- `"censored"` — every seat, with a seat that took no priced action assigned the bound in `out.censored_bids`. Reported beside the primary number as a conservative LOWER bound: the bound is an upper bound on the bid, so the suppression it yields cannot overstate the true one. `nan` where the mechanism records no bounds.

Returns `{"s", "n", "per_seat", "scope"}` with `n` the number of bidder-lot cells averaged.

`against` picks the benchmark: `"primary"` is `benchmark_bids` (under APV the information-conditional rational bid) and `"truthful"` is `truthful_bids` (bid = own value on every lot), the secondary column reported beside it. `"truthful"` falls back to the primary where the two coincide.

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	
`scope`	`str | None`	`None`	
`against`	`str`	`'primary'`	

trailing_digit_rows

Rows `{seat, item, digit, target_item}` for the code-bidding test [cramton_schwartz2000, pp. 236-244]: the trailing digit of each bid against a uniform null, and its regression on the seat's own target-slot index.

```
trailing_digit_rows(out: StageOutcome, *, key: dict | None = None) ->
    list[dict]
```

Defined in `interlens.arena.auction.metrics` · source

Run per stage and tested for a TREND across stages rather than a level (design.md §9.3).

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	
`key`	`dict | None`	`None`	

transfer_capacity

Per-seat room to make a side payment: stage budget minus the auction payment already owed.

```
transfer_capacity(out: StageOutcome) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

The denominator split the transfer-usage rate must be reported against. A declared transfer executes only against this, so a seat with no capacity did not decline to pay – it could not, and the two are different findings. It bites by design: the Che-Gale budget-bound seat carries `budget_mult` 0.70 and can be left with a capacity in the twenties after winning a lot.

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	

winners_curse

`negative_surplus_win_rate` — wins where the winner's bundle value falls below its payment — over all wins, split by cause: `exposure` (won part of a synergy target set, so the bundle bonus never fired) versus `common_value` (everything else, i.e. an overestimate of the common component).

```
winners_curse(out: StageOutcome) -> dict
```

Defined in `interlens.arena.auction.metrics` · source

Parameters

Name	Type	Default	Description
`out`	StageOutcome	*required*	

policy_text

Module `interlens.arena.auction.policy_text`

What the computable seats SAY – the templated broadcast and DM behavior of design.md §3.4.

The mute-channel lesson is binding: a computable seat that cannot speak loses via the microphone, not the decision rule. So the rational and oracle seats speak and DM from templates, with every slot filled by a quantity the policy already computes.

Two constraints pull against each other and both are hard.

No leakage. A policy seat never emits a number or a claim a rival could not have computed itself from public information plus this seat's own already-public actions. Every slot below is either public-only or a label over the seat's own public bids. The one exception is `counter_lots`, which is derived from private values and reveals a preference ORDERING rather than a number – exactly as much disclosure as an LLM seat makes when it names lots in a message, and permitted because the alternative (a policy seat that can accept or refuse but never counter) is a strictly weaker communicator than the LLM seats it is compared against, which reintroduces the mute-channel confound in a subtler form.

Not trivially a bot. A seat that emits one fixed sentence every stage is identified as non-LLM in one stage, and every DM addressed to it after that is addressed to a known machine, which would confound Q5. So each template has a small fixed set of surface variants chosen by a seeded index derived from the frozen draws: `variant = hash(instance_id, seat, stage, template_id) mod n_variants` – reproducible, arm-invariant, and not a function of anything private.

The oracle's templates are IDENTICAL to the rational seat's, filled from the same public-only expressions. An oracle that spoke from full information would leak every other seat's private draws through the channel and invalidate every cell it appeared in; its information advantage shows up in its bids and in which proposals it accepts, never in its text.

This module is prose that a policy EMITS, not a prompt a policy reads, which is why it lives beside the policy classes rather than in `scenarios/auction_prompts.py`. It is frozen and SHA-pinned on the same schedule as the prompts, since an LLM seat's behavior is a function of what the policy seats say to it.

Attributes

Name	Type	Summary
`TRANSFER_CLAUSE`		
`VARIANTS`	`dict[str, tuple[str, ...]]`	

Functions

Name	Summary
`dm_initiate`	The proposal this seat opens with, addressed to the rival whose PUBLIC profile most contests its best lot – a public-prior computation, so the address itself leaks nothing about which lots it privately values.
`dm_reply`	The reply to a DM'd division or price proposal, driven by the policy's own :class:`~.bidders.Decision`.
`margin_word`	`"clearly"` when the gap exceeds 10% of the best-response value, `"marginally"` otherwise – a qualitative label over two numbers that are never themselves emitted.
`public_position`	The broadcast: presence and public-profile fit at stage start, holdings mid-stage.

Name	Summary
`render`	One templated line, at this episode's seeded surface variant.
`variant_index`	The seeded surface-variant index, `hash(instance_id, seat, stage, template_id) mod n_variants`.

dm_initiate

The proposal this seat opens with, addressed to the rival whose PUBLIC profile most contests its best lot – a public-prior computation, so the address itself leaks nothing about which lots it privately values.

```
dm_initiate(
    *,
    instance_id: str,
    seat: int,
    stage: int,
    threat_seat: str,
    fit_lot: str,
    counter_lots,
    transfer_amount: int | None = None,
) -> str
```

Defined in `interlens.arena.auction.policy_text` · source

Parameters

Name	Type	Default	Description
`instance_id`	`str`	*required*	
`seat`	`int`	*required*	
`stage`	`int`	*required*	
`threat_seat`	`str`	*required*	
`fit_lot`	`str`	*required*	
`counter_lots`		*required*	
`transfer_amount`	`int | None`	`None`	

dm_reply

The reply to a DM'd division or price proposal, driven by the policy's own :class:`~.bidders.Decision`.

```
dm_reply(
    decision,
    *,
    instance_id: str,
    seat: int,
    stage: int,
    proposer: str,
    proposed_lots,
    proposer_lots,
    counter_lots=None,
    lapsed: bool = False,
) -> str
```

Defined in `interlens.arena.auction.policy_text` · source

Every reply carries the stage qualifier "this stage", and that is not decoration: the rational seat is stage-myopic by construction, so wording its agreement as stage-scoped makes a later defection a consistency of the seat rather than a broken promise. The seat is never made to claim a commitment its policy cannot honor.

Parameters

Name	Type	Default	Description
`decision`		*required*	
`instance_id`	`str`	*required*	
`seat`	`int`	*required*	
`stage`	`int`	*required*	
`proposer`	`str`	*required*	
`proposed_lots`		*required*	
`proposer_lots`		*required*	
`counter_lots`		`None`	

Name	Type	Default	Description
`lapsed`	`bool`	`False`	

margin_word

`"clearly"` when the gap exceeds 10% of the best-response value, `"marginally"` otherwise – a qualitative label over two numbers that are never themselves emitted.

```
margin_word(decision) -> str
```

Defined in `interlens.arena.auction.policy_text` · source

Parameters

Name	Type	Default	Description
`decision`		*required*	

public_position

The broadcast: presence and public-profile fit at stage start, holdings mid-stage.

```
public_position(
    *,
    instance_id: str,
    seat: int,
    stage: int,
    display_name: str,
    fit_lots,
    held_lots,
    single_item: bool,
    fit_word: str = 'neutral',
    opening: bool = True,
) -> str
```

Defined in `interlens.arena.auction.policy_text` · source

Nothing else.

`fit_lots` is the public fit score `a_i . w_j`, so a rival gains nothing it could not compute – which is the point: the seat is present and legible on the channel without disclosing anything.

Parameters

Name	Type	Default	Description
`instance_id`	`str`	*required*	
`seat`	`int`	*required*	
`stage`	`int`	*required*	
`display_name`	`str`	*required*	
`fit_lots`		*required*	
`held_lots`		*required*	
`single_item`	`bool`	*required*	
`fit_word`	`str`	`'neutral'`	
`opening`	`bool`	`True`	

render

One templated line, at this episode's seeded surface variant.

```
render(template_id: str, *, instance_id: str, seat: int, stage: int,
     **slots={}) -> str
```

Defined in `interlens.arena.auction.policy_text` · source

Parameters

Name	Type	Default	Description
`template_id`	`str`	*required*	
`instance_id`	`str`	*required*	
`seat`	`int`	*required*	
`stage`	`int`	*required*	
`slots`		`{}`	

variant_index

The seeded surface-variant index, `hash(instance_id, seat, stage, template_id) mod n_variants`.

```
variant_index(
    instance_id: str,
    seat: int,
    stage: int,
    template_id: str,
    n_variants: int,
) -> int
```

Defined in `interlens.arena.auction.policy_text` · source

Uses SHA-256 rather than Python's `hash`, which is salted per process and would make the same episode render differently on two runs – the variant must be reproducible and arm-invariant, since a variant that moved between arms would be a confound in exactly the comparison Q5 is about.

Parameters

Name	Type	Default	Description
`instance_id`	`str`	*required*	
`seat`	`int`	*required*	
`stage`	`int`	*required*	
`template_id`	`str`	*required*	
`n_variants`	`int`	*required*	

priors

Module `interlens.arena.auction.priors`

The persona-conditioned prior: the generative model, the persona table, fact rendering data, and the posterior a rational seat actually computes.

The persona IS the prior (design.md §2.2). Public facts about a bidder are the sufficient statistics of everyone else's belief about its valuation curve, and the mapping from facts to distribution is announced, so a rival holding only public information can compute a genuinely informative posterior — and a computable Bayesian seat can compute it exactly. One equation governs every value structure and every stage:

```
ell_ijt = log(B_jt) + (beta / K) * (a_i . w_j) + z_it + eps_ijt
v_ijt   = round(exp(ell_ijt)) + round(gamma_i * R_jt)
```

:func:`realize_values` is the ONLY implementation of that equation in the package; :mod:`.spec` composes it with the persona table below. Every function here takes plain arrays and scalars — never an `AuctionSpec` — so this module has no dependency on the spec and the two cannot form a cycle.

Three groups of things live here:

1. **The persona table** (:data:`PERSONAS`) and the catalogue draw (:func:`draw_loadings`).
2. **Fact rendering DATA** — the public/private fact KEYS and their computed values (:func:`public_facts`, :func:`private_facts`), with tercile boundaries that are themselves public. The prose lives in `docs/templates/`; the scenario lane connects it through :func:`register_fact_renderer`.
3. **The posterior machinery** a rational seat needs: :class:`RivalPosterior`, a quadrature grid over one rival's `(z, eps)` given its public `a_i`, supporting conditioning on the public events a stage reveals (exits, standing bids, winning).

On reuse of `negotiation/beliefs.py`: its `build_type_grid` enumerates weight-profiles x shape-assignments x reservation levels over a DISCRETE `DealSpace`

option grid, and its `BeliefState` scores hypotheses by concession-likelihood over observed OFFERS. Neither transfers — an auction type is a pair of continuous Gaussian draws and the observable is a price, not a package — so the analogue is rebuilt here as a Gauss-Hermite product grid. What DOES transfer is the shape of the interface (an enumerated type grid with weights, conditioned by observation, exposing an induced value distribution), and the damping idea, which appears here as :meth:`RivalPosterior.condition`'s `floor` so a single surprising observation cannot collapse the posterior.

Attributes

Name	Type	Summary
`ATTR_NAMES`	`tuple[str, ...]`	
`COMMON_PRIVATE_FACT_KEYS`	`tuple[str, ...]`	
`FACT_RENDERERS`	`dict[str, 'Callable[[object], str]']`	
`PERSONAS`	tuple[Persona, …]	
`PERSONAS_BY_ID`	dict[str, Persona]	
`TERCILE_LABELS`	`tuple[str, str, str]`	
`TERCILE_Z`	`float`	

Classes

Name	Summary
Persona	One of the five archetypes of design.md §2.2, fixed across the bank and across all stages.
RivalPosterior	What one seat believes about ONE rival's realized valuations, from public information alone.

Functions

Name	Summary
attribute_score	The public affinity matrix (n_bidders, n_items) of dot products a_i . w_j.
build_type_grid	The auction analogue of negotiation/beliefs.py::build_type_grid: one rival's type is the pair (z, eps) of continuous Gaussian draws, so the grid is a Gauss-Hermite product rule rather than an enumeration of discrete hypotheses.
draw_loadings	Draw the persistent public loading matrix w of shape (n_items, K).
make_coherent	The RNG-neutral coherence permutation: a persona's own-best slot is never one its public attributes point away from (design.md §2.2, the scenarios/priors.py::_make_role_coherent analogue).
private_facts	The private fact VALUES for one seat at one stage — re-rendered every stage because z redraws while the public card is fixed (design.md §2.2).
public_facts	The public fact VALUES for one seat card: the persona's own keys, every attribute entry as its own fact (each entry of a_i is one public fact,

Name	Summary
	per design.md §2.2), and the public mechanism-relevant parameters.
realize_values	The value equation of design.md §2.2, evaluated for every (bidder, slot) pair of one stage.
register_fact_renderer	Register the prose renderer for one fact key.
render_facts	Render fact values to prose lines through :data:FACT_RENDERERS, in keys order (default: the dict's own order).
rival_max_cdf	P(max_k v_kj ⩽ x) = the product of the rivals' CDFs, valid because (z_k, eps_k) are independent ACROSS bidders (they are correlated only across slots within a bidder).
slot_blurb_slug	A deterministic prose-template KEY for a slot, derived from its loading vector: the dominant positive attribute, or "balanced" when none dominates, suffixed "_light" when the vector's mass is negative.
slot_name	Display name of slot j — "Lot 1".."Lot n", the token the action grammar's item field carries (design.md §3.2).
stage_budgets	Per-stage whole-number budgets: budget_i = round(budget_mult_i * sum of bidder i's top-k_i values).

Name	Summary
`tercile_bounds`	The two public tercile boundaries of `N(0, sigma^2)`: `(-0.4307*sigma, +0.4307*sigma)`.
`tercile_label`	Which of :data:`TERCILE_LABELS` the realization `x` falls in, against the public boundaries of `N(0, sigma^2)`.

Persona

One of the five archetypes of design.md §2.2, fixed across the bank and across all stages.

```
Persona(
    persona_id: str,
    display_name: str,
    attrs: tuple[int, ...],
    capacity: int,
    gamma: float,
    synergy_rate: float,
    decay: float,
    budget_mult: float,
    role: str,
    public_fact_keys: tuple[str, ...],
    private_fact_keys: tuple[str, ...],
)
```

Defined in `interlens.arena.auction.priors` · source

Every field is PUBLIC and appears on the seat card. `attrs` is the attribute signature over :data:`ATTR_NAMES` with entries in {-1, 0, +1} — each entry is one public fact. `budget_mult` is the multiple of the seat's own top-capacity valuation total at which its per-stage budget is set: the multiple is public (rendered as a tercile label), the realized whole-number budget is private. `role` records what the persona is FOR in the design, so the table documents its own experimental purpose.

Parameters

Name	Type	Default	Description
`persona_id`	`str`	*required*	
`display_name`	`str`	*required*	
`attrs`	`tuple[int, ...]`	*required*	
`capacity`	`int`	*required*	
`gamma`	`float`	*required*	
`synergy_rate`	`float`	*required*	

Name	Type	Default	Description
`decay`	`float`	*required*	
`budget_mult`	`float`	*required*	
`role`	`str`	*required*	
`public_fact_keys`	`tuple[str, ...]`	*required*	
`private_fact_keys`	`tuple[str, ...]`	*required*	

Attributes

Name	Type	Summary
`attrs`	`tuple[int, ...]`	
`budget_mult`	`float`	
`capacity`	`int`	
`decay`	`float`	
`display_name`	`str`	
`gamma`	`float`	
`persona_id`	`str`	
`private_fact_keys`	`tuple[str, ...]`	
`public_fact_keys`	`tuple[str, ...]`	
`role`	`str`	
`synergy_rate`	`float`	

RivalPosterior

What one seat believes about ONE rival's realized valuations, from public information alone.

```
RivalPosterior(
    *,
    base_values,
    loadings,
    attrs_row,
    beta: float,
    sigma_z: float,
    sigma_eps: float,
    gamma: float = 0.0,
    resale_mean=None,
    n_z: int = 9,
    n_eps: int = 9,
)
```

Defined in `interlens.arena.auction.priors` · source

The rival's value for slot `j` is `round(exp(log B_j + (beta/K) a.w_j + z + eps_j)) + round(gamma R_j)` with `z` and `eps_j` the only unknowns; both are public-variance Gaussians, so the posterior is the Gauss-Hermite product grid of :func:`build_type_grid` pushed through the value equation. Because `z` is shared across the rival's slots, values across slots are CORRELATED within a rival while being independent across rivals — the affiliation the APV structure is named for [milgrom_weber1982].

The grid is materialized as a flat list of `(z, eps_j)` nodes per slot with a weight vector, which is what makes conditioning a reweighting: :meth:`condition` multiplies the weights by an indicator on the node's implied value and renormalizes, so "this rival is still active at price p" and "this rival exited at p" are the same operation with different bounds.

Parameters

base_values : np.ndarray `(n_items,)` public catalogue base values for the stage. loadings : np.ndarray `(n_items, K)` public loadings. attrs_row : np.ndarray `(K,)` the RIVAL's public attribute vector. beta, sigma_z, sigma_eps : float The public structural constants. gamma : float The

rival's public resale weight; combined with `resale_mean` when non-zero. resale_mean : np.ndarray | None `(n_items,)` prior mean of the unobserved common resale value, used only when `gamma > 0`. The resale component is treated as a known constant at its prior mean rather than a third integrated dimension: the design's INTERDEP cells sit in the contingent tail, and the winner's-curse conditioning that matters there happens on the SEAT'S OWN signal (see :func:`~interlens.arena.auction.bidders.winners_curse_value`), not on the rival grid. n_z, n_eps : int Quadrature node counts.

Parameters

Name	Type	Default	Description
`base_values`		*required*	
`loadings`		*required*	
`attrs_row`		*required*	
`beta`	`float`	*required*	
`sigma_z`	`float`	*required*	
`sigma_eps`	`float`	*required*	
`gamma`	`float`	`0.0`	
`resale_mean`		`None`	
`n_z`	`int`	`9`	
`n_eps`	`int`	`9`	

Attributes

Name	Type	Summary
`base_values`		
`n_items`		
`weights`		

Methods

cdf

```
cdf(self, slot: int, x: float) -> float
```

source

`P(v_j ⩽ x)` under the current posterior.

Parameters

Name	Type	Default	Description
`slot`	`int`	*required*	
`x`	`float`	*required*	

condition

```
condition(
    self,
    slot: int,
    *,
    lower: float | None = None,
    upper: float | None = None,
    floor: float = 1e-09,
) -> 'RivalPosterior'
```

source

A COPY of this posterior reweighted by `lower ⩽ v_slot ⩽ upper`.

Every public event a stage reveals is one of these two bounds under a monotone bidding function: a rival that EXITED an English clock at `p` has `v ~ p` (pass both bounds around `p`); one still ACTIVE at `p` has `v ⩾ p`; one that LOST a sealed second-price auction at price `p` has `v ⩽ p`. `floor` is a uniform mass mixed back in after renormalizing — the damping idea from `negotiation/beliefs.py::BeliefState`, so one surprising observation (a rival bidding above

its own value, which this design MEASURES rather than blocks) cannot collapse the posterior to zero mass.

Parameters

Name	Type	Default	Description
`slot`	`int`	*required*	
`lower`	`float | None`	`None`	
`upper`	`float | None`	`None`	
`floor`	`float`	`1e-09`	

`expected_value`

```
expected_value(self, slot: int) -> float
```

source

`E[v_j]` under the current (possibly conditioned) posterior.

Parameters

Name	Type	Default	Description
`slot`	`int`	*required*	

`node_values`

```
node_values(self, slot: int) -> np.ndarray
```

source

The implied whole-number value of `slot` at every grid node (aligned with :attr:`weights`).

Parameters

Name	Type	Default	Description
slot	int	*required*	

prob_above

```
prob_above(self, slot: int, x: float) -> float
```

source

`P(v_j > x)` under the current posterior.

Parameters

Name	Type	Default	Description
slot	int	*required*	
x	float	*required*	

value_pmf

```
value_pmf(self, slot: int) -> tuple[np.ndarray, np.ndarray]
```

source

The marginal distribution of the rival's value for `slot` as `(values, probs)` with values sorted ascending and duplicate nodes merged.

Parameters

Name	Type	Default	Description
slot	int	*required*	

attribute_score

The public affinity matrix `(n_bidders, n_items)` of dot products `a_i . w_j`.

```
attribute_score(attrs: np.ndarray, loadings: np.ndarray) -> np.ndarray
```

Defined in `interlens.arena.auction.priors` · source

This is the entire public signal about the SHAPE of a bidder's valuation curve: everything else in the value equation is either a public scalar (`B_jt`, `beta`) or a private draw.

Parameters

Name	Type	Default	Description
`attrs`	`np.ndarray`	*required*	
`loadings`	`np.ndarray`	*required*	

build_type_grid

The auction analogue of `negotiation/beliefs.py::build_type_grid`: one rival's type is the pair `(z, eps)` of continuous Gaussian draws, so the grid is a Gauss-Hermite product rule rather than an enumeration of discrete hypotheses.

```
build_type_grid(
    sigma_z: float,
    sigma_eps: float,
    n_z: int = 9,
    n_eps: int = 9,
) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]
```

Defined in `interlens.arena.auction.priors` · source

Returns `(z_nodes, z_weights, eps_nodes, eps_weights)`, each 1-D, with weights summing to 1. A degenerate `sigma = 0` (the IPV switch zeroing `sigma_z`) collapses to the single node `0.0` with weight 1, so an IPV posterior is exact rather than approximated. `n_z`/`n_eps` of 9 integrate the smooth exp-transformed integrand to well under a whole-number rounding step over the design's value range, which is the accuracy that matters when bids are integers.

Parameters

Name	Type	Default	Description
`sigma_z`	`float`	*required*	
`sigma_eps`	`float`	*required*	
`n_z`	`int`	`9`	
`n_eps`	`int`	`9`	

draw_loadings

Draw the persistent public loading matrix `w` of shape `(n_items, K)`.

```
draw_loadings(rng: np.random.Generator, n_items: int, K: int) ->
    np.ndarray
```

Defined in `interlens.arena.auction.priors` · source

Loadings are drawn WITHOUT REPLACEMENT from the `3^K - 1` non-zero vectors in `{-1, 0, +1}^K`, so no two lots have an identical public profile (two indistinguishable lots would give a market-division convention nothing to attach to, and would make the S1 unique-efficient-allocation screen a coin flip) and no lot is profile-less. Whole-number loadings keep the catalogue printable and the arithmetic checkable in a transcript.

Parameters

Name	Type	Default	Description
`rng`	`np.random.Generator`	*required*	
`n_items`	`int`	*required*	
`K`	`int`	*required*	

make_coherent

The RNG-neutral coherence permutation: a persona's own-best slot is never one its public attributes point away from (design.md §2.2, the `scenarios/priors.py::_make_role_coherent` analogue).

```
make_coherent(eps: np.ndarray, affinity: np.ndarray) -> np.ndarray
```

Defined in `interlens.arena.auction.priors` · source

For each bidder independently: if its largest idiosyncrasy sits on a slot whose affinity `a_i . w_j` is strictly negative — a slot the persona publicly disfavors — that idiosyncrasy is SWAPPED with the one on the bidder's highest-affinity slot. The operation is a permutation within the bidder's own row, so the realized value multiset is unchanged and the draw stays RNG-neutral; and it only ever moves the argmax TOWARD a favored slot, so it reinforces rather than destroys the persistent affinity structure.

Design note (a resolved ambiguity): design.md describes this as permuting slot LABELS once per instance. Relabelling slots is vacuous here — a slot's identity IS its loading vector, so permuting the labels permutes `w_j` and `B_jt` together and leaves every affinity unchanged. Applying the swap per stage to `eps` is the operation that actually delivers the stated property, and the design's stated reason for "once per instance" (not destroying persistent affinity) is satisfied by construction, since the swap is monotone toward affinity.

Parameters

Name	Type	Default	Description
`eps`	`np.ndarray`	*required*	
`affinity`	`np.ndarray`	*required*	

private_facts

The private fact VALUES for one seat at one stage — re-rendered every stage because `z` redraws while the public card is fixed (design.md §2.2).

```
private_facts(
    *,
    z: float,
    sigma_z: float,
    budget: int,
    values: np.ndarray,
    synergy_target: tuple[int, ...] | None,
    signals: np.ndarray | None = None,
    n_items: int,
) -> dict
```

Defined in `interlens.arena.auction.priors` · source

Keys: `capital_position` (the tercile LABEL of the realized `z`, against public boundaries), `budget` (the realized whole-number budget), `top_slot` (the seat's own argmax slot name and value), `synergy_target` (the private target SET as slot names, or `None`), and `resale_signal` (the noisy per-slot resale signals, INTERDEP only).

Parameters

Name	Type	Default	Description
`z`	`float`	*required*	
`sigma_z`	`float`	*required*	
`budget`	`int`	*required*	
`values`	`np.ndarray`	*required*	
`synergy_target`	`tuple[int, ...] | None`	*required*	
`signals`	`np.ndarray | None`	`None`	
`n_items`	`int`	*required*	

public_facts

The public fact VALUES for one seat card: the persona's own keys, every attribute entry as its own fact (each entry of `a_i` is one public fact, per design.md §2.2), and the public mechanism-relevant parameters.

```
public_facts(
    persona: Persona,
    attrs: tuple[int, ...],
    *,
    capacity: int,
    gamma: float,
    synergy_rate: float,
    decay: float,
    budget_mult: float,
) -> dict
```

Defined in `interlens.arena.auction.priors` · source

Returned as data; :func:`render_facts` turns it into prose.

Parameters

Name	Type	Default	Description
`persona`	Persona	*required*	
`attrs`	`tuple[int, ...]`	*required*	
`capacity`	`int`	*required*	
`gamma`	`float`	*required*	
`synergy_rate`	`float`	*required*	
`decay`	`float`	*required*	
`budget_mult`	`float`	*required*	

realize_values

The value equation of design.md §2.2, evaluated for every `(bidder, slot)` pair of one stage.

```
realize_values(
    *,
    base_values: np.ndarray,
    loadings: np.ndarray,
    attrs: np.ndarray,
    beta: float,
    z: np.ndarray,
    eps: np.ndarray,
    gammas: np.ndarray,
    resale: np.ndarray | None = None,
) -> np.ndarray
```

Defined in `interlens.arena.auction.priors` · source

`ell_ij = log(B_j) + (beta / K) * (a_i . w_j) + z_i + eps_ij` and `v_ij = round(exp(ell_ij)) + round(gamma_i * R_j)`, with the result clipped at 1 so a deep negative draw cannot produce a zero- or negative-valued lot (which would make `bid / value` undefined).

Parameters

base_values : np.ndarray `(n_items,)` public whole-number catalogue base values `B_jt`. loadings : np.ndarray `(n_items, K)` public loadings `w_j`. attrs : np.ndarray `(n_bidders, K)` public attribute vectors `a_i`. beta : float Public strength of the persona term; `0` under IPV, which is what makes public facts uninformative about values there. z : np.ndarray `(n_bidders,)` realized private bidder-level shifters (already scaled by `sigma_z`). eps : np.ndarray `(n_bidders, n_items)` realized private idiosyncrasies (already scaled by `sigma_eps`). gammas : np.ndarray `(n_bidders,)` public resale weights; all zero outside INTERDEP. resale : np.ndarray | None `(n_items,)` common resale values `R_jt`, known to nobody. `None` outside INTERDEP.

Returns

np.ndarray `(n_bidders, n_items)` whole-number valuations, dtype `int64`.

Parameters

Name	Type	Default	Description
`base_values`	`np.ndarray`	*required*	
`loadings`	`np.ndarray`	*required*	
`attrs`	`np.ndarray`	*required*	
`beta`	`float`	*required*	
`z`	`np.ndarray`	*required*	
`eps`	`np.ndarray`	*required*	
`gammas`	`np.ndarray`	*required*	
`resale`	`np.ndarray | None`	`None`	

register_fact_renderer

Register the prose renderer for one fact key.

```
register_fact_renderer(
    key: str,
    fn: 'Callable[[object], str]',
    *,
    overwrite: bool = False,
) -> None
```

Defined in `interlens.arena.auction.priors` · source

Re-registering without `overwrite=True` is an error, so two prompt variants cannot silently share a key (the same fail-fast rule as `negotiation/sheets.py::register_constraint`).

Parameters

Name	Type	Default	Description
`key`	`str`	*required*	
`fn`	`'Callable[[object], str]'`	*required*	
`overwrite`	`bool`	`False`	

render_facts

Render fact values to prose lines through :data:`FACT_RENDERERS`, in `keys` order (default: the dict's own order).

```
render_facts(facts: dict, keys: tuple[str, ...] | None = None) ->
    list[str]
```

Defined in `interlens.arena.auction.priors` · source

A key with no registered renderer falls back to `"<key>: <value>"`, so a bare harness and the tests work before the scenario lane connects its templates.

Parameters

Name	Type	Default	Description
`facts`	`dict`	*required*	
`keys`	`tuple[str, ...] | None`	`None`	

rival_max_cdf

`P(max_k v_kj ⩽ x)` = the product of the rivals' CDFs, valid because `(z_k, eps_k)` are independent ACROSS bidders (they are correlated only across slots within a bidder).

```
rival_max_cdf(posteriors: 'list[RivalPosterior]', slot: int, x: float) -
    > float
```

Defined in `interlens.arena.auction.priors` · source

This is the distribution every conditional-on-winning calculation and every first-price best response reads.

Parameters

Name	Type	Default	Description
`posteriors`	`'list[RivalPosterior]'`	*required*	
`slot`	`int`	*required*	
`x`	`float`	*required*	

slot_blurb_slug

A deterministic prose-template KEY for a slot, derived from its loading vector: the dominant positive attribute, or `"balanced"` when none dominates, suffixed `"_light"` when the vector's mass is negative.

```
slot_blurb_slug(loading: np.ndarray) -> str
```

Defined in `interlens.arena.auction.priors` · source

The prose itself lives in `docs/templates/` — a slug keeps the wording out of the payload, so a prompt-wording change never edits a stored instance.

Parameters

Name	Type	Default	Description
`loading`	`np.ndarray`	*required*	

slot_name

Display name of slot `j` — `"Lot 1"`..`"Lot n"`, the token the action grammar's `item` field carries (design.md §3.2).

```
slot_name(j: int, n_items: int) -> str
```

Defined in `interlens.arena.auction.priors` · source

Uniform across single- and multi-item stages so one parser serves both.

Parameters

Name	Type	Default	Description
`j`	`int`	*required*	
`n_items`	`int`	*required*	

stage_budgets

Per-stage whole-number budgets: `budget_i = round(budget_mult_i * sum of bidder i's top-k_i values)`.

```
stage_budgets(
    values: np.ndarray,
    capacities: np.ndarray,
    budget_mults: np.ndarray,
) -> np.ndarray
```

Defined in `interlens.arena.auction.priors` · source

Deterministic given the realized values, so the budget carries no information the seat does not already have from its own value table, and a `budget_mult` below 1 makes the constraint genuinely bind on the seat's own preferred bundle (the Che-Gale subject). Budgets are replenished each stage and never carried (design.md §2.4).

Parameters

Name	Type	Default	Description
`values`	`np.ndarray`	*required*	
`capacities`	`np.ndarray`	*required*	
`budget_mults`	`np.ndarray`	*required*	

tercile_bounds

The two public tercile boundaries of `N(0, sigma^2)`: `(-0.4307*sigma, +0.4307*sigma)`.

```
tercile_bounds(sigma: float) -> tuple[float, float]
```

Defined in `interlens.arena.auction.priors` · source

A degenerate `sigma = 0` (the IPV switch) returns `(0.0, 0.0)`, so every draw renders `"typical"`.

Parameters

Name	Type	Default	Description
`sigma`	`float`	*required*	

tercile_label

Which of :data:`TERCILE_LABELS` the realization `x` falls in, against the public boundaries of `N(0, sigma^2)`.

```
tercile_label(x: float, sigma: float) -> str
```

Defined in `interlens.arena.auction.priors` · source

This is the whole content of the “unusually strong capital position this cycle” private fact: the label is private, the boundaries are public.

Parameters

Name	Type	Default	Description
`x`	`float`	*required*	
`sigma`	`float`	*required*	

references

Module `interlens.arena.auction.references`

Citation-key registry for the auction mechanism, benchmark, and collusion-metric modules.

The sibling of `negotiation/references.py` and the same shape: every algorithm and every preregistered benchmark in this package cites its primary source by a short key (e.g. `[vickrey1961]`) in the relevant docstring, and this module maps each key to the full citation plus the **exact page range** the module relies on. `design.md` §12 item 1 names the required entries; `prompt.md` requires the page numbers.

Usage — the registry is bibliography DATA (no accessor API); read it directly:

```
from interlens.arena.auction.references import REFERENCES
REFERENCES["vickrey1961"].url          # → 'https://www.jstor.org/stable/
    2977633'
str(REFERENCES["robinson1985"])        # "<citation> <url> — <note>" for a
    header line
```

Attributes

Name	Type	Summary
`REFERENCES`	dict[str, Reference]	

spec

Module `interlens.arena.auction.spec`

The frozen auction specification: item slots, bidders, per-stage draws, and the mechanism config.

The sibling of `negotiation/sheets.py::GameSpec` — one object carrying everything a repeated-auction episode needs, round-tripping through a plain JSON dict so it drops straight into an arena `Instance.payload` and back. `DealSpace` deliberately does NOT transfer: bids are effectively continuous and multi-item allocation is combinatorial, so the full-enumeration assumption in `utility_matrix`/`feasible_mask` breaks (design.md §2.1).

Structure of the object, per design.md §2.1 and §2.4:

- **persistent** across all `T` stages — :class:`ItemSlot` identities and their attribute loadings `w_j`, the five :class:`BidderSpec` personas with their PUBLIC attribute vectors `a_i`, capacities, synergy and decay rates, resale weights; the mechanism; the channel;
- **redrawn** each stage into a :class:`StageDraw` — base values `B_jt`, the private shifters `z_it`, the per-item idiosyncrasies `eps_ijt`, realized whole-number valuations, budgets, synergy target sets, resale values and signals, and the seeded tie-break permutation.

The generative model itself (the `ell_ijt` equation of design.md §2.2) lives in exactly one place, :func:`~interlens.arena.auction.priors.realize_values`; :func:`generate_spec` composes it with the persona table and the coherence permutation. Every constant of the environment (base-value range, default variances, default mechanism parameters) is defined here and nowhere else.

Example:

```
spec = generate_spec(seed=7,
     mechanism=Mechanism.sealed(pricing="second_price"),
                       value_structure="apv", n_items=1, horizon=8,
     channel="dm")
```

```
spec.stage(1).values[0]              # bidder 0's whole-number valuations
     at stage 1
AuctionSpec.from_json(spec.to_json()) == spec      # exact round-trip
```

Attributes

Name	Type	Summary
BANK_STAGES	int	
BASE_VALUE_RANGE	tuple[int, int]	
CHANNELS	tuple[str, ...]	
DEFAULT_BETA	float	
DEFAULT_BID_GRANULARITY	int	
DEFAULT_CLOCK_ROUND_CAP	int	
DEFAULT_DM_CAP	int	
DEFAULT_INCREMENT	int	
DEFAULT_RESERVE	int	
DEFAULT_SIGMA_EPS	float	
DEFAULT_SIGMA_NU	float	
DEFAULT_SIGMA_Z	float	
DEFAULT_TALK_ROUNDS	int	
DM_CHANNELS	tuple[str, ...]	
ESCROW_CHANNELS	tuple[str, ...]	
FAMILIES	tuple[str, ...]	
N_BIDDERS	int	
PRICING_BY_FAMILY	dict[str, tuple[str, ...]]	
PUBLIC_CARD_FIELDS	tuple[str, ...]	
SAA_ROUND_CAP_LARGE	int	
SAA_ROUND_CAP_SMALL	int	
SYNERGY_TARGET_LARGE_THRESHOLD	int	
SYNERGY_TARGET_SIZE_LARGE	int	

Name	Type	Summary
SYNERGY_TARGET_SIZE_SMALL	int	
TRANSFER_CHANNELS	tuple[str, ...]	
VALUE_STRUCTURES	tuple[str, ...]	

Classes

Name	Summary
AuctionSpec	A complete repeated-auction episode specification (design.md §2.1).
BidderSpec	One seat's persistent structure.
ItemSlot	One lot in the catalogue.
Mechanism	The auction format as a CONFIG, never a separate runner (design.md §3).
RingSpec	An instructed bidding ring: which seats are in it, and whether the instruction is given in the prompt.
StageDraw	One stage's realized draws — everything that redraws from the same persona priors (design.md §2.4).

Functions

Name	Summary
card_scramble_seed	The frozen scramble seed for one bank instance, derived from its instance_id.

Name	Summary
derangement	A seeded permutation of 0..n-1 with NO fixed point: perm[i] ≠ i for every i.
generate_spec	Generate a complete :class:AuctionSpec deterministically from seed.
scramble_public_cards	X1: permute the five PUBLIC CARDS across the seats under a seeded derangement, keeping every draw.
synergy_target_size	Size of the private synergy target SET at n_items lots: a pair below :data:SYNERGY_TARGET_LARGE_THR lots, a triple at or above (design.md §2.2).

AuctionSpec

A complete repeated-auction episode specification (design.md §2.1).

```
AuctionSpec(
    item_slots: tuple[ItemSlot, ...],
    attr_names: tuple[str, ...],
    bidders: tuple[BidderSpec, ...],
    horizon: int,
    stages: tuple[StageDraw, ...],
    mechanism: Mechanism,
    beta: float = DEFAULT_BETA,
    sigma_z: float = DEFAULT_SIGMA_Z,
    sigma_eps: float = DEFAULT_SIGMA_EPS,
    sigma_nu: float = DEFAULT_SIGMA_NU,
    value_structure: str = 'apv',
    channel: str = 'silent',
    dm_cap: int = DEFAULT_DM_CAP,
    talk_rounds: int = DEFAULT_TALK_ROUNDS,
    disclose_public_facts: bool = False,
    ring: RingSpec | None = None,
    framing: str = 'datacenter',
    meta: dict = dict(),
)
```

Defined in `interlens.arena.auction.spec` · source

Parameters

item_slots : tuple[ItemSlot, …] The persistent catalogue; `len` must equal `mechanism.n_items`. attr_names : tuple[str, …] Names of the `K` public attribute dimensions, aligned with every `a_i` and `w_j`. beta, sigma_z, sigma_eps, sigma_nu : float The PUBLIC structural constants of the generative model, announced in the rules so a rival can compute a genuinely informative posterior (design.md §2.2). bidders : tuple[BidderSpec, …] Exactly :data:`N_BIDDERS` seats; personas persist across stages. horizon : int `T`, the number of auction stages in the episode. `len(stages)` must equal it. stages : tuple[StageDraw, …] The frozen per-stage realizations, in stage order. mechanism : Mechanism The format config. Formats never fork a runner. value_structure : str One of :data:`VALUE_STRUCTURES`; consistency with `beta`/`sigma_z`/`gamma` is validated here so an `"ipv"` spec cannot silently carry a live persona term. channel, dm_cap, talk_rounds : str, int, int The communication affordances

(design.md §3.2, §3.4). disclose_public_facts : bool The linkage-principle switch [milgrom_weber1982]: whether the stage publishes extra public information. ring : RingSpec | None An instructed/designated ring, or `None` (the committed cells). framing : str `"datacenter"` (default) or `"neutral"` — a prompt-surface flag carried on the spec so the analysis can never pool the two by accident. meta : dict Anything scenario- or generator-private (provenance, bank position, difficulty tags).

Parameters

Name	Type	Default	Description
`item_slots`	tuple[ItemSlot, …]	*required*	
`attr_names`	`tuple[str, ...]`	*required*	
`bidders`	tuple[BidderSpec, …]	*required*	
`horizon`	`int`	*required*	
`stages`	tuple[StageDraw, …]	*required*	
`mechanism`	Mechanism	*required*	
`beta`	`float`	`DEFAULT_BETA`	
`sigma_z`	`float`	`DEFAULT_SIGMA_Z`	
`sigma_eps`	`float`	`DEFAULT_SIGMA_EPS`	
`sigma_nu`	`float`	`DEFAULT_SIGMA_NU`	
`value_structure`	`str`	`'apv'`	
`channel`	`str`	`'silent'`	
`dm_cap`	`int`	`DEFAULT_DM_CAP`	
`talk_rounds`	`int`	`DEFAULT_TALK_ROUNDS`	
`disclose_public_facts`	`bool`	`False`	
`ring`	RingSpec | None	`None`	

Name	Type	Default	Description
`framing`	`str`	`'datacenter'`	
`meta`	`dict`	`dict()`	

Attributes

Name	Type	Summary
`K`	`int`	Number of public attribute dimensions.
`attr_names`	`tuple[str, ...]`	
`attrs`	`np.ndarray`	The `(n_bidders, K)` public attribute matrix `a`.
`beta`	`float`	
`bidders`	tuple[BidderSpec, …]	
`capacities`	`tuple[int, ...]`	Per-seat capacity `k_i` (max lots won per stage).
`channel`	`str`	
`decays`	`tuple[float, ...]`	Per-seat diminishing-returns factor `d_i`.
`disclose_public_facts`	`bool`	
`dm_cap`	`int`	
`framing`	`str`	
`gammas`	`tuple[float, ...]`	Per-seat resale weight `gamma_i` (all zero outside INTERDEP).
`horizon`	`int`	
`item_slots`	tuple[ItemSlot, …]	

Name	Type	Summary
`loadings`	`np.ndarray`	The `(n_items, K)` public loading matrix `w`.
`mechanism`	Mechanism	
`meta`	`dict`	
`n_bidders`	`int`	Number of seats (always :data:`N_BIDDERS`).
`n_items`	`int`	Number of distinct lots per stage.
`ring`	RingSpec | None	
`sigma_eps`	`float`	
`sigma_nu`	`float`	
`sigma_z`	`float`	
`stages`	tuple[StageDraw, …]	
`synergy_rates`	`tuple[float, ...]`	Per-seat complementarity rate `c_i`.
`talk_rounds`	`int`	
`value_structure`	`str`	

Methods

`attribute_score`

```
attribute_score(self) -> np.ndarray
```

source

The public affinity matrix `(n_bidders, n_items)` of `a_i . w_j` — everything a rival holding only public facts knows about the shape of `i`'s valuation curve (design.md §2.2).

Zero under IPV only in effect, not in value: the matrix is still computable, but `beta = 0` makes it carry no information about realized values, which is what the IPV seat cards say.

from_json

```
from_json(d: dict) -> 'AuctionSpec'
```

source

Rebuild an :class:`AuctionSpec` from :meth:`to_json` output — an exact round-trip, so a stored instance replays the SAME auction.

Parameters

Name	Type	Default	Description
d	dict	*required*	

prefix

```
prefix(self, horizon: int) -> 'AuctionSpec'
```

source

This spec truncated to its first `horizon` stages — how a `T = 6` or `T = 8` cell consumes a bank instance that carries :data:`BANK_STAGES` draws (design.md §7.1).

A strict prefix, so the long-horizon cell is an extension of the committed cells on identical instances.

Parameters

Name	Type	Default	Description
horizon	int	*required*	

stage

```
stage(self, t: int) -> StageDraw
```

source

The stage-`t` draw, `t` 1-indexed as in design.md §3.1.

Raises `IndexError` past the horizon.

Parameters

Name	Type	Default	Description
t	int	*required*	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict of the whole spec (drops straight into `Instance.payload`).

BidderSpec

One seat's persistent structure.

```
BidderSpec(
    seat: int,
    persona_id: str,
    display_name: str,
    attrs: tuple[int, ...],
    capacity: int,
    gamma: float,
    synergy_rate: float,
    decay: float,
    budget_mult: float,
    public_fact_keys: tuple[str, ...] = (),
    private_fact_keys: tuple[str, ...] = (),
)
```

Defined in `interlens.arena.auction.spec` · source

Everything here persists across all `T` stages (design.md §2.4).

Every field except `private_fact_keys` is PUBLIC and rendered on every seat's card: the attribute vector `attrs` (entries in {-1, 0, +1}, one public fact each), `capacity` (max lots per stage), `gamma` (resale weight), `synergy_rate` (the complementarity rate `c_i` — its EXISTENCE and RATE are public while the target set is private, which is what makes exposure real), `decay` (the diminishing-returns factor `d_i`), and `budget_mult` (the multiple of its own top-capacity valuation total that the seat's per-stage budget is set to — public as a tercile label on the card, while the realized whole-number budget is private).

`public_fact_keys` / `private_fact_keys` are template KEYS, not prose: rendering data lives in `priors.py` and the prose in docs/templates/, so a prompt-wording change never edits a stored spec.

Parameters

Name	Type	Default	Description
`seat`	`int`	*required*	
`persona_id`	`str`	*required*	

Name	Type	Default	Description
`display_name`	`str`	*required*	
`attrs`	`tuple[int, ...]`	*required*	
`capacity`	`int`	*required*	
`gamma`	`float`	*required*	
`synergy_rate`	`float`	*required*	
`decay`	`float`	*required*	
`budget_mult`	`float`	*required*	
`public_fact_keys`	`tuple[str, ...]`	`()`	
`private_fact_keys`	`tuple[str, ...]`	`()`	

Attributes

Name	Type	Summary
`attrs`	`tuple[int, ...]`	
`budget_mult`	`float`	
`capacity`	`int`	
`decay`	`float`	
`display_name`	`str`	
`gamma`	`float`	
`persona_id`	`str`	
`private_fact_keys`	`tuple[str, ...]`	
`public_fact_keys`	`tuple[str, ...]`	
`seat`	`int`	
`synergy_rate`	`float`	

Methods

from_json

```
from_json(d: dict) -> 'BidderSpec'
```

source

Rebuild a :class:`BidderSpec` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
d	dict	*required*	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict.

ItemSlot

One lot in the catalogue.

```
ItemSlot(slot_id: int, name: str, blurb_slug: str, loading:
    tuple[float, ...])
```

Defined in `interlens.arena.auction.spec` · source

PERSISTS across all stages; only its base value `B_jt` redraws.

`loading` is the public attribute-loading vector `w_j` (length `K`, aligned with `AuctionSpec.attr_names`) printed numerically in the catalogue as well as narrated in `blurb`. `blurb_slug` keys the prose template the scenario lane renders (the prose lives in docs/templates/, not here), so wording changes never touch the payload.

Parameters

Name	Type	Default	Description
`slot_id`	`int`	*required*	
`name`	`str`	*required*	
`blurb_slug`	`str`	*required*	
`loading`	`tuple[float, ...]`	*required*	

Attributes

Name	Type	Summary
`blurb_slug`	`str`	
`loading`	`tuple[float, ...]`	
`name`	`str`	
`slot_id`	`int`	

Methods

from_json

```
from_json(d: dict) -> 'ItemSlot'
```

source

Rebuild an :class:`ItemSlot` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
d	dict	*required*	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict.

Mechanism

The auction format as a CONFIG, never a separate runner (design.md §3).

```
Mechanism(
    family: str,
    pricing: str,
    n_items: int = 1,
    n_units: int = 1,
    increment: int = DEFAULT_INCREMENT,
    start_price: int = DEFAULT_RESERVE,
    reserve: int = DEFAULT_RESERVE,
    round_cap: int = 1,
    activity_rule: str = 'none',
    bid_granularity: int = DEFAULT_BID_GRANULARITY,
)
```

Defined in `interlens.arena.auction.spec` · source

Parameters

family : str One of :data:`FAMILIES`; fixes the round structure and the action grammar. pricing : str What the winner pays; must be legal for `family` per :data:`PRICING_BY_FAMILY`. n_items : int Number of distinct lots auctioned per stage (1 for the single-item families). n_units : int Number of identical units for the multi-unit families (`uniform_price` / `clinching`); 1 elsewhere. increment : int Minimum whole-number bid increment, and the clock step for the ascending/descending families. start_price : int Clock start. Ascending clocks start at `reserve`; a descending (Dutch) clock starts above the maximum realized valuation, which the generator sets per stage. reserve : int Reserve price; a lot below reserve goes unsold. round_cap : int Hard cap on bidding rounds within one stage. For clock families it is set so the clock can exceed the maximum realized valuation; hitting it stamps the stage `clock_ceiling` (G1 fails a cell above 5%). activity_rule : str `"none"` or `"eligibility_ratchet"` — under the ratchet a bidder that passes on lot j in round r may not bid on j later in that stage (design.md §3.3, SAA). bid_granularity : int Bids must be multiples of this. `1` is the open channel; setting it to `increment` is the bid-rounding sub-arm that CLOSES the trailing-digit channel (design.md §8).

Parameters

Name	Type	Default	Description
`family`	`str`	*required*	
`pricing`	`str`	*required*	
`n_items`	`int`	`1`	
`n_units`	`int`	`1`	
`increment`	`int`	`DEFAULT_INCREMENT`	
`start_price`	`int`	`DEFAULT_RESERVE`	
`reserve`	`int`	`DEFAULT_RESERVE`	
`round_cap`	`int`	`1`	
`activity_rule`	`str`	`'none'`	
`bid_granularity`	`int`	`DEFAULT_BID_GRANULARITY`	

Attributes

Name	Type	Summary
`activity_rule`	`str`	
`bid_granularity`	`int`	
`family`	`str`	
`increment`	`int`	
`is_clock`	`bool`	Whether the format runs a price clock across rounds rather than one sealed round.
`is_multi_item`	`bool`	Whether a stage allocates more than one distinct lot (the combinatorial/exposure regime).

Name	Type	Summary
n_items	int	
n_units	int	
pricing	str	
reserve	int	
round_cap	int	
start_price	int	

Methods

clinching

```
clinching(n_units: int = 3, **kw={}) -> 'Mechanism'
```

source

Ausubel ascending clock on `n_units` identical units [ausubel2004, pp. 1454-1460].

Parameters

Name	Type	Default	Description
n_units	int	3	
kw		{}	

dutch

```
dutch(**kw={}) -> 'Mechanism'
```

source

Descending clock, nothing revealed between rounds — strategically equivalent to first-price [vickrey1961, pp. 20-23], which is why no separate sealed first-price cell exists.

Parameters

Name	Type	Default	Description
kw		{}	

english

```
english(**kw={}) -> 'Mechanism'
```

source

Ascending clock with public irrevocable exits; the winner pays the second-to-last exit price.

Parameters

Name	Type	Default	Description
kw		{}	

from_json

```
from_json(d: dict) -> 'Mechanism'
```

source

Rebuild a :class:`Mechanism` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
d	dict	*required*	

saa

```
saa(n_items: int, **kw={}) -> 'Mechanism'
```

source

Simultaneous ascending auction on `n_items` lots with the eligibility ratchet.

The round cap follows the lot count (3 at the small pilot rung, 5 at 20 lots) unless overridden.

Parameters

Name	Type	Default	Description
`n_items`	`int`	*required*	
`kw`		`{}`	

sealed

```
sealed(pricing: str = 'second_price', **kw={}) -> 'Mechanism'
```

source

One simultaneous sealed bidding round (design.md §3.3 row 1).

Parameters

Name	Type	Default	Description
`pricing`	`str`	`'second_price'`	
`kw`		`{}`	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict of every mechanism field.

uniform_price

```
uniform_price(n_units: int = 3, **kw={}) -> 'Mechanism'
```

source

Sealed uniform-price sale of `n_units` identical units; winners pay the highest rejected bid.

Parameters

Name	Type	Default	Description
n_units	int	3	
kw		{}	

RingSpec

An instructed bidding ring: which seats are in it, and whether the instruction is given in the prompt.

```
RingSpec(members: tuple[int, ...], instructed: bool = False)
```

Defined in `interlens.arena.auction.spec` · source

`instructed=False` records a ring the ANALYSIS designated (for a counterfactual) without telling the seats; the committed cells never instruct a ring, so this is a tail/robustness knob.

Parameters

Name	Type	Default	Description
`members`	`tuple[int, ...]`	*required*	
`instructed`	`bool`	`False`	

Attributes

Name	Type	Summary
`instructed`	`bool`	
`members`	`tuple[int, ...]`	

Methods

`from_json`

```
from_json(d: dict) -> 'RingSpec'
```

source

Rebuild a :class:`RingSpec` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
d	dict	*required*	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict.

StageDraw

One stage's realized draws — everything that redraws from the same persona priors (design.md §2.4).

```
StageDraw(
    stage: int,
    base_values: tuple[int, ...],
    z: tuple[float, ...],
    eps: tuple[tuple[float, ...], ...],
    values: tuple[tuple[int, ...], ...],
    budgets: tuple[int, ...],
    synergy_target: tuple[tuple[int, ...] | None, ...],
    tie_break: tuple[int, ...],
    clock_ceiling: int,
    resale: tuple[int, ...] | None = None,
    signals: tuple[tuple[int, ...], ...] | None = None,
)
```

Defined in `interlens.arena.auction.spec` · source

Attributes

stage : int 1-indexed stage number within the episode. base_values : tuple[int, …] `B_jt`, the PUBLIC catalogue base value per slot for this stage. z : tuple[float, …] `z_it`, the PRIVATE bidder-level shifter (its SD `sigma_z` is public). eps : tuple[tuple[float, …], …] `eps_ijt`, the PRIVATE per-(bidder, slot) idiosyncrasy (its SD `sigma_eps` is public). values : tuple[tuple[int, …], …] `v_ijt`, the PRIVATE realized whole-number valuations — the equation of design.md §2.2 evaluated and rounded. budgets : tuple[int, …] PRIVATE whole-number per-stage budget, replenished each stage (never carried; design.md §2.4). synergy_target : tuple[tuple[int, …] | None, …] PRIVATE synergy target SET per bidder (`None` for a bidder with `synergy_rate == 0`). Its size follows the lot count; its identity redraws each stage. resale : tuple[int, …] | None `R_jt`, the common resale value — known to NOBODY, INTERDEP only. signals : tuple[tuple[int, …], …] | None PRIVATE noisy resale signals `R_jt + nu_ijt` (`sigma_nu` public), INTERDEP only. tie_break : tuple[int, …] Seeded seat permutation, announced before bidding; ties are resolved by position in this list. clock_ceiling : int The price at which a clock family's round cap binds — set

above the maximum realized valuation so the ceiling is reachable only by a bidder bidding above every value in the stage.

Parameters

Name	Type	Default	Description
`stage`	`int`	*required*	
`base_values`	`tuple[int, ...]`	*required*	
`z`	`tuple[float, ...]`	*required*	
`eps`	`tuple[tuple[float, ...], ...]`	*required*	
`values`	`tuple[tuple[int, ...], ...]`	*required*	
`budgets`	`tuple[int, ...]`	*required*	
`synergy_target`	`tuple[tuple[int, ...], ...] | None`	*required*	
`tie_break`	`tuple[int, ...]`	*required*	
`clock_ceiling`	`int`	*required*	
`resale`	`tuple[int, ...] | None`	`None`	
`signals`	`tuple[tuple[int, ...], ...] | None`	`None`	

Attributes

Name	Type	Summary
`base_values`	`tuple[int, ...]`	
`budgets`	`tuple[int, ...]`	
`clock_ceiling`	`int`	
`eps`	`tuple[tuple[float, ...], ...]`	
`resale`	`tuple[int, ...] | None`	
`signals`	`tuple[tuple[int, ...], ...] | None`	

Name	Type	Summary
`stage`	`int`	
`synergy_target`	`tuple[tuple[int, ...] | None, ...]`	
`tie_break`	`tuple[int, ...]`	
`value_array`	`np.ndarray`	The realized valuations as an `(n_bidders, n_items)` integer array — the workhorse every allocation, benchmark, and metric consumes.
`values`	`tuple[tuple[int, ...], ...]`	
`z`	`tuple[float, ...]`	

Methods

`from_json`

```
from_json(d: dict) -> 'StageDraw'
```

source

Rebuild a :class:`StageDraw` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict (nested tuples become nested lists; `None` targets are preserved).

card_scramble_seed

The frozen scramble seed for one bank instance, derived from its `instance_id`.

```
card_scramble_seed(instance_id: str) -> int
```

Defined in `interlens.arena.auction.spec` · source

A content hash rather than a counter, so the derangement is a property of the INSTANCE and is identical in every rerun, on every machine, in any arm order – the scramble is frozen with the bank even though it is applied at cell time.

Parameters

Name	Type	Default	Description
`instance_id`	`str`	*required*	

derangement

A seeded permutation of `0..n-1` with NO fixed point: `perm[i] ≠ i` for every `i`.

```
derangement(n: int, seed: int) -> tuple[int, ...]
```

Defined in `interlens.arena.auction.spec` · source

Drawn by rejection from :class:`numpy.random.Generator`, which terminates almost surely (the derangement share of permutations tends to `1/e`). A fixed point would leave one seat holding its own card, which would make X1 a partial control on that seat – the one failure mode the control cannot tolerate, since G2(b) reads a cross-persona dispersion that a single un-scrambled seat would inflate.

Parameters

Name	Type	Default	Description
`n`	`int`	*required*	
`seed`	`int`	*required*	

generate_spec

Generate a complete :class:`AuctionSpec` deterministically from `seed`.

```
generate_spec(
    seed: int,
    *,
    mechanism: Mechanism,
    value_structure: str = 'apv',
    horizon: int = BANK_STAGES,
    channel: str = 'silent',
    talk_rounds: int = DEFAULT_TALK_ROUNDS,
    dm_cap: int = DEFAULT_DM_CAP,
    beta: float = DEFAULT_BETA,
    sigma_z: float = DEFAULT_SIGMA_Z,
    sigma_eps: float = DEFAULT_SIGMA_EPS,
    sigma_nu: float = DEFAULT_SIGMA_NU,
    persona_order: tuple[int, ...] | None = None,
    coherent: bool = True,
    disclose_public_facts: bool = False,
    framing: str = 'datacenter',
    ring: RingSpec | None = None,
    meta: dict | None = None,
) -> AuctionSpec
```

Defined in `interlens.arena.auction.spec` · source

The generator implements design.md §2.2 exactly, with the value equation itself living in :func:`~interlens.arena.auction.priors.realize_values` so there is one copy of it.

Parameters

seed : int The instance seed. Two calls with the same seed and the same arguments produce byte-identical JSON. mechanism : Mechanism The format config; its `n_items` fixes the catalogue size. value_structure : str `"ipv"` / `"apv"` / `"interdep"`. The switch is applied AFTER the latent draws, so the `eps` stream is identical across structures at a fixed seed — which is what makes O1 <-> O2 a within-bank paired contrast rather than two populations (design.md §7.1). horizon : int Number of stage draws to generate. Bank instances use :data:`BANK_STAGES`; a cell consumes a prefix via :meth:`AuctionSpec.prefix`. channel, talk_rounds, dm_cap : str, int, int Communication affordances, passed through to the spec. beta, sigma_z,

sigma_eps, sigma_nu : float Public structural constants; `beta` and `sigma_z` are forced to 0 under IPV. persona_order : tuple[int, …] | None Permutation of :data:`~interlens.arena.auction.priors.PERSONAS` indices onto seats 0..4. `None` draws one from `seed`, which is how persona/seat identity varies across the bank while the five archetypes stay fixed. The persona-scrambled control X1 does NOT use this: it keeps the draws and permutes the CARDS, via :func:`scramble_public_cards`, which is applied to a bank instance at cell time rather than at generation time so X1 and O1 read the SAME frozen draws. coherent : bool Apply the once-per-instance coherence permutation (design.md §2.2, the `_make_role_coherent` analogue): a persona's expected-argmax slot is never one its public attributes point away from. The permutation moves `eps` labels only, so the realized value multiset is unchanged and the draw stays RNG-neutral. disclose_public_facts, framing, ring, meta Passed through to the spec unchanged.

Returns

AuctionSpec A validated spec with `horizon` stage draws.

Parameters

Name	Type	Default	Description
`seed`	`int`	*required*	
`mechanism`	Mechanism	*required*	
`value_structure`	`str`	`'apv'`	
`horizon`	`int`	`BANK_STAGES`	
`channel`	`str`	`'silent'`	
`talk_rounds`	`int`	`DEFAULT_TALK_ROUNDS`	
`dm_cap`	`int`	`DEFAULT_DM_CAP`	
`beta`	`float`	`DEFAULT_BETA`	
`sigma_z`	`float`	`DEFAULT_SIGMA_Z`	
`sigma_eps`	`float`	`DEFAULT_SIGMA_EPS`	
`sigma_nu`	`float`	`DEFAULT_SIGMA_NU`	

Name	Type	Default	Description
persona_order	tuple[int, ...] | None	None	
coherent	bool	True	
disclose_public_facts	bool	False	
framing	str	'datacenter'	
ring	RingSpec | None	None	
meta	dict | None	None	

scramble_public_cards

X1: permute the five PUBLIC CARDS across the seats under a seeded derangement, keeping every draw.

```
scramble_public_cards(spec: AuctionSpec, *, seed: int) -> AuctionSpec
```

Defined in `interlens.arena.auction.spec` · source

What moves is the whole card as one unit – :data:`PUBLIC_CARD_FIELDS`, i.e. the persona prose (keyed by `persona_id`), the printed public attribute vector `a_i`, and the public per-seat figures beside it. Seat `i` presents as, and *is*, the organization whose card it holds: the addressable seat id, the roster entry, the "your seat" line, and every mechanic the card states (capacity limit, adjacency premium, decay) are the card's, so nothing a bidder reads contradicts anything else it reads.

What does NOT move is every seat's PRIVATE block – its realized valuations, its budget, its synergy target set, its resale signals, and the tie-break permutation – because those live on :class:`StageDraw`, which this function does not touch. **Valuations are not redrawn.** The scramble therefore breaks exactly one thing: the mapping from the public card to the valuation it used to describe. Persona text is still present at every seat (which is what controls Jia et al.'s persona-shifts-competence confound), and it is now uninformative about the draw (which is what destroys the prior's information content, the thing G2(b) tests O1 against).

The computable free arms run under this too, and their posteriors are then systematically wrong, since a rational bidder forms them from the public attribute matrix. That is the intended reading, not a defect: it prices what the public prior was worth to a bidder that used it correctly.

Parameters

spec : AuctionSpec The instance spec, already at the cell's mechanism / horizon / channel. seed : int The derangement seed; use :func:`card_scramble_seed` on the instance id so it is frozen with the bank.

Returns

AuctionSpec A new spec with permuted cards and a `meta["card_scramble"]` provenance block recording the derangement, the seed, and which fields moved.

Raises

ValueError Under `interdep`, where `gamma` selects which seat receives the stage's resale signals: moving the published resale weight away from the seat holding the signals would be a half-scramble across the public/private line, so the control is refused rather than approximated. X1 is an `apv` cell.

Example:

```
spec = scramble_public_cards(spec,
     seed=card_scramble_seed(instance.instance_id))
spec.meta["card_scramble"]["derangement"]      # e.g. [2, 3, 4, 0, 1]
```

Parameters

Name	Type	Default	Description
`spec`	AuctionSpec	*required*	
`seed`	`int`	*required*	

synergy_target_size

Size of the private synergy target SET at `n_items` lots: a pair below :data:`SYNERGY_TARGET_LARGE_THRESHOLD` lots, a triple at or above (design.md §2.2).

```
synergy_target_size(n_items: int) -> int
```

Defined in `interlens.arena.auction.spec` · source

Parameters

Name	Type	Default	Description
`n_items`	`int`	*required*	

engine

Module `interlens.arena.engine`

Episode drivers: play `Scenario` instances through `Participant`s.

The scenario is a pure state machine (it emits `SeatRequest`s and consumes text); the engine owns everything around it — driving the participant, per-episode persistence (atomic write after every applied wave), the one-retry rule, forked provisional elicitations (whose responses never enter state or any transcript), budget enforcement, and usage accounting.

Two drivers, two throughput regimes:

- `EpisodePool` — episodes as independent `asyncio` tasks over any participant (API participants are network-bound; each blocking `generate` runs in a worker thread, so the pool is fully concurrent and its width is bounded by the shared API client's `max_in_flight`). Fully async and free of shared mutable state across episodes, so it also serves as the Inspect solver's engine.
- `BatchedEpisodePool` — synchronous co-stepping for **local** model participants: each tick collects the pending requests of every live episode and runs them as ONE batched `generate_batch` per participant (the 5–20× rollout win), with adaptive batch splitting on GPU OOM.

A failed generation is never silent, and the two drivers are equally honest about it. Both drivers meet the same cuDNN/OOM faults under load; what differed was only the REPORTING. `EpisodePool` always surfaced a failure as `status="error"` with the traceback — legible, and excluded by any "done" filter. `BatchedEpisodePool` instead substituted `EMPTY_TURN_PLACEHOLDER` and swallowed the exception, and because that placeholder parses into a well-formed no-op, a cell in which *every* turn had been fabricated still reported `status="done"` and `parse_ok=True` throughout; the contamination was found months later, from the outcome numbers.

So the batched driver now recovers from a transient fault by splitting the wave and then retrying the lone request, and only if that also fails does it fab-

ricate — at which point it logs at ERROR, stamps `TurnRecord.gen_failed` with the causing exception, counts it, and RAISES `GenerationFailureBudgetExceeded` once such turns exceed `max_fabricated_fraction` of the run. A non-transient error is a bug and always propagates; it is never converted into a turn. Screen stored episodes with `gen_failures(episode)` (it also handles pre-stamp records) and read a run's totals from `BatchedEpisodePool.fabrication_report()`.

Budgets are stop conditions, not ad-hoc counters. An episode's budget is any `StopCondition` (`TokenBudget`, `CostBudget`, a list of both): the engine records each committed turn as an interlens `Message` (usage metadata included) on an internal transcript, checks the condition against it, and applies `turn_cap` to each generation. When the budget fires, the engine sets `state['budget_exhausted']` so the scenario steers to a forced finalization — the matched-compute semantics from the arena experiments (a solo baseline gets the team's median token budget, then must answer with what it has).

Spend is gated by reservation, not post-hoc. Pass a `UsageMeter` and per-job `estimated_cost`: the pool claims the estimate *before* launching each episode and settles it after, so N concurrent episodes can never collectively overrun the meter's budget (in-flight episodes finish; new ones don't start).

Attributes

Name	Type	Summary
`EFFECTIVE_CAP_KEY`		
`EMPTY_TURN_PLACEHOLDER`		
`FABRICATION_FLOOR`		
`GEN_FAILED_KEY`		
`GEN_FAILURE_KEY`		
`MAX_FABRICATED_FRACTION`		
`SINGLE_REQUEST_RETRIES`		
`TRUNCATED_STOPS`		
`TURN_SIGNATURES`		
`logger`		

Classes

Name	Summary
`BatchedEpisodePool`	Synchronous co-stepping for local model participants: each tick gathers every live episode's pending requests and runs them as one batched `generate_batch` per participant — the local-GPU throughput path.
`EpisodePool`	Concurrent episodes as independent asyncio tasks — one participant call at a time per episode, many episodes in flight.
`EpisodeRun`	Per-episode bookkeeping shared by both drivers: state stepping, turn recording, budget checks, retries, and finalization.
`GenerationFailureBudgetExceeded`	Raised by :class:`BatchedEpisodePool` when it has had to fabricate more turns than its budget allows.

Functions

Name	Summary
`gen_failures`	Every turn of an episode whose text the ENGINE fabricated because generation failed.
`truncation_budget`	The output budget one stored turn's `n_tokens_out` may honestly be compared against; `0` for "none,

Name	Summary
	so read truncation from `stop_reason` alone”.
`turn_signatures`	Every failure signature carried by one stored turn (a `TurnRecord.to_json()` dict); `set()` if healthy.

BatchedEpisodePool

Synchronous co-stepping for local model participants: each tick gathers every live episode's pending requests and runs them as one batched `generate_batch` per participant — the local-GPU throughput path.

```
BatchedEpisodePool(
    store: EpisodeStore | None = None,
    *,
    record_views: bool = True,
    max_fabricated_fraction: float = MAX_FABRICATED_FRACTION,
    fabrication_floor: int = FABRICATION_FLOOR,
    single_request_retries: int = SINGLE_REQUEST_RETRIES,
)
```

Defined in `interlens.arena.engine` · source

On CUDA OOM (or the transient cuDNN graph errors long co-stepped batches produce), the wave is split and retried down to single episodes, then that single request is retried a bounded number of times. Only if all of that fails does the pool fabricate a placeholder turn so it can keep moving — and a fabricated turn is LOGGED at error level and STAMPED (`TurnRecord.gen_failed`), because text no model produced must never be mistaken for model behaviour. Past `max_fabricated_fraction` of the pool's turns the run RAISES instead.

Parameters

store : EpisodeStore | None Where to persist each episode after every applied wave. record_views : bool Persist each turn's rendered view into its `TurnRecord` (default on). max_fabricated_fraction : float The fraction of attempted turns the engine may fabricate before raising :class:`GenerationFailureBudgetExceeded` (default :data:`MAX_FABRICATED_FRACTION`, 10%). A healthy run fabricates none, so any nonzero rate is already a defect; the ceiling exists to stop a broken run in its first seconds rather than at analysis time. Set it to `1.0` to never raise (the old behaviour) — which is only defensible if something downstream screens `gen_failed`. fabrication_floor : int Attempted turns required before the fraction

is consulted (default :data:`FABRICATION_FLOOR`), so one transient blip in a tiny run cannot trip the ceiling on a denominator of one. single_request_retries : int Re-attempts of a lone failing request after splitting has narrowed the wave to it, before fabricating (default :data:`SINGLE_REQUEST_RETRIES`).

Parameters

Name	Type	Default	Description
`store`	EpisodeStore | None	`None`	
`record_views`	`bool`	`True`	
`max_fabricated_fraction`	`float`	`MAX_FABRICATED_FRACTION`	
`fabrication_floor`	`int`	`FABRICATION_FLOOR`	
`single_request_retries`	`int`	`SINGLE_REQUEST_RETRIES`	

Attributes

Name	Type	Summary
`attempted_turns`		
`fabricated_turns`		
`fabrication_floor`		
`failures`	`list[dict]`	
`max_fabricated_fraction`		
`record_views`		
`single_request_retries`		
`store`		

Methods

fabrication_report

```
fabrication_report(self) -> dict
```

source

How much of this pool's output the engine had to fabricate: `{attempted, fabricated, fraction, failures}`.

A caller should log this at the end of a run — `fraction == 0.0` is the only healthy value, and anything else bounds how much of the run is not model behaviour.

This is the ONLY complete account of a run's fabrications, and it is why the report exists alongside the per-turn stamp. Committed turns carry `TurnRecord.gen_failed`, but a forked **provisional** probe is stored as an `OracleRecord` rather than a `TurnRecord`, so it has nowhere to carry the stamp: a fabricated provisional shows up here and in `failures` (with `phase == "provisional"`), and in the stored record only as `content == EMPTY_TURN_PLACEHOLDER`. So `fabricated` can legitimately exceed the number of `gen_failed` turns in the episodes; the difference is fabricated provisional probes, whose `score` should be treated as contaminated.

run_pool

```
run_pool(
    self,
    jobs: list[dict],
    progress: Callable[[int, int], None] | None = None,
) -> list[Episode]
```

source

Parameters

Name	Type	Default	Description
`jobs`	`list[dict]`	*required*	
`progress`	`Callable[[int, int], None] | None`	`None`	

EpisodePool

Concurrent episodes as independent asyncio tasks — one participant call at a time per episode, many episodes in flight.

```
EpisodePool(
    store: EpisodeStore | None = None,
    *,
    meter: UsageMeter | None = None,
    max_concurrent: int = 32,
    record_views: bool = True,
    refusal_ladder: RefusalLadder | None = None,
    wave_parallel: bool = True,
)
```

Defined in `interlens.arena.engine` · source

Each blocking `Participant.generate` runs in a worker thread, so hosted-API episodes are throughput-bound by the shared client's `max_in_flight` cap, not by the event loop.

`meter` (a `UsageMeter`) adds run-level spend control: jobs carrying `estimated_cost` are reservation-gated (an episode that doesn't fit under the budget is skipped, returned as `None`), and every episode re-checks the meter's `exhausted` state when it acquires its concurrency slot, so spend accumulated while it queued genuinely stops it from starting.

Parameters

Name	Type	Default	Description
`store`	EpisodeStore | None	`None`	
`meter`	UsageMeter | None	`None`	
`max_concurrent`	`int`	`32`	
`record_views`	`bool`	`True`	
`refusal_ladder`	RefusalLadder | None	`None`	

Name	Type	Default	Description
wave_parallel	bool	True	

Attributes

Name	Type	Summary
meter		
record_views		
refusal_ladder		
store		
wave_parallel		

Methods

run_episode

```
run_episode(
    self,
    scenario: Scenario,
    instance: Instance,
    arm: str,
    participant,
    *,
    seed: int = 0,
    cfg: dict | None = None,
    gen_config: dict | None = None,
    budget: StopCondition | list | None = None,
    estimated_cost: float | None = None,
    capture=None,
    steering=None,
    patch=None,
    gate: Callable[[], bool] | None = None,
    on_wave: Callable[[Episode], None] | None = None,
    prefix: tuple[dict, int | None] | None = None,
) -> Episode | None
```

source

Play one episode to completion.

Returns the `Episode` (status `done` or `error`), or `None` when the episode never started: its cost reservation didn't fit under the meter's budget, the meter was already exhausted, or `gate()` returned True. The launch gates are evaluated once the episode acquires a concurrency slot (`max_concurrent` bounds episodes in flight), so a queued episode really is stopped by spend that accumulated while it waited — in-flight episodes finish, new ones don't start.

`capture` / `steering` / `patch` are per-turn interp hooks threaded into every committed generation (local `ModelParticipant` only — API/scripted participants raise on interp requests); `capture` (a `CaptureRequest`) tags activations by this episode's turn index, so per-turn activation capture *inside* a structured episode works. Forked provisional probes are left clean (no capture/steer).

`on_wave(episode)` is an observer called with the live `Episode` after every wave is persisted and once more after `finalize()` — the seam a live viewer streams from (`arena.live`). It fires AFTER `save()` on purpose: an observer can then never show a turn that is not yet on disk, so a reader who reloads mid-episode sees at least what was streamed to them. It is an OBSERVER, not a hook — its return value is ignored and any exception it raises is logged and swallowed, because a broken viewer must not be able to kill a running episode. Leave it `None` (the default) and the episode is byte-for-byte what it was.

`prefix = (parent_episode_json, upto_turn_idx)` resumes a stored episode from mid-game: the parent's turns with `idx < upto` are replayed into the fresh state (`interlens.arena.replay.apply_prefix`) before the first generation, so this episode continues that game from that exact node. The same key on a `run_episodes`/`run_pool` job dict does the same thing.

Parameters

Name	Type	Default	Description
`scenario`	Scenario	*required*	
`instance`	Instance	*required*	
`arm`	`str`	*required*	
`participant`		*required*	

Name	Type	Default	Description
seed	int	0	
cfg	dict | None	None	
gen_config	dict | None	None	
budget	StopCondition | list | None	None	
estimated_cost	float | None	None	
capture		None	
steering		None	
patch		None	
gate	Callable[[], bool] | None	None	
on_wave	Callable[[Episode], None] | None	None	
prefix	tuple[dict, int | None] | None	None	

run_pool

```
run_pool(
    self,
    jobs: list[dict],
    stop_check: Callable[[], bool] | None = None,
) -> list[Episode]
```

source

Run many episodes concurrently (`max_concurrent` in flight).

Each job is the `run_episode` kwargs (`{scenario, instance, arm, participant, seed?, cfg?, gen_config?, budget?, estimated_cost?}`). `stop_check() → bool` and the meter's `exhausted` state are evaluated when each episode acquires its slot — not at submission — so once either fires, queued episodes are skipped while in-flight ones finish. Skipped episodes are omitted from the result.

Parameters

Name	Type	Default	Description
`jobs`	`list[dict]`	*required*	
`stop_check`	`Callable[[], bool] | None`	`None`	

EpisodeRun

Per-episode bookkeeping shared by both drivers: state stepping, turn recording, budget checks, retries, and finalization.

```
EpisodeRun(
    scenario: Scenario,
    instance: Instance,
    arm: str,
    participant,
    seed: int,
    store: EpisodeStore | None,
    *,
    cfg: dict | None = None,
    gen_config: dict | None = None,
    budget: StopCondition | list | None = None,
    capture=None,
    steering=None,
    patch=None,
    record_views: bool = True,
    prefix: tuple[dict, int | None] | None = None,
)
```

Defined in `interlens.arena.engine` · source

Driver-agnostic — it never talks to a participant itself.

Parameters

Name	Type	Default	Description
`scenario`	Scenario	*required*	
`instance`	Instance	*required*	
`arm`	`str`	*required*	
`participant`		*required*	
`seed`	`int`	*required*	
`store`	EpisodeStore | None	*required*	
`cfg`	`dict | None`	`None`	
`gen_config`	`dict | None`	`None`	

Name	Type	Default	Description
budget	StopCondition | list | None	None	
capture		None	
steering		None	
patch		None	
record_views	bool	True	
prefix	tuple[dict, int | None] | None	None	

Attributes

Name	Type	Summary
budget		
capture		
ep		
instance		
ledger		
participant		
patch		
record_views		
retries	set[tuple]	
scenario		
state		
steering		
store		

Methods

allow_retry

```
allow_retry(self, request: SeatRequest) -> bool
```

source

Parameters

Name	Type	Default	Description
request	SeatRequest	*required*	

annotate

```
annotate(self, request: SeatRequest) -> None
```

source

Run the scenario's inline oracles over the turn just committed and append their typed `OracleRecord`s to the episode's oracle log.

A no-op unless the scenario overrides `annotate_turn` (pure-Python — no extra generation), so scenarios without an oracle stack are unaffected.

Parameters

Name	Type	Default	Description
request	SeatRequest	*required*	

finalize

```
finalize(self, error: str | None = None) -> Episode
```

source

Parameters

Name	Type	Default	Description
`error`	`str | None`	`None`	

pending

```
pending(self) -> list[SeatRequest]
```

source

record_provisional

```
record_provisional(self, request: SeatRequest, message: Message, parsed,
    score) -> None
```

source

Parameters

Name	Type	Default	Description
`request`	SeatRequest	*required*	
`message`	Message	*required*	
`parsed`		*required*	
`score`		*required*	

record_turn

```
record_turn(self, request: SeatRequest, message: Message, cap: int = 0) ->
    dict | None
```

source

Commit one generated turn: strip any leaked reasoning, apply it to the scenario state, log the `TurnRecord`, accumulate usage, and check the budget.

Returns the scenario's retry directive, if any.

Parameters

Name	Type	Default	Description
`request`	SeatRequest	*required*	
`message`	Message	*required*	
`cap`	`int`	`0`	

`retry_request`

```
retry_request(request: SeatRequest, prior_text: str, retry_prompt: str) -
    > SeatRequest
```

source

Parameters

Name	Type	Default	Description
`request`	SeatRequest	*required*	
`prior_text`	`str`	*required*	
`retry_prompt`	`str`	*required*	

`save`

```
save(self) -> None
```

source

score_provisional

```
score_provisional(self, message: Message) -> tuple
```

source

Parameters

Name	Type	Default	Description
message	Message	*required*	

turn_cap

```
turn_cap(self, request: SeatRequest) -> int
```

source

The output cap for one generation: the request's own cap, shrunk by the budget's `turn_cap` (a `TokenBudget` lands the final turn on budget).

A participant-level `turn_token_floor` may raise it back — the thinking-aware tradeoff documented on `APIParticipant`.

Parameters

Name	Type	Default	Description
request	SeatRequest	*required*	

GenerationFailureBudgetExceeded

Raised by :class:`BatchedEpisodePool` when it has had to fabricate more turns than its budget allows.

```
GenerationFailureBudgetExceeded()
```

Defined in `interlens.arena.engine` · source

Inherits from: `RuntimeError`

The alternative — which this replaces — is a run that completes, reports `status="done"` and `parse_ok=True` on every turn, and contains no model behaviour whatsoever. A campaign cell has been observed at 100% fabricated turns while reporting clean. Failing loudly in the first seconds is strictly better than discovering it in the analysis, so the budget defaults to ON.

gen_failures

Every turn of an episode whose text the ENGINE fabricated because generation failed.

```
gen_failures(episode) -> list[dict]
```

Defined in `interlens.arena.engine` · source

`episode` is an :class:`~interlens.arena.schema.Episode` or its `to_json()` dict. Returns a list of `{"idx", "round", "seat", "phase", "reason", "detected_by"}`, one per fabricated turn.

Use this rather than hand-rolling a screen, because the honest test depends on when the episode was recorded:

- **v1.2 and later** carry an explicit `gen_failed` stamp (`detected_by="stamp"`), which also records the exception that caused it.
- **Older episodes** have no stamp, so they fall back to the legacy value signature (`detected_by="legacy_signature"`): content is exactly :data:`EMPTY_TURN_PLACEHOLDER`, zero output tokens, AND `raw is None`.

The legacy fallback is a conjunction, and it is worth being precise about which clause does what, because getting this wrong in either direction has already happened:

- `content == EMPTY_TURN_PLACEHOLDER` does the discriminating work. `raw is None` **alone is useless** on local runs — a local non-thinking model's raw completion equals its content, so `record_turn` stores `raw=None` for essentially every healthy turn too (measured: all 2,683 turns of a clean 32B cell).
- `raw is None` is a GUARD, not the detector. Its job is to exclude the other producer of the same placeholder: a model that genuinely returned empty or reasoning-only text, which `record_turn` also replaces with the placeholder but for which `raw` holds the non-placeholder text it actually got. That is real model behaviour (a thinking model

burning its cap) and a different problem with a different fix, so it must not be counted as an engine failure.

Because the stamp exists from v1.2 on, this fallback is only for pre-stamp records — do not build new tooling on it. And do NOT screen with `parse_ok` or “is the content empty”: the placeholder is a non-empty string that parses into a well-formed no-op action, so a fully fabricated episode passes both.

Parameters

Name	Type	Default	Description
`episode`		*required*	

truncation_budget

The output budget one stored turn's `n_tokens_out` may honestly be compared against; `0` for "none, so read truncation from `stop_reason` alone".

```
truncation_budget(turn: dict, *, cap_floor: int = 0) -> int
```

Defined in `interlens.arena.engine` · source

Two stored fields look like this number and only one is it. `cap` is what the PROTOCOL asked for, and a participant may raise it before sending — `APIParticipant` sends `max(cap, turn_token_floor)` — so on a hosted-API turn the stored `cap` can be a factor of eight below the budget the request ran under. Comparing against it manufactures truncation out of ordinary long turns: measured over the frozen five-arm Opus corpus, all 450 turns a `n_tokens_out` ⩾ `cap` screen flagged carry `stop_reason: end_turn` and *exceed* the stored cap, up to 8,545 tokens against 2,048 (0.208 reported truncation against 0.000 real).

So `effective_cap` (v1.3+) is used when present. On a pre-v1.3 record it is absent and the difference was never stored, so the fallback reserves `cap` for turns carrying **no** `stop_reason` — which is exactly the population the comparison was written for, since local `ModelParticipant` s never populate one and hosted providers always do.

`cap_floor` is the escape hatch for a legacy run whose real budget is known from outside the record (a manifest's `api_request_config.turn_token_floor`, or a `--turn-max-tokens` the stored caps understate): it raises the returned budget, so a turn only counts as a cap hit if it reached the budget the run really had.

Parameters

Name	Type	Default	Description
`turn`	`dict`	*required*	
`cap_floor`	`int`	`0`	

turn_signatures

Every failure signature carried by one stored turn (a `TurnRecord.to_json()` dict); `set()` if healthy.

```
turn_signatures(
    turn: dict,
    *,
    engine_fabricated: bool | None = None,
    cap_floor: int = 0,
) -> set[str]
```

Defined in `interlens.arena.engine` · source

:func:`gen_failures` answers one question — did the ENGINE fabricate this turn — and a run can be perfectly clean by that measure while a quarter of its turns say nothing. This separates the causes that wear the same :data:`EMPTY_TURN_PLACEHOLDER` string, because each needs a different fix:

- `placeholder` — the turn's content IS the placeholder. Always accompanied by exactly one cause below.
- `gen_failed` — the engine fabricated it (no model was ever successfully called). `engine_fabricated` is the authority when given; :func:`gen_failures` computes it, reading the v1.2 `gen_failed` stamp and falling back to the value signature only on older records. Raising the token cap CANNOT fix this.
- `empty_gen` — generation really happened but reduced to nothing after `strip_think`: the model spent its whole budget inside an unterminated `<think>` block and emitted no visible action. This IS model behaviour and the fix is a larger cap or thinking disabled. **Measured at 24.4% of turns for a thinking-on Qwen3-32B at a 2,048-token cap, against 0.0% with thinking off**, while `fabrication` correctly read 0.000 throughout — the engine did its job and a quarter of the turns were still silent.
- `truncated` — a genuine cap hit: a truncating `stop_reason`, or output that reached the turn's budget as :func:`truncation_budget` resolves it (`cap_floor` is passed straight through, for a legacy run whose real budget is only known from its manifest). The token clause is the

only one that fires on local runs, which never populate `stop_reason`, and it is deliberately NOT applied to a pre-v1.3 hosted-API turn, whose stored `cap` is the protocol's number rather than the one the request used.

- `noop_action` / `parse_failed` — behavioural symptoms reported alongside, NOT causes. `noop_action` also fires on legitimate no-op play, and on some placeholder turns does not fire at all.

Do NOT screen for any of this with `parse_ok` or "is the content empty". Both are fooled: the placeholder is a non-empty string that parses into a well-formed no-op, and on a degraded cell `parse_ok` is *anti*-correlated with turn quality (measured 1.000 on the silent arm against 0.958 on the healthy one, because placeholders parse flawlessly and real model prose sometimes does not).

Parameters

Name	Type	Default	Description
`turn`	`dict`	*required*	
`engine_fabricated`	`bool | None`	`None`	
`cap_floor`	`int`	`0`	

export

Module `interlens.arena.export`

Human-readable transcripts from stored episodes: an `EpisodeStore` tree -> one markdown + one self-contained HTML page per episode, plus a per-run index.

Each transcript shows the game setup (issues/options, every seat's private sheet + threshold, the arm / protocol / scaffold config), then every turn (seat, private scratchpad/reasoning, public cheap-talk message, the validated action + offer id, and any per-oracle regret), then the outcome + welfare. It reads a stored `Episode` JSON (`schema.Episode.to_json` shape) plus the `Instance` it was played on; it degrades gracefully on episodes recorded before the per-turn `view` field existed (renders what is there) and surfaces the rendered `view` when present.

Usage (library):

```
from interlens.arena import export
md = export.render_markdown(episode_dict, instance_dict)
export.export_run("runs/episodes", "runs/instances", "runs/transcripts")
     # → writes md+html+index
```

CLI:

```
python -m interlens.arena.export --episodes runs/episodes --instances
     runs/instances --out runs/transcripts
```

Functions

Name	Summary
`export_episode`	Write `<id>.md` and `<id>.html` for one episode into `out_dir`; returns their paths.

Name	Summary
export_run	Render every episode under episodes_path to out_dir (md + html each) plus an index.html / index.md linking them with one-line summaries.
export_transcripts	Alias of :func:export_run with a caller-friendly signature (episodes_dir, out_dir, instances_dir=).
main	
render_html	A self-contained (inline-CSS, no external assets) HTML transcript for one episode.
render_markdown	A full markdown transcript for one episode (setup header + per-turn + outcome/welfare).

export_episode

Write `<id>.md` and `<id>.html` for one episode into `out_dir`; returns their paths.

```
export_episode(episode: dict, instance: dict | None, out_dir: str | Path)
    -> dict
```

Defined in `interlens.arena.export` · source

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`instance`	`dict | None`	*required*	
`out_dir`	`str | Path`	*required*	

export_run

Render every episode under `episodes_path` to `out_dir` (md + html each) plus an `index.html` / `index.md` linking them with one-line summaries.

```
export_run(
    episodes_path: str | Path,
    instances_path: str | Path | None,
    out_dir: str | Path,
) -> dict
```

Defined in `interlens.arena.export` · source

Returns a small manifest.

Parameters

Name	Type	Default	Description
`episodes_path`	`str | Path`	*required*	
`instances_path`	`str | Path | None`	*required*	
`out_dir`	`str | Path`	*required*	

export_transcripts

Alias of :func:`export_run` with a caller-friendly signature (`episodes_dir, out_dir, instances_dir=`).

```
export_transcripts(
    episodes_dir: str | Path,
    out_dir: str | Path,
    *,
    instances_dir: str | Path | None = None,
    annotations_dir: str | Path | None = None,
) -> dict
```

Defined in `interlens.arena.export` · source

`annotations_dir` is accepted for interface stability but IGNORED — per-oracle regret is read from each episode's own inline `round_checkpoints`, so no separate annotation store is needed.

Parameters

Name	Type	Default	Description
`episodes_dir`	`str | Path`	*required*	
`out_dir`	`str | Path`	*required*	
`instances_dir`	`str | Path | None`	`None`	
`annotations_dir`	`str | Path | None`	`None`	

main

```
main(argv: list[str] | None = None) -> int
```

Defined in `interlens.arena.export` · source

Parameters

Name	Type	Default	Description
argv	list[str] | None	None	

render_html

A self-contained (inline-CSS, no external assets) HTML transcript for one episode.

```
render_html(episode: dict, instance: dict | None = None) -> str
```

Defined in `interlens.arena.export` · source

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`instance`	`dict | None`	`None`	

render_markdown

A full markdown transcript for one episode (setup header + per-turn + outcome/welfare).

```
render_markdown(episode: dict, instance: dict | None = None) -> str
```

Defined in `interlens.arena.export` · source

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`instance`	`dict | None`	`None`	

gates

Module `interlens.arena.gates`

Template-fidelity gates: preflight checks before spending GPU-hours on local-model rollouts.

Two silent failure modes cost real runs in the arena experiments, and both are cheap to gate on:

1. **Template drift** — the token ids a model is *actually* conditioned on (`apply_chat_template(tokenize=True)`) can differ from tokenizing the rendered string, e.g. when a template inserts special tokens the string round-trip re-splits. `check_template_fidelity` asserts exact token-id equality over real rendered multi-party views, for the exact id path batched generation feeds to `generate`.
2. **Reasoning leaks** — a `<think>` block from one seat's raw completion ending up inside another seat's later view (observed live when a turn truncates mid-`<think>`). `check_reasoning_leak` scans a played episode for any raw think fragment appearing in a later turn's visible content.

Run both on smoke instances before a full rollout; each returns a report dict with `ok: bool`.

Functions

Name	Summary
`check_reasoning_leak`	Scan a played episode for reasoning leakage: any turn whose raw completion contains a `<think>` block whose content then appears verbatim in a LATER turn's visible content (meaning another seat saw it).

Name	Summary
`check_template_fidelity`	Assert token-id equality between `tokenizer(apply_chat_templat` and `apply_chat_template(tokenize=True)` for every view.
`scenario_smoke_views`	Real rendered views for gating: a fresh team state's first requests, plus the views after one scripted turn (so later views carry assistant/user mixes, which is where templates diverge).

check_reasoning_leak

Scan a played episode for reasoning leakage: any turn whose raw completion contains a `<think>` block whose content then appears verbatim in a LATER turn's visible content (meaning another seat saw it).

```
check_reasoning_leak(episode: Episode | dict, *, fragment_length: int =
    80) -> dict
```

Defined in `interlens.arena.gates` · source

Accepts a live `Episode` or its stored JSON dict.

Parameters

Name	Type	Default	Description
`episode`	Episode | dict	*required*	
`fragment_length`	`int`	`80`	

check_template_fidelity

Assert
token-id equality between `tokenizer(apply_chat_template(tokenize=False))` and `apply_chat_template(tokenize=True)` for every view.

```
check_template_fidelity(tokenizer, views: list[list[dict]],
     **template_kwargs={}) -> dict
```

Defined in `interlens.arena.gates` · source

`template_kwargs` are forwarded to both template calls (e.g. `enable_thinking=True` for Qwen3, matching the generation-time configuration).

Parameters

Name	Type	Default	Description
`tokenizer`		*required*	
`views`	`list[list[dict]]`	*required*	
`template_kwargs`		`{}`	

scenario_smoke_views

Real rendered views for gating: a fresh team state's first requests, plus the views after one scripted turn (so later views carry assistant/user mixes, which is where templates diverge).

```
scenario_smoke_views(
    scenario: Scenario,
    *,
    level: int = 0,
    seed: int = 999101,
    scripted_turn: str = 'I suggest we start by sharing our goals.',
) -> list[list[dict]]
```

Defined in `interlens.arena.gates` · source

Parameters

Name	Type	Default	Description
`scenario`	Scenario	*required*	
`level`	`int`	`0`	
`seed`	`int`	`999101`	
`scripted_turn`	`str`	`'I suggest we start by sharing our goals.'`	

inspect

Package `interlens.arena.inspect`

Optional Inspect (inspect-ai) integration: run arena scenarios under `inspect eval`.

Install with the extra: `pip install interlens[inspect]`. Exposes the bundled scenarios as Inspect `Task`s — instance banks as samples, the arena engine wrapped as a custom solver (any Inspect-supported model plays every seat), and the exact scenario scorers as Inspect scorers:

```
inspect eval interlens.arena.inspect/info_relay --model anthropic/claude-
    sonnet-5
```

See `tasks.py` for the task parameters (level, arm, situational cells, communication mode, instance count).

Modules

- `adapter` — The Inspect adapter core: a Participant backed by Inspect's model, the arena solver, and the scorer.
- `tasks` — The bundled scenarios as Inspect tasks.

Re-exported

Name	Defined in	Summary
`InspectModelParticipant`	`interlens.arena.inspect.adapter`	An interlens `Participant` played by an Inspect model.
`arena_solver`	`interlens.arena.inspect.adapter`	Play one arena instance (from the sample's metadata) with the evaluated model in every seat.

Name	Defined in	Summary
coding_collab	interlens.arena.inspect.tasks	The coding-collaboration scenario as an Inspect task: 3 seats jointly write one Python module against a public pytest suite while each holds private style constraints; level sets how many constraints are dealt.
distributed_longcontext	interlens.arena.inspect.tasks	A distributed long-context task as an Inspect task.
info_relay	interlens.arena.inspect.tasks	The info-relay scenario (wrong-shard epistemics) as an Inspect task.
negotiation	interlens.arena.inspect.tasks	The negotiation scenario as an Inspect task.
scenario_scorer	interlens.arena.inspect.adapter	Score from the scenario's exact outcome (stored by the solver): success (bool) and primary (the scenario's normalized primary metric), with the full outcome dict in the score metadata.

Name	Defined in	Summary
scorable	interlens.arena.inspect.tasks	The repaired ScorableNegotiation as an Inspect task, generated through the game-preset registry (:mod:interlens.arena.negot
security_dilemma	interlens.arena.inspect.tasks	The repeated security dilemma as an Inspect task: 12 rounds of message + simultaneous build/deescalate/attack waves with noisy intelligence; level sets the first-strike bonus and the observation-noise probability.

adapter

Module `interlens.arena.inspect.adapter`

The Inspect adapter core: a Participant backed by Inspect's model, the arena solver, and the scorer.

Thin glue by design — Inspect owns model access, per-sample concurrency, retry, logging, and native token-usage tracking; interlens owns the game (the SAME `EpisodePool`/`Scenario`/`MessagingPolicy` machinery as outside Inspect — no duplicated engine). The bridge is one class: `InspectModelParticipant`, an interlens `Participant` whose `generate` posts an `inspect_ai` model call back onto the event loop. Because the arena engine already runs blocking participants in worker threads, the whole engine — episode pooling, budgets, retries, provisional forking — works under Inspect unchanged, and Inspect's own `--max-samples` concurrency runs many episodes at once (the solver holds no shared mutable state).

What the adapter adds on top of Inspect's native accounting: dollar cost per sample (`metadata['cost_usd']`, priced by `interlens.usage` — Inspect tracks tokens natively but not dollars) and the arena outcome/usage recorded in the sample store for the scorer and the viewer.

Classes

Name	Summary
`InspectModelParticipant`	An interlens `Participant` played by an Inspect model.

Functions

Name	Summary
`arena_solver`	Play one arena instance (from the sample's metadata) with the evaluated model in every seat.
`scenario_scorer`	Score from the scenario's exact outcome (stored by the solver): `success` (bool) and `primary` (the scenario's normalized primary metric), with the full outcome dict in the score metadata.

InspectModelParticipant

An interlens `Participant` played by an Inspect model.

```
InspectModelParticipant(
    name: str,
    model=None,
    *,
    loop=None,
    meter: UsageMeter | None = None,
    max_tokens: int = 2048,
    system_prompt: str | None = None,
)
```

Defined in `interlens.arena.inspect.adapter` · source

Inherits from: Participant

`generate` runs in a worker thread (that is how the engine and `Conversation` drive blocking participants), so it posts the async `model.generate` back onto the solver's event loop and blocks on the future — Inspect's connection limits and retries apply as usual. Usage telemetry lands in `Message.metadata` under the same keys every other participant uses, so budgets, metering, and episode accounting work unchanged.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`model`		`None`	
`loop`		`None`	
`meter`	UsageMeter | None	`None`	
`max_tokens`	`int`	`2048`	
`system_prompt`	`str | None`	`None`	

Attributes

Name	Type	Summary
`loop`		
`max_tokens`		
`meter`		
`model`		
`model_id`		
`name`		
`private_context`		
`requires_alternating_roles`		
`system_prompt`		

Methods

`generate`

```
generate(
    self,
    view: list[dict],
    *,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    turn: int | None = None,
    max_new_tokens: int | None = None,
    seat: str | None = None,
) -> Message
```

source

Parameters

Name	Type	Default	Description
`view`	`list[dict]`	*required*	

Name	Type	Default	Description
steering		None	
capture		None	
patch		None	
return_logprobs	bool	False	
turn	int | None	None	
max_new_tokens	int | None	None	
seat	str | None	None	

arena_solver

Play one arena instance (from the sample's metadata) with the evaluated model in every seat.

```
arena_solver(
    arm: str = 'team',
    communication: str = 'messaging',
    turn_max_tokens: int = 2048,
    messaging_turns: int = 24,
)
```

Defined in `interlens.arena.inspect.adapter` · source

The default `communication="messaging"` runs the autonomous point-to-point variant — each seat gets its private framing and the agents self-organize via `send_message`/`read_message` mailboxes with pings and the priority scheduler (recorded as transcript events), with the finalizer's fenced `{"answer"}`/`{"proposal"}` JSON scored exactly as in the protocol mode. `"round_robin"` runs the scenario's published turn protocol through the arena engine — the mode the shipped v0 transcript dataset was produced under, so use it when comparing against those cells. Scenarios with no sound messaging reduction (the security dilemma) reject messaging; their tasks pin the protocol via `Scenario.default_communication`.

Parameters

Name	Type	Default	Description
`arm`	`str`	`'team'`	
`communication`	`str`	`'messaging'`	
`turn_max_tokens`	`int`	`2048`	
`messaging_turns`	`int`	`24`	

scenario_scorer

Score from the scenario's exact outcome (stored by the solver): `success` (bool) and `primary` (the scenario's normalized primary metric), with the full outcome dict in the score metadata.

```
scenario_scorer()
```

Defined in `interlens.arena.inspect.adapter` · source

tasks

Module `interlens.arena.inspect.tasks`

The bundled scenarios as Inspect tasks.

Each task generates a solver-verified instance bank (deterministic in `seed0`), exposes it as samples (the full instance JSON + situational config in sample metadata, per-seat framings included), plays each sample with the evaluated model in every seat via `arena_solver`, and scores with the scenario's exact scorer. Episode-level budgets map to Inspect's native per-sample limits where an equivalent exists (`token_limit` carries the episode token budget so Inspect enforces and displays it); arena-specific accounting (dollar cost, per-seat usage) is enforced in the solver and reported in sample metadata/score metadata — one budget definition, two enforcement surfaces.

Run e.g.:

```
inspect eval interlens.arena.inspect/info_relay --model anthropic/claude-
     sonnet-5 -T level=2 -T cell=wrong_confident
inspect eval interlens.arena.inspect/negotiation --model openai/gpt-5 -T
     arm=solo
```

Functions

Name	Summary
`coding_collab`	The coding-collaboration scenario as an Inspect task: 3 seats jointly write one Python module against a public pytest suite while each holds private style constraints; `level` sets how many constraints are dealt.
`distributed_longcontext`	A distributed long-context task as an Inspect task.

Name	Summary
info_relay	The info-relay scenario (wrong-shard epistemics) as an Inspect task.
negotiation	The negotiation scenario as an Inspect task.
scorable	The repaired `ScorableNegotiation` as an Inspect task, generated through the game-preset registry (:mod:`interlens.arena.negotiation.ga`
security_dilemma	The repeated security dilemma as an Inspect task: 12 rounds of message + simultaneous build/deescalate/attack waves with noisy intelligence; `level` sets the first-strike bonus and the observation-noise probability.

coding_collab

The coding-collaboration scenario as an Inspect task: 3 seats jointly write one Python module against a public pytest suite while each holds private style constraints; `level` sets how many constraints are dealt.

```
coding_collab(
    level: int = 0,
    n_instances: int = 10,
    seed0: int = 1,
    arm: str = 'team',
    communication: str | None = None,
    turn_max_tokens: int = 2048,
    token_limit: int | None = None,
    messaging_turns: int = 24,
) -> Task
```

Defined in `interlens.arena.inspect.tasks` · source

Scoring runs the sandboxed test suite + AST constraint checks. The default messaging mode scores the latest complete ``python fence in the sends as the submission; pass`round_robin"` for the published protocol (the mode the shipped v0 dataset used).

Parameters

Name	Type	Default	Description
level	int	0	
n_instances	int	10	
seed0	int	1	
arm	str	'team'	
communication	str | None	None	
turn_max_tokens	int	2048	
token_limit	int | None	None	
messaging_turns	int	24	

distributed_longcontext

A distributed long-context task as an Inspect task.

```
distributed_longcontext(
    instances: str,
    task_name: str | None = None,
    n_instances: int | None = None,
    arm: str = 'team',
    communication: str | None = None,
    token_limit: int | None = None,
) -> Task
```

Defined in `interlens.arena.inspect.tasks` · source

Instances embed megabytes of context and are built offline (`interlens.arena.scenarios.dlc.build`); pass the saved bank as `-T instances=/path/to/ dlc_sniah_L0.json`. `task_name` overrides the task inferred from the bank's first instance. `communication="messaging"` runs the scenario's NATIVE directed-messaging arm (`team-msg` — each non-finalizer turn routes private fenced-JSON messages), not the generic mailbox variant, so episodes stay replayable. Per-turn caps come from the task adapter (they are part of the task definition).

Parameters

Name	Type	Default	Description
instances	str	*required*	
task_name	str | None	None	
n_instances	int | None	None	
arm	str	'team'	
communication	str | None	None	
token_limit	int | None	None	

info_relay

The info-relay scenario (wrong-shard epistemics) as an Inspect task.

```
info_relay(
    level: int = 0,
    n_instances: int = 10,
    seed0: int = 1,
    arm: str = 'team',
    communication: str | None = None,
    n_rounds: int | None = None,
    framing: str | None = None,
    honest_persona: str | None = None,
    wrong_persona: str | None = None,
    turn_max_tokens: int = 2048,
    token_limit: int | None = None,
    messaging_turns: int = 24,
) -> Task
```

Defined in `interlens.arena.inspect.tasks` · source

`communication` selects the autonomous messaging variant (the scenario default) or the published round-robin protocol (the mode the shipped v0 dataset used); the situational knobs mirror the scenario's `cfg`. `token_limit` (per sample) is Inspect's native enforcement of an episode budget.

Parameters

Name	Type	Default	Description
level	int	0	
n_instances	int	10	
seed0	int	1	
arm	str	'team'	
communication	str | None	None	
n_rounds	int | None	None	
framing	str | None	None	
honest_persona	str | None	None	
wrong_persona	str | None	None	
turn_max_tokens	int	2048	

Name	Type	Default	Description
`token_limit`	`int | None`	`None`	
`messaging_turns`	`int`	`24`	

negotiation

The negotiation scenario as an Inspect task.

```
negotiation(
    level: int = 0,
    n_parties: int | None = None,
    n_instances: int = 10,
    seed0: int = 1,
    arm: str = 'team',
    coherent: bool = True,
    communication: str | None = None,
    n_rounds: int | None = None,
    stakes: str | None = None,
    personas: str | None = None,
    turn_max_tokens: int = 2048,
    token_limit: int | None = None,
    messaging_turns: int = 24,
) -> Task
```

Defined in `interlens.arena.inspect.tasks` · source

`n_parties` switches to the sweep generator (3-8 seats, fixed deal space); otherwise the 6-party difficulty ladder at `level` (`coherent` per the role-prior table). `communication` defaults to the scenario's messaging mode; pass `"round_robin"` for the published protocol (the mode the shipped v0 dataset used). `stakes`/`personas`/`n_rounds` mirror the scenario's situational config.

Parameters

Name	Type	Default	Description
`level`	`int`	`0`	
`n_parties`	`int | None`	`None`	
`n_instances`	`int`	`10`	
`seed0`	`int`	`1`	
`arm`	`str`	`'team'`	
`coherent`	`bool`	`True`	
`communication`	`str | None`	`None`	
`n_rounds`	`int | None`	`None`	

Name	Type	Default	Description
stakes	str | None	None	
personas	str | None	None	
turn_max_tokens	int	2048	
token_limit	int | None	None	
messaging_turns	int	24	

scorable

The repaired `ScorableNegotiation` as an Inspect task, generated through the game-preset registry (:mod:`interlens.arena.negotiation.games`).

```
scorable(
    game: str = 'scorable',
    level: int = 0,
    n_instances: int = 10,
    seed0: int = 1,
    arm: str = 'moves_chat',
    turn_max_tokens: int = 2048,
    token_limit: int | None = None,
    messaging_turns: int = 24,
) -> Task
```

Defined in `interlens.arena.inspect.tasks` · source

`game` picks the preset — `scorable` (the default multi-party multi-issue game, at difficulty `level`), `ultimatum`, `divide_dollar`, or `bilateral_multiissue` — and the preset's protocol cfg (single-shot / fixed-proposer / majority) rides in each sample's `cfg` so `inspect view` renders the right protocol. The arena solver plays every seat with the evaluated model; `arm` is `moves_chat` / `moves_only` / `team` (`solo` = the single-agent control). Run e.g. `inspect eval interlens.arena.inspect/scorable --model anthropic/claude-sonnet-5 -T game=ultimatum -T arm=moves_only`.

Parameters

Name	Type	Default	Description
`game`	`str`	`'scorable'`	
`level`	`int`	`0`	
`n_instances`	`int`	`10`	
`seed0`	`int`	`1`	
`arm`	`str`	`'moves_chat'`	
`turn_max_tokens`	`int`	`2048`	
`token_limit`	`int | None`	`None`	
`messaging_turns`	`int`	`24`	

security_dilemma

The repeated security dilemma as an Inspect task: 12 rounds of message + simultaneous build/deescalate/attack waves with noisy intelligence; `level` sets the first-strike bonus and the observation-noise probability.

```
security_dilemma(
    level: int = 0,
    n_instances: int = 10,
    seed0: int = 1,
    turn_max_tokens: int = 2048,
    token_limit: int | None = None,
) -> Task
```

Defined in `interlens.arena.inspect.tasks` · source

Team arm only (the game is irreducibly 2-party), round-robin protocol only (a simultaneous-move payoff game has no sound free-messaging reduction).

Parameters

Name	Type	Default	Description
`level`	`int`	`0`	
`n_instances`	`int`	`10`	
`seed0`	`int`	`1`	
`turn_max_tokens`	`int`	`2048`	
`token_limit`	`int | None`	`None`	

live

Package `interlens.arena.live`

Live play: watch an arena episode as it happens, reconfigure its seats, and play one yourself.

The rest of the arena is run-then-inspect — episodes roll out headlessly and the visualizer renders the JSON afterwards. This package adds the interactive half: a localhost server that streams a negotiation turn by turn into the visualizer's own episode page, lets every seat be configured (and re-configured mid-game) between a model, a computable policy, an omniscient oracle and a person, and blocks a human seat on a browser form that produces a move byte-identical in form to a model's.

Three design commitments hold the whole thing together:

1. **One payload code path.** A streamed turn and a reloaded page come out of the same visualizer functions (`viz.episode._turn_payload`, `viz.page._chat_bubble`), so live and static views cannot drift.
2. **The stream follows the disk.** Turns are broadcast from an engine observer that runs after the episode is persisted, so nothing is ever shown that a reload would not find.
3. **The library stays generic.** Instance banks, framings, scaffolds and model credentials enter through a :class:`ScenarioProvider` the experiment implements; interlens never learns what a framing is.

Usage:

```
from interlens.arena.live import run_live_server
run_live_server(MyProvider(), port=8080, run_dir="results/live")
```

Stdlib only (`http.server` + server-sent events), like the rest of the visualizer.

Modules

- `assets` — The live pages' browser layer, as Python strings — same convention as `viz.assets`.
- `events` — The live-play wire protocol: every server-sent event a session can emit, in one place.
- `human` — `HumanParticipant`: a seat played by a person in a browser.
- `lobby_page` — The lobby: choose a game and decide who plays each seat.
- `payload` — Per-turn slices of the visualizer payload, for streaming one turn at a time.
- `play_page` — The live episode page: the visualizer's episode view, plus the controls to play in it.
- `provider` — The seam between the live server and whatever experiment supplies its games.
- `router` — `LiveSeatRouter`: a seat table whose occupants can change while the episode is running.
- `server` — The HTTP surface: a stdlib `ThreadingHTTPServer` serving the lobby, the live page, and the event stream.
- `session` — `LiveSession`: one live game — the engine thread, the event log, and everything the browser can do to it.
- `style` — The rules the lobby and the play page both need: form controls, seat cards, and the two docks.

Re-exported

Name	Defined in	Summary
`BankInfo`	`interlens.arena.live.provider`	One instance bank the lobby may draw a game from.
`HumanParticipant`	`interlens.arena.live.human`	A negotiation seat whose moves come from a browser.

Name	Defined in	Summary
LiveSeatRouter	interlens.arena.live.router	A :class:~interlens.arena.table.SeatRouter that supports mid-episode occupant changes.
LiveSession	interlens.arena.live.session	One live episode plus its subscribers.
ModelInfo	interlens.arena.live.provider	One model the lobby may offer for an llm seat.
PendingRequest	interlens.arena.live.human	One open ask of a human seat — what the browser renders a form from, and what the server validates a submission against.
PreparedGame	interlens.arena.live.provider	One playable game, fully assembled — the provider's answer to a lobby configuration.
SEAT_KINDS	interlens.arena.live.provider	
ScenarioProvider	interlens.arena.live.provider	What an experiment must supply for its games to be playable live.
SeatConfig	interlens.arena.live.provider	What the lobby says should sit in one seat — the unit of both initial configuration and a mid-game swap.
SessionManager	interlens.arena.live.session	The server's session registry. v1 holds ONE

Name	Defined in	Summary
		active session at a time.
`make_live_server`	`interlens.arena.live.server`	Bind (but do not run) the live-play server over `provider`.
`run_live_server`	`interlens.arena.live.server`	Serve live play until interrupted — bind, print the banner (including the `ssh -L` line, reusing `viz.serve.serve_banner`'s form), then block in `serve_forever`.

assets

Package `interlens.arena.live.assets`

The live pages' browser layer, as Python strings — same convention as `viz.assets`.

Only the LIVE parts live here. The chart, hover cards, transcript cards, sidebar and page shell are imported from `viz.assets` unchanged: a live page is the episode page plus an update path, and reimplementing any of the visualizer in a second copy is how the two would start disagreeing about what the same episode looks like.

No external assets, no build step, no CDN — the pages must work on a cluster node behind a firewall.

Modules

- `js_live` — The live page's browser layer: subscribe, merge, redraw, and take the player's move.
- `js_lobby` — The lobby's browser layer: edit the seat lineup, then start the game.

Re-exported

Name	Defined in	Summary
`JS_LIVE`	`interlens.arena.live.assets.js_live`	
`JS_LOBBY`	`interlens.arena.live.assets.js_lobby`	

js_live

Module `interlens.arena.live.assets.js_live`

The live page's browser layer: subscribe, merge, redraw, and take the player's move.

The merge rule is one line and worth stating exactly, because everything else follows from it: a `turn_appended` event's `turn` is PUSHED onto `PAYLOAD.turns` — the array is mutated in place, never replaced — and then the existing draw functions are called again. The episode page already re-renders entirely from the in-memory `PAYLOAD` (that is how its seat and oracle selectors work), so a live page is that same page with a growing array. Nothing about the chart, the regret bars, the hover cards or the transcript cards is reimplemented here.

Per turn: push the row, insert the server-rendered `bubble_html` into the chat pane, `drawChart()` and `drawRegret()`, wire the ONE new turn card, and re-mount the sidebar (it snapshots turns at mount, and a re-mount is cheap enough to be the honest fix).

Reconnects are handled by `EventSource`, which resends `Last-Event-ID` by itself; the session replays its log from there, so a dropped connection costs nothing. A page that has been away long enough to be unsure re-fetches `/state` instead of trusting an incremental merge.

The one merge case an append cannot express. Three fields on a payload row — `published`, `offer_id` and `standing_deal_index` — are properties of a turn's POSITION IN THE SEQUENCE, and a retried turn retroactively flips an EARLIER row's `published` to `False` (`viz.episode.public_ledger`, and see `live.payload .turn_delta`, which re-derives the ledger over the whole accumulated list on the server). The `turn_appended` event carries the new row only, so corrections to earlier rows do not ride along. The client therefore detects the one situation that produces them — an arriving turn occupying a (round, phase, seat) slot some earlier row already holds, which is what a retry is — and reloads the page, which re-renders server-side from `/state` and is correct by construction. Reloading rather than patching is deliberate: the chat bubbles are server-rendered, so there is no client-side renderer to

rebuild a corrected transcript with, and inventing one is exactly the second bubble renderer this design exists to avoid. Retries are rare (one per seat, round and phase at most) and the reload is rate-limited so a pathological run cannot spin.

Why the episode wiring is here rather than imported. `viz.assets.js_episode` wires the static page's draw functions but does its per-card binding inside `drawTurns`, where a live page cannot reach it to wire ONE arriving card. The wiring below is that module's, restructured around :js:func:`wireTurnCard` so a full draw and a single append share it. Everything it draws with — `frontierChart`, `turnCard`, `mountSidebar`, `regretChart`, the hover and counterfactual cards — is imported unchanged from `viz.assets`.

Owned by lane D.

Attributes

Name	Type	Summary
`JS_LIVE`		

js_lobby

Module `interlens.arena.live.assets.js_lobby`

The lobby's browser layer: edit the seat lineup, then start the game.

Every edit is POSTed to `/api/lobby` and the server's response is what the page re-renders from — the server owns the configuration, the page never keeps its own copy. That is what makes a second browser tab on the lobby show the same lineup instead of a private one, and it means the validation that matters (does this model exist, does it accept this thinking mode) happens where the provider is.

`/api/start` returns the session id; the page then navigates to `/play`, which is rendered server-side from the session's snapshot.

Three things are worth knowing before reading the script:

It edits controls, it does not build them. Seat cards are rendered once, in Python (`lobby_page._seat_card`). This layer sets values, toggles `disabled`, and rebuilds two option lists that genuinely depend on another choice — the thinking modes a newly picked model allows, and the instances a newly picked bank contains. When an edit changes the page's SHAPE (a bank with a different party count, so a different number of cards) it reloads and lets the server render it. A JavaScript copy of the card markup is the fast way to end up with two lobbies that disagree about what a seat is.

Field names live in one map. :js:data:`SEAT_FIELDS` and :js:data:`LOBBY_FIELDS` are the only place this file spells a state key, and they are exactly the `data-field` / `data-lobby` attributes the markup carries and exactly `SeatConfig`'s dataclass fields. The lobby test pins all three to each other, so a renamed field fails a test instead of silently editing nothing.

Defaults are computed, never typed. Which model a new LLM seat opens on and which thinking mode it takes come from `defaultModelId` / `defaultThinking`, mirroring `provider.default_model_id` / `provider.default_thinking` — the provider flags its default model, so no model id is spelled in this file. The "all model seats" row writes those same fields into many seats at once

(`applyAll`) and then saves through the ordinary whole-seats POST, which is why a bulk edit needs no wire change of its own.

Only the tab that pressed Start follows the game. The event stream REPLAYS its whole log to every new subscriber (that is what makes a reconnect lossless), so a lobby opened while a game is running hears the old `episode_started` as if it had just happened. Navigating on it would bounce that tab straight to `/play` and put the lobby — and its “End the session” button — out of reach for as long as the game lasts. So the redirect is gated on :js:data:`startedHere`, set by this page’s own Start click; every other tab stays on the lobby and shows the running banner (`lobby_page._running_banner`, rendered on every load and unhidden by `paint`) with its link to the live page.

Validation here is a courtesy. `validate` mirrors the server’s two rules so Start is disabled before it is clicked rather than after — the click that costs money should not be the thing that discovers the budget cap is missing. The server checks again and is the one that refuses.

Owned by lane C.

Attributes

Name	Type	Summary
`JS_LOBBY`		
`JS_LOBBY_PAGE`		

events

Module `interlens.arena.live.events`

The live-play wire protocol: every server-sent event a session can emit, in one place.

This module is the SINGLE SOURCE OF TRUTH for what goes over the wire. The server never builds an event dict by hand and the browser never reads a literal event name — both go through the constants and builders here — so the protocol cannot drift between the two halves of a feature written by different people at the same time.

Transport is plain SSE over `GET /api/session/{sid}/events`. A frame is:

```
id: <seq>
event: <type>
data: <one-line JSON>
(blank line)
```

`seq` is a monotonic per-session sequence number over ALL events the session emitted, and the session keeps the emitted frames in a log, so a browser that reconnects sends `Last-Event-ID: <seq>` and is replayed everything after it. That, not a heartbeat, is what makes a reload mid-episode lossless. `data` is always ONE line (`json.dumps` with no newlines) because a bare newline inside `data` would be read as a field break by the EventSource parser and split one event into two.

Builders return `(event_type, data_dict)` rather than a formatted frame: the session stamps the sequence number when it appends to its log (that is where the ordering is decided), and :func:`format_sse` does the formatting.

Attributes

Name	Type	Summary
`AWAITING_HUMAN`		
`EPISODE_DONE`		

Name	Type	Summary
EPISODE_STARTED		
ERROR		
EVENT_TYPES		
HELLO		
INPUT_REJECTED		
KEEPALIVE_FRAME		
KEEPALIVE_SECONDS		
LEGAL_ACTION_DEFAULTS		
LOBBY_STATE		
SEAT_SWAPPED		
SSE_HEADERS		
TURN_APPENDED		
TURN_STARTED		
UNKNOWN_TURN_IDX		
USAGE		

Functions

Name	Summary
awaiting_human	A human seat is blocked on input — everything the control dock renders from.
episode_done	The episode ended.
episode_started	An episode began.
error	Something failed.
format_sse	Format one event as an SSE frame, ready to write to the socket.
hello	Stream opened.

Name	Summary
`input_rejected`	A human submission was refused before it reached the engine.
`lobby_state`	The lobby's full configuration: the chosen instance bank, framing and instance, the per-seat configs (`SeatConfig.to_json()` dicts, in seat order) and the session's spend cap in dollars.
`seat_swapped`	A seat changed occupant between turns.
`turn_appended`	A committed, persisted turn.
`turn_started`	A seat is being asked to move.
`usage`	Spend so far against the cap.

awaiting_human

A human seat is blocked on input — everything the control dock renders from.

```
awaiting_human(
    seat: str,
    seat_idx: int,
    turn_idx: int,
    round_: int,
    phase: str,
    state: dict,
    sheet: dict,
    legal: dict,
    deadline: int,
) -> tuple[str, dict]
```

Defined in `interlens.arena.live.events` · source

`state` is the machine-readable `negotiation_state` block parsed out of the seat's own view (the offer registry, the standing offer, the round), so the UI reads the same state the seat was conditioned on rather than scraping the prompt. `sheet` is that seat's PRIVATE score sheet (`{values, threshold}`) — it is private to this seat and to this browser, which is the point of playing it. `legal` is the verdict on what may be submitted now (:data:`LEGAL_ACTION_DEFAULTS` names the keys): the server validates anyway, but the form should not offer a move that will be refused. `deadline` is the game's total round count `T`, for the "round r of T" the human is deciding under.

`legal` is NORMALIZED here — every key in :data:`LEGAL_ACTION_DEFAULTS` is present in the emitted event, missing ones taking their "not available" default. So the browser can read `legal.can_reject` directly instead of guarding for undefined, and a capability a caller forgot to compute fails CLOSED (the control is not offered) rather than rendering a button that 400s. Extra keys are passed through untouched, so a later move type does not need this builder changed to reach the page.

`turn_idx` is the one field a PARTICIPANT cannot fill in. The engine passes `turn` to `generate` only alongside a capture request (`EpisodePool._generate_once` puts it in the kwargs inside the `if capture is not None` branch), and live play

never captures — so a human seat does not know the episode's turn index and publishes :data:`UNKNOWN_TURN_IDX`. The SESSION, which counts the turns it has streamed, stamps the real index before the frame goes out, exactly as it does for `turn_started`. Anything else publishing this event owes the same fix-up: a negative `turn_idx` reaching the browser would point the dock's "your move on turn N" at a turn that does not exist.

Parameters

Name	Type	Default	Description
`seat`	`str`	*required*	
`seat_idx`	`int`	*required*	
`turn_idx`	`int`	*required*	
`round_`	`int`	*required*	
`phase`	`str`	*required*	
`state`	`dict`	*required*	
`sheet`	`dict`	*required*	
`legal`	`dict`	*required*	
`deadline`	`int`	*required*	

episode_done

The episode ended.

```
episode_done(status: str, outcome: dict, final: dict | None = None) ->
    tuple[str, dict]
```

Defined in `interlens.arena.live.events` · source

`status` is the episode's own (`"done"` / `"error"`) or a live-play reason (`"stopped"`, `"budget_stopped"`); `outcome` is the scenario's outcome record; `final` is an optional closing payload slice (scores, the agreed deal) for the summary strip without a second fetch.

Parameters

Name	Type	Default	Description
`status`	`str`	*required*	
`outcome`	`dict`	*required*	
`final`	`dict | None`	`None`	

episode_started

An episode began.

```
episode_started(episode_id: str) -> tuple[str, dict]
```

Defined in `interlens.arena.live.events` · source

The client fetches `/api/session/{sid}/state` for the initial payload rather than receiving it here — the first payload carries the whole game geometry and is far too large for one frame.

Parameters

Name	Type	Default	Description
`episode_id`	`str`	*required*	

error

Something failed.

```
error(message: str, fatal: bool = False) -> tuple[str, dict]
```

Defined in `interlens.arena.live.events` · source

`fatal=True` means the session cannot continue and the page should stop waiting for turns; `False` means one action failed and the game is still live.

Parameters

Name	Type	Default	Description
`message`	`str`	*required*	
`fatal`	`bool`	`False`	

format_sse

Format one event as an SSE frame, ready to write to the socket.

```
format_sse(seq: int, event: str, data: dict) -> bytes
```

Defined in `interlens.arena.live.events` · source

`seq` becomes the frame's `id:` (the value a reconnecting client echoes in `Last-Event-ID`), `event` its type — one of the constants above — and `data` its JSON body, dumped with `ensure_ascii=False` because the transcript is UTF-8 prose that would otherwise triple in size as escapes. The body is one line by construction: `json.dumps` escapes every newline inside a string, and a literal newline in `data` would be read as a field break and split one event into two. `default=str` keeps a stray non-JSON value (a Path on a provenance field) from taking the stream down mid-game.

Returns bytes because this is written straight to a socket, not to a text stream.

Parameters

Name	Type	Default	Description
`seq`	`int`	*required*	
`event`	`str`	*required*	
`data`	`dict`	*required*	

hello

Stream opened.

```
hello(sid: str, seq: int, phase: str, occupants: dict) -> tuple[str, dict]
```

Defined in `interlens.arena.live.events` · source

`sid` is the session, `seq` the last sequence number the session has emitted (0 before anything), `phase` where the session is (`"lobby"` | `"running"` | `"awaiting_human"` | `"done"`), and `occupants` the current seat -> occupant-label map so a fresh page can badge seats immediately.

Parameters

Name	Type	Default	Description
`sid`	`str`	*required*	
`seq`	`int`	*required*	
`phase`	`str`	*required*	
`occupants`	`dict`	*required*	

input_rejected

A human submission was refused before it reached the engine.

```
input_rejected(seat: str, reason: str) -> tuple[str, dict]
```

Defined in `interlens.arena.live.events` · source

`reason` is shown verbatim to the player, so it must say what was wrong in their terms. Nothing was enqueued: the seat is still waiting.

Parameters

Name	Type	Default	Description
`seat`	`str`	*required*	
`reason`	`str`	*required*	

lobby_state

The lobby's full configuration: the chosen instance bank, framing and instance, the per-seat configs (`SeatConfig.to_json()` dicts, in seat order) and the session's spend cap in dollars.

```
lobby_state(
    bank: str,
    framing: str,
    instance_id: str,
    seats: list[dict],
    budget_usd: float,
) -> tuple[str, dict]
```

Defined in `interlens.arena.live.events` · source

Parameters

Name	Type	Default	Description
`bank`	`str`	*required*	
`framing`	`str`	*required*	
`instance_id`	`str`	*required*	
`seats`	`list[dict]`	*required*	
`budget_usd`	`float`	*required*	

seat_swapped

A seat changed occupant between turns.

```
seat_swapped(
    seat: str,
    seat_idx: int,
    from_: str | None,
    to: str,
    at_turn: int,
) -> tuple[str, dict]
```

Defined in `interlens.arena.live.events` · source

`from_`/`to` are occupant labels (`None` from-side if the outgoing occupant was unlabelled) and `at_turn` is the turn index the new occupant starts at. JSON keys are `from`/`to`; the parameter carries an underscore only because `from` is a keyword.

Parameters

Name	Type	Default	Description
`seat`	`str`	*required*	
`seat_idx`	`int`	*required*	
`from_`	`str | None`	*required*	
`to`	`str`	*required*	
`at_turn`	`int`	*required*	

turn_appended

A committed, persisted turn.

```
turn_appended(
    turn: dict,
    bubble_html: str,
    rounds_used: int,
    outcome_partial: dict | None = None,
) -> tuple[str, dict]
```

Defined in `interlens.arena.live.events` · source

`turn` is the payload turn dict — byte-identical to what a full `episode_payload` rebuild would produce for it, oracle annotations and occupant included — which is what lets the client push it straight onto `PAYLOAD.turns`. `bubble_html` is that turn's chat bubble, server-rendered by the same function the static page uses. `outcome_partial` is the episode's outcome so far when the scenario has one (a closed deal mid-episode), else `None`.

Parameters

Name	Type	Default	Description
`turn`	`dict`	*required*	
`bubble_html`	`str`	*required*	
`rounds_used`	`int`	*required*	
`outcome_partial`	`dict | None`	`None`	

turn_started

A seat is being asked to move.

```
turn_started(
    turn_idx: int,
    round_: int,
    phase: str,
    seat: str,
    seat_idx: int,
    occupant: str | None,
) -> tuple[str, dict]
```

Defined in `interlens.arena.live.events` · source

`turn_idx` is the index the turn WILL take, `round_`/`phase` locate it in the scenario, and `occupant` is who is answering (`None` when the seat's occupant is unlabelled). Trailing underscore on `round_` only because `round` is a builtin; the JSON key is `round`.

Parameters

Name	Type	Default	Description
`turn_idx`	`int`	*required*	
`round_`	`int`	*required*	
`phase`	`str`	*required*	
`seat`	`str`	*required*	
`seat_idx`	`int`	*required*	
`occupant`	`str | None`	*required*	

usage

Spend so far against the cap.

```
usage(
    tokens_in: int,
    tokens_out: int,
    cost_usd: float,
    cap_usd: float | None,
    exhausted: bool,
) -> tuple[str, dict]
```

Defined in `interlens.arena.live.events` · source

`cap_usd` is `None` for a session with no metered seat; `exhausted` means the meter has stopped the run, which the client should read as the episode being over.

Parameters

Name	Type	Default	Description
`tokens_in`	`int`	*required*	
`tokens_out`	`int`	*required*	
`cost_usd`	`float`	*required*	
`cap_usd`	`float | None`	*required*	
`exhausted`	`bool`	*required*	

human

Module `interlens.arena.live.human`

`HumanParticipant`: a seat played by a person in a browser.

The person is a participant like any other. `Participant.generate` is synchronous and the engine already runs it in `asyncio.to_thread` (`arena/engine.py`), so a seat that blocks for two minutes while somebody decides what to offer blocks a worker thread and nothing else — no async plumbing, no engine change.

The split of responsibility is deliberate and is the thing to preserve:

- the PARTICIPANT parses the state out of its own view, publishes an `awaiting_human` event describing what may legally be done, and blocks on a queue;
- the SERVER validates the submitted form and assembles the message, because validation needs the deal space and the offer registry and belongs on the side that can answer a POST with a 400;
- the message the server enqueues goes through `arena.actions.action_message` — the SAME renderer LLM seats' output is parsed back out of — so a human turn is byte-identical in form to a model turn. Nothing downstream (the scenario parser, the oracles, the visualizer, an exported dataset row) can tell them apart except by the `occupant` stamp, which is exactly the property that makes a mixed human/model game measurable.

Never enqueue an unvalidated submission: the engine reads empty content as a well-formed no-op turn (it substitutes `EMPTY_TURN_PLACEHOLDER`), so a mistyped form would silently become the player passing.

Owned by lane A.

Attributes

Name	Type	Summary
`FINAL_PROPOSAL`		

Name	Type	Summary
`FINAL_VOTE`		
`TURN`		
`logger`		

Classes

Name	Summary
`HumanParticipant`	A negotiation seat whose moves come from a browser.
`PendingRequest`	One open ask of a human seat — what the browser renders a form from, and what the server validates a submission against.
`SessionStopped`	Raised inside a blocked :meth:`HumanParticipant.generate` when the session is stopped.

Functions

Name	Summary
`build_human_message`	Turn a validated browser form into the message a human seat plays — the assembly half of the split above.
`legal_actions`	What this seat may legally submit right now, as `{can_accept, can_reject, can_offer, can_walk, can_pass}` — the form's rules and the server's validation, computed once from one state.

Name	Summary
`phase_of`	The scenario phase this state describes: :data:`FINAL_VOTE` on the forced-final ballot, :data:`FINAL_PROPOSAL` on the turn that tables it, else an ordinary :data:`TURN`.
`sheet_json`	A seat's private score sheet as the dock renders it: `{agent, values, threshold}`.

HumanParticipant

A negotiation seat whose moves come from a browser.

```
HumanParticipant(
    name: str,
    seat: int,
    sheet: Any,
    space: Any,
    deadline: int,
    publisher: Callable[[str, dict], None],
)
```

Defined in `interlens.arena.live.human` · source

Inherits from: Participant

Parameters

name : str Identifier within the conversation, and the `human:<name>` occupant label's detail. seat : int This seat's index into the game's seat-indexed sheets/tables. sheet : object This seat's PRIVATE score sheet (`.utility`/`.surplus`/`.threshold`) — shown in the player's dock, and shown to nobody else. A human plays under the same information a model seat has. space : DealSpace The shared deal space: the issue/option name table the offer builder is generated from, and the decoder (`DealSpace.parse`) a submitted deal is validated with. deadline : int Total rounds `T`, so the dock can say which round of how many is being played. publisher : callable `publisher(event_type, data) → None` — how the participant announces that it is waiting. Injected rather than reaching for the session, so the participant is testable with a list.append.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`seat`	`int`	*required*	
`sheet`	`Any`	*required*	

Name	Type	Default	Description
`space`	`Any`	*required*	
`deadline`	`int`	*required*	
`publisher`	`Callable[[str, dict], None]`	*required*	

Attributes

Name	Type	Summary
`deadline`		
`name`		
`occupant`	`str`	This seat's occupant label, `human:<name>` — stamped on every message the person plays so the transcript records WHO took the turn, not just which seat it was.
`others_role`		
`pending`	PendingRequest | None	The open ask, or `None` when this seat is not waiting.
`private_context`		
`publisher`		
`seat`		
`self_role`		
`sheet`		
`space`		
`system_prompt`		

Methods

generate

```
generate(
    self,
    view: list[dict],
    *,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    turn: int | None = None,
    max_new_tokens: int | None = None,
    seat: str | None = None,
) -> Message
```

source

Ask the person for this turn and block until they answer.

Parses the `negotiation_state` block out of `view`, builds a :class:`PendingRequest`, publishes `awaiting_human`, then blocks on the inbox with NO timeout — a human thinking is not an error, and a deadline that fired mid-decision would fabricate a turn nobody played. Returns the `Message` the server assembled, whose metadata carries `action`, `occupant` (`human:<name>`) and `human_note`.

Raises on any interp request, exactly as `PolicyParticipant` does: there is no model here to steer, capture, patch or read logprobs from, and silently ignoring the request would corrupt an experiment.

`turn_idx` on the published request is :data:`events.UNKNOWN_TURN_IDX`: the participant does not know the episode's turn count (the engine passes `turn` only alongside a capture request), and the SESSION does — it stamps the real index on the event before broadcasting, the same way it supplies `turn_idx` for `turn_started`, which the router cannot know either.

Parameters

Name	Type	Default	Description
`view`	`list[dict]`	*required*	

Name	Type	Default	Description
steering		None	
capture		None	
patch		None	
return_logprobs	bool	False	
turn	int | None	None	
max_new_tokens	int | None	None	
seat	str | None	None	

publish

```
publish(self, request: PendingRequest) -> None
```

source

Announce that this seat is waiting, through the injected publisher.

A publisher that raises must not take the game down with it — the person can still be asked in another browser tab, and a broadcast failure is a UI problem, not a negotiation one — so the exception is logged and swallowed, matching how the engine treats its wave observer.

Parameters

Name	Type	Default	Description
request	PendingRequest	*required*	

submit

```
submit(self, message: Message) -> None
```

source

Hand the server-assembled message to the blocked :meth:`generate`.

Called from the HTTP thread.

Refuses when the seat is not waiting: a message queued against a closed prompt would sit in the inbox and play itself on some LATER turn, under a state the player never saw.

Parameters

Name	Type	Default	Description
`message`	Message	*required*	

`unblock`

```
unblock(self, reason: str = 'stopped') -> None
```

source

Release a blocked :meth:`generate` without a move — the stop path.

Feeds a sentinel that makes `generate` raise, so a stopped session's episode ends as an error rather than as a fabricated pass.

Parameters

Name	Type	Default	Description
`reason`	`str`	`'stopped'`	

PendingRequest

One open ask of a human seat — what the browser renders a form from, and what the server validates a submission against.

```
PendingRequest(
    seat: str,
    seat_idx: int,
    turn_idx: int,
    round: int,
    phase: str,
    state: Any = None,
    view: list[dict] = list(),
    legal: dict = dict(),
    block: dict = dict(),
)
```

Defined in `interlens.arena.live.human` · source

Held by the participant while it blocks and mirrored into the `awaiting_human` event. Carrying the parsed state (rather than re-parsing the prompt on submit) is what guarantees the form's legality rules and the server's validation are computed from the same state the seat was conditioned on.

Parameters

seat : str The seat display name being asked to move. seat_idx : int Its index into the game's seat-indexed sheets/tables. turn_idx : int The index the resulting turn will take. round : int The negotiation round this move belongs to. phase : str The scenario phase (proposal, voting, …) — decides which actions are legal. state : Any The `NegotiationState` reconstructed from the seat's view, including the live offer registry. view : list[dict] The exact view the seat was conditioned on, kept so the dock can show the human the prompt they are answering rather than a paraphrase. legal : dict `{can_accept: [offer_ids], can_reject: [offer_ids], can_offer: bool, can_walk: bool, can_pass: bool}` for this moment — see :func:`legal_actions`. block : dict The raw `negotiation_state` JSON the state was reconstructed from. Kept alongside the typed state because `state` holds live `ScoreSheet`/`DealSpace` objects and the wire needs plain JSON; carrying the block the seat actually read (rather than

re-serializing the typed state) means the browser and the seat are looking at the same bytes.

Parameters

Name	Type	Default	Description
`seat`	`str`	*required*	
`seat_idx`	`int`	*required*	
`turn_idx`	`int`	*required*	
`round`	`int`	*required*	
`phase`	`str`	*required*	
`state`	`Any`	`None`	
`view`	`list[dict]`	`list()`	
`legal`	`dict`	`dict()`	
`block`	`dict`	`dict()`	

Attributes

Name	Type	Summary
`block`	`dict`	
`legal`	`dict`	
`phase`	`str`	
`round`	`int`	
`seat`	`str`	
`seat_idx`	`int`	
`state`	`Any`	
`turn_idx`	`int`	
`view`	`list[dict]`	

Methods

to_json

```
to_json(self) -> dict
```

source

The wire form carried in `awaiting_human` (the view is omitted — it is fetched with the snapshot).

SessionStopped

Raised inside a blocked :meth:`HumanParticipant.generate` when the session is stopped.

```
SessionStopped()
```

Defined in `interlens.arena.live.human` · source

Inherits from: `RuntimeError`

Deliberately an exception rather than a fabricated pass: a stopped session's episode must end as an error, because "the player was still deciding when the server shut down" and "the player chose to do nothing" are different facts and only one of them is behaviour.

build_human_message

Turn a validated browser form into the message a human seat plays — the assembly half of the split above.

```
build_human_message(
    form: dict,
    *,
    name: str,
    space: Any,
    pending: PendingRequest,
) -> Message
```

Defined in `interlens.arena.live.human` · source

`form` is the POST body (`{action, deal, offer_id, message, note}`); `space` decodes a named deal; `pending` supplies the legality rules the submission is checked against. Builds the typed `Action`, renders it with `arena.actions.action_message` so the envelope matches an LLM seat's exactly, and stamps `occupant`/`human_note` metadata for the turn record.

Raises `ValueError` with a player-readable message for anything illegal — an unknown option name, an accept referencing a dead offer, an action the phase does not allow, an empty submission. The caller answers that with a 400 and an `input_rejected` event; nothing is enqueued.

Parameters

Name	Type	Default	Description
`form`	`dict`	*required*	
`name`	`str`	*required*	
`space`	`Any`	*required*	
`pending`	PendingRequest	*required*	

legal_actions

What this seat may legally submit right now, as `{can_accept, can_reject, can_offer, can_walk, can_pass}` — the form's rules and the server's validation, computed once from one state.

```
legal_actions(state: NegotiationState) -> dict
```

Defined in `interlens.arena.live.human` · source

It mirrors `ScorableNegotiation._PHASE_ALLOWED` exactly, because the scenario is the authority and a form that offers a disallowed move burns a real turn on a legality error:

- an ordinary **turn** allows everything: propose, accept/reject any live offer, walk, or stand pat;
- the forced-final **proposal** turn allows propose / accept / walk but NOT reject (the scenario reads a reject here as an economic-legality violation), and not a pass either — this turn tables the last binding package or there is nothing to vote on;
- the forced-final **vote** allows accept / reject of THE offer under vote only (`state.standing`, which the scenario pins to its `final_offer`) or a walk.

`can_accept`/`can_reject` are lists of offer ids in registration order, so the dock renders the ballot in the order the offers were tabled.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

phase_of

The scenario phase this state describes: :data:`FINAL_VOTE` on the forced-final ballot, :data:`FINAL_PROPOSAL` on the turn that tables it, else an ordinary :data:`TURN`.

```
phase_of(state: NegotiationState) -> str
```

Defined in `interlens.arena.live.human` · source

Read off the same two fields the policies read (`must_vote` and `round > deadline`) so a human seat and a policy seat agree about what turn they are playing.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

sheet_json

A seat's private score sheet as the dock renders it: `{agent, values, threshold}`.

```
sheet_json(sheet: Any) -> dict
```

Defined in `interlens.arena.live.human` · source

Sent to exactly one browser — the person playing this seat — which is the whole point of a private sheet.

Parameters

Name	Type	Default	Description
`sheet`	`Any`	*required*	

lobby_page

Module `interlens.arena.live.lobby_page`

The lobby: choose a game and decide who plays each seat.

Built on `viz.page._document`, the same shell the episode pages use, so the lobby inherits the visualizer's CSS, its light/dark handling and its layout without a second stylesheet to keep in step. Pure string-building like the rest of the visualizer: this module renders, it never fetches or mutates.

The one thing the lobby must get right is not offering a configuration that will fail. Model capabilities differ in ways that are hard errors rather than degradations — the Claude-5 models reject a temperature outright, Fable cannot turn thinking off, Haiku refuses adaptive thinking — so the seat cards are generated FROM each model's declared `thinking_modes` and `supports_temperature` rather than from a fixed set of controls. A budget cap is required before any metered seat can start, for the same reason: it is the only thing standing between a click and an unbounded bill.

A seat nobody has configured opens on the provider's own defaults — the model it flagged (:func:`~interlens.arena.live.provider.default_model_id`) with thinking on wherever that model allows it (:func:`~interlens.arena.live.provider.default_thinking`) — and the "all model seats" row (:func:`_all_seats_row`) sets a whole lineup at once. No model id is spelled anywhere in this module. The :func:`_shuffle_bar` button randomizes who plays which party — seat position is not neutral, since the proposer order rotates from the proposer base — and shows the permutation the server applied.

Every control is rendered here, in Python, exactly once. The browser layer changes the VALUES of controls that already exist and rebuilds only option lists (the thinking modes a newly picked model allows, the instances a newly picked bank contains); it never builds a seat card. When an edit changes the shape of the page — a bank with a different party count, so a different number of seat cards — the browser reloads and this function renders it again. A second copy of the card markup in JavaScript would be

the obvious way to do it and the fast way to end up with two lobbies that disagree about what a seat is.

Owned by lane C.

Attributes

Name	Type	Summary
`CSS_LOBBY`		
`LOBBY_RULES`		
`NO_INSTRUCTION_KINDS`		
`SEAT_KIND_LABELS`		

Functions

Name	Summary
`render_lobby_html`	The complete lobby page.

render_lobby_html

The complete lobby page.

```
render_lobby_html(state: dict) -> str
```

Defined in `interlens.arena.live.lobby_page` · source

`state` is `SessionManager.lobby_state()`: the provider's listings (`banks`, `framings`, `models`, `policies`) plus the current selection (`bank`, `framing`, `instance_id`, `seats`, `budget_usd`) and whether a session is already running (`running`, `sid`). Self-contained — inline CSS and JS, no network fetches on load — so it opens the same way the exported visualizer pages do.

Every key is read defensively: a provider with no banks, a state recorded before a key existed, or a bank whose party count is not yet known all render a page that says so rather than raising. A lobby that 500s is a lobby nobody can use to fix the configuration that broke it.

Parameters

state : dict The lobby state described above. Embedded verbatim into the document as an inert JSON script tag, which is what the browser layer edits and POSTs back — so the page needs no fetch to become interactive.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

payload

Module `interlens.arena.live.payload`

Per-turn slices of the visualizer payload, for streaming one turn at a time.

A live page and a static page must show the same thing, and the only way to be sure of that is for both to come out of the same code. So there is no live payload builder: :func:`turn_delta` calls the visualizer's own `viz.episode._turn_payload` and :func:`bubble_html` its own `viz.page._chat_bubble`, which are exactly the per-turn units `episode_payload` and `_chat_bubbles` are built from. A streamed turn is therefore byte-identical to the row a reload rebuilds, and a drift between the two is not a bug that can happen — it would require the shared function to disagree with itself.

Everything episode-scoped (the geometry, the seat kinds, the seat->party map) is computed ONCE per session and passed in, because rebuilding a game's geometry on every turn is the one thing here expensive enough to notice.

Owned by lane B.

Functions

Name	Summary
`bubble_html`	The chat bubble for one turn, server-rendered by the visualizer's own `_chat_bubble`.
`turn_delta`	Append one committed turn's payload row to `rows` and return it — what a `turn_appended` event carries.

bubble_html

The chat bubble for one turn, server-rendered by the visualizer's own `_chat_bubble`.

```
bubble_html(payload: dict, turn: dict) -> str
```

Defined in `interlens.arena.live.payload` · source

Rendered server-side rather than reimplemented in JS for the same reason the static page renders it there: there would otherwise be two bubble renderers to keep in step, and the browser copy would be the one that quietly went stale.

`payload` must carry BOTH the seat table and the transcript so far (`{"seats": [...], "turns": [...]}`), and `turn` should already be among those turns. The seats are the obvious half; the turns are the half that is easy to skip and silently wrong to omit. `_chat_bubble` derives each seat's DEFAULT occupant from the transcript and badges a turn only when it DEPARTS from that default — so against an empty `turns` no seat has a known default, every badge suppresses itself, and a transcript renders as though no seat had ever changed hands. That is not a degraded bubble, it is the swap badge silently doing nothing, which is the one thing a live transcript shows that a static one cannot. Passing the accumulated rows is also what makes the bubble byte-identical to the one a reload rebuilds, since a reload derives the same map from the same rows.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	
`turn`	`dict`	*required*	

turn_delta

Append one committed turn's payload row to `rows` and return it — what a `turn_appended` event carries.

```
turn_delta(
    episode: dict,
    turn: dict,
    rows: list[dict],
    *,
    geometry: Any = None,
    kinds: dict | None = None,
    oracles: dict | None = None,
    seat_party: dict | None = None,
) -> dict
```

Defined in `interlens.arena.live.payload` · source

`episode` is the live episode's `to_json()` (read for the seat list), `turn` the stored turn dict to render, and `rows` the session's ACCUMULATED payload rows, appended to in place. `geometry` is the session's prebuilt `GameGeometry`, `kinds` the `viz.episode.seat_kinds` result, `oracles` the episode's per-turn oracle records and `seat_party` the seat-name -> party-index map: all four are episode-scoped and computed once at session start rather than per turn.

The accumulated list is required, not a convenience. Three fields on a row — `published`, `offer_id` and `standing_deal_index` — are properties of a turn's POSITION IN THE SEQUENCE rather than of the turn (`viz.episode.public_ledger` derives them), and a retried turn retroactively flips an earlier row's `published` to False. So the ledger is re-derived over the whole accumulated list after each append: a few dozen rows of pure Python, next to an engine turn that just spent seconds in a model call. Skipping it is the one way a streamed row can differ from the row a reload rebuilds.

Views are never reconstructed here (replaying the episode once per turn would be absurd on a live path), so a turn the engine did not store a view for is reported `view_source="absent"` rather than reconstructed — the honest answer, and the same one `episode_payload(reconstruct=False)` gives.

`kinds`, `oracles` and `seat_party` are derived from `episode` when omitted, so a caller with nothing cached still gets a correct row. Pass the episode-scoped

ones (`kinds`, `seat_party`, `geometry`) for the live path; `oracles` is the exception that is normally left to default, since a turn's oracle records are written by the engine as the turn is committed and so do not exist before it.

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`turn`	`dict`	*required*	
`rows`	`list[dict]`	*required*	
`geometry`	`Any`	`None`	
`kinds`	`dict | None`	`None`	
`oracles`	`dict | None`	`None`	
`seat_party`	`dict | None`	`None`	

play_page

Module `interlens.arena.live.play_page`

The live episode page: the visualizer's episode view, plus the controls to play in it.

This is deliberately the EXISTING episode page and not a new one. The body is assembled from the same fragments the static page uses (`viz.page._chat_bubbles`/`_sidebar`/`_game_cards`/`_issue_pane`, `viz.chrome`'s top-bar and summary strip) inside the same `viz.page._document` shell, and the same chart/transcript/hover JS runs on the same `PAYLOAD` object. What live play adds is a live UPDATE path (`assets/js_live.py`) and two docks; everything a reader already knows how to read stays exactly where it was.

Rendering server-side from a snapshot, rather than booting an empty page that fetches, is what makes a reload mid-episode land on the full transcript immediately and then attach to the stream — no flash of an empty game, no divergence between what was rendered and what is being streamed.

The two docks:

- the HUMAN CONTROL DOCK — an offer builder generated from the deal space (one selector per issue, so an unrepresentable deal cannot be expressed), accept/reject/walk/pass buttons enabled from the server's own legality verdict, a public message box, a private scratchpad recorded as `TurnRecord.human_note`, and the seat's PRIVATE score sheet with its values and threshold. The player negotiates under exactly the information a model seat has, which is the only way a human turn is comparable to a model one.
- the SWAP DOCK — reassign any seat mid-game.

Everything on both docks that the moment decides — which offers may be accepted, which buttons are legal, what the package under construction is worth — is filled by the browser from the `awaiting_human` event. What the GAME decides — the issues, the options, the sheet — is rendered here, once, so the page does not rearrange itself when a turn becomes the player's.

Owned by lane D.

Attributes

Name	Type	Summary
`SEAT_KIND_LABELS`		

Functions

Name	Summary
`render_live_html`	The complete live page for a session.

render_live_html

The complete live page for a session.

```
render_live_html(snapshot: dict) -> str
```

Defined in `interlens.arena.live.play_page` · source

`snapshot` is `LiveSession.snapshot()`: `{seq, payload, phase, awaiting, occupants, lobby}`. The `payload` is a full `viz.episode_payload` — the same object the exported page is built from — so the page is a correct static rendering of the game so far even before its JS runs, and `seq` is the sequence number the page then subscribes from so nothing is missed between render and attach.

Parameters

Name	Type	Default	Description
`snapshot`	`dict`	*required*	

provider

Module `interlens.arena.live.provider`

The seam between the live server and whatever experiment supplies its games.

The server knows how to run a negotiation live — stream it, block a seat on a browser, swap an occupant — and knows NOTHING about instance banks, framings, scaffolds, oracle stacks or which models an experiment can afford to call. All of that enters through :class:`ScenarioProvider`, which the experiment implements (for the rational agents work, `experiments/rational_agents/live_play.py` implements it over `run.py`'s existing assembly helpers).

That direction of dependency is the whole design: an experiment's scenario assembly is intricate, versioned and changes with the research, and copying any of it into the library would create a second copy that goes stale. The provider hands over already-assembled objects, so interlens never learns what a "framing" is.

Nothing here does any work. These are the shapes the four implementation lanes agree on.

Attributes

Name	Type	Summary
`SEAT_KINDS`		
`THINKING_PREFERENCE`		

Classes

Name	Summary
`BankInfo`	One instance bank the lobby may draw a game from.

Name	Summary
`ModelInfo`	One model the lobby may offer for an `llm` seat.
`PreparedGame`	One playable game, fully assembled — the provider's answer to a lobby configuration.
`ScenarioProvider`	What an experiment must supply for its games to be playable live.
`SeatConfig`	What the lobby says should sit in one seat — the unit of both initial configuration and a mid-game swap.

Functions

Name	Summary
`default_model_id`	Which model a seat that has just become an `llm` seat is pre-selected to.
`default_thinking`	The thinking mode a seat on this model starts in: the first of :data:`THINKING_PREFERENCE` the model accepts, falling back to its first declared mode if it accepts none of them (a provider free to invent mode names must still get a mode that model can take).

BankInfo

One instance bank the lobby may draw a game from.

```
BankInfo(
    bank_id: str,
    label: str,
    instance_ids: tuple[str, ...] = (),
    n_parties: int | None = None,
    description: str = '',
)
```

Defined in `interlens.arena.live.provider` · source

Parameters

bank_id : str Stable identifier passed back to :meth:`ScenarioProvider.prepare` (for the rational-agents launcher, the instances directory name, e.g. `"instances_realistic_demo"`). label : str Display name. instance_ids : tuple[str, …] The instances in the bank, in a stable order. The lobby offers a pick from these plus “random”. n_parties : int | None Seat count, when every instance in the bank shares one (the usual case) — the lobby builds that many seat cards before an instance is chosen. `None` for a mixed bank, where the cards wait for :meth:`prepare`. description : str One line about what the bank contains, shown under the picker.

Parameters

Name	Type	Default	Description
`bank_id`	`str`	*required*	
`label`	`str`	*required*	
`instance_ids`	`tuple[str, ...]`	`()`	
`n_parties`	`int | None`	`None`	
`description`	`str`	`''`	

Attributes

Name	Type	Summary
`bank_id`	`str`	
`description`	`str`	
`instance_ids`	`tuple[str, ...]`	
`label`	`str`	
`n_parties`	`int | None`	

Methods

`to_json`

```
to_json(self) -> dict
```

source

The lobby’s wire form.

ModelInfo

One model the lobby may offer for an `llm` seat.

```
ModelInfo(
    model_id: str,
    label: str,
    provider: str,
    thinking_modes: tuple[str, ...] = ('off',),
    supports_temperature: bool = True,
    available: bool = True,
    unavailable_reason: str | None = None,
    metered: bool = True,
    default: bool = False,
)
```

Defined in `interlens.arena.live.provider` · source

Parameters

model_id : str The provider's id, exactly as a participant factory wants it (`"claude-fable-5"`, a HF repo id, …). label : str What the lobby shows a human. Free text. provider : str Which backend serves it (`"anthropic"`, `"openai"`, `"local"`, …) — drives the "needs an API key" warning and whether the seat is metered. thinking_modes : tuple[str, …] The extended-thinking settings this model actually accepts, in the order the lobby should show them (e.g. `("off", "on")`, or `("on",)` for a model that cannot turn thinking off). The lobby offers ONLY these: the Claude-5 family rejects several combinations outright (Fable cannot disable thinking, Haiku refuses adaptive thinking and effort levels), and a 400 from a live seat mid-game is a wasted session. supports_temperature : bool Whether a temperature may be sent at all. False for the Claude-5 models, which hard-error on any temperature — so the lobby hides the control rather than sending a default and failing. available : bool Whether this model can be used right now (API key present, local weights on disk / a GPU visible). An unavailable model is still LISTED — greyed out with `unavailable_reason` — because silently omitting it looks like the model does not exist. unavailable_reason : str | None Why not, in the user's terms (`"ANTHROPIC_API_KEY is not set"`). `None` when available. metered : bool Whether turns from this model cost money and must count against the

session's budget cap. default : bool Whether this is the model a new `llm` seat starts on. The PROVIDER owns that choice — it is the half that knows which model this experiment actually wants played — so the lobby never spells a model id of its own. Flag at most one; :func:`default_model_id` resolves several (or none) by availability and list order rather than raising, since a lobby is the wrong place to discover a misconfigured flag.

Parameters

Name	Type	Default	Description
`model_id`	`str`	*required*	
`label`	`str`	*required*	
`provider`	`str`	*required*	
`thinking_modes`	`tuple[str, ...]`	`('off',)`	
`supports_temperature`	`bool`	`True`	
`available`	`bool`	`True`	
`unavailable_reason`	`str | None`	`None`	
`metered`	`bool`	`True`	
`default`	`bool`	`False`	

Attributes

Name	Type	Summary
`available`	`bool`	
`default`	`bool`	
`label`	`str`	
`metered`	`bool`	
`model_id`	`str`	
`provider`	`str`	
`supports_temperature`	`bool`	
`thinking_modes`	`tuple[str, ...]`	

Name	Type	Summary
unavailable_reason	str | None	

Methods

to_json

```
to_json(self) -> dict
```

source

The lobby's wire form (see :func:~interlens.arena.live.events.lobby_state).

PreparedGame

One playable game, fully assembled — the provider's answer to a lobby configuration.

```
PreparedGame(
    instance: Any,
    scenario: Any,
    game: Any,
    arm: str,
    deadline: int,
    cfg: dict = dict(),
    seat_names: tuple[str, ...] = (),
    instance_json: dict = dict(),
    scaffold: Any = None,
    manifest: dict = dict(),
)
```

Defined in `interlens.arena.live.provider` · source

Everything the session needs to construct a table and run an episode, and nothing about WHO plays: the seats are configured separately (:class:`SeatConfig`) so the same prepared game can be replayed with a different lineup, and so a mid-game swap has the game objects already in hand.

Parameters

instance : Any The `Instance` to play, already framed (the provider applies the framing before returning). scenario : Any The `Scenario` object (a `ScorableNegotiation` for negotiation play), with its oracle stack attached. game : Any The `GameSpec` behind the instance — the source of the private score sheets, the deal space and the discount. The human dock reads its seat's sheet from here, and computable seats are bound against it. arm : str The arm the episode is played under, recorded on it and passed to `scenario.make_state`. REQUIRED, and deliberately without a default: the scenario validates it and a value it does not know is fatal on the first wave (`ScorableNegotiation` accepts `moves_chat` / `moves_only` / `team` / `solo`, or any `team`-prefixed variant, and raises `unknown arm` otherwise). There is no arm every scenario would accept, so any default here would be a value that happens to work for one scenario and kills the session for the next —

better to make the provider name one. `moves_chat` is the usual choice for live negotiation, since a live game with no talk channel gives a human nothing to do but submit moves. deadline : int Total rounds `T`. REQUIRED for the same reason: it is bound into every computable seat's concession schedule (`policy_seat(deadline=...)`) and shown to the human as the round they are deciding under, so a deadline that disagrees with the game is not a cosmetic default but a table playing to the wrong clock — and silently, unlike a bad `arm`. Read it off the game (`spec.rounds`) rather than restating it. cfg : dict The episode `cfg` (cell id and sweep configuration) passed to `run_episode`. seat_names : tuple[str, …] Seat display names in seat order (`arena.schema.PERSONAS` for negotiation). The routing key. instance_json : dict `instance.to_json()` — what the visualizer's geometry is built from, kept beside the object so the session does not re-serialize it once per turn. scaffold : Any The framing/scaffold record, when the provider has one, for provenance on the page. Optional. manifest : dict Run-level provenance to hand the page (invocation, oracle list, seat kinds), shaped like a run's `manifest.json` so the visualizer reads it with the code it already has.

Parameters

Name	Type	Default	Description
`instance`	`Any`	*required*	
`scenario`	`Any`	*required*	
`game`	`Any`	*required*	
`arm`	`str`	*required*	
`deadline`	`int`	*required*	
`cfg`	`dict`	`dict()`	
`seat_names`	`tuple[str, ...]`	`()`	
`instance_json`	`dict`	`dict()`	
`scaffold`	`Any`	`None`	
`manifest`	`dict`	`dict()`	

Attributes

Name	Type	Summary
arm	str	
cfg	dict	
deadline	int	
game	Any	
instance	Any	
instance_json	dict	
manifest	dict	
scaffold	Any	
scenario	Any	
seat_names	tuple[str, ...]	

ScenarioProvider

What an experiment must supply for its games to be playable live.

```
ScenarioProvider()
```

Defined in `interlens.arena.live.provider` · source

Inherits from: `Protocol`

Implement it anywhere — it is a structural protocol, so no import of interlens is needed to satisfy it — and hand an instance to :func:`~interlens.arena.live.run_live_server`. The listing methods drive the lobby and are called on every lobby render (keep them cheap or cache them); `prepare` and `build_model_seat` are called when a game starts and when a seat is swapped.

Methods

build_model_seat

```
build_model_seat(
    self,
    model_id: str,
    *,
    thinking: str = 'off',
    meter: Any = None,
    extra_instructions: str = '',
) -> Any
```

source

Construct the participant for one `llm` seat and return it.

`thinking` is one of that model's `thinking_modes`; `meter` is the session's `UsageMeter`, which the participant must charge so the budget cap actually binds; `extra_instructions`, when non-empty, is appended to the participant's `private_context` as one labelled segment.

The returned participant must be SAFE TO OWN: the session may set per-seat instructions on it and may hold several seats on the same model at once, so a provider that caches participants by model id has to return a wrapper or copy rather than a shared object — otherwise one seat's private instructions leak into another's view.

Parameters

Name	Type	Default	Description
`model_id`	`str`	*required*	
`thinking`	`str`	`'off'`	
`meter`	`Any`	`None`	
`extra_instructions`	`str`	`''`	

`list_banks`

```
list_banks(self) -> list[BankInfo]
```

source

The instance banks the lobby may draw from, in display order.

May be empty (the lobby then says so rather than offering a broken picker).

`list_framings`

```
list_framings(self) -> list[dict]
```

source

The framings/skins available, as `{"framing_id", "label", "description"}` dicts.

A framing rewrites an instance's surface story (issue names, party names, the cover narrative) without touching its payoff structure, so it is chosen independently of the bank.

list_models

```
list_models(self) -> list[ModelInfo]
```

source

Every model an `llm` seat may be assigned, INCLUDING ones that are currently unavailable (each carrying its own reason).

Availability is reported, never silently filtered — see :class:`ModelInfo`.

prepare

```
prepare(
    self,
    bank: str,
    framing: str,
    instance_id: str | None,
    overrides: dict | None = None,
) -> PreparedGame
```

source

Assemble one playable game.

`bank` and `framing` are ids from the listings; `instance_id` picks an instance from the bank, and `None` means “choose one” (the provider decides how — random, or the bank’s first). `overrides` is the lobby’s free-form extra configuration (seed, deadline, difficulty, oracle selection) — a dict rather than fixed parameters because what is tunable is the experiment’s business, not the server’s.

Raises `ValueError` for an unknown bank/framing/instance, which the server turns into a 400 with the message shown in the lobby.

Parameters

Name	Type	Default	Description
bank	str	*required*	

Name	Type	Default	Description
framing	str	*required*	
instance_id	str | None	*required*	
overrides	dict | None	None	

SeatConfig

What the lobby says should sit in one seat — the unit of both initial configuration and a mid-game swap.

```
SeatConfig(
    kind: str = 'llm',
    model_id: str | None = None,
    policy: str | None = None,
    thinking: str = '',
    instructions: str = '',
    display_name: str = '',
)
```

Defined in `interlens.arena.live.provider` · source

The same dataclass travels in three directions (lobby form -> server, server -> participant construction, server -> browser as part of `lobby_state`), so a seat is described exactly once.

Parameters

kind : str One of :data:`SEAT_KINDS`. model_id : str | None For `kind="llm"`: which model (a :class:`ModelInfo` `model_id`). Ignored otherwise. policy : str | None For `kind="rational"` / `"oracle"`: the policy name (a key of `arena.table.POLICY_FACTORIES`, e.g. `"bayes-rational"`). The kind, not the policy, decides whether the seat gets full-information tables. thinking : str For `kind="llm"`: the extended-thinking mode, restricted to the model's `thinking_modes`. The empty string means "whatever this model's default is" (:func:`default_thinking`, i.e. thinking on wherever the model allows it) and is what an unconfigured seat carries — spelled as its own value rather than as a literal mode because `off` has to keep meaning a deliberate `off`, and a seat that had thinking turned off by hand must not have it turned back on by a re-validation. instructions : str Extra PRIVATE instructions for this seat, appended to its `private_context` as one labelled segment. This is how a live operator gives one seat a persona or a hidden agenda without editing a scaffold. Empty string means none. Meaningless for computable seats (the lobby greys the field out) since a policy does not read prose. display_name :

str Who the transcript says is playing — the `human:<name>` label for a human seat, and the occupant badge for any other. Empty string means “derive it from the kind and model/policy”.

Parameters

Name	Type	Default	Description
`kind`	`str`	`'llm'`	
`model_id`	`str | None`	`None`	
`policy`	`str | None`	`None`	
`thinking`	`str`	`''`	
`instructions`	`str`	`''`	
`display_name`	`str`	`''`	

Attributes

Name	Type	Summary
`display_name`	`str`	
`instructions`	`str`	
`kind`	`str`	
`model_id`	`str | None`	
`policy`	`str | None`	
`thinking`	`str`	

Methods

`from_json`

```
from_json(cls, d: dict) -> 'SeatConfig'
```

source

Rebuild from a browser-supplied dict, ignoring keys this version does not know (the same forward-compatible rule `TurnRecord.from_json` follows).

Does NOT validate — the session validates, because it is the half that knows which models and policies actually exist.

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

occupant_detail

```
occupant_detail(self) -> str
```

source

The half of :meth:`occupant_label` after the colon — the model, policy or person.

Exposed separately for one reason: a `HumanParticipant` stamps `human:<its own name>` on the turns it plays (it knows who is at the keyboard; the table only knows who it was told about), so the session MUST construct it with this exact string or the seat would report two different players for the same person.

occupant_label

```
occupant_label(self) -> str
```

source

The `kind:detail` string stamped on every turn this seat plays (`TurnRecord.occupant`).

One function so the label the router stamps, the badge the transcript draws and the swap event's `from`/`to` are all the same string — an occupant timeline

assembled from three spellings of the same seat would be unreadable. `display_name` overrides the detail when set.

The prefix is the seat's ROLE rather than its `kind` verbatim (`llm` -> `api:`, `rational` -> `policy:`), matching the vocabulary the rest of the arena already uses for seat kinds. The detail defaults to whatever identifies that role: the model id, the policy name, the player's name.

resolved

```
resolved(self, models: Any) -> 'SeatConfig'
```

source

This seat with the choices nobody made filled in from the offered models — the ONE place defaults land.

For an `llm` seat: an empty `model_id` becomes :func:`default_model_id` (the provider's flagged model), and an empty `thinking` becomes that model's :func:`default_thinking` (thinking on wherever it is allowed). Any other seat, and any field already set, is returned untouched — including a deliberate `off`, which is why the unset value is `""` rather than a mode name.

Called by the session on every lobby edit, every start and every swap, so a client that posts a bare `{"kind": "llm"}` gets the same seat the lobby page would have shown it. Idempotent: resolving a resolved config returns it unchanged (the same object, in fact).

Parameters

Name	Type	Default	Description
`models`	`Any`	*required*	

to_json

```
to_json(self) -> dict
```

source

The wire form the lobby page and `lobby_state` events carry.

default_model_id

Which model a seat that has just become an `llm` seat is pre-selected to.

```
default_model_id(models: Any) -> str
```

Defined in `interlens.arena.live.provider` · source

The provider decides, by flagging one :class:`ModelInfo` with `default=True` — the lobby must not know the name of anybody's favourite model, and an id hardcoded in the page would be a second place to edit every time the frontier moves. Preference order, ties broken by list order: the flagged model if it can be used, then any model that can be used, then the flagged one anyway, then the first entry. Pre-selecting a model whose key is missing over one that would actually run is the one case where honouring the flag helps nobody, and the flag still wins when NOTHING is usable, so a lobby with no credential still opens on the right name (and on the "no Anthropic credential" reason next to it). `""` when there is nothing to offer.

Parameters

Name	Type	Default	Description
`models`	`Any`	*required*	

default_thinking

The thinking mode a seat on this model starts in: the first of :data:`THINKING_PREFERENCE` the model accepts, falling back to its first declared mode if it accepts none of them (a provider free to invent mode names must still get a mode that model can take).

```
default_thinking(model: Any) -> str
```

Defined in `interlens.arena.live.provider` · source

`model` is a :class:`ModelInfo` or its wire dict; a missing model resolves to `"off"`, the only mode every backend has.

Parameters

Name	Type	Default	Description
`model`	`Any`	*required*	

router

Module `interlens.arena.live.router`

`LiveSeatRouter`: a seat table whose occupants can change while the episode is running.

`SeatRouter` already presents a heterogeneous lineup to the engine as one participant. This subclass adds the two things live play needs on top:

- **hot-swap.** :meth:`swap` replaces a seat's participant between turns. The engine holds a reference to the TABLE, not to the seat participants, so replacing an entry in the table is all it takes for the next turn to be played by somebody else — no episode restart, no engine involvement.
- **attribution.** Every message the table returns is stamped with the occupant label that produced it (`msg.metadata["occupant"]` -> `TurnRecord.occupant`), so the transcript records who played each turn rather than who holds the seat when you happen to read it.

The lock matters and is easy to get wrong. `generate` runs on an engine worker thread and `swap` on an HTTP thread, so a swap can land at any instant. The rule: the participant AND its label are read together, once, under the lock, and generation then runs OUTSIDE it against that snapshot. Reading them separately could stamp one occupant's label on another's turn, which is the one failure that would make the occupant record a lie; holding the lock across generation would instead block every swap for the length of an API call.

A swap therefore takes effect on the NEXT turn, never mid-turn. That is also why a swap is refused while the seat's human prompt is open: the person is already answering, and the turn is theirs.

Owned by lane A.

Attributes

Name	Type	Summary
`logger`		

Classes

Name	Summary
`LiveSeatRouter`	A :class:`~interlens.arena.table.SeatRouter` that supports mid-episode occupant changes.

LiveSeatRouter

A :class:`~interlens.arena.table.SeatRouter` that supports mid-episode occupant changes.

```
LiveSeatRouter(
    seats: dict[str, Any],
    labels: dict[str, str] | None = None,
    on_turn_start: Callable[[str, str | None], None] | None = None,
    name: str = 'live_table',
)
```

Defined in `interlens.arena.live.router` · source

Inherits from: SeatRouter

Parameters

seats : dict[str, Participant] Seat display name to the participant that starts the episode in it. labels : dict[str, str] | None Seat display name to its occupant label (`"api:claude-fable-5"`, `"human:sid"`, …). Seats missing from the map are stamped with nothing, which is how a turn records “unlabelled occupant” rather than guessing one. on_turn_start : callable | None `on_turn_start(seat, occupant) → None`, called just BEFORE dispatching a turn. This is the hook the live page’s “who is thinking” indicator is driven from: it fires while the model call is in flight, which is the only moment the information exists and the only moment it is useful. Exceptions from it must not reach the engine — a broken indicator cannot be allowed to kill a game. name : str Identifier within the conversation.

Notes

Drive a live table with `EpisodePool`, never `BatchedEpisodePool`. The batched pool resolves a pure-dispatch table (one declaring `participant_for`, which this inherits) to its sub-participants and addresses them directly — which would bypass this class’s `generate` and with it the occupant stamp, so a hot-swapped episode would come back with no record of who played what. Live play issues one request per wave anyway, so there is nothing to batch.

Parameters

Name	Type	Default	Description
`seats`	`dict[str, Any]`	*required*	
`labels`	`dict[str, str] | None`	`None`	
`on_turn_start`	`Callable[[str, str | None], None] | None`	`None`	
`name`	`str`	`'live_table'`	

Attributes

Name	Type	Summary
`labels`		
`on_turn_start`		

Methods

`generate`

```
generate(self, view, *, seat: str | None = None, **kwargs={}) -> Message
```

source

Snapshot (participant, label) for `seat` under the lock, fire `on_turn_start`, generate outside the lock, and return the message with `metadata["occupant"]` set to the snapshotted label.

The label is stamped only when the participant did not set one itself, so a participant that knows better than the table who it is (a human seat naming the player) keeps its own attribution.

Parameters

Name	Type	Default	Description
`view`		*required*	
`seat`	`str | None`	`None`	
`kwargs`		`{}`	

occupants

```
occupants(self) -> dict[str, str | None]
```

source

The current seat -> occupant-label map, read under the lock.

What `hello` reports so a page that just connected badges seats correctly without replaying the whole swap history.

swap

```
swap(self, seat: str, participant: Any, label: str) -> str | None
```

source

Install `participant` as `seat`'s occupant from the next turn on.

Returns the label of the occupant that was replaced (`None` if it had none), which the caller reports as the `from` side of a `seat_swapped` event.

Raises `KeyError` for an unknown seat. Does NOT check whether a human prompt is open on that seat — the session enforces that, because it is the half that knows the session's phase.

Parameters

Name	Type	Default	Description
`seat`	`str`	*required*	
`participant`	`Any`	*required*	

Name	Type	Default	Description
`label`	`str`	*required*	

server

Module `interlens.arena.live.server`

The HTTP surface: a stdlib `ThreadingHTTPServer` serving the lobby, the live page, and the event stream.

Stdlib only, matching `viz/serve.py` — the visualizer's whole point is that it adds no dependency to a package that is otherwise pure string-building, and a live server is not a good enough reason to put a web framework in it. `ThreadingHTTPServer` gives a thread per request, which is what an SSE endpoint needs: those handlers are long-lived, looping on their subscriber queue for the length of the game, and would block every other request on a single-threaded server.

Routes (all JSON in, JSON out, except the two HTML pages):

========== ====== ========== Route Method Purpose ========== ====== ========== / GET The lobby page (HTML) `/play` GET The live episode page (HTML), rendered from the snapshot `/api/lobby` GET Current lobby configuration + provider listings `/api/lobby` POST Partial lobby edit; returns the updated state `/api/start` POST Prepare a game and start a session -> `{sid, episode_id}` `/api/session/{sid}/state` GET Full snapshot: payload, phase, awaiting, occupants, seq `/api/session/{sid}/events` GET The SSE stream (honours `Last-Event-ID`) `/api/session/{sid}/act` POST A human seat's move `/api/session/{sid}/swap` POST Replace a seat's occupant `/api/session/{sid}/stop` POST End the session `/api/reset` POST Drop the session and return to the lobby ========== ====== ==========

The server binds but does not start, exactly like `viz.serve.make_server`, so the caller chooses the thread and the tests can run it on an ephemeral port. Port 0 is the default for the same reason it is there: shared nodes and fixed ports collide.

Owned by lane B.

Attributes

Name	Type	Summary
`DEFAULT_HOST`		
`MAX_BODY_BYTES`		
`SESSION_ROUTE`		
`logger`		

Classes

Name	Summary
`LiveHandler`	One request.

Functions

Name	Summary
`make_live_server`	Bind (but do not run) the live-play server over `provider`.
`run_live_server`	Serve live play until interrupted — bind, print the banner (including the `ssh -L` line, reusing `viz.serve.serve_banner`'s form), then block in `serve_forever`.

LiveHandler

One request.

```
LiveHandler()
```

Defined in `interlens.arena.live.server` · source

Inherits from: `BaseHTTPRequestHandler`

Dispatches the route table above against the server's `SessionManager`.

The manager and the provider hang off the server object (set by :func:`make_live_server`) rather than off the handler, since a handler instance lives for exactly one request.

Attributes

Name	Type	Summary
`manager`	SessionManager	The server's one session registry.
`protocol_version`		
`server_version`		

Methods

do_GET

```
do_GET(self) -> None
```

source

Serve the two pages, the snapshot, and the event stream.

do_POST

```
do_POST(self) -> None
```

source

Handle a lobby edit, a start, a human move, a swap, a stop, or a reset.

log_message

```
log_message(self, fmt: str, *args=()) -> None
```

source

Quiet by default: an SSE stream plus a per-turn poll would otherwise fill the terminal the operator is watching the game in.

Routed through the module logger at debug level instead.

Parameters

Name	Type	Default	Description
`fmt`	`str`	*required*	
`args`		`()`	

make_live_server

Bind (but do not run) the live-play server over `provider`.

```
make_live_server(
    provider: ScenarioProvider,
    host: str = DEFAULT_HOST,
    port: int = 0,
    run_dir=None,
) -> ThreadingHTTPServer
```

Defined in `interlens.arena.live.server` · source

`port=0` asks the OS for a free ephemeral port; read the one chosen back off `server.server_address[1]`. `run_dir` is where episodes are written (a temporary directory when omitted — but a session whose episodes are worth keeping should be given a real one, since the JSON on disk is the durable artifact and the stream is only a view of it).

Returns the bound server so the caller decides how to run it: `serve_forever()` on this thread, or another one in a test. Call `server_close()` when done.

Parameters

Name	Type	Default	Description
`provider`	Scenario-Provider	*required*	
`host`	`str`	`DEFAULT_HOST`	
`port`	`int`	`0`	
`run_dir`		`None`	

run_live_server

Serve live play until interrupted — bind, print the banner (including the `ssh -L` line, reusing `viz.serve.serve_banner`'s form), then block in `serve_forever`.

```
run_live_server(
    provider: ScenarioProvider,
    host: str = DEFAULT_HOST,
    port: int = 0,
    run_dir=None,
) -> None
```

Defined in `interlens.arena.live.server` · source

`Ctrl-C` shuts down and returns normally so the CLI exits 0, and stops any session still running first.

Parameters

Name	Type	Default	Description
`provider`	Scenario-Provider	*required*	
`host`	`str`	`DEFAULT_HOST`	
`port`	`int`	`0`	
`run_dir`		`None`	

session

Module `interlens.arena.live.session`

`LiveSession`: one live game — the engine thread, the event log, and everything the browser can do to it.

A session owns a running episode and the fanout to however many browsers are watching it. Its threading model is small on purpose, because a live server with an LLM in it has enough moving parts already:

- ONE engine thread per session runs `asyncio.run(EpisodePool(...).run_episode(...))` with `max_concurrent=1`. The negotiation scenario issues exactly one request per wave, so turns are strictly sequential — which is what makes a turn-by-turn UI honest rather than a reordering of a concurrent batch.
- HTTP threads (one per request, from `ThreadingHTTPServer`) mutate the session: submit a human move, swap a seat, stop. Every mutation takes ONE `threading.RLock`.
- Streaming is a plain fanout: :meth:`broadcast` appends the event to a sequence-numbered log and `put_nowait`s it on each subscriber's queue. It never blocks and never waits on a socket, because it is called from the engine thread — a slow reader must be able to fall behind or be dropped, never to stall a game.

No asyncio primitives are shared across threads. The engine thread has its own event loop and the only things crossing the boundary are `queue.Queue` handoffs, which are thread-safe by construction. Cross-loop asyncio objects are the classic way this goes wrong and are simply not used.

The event log is what makes reload lossless: a reconnecting browser replays from `Last-Event-ID` and a fresh one renders :meth:`snapshot` server-side, then subscribes from the snapshot's sequence number.

Owned by lane B.

Attributes

Name	Type	Summary
`DEFAULT_BUDGET_USD`		
`INSTRUCTION_HEADER`		
`PHASES`		
`SHUFFLE_REDRAWS`		
`SUBSCRIBER_QUEUE_MAX`		
`logger`		

Classes

Name	Summary
`LiveSession`	One live episode plus its subscribers.
`SessionManager`	The server's session registry. v1 holds ONE active session at a time.

Functions

Name	Summary
`apply_private_instructions`	Append `instructions` to `participant`'s `private_context` as one labelled segment, and return it.
`shuffled_seats`	Permute WHO plays which seat, leaving the seats themselves alone.

LiveSession

One live episode plus its subscribers.

```
LiveSession(
    sid: str,
    provider: ScenarioProvider,
    game: PreparedGame,
    seats: list[SeatConfig],
    run_dir: Any,
    budget_usd: float | None = None,
)
```

Defined in `interlens.arena.live.session` · source

Parameters

sid : str Session id, used in every route and echoed in `hello`. provider : ScenarioProvider Where games and model seats come from. game : PreparedGame The assembled game this session plays (from `provider.prepare`). seats : listSeatConfig Who plays each seat at the start, in seat order. run_dir : str | Path Where the episode JSON is written. The live page links to it, and it is the durable artifact: the stream is a view of the file, never a replacement for it. budget_usd : float | None Hard spend cap for this session, enforced by a `UsageMeter`. Required whenever any seat is a metered model — a live game with a human in it can idle for a long time with an API seat configured, and an uncapped session is how a lobby click turns into a bill.

Parameters

Name	Type	Default	Description
`sid`	`str`	*required*	
`provider`	ScenarioProvider	*required*	
`game`	PreparedGame	*required*	
`seats`	list[SeatConfig]	*required*	

Name	Type	Default	Description
`run_dir`	`Any`	*required*	
`budget_usd`	`float | None`	`None`	

Attributes

Name	Type	Summary
`budget_usd`		
`episode_id`	`str | None`	The running episode's id, or `None` until the first wave lands.
`game`		
`meter`		
`occupants`	`dict`	The current seat -> occupant-label map, or `{}` before the table is built.
`phase`		
`provider`		
`run_dir`		
`seats`		
`seq`	`int`	The sequence number of the last event this session emitted (0 before any).
`sid`		

Methods

broadcast

```
broadcast(self, event: str, data: dict) -> int
```

source

Append one event to the session log under the next sequence number and push it to every subscriber.

Returns the sequence number assigned. Never blocks: it is called from the engine thread between turns, so a subscriber whose queue is full is dropped rather than allowed to stall the game.

This is also where the session's own phase is kept: the events that move it (a seat blocking on a person, a turn landing, the episode ending) are exactly the events that go over the wire, so reading the phase off the stream keeps `/state` and the stream from ever disagreeing about where the session is.

Parameters

Name	Type	Default	Description
`event`	`str`	*required*	
`data`	`dict`	*required*	

lobby

```
lobby(self) -> dict
```

source

This session's own configuration, in the shape the `lobby_state` event carries.

Carries the provider's `models` and `policies` listings as well as the current lineup, because the live page's swap dock reuses the lobby's seat editor and must be able to offer the same choices without a second round trip to `/api/lobby`.

snapshot

```
snapshot(self) -> dict
```

source

The session's complete current state, for a page load or a reload mid-episode.

`{seq, payload, phase, awaiting, occupants, lobby}` — where `payload` is a full `viz.episode_payload` over the live episode's `to_json()` and `seq` is the sequence number that payload is current as of, so the client subscribes with `Last-Event-ID: seq` and misses nothing. Using the visualizer's own payload builder (rather than a live-specific one) is what guarantees a reloaded page and a streamed one are showing the same thing.

start

```
start(self) -> str
```

source

Build the table, spawn the engine thread, and return the episode id.

Constructs each seat from its :class:`SeatConfig` (model seats through `provider.build_model_seat`, computable seats through `arena.table.policy_seat`, human seats as `HumanParticipant`), wraps them in a :class:`~interlens.arena.live.router.LiveSeatRouter`, and runs the episode with `on_wave` bound to :meth:`_on_wave`. Fails fast — BEFORE the thread starts — if a configured API seat has no credentials or a metered seat has no budget cap, so a misconfiguration surfaces in the lobby instead of as a dead game.

Returns the episode id, which is `""` until the engine mints one (see :attr:`episode_id`).

stop

```
stop(self) -> None
```

source

End the session: unblock any waiting human seat, ask the engine thread to wind down, and broadcast `episode_done{status:"stopped"}`.

Idempotent — a double-click on Stop must not raise.

A blocked human seat is released immediately (its `generate` raises, and the engine finalizes the episode as an error, which is the honest record: a stopped game did not play itself out). A seat that is mid-generation cannot be interrupted — an API call in flight is in flight — so a stopped session ends when that turn returns, and `episode_done` is broadcast then rather than optimistically now.

submit_human

```
submit_human(self, seat: str, form: dict) -> None
```

source

Validate a human submission and hand it to the blocked seat.

Raises `ValueError` (message shown to the player) if the seat is not waiting, or the move is illegal / unparseable / empty. Nothing is enqueued on a rejection: the seat stays blocked and the caller answers 400 and broadcasts `input_rejected`. This is the guard that keeps a typo from becoming a silent pass — the engine would read empty content as a well-formed no-op turn.

Parameters

Name	Type	Default	Description
`seat`	`str`	*required*	
`form`	`dict`	*required*	

subscribe

```
subscribe(self, last_event_id: int | None = None) -> Any
```

source

Attach a client.

Returns a `queue.Queue` of `(seq, event, data)` tuples, pre-loaded with every logged event after `last_event_id` — the replay that makes a reconnect lossless.

Parameters

Name	Type	Default	Description
`last_event_id`	`int | None`	`None`	

swap_seat

```
swap_seat(self, seat_idx: int, config: SeatConfig) -> None
```

source

Replace a seat's occupant, effective on its next turn.

Builds the new participant from `config`, installs it via the router, and broadcasts `seat_swapped`. Refused (`ValueError`) while that seat's human prompt is open: the person is mid-decision and the turn is already theirs. v1 applies swaps between turns only.

Parameters

Name	Type	Default	Description
`seat_idx`	`int`	*required*	
`config`	SeatConfig	*required*	

unsubscribe

```
unsubscribe(self, q: Any) -> None
```

source

Detach a client's queue (its SSE connection closed).

Parameters

Name	Type	Default	Description
q	Any	*required*	

SessionManager

The server's session registry. v1 holds ONE active session at a time.

```
SessionManager(provider: ScenarioProvider, run_dir: Any, rng:
    random.Random | None = None)
```

Defined in `interlens.arena.live.session` · source

One at a time is a real constraint, not a shortcut: a live game is meant to be watched and played by the person who started it, budgets are per-session, and a second concurrent game would double an API bill with no way to tell whose it was. The manager keeps the lobby configuration between games so starting a second game with the same lineup is one click.

Parameters

Name	Type	Default	Description
`provider`	Scenario-Provider	*required*	
`run_dir`	`Any`	*required*	
`rng`	`random.Random | None`	`None`	

Attributes

Name	Type	Summary
`active`	LiveSession | None	The running session, or `None` when the server is sitting in the lobby.
`provider`		
`run_dir`		

Methods

`get`

```
get(self, sid: str) -> LiveSession
```

source

The session with this id.

Raises `KeyError` when it is unknown or has been replaced.

Parameters

Name	Type	Default	Description
`sid`	`str`	*required*	

`lobby_state`

```
lobby_state(self) -> dict
```

source

The current lobby configuration plus the provider's listings — what the lobby page renders from.

The complete set of keys, since three things render from this one dict (the lobby page, `GET /api/lobby`, and the `lobby_state` event) and a key that exists in only two of them is a drift waiting to happen:

- `banks` / `framings` / `models` — the provider's listings, as their `to_json()` dicts. An unavailable model is LISTED with its `unavailable_reason`, never filtered out.
- `policies` — the names in `table.POLICY_FACTORIES`, for the rational/oracle picker.
- `seat_kinds` — the kinds a seat may take (:data:`~interlens.arena.live.provider.SEAT_KINDS`).

- `seat_names` — seat order for the chosen bank, so the cards can be labelled before a game exists.
- `bank` / `framing` / `instance_id` / `seats` / `budget_usd` — the current selection. `instance_id` is `""` for "let the provider choose", which is also what the lobby may post back.
- `running` / `sid` / `phase` / `episode_id` — whether a session is live and how to reach it. `running` is a plain bool so a page can branch on it; `sid` is what `/play` is keyed by.
- `error` — the last refused edit's message, `""` when the last one was accepted.

reset

```
reset(self) -> None
```

source

Stop any active session and return the server to the lobby.

start

```
start(self, lobby: dict | None = None) -> LiveSession
```

source

Prepare a game from the lobby configuration and start a session on it.

Raises `ValueError` if a session is already running (stop it first) or the configuration is invalid.

Parameters

Name	Type	Default	Description
`lobby`	`dict | None`	`None`	

update_lobby

```
update_lobby(self, patch: dict) -> dict
```

source

Merge a partial lobby edit (one changed seat, a new bank) and return the updated state.

Validates against the provider's listings so an unknown model or policy is refused at edit time, not at start.

Recognized keys: `bank`, `framing`, `instance_id` (`""` = let the provider choose), `budget_usd` (`null` = uncapped, which only a lineup with no metered seat may start), `overrides`, `shuffle` (`true` permutes the current lineup among the seats — see :func:`shuffled_seats` — and records the permutation in `last_shuffle`; any later seat edit clears that record, since a permutation that no longer maps the lineup to a previous one describes nothing), and the seats. `seats` is the whole list; a single-card edit may instead send `{"seat_idx": i, "seat": {...}}` (`index` is accepted as a synonym), so a lobby with six seats does not have to round-trip all six to change one. Both forms are supported — send whichever the page finds simpler.

Parameters

Name	Type	Default	Description
`patch`	`dict`	*required*	

apply_private_instructions

Append `instructions` to `participant`'s `private_context` as one labelled segment, and return it.

```
apply_private_instructions(participant: Any, instructions: str) -> Any
```

Defined in `interlens.arena.live.session` · source

The whole override mechanism: a live operator gives one seat a persona or a hidden agenda without editing a scaffold, and the text folds into that seat's view exactly where its own private material already goes. A no-op for empty instructions or for a participant with no `private_context` (a policy seat reads no prose).

Shared with :meth:`ScenarioProvider.build_model_seat` implementations so a model seat's override and a scripted seat's are the same segment in the same place rather than two spellings of the same idea.

Parameters

Name	Type	Default	Description
`participant`	`Any`	*required*	
`instructions`	`str`	*required*	

shuffled_seats

Permute WHO plays which seat, leaving the seats themselves alone.

```
shuffled_seats(
    seats: list[SeatConfig],
    rng: random.Random,
) -> tuple[list[SeatConfig], list[int]]
```

Defined in `interlens.arena.live.session` · source

Returns the new lineup and the permutation that produced it as source indices: `order[i]` is the index the seat now at position `i` came from, so `new[i] is seats[order[i]]` and `order` is what gets recorded. Seat NAMES and parties do not move — Avery is still party 0 with party 0's score sheet — only the occupant configurations do, which is the thing worth randomizing when seat position carries a protocol advantage (the proposer order rotates from the proposer base, so "the model always opens" is a property of the lineup, not of the model).

Uniform over permutations, re-drawn while the resulting ARRANGEMENT is the one it started from (see :data:`SHUFFLE_REDRAWS`) — so it is uniform over the arrangements that visibly differ, and returns the input unchanged, with the identity order, when no different arrangement exists.

Parameters

seats : listSeatConfig The current lineup, in seat order. rng : random.Random The source of randomness, passed in rather than reached for so a caller can make a shuffle reproducible (and so the test can pin an exact permutation instead of asserting that something moved).

Parameters

Name	Type	Default	Description
`seats`	list[SeatConfig]	*required*	
`rng`	`random.Random`	*required*	

style

Module `interlens.arena.live.style`

The rules the lobby and the play page both need: form controls, seat cards, and the two docks.

Live play is the only part of the visualizer with FORMS in it — everything else renders numbers and prose — so the shared stylesheet (`viz.assets.CSS`) styles no input, select or textarea, and both live pages would otherwise carry their own copy of the same handful of rules. They are here instead, in the vocabulary of the shared sheet's own tokens (`--surface-1`, `--ring`, `--sp-3`), so a live control looks like part of the same page as the transcript beside it and a change to either page's controls is one edit rather than two.

Page-specific layout stays on the page that owns it (the lobby's start bar and problem lists, for instance); what is here is what a seat card, a labelled field and a dock are on both.

Attributes

Name	Type	Summary
`CSS_LIVE`		

negotiation

Package `interlens.arena.negotiation`

Multi-issue, multi-party scorable negotiation: deal spaces, private score sheets, exact solution concepts, computable rational-agent oracles, and an executable strategy zoo.

Each module's docstring carries the primary citations for the algorithm it implements; `references.py` maps every citation key to its full reference.

Modules

- `acceptance` — Optimal-stopping acceptance oracle: when is accepting the standing offer better than holding out?
- `analysis` — Measurement over stored negotiation episodes: how far from rational was this play, and where?
- `belief_accuracy` — How well does a :class:`~interlens.arena.negotiation.beliefs.BeliefState` actually know its opponent?
- `beliefs` — Bayesian (and frequency-model fallback) belief oracle over an enumerated opponent-type grid.
- `bestresponse` — Exact expectimax best-response oracle over (remaining rounds x deal space x type posterior).
- `calibrated` — Behaviourally-calibrated rational negotiation: :class:`CalibratedRationalPolicy`.
- `equilibrium` — Banks-Duggan stationary-equilibrium oracle for the multilateral unanimity bargaining game.
- `fairness` — The **table objective**: one number per deal saying how good that deal is *for the whole table*.
- `games` — Swappable game presets: name a classic bargaining situation, get a ready-to-play game in one call.
- `generate` — Scorable-negotiation scenario generator with the scoresheet repairs the reproducibility studies demand.

- `llm_calibrated` — Private-information rational negotiation against an **empirically fitted LLM opponent model**: :class:`LLMCalibratedRationalPolicy`.
- `oracle_context` — The per-decision-point context the negotiation oracles share, written once.
- `policy_participant` — `PolicyParticipant`: a state-dependent pure-Python seat that computes its move from a bound policy.
- `references` — Citation-key registry for the negotiation solution-concept and generator modules.
- `rewards` — Outcome rewards for RL on scorable negotiation: the smoothed log-Nash objective.
- `sheets` — Private score sheets, the additive utility model, the game specification, and the NumPy utility matrix.
- `solutions` — Exact axiomatic solution concepts over the fully-enumerated deal space.
- `space` — The deal space: issues, their discrete options, and the fully-enumerable Cartesian product of options.
- `strategies` — The executable rational / scripted negotiator zoo as **policies** (`state → action`), the computable opponent pool the LLMs are measured against.
- `talking` — The **talking rational agent**: the composed Bayesian negotiator with a truthful templated voice.

acceptance

Module `interlens.arena.negotiation.acceptance`

Optimal-stopping acceptance oracle: when is accepting the standing offer better than holding out?

Core recursion — Baarslag & Hindriks, “Accepting Optimally in Automated Negotiation with Incomplete Information,” AAMAS 2013, pp. 715-722, Eqs. (1)-(3) (http://www.ifaamas.org/Proceedings/aamas2013/docs/p715.pdf):

```
v_0 = 0
v_j = E[ max(X_{j-1}, v_{j-1}) ] - C          # j = rounds remaining; X_j
     ~ F_j the offer distribution
accept x  iff  x ⩾ v_j                         # Algorithm 1
```

with `F_j` = the offer-distribution mixture induced by the belief posterior (`offer_surplus_pmf` below). Closed form for a uniform opponent (their Prop. 3.1): `v_j = 1/2 + v_{j-1}^2 / 2` (reproduced by the unit test). This module **generalizes** the recursion with a per-round discount `delta` and a disagreement `flow` so the reservation is an *endogenous, time-varying* continuation value tau_i(t) (McCall, “Economics of Information and Job Search,” QJE 84(1):113-126, 1970 — the reservation-wage recursion); the additive cost `C` and the discount `delta` are the two friction conventions and coincide with Baarslag at `delta=1, flow=0`. An optional outside-option floor follows Li, Giampapa & Sycara, IEEE SMC-C 36(1):31-44, 2006 (reservation price = valuation - reservation utility; conservative order-statistic OU).

DEADLINE WARNING (Sandholm & Vulkan, “Bargaining with Deadlines,” AAAI-99, pp. 44-51): with a firm common deadline and NO discounting, the unique sequential-equilibrium play is extreme brinkmanship — both wait until the earlier deadline, then the deadline-bound party concedes the whole surplus. So if the game’s only impatience is a hard turn-count deadline, this oracle’s “hold out” recommendation is rational but degenerate; pass a `discount < 1` (or a breakdown-risk / outside-option) to make interior concession rational.

The single most diagnostic per-turn signal in practice (accept-too-early leaves surplus on the table; never-accept blows the deadline).

Classes

Name	Summary
`AcceptanceOracle`	Values accept / reject / propose / walk at one turn via optimal stopping.
`ThresholdOracle`	The trivial hard-violation detector: accepting or proposing a deal below one's *own* threshold is a strict rationality error (agreeing below your BATNA is worse than no deal — Abdelnabi et al.'s "wrong deals" metric, which runs 7-20% even for strong models).

Functions

Name	Summary
`offer_surplus_pmf`	The distribution `F` of the surplus `agent` expects to *receive*, induced by the belief posterior.
`reservation_values`	Backward-induction reservation curve `[v_0, v_1, ..., v_T]` where `v_j` is the reservation with `j` rounds remaining; **accept an offer of surplus x iff x >= v_j**.

AcceptanceOracle

Values accept / reject / propose / walk at one turn via optimal stopping.

```
AcceptanceOracle(
    agent: int,
    *,
    discount: float | None = None,
    cost: float = 0.0,
    flow: float = 0.0,
    accept_prob_fn=None,
    outside_value: float | None = None,
    accept_prob_vec=None,
)
```

Defined in `interlens.arena.negotiation.acceptance` · source

Inherits from: Oracle

Parameters

agent : int The deciding seat. discount : float Per-round discount `delta` OVERRIDE. Default `None` = read the game's own `discount` / `breakdown_risk` via `effective_discount` (single source of truth); pass a float only to force a value. A nonzero discount is what makes interior concession rational (Sandholm-Vulkan, module docstring). cost : float Additive per-round search cost `C`. flow : float Disagreement flow while unagreed. accept_prob_fn : callable | None `deal → P(all opponents accept)` from the belief oracle; None = full-info uniform-offer proxy. accept_prob_vec : np.ndarray | None The vectorized form of `accept_prob_fn`: a `(|D|,)` array of the same probabilities in `tables.deals` order (see :func:`offer_surplus_pmf`). Prefer it — a caller that already holds the posterior acceptance table can produce it with ONE `passage_probability` call, where the callable form re-solves the same Poisson-binomial once per deal. Takes precedence over `accept_prob_fn`. outside_value : float | None Optional reservation floor (outside option).

Parameters

Name	Type	Default	Description
agent	int	*required*	
discount	float | None	None	
cost	float	0.0	
flow	float	0.0	
accept_prob_fn		None	
outside_value	float | None	None	
accept_prob_vec		None	

Attributes

Name	Type	Summary
accept_prob_fn		
accept_prob_vec		
agent		
cost		
discount		
flow		
name		
outside_value		

Methods

evaluate

```
evaluate(self, game, history, agent, legal)
```

source

Value each legal action in surplus units and flag stopping errors.

`Accept(offer_id)` is valued at its realized surplus for `agent`; `Reject/Propose/Walk` are valued at the continuation reservation `v` (Walk clamped at >= 0). The discount is read from the game (`effective_discount`) unless the oracle was constructed with an explicit override. Flags: `premature_accept` (an available Accept is below reservation), `should_accept` (the best live offer clears reservation), `deadline_brinkmanship` (delta ~ 1 with a hard deadline).

Parameters

Name	Type	Default	Description
`game`		*required*	
`history`		*required*	
`agent`		*required*	
`legal`		*required*	

`reservation`

```
reservation(
    self,
    tables: GameTables,
    rounds_left: int,
    discount: float | None = None,
    agent: int | None = None,
    *,
    objective=None,
) -> float
```

source

The reservation surplus `v_{rounds_left}` given the belief-induced offer distribution.

`agent` is the seat whose offer-surplus distribution sets the reservation; `None` falls back to the constructor's `self.agent`. Pass it whenever one instance serves several seats (a scenario's shared oracle stack) — :meth:`evaluate` passes the seat it resolved, since a stale seat here would silently set every party's stopping threshold from seat 0's sheet and so mis-flag `premature_accept` / `should_accept`.

`objective` substitutes a table-level payoff column for own surplus (see :func:`offer_surplus_pmf`), which is what turns this into a fairness-seeking agent's stopping rule: the returned number is then the table welfare it expects to reach by continuing, and it should accept iff the standing offer's welfare clears it.

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	
`rounds_left`	`int`	*required*	
`discount`	`float | None`	`None`	
`agent`	`int | None`	`None`	
`objective`		`None`	

`reservation_curve`

```
reservation_curve(
    self,
    tables: GameTables,
    T: int,
    discount: float | None = None,
    agent: int | None = None,
) -> list
```

source

The full endogenous reservation curve `tau_i(rounds_left)` for `rounds_left = 0..T`.

`agent` as in :meth:`reservation`.

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	
`T`	`int`	*required*	

Name	Type	Default	Description
`discount`	`float | None`	`None`	
`agent`	`int | None`	`None`	

ThresholdOracle

The trivial hard-violation detector: accepting or proposing a deal below one's *own* threshold is a strict rationality error (agreeing below your BATNA is worse than no deal — Abdelnabi et al.'s "wrong deals" metric, which runs 7-20% even for strong models).

```
ThresholdOracle(agent: int | None = None)
```

Defined in `interlens.arena.negotiation.acceptance` · source

Inherits from: Oracle

Cheap (one surplus lookup per action); values each action at the agent's own surplus (Walk/Reject at 0) and flags the individual-rationality violations.

This complements the stopping oracle: `AcceptanceOracle` says *when* holding out beats accepting a good offer, while `ThresholdOracle` catches the strictly-dominated moves regardless of timing.

Parameters

Name	Type	Default	Description
`agent`	`int | None`	`None`	

Attributes

Name	Type	Summary
`agent`		
`name`		

Methods

evaluate

```
evaluate(self, game, history, agent, legal)
```

source

Value `Accept`/`Propose` at the agent's own surplus (Walk/Reject at 0); `best` is the surplus-maximizing individually-rational move.

Flags: `below_threshold_accept` (an available Accept is below the agent's BATNA) and `below_threshold_propose` (proposing a deal below one's own BATNA).

Parameters

Name	Type	Default	Description
`game`		*required*	
`history`		*required*	
`agent`		*required*	
`legal`		*required*	

offer_surplus_pmf

The distribution `F` of the surplus `agent` expects to *receive*, induced by the belief posterior.

```
offer_surplus_pmf(
    tables: GameTables,
    agent: int,
    accept_prob_fn=None,
    *,
    only_closable: bool = True,
    only_ir: bool = True,
    objective=None,
    accept_weights=None,
)
```

Defined in `interlens.arena.negotiation.acceptance` · source

Each deal `d` is weighted by `P(all opponents accept d)` (so only deals that can actually close carry mass) and contributes its own surplus `tables.surplus[d, agent]` to the support.

Parameters

tables : GameTables Precomputed surplus tables. agent : int Whose received-surplus distribution to build. accept_prob_fn : callable | None `accept_prob_fn(deal) → P(all opponents accept deal)`. Typically `lambda d: prod_j belief_states[j].accept_prob(d)`. If None, all closable = uniform (full-info proxy: every IR deal equally offerable). Ignored when `accept_weights` is given. accept_weights : np.ndarray | None The SAME per-deal acceptance probabilities as a ready `(|D|,)` vector in `tables.deals` order — the vectorized form of `accept_prob_fn`, and the one to prefer. `passage_probability` already solves every deal at once (its Poisson-binomial DP is elementwise across the deal axis, so a row of the batched result is bitwise identical to solving that row alone), whereas calling `accept_prob_fn` here runs one numpy-heavy call per deal — |D| times per turn, per seat, which measured as the single largest cost in a long-horizon private-information episode. only_closable : bool Drop deals with zero acceptance probability. only_ir : bool Drop deals with negative surplus for `agent` (below own threshold — never worth receiving). objective :

np.ndarray | None Optional `(|D|,)` payoff column to use **instead of** `agent`'s own surplus — the objective substitution that makes this recursion serve a fairness-seeking agent (`fairness.mnw_objective`) as well as a self-interested one. `None` (default) reads `tables.surplus[:, agent]`, which is the exact prior behaviour. Whatever is passed must share own-surplus's convention that **no-deal scores zero**, since that is the recursion's `v_0` base case; the `only_ir` filter then reads "non-negative on this column" in the substituted units.

Returns `(values, probs)` — a normalized pmf. Empty support falls back to a point mass at surplus 0 (i.e. "no acceptable offer expected", so the reservation collapses to the no-deal value).

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	
`agent`	`int`	*required*	
`accept_prob_fn`		`None`	
`only_closable`	`bool`	`True`	
`only_ir`	`bool`	`True`	
`objective`		`None`	
`accept_weights`		`None`	

reservation_values

Backward-induction reservation curve `[v_0, v_1, ..., v_T]` where `v_j` is the reservation with `j` rounds remaining; **accept an offer of surplus x iff x >= v_j**.

```
reservation_values(
    values,
    probs,
    T: int,
    *,
    cost: float = 0.0,
    discount: float = 1.0,
    flow: float = 0.0,
    pmfs=None,
    outside_value: float | None = None,
) -> list
```

Defined in `interlens.arena.negotiation.acceptance` · source

v_j = (1 - discount) * flow + discount * E_{X ~ F_{j-1}}[max(X, v_{j-1})] - cost

Parameters

values, probs : sequence[float] The stationary offer-surplus distribution `F` as a discrete pmf (support `values`, masses `probs`). Ignored if `pmfs` is given. T : int Horizon (max rounds remaining). cost : float Additive per-round search cost `c` (Baarslag friction). discount : float Per-round discount / no-breakdown probability `delta` in `(0, 1]` (McCall friction). flow : float Disagreement flow received while unagreed (enters the discounted continuation). pmfs : list[tuple[seq, seq]] | None Optional per-round distributions `F_{j-1}`; `pmfs[j-1] = (values, probs)` used at step `j` for a time-varying offer distribution. Length must be >= `T`. outside_value : float | None Optional floor on every `v_j` (a conservative outside-option reservation utility; Li-Giampapa- Sycara). The reservation never drops below it.

Returns `list[float]` of length `T + 1`.

Parameters

Name	Type	Default	Description
values		*required*	
probs		*required*	
T	int	*required*	
cost	float	0.0	
discount	float	1.0	
flow	float	0.0	
pmfs		None	
outside_value	float | None	None	

analysis

Package `interlens.arena.negotiation.analysis`

Measurement over stored negotiation episodes: how far from rational was this play, and where?

Reading an episode back is the other half of running one, and none of it is specific to a single study — any experiment over this game family asks the same questions. So the measurement stack lives here, next to the game and the oracles, rather than inside one experiment:

- :mod:`surplus` — surplus-vector primitives: Pareto/dominance geometry and distances. (The welfare and inequality SCALARS live in `negotiation.solutions`, with the solution concepts they are compared against.)
- :mod:`game_analysis` — :class:`~.game_analysis.GameAnalysis`, the solved-game bundle the metrics read: the frontier, the axiomatic solution points, and the per-party scales, built from a stored `Instance` or a `GameSpec`.
- :mod:`episode_view` — :class:`~.episode_view.EpisodeView` / `TurnView`: a stored episode parsed into the per-turn move ledger (offers, votes, walks) the metrics iterate.
- :mod:`curves` — concession-curve fitting (the tanh τ / CRI summary) and the Park et al. no-regret trend tests.
- :mod:`annotations` — the per-turn annotation records (`TurnAnnotation` / `EpisodeAnnotation` / `DivergenceSummary`) and the readers that group an episode's inline `OracleRecord` rows.
- :mod:`metrics` — the outcome and per-turn metric functions (welfare/distance-to-frontier, regret series, IR and dominated-proposal detection, concession summaries).
- :mod:`taxonomy` — the failure taxonomy: named divergence rows tiered from fully mechanical to judge-scored.
- :mod:`rollout` — counterfactual-rollout regret (replay a turn under a reference policy and re-score).
- :mod:`cot_localize` — within-chain-of-thought localization of the first divergent reasoning step.

Framing a run's numbers into a report (the "atlas") stays with the experiment that defines the run — this package supplies the measurements, not the presentation.

Submodules are imported explicitly (`from interlens.arena.negotiation.analysis import metrics`) rather than re-exported here, so importing one measurement does not pull numpy-heavy siblings you did not ask for.

Modules

- `annotations` — Per-turn annotation records: the divergence data model `annotate.py` writes and `taxonomy.py` / `report.py` read (disk I/O lives in `runio.AnnotationStore`).
- `cot_localize` — OmegaPRM-style within-CoT divergence localization: binary-search the first reasoning step that flips the induced action to a divergent one, in O(log n) oracle calls (arXiv:2406.06592 — exploit prefix monotonicity: correct until the first error, wrong after).
- `curves` — Trajectory-shape metrics over a *series* (not a single turn).
- `episode_view` — `EpisodeView`: a stored arena `Episode` JSON parsed into a normalized negotiation action series that the metrics read instead of raw `parsed_action` — an ordered `TurnView` list (typed action, deal canonicalized to an index tuple, offer id, any acceptable-offer/belief note, private thinking), the offer registry, the per-round standing offer, and the final deal / reached flag.
- `game_analysis` — `GameAnalysis`: the solved-game bundle the metrics read — the adapter seam between interlens' `GameSpec`/`solutions.py` and the pure metric math.
- `metrics` — The divergence metric suite: outcome-, turn-, and faithfulness-level measures over one episode.
- `rollout` — Counterfactual-rollout regret: label a divergence by Δ expected surplus, not by action mismatch.
- `surplus` — Pure surplus-vector math: Pareto geometry, dominance, and distances.
- `taxonomy` — The 12-row LLM-negotiation failure taxonomy as executable checks.

annotations

Module `interlens.arena.negotiation.analysis.annotations`

Per-turn annotation records: the divergence data model `annotate.py` writes and `taxonomy.py` / `report.py` read (disk I/O lives in `runio.AnnotationStore`).

An `EpisodeAnnotation` is one episode's divergence record — a `TurnAnnotation` per turn (oracle verdicts, headline regret, hard-violation flags, counterfactual regret, CoT first-divergent-step) plus a rolled-up `DivergenceSummary` — kept separate from the immutable raw `Episode` so re-annotating never rewrites it. `TurnAnnotation.oracle` mirrors interlens' per-oracle `OracleRecord` (name → verdict).

Attributes

Name	Type	Summary
`PRIMARY_ORACLES`		

Classes

Name	Summary
`DivergenceSummary`	Episode-level roll-up of the turn annotations: the regret series and its aggregates, the divergence-point turns (regret above the noise threshold), and hard-flag counts by taxonomy row.
`EpisodeAnnotation`	One episode's full divergence record: the summary plus every turn annotation.
`TurnAnnotation`	One turn's oracle annotation.

Functions

Name	Summary
annotation_from_episode	Build an EpisodeAnnotation purely from an episode's inline oracle rows (round_checkpoints).
group_oracle_rows	Group inline oracle rows by turn into [{turn_idx, round, seat, oracle: {name:row}, regret, flags, belief}] (turn order).
inline_oracle_rows	The episode's INLINE oracle annotation rows in round_checkpoints — the OracleRecord.to_json() dicts, identified by a "verdict" key (forked provisional-probe rows have no verdict and are excluded).
summarize_turns	Roll a list of turn annotations up into a DivergenceSummary (shared by annotate.py and the inline-rows reader so the summary math lives in one place).

DivergenceSummary

Episode-level roll-up of the turn annotations: the regret series and its aggregates, the divergence-point turns (regret above the noise threshold), and hard-flag counts by taxonomy row.

```
DivergenceSummary(
    episode_id: str,
    model: str,
    arm: str,
    n_turns: int,
    n_flagged: int,
    regret_series: list[float],
    mean_regret: float,
    total_regret: float,
    divergence_turns: list[int],
    flag_counts: dict[str, int],
    outcome: dict = dict(),
)
```

Defined in `interlens.arena.negotiation.analysis.annotations` · source

Parameters

Name	Type	Default	Description
`episode_id`	`str`	*required*	
`model`	`str`	*required*	
`arm`	`str`	*required*	
`n_turns`	`int`	*required*	
`n_flagged`	`int`	*required*	
`regret_series`	`list[float]`	*required*	
`mean_regret`	`float`	*required*	
`total_regret`	`float`	*required*	
`divergence_turns`	`list[int]`	*required*	
`flag_counts`	`dict[str, int]`	*required*	
`outcome`	`dict`	`dict()`	

Attributes

Name	Type	Summary
arm	str	
divergence_turns	list[int]	
episode_id	str	
flag_counts	dict[str, int]	
mean_regret	float	
model	str	
n_flagged	int	
n_turns	int	
outcome	dict	
regret_series	list[float]	
total_regret	float	

Methods

from_json

```
from_json(d: dict) -> 'DivergenceSummary'
```

source

Parameters

Name	Type	Default	Description
d	dict	*required*	

to_json

```
to_json(self) -> dict
```

source

EpisodeAnnotation

One episode's full divergence record: the summary plus every turn annotation.

```
EpisodeAnnotation(
    episode_id: str,
    summary: DivergenceSummary,
    turns: list[TurnAnnotation] = list(),
)
```

Defined in `interlens.arena.negotiation.analysis.annotations` · source

Parameters

Name	Type	Default	Description
`episode_id`	`str`	*required*	
`summary`	DivergenceSummary	*required*	
`turns`	list[TurnAnnotation]	`list()`	

Attributes

Name	Type	Summary
`episode_id`	`str`	
`summary`	DivergenceSummary	
`turns`	list[TurnAnnotation]	

Methods

from_json

```
from_json(d: dict) -> 'EpisodeAnnotation'
```

source

Parameters

Name	Type	Default	Description
d	dict	*required*	

to_json

```
to_json(self) -> dict
```

source

TurnAnnotation

One turn's oracle annotation.

```
TurnAnnotation(
    turn_idx: int,
    round: int,
    seat: str,
    oracle: dict[str, dict] = dict(),
    regret: float | None = None,
    flags: list[str] = list(),
    cf_regret: float | None = None,
    cot_first_divergent_step: int | None = None,
    oracle_belief: Any = None,
    stated_belief: Any = None,
    counterfactuals: dict[str, dict] = dict(),
)
```

Defined in `interlens.arena.negotiation.analysis.annotations` · source

`oracle` maps an oracle name to its scored verdict dict. `counterfactuals` stores named decision references such as `rational_private` and `oracle_omniscient`; each value may carry `action`, `deal` or `deal_index`, `value`, and `information`. Keeping decision references separate from scored oracle verdicts lets a visualizer compare the action actually taken with both an information-feasible policy and a privileged hindsight optimum without pretending that they share an information set. Older annotation files omit the field and deserialize to an empty mapping.

`regret` is the headline per-turn surplus loss from the primary value oracle, while `flags` are the taxonomy row identifiers that fired for this turn.

Parameters

Name	Type	Default	Description
`turn_idx`	`int`	*required*	
`round`	`int`	*required*	
`seat`	`str`	*required*	
`oracle`	`dict[str, dict]`	`dict()`	

Name	Type	Default	Description
regret	float | None	None	
flags	list[str]	list()	
cf_regret	float | None	None	
cot_first_divergent_step	int | None	None	
oracle_belief	Any	None	
stated_belief	Any	None	
counterfactuals	dict[str, dict]	dict()	

Attributes

Name	Type	Summary
cf_regret	float | None	
cot_first_divergent_step	int | None	
counterfactuals	dict[str, dict]	
flags	list[str]	
oracle	dict[str, dict]	
oracle_belief	Any	
regret	float | None	
round	int	
seat	str	
stated_belief	Any	
turn_idx	int	

Methods

from_json

```
from_json(d: dict) -> 'TurnAnnotation'
```

source

Parameters

Name	Type	Default	Description
d	dict	*required*	

to_json

```
to_json(self) -> dict
```

source

annotation_from_episode

Build an `EpisodeAnnotation` purely from an episode's inline oracle rows (`round_checkpoints`).

```
annotation_from_episode(
    episode: dict,
    *,
    threshold: float = 1e-06,
) -> 'EpisodeAnnotation | None'
```

Defined in `interlens.arena.negotiation.analysis.annotations` · source

Returns `None` when the episode carries no inline oracle rows (a v0 dataset or a not-yet-annotated run), so callers fall back to the mechanical-only path. Used by `report.py` to show oracle regret straight from a stored run without a separate annotate pass.

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`threshold`	`float`	`1e-06`	

group_oracle_rows

Group inline oracle rows by turn into `[{turn_idx, round, seat, oracle: {name:row}, regret, flags, belief}]` (turn order).

```
group_oracle_rows(rows: list[dict], *, primary: tuple = PRIMARY_ORACLES) -
    > list[dict]
```

Defined in `interlens.arena.negotiation.analysis.annotations` · source

The per-turn `regret` is the `divergence` of the first oracle in `primary` present, else the max-divergence oracle; `flags` are unioned across that turn's oracles. Shared by `annotation_from_episode` and `annotate.py`'s inline merge.

Parameters

Name	Type	Default	Description
`rows`	`list[dict]`	*required*	
`primary`	`tuple`	`PRIMARY_ORACLES`	

inline_oracle_rows

The episode's INLINE oracle annotation rows in `round_checkpoints` — the `OracleRecord.to_json()` dicts, identified by a `"verdict"` key (forked provisional-probe rows have no verdict and are excluded).

```
inline_oracle_rows(episode: dict) -> list[dict]
```

Defined in `interlens.arena.negotiation.analysis.annotations` · source

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	

summarize_turns

Roll a list of turn annotations up into a `DivergenceSummary` (shared by `annotate.py` and the inline-rows reader so the summary math lives in one place).

```
summarize_turns(
    episode_id: str,
    model: str,
    arm: str,
    outcome: dict,
    turn_anns: list[TurnAnnotation],
    threshold: float,
) -> DivergenceSummary
```

Defined in `interlens.arena.negotiation.analysis.annotations` · source

Parameters

Name	Type	Default	Description
`episode_id`	`str`	*required*	
`model`	`str`	*required*	
`arm`	`str`	*required*	
`outcome`	`dict`	*required*	
`turn_anns`	list[TurnAnnotation]	*required*	
`threshold`	`float`	*required*	

cot_localize

Module `interlens.arena.negotiation.analysis.cot_localize`

OmegaPRM-style within-CoT divergence localization: binary-search the first reasoning step that flips the induced action to a divergent one, in O(log n) oracle calls (arXiv:2406.06592 — exploit prefix monotonicity: correct until the first error, wrong after). Re-deriving an action from a truncated CoT is the LLM hook (`induced_action_hook`); the search skeleton here is model-agnostic and tested against a synthetic oracle.

Functions

Name	Summary
`induced_action_hook`	HOOK (unimplemented): re-derive the action a truncated CoT induces by force-continuing the model from the joined `prefix_steps`; wrap with the oracle's divergence check to form `is_divergent` for `localize_first_divergence`.
`localize_first_divergence`	Binary-search the earliest prefix length `j` (1..n_steps) whose induced action is divergent.
`split_steps`	Split a CoT scratchpad into reasoning steps.

induced_action_hook

HOOK (unimplemented): re-derive the action a truncated CoT induces by force-continuing the model from the joined `prefix_steps`; wrap with the oracle's divergence check to form `is_divergent` for `localize_first_divergence`.

```
induced_action_hook(prefix_steps: list[str])
```

Defined in `interlens.arena.negotiation.analysis.cot_localize` · source

Model generation is out of the metrics layer's scope.

Parameters

Name	Type	Default	Description
`prefix_steps`	`list[str]`	*required*	

localize_first_divergence

Binary-search the earliest prefix length `j` (1..n_steps) whose induced action is divergent.

```
localize_first_divergence(n_steps: int, is_divergent: Callable[[int],
    bool]) -> int | None
```

Defined in `interlens.arena.negotiation.analysis.cot_localize` · source

`is_divergent(j)` reports whether the action induced by the first `j` steps diverges from the oracle; it is assumed monotone (non-divergent for short prefixes, divergent once the erroneous step is included), which is the OmegaPRM prefix-correctness assumption. Returns the first divergent step index (1-based), or `None` if even the full CoT does not induce a divergent action. Makes O(log n_steps) calls to `is_divergent` — each call is one (expensive) re-derivation + oracle check in production.

Parameters

Name	Type	Default	Description
`n_steps`	`int`	*required*	
`is_divergent`	`Callable[[int], bool]`	*required*	

split_steps

Split a CoT scratchpad into reasoning steps.

```
split_steps(scratchpad: str) -> list[str]
```

Defined in `interlens.arena.negotiation.analysis.cot_localize` · source

Default heuristic: numbered/bulleted lines (“1.”, “-”, “Step 3:”) start new steps, otherwise blank-line-separated paragraphs, otherwise single newlines. A scenario that emits already-structured steps should pass them in directly rather than re-splitting.

Parameters

Name	Type	Default	Description
`scratchpad`	`str`	*required*	

curves

Module `interlens.arena.negotiation.analysis.curves`

Trajectory-shape metrics over a *series* (not a single turn). Pure numpy (no scipy).

- **Concession-curve fit** `y(x) = d + b*tanh(a*x - c)`, burstiness `tau = |a|*|b|` and Concession-Rigidity Index `CRI = 1 - 1.32/(|a|*T)` (LLM Rationalis arXiv:2512.13063 §3) — smooth vs rigid all-or-nothing concession. Fit by multi-start damped Gauss–Newton (the tanh fit is non-convex).
- **No-regret trend tests** (Park et al. arXiv:2403.16843 §3.1) over a per-turn regret series: a Mann–Kendall trend test for a decreasing average regret `Regret_t/t`, and a log–log slope of cumulative regret (`b0 < 1` ⇒ sublinear ⇒ no-regret).

Classes

Name	Summary
`ConcessionFit`	Fitted `y(x) = d + b*tanh(a*x - c)` on normalized turn-fraction x∈[0,1] and normalized concession y∈[0,1], plus the derived shape metrics.
`LogLogRegret`	Log–log regression `log R_t = b0 log t + b1` of cumulative regret (Park §3.1).
`NoRegretTrend`	Mann–Kendall test for a *decreasing* trend in average regret `Regret_t / t` (Park §3.1 / Prop. 1).

Functions

Name	Summary
`fit_concession_curve`	Fit the tanh concession model to a sequence of a party's successive offer *values* (in the order made).
`loglog_regret_slope`	Fit the log–log slope of cumulative regret.
`no_regret_trend_test`	Park trend test on a per-turn regret series (r_t >= 0, surplus-loss units).

ConcessionFit

Fitted `y(x) = d + b*tanh(a*x - c)` on normalized turn-fraction x∈[0,1] and normalized concession y∈[0,1], plus the derived shape metrics.

```
ConcessionFit(
    a: float,
    b: float,
    c: float,
    d: float,
    tau: float,
    cri: float,
    rmse: float,
    n: int,
)
```

Defined in `interlens.arena.negotiation.analysis.curves` · source

`n` is the number of offers fitted; `rmse` the fit residual (a large rmse means the tanh family does not describe this trajectory — read τ/CRI with care).

Parameters

Name	Type	Default	Description
a	float	*required*	
b	float	*required*	
c	float	*required*	
d	float	*required*	
tau	float	*required*	
cri	float	*required*	
rmse	float	*required*	
n	int	*required*	

Attributes

Name	Type	Summary
a	float	
b	float	
c	float	
cri	float	
d	float	
n	int	
rmse	float	
tau	float	

Methods

to_json

```
to_json(self) -> dict
```

source

LogLogRegret

Log–log regression `log R_t = b0 log t + b1` of cumulative regret (Park §3.1).

```
LogLogRegret(beta0: float, intercept: float, r2: float, sublinear: bool,
    n_points: int)
```

Defined in `interlens.arena.negotiation.analysis.curves` · source

`b0 < 1` ⇒ sublinear cumulative regret ⇒ no-regret. `r2` is the fit quality; `n_points` the turns with positive cumulative regret (log is undefined at zero, so leading zero-regret turns are dropped).

Parameters

Name	Type	Default	Description
`beta0`	`float`	*required*	
`intercept`	`float`	*required*	
`r2`	`float`	*required*	
`sublinear`	`bool`	*required*	
`n_points`	`int`	*required*	

Attributes

Name	Type	Summary
`beta0`	`float`	
`intercept`	`float`	
`n_points`	`int`	
`r2`	`float`	
`sublinear`	`bool`	

Methods

to_json

```
to_json(self) -> dict
```

source

NoRegretTrend

Mann–Kendall test for a *decreasing* trend in average regret `Regret_t / t` (Park §3.1 / Prop. 1).

```
NoRegretTrend(
    n: int,
    mk_s: int,
    z: float,
    trend_pvalue: float,
    no_regret_evidence: bool,
    mean_avg_regret: float,
)
```

Defined in `interlens.arena.negotiation.analysis.curves` · source

`trend_pvalue` is the one-sided p-value for the decreasing alternative: small p ⇒ average regret is significantly decreasing ⇒ evidence of no-regret behavior (`no_regret_evidence` at α=0.05). `mk_s` is the Mann–Kendall statistic (negative = downward), `mean_avg_regret` the mean of `Regret_t/t`.

Parameters

Name	Type	Default	Description
`n`	`int`	*required*	
`mk_s`	`int`	*required*	
`z`	`float`	*required*	
`trend_pvalue`	`float`	*required*	
`no_regret_evidence`	`bool`	*required*	
`mean_avg_regret`	`float`	*required*	

Attributes

Name	Type	Summary
`mean_avg_regret`	`float`	

Name	Type	Summary
mk_s	int	
n	int	
no_regret_evidence	bool	
trend_pvalue	float	
z	float	

Methods

to_json

```
to_json(self) -> dict
```

source

fit_concession_curve

Fit the tanh concession model to a sequence of a party's successive offer *values* (in the order made).

```
fit_concession_curve(values, *, normalize: bool = True) -> ConcessionFit
```

Defined in `interlens.arena.negotiation.analysis.curves` · source

`values` is the party's own-surplus (or its own offered price) at each of its offers; the x-axis is the turn fraction of that offer (evenly spaced in [0,1]). With `normalize=True` (the LLM Rationalis convention) y is min-max scaled to [0,1] so `a`/`b`/`tau` are comparable across parties and games; the burstiness `tau` and CRI are always computed on the normalized fit. Needs >= 3 offers (returns a fit with `n < 3` and `nan` shape metrics otherwise, since a step cannot be identified from two points).

Parameters

Name	Type	Default	Description
`values`		*required*	
`normalize`	`bool`	`True`	

loglog_regret_slope

Fit the log–log slope of cumulative regret.

```
loglog_regret_slope(per_turn_regret) -> LogLogRegret
```

Defined in `interlens.arena.negotiation.analysis.curves` · source

Returns `nan` slope if fewer than 2 turns have positive cumulative regret (a flat zero-regret series is trivially sublinear but the slope is unidentified).

Parameters

Name	Type	Default	Description
`per_turn_regret`		*required*	

no_regret_trend_test

Park trend test on a per-turn regret series (r_t >= 0, surplus-loss units).

```
no_regret_trend_test(per_turn_regret, *, alpha: float = 0.05) ->
    NoRegretTrend
```

Defined in `interlens.arena.negotiation.analysis.curves` · source

Forms cumulative regret `R_t = sum_{s⩽t} r_s` then average regret `a_t = R_t/t` and runs Mann–Kendall for a monotone decreasing trend in `a_t`. Needs >= 3 turns. Ties are corrected in the variance; a continuity correction is applied to S.

Parameters

Name	Type	Default	Description
`per_turn_regret`		*required*	
`alpha`	`float`	`0.05`	

episode_view

Module `interlens.arena.negotiation.analysis.episode_view`

`EpisodeView`: a stored arena `Episode` JSON parsed into a normalized negotiation action series that the metrics read instead of raw `parsed_action` — an ordered `TurnView` list (typed action, deal canonicalized to an index tuple, offer id, any acceptable-offer/belief note, private thinking), the offer registry, the per-round standing offer, and the final deal / reached flag.

The field-name maps below are the single adapter for the arena action layer's serialization: tolerant to the scorable scenario's `{atype, deal_named, offer}` shape (offer ids `P{n}` in proposal order), the canonical typed `{"action": "propose", "deal": [idx...]}` shape, and the older v1 `{proposal, support}` shape.

Classes

Name	Summary
`EpisodeView`	A parsed episode: seats, the turn series, the offer registry, per-round standing offer, and outcome.
`TurnView`	One turn, normalized.

EpisodeView

A parsed episode: seats, the turn series, the offer registry, per-round standing offer, and outcome.

```
EpisodeView(
    episode_id: str,
    arm: str,
    model: str,
    seats: list[str],
    turns: list[TurnView],
    proposals: dict[Any, dict] = dict(),
    final_deal: Deal | None = None,
    reached: bool = False,
    outcome: dict = dict(),
    standing_offer_by_round: dict[int, Deal | None] = dict(),
)
```

Defined in `interlens.arena.negotiation.analysis.episode_view` · source

Parameters

Name	Type	Default	Description
`episode_id`	`str`	*required*	
`arm`	`str`	*required*	
`model`	`str`	*required*	
`seats`	`list[str]`	*required*	
`turns`	list[TurnView]	*required*	
`proposals`	`dict[Any, dict]`	`dict()`	
`final_deal`	`Deal | None`	`None`	
`reached`	`bool`	`False`	
`outcome`	`dict`	`dict()`	
`standing_offer_by_round`	`dict[int, Deal | None]`	`dict()`	

Attributes

Name	Type	Summary
arm	str	
episode_id	str	
final_deal	Deal | None	
model	str	
n_agents	int	
outcome	dict	
proposals	dict[Any, dict]	
reached	bool	
seats	list[str]	
standing_offer_by_round	dict[int, Deal | None]	
turns	list[TurnView]	

Methods

from_episode

```
from_episode(cls, episode: dict, game: GameAnalysis) -> 'EpisodeView'
```

source

Parse a stored `Episode.to_json()` dict into an `EpisodeView` using `game` to canonicalize deals.

Robust to unknown seats (falls back to the episode's `seats` list) and to actions that fail to canonicalize (kept as a turn with `parse_ok=False` and no deal, so nothing is silently dropped).

Parameters

Name	Type	Default	Description
episode	dict	*required*	
game	GameAnalysis	*required*	

proposals_by

```
proposals_by(self, seat: str) -> list[TurnView]
```

source

Every turn on which `seat` registered a proposal (in order).

Parameters

Name	Type	Default	Description
seat	str	*required*	

seat_index

```
seat_index(self, seat: str) -> int
```

source

Parameters

Name	Type	Default	Description
seat	str	*required*	

TurnView

One turn, normalized.

```
TurnView(
    idx: int,
    round: int,
    seat: str,
    action_type: str,
    proposed_deal: Deal | None = None,
    accepted_offer_id: Any = None,
    rejected_offer_id: Any = None,
    walked: bool = False,
    stated_offer: Deal | None = None,
    stated_belief: Any = None,
    thinking: str | None = None,
    message: str | None = None,
    parse_ok: bool = True,
    raw_action: Any = None,
)
```

Defined in `interlens.arena.negotiation.analysis.episode_view` · source

`action_type` is the primary typed action; the granular fields coexist so a v1 turn that both registers a proposal and supports an id populates `proposed_deal` and `accepted_offer_id` together.

Parameters

Name	Type	Default	Description
`idx`	`int`	*required*	
`round`	`int`	*required*	
`seat`	`str`	*required*	
`action_type`	`str`	*required*	
`proposed_deal`	`Deal | None`	`None`	
`accepted_offer_id`	`Any`	`None`	
`rejected_offer_id`	`Any`	`None`	
`walked`	`bool`	`False`	
`stated_offer`	`Deal | None`	`None`	

Name	Type	Default	Description
stated_belief	Any	None	
thinking	str | None	None	
message	str | None	None	
parse_ok	bool	True	
raw_action	Any	None	

Attributes

Name	Type	Summary
accepted_offer_id	Any	
action_type	str	
idx	int	
message	str | None	
parse_ok	bool	
proposed_deal	Deal | None	
raw_action	Any	
rejected_offer_id	Any	
round	int	
seat	str	
stated_belief	Any	
stated_offer	Deal | None	
thinking	str | None	
walked	bool	

game_analysis

Module `interlens.arena.negotiation.analysis.game_analysis`

`GameAnalysis`: the solved-game bundle the metrics read — the adapter seam between interlens' `GameSpec`/`solutions.py` and the pure metric math. Holds everything the metrics need about the *game* (not any one episode): thresholds, score sheets (to score any deal into a surplus vector), the Pareto frontier, and the named solution points (NBS/KS/MNW/utilitarian/egalitarian) as surplus vectors, plus §2.1 descriptors.

Constructed either from a stored `Instance` (`from_instance` — reads interlens' precomputed analysis, no re-solving) or from raw sheets (`from_sheets` — self-contained fallback for fixtures; enumerates the frontier and welfare argmaxes, leaves the axiomatic NBS/KS/MNW `None` unless supplied). `Deal` is `tuple[int, ...]` (one option index per issue); `canonical_deal` maps the forms a stored action may carry (index tuple/list, or `{issue_name: option}` dict) onto it.

Attributes

Name	Type	Summary
`Deal`		
`POINT_NAMES`		

Classes

Name	Summary
`GameAnalysis`	Solved-game bundle: surplus geometry + solution points + descriptors for one instance.

GameAnalysis

Solved-game bundle: surplus geometry + solution points + descriptors for one instance.

```
GameAnalysis(
    n_agents: int,
    thresholds: tuple[float, ...],
    issues: list[tuple[str, list[str]]] = list(),
    sheets: list[list[list[float]]] | None = None,
    frontier: list[tuple[float, ...]] = list(),
    ir_deals: list[Deal] = list(),
    points: dict[str, tuple[float, ...] | None] = dict(),
    descriptors: dict[str, Any] = dict(),
    scale: tuple[float, ...] | None = None,
)
```

Defined in `interlens.arena.negotiation.analysis.game_analysis` · source

Parameters

Name	Type	Default	Description
`n_agents`	`int`	*required*	
`thresholds`	`tuple[float, ...]`	*required*	
`issues`	`list[tuple[str, list[str]]]`	`list()`	
`sheets`	`list[list[list[float]]] | None`	`None`	
`frontier`	`list[tuple[float, ...]]`	`list()`	
`ir_deals`	`list[Deal]`	`list()`	
`points`	`dict[str, tuple[float, ...] | None]`	`dict()`	
`descriptors`	`dict[str, Any]`	`dict()`	
`scale`	`tuple[float, ...] | None`	`None`	

Attributes

Name	Type	Summary
`descriptors`	`dict[str, Any]`	

Name	Type	Summary
`frontier`	`list[tuple[float, ...]]`	
`ir_deals`	`list[Deal]`	
`issues`	`list[tuple[str, list[str]]]`	
`n_agents`	`int`	
`points`	`dict[str, tuple[float, ...] | None]`	
`scale`	`tuple[float, ...] | None`	
`sheets`	`list[list[list[float]]] | None`	
`thresholds`	`tuple[float, ...]`	

Methods

`canonical_deal`

```
canonical_deal(self, deal_like: Any) -> Deal
```

source

Map a proposed deal onto `tuple[int, ...]` (option index per issue).

Accepts an index tuple/list, or a `{issue_name: option}` dict whose values are option names or indices. Raises `ValueError` on an incomplete/unknown deal so a malformed proposal is a loud parse failure, not a silent mis-score.

Parameters

Name	Type	Default	Description
`deal_like`	`Any`	*required*	

`deal_frontier_distance`

```
deal_frontier_distance(self, deal_like: Any) -> float
```

source

Scale-invariant distance from a realized deal to the Pareto frontier (0 iff efficient).

Delegates to the sibling `solutions.distance_to_frontier` (normalized-surplus space) for instances built via `from_instance`; falls back to the Euclidean-on-surplus metric for hand-built fixtures.

Parameters

Name	Type	Default	Description
`deal_like`	`Any`	*required*	

deal_point_distance

```
deal_point_distance(self, deal_like: Any, name: str) -> float
```

source

Scale-invariant distance from a realized deal to a named solution point (`nbs`/`ks`/…).

Delegates to `solutions.distance_to_solution` for `from_instance` games, else Euclidean fallback. `nan` if that point was not computed.

Parameters

Name	Type	Default	Description
`deal_like`	`Any`	*required*	
`name`	`str`	*required*	

distance_to_frontier

```
distance_to_frontier(self, x: Sequence[float], *, normalized: bool = True)
    -> float
```

source

Euclidean distance from surplus vector `x` to the Pareto frontier (0 if efficient).

With `normalized` each coordinate is divided by that party's `scale` so parties are commensurable.

Parameters

Name	Type	Default	Description
`x`	`Sequence[float]`	*required*	
`normalized`	`bool`	`True`	

`distance_to_point`

```
distance_to_point(
    self,
    x: Sequence[float],
    name: str,
    *,
    normalized: bool = True,
) -> float
```

source

Euclidean distance from `x` to a named solution point (`nbs`/`ks`/…).

`nan` if that point was not computed for this instance.

Parameters

Name	Type	Default	Description
`x`	`Sequence[float]`	*required*	
`name`	`str`	*required*	
`normalized`	`bool`	`True`	

from_instance

```
from_instance(cls, instance: Any) -> 'GameAnalysis'
```

source

Build from a stored `Instance` (or its `.to_json()` dict) whose `payload` is a `GameSpec`.

Delegates all solving to interlens' `solutions.py` (frontier via `pareto_mask`, points + descriptors via `analyze`, read from `Instance.solution` when present, recomputed otherwise). interlens is imported lazily so the pure metric math (and `from_sheets`) never require it.

Parameters

Name	Type	Default	Description
instance	Any	*required*	

from_sheets

```
from_sheets(
    cls,
    sheets: list[list[list[float]]],
    thresholds: Any,
    *,
    issues: list[tuple[str, list[str]]] | None = None,
    points: dict[str, tuple] | None = None,
) -> 'GameAnalysis'
```

source

Enumerate the deal space from raw score sheets and compute the frontier, IR set, welfare argmaxes, and descriptors.

A self-contained fallback and the tests' constructor — `solutions.py` is the authority for the axiomatic NBS/KS/MNW points, which are only filled here if passed in `points`.

Parameters

Name	Type	Default	Description
`sheets`	`list[list[list[float]]]`	*required*	
`thresholds`	`Any`	*required*	
`issues`	`list[tuple[str, list[str]]] | None`	`None`	
`points`	`dict[str, tuple] | None`	`None`	

surplus

```
surplus(self, deal_like: Any) -> tuple[float, ...]
```

source

Surplus vector `(u_i(deal) - tau_i)_i` for a deal in any accepted form.

Computed from the score sheets when present (exact for any deal), else looked up in the cached enumeration.

Parameters

Name	Type	Default	Description
`deal_like`	`Any`	*required*	

metrics

Module `interlens.arena.negotiation.analysis.metrics`

The divergence metric suite: outcome-, turn-, and faithfulness-level measures over one episode.

Pure functions of a solved game (`GameAnalysis`), the parsed episode (`EpisodeView`), and — for the regret/belief measures — the oracle annotations (`EpisodeAnnotation`). Composes the `surplus` and `curves` math layers (never re-derives); `taxonomy.py` and `report.py` call these. Groups: `outcome_metrics` (deal, surplus, distance-to-frontier/NBS/KS, USW/ESW/NSW/Gini, U vs U*, welfare trajectory); turn-level primitives (regret series + no-regret tests, the mechanical IR/dominated detectors, concession fits); and faithfulness (stated-offer vs offers made, stated-belief vs oracle posterior). Every function degrades to `nan`/empty rather than raising when an input (a solution point, an annotation) is absent.

Attributes

Name	Type	Summary
`DEFAULT_REGRET_THRESHOLD`		

Functions

Name	Summary
`belief_calibration`	Belief-calibration error: mean L1 distance between the model's stated belief and the oracle posterior over the turns where both are present, when both are numeric vectors/distributions.

Name	Summary
`concession_fits`	Per-party concession-curve fit on the sequence of the party's OWN surplus across its OWN successive proposals (the offers it puts on the table).
`divergence_turns`	Turn indices whose per-turn regret exceeds `threshold` — the localized divergence points.
`dominated_proposals`	Turns on which the proposed deal is Pareto-dominated — a strictly-better-for-everyone alternative existed.
`episode_metrics`	All divergence metrics for one episode, in one dict — the row the atlas aggregates over.
`ir_violations`	Turns on which a party proposed or accepted a deal scoring below its OWN threshold (surplus < 0) — the individual-rationality / 'wrong deal' failure (Abdelnabi wrong-deal rate 7–20%; multi-buyer sell-below-cost up to 38.3%).
`no_regret_tests`	Park no-regret trend + log–log tests over the per-turn regret series.
`normalized_nash_welfare`	Bounded Nash-welfare endpoint in per-party feasible-surplus units.
`outcome_metrics`	Outcome-level divergence for one episode.
`regret_series`	The per-turn headline regret series (surplus loss) from an annotation,

Name	Summary
	0.0 where a turn has no recorded regret.
`stated_offer_faithfulness`	Internal faithfulness (LAMEN, Davidson 2024): are a party's actual proposals consistent with its own machine-readable 'currently acceptable offer' note?
`welfare_trajectory`	Per-round utilitarian welfare of the standing (tabled) deal, with an OLS slope and variance.

belief_calibration

Belief-calibration error: mean L1 distance between the model's stated belief and the oracle posterior over the turns where both are present, when both are numeric vectors/distributions.

```
belief_calibration(annotation: EpisodeAnnotation | None) -> dict
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

`nan` when the scenario elicits no beliefs or the oracle recorded none (belief calibration is the BeliefOracle-dependent metric).

Parameters

Name	Type	Default	Description
`annotation`	EpisodeAnnotation | None	*required*	

concession_fits

Per-party concession-curve fit on the sequence of the party's OWN surplus across its OWN successive proposals (the offers it puts on the table).

```
concession_fits(game: GameAnalysis, view: EpisodeView) -> dict[str,
    curves.ConcessionFit]
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

Parties with < 3 proposals get an `n<3` fit with `nan` shape metrics. Read τ (burstiness) and CRI (rigidity) off each fit.

Parameters

Name	Type	Default	Description
`game`	GameAnalysis	*required*	
`view`	EpisodeView	*required*	

divergence_turns

Turn indices whose per-turn regret exceeds `threshold` — the localized divergence points.

```
divergence_turns(
    annotation: EpisodeAnnotation | None,
    threshold: float = DEFAULT_REGRET_THRESHOLD,
) -> list[int]
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

Parameters

Name	Type	Default	Description
`annotation`	EpisodeAnnotation | None	*required*	
`threshold`	`float`	`DEFAULT_REGRET_THRESHOLD`	

dominated_proposals

Turns on which the proposed deal is Pareto-dominated — a strictly-better-for-everyone alternative existed.

```
dominated_proposals(game: GameAnalysis, view: EpisodeView) -> list[dict]
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

Reports both dominance vs the Pareto frontier and, more sharply, vs the IR (feasible) set (a Pareto improvement everyone would have accepted). Mechanical.

Parameters

Name	Type	Default	Description
`game`	GameAnalysis	*required*	
`view`	EpisodeView	*required*	

episode_metrics

All divergence metrics for one episode, in one dict — the row the atlas aggregates over.

```
episode_metrics(
    game: GameAnalysis,
    view: EpisodeView,
    annotation: EpisodeAnnotation | None = None,
) -> dict
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

Parameters

Name	Type	Default	Description
`game`	GameAnalysis	*required*	
`view`	EpisodeView	*required*	
`annotation`	EpisodeAnnotation | None	`None`	

ir_violations

Turns on which a party proposed or accepted a deal scoring below its OWN threshold (surplus < 0) — the individual-rationality / 'wrong deal' failure (Abdelnabi wrong-deal rate 7–20%; multi-buyer sell-below-cost up to 38.3%).

```
ir_violations(game: GameAnalysis, view: EpisodeView) -> list[dict]
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

Fully mechanical: reads the acting party's surplus off the score sheet.

Parameters

Name	Type	Default	Description
`game`	GameAnalysis	*required*	
`view`	EpisodeView	*required*	

no_regret_tests

Park no-regret trend + log–log tests over the per-turn regret series.

```
no_regret_tests(annotation: EpisodeAnnotation | None) -> dict
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

Returns `None` fields when there is no annotation or the series is too short.

Parameters

Name	Type	Default	Description
`annotation`	EpisodeAnnotation | None	*required*	

normalized_nash_welfare

Bounded Nash-welfare endpoint in per-party feasible-surplus units.

```
normalized_nash_welfare(game: GameAnalysis, surplus: tuple[float, ...] |
    None) -> float
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

Each surplus is divided by that party's maximum surplus over the individually rational set (`game.scale`). A missing outcome, a non-positive surplus, or an unavailable/invalid scale scores zero. The geometric mean is in `[0, 1]` for solved finite games and is invariant to independent positive affine rescalings of the parties' utility sheets.

Parameters

Name	Type	Default	Description
`game`	GameAnalysis	*required*	
`surplus`	`tuple[float, ...] | None`	*required*	

outcome_metrics

Outcome-level divergence for one episode.

```
outcome_metrics(game: GameAnalysis, view: EpisodeView) -> dict
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

On a no-deal episode the welfare fields are 0 (USW/ESW/NSW) and Gini is `nan` (no distribution), while `surplus` is `None` — this is the U-vs-U* distinction: the unconditional welfare counts no-deal as 0, the conditional (`*_conditional`) is `None` and excluded from deal-only aggregates upstream.

Parameters

Name	Type	Default	Description
`game`	GameAnalysis	*required*	
`view`	EpisodeView	*required*	

regret_series

The per-turn headline regret series (surplus loss) from an annotation, 0.0 where a turn has no recorded regret.

```
regret_series(annotation: EpisodeAnnotation | None) -> list[float]
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

Empty when there is no annotation (no oracle pass has been run).

Parameters

Name	Type	Default	Description
`annotation`	EpisodeAnnotation | None	*required*	

stated_offer_faithfulness

Internal faithfulness (LAMEN, Davidson 2024): are a party's actual proposals consistent with its own machine-readable 'currently acceptable offer' note?

```
stated_offer_faithfulness(game: GameAnalysis, view: EpisodeView) -> dict
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

For each proposal made after the party states an acceptable offer, faithful iff the proposal is at least as good for the party as its stated floor (own-surplus(proposal) >= own-surplus(stated floor) – ε). Returns the overall rate and per-party rates; the rate is `nan` when the scenario elicits no acceptable-offer note (none of the turns carry `stated_offer`).

Parameters

Name	Type	Default	Description
`game`	GameAnalysis	*required*	
`view`	EpisodeView	*required*	

welfare_trajectory

Per-round utilitarian welfare of the standing (tabled) deal, with an OLS slope and variance.

```
welfare_trajectory(game: GameAnalysis, view: EpisodeView) -> dict
```

Defined in `interlens.arena.negotiation.analysis.metrics` · source

A negative slope = deals get collectively *worse* over rounds (Study B: Phi-3.5 USW slope −5.8); high variance = the table thrashes. `nan` slope when fewer than two rounds carry a standing offer.

Parameters

Name	Type	Default	Description
`game`	GameAnalysis	*required*	
`view`	EpisodeView	*required*	

rollout

Module `interlens.arena.negotiation.analysis.rollout`

Counterfactual-rollout regret: label a divergence by Δ expected surplus, not by action mismatch. At turn t, roll k continuations from the model's action and from the oracle's action against the *same frozen counterpart policies* and score the gap in the acting party's expected surplus (mismatch alone over-labels — many actions are near-optimal). Engine-agnostic: works against a small `RolloutEnv` protocol + a per-seat `Policy` map; `policy_rollout_env` is the (unimplemented) hook to bind `ScorableNegotiation` + the rational policies.

Attributes

Name	Type	Summary
`Action`		
`Policy`		
`State`		

Classes

Name	Summary
`CounterfactualRegret`	Δ expected surplus between the oracle action and the model action for the acting `agent`: positive = the oracle's action leads to more surplus (a real divergence), 0 = the model's action was as good.
`RolloutEnv`	The negotiation transition a rollout needs.

Functions

Name	Summary
`counterfactual_regret`	Roll `k` continuations from the model's action and from the oracle's action (both taken by `agent` at `state`, then everyone — including `agent` — follows `policies`) and report the mean-surplus gap.
`policy_rollout_env`	HOOK (unimplemented): bind the rational policies as rollout counterparts — `NegotiationState` as the env `State`, `ScorableNegotiation.apply` as the transition, each frozen counterpart a `policy(state)→action`.
`rollout`	Drive `state` to a terminal state, each seat acting by its policy in `policies`.

CounterfactualRegret

Δ expected surplus between the oracle action and the model action for the acting `agent`: positive = the oracle's action leads to more surplus (a real divergence), 0 = the model's action was as good.

```
CounterfactualRegret(
    agent: str,
    surplus_model: float,
    surplus_oracle: float,
    delta: float,
    k: int,
)
```

Defined in `interlens.arena.negotiation.analysis.rollout` · source

Parameters

Name	Type	Default	Description
`agent`	`str`	*required*	
`surplus_model`	`float`	*required*	
`surplus_oracle`	`float`	*required*	
`delta`	`float`	*required*	
`k`	`int`	*required*	

Attributes

Name	Type	Summary
`agent`	`str`	
`delta`	`float`	
`k`	`int`	
`surplus_model`	`float`	
`surplus_oracle`	`float`	

Methods

to_json

```
to_json(self) -> dict
```

source

RolloutEnv

The negotiation transition a rollout needs.

```
RolloutEnv()
```

Defined in `interlens.arena.negotiation.analysis.rollout` · source

Inherits from: `Protocol`

Implementations must not mutate the input state in `step` (return a fresh/cloned state) so a single state can seed many counterfactual branches.

Methods

`legal_actions`

```
legal_actions(self, state: State, seat: str) -> list
```

source

Legal actions for `seat` at `state` (used to validate / enumerate; may be empty if unbounded).

Parameters

Name	Type	Default	Description
`state`	`State`	*required*	
`seat`	`str`	*required*	

`next_seat`

```
next_seat(self, state: State) -> str | None
```

source

The seat to move now, or `None` if the state is terminal.

Parameters

Name	Type	Default	Description
`state`	`State`	*required*	

step

```
step(self, state: State, seat: str, action: Action) -> State
```

source

Apply `action` by `seat`; return the successor state (no in-place mutation).

Parameters

Name	Type	Default	Description
`state`	`State`	*required*	
`seat`	`str`	*required*	
`action`	`Action`	*required*	

surplus

```
surplus(self, state: State, agent: str) -> float
```

source

Terminal surplus for `agent` (call only at a terminal state).

Parameters

Name	Type	Default	Description
`state`	`State`	*required*	

Name	Type	Default	Description
`agent`	`str`	*required*	

counterfactual_regret

Roll `k` continuations from the model's action and from the oracle's action (both taken by `agent` at `state`, then everyone — including `agent` — follows `policies`) and report the mean-surplus gap.

```
counterfactual_regret(
    env: RolloutEnv,
    state: State,
    agent: str,
    model_action: Action,
    oracle_action: Action,
    policies: dict[str, Policy],
    *,
    k: int = 8,
) -> CounterfactualRegret
```

Defined in `interlens.arena.negotiation.analysis.rollout` · source

With deterministic policies the k rollouts coincide and `k` is effectively 1; `k` matters once a policy is stochastic (a mixed strategy or, later, an LLM counterpart at temperature). `policies` must include a continuation policy for `agent` itself (its post-branch behavior).

Parameters

Name	Type	Default	Description
`env`	RolloutEnv	*required*	
`state`	`State`	*required*	
`agent`	`str`	*required*	
`model_action`	`Action`	*required*	
`oracle_action`	`Action`	*required*	
`policies`	`dict[str, Policy]`	*required*	
`k`	`int`	`8`	

policy_rollout_env

HOOK (unimplemented): bind the rational policies as rollout counterparts — `NegotiationState` as the env `State`, `ScorableNegotiation.apply` as the transition, each frozen counterpart a `policy(state)→action`.

```
policy_rollout_env(scenario: Any, instance: Any)
```

Defined in `interlens.arena.negotiation.analysis.rollout` · source

The env-agnostic `counterfactual_regret`/`rollout` core is already tested via a synthetic env.

Parameters

Name	Type	Default	Description
`scenario`	`Any`	*required*	
`instance`	`Any`	*required*	

rollout

Drive `state` to a terminal state, each seat acting by its policy in `policies`.

```
rollout(
    env: RolloutEnv,
    state: State,
    policies: dict[str, Policy],
    *,
    max_steps: int = 500,
) -> State
```

Defined in `interlens.arena.negotiation.analysis.rollout` · source

`max_steps` guards against a non-terminating env (a policy that never closes).

Parameters

Name	Type	Default	Description
`env`	RolloutEnv	*required*	
`state`	`State`	*required*	
`policies`	`dict[str, Policy]`	*required*	
`max_steps`	`int`	`500`	

surplus

Module `interlens.arena.negotiation.analysis.surplus`

Pure surplus-vector math: Pareto geometry, dominance, and distances.

Every function operates on a *surplus vector* `x = (x_i)` in surplus units `x_i = u_i(deal) - tau_i` — the scale-invariant analysis object across parties with private point scales.

The **welfare and inequality scalars** (`utilitarian_welfare`/`egalitarian_welfare`/ `nash_welfare`/ `nash_welfare_geomean`/`gini`) are re-exported from interlens' `arena.negotiation.solutions`, which is their one home: the scenario scores each episode's outcome with the same functions this layer aggregates, so a run's reported welfare and the frontier it is compared against cannot drift apart. The names here are the local aliases `metrics.py`/`taxonomy.py` already use.

The axiomatic solution points (NBS/KS/MNW and the frontier) come from `solutions.py` via `game_analysis.py`; the Pareto helpers below (`pareto_frontier`, `dominating_alternatives`) are the small domination primitives the metrics need (dominated-proposal detection) plus a fixture fallback, not a re-implementation of those.

Attributes

Name	Type	Summary
`Vec`		

Functions

Name	Summary
`distance_to_set`	Minimum Euclidean distance from `x` to any vector in `points` (0 if `x` is one of them).

Name	Summary
dominates	y Pareto-dominates x: y_i ⩾ x_i for all i and (if strict_any) strictly greater for at least one i.
dominating_alternatives	Every candidate surplus vector that Pareto-dominates x.
euclidean	Euclidean distance ||x - y|| in surplus space, optionally after dividing each coordinate by a per-party scale (e.g. that party's max feasible surplus) so parties on different point scales are commensurable.
pareto_frontier	The non-dominated subset of a set of surplus vectors (the discrete Pareto frontier).

distance_to_set

Minimum Euclidean distance from `x` to any vector in `points` (0 if `x` is one of them).

```
distance_to_set(x: Vec, points: Sequence[Vec], *, scale: Vec | None =
    None) -> float
```

Defined in `interlens.arena.negotiation.analysis.surplus` · source

Used for distance-to-Pareto-frontier: how far the realized outcome sits from the efficient set.

Parameters

Name	Type	Default	Description
`x`	`Vec`	*required*	
`points`	`Sequence[Vec]`	*required*	
`scale`	`Vec | None`	`None`	

dominates

`y` Pareto-dominates `x`: `y_i ⩾ x_i` for all i and (if `strict_any`) strictly greater for at least one i.

```
dominates(y: Vec, x: Vec, *, strict_any: bool = True) -> bool
```

Defined in `interlens.arena.negotiation.analysis.surplus` · source

With `strict_any=False` this is weak dominance (`y_i ⩾ x_i` for all i).

Parameters

Name	Type	Default	Description
`y`	`Vec`	*required*	
`x`	`Vec`	*required*	
`strict_any`	`bool`	`True`	

dominating_alternatives

Every candidate surplus vector that Pareto-dominates `x`.

```
dominating_alternatives(x: Vec, candidates: Sequence[Vec]) -> list[Vec]
```

Defined in `interlens.arena.negotiation.analysis.surplus` · source

Used to flag a *dominated proposal*: a party proposed/accepted `x` when some other feasible deal made everyone at least as well off and someone better (a left-on-the-table Pareto improvement).

Parameters

Name	Type	Default	Description
`x`	`Vec`	*required*	
`candidates`	`Sequence[Vec]`	*required*	

euclidean

Euclidean distance `||x - y||` in surplus space, optionally after dividing each coordinate by a per-party `scale` (e.g. that party's max feasible surplus) so parties on different point scales are commensurable.

```
euclidean(x: Vec, y: Vec, *, scale: Vec | None = None) -> float
```

Defined in `interlens.arena.negotiation.analysis.surplus` · source

With `scale=None` distances are in raw surplus units.

Parameters

Name	Type	Default	Description
`x`	`Vec`	*required*	
`y`	`Vec`	*required*	
`scale`	`Vec | None`	`None`	

pareto_frontier

The non-dominated subset of a set of surplus vectors (the discrete Pareto frontier).

```
pareto_frontier(surplus_vectors: Sequence[Vec]) -> list[tuple[float, ...]]
```

Defined in `interlens.arena.negotiation.analysis.surplus` · source

O(|S|^2) brute force, exact at an enumerable deal-space scale. This is a fallback/utility for fixtures and audits — production frontiers are read from the precomputed `Instance.solution`.

Parameters

Name	Type	Default	Description
`surplus_vectors`	`Sequence[Vec]`	*required*	

taxonomy

Module `interlens.arena.negotiation.analysis.taxonomy`

The 12-row LLM-negotiation failure taxonomy as executable checks.

Each check runs over (solved game, parsed episode, oracle annotation) and returns a `CategoryResult` (fired, rate, evidence). Rows are tiered by checkability: Tier 1 mechanical (1,2,6,8,9 — deterministic, though 2/6 need an acceptance/belief oracle pass and report `nan` without one), Tier 2 curve/statistical (3,4,7,10 — from the concession fits; 10 is cross-condition and defers to `report.py`), Tier 3 judge-dependent (5,11,12 — `implemented=False` stubs with an LLM-judge hook). `taxonomy_report` runs every row over one episode; `TAXONOMY` is the ordered row list.

Attributes

Name	Type	Summary
`FLAGS`		
`ORACLE_FLAGS`		
`TAXONOMY`	list[tuple[int, str, Tier, list[str], Callable]]	

Classes

Name	Summary
`CategoryResult`	Outcome of one taxonomy check on one episode.
`Tier`	

Functions

Name	Summary
check_anchoring	Rigid extreme anchoring: a party makes >=2 offers whose own-surplus barely moves (range below 5% of its feasible scale) or fits a near-vertical/degenerate concession curve.
check_arithmetic_error	Recompute a party's stated own-score for a deal against the true score sheet.
check_belief_miscalibration	
check_deal_closing	
check_exploration_failure	Commits (first accept / final proposal) before gathering information: accepts within the first two turns with no prior counter-proposal from others (multi-buyer 'fails to explore the pool').
check_goal_inconsistency	A party's stated acceptable-offer floor drifts with no new incoming offer since its last statement (Davidson 2024: target/limit drifts without new information).
check_ir_violation	
check_manipulation_susceptibility	STUB (Tier 3).
check_power_insensitivity	Identical strategy across power-asymmetry conditions where the oracle strategy differs — inherently CROSS-condition.

Name	Summary
check_preference_action_gap	STUB (Tier 3).
check_premature_concession	Excess/early concession: a party gives up a large fraction of its own surplus across its offers (high burstiness τ with a downward step), especially early.
check_tactic_deficit	STUB (Tier 3).
taxonomy_report	Run every taxonomy row over one episode, in row order.
taxonomy_rows	The taxonomy as a static table (id, name, tier, citations, implemented) for docs/report headers.

Tier

```
Tier()
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Inherits from: `IntEnum`

Attributes

Name	Type	Summary
`CURVE`		
`JUDGE`		
`MECHANICAL`		

CategoryResult

Outcome of one taxonomy check on one episode.

```
CategoryResult(
    category_id: int,
    name: str,
    tier: Tier,
    fired: bool,
    rate: float,
    evidence: list[dict] = list(),
    implemented: bool = True,
    note: str = '',
)
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Parameters

Name	Type	Default	Description
`category_id`	`int`	*required*	
`name`	`str`	*required*	
`tier`	Tier	*required*	
`fired`	`bool`	*required*	
`rate`	`float`	*required*	
`evidence`	`list[dict]`	`list()`	
`implemented`	`bool`	`True`	
`note`	`str`	`''`	

Attributes

Name	Type	Summary
`category_id`	`int`	
`evidence`	`list[dict]`	
`fired`	`bool`	

Name	Type	Summary
`implemented`	`bool`	
`name`	`str`	
`note`	`str`	
`rate`	`float`	
`tier`	Tier	

Methods

to_json

```
to_json(self) -> dict
```

source

check_anchoring

Rigid extreme anchoring: a party makes >=2 offers whose own-surplus barely moves (range below 5% of its feasible scale) or fits a near-vertical/ degenerate concession curve.

```
check_anchoring(game, view, annotation) -> CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Read from the concession fits.

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

check_arithmetic_error

Recompute a party's stated own-score for a deal against the true score sheet.

```
check_arithmetic_error(game, view, annotation) -> CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Mechanical when the action schema carries a machine-readable `stated_score`; annotation flags (written by an oracle pass that parsed the CoT) are also honored.

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

check_belief_miscalibration

```
check_belief_miscalibration(game, view, annotation) -> CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

check_deal_closing

```
check_deal_closing(game, view, annotation) -> CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

check_exploration_failure

Commits (first accept / final proposal) before gathering information: accepts within the first two turns with no prior counter-proposal from others (multi-buyer ‘fails to explore the pool’).

```
check_exploration_failure(game, view, annotation) -> CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

check_goal_inconsistency

A party's stated acceptable-offer floor drifts with no new incoming offer since its last statement (Davidson 2024: target/limit drifts without new information).

```
check_goal_inconsistency(game, view, annotation) -> CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Mechanical proxy over the stated-offer notes.

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

check_ir_violation

```
check_ir_violation(game, view, annotation) -> CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

check_manipulation_susceptibility

STUB (Tier 3).

```
check_manipulation_susceptibility(game, view, annotation) ->
    CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Outcome delta when the counterpart uses scripted pressure tactics (NegotiationArena; converges with the conv_dataset confidence-vs-logic finding). Hook: paired ±scripted-adversary arms; the report contrasts the party's surplus/capitulation with vs without the pressure participant.

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

check_power_insensitivity

Identical strategy across power-asymmetry conditions where the oracle strategy differs — inherently CROSS-condition.

```
check_power_insensitivity(game, view, annotation) -> CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Cannot fire on a single episode; `report.py` computes it across matched arms.

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

check_preference_action_gap

STUB (Tier 3).

```
check_preference_action_gap(game, view, annotation) -> CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Correct opponent-preference inference in the CoT but the offer is unchanged vs the best-response oracle (Counterparty Modeling, arXiv:2605.16575). Hook: an LLM judge extracts the inferred opponent preference from `TurnView.thinking`; compare the made offer to the BestResponseOracle action given that (correct) inference. Fires when inference is correct but the action does not move toward BR.

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

check_premature_concession

Excess/early concession: a party gives up a large fraction of its own surplus across its offers (high burstiness τ with a downward step), especially early.

```
check_premature_concession(game, view, annotation) -> CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Read from the concession fits (τ, CRI) and the own-surplus drop.

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

check_tactic_deficit

STUB (Tier 3).

```
check_tactic_deficit(game, view, annotation) -> CategoryResult
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Tactic-frequency profile vs successful humans (Diplomacy tactic taxonomy, arXiv:2512.18292). Hook: an LLM tactic classifier (validated to Gwet's AC1 >= 0.65) labels each message with the 8-tactic scheme; compare the frequency profile to a human reference.

Parameters

Name	Type	Default	Description
`game`		*required*	
`view`		*required*	
`annotation`		*required*	

taxonomy_report

Run every taxonomy row over one episode, in row order.

```
taxonomy_report(
    game: GameAnalysis,
    view: EpisodeView,
    annotation: EpisodeAnnotation | None = None,
) -> list[CategoryResult]
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

Parameters

Name	Type	Default	Description
`game`	GameAnalysis	*required*	
`view`	EpisodeView	*required*	
`annotation`	EpisodeAnnotation | None	`None`	

taxonomy_rows

The taxonomy as a static table (id, name, tier, citations, implemented) for docs/report headers.

```
taxonomy_rows() -> list[dict]
```

Defined in `interlens.arena.negotiation.analysis.taxonomy` · source

belief_accuracy

Module `interlens.arena.negotiation.belief_accuracy`

How well does a :class:`~interlens.arena.negotiation.beliefs.BeliefState` actually know its opponent?

Three read-only scores of one posterior against the opponent's TRUE private sheet, all computed over the whole enumerated deal space as gemvs against the belief's cached type-by-deal matrices:

`posterior_mass_true_type` Posterior probability on the single grid type nearest the true sheet (:func:`nearest_type_index`) — the identification score. `1.0` = the posterior has collapsed onto the truth. `expected_utility_rmse` Root-mean-square error between posterior-expected opponent utility and true opponent utility, per deal, on the type grid's `[0, 1]` scale — the calibration score. `0.0` = exact. `accept_set_f1` F1 between the set of deals the posterior says the opponent accepts (accept probability `> 0.5`) and the set it really accepts — the decision-relevant score, since acceptance is the only thing a best-response policy consumes the belief FOR. `1.0` = exact.

Everything here is pure observation: nothing mutates the belief, the sheet, or any policy state, so a caller may compute these beside a live negotiation without changing a single move.

Scale. An :class:`~interlens.arena.negotiation.beliefs.OpponentType` scores deals in `[0, 1]`; a real :class:`~interlens.arena.negotiation.sheets.ScoreSheet` scores them in points. Because both are ADDITIVE, the sheet's min-max normalization over the deal space is exactly the Hindriks-Tykhonov form the grid enumerates (per-issue weights `propto` the issue's point range, per-issue evaluators rescaled to `[0, 1]`), so :func:`true_opponent` maps a sheet onto the grid's scale without approximation, and a sheet whose weights and shapes happen to be on the grid lands exactly on its own type. The accept set is always read off the RAW points, where it is exact by definition.

Attributes

Name	Type	Summary
METRICS		
PERFECT		

Classes

Name	Summary
TrueOpponent	One opponent's ground truth, precomputed once per episode against a fixed deal ordering.

Functions

Name	Summary
auc	Trapezoidal area under a metric's turn series, divided by the number of intervals — i.e. the TIME-MEAN of the metric, on the metric's own scale rather than an area that grows with episode length (so a 4-round and a 12-round episode are comparable).
f1	F1 between two boolean sets over the same index.
metrics	The three scores of belief against truth, as a JSON-safe dict keyed by :data:METRICS.

Name	Summary
`nearest_type_index`	The grid type closest to a true `(utility, threshold)` pair, and its distance.
`normalized_utility`	`(utility, threshold, raw_utility)` for `sheet` over `deals_arr` (`(D, J)` option indices).
`true_opponent`	Precompute one opponent's :class:`TrueOpponent` against `belief`'s grid — do this ONCE per episode.

TrueOpponent

One opponent's ground truth, precomputed once per episode against a fixed deal ordering.

```
TrueOpponent(
    utility: np.ndarray,
    threshold: float,
    accepts: np.ndarray,
    type_index: int,
    type_distance: float,
)
```

Defined in `interlens.arena.negotiation.belief_accuracy` · source

Attributes

utility : np.ndarray `(D,)` true utility on the type grid's `[0, 1]` scale (min-max over the deal space). threshold : float The true reservation on that same scale. accepts : np.ndarray `(D,)` bool — the true accept set, read off RAW points (`utility ≥ threshold`) so no normalization can move its boundary. type_index : int Index into the belief grid of the type nearest this sheet (:func:`nearest_type_index`). type_distance : float That type's distance (0.0 when the sheet lies exactly on the grid), kept so a caller can report how well the grid could POSSIBLY have done before reading how well it did.

Parameters

Name	Type	Default	Description
`utility`	`np.ndarray`	*required*	
`threshold`	`float`	*required*	
`accepts`	`np.ndarray`	*required*	
`type_index`	`int`	*required*	
`type_distance`	`float`	*required*	

Attributes

Name	Type	Summary
`accepts`	`np.ndarray`	
`threshold`	`float`	
`type_distance`	`float`	
`type_index`	`int`	
`utility`	`np.ndarray`	

auc

Trapezoidal area under a metric's turn series, divided by the number of intervals — i.e. the TIME-MEAN of the metric, on the metric's own scale rather than an area that grows with episode length (so a 4-round and a 12-round episode are comparable).

```
auc(series: list[float]) -> float
```

Defined in `interlens.arena.negotiation.belief_accuracy` · source

A single point returns itself; an empty series returns `0.0`.

Parameters

Name	Type	Default	Description
`series`	`list[float]`	*required*	

f1

F1 between two boolean sets over the same index.

```
f1(predicted: np.ndarray, actual: np.ndarray) -> float
```

Defined in `interlens.arena.negotiation.belief_accuracy` · source

Two EMPTY sets score `1.0` (they agree exactly); one empty and one not scores `0.0`.

Parameters

Name	Type	Default	Description
`predicted`	`np.ndarray`	*required*	
`actual`	`np.ndarray`	*required*	

metrics

The three scores of `belief` against `truth`, as a JSON-safe dict keyed by :data:`METRICS`.

```
metrics(belief: BeliefState, deals_arr: np.ndarray, truth: TrueOpponent) -
    > dict
```

Defined in `interlens.arena.negotiation.belief_accuracy` · source

Three gemvs against the belief's cached matrices (posterior mass, expected utility, accept probability); no loop over deals or types, and nothing is mutated.

Parameters

Name	Type	Default	Description
`belief`	BeliefState	*required*	
`deals_arr`	`np.ndarray`	*required*	
`truth`	TrueOpponent	*required*	

nearest_type_index

The grid type closest to a true `(utility, threshold)` pair, and its distance.

```
nearest_type_index(
    belief: BeliefState,
    deals_arr: np.ndarray,
    utility: np.ndarray,
    threshold: float,
) -> tuple[int, float]
```

Defined in `interlens.arena.negotiation.belief_accuracy` · source

Distance is `rmse(type utility, true utility) + |type tau - true tau|`: agreement of the whole utility FUNCTION across every deal (not of a parameterization, which the grid samples only coarsely) plus agreement of the reservation, both already on the same `[0, 1]` scale so they add without a weight to argue about. A sheet that lies exactly on the grid scores 0.0 and therefore maps to itself, which is the property that makes `posterior_mass_true_type` mean what its name says; a sheet off the grid maps to the type whose induced behaviour over the deal space is closest, which is the only thing a posterior over this grid could ever concentrate on.

Parameters

Name	Type	Default	Description
`belief`	BeliefState	*required*	
`deals_arr`	`np.ndarray`	*required*	
`utility`	`np.ndarray`	*required*	
`threshold`	`float`	*required*	

normalized_utility

`(utility, threshold, raw_utility)` for `sheet` over `deals_arr` (`(D, J)` option indices).

```
normalized_utility(sheet, deals_arr: np.ndarray) -> tuple[np.ndarray,
    float, np.ndarray]
```

Defined in `interlens.arena.negotiation.belief_accuracy` · source

The first two are min-max normalized onto the type grid's `[0, 1]` scale (the additive sheet's span is `sum_j (max_j - min_j)`, so this is the exact counterpart of the grid's construction); the third is the raw points, which the accept set is defined on. A degenerate sheet (every deal worth the same) normalizes to all-zeros with a threshold of `0.0` when it is met and `1.0` when it is not, so the accept set the normalized pair implies still agrees with the raw one.

Parameters

Name	Type	Default	Description
`sheet`		*required*	
`deals_arr`	`np.ndarray`	*required*	

true_opponent

Precompute one opponent's :class:`TrueOpponent` against `belief`'s grid — do this ONCE per episode.

```
true_opponent(sheet, deals_arr: np.ndarray, belief: BeliefState) ->
    TrueOpponent
```

Defined in `interlens.arena.negotiation.belief_accuracy` · source

`belief` is used only for its (immutable, process-cached) type grid, so any belief state built on the same option counts gives the same answer.

Parameters

Name	Type	Default	Description
`sheet`		*required*	
`deals_arr`	`np.ndarray`	*required*	
`belief`	BeliefState	*required*	

beliefs

Module `interlens.arena.negotiation.beliefs`

Bayesian (and frequency-model fallback) belief oracle over an enumerated opponent-type grid.

Literature grounding:

- **Hypothesis space** = issue-weight rankings x per-issue evaluator shapes x reservation (tau) levels — Hindriks & Tykhonov, "Opponent modelling in automated multi-issue negotiation using Bayesian learning," AAMAS 2008, pp. 331-338. https://research.vu.nl/en/publications/opponent-modelling-in-automated-multi-issue-negotiation-using-bay Their **separate-learning scalability trick** (learn weight-hypotheses and evaluator-hypotheses independently, each conditioned on the mean of the others) is implemented as `mode="separate"` so the full product grid never has to be materialized when it is large.
- **Concession likelihood** `P(b_t | h, b_{t-1}) proportional to exp(-delta+ / 2 sigma^2)` where `delta+` is the *positive part* of the opponent's own-utility increase `h(b_t) - h(b_{t-1})` — Chang & Fujita, "A Scalable Opponent Model Using Bayesian Learning …," AAMAS 2023, pp. 2487-2489, Eqs. 2-5. https://www.southampton.ac.uk/~eg/AAMAS2023/pdfs/p2487.pdf Hypotheses under which the opponent's *own* utility went UP are penalized (a rational opponent concedes, i.e. weakly lowers its own demanded utility).
- **Frequency model** cheap fallback (issue-stability weights + option-frequency values) — HardHeaded / Baarslag, Hendrikx, Hindriks & Jonker, "Learning about the opponent …," JAAMAS 30(5):849-898, 2016, which repeatedly finds simple frequency heuristics often beat Bayesian models.
- **Soft / damped updates** (per-observation likelihood tempered by `lam < 1` + a uniform floor) so a deceptive / adversarial trace cannot catastrophically corrupt the posterior — Hua et al., "Game-Theo-

retic LLM …," arXiv:2411.05990, Remark 1 (exact-rationality belief updates are corrupted by deception).

Multilateral handling: one independent model per opponent (the standard independence assumption).

Attributes

Name	Type	Summary
`REPLAY_CACHE_ENTRIES`		

Classes

Name	Summary
`BeliefOracle`	Maintains a per-opponent `BeliefState` for one agent and exposes the posteriors as the `beliefs` payload of an `OracleVerdict` (this oracle annotates beliefs; it does not itself value moves, so `action_values` is left empty and `best` None).
`BeliefState`	A single opponent's belief model: a damped Bayesian posterior over an enumerated `OpponentType` grid, with an optional Hindriks-Tykhonov *separate-learning* factorization for large grids and a frequency-model shadow kept in parallel as the cheap control readout.
`FrequencyModel`	Count-based opponent model: issue *stability* -> weights (an issue whose chosen option stays fixed

Name	Summary
	across consecutive offers is important), option *frequency* -> values (a frequently-offered option is preferred).
`OpponentType`	One enumerated hypothesis about an opponent: normalized issue `weights` (sum 1), a per-issue evaluator `shapes` tuple, and a reservation `threshold` on the induced `[0, 1]` utility scale.

Functions

Name	Summary
`build_type_grid`	Enumerate the opponent-type hypothesis grid = weight-profiles x shape-assignments x tau-levels.
`clear_replay_cache`	Drop every cached offer-prefix posterior.
`replay_belief`	A :class:`BeliefState` that has observed `offers` in order — reusing the longest cached prefix.

BeliefOracle

Maintains a per-opponent `BeliefState` for one agent and exposes the posteriors as the `beliefs` payload of an `OracleVerdict` (this oracle annotates beliefs; it does not itself value moves, so `action_values` is left empty and `best` None).

```
BeliefOracle(
    agent: int,
    *,
    sigma: float = 0.25,
    lam: float = 1.0,
    floor: float = 0.001,
    mode: str = 'auto',
    anchor_first: bool = True,
    seed: int = 0,
    types=None,
)
```

Defined in `interlens.arena.negotiation.beliefs` · source

Inherits from: Oracle

Consumed by the acceptance / best-response oracles and the `BayesianRationalPolicy` for their induced `(utility, threshold)` distributions.

Parameters mirror `BeliefState` (`sigma`/`lam`/`floor`/`mode`/`anchor_first`); `agent` is the seat whose opponents are modeled.

Parameters

Name	Type	Default	Description
`agent`	`int`	*required*	
`sigma`	`float`	`0.25`	
`lam`	`float`	`1.0`	
`floor`	`float`	`0.001`	
`mode`	`str`	`'auto'`	
`anchor_first`	`bool`	`True`	
`seed`	`int`	`0`	
`types`		`None`	

Attributes

Name	Type	Summary
`agent`		
`name`		
`states`	dict[int, BeliefState]	

Methods

`evaluate`

```
evaluate(self, game, history, agent, legal)
```

source

Annotate the current turn with per-opponent posteriors.

Reads opponent offers from `history` (turns with a `Propose` action) if present; otherwise assumes `update_from_offers` was already called. Returns a verdict whose `beliefs` is a JSON-safe per-opponent summary (MAP type + tau posterior + entropy), with the rich `(OpponentType, prob)` induced distributions in `extra`.

Parameters

Name	Type	Default	Description
`game`		*required*	
`history`		*required*	
`agent`		*required*	
`legal`		*required*	

update_from_offers

```
update_from_offers(self, offers_by_opponent: dict, option_counts)
```

source

Feed each opponent's ordered list of proposed deals into its belief state (idempotent rebuild: resets and replays, so it is safe to call each turn with the full history).

The replay goes through :func:`replay_belief`, which reuses the longest cached offer prefix, so "rebuild from scratch every turn" costs one folded observation per turn rather than the whole history. The result is identical either way — that function returns a private copy of a pure fold.

Parameters

Name	Type	Default	Description
`offers_by_opponent`	`dict`	*required*	
`option_counts`		*required*	

BeliefState

A single opponent's belief model: a damped Bayesian posterior over an enumerated `OpponentType` grid, with an optional Hindriks-Tykhonov *separate-learning* factorization for large grids and a frequency-model shadow kept in parallel as the cheap control readout.

```
BeliefState(
    option_counts,
    types=None,
    *,
    sigma: float = 0.25,
    lam: float = 1.0,
    floor: float = 0.001,
    mode: str = 'auto',
    anchor_first: bool = True,
    joint_cap: int = 20000,
    seed: int = 0,
    tau_levels: tuple = (0.35, 0.55, 0.75),
    max_rankings: int = 24,
)
```

Defined in `interlens.arena.negotiation.beliefs` · source

Parameters

option_counts : sequence[int] Per-issue option counts. types : listOpponentType | None Explicit grid (else built via `build_type_grid`). sigma : float Concession-likelihood scale `sigma` (Chang-Fujita); larger = softer discrimination. lam : float Damping `lambda in (0, 1]` applied to each observation's log-likelihood (`1` = plain Bayes); `< 1` tempers updates so a deceptive trace cannot collapse the posterior. floor : float Uniform mass mixed into the posterior after every update (robustness; keeps every type reachable). mode : str `"joint"` (full product posterior) or `"separate"` (factor into weight / shape / tau marginals, each scored against the mean of the others — the AAMAS-2008 scalability trick). `"auto"` picks `separate` when the grid exceeds `joint_cap`. anchor_first : bool If True, the first observed offer is scored by how far it sits below each type's *ideal* utility (a rational opener bids near its max), which sharpens weight identification early. joint_cap : int Grid-size threshold for `mode="auto"`.

Parameters

Name	Type	Default	Description
option_counts		*required*	
types		None	
sigma	float	0.25	
lam	float	1.0	
floor	float	0.001	
mode	str	'auto'	
anchor_first	bool	True	
joint_cap	int	20000	
seed	int	0	
tau_levels	tuple	(0.35, 0.55, 0.75)	
max_rankings	int	24	

Attributes

Name	Type	Summary
anchor_first		
floor		
lam		
mode		
option_counts		
sigma		
types		

Methods

accept_prob

```
accept_prob(self, deal: Deal) -> float
```

source

Posterior probability the opponent accepts `deal` (mass of types whose utility >= their tau).

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	

accept_prob_matrix

```
accept_prob_matrix(self, deals_arr: np.ndarray) -> np.ndarray
```

source

Vectorized `accept_prob` for a whole batch of deals at once: `deals_arr` is the `(D, J)` int option-index array (`GameTables.deals_arr`); returns a `(D,)` acceptance-probability vector.

When the deal batch is the full enumerated space (the common case — the arena passes all deals), this reuses the precomputed cached `accept_matrix` and collapses to a single `posterior @ accept_matrix` matmul (the expensive (T x D) utility tensor is built ONCE per game, not per opponent per turn). Otherwise it builds the tensor on the fly for the given deal subset.

Parameters

Name	Type	Default	Description
`deals_arr`	`np.ndarray`	*required*	

copy

```
copy(self) -> 'BeliefState'
```

source

An independent belief with the same posterior, sharing the immutable type grid.

Only the fold state is duplicated (log-posterior, posterior, last observed offer, frequency counts); the grid arrays and the cached type-by-deal matrices are read-only and shared, so a copy costs two `|types|` vectors rather than a grid rebuild. This is what :func:`replay_belief` hands callers, so a cached posterior can never be mutated by whoever received it.

expected_normalized_surplus_matrix

```
expected_normalized_surplus_matrix(self, deals_arr: np.ndarray) ->
    np.ndarray
```

source

Posterior-expected **normalized surplus** for a whole batch of deals: `deals_arr` is the `(D, J)` int option-index array (`GameTables.deals_arr`); returns a `(D,)` vector in `[0, 1]`.

Per candidate type, the normalized surplus of a deal is `clip(u_type(d) - tau_type, 0)` divided by that type's best attainable surplus, so every type contributes on a common `[0, 1]` scale regardless of the point scale it implies; the readout is then the posterior mixture of those. This is the opponent-side input the fairness objective needs (`fairness.expected_objective`) and is the exact analogue of :meth:`accept_prob_matrix`, which mixes the 0/1 *indicator* of the same surplus being positive — this mixes its magnitude.

Uses the cached type-by-deal matrix when the batch is the full enumerated space (one gemv), and builds the tensor on the fly otherwise. Both paths normalize by the type's ideal surplus over the WHOLE deal space (`ideal - tau`, exact for an additive type) rather than over whatever subset was passed,

so a partial batch returns the same numbers as the corresponding slice of a full one.

Parameters

Name	Type	Default	Description
`deals_arr`	`np.ndarray`	*required*	

`expected_utility`

```
expected_utility(self, deal: Deal) -> float
```

source

Posterior-mean opponent utility of `deal`.

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	

`frequency_utility`

```
frequency_utility(self, deal: Deal) -> float
```

source

The cheap frequency-model readout of the opponent's utility of `deal` (control comparison).

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	

induced_distribution

```
induced_distribution(self)
```

source

The induced distribution over `(utility_fn, threshold)`: a list of `(OpponentType, prob)`.

map_type

```
map_type(self) -> OpponentType
```

source

Maximum-a-posteriori opponent type.

observe

```
observe(self, offer: Deal) -> 'BeliefState'
```

source

Update the posterior from one observed opponent offer (a proposed `Deal`).

Parameters

Name	Type	Default	Description
`offer`	`Deal`	*required*	

observe_response

```
observe_response(
    self,
    deal: Deal,
    accepted: bool,
    *,
```

```
    reliability: float = 0.75,
    strength: float = 0.35,
) -> 'BeliefState'
```

source

Soft-update from an opponent's public accept/reject response to `deal`.

A type predicts acceptance iff the deal clears its reservation threshold. `reliability` is the probability that the observed response follows that myopic prediction; values below 1 explicitly allow strategic rejection, mistakes, and cheap-talk inconsistency. `strength` tempers this evidence relative to a self-authored proposal, because a vote is informative about the threshold but much less diagnostic of the opponent's complete preference ordering.

The update is deliberately soft and implementable: it consumes only a public formal response and the referenced public deal, never the opponent's hidden score sheet.

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	
`accepted`	`bool`	*required*	
`reliability`	`float`	`0.75`	
`strength`	`float`	`0.35`	

`posterior`

```
posterior(self) -> np.ndarray
```

source

Posterior probability vector aligned with `self.types`.

Under `mode="separate"` this is the product-of-marginals reconstruction (weights, shapes, tau treated independent).

threshold_distribution

```
threshold_distribution(self)
```

source

Posterior distribution over the opponent's reservation `tau`: `{tau: prob}`.

type_thresholds

```
type_thresholds(self) -> np.ndarray
```

source

The `(|types|,)` reservation `tau` of every grid type, aligned with `self.types` (read-only on the shared default grid).

The vector form of the per-type `threshold` attribute, exposed so a diagnostic can compare a whole grid against a known reservation without rebuilding it from the dataclasses.

type_utility_matrix

```
type_utility_matrix(self, deals_arr: np.ndarray) -> np.ndarray
```

source

The `(|types|, D)` per-type utility tensor for a batch of deals: entry `[t, d]` is type `t`'s `[0, 1]`-scale utility of deal `d`.

The raw quantity :meth:`accept_prob_matrix` thresholds and :meth:`expected_normalized_surplus_matrix` rescales, exposed because a *diagnostic* needs the utilities themselves — posterior-expected opponent utility is `posterior() @ type_utility_matrix(...)`, one gemv. Returns the cached read-only tensor when the batch is the full enumerated space (the common case) and builds it on the fly otherwise, so the two paths agree numerically.

Parameters

Name	Type	Default	Description
`deals_arr`	`np.ndarray`	*required*	

FrequencyModel

Count-based opponent model: issue *stability* -> weights (an issue whose chosen option stays fixed across consecutive offers is important), option *frequency* -> values (a frequently-offered option is preferred).

```
FrequencyModel(option_counts)
```

Defined in `interlens.arena.negotiation.beliefs` · source

Induces a utility function; `update(offer)` is O(J). Often competitive with Bayes.

Parameters

Name	Type	Default	Description
`option_counts`		*required*	

Attributes

Name	Type	Summary
`J`		
`option_counts`		

Methods

copy

```
copy(self) -> 'FrequencyModel'
```

source

An independent model with the same counts — the fold state a cached replay hands out.

update

```
update(self, offer: Deal) -> 'FrequencyModel'
```

source

Fold one observed opponent offer into the counts.

Parameters

Name	Type	Default	Description
offer	Deal	*required*	

utility

```
utility(self, deal: Deal) -> float
```

source

Induced utility of `deal` in `[0, 1]` (per-issue value normalized to its max).

Parameters

Name	Type	Default	Description
deal	Deal	*required*	

weights

```
weights(self) -> np.ndarray
```

source

Normalized issue weights.

OpponentType

One enumerated hypothesis about an opponent: normalized issue `weights` (sum 1), a per-issue evaluator `shapes` tuple, and a reservation `threshold` on the induced `[0, 1]` utility scale.

```
OpponentType(weights: tuple, shapes: tuple, threshold: float,
    option_counts: tuple)
```

Defined in `interlens.arena.negotiation.beliefs` · source

`utility(deal) = sum_j weights[j] * evaluator(shapes[j])[deal[j]]` lies in `[0, 1]`; `accepts(deal)` iff that utility is at least `threshold`.

Parameters

Name	Type	Default	Description
`weights`	`tuple`	*required*	
`shapes`	`tuple`	*required*	
`threshold`	`float`	*required*	
`option_counts`	`tuple`	*required*	

Attributes

Name	Type	Summary
`option_counts`	`tuple`	
`shapes`	`tuple`	
`threshold`	`float`	
`weights`	`tuple`	

Methods

accepts

```
accepts(self, deal: Deal) -> bool
```

source

Whether a myopically-rational opponent of this type would accept `deal`.

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	

evaluator_table

```
evaluator_table(self) -> list
```

source

Per-issue option-value arrays (cached-free; cheap).

utility

```
utility(self, deal: Deal) -> float
```

source

Additive utility of `deal` in `[0, 1]`.

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	

build_type_grid

Enumerate the opponent-type hypothesis grid = weight-profiles x shape-assignments x tau-levels.

```
build_type_grid(
    option_counts,
    tau_levels=(0.35, 0.55, 0.75),
    max_rankings: int = 24,
    seed: int = 0,
) -> list
```

Defined in `interlens.arena.negotiation.beliefs` · source

Parameters

option_counts : sequence[int] Per-issue option counts `(O_1, ..., O_J)`. tau_levels : sequence[float] Candidate reservation thresholds on the `[0, 1]` utility scale. max_rankings : int Cap on the number of weight-ranking hypotheses (all `J!` if fewer, else a deterministic sample). seed : int Seed for the ranking sample (determinism).

Parameters

Name	Type	Default	Description
`option_counts`		*required*	
`tau_levels`		`(0.35, 0.55, 0.75)`	
`max_rankings`	`int`	`24`	
`seed`	`int`	`0`	

clear_replay_cache

Drop every cached offer-prefix posterior.

```
clear_replay_cache() -> None
```

Defined in `interlens.arena.negotiation.beliefs` · source

Only needed to reclaim memory or to measure cold timings — correctness never depends on the cache, since a hit and a miss produce the same state.

replay_belief

A :class:`BeliefState` that has observed `offers` in order — reusing the longest cached prefix.

```
replay_belief(option_counts, offers, **kwargs={}) -> BeliefState
```

Defined in `interlens.arena.negotiation.beliefs` · source

`observe` is a pure left fold: the state after `[d1 … dk]` depends on nothing but that sequence, so replaying a prefix and then folding in the remaining offers gives *exactly* the state a from-scratch replay would (same operations, same order, same float64). A negotiation turn extends its opponent's offer list by at most one entry, so the cached prefix is almost always all but the last offer, and each turn folds ONE observation instead of the whole history — which is what stops a seat's per-turn belief cost growing with the round number.

Returns a private :meth:`BeliefState.copy` every time, so the caller may mutate or further `observe` on the result without touching the cache. Grid-identity `kwargs` (`sigma`/`lam`/`floor`/`mode`/ `anchor_first`/`seed`/`tau_levels`/`max_rankings`) are part of the cache key; passing an explicit `types=` grid bypasses the cache entirely, since a caller-supplied grid has no stable identity.

Parameters

Name	Type	Default	Description
`option_counts`		*required*	
`offers`		*required*	
`kwargs`		`{}`	

bestresponse

Module `interlens.arena.negotiation.bestresponse`

Exact expectimax best-response oracle over (remaining rounds x deal space x type posterior).

Yields the headline **per-turn surplus-loss** metric `V(oracle action) - V(agent action)` in surplus units (the centipawn-loss analog: Regan & Haworth, "Intrinsic Chess Ratings," AAAI 2011; McIlroy-Young et al., KDD 2020) and **revealed-strategy exploitability** against fixed counterparts (Johanson, Waugh, Bowling & Zinkevich, "Accelerating Best Response Calculation in Large Extensive Games," IJCAI 2011, pp. 258-265).

WHY exact posterior-averaging rather than MCTS/determinization: at our scale (|D| ~ 720, T ~ 24, |types| ~ 10^3) the backward induction is ~10⁸⁻⁹6 elementary ops, so no sampling is needed; and averaging the value over the *type posterior* (rather than solving determinized full-information games and averaging outcomes) avoids strategy-fusion / non-locality bias — Frank & Basin, "Search in games with incomplete information," AIJ 100(1-2):87-123, 1998; ISMCTS: Cowling, Powley & Whitehouse, IEEE TCIAIG 4(2):120-143, 2012.

Protocol modeled: each round a (rotating) proposer offers a deal; all other seats accept/reject; the deal closes iff the game's fixed acceptance quorum (and every veto seat) accepts; otherwise play continues to the next round with the discount `delta`; after the deadline, no-deal pays surplus 0. Two regimes share one backward induction:

- **Full information** (`value_to_go_full_info`): all sheets known; opponents accept iff the offer beats their own discounted continuation, and each proposer offers its value-maximizing all-accepted deal. Exact; this is the path the unit tests pin.
- **Belief-averaged** (`value_to_go_beliefs`): opponents' acceptance is the posterior mass of accepting types (`accept_prob_fn`) and opponent proposals are modeled from the posterior; the agent's own acceptance uses its (known) surplus vs its continuation. A documented approximation for LLM-divergence use.

Complexity: building the per-deal all-accept masks is `O(T * n * |D|)` (vectorized); the belief path adds the `O(n * |types| * |D|)` acceptance-probability tensor once. Sub-second at the target scale.

Classes

Name	Summary
`BestResponseOracle`	Per-turn expectimax best response for one seat.

Functions

Name	Summary
`conditional_vote_values`	Value this seat's yes/no vote after conditioning on votes already cast.
`conditional_vote_values_batch`	:func:`conditional_vote_values` for `K` offers at once; returns four `(K,)` arrays.
`exploitability`	`BR_value - revealed_value`: how much surplus the agent leaves on the table vs its exact best response, holding the (full-information) counterpart continuation fixed.
`passage_probability`	Probability each deal passes under independent responder votes.
`value_to_go_beliefs`	Agent-`agent` continuation `Vi[t]` (shape `(T+2,)`) under the posterior.
`value_to_go_full_info`	Joint continuation values `V[t]` (shape `(T+2, n)`) for *every* seat under subgame-perfect alternating-offers play with the fixed `min_accept` quo-

Name	Summary
	rum, required `veto_seats`, and no-deal surplus 0.
`value_to_go_full_info_cached`	:func:`value_to_go_full_info` memoized on the game tables — the whole `v` curve, computed ONCE.

BestResponseOracle

Per-turn expectimax best response for one seat.

```
BestResponseOracle(
    agent: int,
    *,
    discount: float | None = None,
    accept_prob=None,
    opp_proposal=None,
    min_accept: int | None = None,
    veto_seats=(),
)
```

Defined in `interlens.arena.negotiation.bestresponse` · source

Inherits from: Oracle

Parameters

agent : int The deciding seat. discount : float | None Per-round discount `delta` OVERRIDE; default `None` = read the game's own `discount` / `breakdown_risk` via `effective_discount` (single source of truth). Pass a float only to force it. accept_prob : np.ndarray | None Optional `(D, n)` posterior acceptance-probability table (belief regime). None => full information. opp_proposal : dict[int, int] | None Optional stationary modeled opponent proposals (belief regime). If None in the belief regime, each opponent is modeled as proposing its own posterior-expected-utility-maximizing deal (via `accept_prob` as a utility proxy is avoided; falls back to full-info proposal if sheets known).

Parameters

Name	Type	Default	Description
`agent`	`int`	*required*	
`discount`	`float | None`	`None`	
`accept_prob`		`None`	
`opp_proposal`		`None`	
`min_accept`	`int | None`	`None`	

Name	Type	Default	Description
`veto_seats`		`()`	

Attributes

Name	Type	Summary
`accept_prob`		
`agent`		
`discount`		
`min_accept`		
`name`		
`opp_proposal`		
`veto_seats`		

Methods

`evaluate`

```
evaluate(self, game, history, agent, legal)
```

source

Value each legal action; `best` is the surplus-maximizing one.

`extra` carries the per-turn `surplus_loss` of every action (`V(best) - V(action)`) and the best-response proposal deal.

Ties in the argmax are resolved by :meth:`_argmax_action`, which prefers any action over accepting an offer that pays this seat strictly negative surplus, and only then falls back to the canonical `action_key` order. A strict optimum is never overridden.

Parameters

Name	Type	Default	Description
`game`		*required*	
`history`		*required*	
`agent`		*required*	
`legal`		*required*	

propose_values

```python
propose_values(
    self,
    tables: GameTables,
    cont: np.ndarray,
    agent: int | None = None,
    *,
    min_accept: int | None = None,
    veto_seats=None,
    objective=None,
) -> np.ndarray
```

source

Expected value to `agent` of proposing each deal now, given the continuation vector `cont` (full-info: `cont` is the length-n discounted continuation; belief: pass agent scalar via a length-n vector with opponents' acceptance folded into `accept_prob`).

`agent` is the PROPOSING seat; `None` falls back to the constructor's `self.agent`. Pass it explicitly whenever one oracle instance serves several seats — which is what a scenario's shared oracle stack does (`BestResponseOracle(0)` reused for every seat), and what :meth:`evaluate` now does with the seat it resolved. Getting this wrong is silent: the acceptance mask and the surplus column both come from this seat, so a stale seat prices every proposal from the wrong sheet while accept/reject/walk values (computed from the resolved seat) stay correct.

`objective` is an optional `(|D|,)` payoff column replacing `agent`'s own surplus as *what the proposal is worth if it passes* — the substitution that makes this best-response machinery serve a fairness-seeking proposer

(`fairness.mnw_objective`). Only the payoff changes: which deals can pass is still governed by the other seats' own-surplus acceptance, since the opponents remain self-interested however this seat scores the outcome. `None` (default) is the exact prior behaviour.

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	
`cont`	`np.ndarray`	*required*	
`agent`	`int | None`	`None`	
`min_accept`	`int | None`	`None`	
`veto_seats`		`None`	
`objective`		`None`	

conditional_vote_values

Value this seat's yes/no vote after conditioning on votes already cast.

```
conditional_vote_values(
    accept_prob: np.ndarray,
    proposer: int | None,
    agent: int,
    deal_index: int,
    deal_surplus: float,
    continuation: float,
    *,
    min_accept: int | None = None,
    veto_seats=(),
    forced_yes=(),
    forced_no=(),
) -> tuple[float, float, float, float]
```

Defined in `interlens.arena.negotiation.bestresponse` · source

Returns `(yes_value, no_value, p_pass_if_yes, p_pass_if_no)`. This is load-bearing for quorum games: a yes vote may be insufficient to pass, while a no vote may be non-pivotal. Existing supporters/rejecters and walkers are forced through `forced_yes`/`forced_no`; all uncast votes retain their deterministic full-information or posterior probability. Agreement pays `deal_surplus` and failure pays `continuation`.

`proposer=None` values a vote on a facilitator-tabled offer, which no seat implicitly supports (see :func:`passage_probability`).

One offer is the `K = 1` case of :func:`conditional_vote_values_batch`, which is where the arithmetic actually lives — a turn with many live offers should call that instead of this in a loop.

Parameters

Name	Type	Default	Description
`accept_prob`	`np.ndarray`	*required*	
`proposer`	`int | None`	*required*	
`agent`	`int`	*required*	

Name	Type	Default	Description
`deal_index`	`int`	*required*	
`deal_surplus`	`float`	*required*	
`continuation`	`float`	*required*	
`min_accept`	`int | None`	`None`	
`veto_seats`		`()`	
`forced_yes`		`()`	
`forced_no`		`()`	

conditional_vote_values_batch

:func:`conditional_vote_values` for `K` offers at once; returns four `(K,)` arrays.

```
conditional_vote_values_batch(
    accept_prob: np.ndarray,
    proposers,
    agent: int,
    deal_indices,
    deal_surpluses,
    continuation: float,
    *,
    min_accept: int | None = None,
    veto_seats=(),
    forced_yes=None,
    forced_no=None,
) -> tuple
```

Defined in `interlens.arena.negotiation.bestresponse` · source

Same semantics per offer — each gets its own proposer, deal, already-cast `forced_yes` / `forced_no` votes, and surplus — but the Poisson-binomial solves are BATCHED. The DP in :func:`passage_probability` is elementwise across its deal axis, so stacking K independent single-offer rows into one call returns each row's own answer, bitwise identical to solving it alone. Offers are grouped by proposer (the one argument the DP takes per call rather than per row), so a turn costs at most two solves per distinct proposer instead of two per offer — the difference between O(live offers) and O(seats) numpy calls on a turn, and the reason a long-horizon episode, whose live-offer list grows every round, stops being quadratic.

Parameters

accept_prob : np.ndarray `(D, n)` acceptance probabilities (full-information 0/1 or posterior). proposers : sequence[int | None] Per-offer proposing seat, or `None` for a facilitator-tabled offer with no implicit supporter. agent : int The seat whose yes/no vote is being valued (the same seat for every offer). deal_indices, deal_surpluses : sequence Per-offer row into `accept_prob` and the payoff agreement pays `agent`. continuation : float What failing to pass is worth to `agent` — shared by every offer, since it is a property of the turn.

forced_yes, forced_no : sequence[sequence[int]] | None Per-offer already-cast votes; `None` means none for any offer.

Parameters

Name	Type	Default	Description
`accept_prob`	`np.ndarray`	*required*	
`proposers`		*required*	
`agent`	`int`	*required*	
`deal_indices`		*required*	
`deal_surpluses`		*required*	
`continuation`	`float`	*required*	
`min_accept`	`int | None`	`None`	
`veto_seats`		`()`	
`forced_yes`		`None`	
`forced_no`		`None`	

exploitability

`BR_value - revealed_value`: how much surplus the agent leaves on the table vs its exact best response, holding the (full-information) counterpart continuation fixed.

```
exploitability(
    tables: GameTables,
    agent: int,
    revealed_value: float,
    proposer_seq,
    T: int,
    discount: float = 0.95,
) -> float
```

Defined in `interlens.arena.negotiation.bestresponse` · source

`revealed_value` is the agent's realized/expected surplus under its own policy (computed by the caller, e.g. via a rollout). Non-negative up to estimation noise; larger = more exploitable.

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	
`agent`	`int`	*required*	
`revealed_value`	`float`	*required*	
`proposer_seq`		*required*	
`T`	`int`	*required*	
`discount`	`float`	`0.95`	

passage_probability

Probability each deal passes under independent responder votes.

```
passage_probability(
    accept_prob: np.ndarray,
    proposer: int | None,
    *,
    min_accept: int | None = None,
    veto_seats=(),
) -> np.ndarray
```

Defined in `interlens.arena.negotiation.bestresponse` · source

`accept_prob` has shape `(D, n)`. The proposer implicitly supports its own offer, `min_accept` is a fixed count out of the original `n` seats, and every veto seat must support. `None` preserves the historical unanimity rule. A small Poisson-binomial DP computes `P(# yes ⩾ quorum)` exactly, rather than multiplying every opponent probability (which silently turns every quorum into unanimity).

`proposer=None` means the offer has NO implicit supporter — a package tabled by the protocol's neutral facilitator (:data:`~interlens.arena.actions.FACILITATOR`), which holds no sheet and casts no vote. Every seat then votes on its own merits: the quorum must be met out of the responders alone, and each veto seat's probability enters the product rather than being satisfied for free by proposing.

Parameters

Name	Type	Default	Description
`accept_prob`	`np.ndarray`	*required*	
`proposer`	`int | None`	*required*	
`min_accept`	`int | None`	`None`	
`veto_seats`		`()`	

value_to_go_beliefs

Agent-`agent` continuation `Vi[t]` (shape `(T+2,)`) under the posterior.

```
value_to_go_beliefs(
    tables: GameTables,
    agent: int,
    proposer_seq,
    T: int,
    discount: float,
    accept_prob,
    opp_proposal,
    *,
    min_accept: int | None = None,
    veto_seats=(),
    objective=None,
) -> np.ndarray
```

Defined in `interlens.arena.negotiation.bestresponse` · source

Parameters

accept_prob : np.ndarray `(D, n)` posterior probability each seat accepts each deal (self-column is treated deterministically below, so it is ignored for `agent`). opp_proposal : dict[int, int] Stationary modeled proposal (deal index) per opponent seat. objective : np.ndarray | None Optional `(|D|,)` payoff column replacing `agent`'s own surplus, so the same rollout produces the continuation value of a **fairness-seeking** seat (`fairness.mnw_objective`) rather than a self-interested one. Only this seat's payoff changes; the modeled opponents keep behaving as self-interested acceptors/proposers. `None` (default) is the exact prior behaviour.

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	
`agent`	`int`	*required*	
`proposer_seq`		*required*	

Name	Type	Default	Description
`T`	`int`	*required*	
`discount`	`float`	*required*	
`accept_prob`		*required*	
`opp_proposal`		*required*	
`min_accept`	`int | None`	`None`	
`veto_seats`		`()`	
`objective`		`None`	

value_to_go_full_info

Joint continuation values `V[t]` (shape `(T+2, n)`) for *every* seat under subgame-perfect alternating-offers play with the fixed `min_accept` quorum, required `veto_seats`, and no-deal surplus 0.

```
value_to_go_full_info(
    tables: GameTables,
    proposer_seq,
    T: int,
    discount: float = 0.95,
    *,
    min_accept: int | None = None,
    veto_seats=(),
) -> np.ndarray
```

Defined in `interlens.arena.negotiation.bestresponse` · source

`min_accept=None` preserves unanimity.

`V[t, i]` = seat `i`'s expected surplus-to-go at the start of round `t` (t = 1..T; `V[T+1] = 0`). At round `t` the proposer `p = proposer_seq[(t-1) % len]` offers the all-accepted deal maximizing its own surplus if that beats delaying, else delays; responders accept iff their surplus >= their discounted continuation.

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	
`proposer_seq`		*required*	
`T`	`int`	*required*	
`discount`	`float`	`0.95`	
`min_accept`	`int | None`	`None`	
`veto_seats`		`()`	

value_to_go_full_info_cached

:func:`value_to_go_full_info` memoized on the game tables — the whole `V` curve, computed ONCE.

```
value_to_go_full_info_cached(
    tables: GameTables,
    proposer_seq,
    T: int,
    discount: float = 0.95,
    *,
    min_accept: int | None = None,
    veto_seats=(),
) -> np.ndarray
```

Defined in `interlens.arena.negotiation.bestresponse` · source

The joint value curve depends only on `(tables, proposer_seq, T, discount, min_accept, veto_seats)`, every one of which is fixed for an episode; a turn varies only which ROW of `V` it reads. Recomputing the full backward induction on every turn therefore costs `O(T)` Poisson-binomial solves per turn and `O(T^2)` per episode, which is what made a long-horizon omniscient seat expensive the moment it started planning the real deadline. The cache lives on the `GameTables` instance (one per game, already cached on the game itself), so it is per-game rather than global and needs no eviction policy; the stored array is marked read-only so a caller cannot mutate a shared curve. Falls back to a plain recompute if the tables object refuses the attribute.

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	
`proposer_seq`		*required*	
`T`	`int`	*required*	
`discount`	`float`	`0.95`	
`min_accept`	`int | None`	`None`	
`veto_seats`		`()`	

calibrated

Module `interlens.arena.negotiation.calibrated`

Behaviourally-calibrated rational negotiation: :class:`CalibratedRationalPolicy`.

The composed Bayesian agent (:class:`~interlens.arena.negotiation.strategies.BayesianRationalPolicy`) is optimal against the opponent model it is given, and the opponent model it is given is **wrong for LLMs**. Under full information it assumes an opponent accepts a deal iff that deal clears the opponent's reservation — a step function at surplus 0. Measured LLM seats do not behave that way: they accept below their threshold 20-94% of the time, and they refuse plenty of offers that clear it.

This module keeps *every* other part of the agent — the optimal-stopping reservation, the expectimax best-response, the individual-rationality floor on its own acceptances, the walk-if-hopeless rule — and swaps that one step function for an **empirically fitted** `P(accept | z, rounds_left)` curve per opponent model. The whole intervention is the override of :meth:`~interlens.arena.negotiation.strategi` nothing else is touched, which is what makes the Bayes-vs-calibrated contrast a clean single-factor comparison rather than two different agents.

Why that is the interesting knob: the Bayesian agent's proposals are *generous* precisely because it models opponents as refusers. Tell it the truth — that these opponents cave — and it should best-respond by demanding more. The preregistered question (H2-strong) is whether it then extracts more than the Bayesian agent does, and whether the table gets **less fair** as the extraction gets more cynical.

FULL INFORMATION ONLY (deliberate). Computing `z` needs the opponent's own utility for a deal, which only exists under `--info full`. Under private info this class falls straight through to the inherited belief-oracle path and is therefore *identical* to :class:`BayesianRationalPolicy` — it does not silently substitute a half-calibrated model. Calling code that wants the calibrated behaviour must run full-info tables; the policy records which path it took on `last_path` so a run can be audited rather than assumed.

Worked example

```
from interlens.arena.negotiation.calibrated import AcceptanceCurveSet,
    CalibratedRationalPolicy
curves = AcceptanceCurveSet.from_fit_artifact("self_benefit/
    acceptance_curves.json")
policy = CalibratedRationalPolicy(curves=curves, opponent_model="Qwen/
    Qwen3-8B")
# ... seat it exactly as BayesianRationalPolicy is seated
    (table.policy_seat / mixed_table)
```

A step-function curve reproduces the Bayesian agent **exactly** — that equivalence is the regression test in `tests/test_negotiation_calibrated.py` and it is the reason the curve object can express a step at all:

```
step = AcceptanceCurveSet.step()          # P(accept) = 1 iff surplus ⩾ 0
CalibratedRationalPolicy(curves=step, opponent_model="anything")   # ==
    BayesianRationalPolicy
```

Attributes

Name	Type	Summary
`Z_SPACES`		

Classes

Name	Summary
`AcceptanceCurve`	One opponent model's fitted `P(accept | z, rounds_left)`.
`AcceptanceCurveSet`	A fitted curve per opponent model, plus the `z` convention they were all fit in.

Name	Summary
`CalibratedRationalPolicy`	:class:`BayesianRationalPolicy` with its opponent acceptance model replaced by fitted behavioural curves.

AcceptanceCurve

One opponent model's fitted `P(accept | z, rounds_left)`.

```
AcceptanceCurve(
    form: str,
    params: dict = dict(),
    rounds_feature: str = 'rounds_left',
    p_min: float = 0.0,
    p_max: float = 1.0,
    n_events: int = 0,
    n_accepts: int = 0,
    heldout_ece: float | None = None,
    low_power: bool = False,
    notes: str = '',
)
```

Defined in `interlens.arena.negotiation.calibrated` · source

Parameters

form : str Functional form. `"step"` = the Bayesian agent's own model (1.0 iff `z ⩾ 0`), kept so the calibrated policy can be driven back to exact Bayes equivalence for the regression test. `"logistic"` = `sigmoid(a + b*z)`. `"logistic_rounds"` = `sigmoid(a + b*z + c*r)` where `r` is the round position (see `rounds_feature`). `"bins"` = a piecewise-constant lookup over `z` edges, optionally one row per `rounds_left` bin — the assumption-free form, at the cost of needing enough events per cell. params : dict Form-specific coefficients. logistic: `a`, `b` (and `c` with rounds). bins: `z_edges` (K-1 ascending interior edges), `p` (K probabilities, or a list of K-wide rows when `r_edges` is given) and optional `r_edges`. rounds_feature : str What `r` means in the logistic forms. `"rounds_left"` (the default) = the raw integer count of rounds remaining AFTER the current one, 0 on the forced final — this is the fitting lane's convention and the default is deliberately the same as theirs, because a curve fitted on raw counts and evaluated on a fraction is wrong by a silent rescaling that nothing would raise on. `"frac_elapsed"` = `1 - rounds_left/total` in `[0, 1]`, for a fit that chose scale-free pressure. Ignored by `step`/`logistic`. p_min, p_max : float Clamp on the returned probability. The default floor/ceiling of 0/1 leaves the curve untouched; a small positive floor is worth setting when the fit is extrapolating

far below the observed `z` range, since a hard 0 tells the expectimax that a deal is *impossible* rather than merely unlikely and can make the agent walk on a technicality. n_events, n_accepts : int Sample size behind the fit. Carried so a ladder result can never be quoted without the reader being able to see it was fit on 30 events. heldout_ece : float | None Held-out expected calibration error, if the fitting lane computed one. low_power : bool The fitting lane's own flag that this curve is not quotable — too few events, too few game clusters, or no variation in the outcome. Carried through so a consumer can REFUSE it rather than discover the problem in a results table. The fit that motivated this (`Qwen3-8B:thinking-on`) has 60 events across 3 games and zero below-threshold votes. notes : str Free text from the fitting lane (run dirs, exclusions, caveats).

Parameters

Name	Type	Default	Description
`form`	`str`	*required*	
`params`	`dict`	`dict()`	
`rounds_feature`	`str`	`'rounds_left'`	
`p_min`	`float`	`0.0`	
`p_max`	`float`	`1.0`	
`n_events`	`int`	`0`	
`n_accepts`	`int`	`0`	
`heldout_ece`	`float | None`	`None`	
`low_power`	`bool`	`False`	
`notes`	`str`	`''`	

Attributes

Name	Type	Summary
`form`	`str`	
`heldout_ece`	`float | None`	

Name	Type	Summary
`is_step`	`bool`	Whether this curve IS the Bayesian agent's own step model (so a calibrated policy carrying only step curves must behave identically to :class:`BayesianRationalPolicy`).
`low_power`	`bool`	
`n_accepts`	`int`	
`n_events`	`int`	
`notes`	`str`	
`p_max`	`float`	
`p_min`	`float`	
`params`	`dict`	
`rounds_feature`	`str`	

Methods

`from_dict`

```
from_dict(cls, d: dict) -> 'AcceptanceCurve'
```

source

Build from one model's JSON block, ignoring keys the schema does not define (the fitting lane is free to carry extra diagnostics alongside the coefficients).

Parameters

Name	Type	Default	Description
d	dict	*required*	

prob

```
prob(self, z: np.ndarray, rounds_left: int = 1, total_rounds: int = 0) -
    > np.ndarray
```

source

Vectorized `P(accept)` over an array of `z` values, clamped to `[p_min, p_max]`.

`rounds_left` is the number of rounds still playable INCLUDING the current one; `total_rounds` is the episode's deadline in the same units, needed only by the `frac_elapsed` rounds feature.

Parameters

Name	Type	Default	Description
z	np.ndarray	*required*	
rounds_left	int	1	
total_rounds	int	0	

AcceptanceCurveSet

A fitted curve per opponent model, plus the `z` convention they were all fit in.

```
AcceptanceCurveSet(
    z_space: str,
    curves: dict[str, AcceptanceCurve],
    default: AcceptanceCurve | None = None,
    provenance: dict = dict(),
)
```

Defined in `interlens.arena.negotiation.calibrated` · source

Parameters

z_space : str Which of :data:`Z_SPACES` the curves are functions of. Set by the fitting lane; the policy computes `z` the same way or the numbers mean nothing. curves : dict[str, AcceptanceCurve] Model id (as it appears in the lineup, e.g. `"Qwen/Qwen3-8B"`) to its fitted curve. default : AcceptanceCurve | None Curve for a model with no entry of its own. `None` (the default) makes an unknown model an ERROR rather than a silent fallback — seating a calibrated agent against a model nobody fit a curve for is a design mistake, not a runtime condition to paper over. provenance : dict Whatever the fitting lane recorded about how these were produced (run dirs, episode counts, date).

Parameters

Name	Type	Default	Description
`z_space`	`str`	*required*	
`curves`	dict[str, AcceptanceCurve]	*required*	
`default`	AcceptanceCurve | None	`None`	
`provenance`	`dict`	`dict()`	

Attributes

Name	Type	Summary
`all_step`	`bool`	Whether every curve here (including the default) is a step function — i.e. this set drives :class:`CalibratedRationalPoli` back to exact Bayes behaviour.
`curves`	dict[str, AcceptanceCurve]	
`default`	AcceptanceCurve | None	
`provenance`	`dict`	
`z_space`	`str`	

Methods

`for_model`

```
for_model(self, model_id: str) -> AcceptanceCurve
```

source

The curve for `model_id`, or `default`.

Raises :class:`KeyError` when neither exists, naming the models that ARE fit — a calibrated cell run against an uncalibrated opponent is uninterpretable, so this fails at seat-construction time rather than quietly reverting to the step model.

Parameters

Name	Type	Default	Description
model_id	str	*required*	

from_fit_artifact

```
from_fit_artifact(cls, path: str | Path, *, spec: str = 'base') ->
    'AcceptanceCurveSet'
```

source

Load the acceptance-curve fitting lane's NATIVE artifact (`schema == "rational_agents/acceptance_curves/v1"`), as written by `experiments/rational_agents/self_benefit/fit_acceptance.py`.

That artifact is organised around the fit rather than around the consumer: each responder carries several candidate `specs` plus validation blocks, coefficients are a flat `[intercept, *slopes]` list, and the regressors are named in `base_features`. This adapter reads it as-is rather than asking the fitting lane to reshape its output — the artifact of record should stay the fitter's own.

Parameters

path : str | Path The fitted JSON. spec : str Which candidate fit to consume. `"base"` (`logit p = b0 + b_z z + b_r rounds_left`) is the preregistered one and the default; `"interaction"` adds `z * rounds_left`. Selecting a non-preregistered spec is a deliberate act and is recorded in `provenance["spec"]`.

The fit's `z` is `(u_i(d) - tau_i) / (max_d' u_i(d') - tau_i)` — this module's `surplus_norm` — and its `rounds_left` is the count AFTER the current round. Both are asserted against the artifact's own `base_features` so a change of regressors upstream fails loudly here instead of being evaluated under the old meaning.

Parameters

Name	Type	Default	Description
path	str | Path	*required*	
spec	str	'base'	

from_json

```
from_json(cls, path: str | Path) -> 'AcceptanceCurveSet'
```

source

Load the fitting lane's artifact.

Expected shape (extra keys are ignored, so the fitting lane can carry diagnostics):

```
{"schema_version": 1,
 "z_space": "surplus_norm",
 "models": {"Qwen/Qwen3-8B": {"form": "logistic_rounds", "params":
     {"a": .., "b": .., "c": ..},
                              "n_events": 1234, "n_accepts": 456,
     "heldout_ece": 0.03}},
 "default": {...},                       # optional
 "provenance": {"run_dirs": [...], ...}} # optional
```

Parameters

Name	Type	Default	Description
path	str | Path	*required*	

step

```
step(cls, z_space: str = 'surplus') -> 'AcceptanceCurveSet'
```

source

The degenerate set every model of which is the Bayesian agent's own step model.

Exists for the equivalence regression test and as a sanity control cell (a "calibrated" run that must reproduce the Bayes replication exactly).

Parameters

Name	Type	Default	Description
z_space	str	'surplus'	

vote_grain_from_fit_artifact

```
vote_grain_from_fit_artifact(cls, path: str | Path) ->
    'AcceptanceCurveSet'
```

source

Build the **vote-conditional** curve set from the fit artifact's `empirical` block.

WHY A SECOND GRAIN EXISTS. The fitted logistic is a **panel-grain** quantity: "next time this seat moves, does it accept this offer" — which folds in crowding and inattention, because a seat facing five standing offers can formally take at most one. Measured panel acceptance is 0.15-0.45. But the protocol ends in a **forced vote**, and among offers actually voted on, acceptance is ~0.99 (Qwen3-8B 0.992 over n=7,420; gemma 0.991 over n=8,971) — even *below* the voter's own reservation it is 0.81 and 0.96 respectively. Feeding the panel curve into the endgame therefore tells the agent that closure is nearly impossible exactly where it is nearly certain, which is a large and one-directional error.

No new fit is needed: the three vote-conditional rates the artifact already reports (`accept_rate_below_ir_when_voted` for `z < 0`, `accept_rate_high_z_when_voted` for `z ⩾ 0.5`, and `accept_rate_when_voted` over all votes) determine the middle bin exactly, since the overall rate is their count-weighted mixture. The result is a three-bin step in `z` per responder — assumption-free and reported rather than extrapolated.

Use with :class:`CalibratedRationalPolicy`'s `vote_curves` argument, which applies it only in the endgame; during regular rounds the panel curve is the right object.

Parameters

Name	Type	Default	Description
`path`	`str | Path`	*required*	

z_of

```
z_of(self, utility: np.ndarray, thresholds: np.ndarray, seat: int) ->
     np.ndarray
```

source

`z` for every deal from `seat`'s point of view, under this set's declared convention.

Parameters

Name	Type	Default	Description
`utility`	`np.ndarray`	*required*	
`thresholds`	`np.ndarray`	*required*	
`seat`	`int`	*required*	

CalibratedRationalPolicy

:class:`BayesianRationalPolicy` with its opponent acceptance model replaced by fitted behavioural curves.

```
CalibratedRationalPolicy(
    *,
    curves: AcceptanceCurveSet,
    opponent_model: str,
    seat_models: dict[int, str] | None = None,
    vote_curves: AcceptanceCurveSet | None = None,
    endgame_rounds: int = 0,
    discount: float | None = None,
    walk_if_hopeless: bool = True,
    name: str = 'calibrated-rational',
)
```

Defined in `interlens.arena.negotiation.calibrated` · source

Inherits from: BayesianRationalPolicy

Optimal stopping, expectimax best-response, the IR floor on its own acceptances and the walk-if-hopeless rule are inherited unchanged.

Parameters

curves : AcceptanceCurveSet The fitted curves and their `z` convention. opponent_model : str Model id filling the OTHER seats, used to look up the curve. The lineup knows this (the run's `--models`); the policy cannot infer it from the state, which is why it is required. seat_models : dict[int, str] | None Per-seat override when the table is heterogeneous — seat index to model id. Seats absent here use `opponent_model`. Supply this for any mixed-model lineup; with a homogeneous lineup leave it `None`. discount, walk_if_hopeless, name As :class:`BayesianRationalPolicy`.

Attributes

last_path : str | None `"calibrated"` or `"belief"` — which acceptance model the most recent turn actually used. Set every turn so a run can be AUDITED for

whether the calibrated path was live, instead of the cell's label being taken on trust. A full-info run whose policy reports `"belief"` is mislabelled.

Parameters

Name	Type	Default	Description
`curves`	AcceptanceCurveSet	*required*	
`opponent_model`	`str`	*required*	
`seat_models`	`dict[int, str] | None`	`None`	
`vote_curves`	AcceptanceCurveSet | None	`None`	
`endgame_rounds`	`int`	`0`	
`discount`	`float | None`	`None`	
`walk_if_hopeless`	`bool`	`True`	
`name`	`str`	`'calibrated-rational'`	

Attributes

Name	Type	Summary
`curves`		
`endgame_rounds`		
`last_path`	`str | None`	
`opponent_model`		
`seat_models`		
`vote_curves`		

Methods

`model_for_seat`

```
model_for_seat(self, seat: int) -> str
```

source

Model id seated at `seat` — the per-seat override if given, else `opponent_model`.

Parameters

Name	Type	Default	Description
`seat`	`int`	*required*	

equilibrium

Module `interlens.arena.negotiation.equilibrium`

Banks-Duggan stationary-equilibrium oracle for the multilateral unanimity bargaining game.

Model (random-proposer, closed rule, unanimity): Baron & Ferejohn, “Bargaining in Legislatures,” APSR 83(4):1181-1206, 1989; generalized to arbitrary alternative spaces / utilities by Banks & Duggan, “A Bargaining Model of Collective Choice,” APSR 94(1):73-88, 2000, and QJPS 1(1):49-85, 2006. No-delay stationary characterization with continuation values `v` and disagreement flows `tau`:

```
A_i(v) = { d in D : u_i(d) ≥ (1 - delta) * tau_i + delta * v_i }     #
     i's acceptance set
A(v)   = intersection_i A_i(v)                                       #
     unanimity social acceptance set
x_j*(v) = argmax_{d in A(v)} u_j(d)                                  #
     proposer j best-in-set (else delay)
v_i     = sum_j p_j * u_i(x_j*(v))   (+ delay branch)                #
     fixed point
```

Solved by damped fixed-point iteration `v^{k+1} = (1 - lambda) v^k + lambda * T(v^k)` over the enumerated `D` (each sweep `O(n * |D|)`); a softmax-over-ties proposer rule (`tie_temperature > 0`) is available as the fallback when the discrete argmax correspondence makes the hard iteration cycle.

Sanity anchor (built in, `divide_the_dollar` + `okada_closed_form`): Okada, “A Noncooperative Coalitional Bargaining Game with Random Proposers,” GEB 16(1):97-108, 1996 — the unanimity closed form is proposer keeps `1 - delta (n-1)/n`, each responder gets `delta/n`, and `v_i = 1/n`.

Caveats (docstring, not asserted): Eraslan, “Uniqueness of Stationary Equilibrium Payoffs in the Baron- Ferejohn Model,” JET 103(1):11-30, 2002, gives uniqueness for `q < n` rules; general *unanimity* games can have delay / multiplicity / non-existence of a pure no-delay equilibrium (Britz, Herings & Predtetchinski). Existence here is in mixed proposal strategies with pure stage-undominated voting on the finite (compact) D.

Classes

Name	Summary
`EquilibriumOracle`	Mounts the stationary equilibrium as a per-turn reference: what the standing offer *should* look like for whichever seat currently proposes, plus the proposer-power decomposition `v*`.
`EquilibriumSolution`	Result of the fixed-point solve.

Functions

Name	Summary
`divide_the_dollar`	A discrete divide-the-dollar TU game as `GameTables`: deals = integer allocations of `steps` units among `n` players (compositions), `u_i = share_i = units_i / steps`, `tau = 0`.
`okada_closed_form`	The Okada 1996 unanimity closed form: `{"proposer_keeps": 1 - delta(n-1)/n, "responder_gets": delta/n, "value": 1/n}`.
`solve_equilibrium`	Damped fixed-point solve for the Banks-Duggan stationary continuation values.

EquilibriumOracle

Mounts the stationary equilibrium as a per-turn reference: what the standing offer *should* look like for whichever seat currently proposes, plus the proposer-power decomposition `v*`.

```
EquilibriumOracle(
    agent: int | None = None,
    *,
    discount: float | None = None,
    damping: float = 0.5,
    max_iter: int = 1000,
    tol: float = 1e-09,
    tie_temperature: float = 0.0,
    proposer_probs=None,
)
```

Defined in `interlens.arena.negotiation.equilibrium` · source

Inherits from: Oracle

Parameters mirror `solve_equilibrium` (`discount`/`damping`/`max_iter`/`tol`/`tie_temperature`/`proposer_probs`). `discount` defaults to `None` = read the game's own `discount` / `breakdown_risk` (single source of truth). `agent` (optional) is the seat whose actions are valued.

Parameters

Name	Type	Default	Description
`agent`	`int | None`	`None`	
`discount`	`float | None`	`None`	
`damping`	`float`	`0.5`	
`max_iter`	`int`	`1000`	
`tol`	`float`	`1e-09`	
`tie_temperature`	`float`	`0.0`	
`proposer_probs`		`None`	

Attributes

Name	Type	Summary
`agent`		
`damping`		
`discount`		
`max_iter`		
`name`		
`proposer_probs`		
`tie_temperature`		
`tol`		

Methods

`evaluate`

```
evaluate(self, game, history, agent, legal)
```

source

Value the current proposer's legal `Propose` actions against the equilibrium best-in-set: the equilibrium proposal is `best`; each `Propose(deal)` is valued by the proposer's utility of that deal (0 outside the social acceptance set is not imposed — the value is the raw utility, and the flag `outside_social_set` marks proposals `A(v*)` would not sustain).

`extra` carries `v*` and the per-proposer equilibrium deals.

Parameters

Name	Type	Default	Description
`game`		*required*	
`history`		*required*	

Name	Type	Default	Description
`agent`		*required*	
`legal`		*required*	

solve

```
solve(self, game) -> EquilibriumSolution
```

source

Solve the stationary equilibrium for `game` (cached on the game object).

The discount is read from the game (`effective_discount`) unless an explicit override was passed to the constructor.

Parameters

Name	Type	Default	Description
`game`		*required*	

EquilibriumSolution

Result of the fixed-point solve.

```
EquilibriumSolution(
    values: np.ndarray,
    proposals: dict,
    residual: float,
    converged: bool,
    social_set_size: int,
)
```

Defined in `interlens.arena.negotiation.equilibrium` · source

Attributes

values : np.ndarray Stationary continuation values `v*` (shape `(n,)`). proposals : dict[int, int] Per-proposer equilibrium deal index `x_j*` (`-1` if the proposer delays / no social set). residual : float Final fixed-point residual `max|T(v) - v|`. converged : bool Whether `residual < tol` within `max_iter`. social_set_size : int `|A(v*)|` at the solution.

Parameters

Name	Type	Default	Description
`values`	`np.ndarray`	*required*	
`proposals`	`dict`	*required*	
`residual`	`float`	*required*	
`converged`	`bool`	*required*	
`social_set_size`	`int`	*required*	

Attributes

Name	Type	Summary
`converged`	`bool`	

Name	Type	Summary
`proposals`	`dict`	
`residual`	`float`	
`social_set_size`	`int`	
`values`	`np.ndarray`	

divide_the_dollar

A discrete divide-the-dollar TU game as `GameTables`: deals = integer allocations of `steps` units among `n` players (compositions), `u_i = share_i = units_i / steps`, `tau = 0`.

```
divide_the_dollar(n: int, steps: int) -> GameTables
```

Defined in `interlens.arena.negotiation.equilibrium` · source

Used as the Okada unanimity sanity anchor for the equilibrium solver.

Parameters

Name	Type	Default	Description
`n`	`int`	*required*	
`steps`	`int`	*required*	

okada_closed_form

The Okada 1996 unanimity closed form: `{"proposer_keeps": 1 - delta(n-1)/n, "responder_gets": delta/n, "value": 1/n}`.

```
okada_closed_form(n: int, delta: float) -> dict
```

Defined in `interlens.arena.negotiation.equilibrium` · source

Parameters

Name	Type	Default	Description
`n`	`int`	*required*	
`delta`	`float`	*required*	

solve_equilibrium

Damped fixed-point solve for the Banks-Duggan stationary continuation values.

```
solve_equilibrium(
    tables: GameTables,
    *,
    discount: float = 0.95,
    thresholds=None,
    proposer_probs=None,
    damping: float = 0.5,
    max_iter: int = 1000,
    tol: float = 1e-09,
    tie_temperature: float = 0.0,
    v_init=None,
) -> EquilibriumSolution
```

Defined in `interlens.arena.negotiation.equilibrium` · source

Parameters

tables : GameTables Utility tables for the game. discount : float Common discount `delta` in `(0, 1]`. thresholds : sequence[float] | None Disagreement flows `tau_i` (default: `tables.thresholds`). proposer_probs : sequence[float] | None Recognition probabilities `p_j` (default uniform `1/n`). damping : float Relaxation `lambda` in `(0, 1]` for `v ← (1-lambda) v + lambda T(v)`. max_iter, tol : int, float Iteration budget and convergence tolerance on the residual. tie_temperature : float If `> 0`, the proposer's outcome is a softmax-over-`A(v)` average of utilities (smooths cycles) rather than a hard best-in-set. v_init : sequence[float] | None Optional initial `v` (default column means of utility).

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	
`discount`	`float`	`0.95`	
`thresholds`		`None`	

Name	Type	Default	Description
`proposer_probs`		`None`	
`damping`	`float`	`0.5`	
`max_iter`	`int`	`1000`	
`tol`	`float`	`1e-09`	
`tie_temperature`	`float`	`0.0`	
`v_init`		`None`	

fairness

Module `interlens.arena.negotiation.fairness`

The **table objective**: one number per deal saying how good that deal is *for the whole table*.

This is the drop-in replacement for the per-seat surplus column `tables.surplus[:, seat]` that every self-interested oracle in this package maximizes. Substituting it turns the existing best-response / optimal-stopping stack into a fairness-seeking negotiator without touching a line of the recursion: the proposal argmax, the reservation curve and the vote comparison all keep their shape and only change units.

The objective is :func:`mnw_objective` — a scalar flattening of Maximum Nash Welfare's two-stage rule [caragiannis2019] over the **normalized** surplus vector:

```
z_i(d) = clip(u_i(d) - tau_i, 0) / b_i          # b_i = party i's best
     surplus over the IR set
k(d)   = #{i : u_i(d) - tau_i > 0}              # how many parties the
     deal actually satisfies
obj(d) = k(d)/n  +  geomean_{i in S(d)} z_i(d) / n
```

The normalization by `b_i` is what makes this comparable across parties holding arbitrary private point scales, and it is the same coordinate as the analysis layer's `normalized_nash_welfare` (whose `game.scale` is likewise the per-party max surplus over the IR set), so a policy's objective and the metric it is judged on are the same quantity rather than two things that can drift apart.

Why the flattening, and what it buys. On the region that matters — deals where every party clears its threshold — `k(d) = n` and `obj(d) = 1 + NNW(d)/n`, so the ordering is *exactly* the normalized-Nash-welfare ordering and `argmax obj` is the discrete NBS point. Off that region raw NNW is identically zero and gives a maximizer nothing to steer by, whereas the flattening keeps ranking deals by how many parties they satisfy and then by the product over those parties. The coalition term dominates by construction (`geomean z ⩽ 1` so the whole second term is `⩽ 1/n`, the width of one step in `k/n`), which is precisely MNW's lexicographic "satisfy

the largest coalition first" rule. Hence :func:`max_objective_index` agrees with `solutions.nash_bargaining_index` whenever the strict-IR set is non-empty and with `solutions.max_nash_welfare_index` when it is not — pinned by property tests.

Units. `obj` lies in `[0, 1 + 1/n]`, and no-deal scores `0` — the same convention as own-surplus (where no-deal is also 0), which is what lets the optimal-stopping recursion in `acceptance.py` be reused verbatim: its `v_0 = 0` base case still means "the value of never agreeing".

Private information. :func:`expected_objective` computes the same quantity when the opponents' sheets are unknown, substituting each opponent's *posterior-expected* normalized surplus (from :meth:`~interlens.arena.negotiation.beliefs.BeliefState.ex` for the exact one. This is a plug-in / mean-field estimate `obj(E[z])` rather than the true `E[obj(z)]`; the two differ by a Jensen gap and the plug-in is the optimistic side, because the geometric mean is concave. That bias is the point of the `fairness-rational` vs `fairness-oracle` contrast rather than a defect to be hidden.

Functions

Name	Summary
`expected_objective`	The table objective under a posterior over the other seats' hidden sheets — what the **fairness algorithmic** agent maximizes.
`max_objective_index`	The objective-maximizing deal row, ties broken by lowest index so a policy built on this is deterministic — the same canonical tie-break `solutions._argmax_ties` and `np.argmax` use.

Name	Summary
mnw_objective	The full-information table objective of shape (|D|,) — what the **fairness oracle** maximizes.
normalized_surplus_matrix	Per-deal per-party normalized surplus z of shape (|D|, n): clip(surplus, 0) / b_i.
objective_from_normalized	The table objective of shape (|D|,) from a normalized-surplus matrix z of shape (|D|, n).

expected_objective

The table objective under a posterior over the other seats' hidden sheets — what the **fairness algorithmic** agent maximizes.

```
expected_objective(
    tables: GameTables,
    seat: int,
    expected_z: dict[int, np.ndarray],
) -> np.ndarray
```

Defined in `interlens.arena.negotiation.fairness` · source

The acting seat's own column is exact (it holds its own sheet, so its normalized surplus is known); every other seat's column is the posterior-expected normalized surplus supplied in `expected_z`. Any seat missing from `expected_z` and not the acting seat is treated as fully satisfied (`z = 1`), which is the only neutral choice: scoring it 0 would make every deal look like it satisfies nobody and would collapse the objective to a constant.

Parameters

tables : GameTables Tables whose `surplus` column for `seat` is populated. Under private information the other columns are padding (the caller's `_tables` fills them with zeros) and are deliberately never read here. seat : int The acting seat, whose own normalized surplus is computed exactly from its own sheet. expected_z : dict[int, np.ndarray] `{opponent_seat: (\|D\|,) posterior-expected normalized surplus}`, e.g. from :meth:`~interlens.arena.negotiation.beliefs.BeliefState.expected_r`

Returns the `(|D|,)` expected objective. See the module docstring on the plug-in (Jensen) bias.

One further deviation from the full-information case, forced by the information condition: the acting seat's normalizer is its best surplus over ALL deals, not over the IR set, because identifying the IR set requires the thresholds it does not have. This cannot move the ranking within its own column (a positive constant), but it does shift how its own gains trade against the opponents' in the geometric mean.

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	
`seat`	`int`	*required*	
`expected_z`	`dict[int, np.ndarray]`	*required*	

max_objective_index

The objective-maximizing deal row, ties broken by lowest index so a policy built on this is deterministic — the same canonical tie-break `solutions._argmax_ties` and `np.argmax` use.

```
max_objective_index(objective: np.ndarray) -> int
```

Defined in `interlens.arena.negotiation.fairness` · source

Parameters

Name	Type	Default	Description
`objective`	`np.ndarray`	*required*	

mnw_objective

The full-information table objective of shape `(|D|,)` — what the **fairness oracle** maximizes.

```
mnw_objective(tables: GameTables) -> np.ndarray
```

Defined in `interlens.arena.negotiation.fairness` · source

Composition of :func:`normalized_surplus_matrix` and :func:`objective_from_normalized`. Its argmax is the discrete NBS deal when some deal clears every threshold, and the Maximum-Nash-Welfare deal otherwise.

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	

normalized_surplus_matrix

Per-deal per-party normalized surplus `z` of shape `(|D|, n)`: `clip(surplus, 0) / b_i`.

```
normalized_surplus_matrix(tables: GameTables) -> np.ndarray
```

Defined in `interlens.arena.negotiation.fairness` · source

`b_i` is party `i`'s largest surplus over the **individually rational** set (every party at or above threshold) — the scale-invariant unit the whole fairness objective is expressed in, and the same `game.scale` the analysis layer's `normalized_nash_welfare` divides by. Falls back to the max over all deals when no deal is individually rational for everyone, and to `1.0` for a party whose best surplus is non-positive (it can never be made better off, so dividing by its "ideal" is meaningless). Below-threshold surpluses clip to 0 rather than going negative: they are not partial progress, they are unacceptable.

Values can exceed `1` *outside* the IR set — `b_i` is the best party `i` can do in a deal everyone could sign, and a deal that tramples somebody else may pay `i` more than that. :func:`objective_from_normalized` caps at 1 for exactly this reason; see its docstring.

Parameters

tables : GameTables Precomputed dense game tables. Only `surplus` is read, so this works on any table whose surplus column is populated for the parties in question.

Parameters

Name	Type	Default	Description
`tables`	GameTables	*required*	

objective_from_normalized

The table objective of shape `(|D|,)` from a normalized-surplus matrix `z` of shape `(|D|, n)`.

```
objective_from_normalized(z: np.ndarray) -> np.ndarray
```

Defined in `interlens.arena.negotiation.fairness` · source

`obj = k/n + geomean_{i in S} z_i / n` with `S` the strictly-satisfied set and `k = |S|` (see the module docstring for why the two terms compose into MNW's lexicographic rule). A deal satisfying nobody scores exactly `0`, matching the no-deal payoff. The geometric mean is taken in log space because a five- or six-party product of small normalized gains underflows well before the mean does.

`z` is **capped at 1** first, and the cap is load-bearing rather than hygiene. The coalition term only dominates the welfare term if the latter cannot exceed `1/n`, and a party can beat its IR-set ideal on a deal that leaves somebody else below threshold — uncapped, such a deal can outscore one that satisfies everybody, which would break both the lexicographic rule and the identity with the Nash Bargaining Solution. The cap is inert on the IR set (where `z ⩽ 1` by construction), so it costs nothing where the objective is actually operating; it only refuses to reward paying one party out of another's threshold.

Split out from :func:`mnw_objective` so the private-information path can feed it *expected* normalized surpluses through :func:`expected_objective` without re-deriving the flattening rule.

Parameters

Name	Type	Default	Description
`z`	`np.ndarray`	*required*	

games

Module `interlens.arena.negotiation.games`

Swappable game presets: name a classic bargaining situation, get a ready-to-play game in one call.

A **preset** is a factory returning `(GameSpec, analysis, protocol_cfg)` — the game spec, its enumeration-verified :func:`~interlens.arena.negotiation.solutions.analyze` dict, and the scenario protocol knobs that turn the shared `ScorableNegotiation` state machine into that specific game (single-shot / fixed-proposer / majority). Presets are parameterizations, not new engines: the deal space, the oracle stack, and the annotation layer all run unchanged on any of them.

Presets shipped:

========== ========== ========== Preset Encoding Rational anchor ========== ========== ========== `scorable` (default) the full §2 generator (a thin alias – defaults in one place) the §4 oracle stack `ultimatum` N=2, J=1 pie split, tau=0, rounds=1, fixed proposer, SPE (Rubinstein 1982 §5 responder-only single-shot vote one-round limit); human rejection [guth1982] `divide_dollar` N parties, J=1 discrete-shares split, rotating proposer, Baron-Ferejohn 1989 / multi-round, unanimity or majority Okada 1996 (v = 1/n) `bilateral_multiissue` DoND-style N=2, J issues, private values (the generator) Lewis et al. 2017 lineage ========== ========== ==========

Usage:

```
from interlens.arena.negotiation import games
game, analysis, protocol_cfg = games.make_preset("ultimatum", pie=10,
     n_options=11)
game.n_parties                              # 2
analysis["deal_space_size"]                 # 11 (one split per option)
# the arena bridge: a solver-verified Instance + the scenario cfg to play
     it under
```

```
instance, protocol_cfg = games.build_preset_instance("ultimatum")
from interlens.arena.scenarios import ScorableNegotiation
ScorableNegotiation().make_state(instance, "moves_only", seed=0,
     cfg=protocol_cfg)   # single-shot ultimatum
```

Each preset's own docstring carries its citation keys; `references.py` maps every key to the full reference. The `scorable` alias re-uses the generator's defaults verbatim (never re-defaulted here).

Attributes

Name	Type	Summary
`PRESETS`	`dict[str, Preset]`	

`Preset`

Functions

Name	Summary
`bilateral_multiissue`	A **DoND-style bilateral, multi-issue, private-value** negotiation [lewis2017]: two parties bargain over `n_issues` item types, each with private per-item values – the classic "Deal or No Deal" item-division game, reproduced here as the scorable generator restricted to `n_parties=2`.
`build_preset_instance`	Build a solver-verified arena :class:`~interlens.arena.schema.Ins from a preset, plus the scenario `protocol_cfg` to play it under – the one-call bridge from a preset name to a runnable game.

Name	Summary
`divide_dollar`	The **divide-the-dollar / legislative-bargaining** testbed [baron_ferejohn1989]: `n_parties` split a unit pie over multiple rounds with a rotating proposer, under a unanimity or majority agreement rule.
`make_preset`	Look up a preset by `name` and build it, returning `(GameSpec, analysis, protocol_cfg)`.
`scorable`	The default multi-party, multi-issue **scorable** game – a thin alias to the existing generator so the whole preset machinery has one uniform entry point.
`ultimatum`	The **ultimatum game** [guth1982] as a scorable game: one proposer offers a split of a fixed `pie`, one responder accepts (both get the split) or rejects (both get 0), take-it-or-leave-it.

bilateral_multiissue

A **DoND-style bilateral, multi-issue, private-value** negotiation [lewis2017]: two parties bargain over `n_issues` item types, each with private per-item values – the classic “Deal or No Deal” item-division game, reproduced here as the scorable generator restricted to `n_parties=2`.

```
bilateral_multiissue(
    *,
    n_issues: int = 3,
    n_options: int = 5,
    info: str = 'private',
    seed: int = 0,
    **overrides={},
) -> tuple[GameSpec, dict, dict]
```

Defined in `interlens.arena.negotiation.games` · source

This is a thin call into :func:`~interlens.arena.negotiation.generate.generate_game` with `n_parties=2`, so it inherits every score-sheet repair (the Pareto-slack / feasible-size / sparsity-IoU knobs) and its verified analysis; no repair is re-implemented. Standard multi-round protocol (`protocol_cfg = {}`).

Parameters

n_issues : number of item-type issues `J` (Hua et al. / DoND use ~3). n_options : options per issue (the item-count levels). info : `"private"` (each party’s values are private – the DoND setting, where some inefficiency is rational by Myerson-Satterthwaite) or `"full"` (common knowledge, an efficiency upper bound). seed : generator seed (the game IS randomized here – different seeds give different value sheets). **overrides : any other :func:`generate_game` knob (`dominated_target`, `mix`, `feasible_fraction`, `rounds`, `discount`, …); defaults come from `generate_game` and are never re-set here.

Returns `(GameSpec, analysis, protocol_cfg)`.

Parameters

Name	Type	Default	Description
`n_issues`	`int`	`3`	
`n_options`	`int`	`5`	
`info`	`str`	`'private'`	
`seed`	`int`	`0`	
`overrides`		`{}`	

build_preset_instance

Build a solver-verified arena :class:`~interlens.arena.schema.Instance` from a preset, plus the scenario `protocol_cfg` to play it under – the one-call bridge from a preset name to a runnable game.

```
build_preset_instance(
    name: str,
    *,
    level: int = 0,
    seed: int = 0,
    instance_name: str = 'scorable_negotiation',
    **kwargs={},
) -> tuple[Instance, dict]
```

Defined in `interlens.arena.negotiation.games` · source

Reuses :func:`~interlens.arena.negotiation.generate.build_instance` for the `(GameSpec, analysis) → Instance` wrap (`payload = GameSpec.to_json()`, `solution` = the analysis dict, exact ceiling/floor), so a preset instance is indistinguishable from a generated one to the scenario, oracle, annotation, and atlas layers. `seed` is threaded to the preset factory (used by randomized presets like `bilateral_multiissue` / `scorable`, ignored by the deterministic `ultimatum` / `divide_dollar`) and stamped on the Instance; `level` is the Instance difficulty tag and (for `scorable`) the generator ladder level. `kwargs` are the preset's knobs.

Returns `(instance, protocol_cfg)` – pass `protocol_cfg` as the scenario `cfg` (or merged into it) so the shared `ScorableNegotiation` state machine runs the preset's protocol variant.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`level`	`int`	`0`	
`seed`	`int`	`0`	
`instance_name`	`str`	`'scorable_negotiation'`	
`kwargs`		`{}`	

divide_dollar

The **divide-the-dollar / legislative-bargaining** testbed [baron_ferejohn1989]: `n_parties` split a unit pie over multiple rounds with a rotating proposer, under a unanimity or majority agreement rule.

```
divide_dollar(
    *,
    n_parties: int = 3,
    steps: int = 12,
    rounds: int | None = None,
    rule: str = 'unanimity',
    discount: float = 0.99,
    seed: int = 0,
) -> tuple[GameSpec, dict, dict]
```

Defined in `interlens.arena.negotiation.games` · source

Encoding: J=1 issue ("Allocation") whose options are every integer allocation of `steps` units among the parties (the compositions summing to `steps`); party `i`'s utility for an allocation is its own share `units_i / steps in [0, 1]`, thresholds `tau=0`. The deal space is reused from the equilibrium oracle's Okada anchor (:func:`~interlens.arena.negotiation.equilibrium.divide_the_dollar`), so the same game the oracle solves is the one the scenario plays. Rotating proposer + multi-round is the scenario's default protocol (`protocol_cfg = {}`).

Rational anchor: the Banks-Duggan / Baron-Ferejohn stationary equilibrium. For the **unanimity** rule the Okada 1996 closed form [okada1996] gives the sanity value `v_i = 1/n` (proposer keeps `1 - delta(n-1)/n`, each responder `delta/n`) – exactly what :class:`~interlens.arena.negotiation.equilibrium.EquilibriumOracle` recovers on this preset. The **majority** rule (`min_accept = floor(n/2)+1`) is the minimal-winning-coalition Baron-Ferejohn game; the equilibrium oracle models unanimity social acceptance, so the `v = 1/n` anchor is stated only for `rule="unanimity"`.

Like `ultimatum` this is a pure division: every allocation is Pareto-optimal (`dominated_acceptable_fraction = 0`), so the score-sheet-repair knob does not apply.

Parameters

n_parties : number of parties splitting the pie (N in {2, 3} is the usual testbed; larger works but the deal space is `C(steps+n-1, n-1)` compositions – keep it enumerable). steps : allocation granularity (the pie is `steps` indivisible units). Finer `steps` = the equilibrium value tracks `1/n` more precisely but a larger deal space. rounds : round-robin rounds before the forced final (default `2 * n_parties`, so each seat proposes ~twice). rule : `"unanimity"` (all parties must clear their threshold – the Okada `v=1/n` anchor) or `"majority"` (`floor(n/2)+1` parties – the minimal-winning-coalition game). discount : per-round discount `delta` in `(0, 1]` stored on the game (the equilibrium/acceptance oracles' single source of truth); `< 1` makes interior concession rational rather than deadline brinkmanship. seed : accepted for a uniform preset interface; the game is deterministic, so it is unused.

Returns `(GameSpec, analysis, protocol_cfg)`.

Parameters

Name	Type	Default	Description
`n_parties`	`int`	`3`	
`steps`	`int`	`12`	
`rounds`	`int | None`	`None`	
`rule`	`str`	`'unanimity'`	
`discount`	`float`	`0.99`	
`seed`	`int`	`0`	

make_preset

Look up a preset by `name` and build it, returning `(GameSpec, analysis, protocol_cfg)`.

```
make_preset(name: str, **kwargs={}) -> tuple[GameSpec, dict, dict]
```

Defined in `interlens.arena.negotiation.games` · source

`kwargs` are the preset's own knobs (see each factory). Raises `KeyError` (listing the known presets) on an unknown name.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`kwargs`		`{}`	

scorable

The default multi-party, multi-issue **scorable** game – a thin alias to the existing generator so the whole preset machinery has one uniform entry point.

```
scorable(
    *,
    level: int = 0,
    seed: int = 0,
    ladder: list[dict] | None = None,
    **overrides={},
) -> tuple[GameSpec, dict, dict]
```

Defined in `interlens.arena.negotiation.games` · source

Delegates to :func:`~interlens.arena.negotiation.generate.game_at_level`, which maps the difficulty `level` through the shipped :data:`~interlens.arena.negotiation.generate.INSTANCE_LADDER` (feasible-set size, the dominated-acceptable repair, the per-round discount) to :func:`generate_game`. **Every default lives in generate.py** and is never re-set here; `**overrides` (e.g. `n_parties`, `n_issues`, `n_options`, `info`, `dominated_target`) pass straight through. Standard protocol (`protocol_cfg = {}`).

Parameters

level : difficulty ladder index (0 = easiest / largest acceptable set). seed : generator seed. ladder : optional custom difficulty ladder (default :data:`INSTANCE_LADDER`). **overrides : any :func:`generate_game` knob (overrides the ladder's).

Returns `(GameSpec, analysis, protocol_cfg)`.

Parameters

Name	Type	Default	Description
`level`	`int`	`0`	
`seed`	`int`	`0`	

Name	Type	Default	Description
`ladder`	`list[dict] | None`	`None`	
`overrides`		`{}`	

ultimatum

The **ultimatum game** [guth1982] as a scorable game: one proposer offers a split of a fixed `pie`, one responder accepts (both get the split) or rejects (both get 0), take-it-or-leave-it.

```
ultimatum(
    *,
    pie: float = 10.0,
    n_options: int = 11,
    seed: int = 0,
) -> tuple[GameSpec, dict, dict]
```

Defined in `interlens.arena.negotiation.games` · source

Encoding: N=2, J=1 issue ("Split") whose `n_options` discrete options are the shares – option `o` gives the proposer `o * pie/(n_options-1)` and the responder the rest – with thresholds `tau=0` (the BATNA is the no-deal payoff 0), `rounds=1`, a fixed proposer (seat 0), and the responder-only single-shot vote (`protocol_cfg = {"single_shot": True, "fixed_proposer": True}`).

Rational anchor – the subgame-perfect equilibrium (the T=1 limit of Rubinstein 1982 alternating offers [rubinstein1982] §5): the responder accepts **any positive share** (rejecting pays 0, so any surplus >= 0 is weakly better), so the proposer keeps the whole pie. Under the harness's `⩾ 0` individual-rationality convention (`solutions.ir_mask` default `strict=False`) the responder is *indifferent* at a 0 share and (weakly) accepts, so the discrete SPE has the proposer keeping the entire pie (option `n_options-1`); the classic "pie - epsilon" is the strict/continuous statement. The behavioral contrast [guth1982] – humans reject low positive offers – is the LLM-vs-rational gap this preset exposes.

Note: this is a **pure division** (every split has the same total `pie`), so EVERY split is Pareto-optimal and the acceptable set has `dominated_acceptable_fraction = 0` – the score-sheet-repair knob (`dominated_target`) does not apply here (there is no dominated slack to leave in play).

Parameters

pie : total value to split (proposer share + responder share = `pie` for every option). n_options : number of discrete splits (>= 2). `11` gives whole-unit shares `0..pie` for `pie=10`; a larger value = finer granularity (closer to the continuous game). seed : accepted for a uniform preset interface; the ultimatum game is deterministic, so it is unused.

Returns `(GameSpec, analysis, protocol_cfg)`.

Parameters

Name	Type	Default	Description
`pie`	`float`	`10.0`	
`n_options`	`int`	`11`	
`seed`	`int`	`0`	

generate

Module `interlens.arena.negotiation.generate`

Scorable-negotiation scenario generator with the score-sheet repairs the reproducibility studies demand.

The generator draws private additive score sheets from an explicit issue-type mix, calibrates thresholds to a target acceptable-set size, and – the central repair – rejection-samples over seeds to hit a target fraction of acceptable deals that are Pareto-*dominated*, so the game is not near-zero-sum the moment it is feasible [reproA2025]. Every generated game ships with a precomputed analysis dict (:func:`interlens.arena.negotiation.solutions.analyze`) whose descriptors are *verified by enumeration*.

Issue types (the `mix` / `issue_types` knobs), which shape the cross-party correlation of the value columns:

- `distributive` – opposed preferences (a fixed pie): parties split into camps with reversed option rankings and high weight, so acceptable deals get pushed onto the frontier (near-zero-sum).
- `compatible` – a shared option ranking: everyone prefers the same option, so the worse options are Pareto-dominated-for-all yet may stay acceptable under loose thresholds (a prime source of dominated slack).
- `integrative` – parties weight *different* issues (non-carers score the issue 0, creating sparsity), so conceding an issue you don't care about to someone who does is a free logroll; failing to do so lands you at a dominated interior deal.

So the mix is the geometric lever on the dominated fraction, while threshold calibration fixes the acceptable-set *size* independently.

Example:

```
game, analysis = generate_game(n_parties=5, n_issues=5, n_options=4,
```

```
                                feasible_fraction=0.12,
    dominated_target=0.6, seed=1)
analysis["dominated_acceptable_fraction"]   # ~0.6, verified by
    enumeration
analysis["ir_count"], game.n_parties
```

Attributes

Name	Type	Summary
INSTANCE_LADDER	list[dict]	
LADDER_DISCOUNT		
SCALE		

Classes

Name	Summary
IssueType	The three issue archetypes that set how parties' value columns correlate (see module docstring).

Functions

Name	Summary
build_instance	Wrap a generated (GameSpec, analysis) into a solver-verified arena :class:~interlens.arena.sc
game_at_level	The (GameSpec, analysis) at difficulty level from seed – the level -> generator-knob mapping shared by :func:generate_instance and the games.scorable preset, so the difficulty ladder and the per-round dis-

Name	Summary
	count default live in exactly one place (never re-defaulted at a call site).
`generate_game`	Generate one scorable-negotiation game plus its enumeration-verified analysis dict.
`generate_games`	Generate `count` independent games with consecutive base seeds `seed`, `seed+1`, … (all other knobs forwarded to :func:`generate_game`).
`generate_instance`	Generate one solver-verified arena :class:`~interlens.arena.sch at difficulty `level` from `seed` – the bridge a `ScorableNegotiation` scenario's `generate_instance` delegates to.

IssueType

The three issue archetypes that set how parties' value columns correlate (see module docstring).

```
IssueType()
```

Defined in `interlens.arena.negotiation.generate` · source

Inherits from: `str`, `Enum`

Attributes

Name	Type	Summary
`COMPATIBLE`		
`DISTRIBUTIVE`		
`INTEGRATIVE`		

build_instance

Wrap a generated `(GameSpec, analysis)` into a solver-verified arena :class:`~interlens.arena.schema.Instance`.

```
build_instance(
    game: GameSpec,
    analysis: dict,
    *,
    name: str,
    level: int,
    seed: int,
    id_prefix: str | None = None,
) -> Instance
```

Defined in `interlens.arena.negotiation.generate` · source

Primary-score convention (matching the shipped `negotiation` scenario): the episode's primary score is the sum of per-party normalized surplus captures divided by its maximum over feasible deals (no deal = 0). Thus the exact `ceiling` is `1.0` and the reference `floor` is the mean normalized primary over feasible deals – an average-feasible-deal policy. Both are invariant to independent positive affine transformations of the parties' score sheets. `payload` is `GameSpec.to_json()` (round-trips back via `GameSpec.from_json`); `solution` is the full :func:`~interlens.arena.negotiation.solutions.analyze` dict (descriptors + every solution point), so scorers and analyses read it straight from the stored Instance without re-solving. If the feasible set is empty (no-deal is the rational outcome) ceiling/floor are both `0.0`. `name` is the scenario name the calling scenario passes in, so `Instance.scenario` matches it.

Parameters

Name	Type	Default	Description
`game`	GameSpec	*required*	
`analysis`	`dict`	*required*	
`name`	`str`	*required*	
`level`	`int`	*required*	

Name	Type	Default	Description
`seed`	`int`	*required*	
`id_prefix`	`str | None`	`None`	

game_at_level

The `(GameSpec, analysis)` at difficulty `level` from `seed` – the level -> generator-knob mapping shared by :func:`generate_instance` and the `games.scorable` preset, so the difficulty ladder and the per-round discount default live in exactly one place (never re-defaulted at a call site).

```
game_at_level(
    level: int,
    seed: int,
    *,
    ladder: list[dict] | None = None,
    **overrides={},
) -> tuple[GameSpec, dict]
```

Defined in `interlens.arena.negotiation.generate` · source

Maps `level` through `ladder` (default :data:`INSTANCE_LADDER`, shrinking feasible set with level) to :func:`generate_game` knobs – the bank's per-round discount :data:`LADDER_DISCOUNT` (< 1, so interior concession is rational rather than deadline brinkmanship [sandholm_vulkan1999]) plus the level's feasible-fraction/dominated-target – then generates the game and its enumeration-verified analysis. `**overrides` win over the ladder knobs (e.g. `discount=1.0` for a brinkmanship-baseline arm, or `n_parties`/`n_issues`/`n_options`/`mix`/`info`). No `Instance` wrapping – see :func:`generate_instance` for that.

Parameters

Name	Type	Default	Description
`level`	`int`	*required*	
`seed`	`int`	*required*	
`ladder`	`list[dict] | None`	`None`	
`overrides`		`{}`	

generate_game

Generate one scorable-negotiation game plus its enumeration-verified analysis dict.

```
generate_game(
    n_parties: int = 6,
    n_issues: int = 5,
    n_options: int | Sequence[int] = 4,
    mix: tuple[float, float, float] = (0.4, 0.4, 0.2),
    issue_types: Sequence[str] | None = None,
    feasible_fraction: float = 0.1,
    feasible_tol: float = 0.6,
    dominated_target: float | None = 0.6,
    dominated_tol: float = 0.12,
    auto_mix: bool = True,
    decorrelate: bool = True,
    rounds: int = 4,
    info: str = 'full',
    chat: bool = True,
    proposer: int = 0,
    veto: int | None = 1,
    discount: float = 1.0,
    breakdown_risk: float = 0.0,
    max_tries: int = 600,
    seed: int = 0,
) -> tuple[GameSpec, dict]
```

Defined in `interlens.arena.negotiation.generate` · source

Parameters

n_parties : number of negotiating parties `n`. n_issues : number of issues `J`. n_options : options per issue – one int (same for all issues) or a per-issue sequence of length `n_issues`. Deal space size is `prod` of these; keep it enumerable (<= ~3125). mix : `(distributive, integrative, compatible)` fractions used to assign each issue a type (normalized internally). The geometric lever on the dominated-acceptable fraction: more compatible/integrative content -> more dominated-but-acceptable slack; more distributive content -> nearer zero-sum. issue_types : explicit per-issue type list (strings/`IssueType`); overrides `mix` when given. feasible_fraction : target size of the unanimity acceptable (IR) set as a fraction of `|D|` – the difficulty dial (smaller = harder to find a deal). Thresholds are calibrated to hit it. feasible_tol : relative

tolerance on the acceptable count; a candidate counts as size-OK when its count is within `feasible_tol` of the target (always at least +/-1 deal of slack). dominated_target : target fraction of acceptable deals that are Pareto-dominated (the central repair). Set `None` to skip the search and just return the first size-OK candidate. dominated_tol : accept-and-stop tolerance on the dominated fraction; otherwise the closest candidate over `max_tries` seeds is returned. auto_mix : when True and `dominated_target` is set and `issue_types` is not given, also sweep the distributive fraction of the issue mix (more distributive -> nearer zero-sum -> lower dominated fraction) so a low or high target is reachable that a single fixed mix could not hit by reseeding alone. `mix` is the sweep center; set `auto_mix=False` to hold `mix` fixed. decorrelate : when True, permute each issue's option order and use neutral option labels so no public role text can leak preferences (the mandatory control from Study A) [reproA2025]. rounds, info, chat, proposer, veto : protocol knobs stored on the returned `GameSpec` (see :class:`~interlens.arena.negotiation.sheets.GameSpec`). Acceptable-set analysis uses unanimity IR. discount : per-round discount `delta` in (0, 1] stored on the game (default 1.0 = no impatience). With only a hard deadline and `delta = 1`, the rational baseline is deadline brinkmanship, so a game meant to elicit interior concession should set `delta < 1` [sandholm_vulkan1999] (the instance ladder does). The equilibrium/acceptance oracles read this as their single source of truth. breakdown_risk : per-round exogenous-breakdown probability in [0, 1) stored on the game (default 0.0). An alternative to discounting for making interior concession rational. max_tries : rejection-sampling budget (distinct seeds tried) when `dominated_target` is set. seed : base RNG seed; try `t` uses `seed*100003 + t` so different base seeds give different games.

Returns

`(GameSpec, analysis)` where `analysis` is :func:`solutions.analyze`'s dict (all descriptors verified by enumeration). `GameSpec.meta` records the generator provenance and the achieved fractions, and `analysis["generator"]` mirrors the request/achievement so failures to hit a target are visible, never silent.

Parameters

Name	Type	Default	Description
n_parties	int	6	
n_issues	int	5	
n_options	int | Sequence[int]	4	
mix	tuple[float, float, float]	(0.4, 0.4, 0.2)	
issue_types	Sequence[str] | None	None	
feasible_fraction	float	0.1	
feasible_tol	float	0.6	
dominated_target	float | None	0.6	
dominated_tol	float	0.12	
auto_mix	bool	True	
decorrelate	bool	True	
rounds	int	4	
info	str	'full'	
chat	bool	True	
proposer	int	0	
veto	int | None	1	
discount	float	1.0	
breakdown_risk	float	0.0	
max_tries	int	600	
seed	int	0	

generate_games

Generate `count` independent games with consecutive base seeds `seed`, `seed+1`, … (all other knobs forwarded to :func:`generate_game`).

```
generate_games(count: int, seed: int = 0, **kwargs={}) ->
     list[tuple[GameSpec, dict]]
```

Defined in `interlens.arena.negotiation.generate` · source

Parameters

Name	Type	Default	Description
`count`	`int`	*required*	
`seed`	`int`	`0`	
`kwargs`		`{}`	

generate_instance

Generate one solver-verified arena :class:`~interlens.arena.schema.Instance` at difficulty `level` from `seed` – the bridge a `ScorableNegotiation` scenario's `generate_instance` delegates to.

```
generate_instance(
    level: int,
    seed: int,
    *,
    name: str = 'scorable_negotiation',
    ladder: list[dict] | None = None,
    **overrides={},
) -> Instance
```

Defined in `interlens.arena.negotiation.generate` · source

Maps `level` through `ladder` (default :data:`INSTANCE_LADDER`, shrinking feasible set with level) to :func:`generate_game` knobs via :func:`game_at_level`, generates the game plus its enumeration-verified analysis, and wraps them via :func:`build_instance`. The bank ships with per-round discount :data:`LADDER_DISCOUNT` (< 1, so interior concession is rational rather than deadline brinkmanship [sandholm_vulkan1999]); pass `discount=1.0` (or any other knob) via `**overrides` to change it – e.g. a brinkmanship-baseline ablation arm. Other overridable knobs: `n_parties`, `n_issues`, `n_options`, `mix`, `max_tries`, `breakdown_risk`. `name` sets `Instance.scenario` so a scenario passes `self.name`. The payload is deterministic per `(level, seed)` (the instance id is fresh each call, per the arena convention).

Parameters

Name	Type	Default	Description
`level`	`int`	*required*	
`seed`	`int`	*required*	
`name`	`str`	`'scorable_negotiation'`	
`ladder`	`list[dict] | None`	`None`	
`overrides`		`{}`	

llm_calibrated

Module `interlens.arena.negotiation.llm_calibrated`

Private-information rational negotiation against an **empirically fitted LLM opponent model**: :class:`LLMCalibratedRationalPolicy`.

Why this exists. :class:`~interlens.arena.negotiation.strategies.BayesianRationalPolicy` is optimal against opponents whose signal lives in their offer sequence and who accept iff a deal clears their reservation. Notes 0057/0045/0053 established that at a real LLM table both assumptions fail: the belief posterior is signal-starved (LLM offers barely move, so the concession likelihood extracts ~nothing), and measured LLM acceptance is nothing like a step at surplus 0. The result is the 0.20-0.23 closure collapse. This module keeps the composed agent's *decision machinery* — optimal stopping, expectimax proposals, the IR floor, the walk rule — and swaps the two RATIONALISTIC priors for quantities **measured from ~1,500 frozen LLM episodes**:

1. **Acceptance**: `P(opponent accepts deal d)` becomes the posterior mixture of a fitted acceptance curve evaluated on each hypothesis type's normalized surplus, instead of the posterior mass of types whose step-model would accept. The same Hindriks–Tykhonov posterior; only the within-type response model changes. (The full-information version of this swap is :class:`~interlens.arena.negotiation.calibrated.CalibratedRa` this class is its private-information counterpart and reuses its :class:`~interlens.arena.negotiation.calibrated.AcceptanceCurveSet` containers unchanged.)
2. **Future offers**: the optimal-stopping reservation `v_j = delta * E[max(X, v_{j-1})]` is computed over the EMPIRICAL distribution of what LLM opponents actually table at each round position (:class:`OfferCurveSet`), instead of the belief-induced "deals that could close, weighted by posterior passage" distribution. This is the change note 0045 points at: the Bayesian reservation never decays into a known-good standing offer because its imagined offer distribution stays optimistic; the measured incoming-z distri-

bution is far thinner, so waiting is worth less and the reservation admits real offers.

Everything is in the program's affine-invariant `z` space (`surplus_norm`: own surplus over own best attainable surplus), so one fitted model pools across games and sheets. The policy is deterministic given `(state, fitted model)`.

Worked example:

```
from interlens.arena.negotiation.llm_calibrated import LLMOpponentModel,
    LLMCalibratedRationalPolicy
model = LLMOpponentModel.from_json("opponent_model.json")   # written by
    the fitting lane
policy = LLMCalibratedRationalPolicy(model=model, opponent_model="claude-
    opus-5")
# seat it exactly as BayesianRationalPolicy is seated (PolicyParticipant /
    mixed_table)
```

Degenerate-control identities (pinned by `tests/test_negotiation_llm_calibrated.py`):

- step acceptance curves reproduce the parent's posterior acceptance table bit-for-bit (a step in per-type `z` is exactly "this type's utility clears its tau", which is what the parent's mixture thresholds);
- with `offers=None` the whole `act`/`vote` path is the parent's own (only the acceptance table is swapped), so step curves + no offer model = byte-identical actions to `BayesianRationalPolicy`.

Classes

Name	Summary
`LLMCalibratedRationalPolicy`	:class:`BayesianRationalPolicy` for PRIVATE-information tables with both rationalistic priors replaced by the fitted :class:`LLMOpponentModel` (module docstring).

Name	Summary
`LLMOpponentModel`	Everything the calibrated policy consumes, loaded from ONE fitted artifact.
`OfferCurve`	One opponent model's fitted distribution of the `z` an offer delivers to its RECIPIENT, as a function of round position.
`OfferCurveSet`	A fitted :class:`OfferCurve` per opponent model.

LLMCalibratedRationalPolicy

:class:`BayesianRationalPolicy` for PRIVATE-information tables with both rationalistic priors replaced by the fitted :class:`LLMOpponentModel` (module docstring).

```
LLMCalibratedRationalPolicy(
    *,
    model: LLMOpponentModel,
    opponent_model: str,
    seat_models: dict[int, str] | None = None,
    endgame_rounds: int = 1,
    discount: float | None = None,
    walk_if_hopeless: bool = True,
    name: str = 'llm-calibrated-rational',
)
```

Defined in `interlens.arena.negotiation.llm_calibrated` · source

Inherits from: BayesianRationalPolicy

Inherits the belief posterior, the IR floor on its own acceptances, the quorum-aware vote valuation and the walk rule unchanged.

Parameters

model : LLMOpponentModel The fitted opponent model. opponent_model : str Model id filling the other seats (the fitting key, e.g. `"claude-opus-5"`). Required because the policy cannot infer from the state who it is playing. seat_models : dict[int, str] | None Per-seat override for heterogeneous tables. A seat mapped to a key starting with `"policy:"` is a computable seat: it keeps the parent's step-posterior acceptance column and contributes no offer curve, since the fitted curves describe LLMs, not the project's own agents. endgame_rounds : int How many final regular rounds (plus the forced final itself) are priced at the vote grain when `model.acceptance_vote` exists. 1 covers the last regular round + forced final. discount, walk_if_hopeless, name As :class:`BayesianRationalPolicy`.

Attributes

last_path : str | None `"calibrated"`, `"calibrated-vote"` or `"belief"` (an opponent column that fell back to the parent's model) — set every turn so a run can be audited for which model was live.

Parameters

Name	Type	Default	Description
`model`	LLMOpponent-Model	*required*	
`opponent_model`	`str`	*required*	
`seat_models`	`dict[int, str] | None`	`None`	
`endgame_rounds`	`int`	`1`	
`discount`	`float | None`	`None`	
`walk_if_hopeless`	`bool`	`True`	
`name`	`str`	`'llm-calibrated-rational'`	

Attributes

Name	Type	Summary
`endgame_rounds`		
`last_path`	`str | None`	
`model`		
`opponent_model`		
`seat_models`		

Methods

act

```
act(self, state)
```

source

Parent machinery with the fitted acceptance table when no offer model is fitted (the acceptance-only ablation, and the exact-Bayes regression path under step curves); otherwise the same accept / propose / walk skeleton with the reservation and the proposal continuation both priced on the empirical incoming-offer distribution.

Parameters

Name	Type	Default	Description
`state`		*required*	

model_for_seat

```
model_for_seat(self, seat: int) -> str
```

source

Fitting key seated at `seat` — the per-seat override if given, else `opponent_model`.

Parameters

Name	Type	Default	Description
`seat`	`int`	*required*	

`reservation`

```
reservation(self, state) -> float
```

source

The reservation surplus this policy holds at `state` — the number an offline audit plots against a standing offer's surplus.

Under the empirical offer model when fitted, else the parent's.

Parameters

Name	Type	Default	Description
`state`		*required*	

LLMOpponentModel

Everything the calibrated policy consumes, loaded from ONE fitted artifact.

```
LLMOpponentModel(
    acceptance: AcceptanceCurveSet,
    acceptance_vote: AcceptanceCurveSet | None = None,
    offers: OfferCurveSet | None = None,
    provenance: dict = dict(),
)
```

Defined in `interlens.arena.negotiation.llm_calibrated` · source

Parameters

acceptance : AcceptanceCurveSet Panel-grain `P(accept | z, rounds_left)` per opponent model (“next time this seat moves, does it take this offer” — folds in crowding). acceptance_vote : AcceptanceCurveSet | None Vote-conditional grain, applied in the endgame (the forced final and the `endgame_rounds` before it). Measured LLM vote acceptance is ~0.99 even below the voter’s own reservation, an order of magnitude above the panel rate, and pricing the endgame at the panel grain tells the agent closure is near-impossible exactly where it is near-certain (the lesson of the full-info calibrated lane). offers : OfferCurveSet | None The empirical incoming-offer distribution per opponent model. `None` = keep the parent’s belief-induced offer distribution (the acceptance-only variant, a legitimate ablation arm). provenance : dict The fitting lane’s record (runs, fit/held-out instance split, invocation).

Parameters

Name	Type	Default	Description
`acceptance`	AcceptanceCurveSet	*required*	
`acceptance_vote`	AcceptanceCurveSet | None	`None`	

Name	Type	Default	Description
`offers`	OfferCurveSet | None	`None`	
`provenance`	`dict`	`dict()`	

Attributes

Name	Type	Summary
`SCHEMA`		
`acceptance`	AcceptanceCurveSet	
`acceptance_vote`	AcceptanceCurveSet | None	
`offers`	OfferCurveSet | None	
`provenance`	`dict`	

Methods

`from_json`

```
from_json(cls, path: str | Path) -> 'LLMOpponentModel'
```

source

Load the fitting lane's combined artifact (schema :data:`SCHEMA`); see the fitter for the shape.

Parameters

Name	Type	Default	Description
`path`	`str | Path`	*required*	

step

```
step(cls) -> 'LLMOpponentModel'
```

source

The degenerate model that drives :class:`LLMCalibratedRationalPolicy` back to exact Bayes behaviour (step acceptance, no offer model) — the equivalence-test control.

OfferCurve

One opponent model's fitted distribution of the `z` an offer delivers to its RECIPIENT, as a function of round position.

```
OfferCurve(
    frac_edges: tuple,
    z_quantiles: tuple,
    n_offers: tuple = (),
    low_power: bool = False,
    notes: str = '',
)
```

Defined in `interlens.arena.negotiation.llm_calibrated` · source

This is the concession curve seen from the receiving side, which is the side the stopping rule needs: it answers "if I wait, what is the next package worth to me", in my own affine-invariant units, without knowing the proposer's sheet at all — the recipient's `z` is computed from the recipient's own sheet, so the fit needs no revelation and the runtime evaluation needs no belief.

Parameters

frac_edges : list[float] `K-1` strictly ascending interior edges partitioning round fraction `t = (round-1)/deadline` in `[0, 1]` into `K` bins (`searchsorted` right, exactly like the acceptance bins). Scale-free so a curve fitted on 4-round games evaluates on any deadline. z_quantiles : list[list[float]] `K` rows of equiprobable `z` support points (fitted quantiles of the observed recipient-`z` distribution in that round bin). Row lengths may differ; each row is one discrete pmf with uniform masses. n_offers : list[int] Observation count behind each row, carried so a consumer can see a bin fitted on 12 offers. low_power : bool The fitting lane's own not-quotable flag, carried through like the acceptance curves'. notes : str Free text from the fitting lane (runs, exclusions, caveats).

Parameters

Name	Type	Default	Description
`frac_edges`	`tuple`	*required*	

Name	Type	Default	Description
z_quantiles	tuple	*required*	
n_offers	tuple	()	
low_power	bool	False	
notes	str	''	

Attributes

Name	Type	Summary
frac_edges	tuple	
low_power	bool	
n_offers	tuple	
notes	str	
z_quantiles	tuple	

Methods

from_dict

```
from_dict(cls, d: dict) -> 'OfferCurve'
```

source

Build from one model's JSON block, ignoring keys the schema does not define.

Parameters

Name	Type	Default	Description
d	dict	*required*	

pmf

```
pmf(self, frac: float) -> tuple
```

source

The discrete pmf `(z_values, probs)` of an incoming offer's recipient-`z` at round fraction `frac` (clipped to `[0, 1]`): the fitted quantiles of that round bin with uniform masses.

Parameters

Name	Type	Default	Description
`frac`	`float`	*required*	

OfferCurveSet

A fitted :class:`OfferCurve` per opponent model.

```
OfferCurveSet(curves: dict, default: OfferCurve | None = None, provenance:
    dict = dict())
```

Defined in `interlens.arena.negotiation.llm_calibrated` · source

Same lookup contract as :class:`~interlens.arena.negotiation.calibrated.AcceptanceCurveSet`: an unknown model with no default is an ERROR at seat construction, never a silent fallback.

Parameters

Name	Type	Default	Description
`curves`	`dict`	*required*	
`default`	OfferCurve | None	`None`	
`provenance`	`dict`	`dict()`	

Attributes

Name	Type	Summary
`curves`	`dict`	
`default`	OfferCurve | None	
`provenance`	`dict`	

Methods

`for_model`

```
for_model(self, model_id: str) -> OfferCurve
```

source

The curve for `model_id`, or `default`; :class:`KeyError` naming the fitted models otherwise.

Parameters

Name	Type	Default	Description
`model_id`	`str`	*required*	

`from_json_dict`

```
from_json_dict(cls, blob: dict) -> 'OfferCurveSet'
```

source

Read the fitting lane's `offers` block: `{"models": {id: {...}}, "default": {...}?}`.

Parameters

Name	Type	Default	Description
`blob`	`dict`	*required*	

oracle_context

Module `interlens.arena.negotiation.oracle_context`

The per-decision-point context the negotiation oracles share, written once.

The four numeric oracles (beliefs / acceptance / bestresponse / equilibrium) each need the same three things at a decision point, so they live here rather than in any one of them: the `|D|×n` utility/surplus tables (:class:`GameTables`, built once per game and cached on it by :func:`game_tables`), READERS that recover the turn context — standing offers, round, rounds left, continuation discount, proposer rotation — from the loose history/offer-registry shapes a scenario emits, and the :func:`make_verdict` constructor. The typed actions and the `Oracle` ABC / `OracleVerdict` are imported from interlens-core (`arena/actions.py`, `arena/oracles.py`); the structured state a *policy* reads is `strategies.NegotiationState`.

A *game* is duck-typed: any object exposing `.space` and seat-indexed `.sheets` works, plus optionally `.rounds` / `.info` / `.discount` / `.proposer` / `.veto` (the real one is :class:`~interlens.arena.negotiation.sheets.GameSpec`). `Deal = tuple[int, ...]` is one option index per issue.

Attributes

Name	Type	Summary
`Deal`		

Classes

Name	Summary
`GameTables`	Precomputed dense tables for a game — built once, shared by every oracle so the `O(n*J*|D|)` utility pass is never duplicated.

Functions

Name	Summary
current_round	1-indexed current round.
deal_list	Materialize the deal space in a *stable* order (matrix rows below use this exact order).
effective_discount	The per-round continuation factor the acceptance/best-response/equilibrium oracles should use, read from the game as the single source of truth: discount * (1 - breakdown_risk) (time preference times the per-round no-breakdown survival probability — the BRW 1986 breakdown model).
game_tables	GameTables for game, cached on the game object when possible.
issue_sizes	Per-issue option counts (O_1, ..., O_J), discovered from (in order): an .issue_sizes / .n_options attribute on the space; a sheet's .values rows; or the max option index seen in deals.
make_verdict	Build an OracleVerdict with its free-form extra diagnostics coerced JSON-safe up front (the |D| ×n numpy tables / typed actions the oracles stash) via the shared _jsonify, so to_json and the episode save never crash.
n_agents	

Name	Summary
`normalize`	Return a probability vector from nonnegative weights, optionally mixed with `floor` uniform mass.
`offer_registry`	Recover `{offer_id: Deal}` from the game/history.
`proposer_sequence`	Per-round proposer seat indices.
`rounds_left`	Rounds remaining (this turn inclusive).
`seat_index`	Resolve `agent` to a seat index.
`softmax`	Tempered softmax; `temperature` → `0` approaches a hard argmax (used to break equilibrium cycles).

GameTables

Precomputed dense tables for a game — built once, shared by every oracle so the `O(n*J*|D|)` utility pass is never duplicated.

```
GameTables(
    deals: list[Deal],
    index: dict[Deal, int],
    deals_arr: np.ndarray,
    utility: np.ndarray,
    surplus: np.ndarray,
    thresholds: np.ndarray,
)
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Attributes

deals : list[Deal] The deal space in stable order. index : dict[Deal, int] Inverse map `deal → row`. deals_arr : np.ndarray `(|D|, J)` int array of option indices. utility : np.ndarray `(|D|, n)` per-deal per-agent utility. surplus : np.ndarray `(|D|, n)` per-deal per-agent surplus `utility - threshold`. thresholds : np.ndarray `(n,)` per-agent reservation thresholds.

Parameters

Name	Type	Default	Description
`deals`	`list[Deal]`	*required*	
`index`	`dict[Deal, int]`	*required*	
`deals_arr`	`np.ndarray`	*required*	
`utility`	`np.ndarray`	*required*	
`surplus`	`np.ndarray`	*required*	
`thresholds`	`np.ndarray`	*required*	

Attributes

Name	Type	Summary
deals	list[Deal]	
deals_arr	np.ndarray	
index	dict[Deal, int]	
n_agents	int	
n_deals	int	
surplus	np.ndarray	
thresholds	np.ndarray	
utility	np.ndarray	

Methods

from_game

```
from_game(cls, game) -> 'GameTables'
```

source

Build tables from a GameSpec-like object exposing .space and .sheets.

Reuses the game's own utility_matrix() when available (identical mixed-radix row order, no recomputation); otherwise vectorizes utilities from each sheet's .values rows (single fancy-index per issue), falling back to sheet.utility per deal. The single tables builder — there is no separate build entry point.

Parameters

Name	Type	Default	Description
game		*required*	

current_round

1-indexed current round.

```
current_round(game, history) -> int
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Parameters

Name	Type	Default	Description
`game`		*required*	
`history`		*required*	

deal_list

Materialize the deal space in a *stable* order (matrix rows below use this exact order).

```
deal_list(space: DealSpace) -> list[Deal]
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Parameters

Name	Type	Default	Description
`space`	DealSpace	*required*	

effective_discount

The per-round continuation factor the acceptance/best-response/equilibrium oracles should use, read from the game as the single source of truth: `discount * (1 - breakdown_risk)` (time preference times the per-round no-breakdown survival probability — the BRW 1986 breakdown model).

```
effective_discount(game, override=None) -> float
```

Defined in `interlens.arena.negotiation.oracle_context` · source

An explicit `override` (a non-None oracle-level `discount`) wins, so a caller can still force a value; otherwise the game's own impatience is honored. `GameSpec` defaults `discount=1.0` (neutral) / `breakdown_risk=0.0`, which yields the Sandholm-Vulkan brinkmanship baseline — set `discount < 1` on the game for interior concession to be rational.

Parameters

Name	Type	Default	Description
`game`		*required*	
`override`		`None`	

game_tables

`GameTables` for `game`, cached on the game object when possible.

```
game_tables(game) -> GameTables
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Parameters

Name	Type	Default	Description
`game`		*required*	

issue_sizes

Per-issue option counts `(O_1, ..., O_J)`, discovered from (in order): an `.issue_sizes` / `.n_options` attribute on the space; a sheet's `.values` rows; or the max option index seen in `deals`.

```
issue_sizes(
    space: DealSpace | None = None,
    sheets: Iterable[ScoreSheet] | None = None,
    deals: list[Deal] | None = None,
) -> tuple[int, ...]
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Needed by the belief oracle to build per-issue evaluator hypotheses.

Parameters

Name	Type	Default	Description
`space`	DealSpace | None	`None`	
`sheets`	Iterable[ScoreSheet] | None	`None`	
`deals`	`list[Deal] | None`	`None`	

make_verdict

Build an `OracleVerdict` with its free-form `extra` diagnostics coerced JSON-safe up front (the `|D|×n` numpy tables / typed actions the oracles stash) via the shared `_jsonify`, so `to_json` and the episode save never crash.

```
make_verdict(
    action_values,
    best=None,
    *,
    beliefs=None,
    flags=None,
    extra=None,
) -> OracleVerdict
```

Defined in `interlens.arena.negotiation.oracle_context` · source

`beliefs` is coerced later by `OracleVerdict.to_json`.

Parameters

Name	Type	Default	Description
`action_values`		*required*	
`best`		`None`	
`beliefs`		`None`	
`flags`		`None`	
`extra`		`None`	

n_agents

```
n_agents(game) -> int
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Parameters

Name	Type	Default	Description
`game`		*required*	

normalize

Return a probability vector from nonnegative weights, optionally mixed with `floor` uniform mass.

```
normalize(w: np.ndarray, floor: float = 0.0) -> np.ndarray
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Parameters

Name	Type	Default	Description
`w`	`np.ndarray`	*required*	
`floor`	`float`	`0.0`	

offer_registry

Recover `{offer_id: Deal}` from the game/history.

```
offer_registry(game, history) -> dict
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Prefers an explicit registry, read as either an `offers` ATTRIBUTE (`history.offers` / `game.offers`) or, when `history` is a mapping, an `offers` KEY – the shape the scenario's per-turn history snapshot carries (`ScorableNegotiation._history_snapshot` stores `offers` as a LIST of serialized `Offer` dicts, each with `offer_id` + `deal`). The registry may thus be a `{id: offer}` mapping OR a list of `Offer.to_json()` dicts / `Offer` objects. Falls back to scanning turns for `Propose` actions, assigning sequential ids `O1, O2, …` in order of appearance.

Getting this right is load-bearing: if the standing offers are lost, the acceptance / threshold / best-response oracles value every `Accept` at the no-deal continuation instead of the offer's realized surplus (the single-shot mis-scoring bug), so a rational accept reads as 0 regret AND 0 value.

Parameters

Name	Type	Default	Description
`game`		*required*	
`history`		*required*	

proposer_sequence

Per-round proposer seat indices.

```
proposer_sequence(game) -> list
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Uses `game.proposer_sequence` if present; else a rotation starting at `game.proposer` (default 0) over `range(n)` — the rotating-proposer default.

Parameters

Name	Type	Default	Description
`game`		*required*	

rounds_left

Rounds remaining (this turn inclusive).

```
rounds_left(game, history) -> int
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Uses `game.rounds` and completed rounds when discoverable; defaults to `game.rounds` (or 1).

Parameters

Name	Type	Default	Description
`game`		*required*	
`history`		*required*	

seat_index

Resolve `agent` to a seat index.

```
seat_index(game, agent) -> int
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Accepts an int (returned as-is) or a seat *name* (str), which is matched against `sheet.agent` on each score sheet, then against a `game.seats`/`seat_names`/ `agents` name list, then an int-like string. The real `Oracle` ABC types `agent` as `str` (the seat name), so oracles call this first before indexing the seat-indexed utility tables.

Parameters

Name	Type	Default	Description
`game`		*required*	
`agent`		*required*	

softmax

Tempered softmax; `temperature` → `0` approaches a hard argmax (used to break equilibrium cycles).

```
softmax(a: np.ndarray, temperature: float = 1.0, axis=-1) -> np.ndarray
```

Defined in `interlens.arena.negotiation.oracle_context` · source

Parameters

Name	Type	Default	Description
`a`	`np.ndarray`	*required*	
`temperature`	`float`	`1.0`	
`axis`		`-1`	

policy_participant

Module `interlens.arena.negotiation.policy_participant`

`PolicyParticipant`: a state-dependent pure-Python seat that computes its move from a bound policy.

Where `ScriptedParticipant` cycles fixed strings ignoring the conversation, a `PolicyParticipant` *reads* the view, reconstructs the structured negotiation state (the offer registry, standing offer, its own and opponents' past proposals, the round), asks a bound `policy(state) → action` for a typed action (`Propose` / `Accept` / `Reject` / `Walk`), and emits it in the **same fenced-JSON envelope an LLM seat produces** — so a computable rational agent and an LLM are interchangeable seats in one scenario.

It holds no model/activations, so (like `ScriptedParticipant`) it raises on any interp request (steering / capture / patch / logprobs) rather than silently ignoring it.

Two ways to supply the state each turn:

- **default view reconstruction** — parse the fenced-JSON actions already in the view (role `assistant` = this seat's past turns, role `user` = others'), rebuild the offer ledger with monotonic ids, and infer the round from this seat's completed turns. This keeps the participant symmetric with LLM seats (it reads exactly what an LLM reads) without any English NLP.
- **`state_provider`** — an injected `callable(view) → NegotiationState` for scenarios that already track structured state (e.g. the arena scenario handing over its authoritative registry).

Classes

Name	Summary
`PolicyParticipant`	A pure-Python negotiation seat driven by a bound `policy`.

PolicyParticipant

A pure-Python negotiation seat driven by a bound `policy`.

```
PolicyParticipant(
    name: str,
    policy,
    *,
    seat: int,
    sheet,
    space,
    deadline: int,
    discount: float = 1.0,
    opponents: tuple = (),
    n_seats: int | None = None,
    tables=None,
    state_provider=None,
    registry_prefix: str = 'O',
    system_prompt: str | None = None,
    private_context: tuple = (),
    min_accept: int | None = None,
    veto_seats: tuple = (),
)
```

Defined in `interlens.arena.negotiation.policy_participant` · source

Inherits from: Participant

Parameters

name : str Identifier within the conversation. policy : callable `policy(state: NegotiationState) → Action` — e.g. any policy from `interlens.arena.negotiation.strategies` or an oracle wrapped as a policy. seat : int This seat's index into the game's seat-indexed sheets/tables. sheet : object This seat's private score sheet (exposes `.utility`/`.surplus`/`.threshold`). space : DealSpace The shared deal space. Also the seat's issue/option NAME table: proposals are emitted with names (`space.named`) and incoming name-based deals decoded (`space.parse`), so the transcript is LLM-legible and there is no second copy of the issue names to keep in sync. deadline : int Total number of rounds `T` (for the policy's time-dependent concession). discount : float Per-round discount `delta` carried into the state. opponents : tuple[int, …] Opponent seat indices (default: inferred as all seats != `seat` up

to `n_seats`). n_seats : int | None Total seat count (used to default `opponents` when not given). tables : object | None Optional full-information `GameTables` to attach to the state (enables exact full-info policies). min_accept : int | None Fixed acceptance quorum out of the original seats; `None` means unanimity. veto_seats : tuple[int, …] Seats whose support is mandatory. state_provider : callable | None Optional `callable(view) → NegotiationState` overriding the default view reconstruction. registry_prefix : str Offer-id prefix for the reconstructed registry (default `"O"`, matching `OfferRegistry`). system_prompt : str | None Optional system framing (recorded for view/transcript symmetry; unused by the policy).

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`policy`		*required*	
`seat`	`int`	*required*	
`sheet`		*required*	
`space`		*required*	
`deadline`	`int`	*required*	
`discount`	`float`	`1.0`	
`opponents`	`tuple`	`()`	
`n_seats`	`int | None`	`None`	
`tables`		`None`	
`state_provider`		`None`	
`registry_prefix`	`str`	`'O'`	
`system_prompt`	`str | None`	`None`	
`private_context`	`tuple`	`()`	
`min_accept`	`int | None`	`None`	
`veto_seats`	`tuple`	`()`	

Attributes

Name	Type	Summary
`deadline`		
`discount`		
`min_accept`		
`name`		
`opponents`		
`others_role`		
`policy`		
`private_context`		
`registry_prefix`		
`seat`		
`self_role`		
`sheet`		
`space`		
`state_provider`		
`system_prompt`		
`tables`		
`veto_seats`		

Methods

`act`

```
act(self, state: NegotiationState)
```

source

Compute this seat's action from a `NegotiationState` directly — no view parsing, no engine.

The pure entry a counterfactual-rollout loop calls (equivalent to `self.policy(state)`). Note the bound `self.policy` is itself a public callable `policy(NegotiationState) → Action`, so a rollout can skip the participant wrapper entirely and call the policy on a reconstructed state.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

generate

```
generate(
    self,
    view: list[dict],
    *,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    turn: int | None = None,
    max_new_tokens: int | None = None,
    seat: str | None = None,
) -> Message
```

source

Reconstruct the negotiation state from `view`, ask the bound policy for an action, and return it as a fenced-JSON message (the same envelope LLM seats emit).

Raises on any interp request — a pure-Python seat has no model to steer/capture/patch or read logprobs from.

Parameters

Name	Type	Default	Description
`view`	`list[dict]`	*required*	

Name	Type	Default	Description
steering		None	
capture		None	
patch		None	
return_logprobs	bool	False	
turn	int | None	None	
max_new_tokens	int | None	None	
seat	str | None	None	

references

Module `interlens.arena.negotiation.references`

Citation-key registry for the negotiation solution-concept and generator modules.

Every algorithm in this package cites its primary source by a short key (e.g. `[nash1950]`) in the relevant docstring; this module maps each key to the full citation (authors, year, title, venue, volume/pages, DOI) and a stable URL, so the citations live in exactly one place and the docstrings stay terse. Keys and page/section page and section references were verified against the fetched primary PDFs.

Usage — the registry is bibliography DATA (no accessor API); read it directly:

```
from interlens.arena.negotiation.references import REFERENCES
REFERENCES["nash1950"].url         # → 'https://www.jstor.org/stable/
    1907266'
str(REFERENCES["ks1975"])          # "<citation> <url> — <note>" for a
    header line
```

Attributes

Name	Type	Summary
`REFERENCES`	dict[str, Reference]	

Classes

Name	Summary
`Reference`	One bibliographic entry.

Reference

One bibliographic entry.

```
Reference(key: str, citation: str, url: str, note: str = '')
```

Defined in `interlens.arena.negotiation.references` · source

`key` is the short citation key used in docstrings; `citation` the full human-readable reference (authors, year, title, venue, volume(issue):pages, DOI where available); `url` a stable link (publisher landing page, JSTOR, arXiv, or open PDF). `note` optionally records what specifically this reference grounds (e.g. the exact theorem/algorithm/page a module relies on).

Parameters

Name	Type	Default	Description
`key`	`str`	*required*	
`citation`	`str`	*required*	
`url`	`str`	*required*	
`note`	`str`	`''`	

Attributes

Name	Type	Summary
`citation`	`str`	
`key`	`str`	
`note`	`str`	
`url`	`str`	

rewards

Module `interlens.arena.negotiation.rewards`

Outcome rewards for RL on scorable negotiation: the smoothed log-Nash objective.

Everything here is a pure function of the **normalized surplus vector** `z = (z_i)`, `z_i = (u_i(d) - tau_i) / c_i`, where `c_i` is party `i`'s unconstrained maximum surplus. That is exactly the `outcome.normalized_realized_surplus` an episode already records, so a reward computed here is **text-blind**: it never reads a token the policy generated, only the engine's scoring of the deal it closed.

Why this shape (see the proposal for the full argument):

- `smoothed_log_utility` is `log z` above `eps` — so `table_reward` is a monotone transform of normalized Nash welfare wherever every party clears threshold, and training optimizes the judgment metric rather than a proxy. Concavity does the fairness work: moving surplus from a rich party to a poor one always raises the mean.
- Below `eps` it continues **linearly** with the slope `log` has at `eps`. Plain `log`/NNW is flat (zero) everywhere a party is below threshold, which is precisely where the observed pathology lives, so RL would get no gradient there. The linear branch restores slope while preserving the ordering — a below-threshold outcome still scores worse than every individually rational one.
- Walking away pays `no_deal_utility` (`= g(0)`) to every seat: worse than any weakly-IR deal, better than a deeply below-threshold one. Walking is priced, not free and not catastrophic.
- `z` is affine-invariant in each party's private score sheet, so no arm can win by rescaling its own points.

Every function takes an optional `g_floor`, added for fairness-GRPO **v2**. Default `None` reproduces the shape above exactly, so the v1 pilot's numbers stay byte-reproducible. With a floor the per-party utility is clipped from below at a constant, which bounds the violation branch's dynamic range:

v1 measured that the unbounded tail made the objective behave as an IR-violation penalty rather than as Nash welfare (one party at `z = -0.2` is worth `-25.6`), so nearly all gradient landed on “did anyone get shorted” and none on “was the surplus divided well”. `violation_decomposition` is the measurement that turns that diagnosis into a number and calibrates the floor.

`mixture_rewards` implements the per-seat objective `R_i(lam) = (1 - lam) * g(z_i) + lam * R_table`, the self-interest/table-welfare mixture whose sweep is the experiment. `potential` is the potential-based shaping term `Phi(s) = R_table(standing offer at s)`; `shaping_reward` forms `gamma * Phi(s') - Phi(s)`, which by Ng-Harada-Russell leaves the optimal policy unchanged while densifying credit over a long episode.

Attributes

Name	Type	Summary
`DEFAULT_EPS`		Threshold below which `smoothed_log_utility` switches from `log` to its linear continuation.

Functions

Name	Summary
`mixture_rewards`	Per-seat episode rewards `R_i(lam) = (1 - lam) * g(z_i) + lam * R_table`, one per party in seat order.
`no_deal_utility`	The utility every seat receives when the episode ends with no deal: `g(0) = log(eps) - 1` (`-5.605` at the default `eps`).

Name	Summary
potential	Shaping potential `Phi(s) = max(R_table(standing offer at s), g(0))` — how good the table is right now.
shaping_reward	Potential-based shaping increment `gamma * Phi(s') - Phi(s)` for one turn: “did your move pull the standing offer toward or away from a fair deal?”.
smoothed_log_utility	Per-party utility `g(z)` of a normalized surplus: `log z` for `z ⩾ eps`, and below that the tangent line `log(eps) + (z - eps) / eps` (same value and slope at `eps`, so `g` is C1 and strictly increasing on the whole real line).
table_reward	Table-welfare term `R_table = mean_i g(z_i)` on a closed deal, the smoothed log form of normalized Nash welfare.
violation_decomposition	Split `table_reward` into `(among_ir, violation)` with `among_ir + violation == table_reward(z)`.

mixture_rewards

Per-seat episode rewards `R_i(lam) = (1 - lam) * g(z_i) + lam * R_table`, one per party in seat order.

```python
mixture_rewards(
    z: Sequence[float] | None,
    *,
    lam: float,
    n_agents: int | None = None,
    eps: float = DEFAULT_EPS,
    g_floor: float | None = None,
) -> list[float]
```

Defined in `interlens.arena.negotiation.rewards` · source

`lam=0` is pure self-interest (the manipulation check: it should reproduce the observed pathologies), `lam=1` is pure table welfare (identical for every seat), and intermediate values trace the frontier between them. On a no-deal episode (`z=None`) every seat gets `no_deal_utility` at any `lam`, so `n_agents` is required to size the output.

Parameters

Name	Type	Default	Description
`z`	`Sequence[float] | None`	*required*	
`lam`	`float`	*required*	
`n_agents`	`int | None`	`None`	
`eps`	`float`	`DEFAULT_EPS`	
`g_floor`	`float | None`	`None`	

no_deal_utility

The utility every seat receives when the episode ends with no deal: `g(0) = log(eps) - 1` (`-5.605` at the default `eps`).

```
no_deal_utility(*, eps: float = DEFAULT_EPS, g_floor: float | None = None)
    -> float
```

Defined in `interlens.arena.negotiation.rewards` · source

Sits strictly below any weakly-IR deal (`z ⩾ 0` scores `⩾ g(0)`) and strictly above deals that push a party below `z = 0`. `g_floor` is accepted (and validated) for signature symmetry with the rest of the module but cannot change this value, since a legal floor is by definition below it.

Parameters

Name	Type	Default	Description
`eps`	`float`	`DEFAULT_EPS`	
`g_floor`	`float | None`	`None`	

potential

Shaping potential `Phi(s) = max(R_table(standing offer at s), g(0))` — how good the table is right now.

```
potential(
    z: Sequence[float] | None,
    *,
    eps: float = DEFAULT_EPS,
    g_floor: float | None = None,
) -> float
```

Defined in `interlens.arena.negotiation.rewards` · source

With no standing offer this is `no_deal_utility`, so the first offer is credited by how much better than walking it is.

The floor at `g(0)` is what makes this usable as a shaping term. `table_reward` is unbounded below (the linear branch is deliberately so, to keep gradient on threshold violations), so an absurd proposal could otherwise drive `Phi` to −100 and produce a shaping increment an order of magnitude larger than the episode reward it is supposed to densify. Flooring is not a fudge: a standing offer worse than no agreement is worth no more than no agreement, because any party can simply refuse it. `Phi` is still a pure function of the state, so potential-based shaping's policy-invariance guarantee is untouched, and the increment is now bounded by `|g(0)|`. `g_floor` (see :func:`smoothed_log_utility`) additionally bounds each party's term before the mean is taken; with a floor in play this `max` is rarely the binding constraint, but it is kept so `Phi` has the same meaning under both reward shapes.

Parameters

Name	Type	Default	Description
`z`	`Sequence[float] | None`	*required*	
`eps`	`float`	`DEFAULT_EPS`	
`g_floor`	`float | None`	`None`	

shaping_reward

Potential-based shaping increment `gamma * Phi(s') - Phi(s)` for one turn: “did your move pull the standing offer toward or away from a fair deal?”.

```
shaping_reward(
    z_before: Sequence[float] | None,
    z_after: Sequence[float] | None,
    *,
    gamma: float = 1.0,
    eps: float = DEFAULT_EPS,
    g_floor: float | None = None,
) -> float
```

Defined in `interlens.arena.negotiation.rewards` · source

Potential-based shaping provably preserves the optimal policy (Ng, Harada & Russell 1999), so this densifies credit assignment over a ~25-round episode without changing what is being optimized. `gamma` should match the discount used by the learner (`1.0` for the undiscounted episodic setting).

Parameters

Name	Type	Default	Description
`z_before`	`Sequence[float] | None`	*required*	
`z_after`	`Sequence[float] | None`	*required*	
`gamma`	`float`	`1.0`	
`eps`	`float`	`DEFAULT_EPS`	
`g_floor`	`float | None`	`None`	

smoothed_log_utility

Per-party utility `g(z)` of a normalized surplus: `log z` for `z ⩾ eps`, and below that the tangent line `log(eps) + (z - eps) / eps` (same value and slope at `eps`, so `g` is C1 and strictly increasing on the whole real line).

```
smoothed_log_utility(
    z: float,
    *,
    eps: float = DEFAULT_EPS,
    g_floor: float | None = None,
) -> float
```

Defined in `interlens.arena.negotiation.rewards` · source

Unbounded below by default, which was the original intent: deep threshold violations are meant to hurt.

`g_floor` clips that unbounded tail from below at a constant, `max(g(z), g_floor)`. Leave it `None` for the original unbounded shape. Pass a value when the *dynamic range* of the violation branch is the problem rather than its direction: because the linear branch is unbounded, one party at `z = -0.2` contributes `-25.6` against a well-behaved deal's `-2`ish, so nearly all variance in `table_reward` — and therefore nearly all gradient — lives in "did anybody get shorted" rather than in "was the surplus divided well". A floor bounds the violation term's contribution and lets the among-IR region carry the signal, at the cost of making `g` non-strict (flat) below the crossing point `z*` where `g(z*) = g_floor`.

The floor must sit strictly below :func:`no_deal_utility` (`-5.605` at the default `eps`) or the ordering inverts and a below-threshold agreement becomes worth more than walking away, which is the one ordering the linear branch exists to protect. This is checked, not documented-and-hoped.

Parameters

Name	Type	Default	Description
`z`	`float`	*required*	
`eps`	`float`	`DEFAULT_EPS`	

Name	Type	Default	Description
g_floor	float | None	None	

table_reward

Table-welfare term `R_table = mean_i g(z_i)` on a closed deal, the smoothed log form of normalized Nash welfare.

```
table_reward(
    z: Sequence[float] | None,
    *,
    n_agents: int | None = None,
    eps: float = DEFAULT_EPS,
    g_floor: float | None = None,
) -> float
```

Defined in `interlens.arena.negotiation.rewards` · source

`z=None` means no deal and returns `no_deal_utility` (`n_agents` is then unused — the mean of a constant is that constant). `g_floor` clips each party's term from below; see :func:`smoothed_log_utility`.

Parameters

Name	Type	Default	Description
`z`	`Sequence[float] | None`	*required*	
`n_agents`	`int | None`	`None`	
`eps`	`float`	`DEFAULT_EPS`	
`g_floor`	`float | None`	`None`	

violation_decomposition

Split `table_reward` into `(among_ir, violation)` with `among_ir + violation == table_reward(z)`.

```
violation_decomposition(
    z: Sequence[float] | None,
    *,
    eps: float = DEFAULT_EPS,
    g_floor: float | None = None,
) -> tuple[float, float]
```

Defined in `interlens.arena.negotiation.rewards` · source

`among_ir = mean_i g(max(z_i, eps))` is what the table would score if every shorted party were pinned at its own threshold — the pure *distribution* term, which varies only with how well the surplus is divided among parties that cleared their walk-away. `violation = mean_i [g(z_i) - g(max(z_i, eps))] ⩽ 0` is the penalty for shorting parties, and is exactly zero on an individually rational deal.

The split exists to be *measured*: taking the variance of each component over a campaign answers “how much of the reward’s dynamic range is the violation branch”, which is the diagnosis of the v1 fairness-GRPO negative (the objective was nominally Nash welfare but behaviourally an IR-violation penalty) and the calibration target for `g_floor`. A no-deal episode is not a violation: it returns `(no_deal_utility, 0.0)`.

Parameters

Name	Type	Default	Description
`z`	`Sequence[float] | None`	*required*	
`eps`	`float`	`DEFAULT_EPS`	
`g_floor`	`float | None`	`None`	

sheets

Module `interlens.arena.negotiation.sheets`

Private score sheets, the additive utility model, the game specification, and the NumPy utility matrix.

Each party holds a secret **score sheet**: an integer/real value for every option of every issue, plus a private acceptance **threshold** `tau_i` (its BATNA / reservation value). Utility is the Keeney-Raiffa additive form `u_i(d) = sum_j s_ij(d_j)` [keeney_raiffa1976]; the analysis object is always the **surplus** `x_i(d) = u_i(d) - tau_i` – raw points sit on arbitrary private scales, so only scale-invariant solution concepts (Nash product, Kalai-Smorodinsky) are defensible across parties (see `solutions.py`).

`GameSpec` bundles the deal space, the sheets, and the protocol knobs (rounds, full/private info, chat on/off, proposer/veto seats, agreement rule) into one object that round-trips through a plain JSON dict, so a whole game drops straight into an arena `Instance.payload` and back.

The **workhorse** every solution concept consumes is :func:`utility_matrix` – the dense `|D| x n` array `U[k, i] = u_i(deal_at(k))` built once via a vectorized mixed-radix accumulation (no Python loop over deals).

Example:

```
space = DealSpace((Issue("A", ("a0", "a1")), Issue("B", ("b0", "b1",
     "b2"))))
alice = ScoreSheet("Alice", ((10, 0), (0, 3, 6)), threshold=5.0)
alice.utility((1, 2))            # 0 + 6 = 6.0
alice.surplus((1, 2))            # 6 - 5 = 1.0
U = utility_matrix(space, (alice, bob))     # shape (6, 2)
```

Attributes

Name	Type	Summary
`CONSTRAINTS`	`dict[str, 'Callable[[Deal], bool]']`	

Classes

Name	Summary
GameSpec	A complete negotiation game: the deal space, every party's private sheet, and the protocol knobs.
ScoreSheet	One party's private additive score sheet.

Functions

Name	Summary
pairwise_iou	Mean pairwise Intersection-over-Union of the parties' *value supports* – the score-function-overlap descriptor of the TMLR reproduction [reproB_tmlr] (their games run 18.8-29.8%).
register_constraint	Register a structural deal predicate under `name` for `GameSpec(constraint=name)`.
sparsity	Fraction of option cells (over all sheets, issues, options) whose value is exactly zero – the score-sheet sparsity descriptor of the TMLR reproduction [reproB_tmlr] (their games run 23.7-43.0%).
surplus_matrix	The `|D| x n` surplus matrix `U - tau`.
utility_matrix	Build the dense `|D| x n` utility matrix `U[k, i] = u_i(space.deal_at(k))`.

GameSpec

A complete negotiation game: the deal space, every party's private sheet, and the protocol knobs.

```
GameSpec(
    space: DealSpace,
    sheets: tuple[ScoreSheet, ...],
    rounds: int = 4,
    info: str = 'full',
    chat: bool = True,
    proposer: int = 0,
    veto: int | list[int] | None = None,
    min_accept: int | None = None,
    discount: float = 1.0,
    breakdown_risk: float = 0.0,
    constraint: str | None = None,
    meta: dict = dict(),
)
```

Defined in `interlens.arena.negotiation.sheets` · source

Beyond `space`/`sheets` the fields carry the protocol arms: `rounds` (round-robin rounds before a forced final), `info` (`"full"` = sheets common knowledge, `"private"` = only a prior over types), `chat` (whether a public cheap-talk channel exists alongside formal moves), and the agreement structure – `proposer` seat (a rotating-proposer scenario may ignore this default), `veto` (one seat, a list of seats, or `None` – see :attr:`veto_seats`), and `min_accept` (how many parties must clear their threshold; `None` = unanimity). Impatience knobs the equilibrium/acceptance oracles read as their single source of truth: `discount` (per-round delta in (0, 1], 1.0 = none) and `breakdown_risk` (per-round exogenous-breakdown probability in [0, 1), 0.0 = none). `meta` holds anything scenario-private (e.g. generator provenance, issue-type labels). Serializes to/from a plain JSON dict for `Instance.payload`.

Parameters

Name	Type	Default	Description
`space`	DealSpace	*required*	

Name	Type	Default	Description
`sheets`	tuple[ScoreSheet, …]	*required*	
`rounds`	`int`	`4`	
`info`	`str`	`'full'`	
`chat`	`bool`	`True`	
`proposer`	`int`	`0`	
`veto`	`int | list[int] | None`	`None`	
`min_accept`	`int | None`	`None`	
`discount`	`float`	`1.0`	
`breakdown_risk`	`float`	`0.0`	
`constraint`	`str | None`	`None`	
`meta`	`dict`	`dict()`	

Attributes

Name	Type	Summary
`agents`	`list[str]`	Party names in seat order.
`breakdown_risk`	`float`	
`chat`	`bool`	
`constraint`	`str | None`	
`discount`	`float`	
`info`	`str`	
`meta`	`dict`	
`min_accept`	`int | None`	
`n_parties`	`int`	Number of parties `n`.
`proposer`	`int`	
`rounds`	`int`	

Name	Type	Summary
`sheets`	tuple[ScoreSheet, …]	
`space`	DealSpace	
`thresholds`	`np.ndarray`	The reservation vector `tau` of shape `(n,)`.
`veto`	`int | list[int] | None`	
`veto_seats`	`list[int]`	The veto seats as a list (`[]` if none): normalizes the `veto` field, which may be a single seat index, a list of seat indices, or `None`.

Methods

`constraint_fn`

```
constraint_fn(self) -> 'Callable[[Deal], bool] | None'
```

source

This game's structural deal predicate, resolved from the `constraint` NAME via :data:`CONSTRAINTS`, or `None` when it declares none.

Raises `KeyError` for an unregistered name.

`feasible`

```
feasible(self, deal: Deal) -> bool
```

source

Whether ONE deal may close under this game's agreement rule — the single-deal counterpart of :meth:`feasible_mask` (it indexes that mask, so the two can never disagree).

Parameters

Name	Type	Default	Description
deal	Deal	*required*	

feasible_mask

```
feasible_mask(self, U: np.ndarray | None = None) -> np.ndarray
```

source

Boolean `(|D|,)` mask of deals that pass this game's agreement rule: the `proposer` clears its threshold, the `veto` seat (if any) clears its threshold, at least `min_accept` parties clear theirs (`min_accept=None` => all `n` => plain unanimity / the IR set), AND the deal satisfies this game's `constraint` (if it declares one).

Pass a precomputed `U` to avoid rebuilding it.

Because every exact solution concept, the oracles, and the scenario's surplus ceiling all read this mask, a constraint declared here flows into all of them for free — it is the single definition of "may this deal close?". :meth:`feasible` is the single-deal form of the same rule.

Parameters

Name	Type	Default	Description
U	np.ndarray | None	None	

from_json

```
from_json(d: dict) -> 'GameSpec'
```

source

Rebuild a `GameSpec` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

normalized_geometry

```
normalized_geometry(self) -> dict
```

source

The game's scale-invariant surplus geometry, and the deal that attains its ceiling.

Each party is normalized by its own **capacity** `c_i = max_d x_i(d)` (its best surplus over the whole deal space), giving `z_i = x_i / c_i`. The **ceiling** is `max` over the FEASIBLE set (whatever :meth:`feasible_mask` admits) of `sum_i z_i`, and the **ceiling deal** is the argmax — ties broken by the smallest enumeration index, so it is deterministic.

Returns `{"feasible", "capacities", "normalized_ceiling", "raw_ceiling", "ceiling_index", "ceiling_deal"}`; `ceiling_index`/`ceiling_deal` are `None` when the game admits no feasible deal or some party has zero capacity (a degenerate sheet), which is also when `normalized_ceiling` is 0.

This is the single definition of "the best deal on the table": `normalized_ceiling` is the denominator of the scenario's `primary` score (so score 1.0 *is* the ceiling deal), and `ceiling_deal` is what an experiment seeds when it wants the optimum already tabled.

surplus_matrix

```
surplus_matrix(self) -> np.ndarray
```

source

The `|D| x n` surplus matrix `U - tau` for this game.

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict of the whole game (drops straight into `Instance.payload`).

utility_matrix

```
utility_matrix(self) -> np.ndarray
```

source

The `|D| x n` utility matrix for this game (see module-level :func:`utility_matrix`).

ScoreSheet

One party's private additive score sheet.

```
ScoreSheet(agent: str, values: tuple[tuple[float, ...], ...], threshold:
    float)
```

Defined in `interlens.arena.negotiation.sheets` · source

`values[j]` is the tuple of per-option values for issue `j` (same length as that issue's options); `threshold` is the party's private acceptance minimum `tau_i` (its BATNA). Utility is the sum of the chosen options' values; surplus is utility minus threshold. Frozen/hashable so a tuple of sheets can key a cached utility matrix.

Parameters

Name	Type	Default	Description
`agent`	`str`	*required*	
`values`	`tuple[tuple[float, ...], ...]`	*required*	
`threshold`	`float`	*required*	

Attributes

Name	Type	Summary
`agent`	`str`	
`max_utility`	`float`	Best attainable utility (pick each issue's highest-valued option).
`min_utility`	`float`	Worst attainable utility (pick each issue's lowest-valued option).

Name	Type	Summary
`n_issues`	`int`	Number of issues this sheet scores.
`threshold`	`float`	
`values`	`tuple[tuple[float, ...], ...]`	

Methods

from_json

```
from_json(d: dict) -> 'ScoreSheet'
```

source

Rebuild a `ScoreSheet` from :meth:`to_json` output (lists -> tuples).

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

rescaled

```
rescaled(self, a: float, c: float = 0.0) -> 'ScoreSheet'
```

source

This sheet under the positive affine transform of the *utility function* `u → a*u + c` with the threshold moved the same way (`tau → a*tau + c`).

Requires `a > 0`. Because the disagreement point moves with the scale, every surplus obeys `x → a*x` exactly; scale-invariant concepts (NBS, KS) are unchanged, non-invariant ones (utilitarian, egalitarian) generally are not – the property the tests exercise. The additive constant `c` is realized on the

additive sheet by adding it to a single issue's options (issue 0), so that the *total* utility shifts by `c` exactly once (not once per issue).

Parameters

Name	Type	Default	Description
`a`	`float`	*required*	
`c`	`float`	`0.0`	

surplus

```
surplus(self, deal: Deal) -> float
```

source

Surplus `x(d) = u(d) - threshold` – positive iff the deal clears this party's BATNA.

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict (values as nested lists).

utility

```
utility(self, deal: Deal) -> float
```

source

Additive utility `u(d) = sum_j values[j][d_j]` of a deal.

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	

pairwise_iou

Mean pairwise Intersection-over-Union of the parties' *value supports* – the score-function-overlap descriptor of the TMLR reproduction [reproB_tmlr] (their games run 18.8-29.8%).

```
pairwise_iou(sheets: tuple[ScoreSheet, ...] | list[ScoreSheet]) -> float
```

Defined in `interlens.arena.negotiation.sheets` · source

Each party's support is the set of `(issue, option)` cells it scores with a nonzero value; for every pair of parties IoU = `|support_a & support_b| / |support_a | support_b|`, averaged over all pairs. Low IoU = parties care about disjoint parts of the deal (integrative, logrolling-friendly); high IoU = they fight over the same cells (distributive).

Parameters

Name	Type	Default	Description
`sheets`	tuple[ScoreSheet, …] | list[ScoreSheet]	*required*	

register_constraint

Register a structural deal predicate under `name` for `GameSpec(constraint=name)`.

```
register_constraint(name: str, predicate: 'Callable[[Deal], bool]') ->
    None
```

Defined in `interlens.arena.negotiation.sheets` · source

Re-registering the same name is an error – two games silently sharing a name would make a stored instance ambiguous.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`predicate`	`'Callable[[Deal], bool]'`	*required*	

sparsity

Fraction of option cells (over all sheets, issues, options) whose value is exactly zero – the score-sheet sparsity descriptor of the TMLR reproduction [reproB_tmlr] (their games run 23.7-43.0%).

```
sparsity(sheets: tuple[ScoreSheet, ...] | list[ScoreSheet]) -> float
```

Defined in `interlens.arena.negotiation.sheets` · source

Zero options are the `don't care` slots that create logrolling room and Pareto-dominated-but-acceptable deals.

Parameters

Name	Type	Default	Description
`sheets`	tuple[ScoreSheet, …] | list[ScoreSheet]	*required*	

surplus_matrix

The `|D| x n` surplus matrix `U - tau`.

```
surplus_matrix(
    space: DealSpace,
    sheets: tuple[ScoreSheet, ...] | list[ScoreSheet],
) -> np.ndarray
```

Defined in `interlens.arena.negotiation.sheets` · source

Parameters

Name	Type	Default	Description
`space`	DealSpace	*required*	
`sheets`	tuple[ScoreSheet, …] | list[ScoreSheet]	*required*	

utility_matrix

Build the dense `|D| x n` utility matrix `U[k, i] = u_i(space.deal_at(k))`.

```
utility_matrix(
    space: DealSpace,
    sheets: tuple[ScoreSheet, ...] | list[ScoreSheet],
) -> np.ndarray
```

Defined in `interlens.arena.negotiation.sheets` · source

This is the array every solution concept in `solutions.py` consumes. It is built without a Python loop over the `|D|` deals: for each issue `j` the per-option, per-party value block (shape `(|options_j|, n)`) is gathered by the mixed-radix option-index pattern `(arange(|D|) // stride_j) % |options_j|` and added in, so the cost is `O(J * |D| * n)` vectorized. Row order matches :meth:`DealSpace.deal_at` / :meth:`DealSpace.enumerate`.

Parameters

Name	Type	Default	Description
`space`	DealSpace	*required*	
`sheets`	tuple[ScoreSheet, ...] | list[ScoreSheet]	*required*	

solutions

Module `interlens.arena.negotiation.solutions`

Exact axiomatic solution concepts over the fully-enumerated deal space.

Everything here operates on the `|D| x n` utility matrix `U` (`sheets.utility_matrix`) and the reservation vector `tau`; the analysis object is the surplus `X = U - tau`. Because the space is enumerated, every concept is computed exactly (no sampling, no convex-hull approximation). Each function's docstring carries the primary citation by key – full text in `references.py`.

Two solution concepts are **exactly scale-invariant** and therefore the only ones defensible across arbitrary private score-sheet scales: the Nash Bargaining Solution (argmax of the surplus product) and Kalai-Smorodinsky (argmax of the min normalized surplus). Utilitarian and egalitarian are **not** scale-invariant – they are only meaningful on a shared or normalized scale, and their docstrings flag this. Maximum Nash Welfare provides the two-stage fallback when no deal clears every party's threshold (empty strict-IR set), where "no deal" is itself the rational outcome.

Key terms: **IR set** = individually rational deals (all surpluses >= 0; strict > 0 for product solutions); **ideal point** `b_i` = the best surplus party `i` can get among IR deals; **Pareto frontier** = deals not utility-dominated by any other deal.

Low-level `*_index` functions return `(best_index, tie_indices, note)` on `(U, tau)` arrays; the high-level :func:`all_solutions` / :func:`analyze` wrap them into :class:`SolutionPoint` objects and the per-instance descriptor dict.

Classes

Name	Summary
`SolutionPoint`	One solution concept's chosen deal, resolved against a concrete game.

Functions

Name	Summary
`all_solutions`	Compute every solution concept for a game, returning `{concept_name: SolutionPoint}`.
`analyze`	The precomputed per-instance analysis dict every generated game ships with.
`distance_to_frontier`	Euclidean distance, in normalized-surplus space, from deal `index` to the nearest Pareto-frontier deal (0 iff `index` is itself Pareto-optimal).
`distance_to_solution`	Euclidean distance, in normalized-surplus space, between deal `index` and a reference solution deal `target_index` (e.g. the NBS or KS index).
`egalitarian_index`	Discrete **egalitarian (Kalai proportional) solution**: the IR deal maximizing `min_i x_i(d)`, leximin refined [kalai1977].
`egalitarian_welfare`	Egalitarian (Rawlsian) welfare `ESW = min x_i` — “is any single party’s interest being ignored”.
`gini`	Gini coefficient of the surplus distribution (0 = perfectly equal, $\to 1$ = maximally unequal).
`ideal_surplus`	The ideal-point surplus vector `b` of shape `(n,)`: `b_i` = the largest surplus party `i` attains, over the IR set (`restrict_ir=True`) or over all deals.

Name	Summary
ir_mask	Boolean (|D|,) mask of individually rational (acceptable) deals: every party's surplus is ⩾ 0 (strict=True requires > 0, the domain of the product solutions).
kalai_smorodinsky_index	Discrete **Kalai-Smorodinsky solution**: the Pareto-optimal IR deal maximizing the minimum normalized surplus min_i x_i(d)/b_i (b = ideal point over IR), refined by leximin over the normalized surplus vector [ks1975].
max_nash_welfare_index	**Maximum Nash Welfare** with the two-stage empty-product rule [caragiannis2019] (Def. 3.1 + Algorithm 1, pp.12:7-12:8): first find the largest number of parties that can be made simultaneously positive-surplus by a single deal, then, among deals achieving that count, maximize the product of the positive surpluses (via sum log over the satisfied parties).
nash_bargaining_index	Discrete **Nash Bargaining Solution**: argmax_{d: x_i(d) > 0 for all i} prod_i x_i(d), computed as argmax sum_i log x_i(d) for overflow safety [nash1950] (solution statement p.159; the non-convex/finite axiomatization is [mariotti1998]; the symmetric n-player product is [harsanyi1963]).

Name	Summary
nash_geomean	Geometric mean of surpluses NSW^(1/n), in surplus units (0.0 if any party is non-positive, per the Nash convention above).
nash_welfare	Nash social welfare NSW = prod x_i when every surplus is strictly positive, else 0.0.
normalized_surplus	Per-deal nonnegative normalized surplus clip(x_i, 0) / b_i (b_i = ideal over all deals), the scale-invariant coordinate the distance metrics live in.
pareto_mask	Boolean (|D|,) mask of Pareto-optimal deals: deal d is on the frontier iff no other deal weakly dominates it on every party and strictly on at least one.
utilitarian_index	Discrete **utilitarian solution**: the IR deal maximizing the surplus sum sum_i x_i(d) [harsanyi1955].
welfare	Utilitarian social welfare USW = sum x_i.

SolutionPoint

One solution concept's chosen deal, resolved against a concrete game.

```
SolutionPoint(
    concept: str,
    index: int,
    deal: Deal,
    named: dict[str, str],
    utilities: tuple[float, ...],
    surpluses: tuple[float, ...],
    ties: tuple[int, ...],
    note: str = '',
    scale_invariant: bool = True,
)
```

Defined in `interlens.arena.negotiation.solutions` · source

`index` is the deal's flat position (row in `U`), `deal` the option-index tuple, `named` its human-readable form; `utilities` and `surpluses` are the per-party vectors at that deal; `ties` lists every equally-optimal deal (>=1); `note` records fallbacks/caveats; `scale_invariant` flags whether the concept is defensible across arbitrary private scales.

Parameters

Name	Type	Default	Description
`concept`	`str`	*required*	
`index`	`int`	*required*	
`deal`	`Deal`	*required*	
`named`	`dict[str, str]`	*required*	
`utilities`	`tuple[float, ...]`	*required*	
`surpluses`	`tuple[float, ...]`	*required*	
`ties`	`tuple[int, ...]`	*required*	
`note`	`str`	`''`	
`scale_invariant`	`bool`	`True`	

Attributes

Name	Type	Summary
concept	str	
deal	Deal	
index	int	
named	dict[str, str]	
note	str	
scale_invariant	bool	
surpluses	tuple[float, ...]	
ties	tuple[int, ...]	
utilities	tuple[float, ...]	

Methods

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict.

all_solutions

Compute every solution concept for a game, returning `{concept_name: SolutionPoint}`.

```
all_solutions(
    space: DealSpace,
    sheets: tuple[ScoreSheet, ...] | list[ScoreSheet],
) -> dict[str, SolutionPoint]
```

Defined in `interlens.arena.negotiation.solutions` · source

Builds the utility matrix once and reuses it.

Parameters

Name	Type	Default	Description
`space`	DealSpace	*required*	
`sheets`	tuple[ScoreSheet, …] | list[ScoreSheet]	*required*	

analyze

The precomputed per-instance analysis dict every generated game ships with.

```
analyze(
    space: DealSpace,
    sheets: tuple[ScoreSheet, ...] | list[ScoreSheet],
    acceptable_mask: np.ndarray | None = None,
) -> dict
```

Defined in `interlens.arena.negotiation.solutions` · source

Contains: `deal_space_size` |D|, `n_parties`, `pareto_count`, `ir_count` |IR|, `ir_pareto_count` |IR∩Pareto|, `ir_pareto_fraction` |IR∩Pareto|/|IR| (how near-zero-sum the acceptable set is), `dominated_acceptable_fraction` = 1 - that (the central score-sheet-repair target [reproA2025]), `empty_ir`, `ideal_surplus` (over IR), the score-sheet descriptors `sparsity` and `pairwise_iou` [reproB_tmlr], and every solution point under `solutions`. Pass `acceptable_mask` to score the fraction against a custom agreement rule (e.g. `GameSpec.feasible_mask`); default is unanimity IR (all surplus >= 0). Fully JSON-serializable.

Parameters

Name	Type	Default	Description
`space`	DealSpace	*required*	
`sheets`	tuple[ScoreSheet, …] | list[ScoreSheet]	*required*	
`acceptable_mask`	`np.ndarray | None`	`None`	

distance_to_frontier

Euclidean distance, in normalized-surplus space, from deal `index` to the nearest Pareto-frontier deal (0 iff `index` is itself Pareto-optimal).

```
distance_to_frontier(U: np.ndarray, tau: np.ndarray, index: int) -> float
```

Defined in `interlens.arena.negotiation.solutions` · source

Scale-invariant. This is the centipawn-loss-style denominator for per-turn divergence: how far a chosen deal sits below the efficient frontier.

Parameters

Name	Type	Default	Description
`U`	`np.ndarray`	*required*	
`tau`	`np.ndarray`	*required*	
`index`	`int`	*required*	

distance_to_solution

Euclidean distance, in normalized-surplus space, between deal `index` and a reference solution deal `target_index` (e.g. the NBS or KS index).

```
distance_to_solution(
    U: np.ndarray,
    tau: np.ndarray,
    index: int,
    target_index: int,
) -> float
```

Defined in `interlens.arena.negotiation.solutions` · source

Scale-invariant; 0 iff the two deals give identical normalized surplus.

Parameters

Name	Type	Default	Description
`U`	`np.ndarray`	*required*	
`tau`	`np.ndarray`	*required*	
`index`	`int`	*required*	
`target_index`	`int`	*required*	

egalitarian_index

Discrete **egalitarian (Kalai proportional) solution**: the IR deal maximizing `min_i x_i(d)`, leximin refined [kalai1977].

```
egalitarian_index(U: np.ndarray, tau: np.ndarray) -> tuple[int,
    tuple[int, ...], str]
```

Defined in `interlens.arena.negotiation.solutions` · source

NOT scale invariant – it presupposes interpersonal utility comparability (in the title of the paper), so on arbitrary private scales the raw egalitarian point is meaningless across parties; on normalized surpluses `x_i/b_i` it collapses into KS. Only interpret it when the sheets share a scale by design (e.g. a common 0-100 budget).

Parameters

Name	Type	Default	Description
`U`	`np.ndarray`	*required*	
`tau`	`np.ndarray`	*required*	

egalitarian_welfare

Egalitarian (Rawlsian) welfare `ESW = min x_i` — “is any single party’s interest being ignored”.

```
egalitarian_welfare(x: 'Sequence[float]') -> float
```

Defined in `interlens.arena.negotiation.solutions` · source

The quantity discrete-KS maximizes, up to per-party normalization. `0.0` for an empty vector.

Parameters

Name	Type	Default	Description
x	`'Sequence[float]'`	*required*	

gini

Gini coefficient of the surplus distribution (0 = perfectly equal, →1 = maximally unequal).

```
gini(x: 'Sequence[float]', *, shift_negative: bool = False) -> float
```

Defined in `interlens.arena.negotiation.solutions` · source

Mean-absolute-difference form `G = sum_ij |x_i - x_j| / (2 n sum_k x_k)`, which is the standard Gini and is algebraically identical to the sorted-rank form on non-negative data.

Gini is only defined for a non-negative distribution with positive total, and a surplus vector can be negative when a party accepted below its threshold (an IR violation). `shift_negative` selects the policy for that case, because the two callers want different things and the choice changes the number:

- `False` (default) — report `nan` when the total is non-positive, and take negatives at face value. "Undefined" is surfaced as undefined rather than silently reported as perfect equality. This is the analysis-layer policy.
- `True` — translate the vector so its minimum is at 0 first, and return `0.0` for a non-positive total. Always yields a number, at the cost of measuring the *shifted* distribution's inequality. This is the per-episode outcome policy.

The two agree exactly whenever every surplus is non-negative.

Parameters

Name	Type	Default	Description
`x`	`'Sequence[float]'`	*required*	
`shift_negative`	`bool`	`False`	

ideal_surplus

The ideal-point surplus vector `b` of shape `(n,)`: `b_i` = the largest surplus party `i` attains, over the IR set (`restrict_ir=True`) or over all deals.

```
ideal_surplus(U: np.ndarray, tau: np.ndarray, restrict_ir: bool = True) -
    > np.ndarray
```

Defined in `interlens.arena.negotiation.solutions` · source

This is the normalizer for KS. If the IR set is empty, falls back to the max over all deals.

Parameters

Name	Type	Default	Description
`U`	`np.ndarray`	*required*	
`tau`	`np.ndarray`	*required*	
`restrict_ir`	`bool`	`True`	

ir_mask

Boolean `(|D|,)` mask of individually rational (acceptable) deals: every party's surplus is `⩾ 0` (`strict=True` requires `> 0`, the domain of the product solutions).

```
ir_mask(U: np.ndarray, tau: np.ndarray, strict: bool = False) ->
    np.ndarray
```

Defined in `interlens.arena.negotiation.solutions` · source

An empty mask means "no deal" is the rational outcome (Nash 1950 p.158 – the disagreement point) [nash1950].

Parameters

Name	Type	Default	Description
`U`	`np.ndarray`	*required*	
`tau`	`np.ndarray`	*required*	
`strict`	`bool`	`False`	

kalai_smorodinsky_index

Discrete **Kalai-Smorodinsky solution**: the Pareto-optimal IR deal maximizing the minimum normalized surplus `min_i x_i(d)/b_i` (`b` = ideal point over IR), refined by leximin over the normalized surplus vector [ks1975].

```
kalai_smorodinsky_index(
    U: np.ndarray,
    tau: np.ndarray,
) -> tuple[int, tuple[int, ...], str]
```

Defined in `interlens.arena.negotiation.solutions` · source

Exactly scale invariant (`a_i` cancels in `x_i/b_i`). Original is 2-player: for `n>2` KS can miss Pareto optimality and no solution keeps all its axioms, so we take the Pareto-restricted max-min-normalized point with leximin ties [roth1979]; non-convex/finite cover [conley_wilkie1991]. Empty IR set falls back to Maximum Nash Welfare.

Parameters

Name	Type	Default	Description
`U`	`np.ndarray`	*required*	
`tau`	`np.ndarray`	*required*	

max_nash_welfare_index

Maximum Nash Welfare with the two-stage empty-product rule [caragiannis2019] (Def. 3.1 + Algorithm 1, pp.12:7-12:8): first find the largest number of parties that can be made simultaneously positive-surplus by a single deal, then, among deals achieving that count, maximize the product of the positive surpluses (via `sum log` over the satisfied parties).

```
max_nash_welfare_index(U: np.ndarray, tau: np.ndarray) -> tuple[int,
    tuple[int, ...], str]
```

Defined in `interlens.arena.negotiation.solutions` · source

When some deal clears every party's threshold this coincides exactly with the Nash Bargaining Solution; otherwise it is the least-bad diagnostic point for an empty strict-IR game. Scale-free [caragiannis2019] p.12:2.

Parameters

Name	Type	Default	Description
`U`	`np.ndarray`	*required*	
`tau`	`np.ndarray`	*required*	

nash_bargaining_index

Discrete **Nash Bargaining Solution**: `argmax_{d: x_i(d) > 0 for all i} prod_i x_i(d)`, computed as `argmax sum_i log x_i(d)` for overflow safety [nash1950] (solution statement p.159; the non-convex/finite axiomatization is [mariotti1998]; the symmetric n-player product is [harsanyi1963]).

```
nash_bargaining_index(U: np.ndarray, tau: np.ndarray) -> tuple[int,
    tuple[int, ...], str]
```

Defined in `interlens.arena.negotiation.solutions` · source

Exactly scale invariant – `u_i` → `a_i u_i + c_i` (with `tau_i` moved likewise) multiplies the product by `prod a_i` and leaves the argmax unchanged. If the strict-IR set is empty, falls back to Maximum Nash Welfare (:func:`max_nash_welfare_index`).

Parameters

Name	Type	Default	Description
`U`	`np.ndarray`	*required*	
`tau`	`np.ndarray`	*required*	

nash_geomean

Geometric mean of surpluses `NSW^(1/n)`, in surplus units (`0.0` if any party is non-positive, per the Nash convention above).

```
nash_geomean(x: 'Sequence[float]') -> float
```

Defined in `interlens.arena.negotiation.solutions` · source

The readable display form: raw `prod x_i` explodes with `n` (a 6-party game runs to 1e10) while the geometric mean stays on the same scale as USW/ESW, so cells compare at a glance.

Parameters

Name	Type	Default	Description
x	`'Sequence[float]'`	*required*	

nash_welfare

Nash social welfare `NSW = prod x_i` when every surplus is strictly positive, else `0.0`.

```
nash_welfare(x: 'Sequence[float]') -> float
```

Defined in `interlens.arena.negotiation.solutions` · source

The Nash objective is `argmax prod x_i` over strictly-IR deals, so a party at or below its threshold is by convention outside the Nash set and contributes zero rather than a negative or spurious product — matching the discrete-NBS convention in :func:`max_nash_welfare_index`.

Parameters

Name	Type	Default	Description
x	`'Sequence[float]'`	*required*	

normalized_surplus

Per-deal nonnegative normalized surplus `clip(x_i, 0) / b_i` (`b_i` = ideal over all deals), the scale-invariant coordinate the distance metrics live in.

```
normalized_surplus(U: np.ndarray, tau: np.ndarray) -> np.ndarray
```

Defined in `interlens.arena.negotiation.solutions` · source

Below-threshold “gains” are clipped to 0 so they do not count as progress.

Public because it is the ONE coordinate system in which deals are compared across parties on arbitrary private scales: the distance metrics below and the frontier visualizer (`arena.viz`) must plot in the same space, so they share this function rather than each re-deriving a normalizer.

Parameters

Name	Type	Default	Description
`U`	`np.ndarray`	*required*	
`tau`	`np.ndarray`	*required*	

pareto_mask

Boolean `(|D|,)` mask of Pareto-optimal deals: deal `d` is on the frontier iff no other deal weakly dominates it on every party and strictly on at least one.

```
pareto_mask(U: np.ndarray) -> np.ndarray
```

Defined in `interlens.arena.negotiation.solutions` · source

Brute force `O(|D|^2 * n)` – milliseconds at |D|<=3125 [kung1975]. Deals with identical utility vectors do not dominate each other, so exact duplicates both stay on the frontier.

Parameters

Name	Type	Default	Description
`U`	`np.ndarray`	*required*	

utilitarian_index

Discrete **utilitarian solution**: the IR deal maximizing the surplus sum `sum_i x_i(d)` [harsanyi1955].

```
utilitarian_index(U: np.ndarray, tau: np.ndarray) -> tuple[int,
    tuple[int, ...], str]
```

Defined in `interlens.arena.negotiation.solutions` · source

NOT scale invariant – rescaling one party's sheet reweights the sum, so it is only meaningful on a shared or normalized scale (ANAC/GENIUS normalize to [0,1] first). The IR filter matters (a below-threshold party makes the sum meaningless); the `tau`-shift alone does not move the argmax.

Parameters

Name	Type	Default	Description
`U`	`np.ndarray`	*required*	
`tau`	`np.ndarray`	*required*	

welfare

Utilitarian social welfare `USW = sum x_i`.

```
welfare(x: 'Sequence[float]') -> float
```

Defined in `interlens.arena.negotiation.solutions` · source

NOT scale-invariant across parties — meaningful on normalized surpluses, and reported alongside ESW/NSW rather than alone [reproB_tmlr].

Parameters

Name	Type	Default	Description
x	`'Sequence[float]'`	*required*	

space

Module `interlens.arena.negotiation.space`

The deal space: issues, their discrete options, and the fully-enumerable Cartesian product of options.

A negotiation is over `J` issues, each with a fixed list of discrete options. A **deal** picks one option per issue and is represented as a `Deal` – a `tuple[int, ...]` of option indices, one per issue (the frozen cross-team contract). The whole deal space `D = prod_j |options_j|` is small enough (|D| ~ 243-3125 in the target regime) to enumerate exactly, which is what makes every normative benchmark in `solutions.py` exact rather than sampled.

Enumeration order is `itertools.product` order: issue 0 is most significant and the last issue varies fastest. `deal_at`/`index_of` are the mixed-radix encode/decode for that same order, so a deal's position in `enumerate()` equals its row in the `|D| x n` utility matrix that `sheets.utility_matrix` builds – the NumPy workhorse the solution concepts consume.

Example:

```
space = DealSpace((Issue("Site", ("North", "South")), Issue("Fund",
     ("None", "1M", "5M"))))
space.size                       # 6
list(space.enumerate())          # [(0,0),(0,1),(0,2),(1,0),(1,1),(1,2)]
space.index_of((1, 2))           # 5
space.deal_at(5)                 # (1, 2)
space.named((1, 2))              # {'Site': 'South', 'Fund': '5M'}
```

Classes

Name	Summary
`DealSpace`	The Cartesian product of all issues' options – the full, enumerable set of possible deals.

Name	Summary
`Issue`	One negotiable issue: a name and its discrete options (order defines the option indices).

DealSpace

The Cartesian product of all issues' options – the full, enumerable set of possible deals.

```
DealSpace(issues: tuple[Issue, ...])
```

Defined in `interlens.arena.negotiation.space` · source

Frozen and hashable so it can back a cached utility matrix. All indexing (`enumerate`, `deal_at`, `index_of`) uses one consistent mixed-radix order (issue 0 most significant, last issue fastest), matching `itertools.product` and the row order of the utility matrix.

Parameters

Name	Type	Default	Description
`issues`	tuple[Issue, …]	*required*	

Attributes

Name	Type	Summary
`issues`	tuple[Issue, …]	
`n_issues`	`int`	Number of issues `J`.
`shape`	`tuple[int, ...]`	Per-issue option counts `(|options_0|, ..., |options_{J-1}|)` – the mixed-radix base.
`size`	`int`	Total number of deals `|D| = prod_j |options_j|`.

Methods

deal_at

```
deal_at(self, index: int) -> Deal
```

source

The deal at flat position `index` in :meth:`enumerate` order (mixed-radix decode).

Parameters

Name	Type	Default	Description
index	int	*required*	

deals

```
deals(self) -> list[Deal]
```

source

All deals as a list (convenience wrapper over :meth:`enumerate`).

enumerate

```
enumerate(self) -> Iterator[Deal]
```

source

Iterate all `|D|` deals in mixed-radix order (issue 0 most significant, last issue fastest).

from_json

```
from_json(d: dict) -> 'DealSpace'
```

source

Rebuild a `DealSpace` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

index_of

```
index_of(self, deal: Deal) -> int
```

source

The flat position of `deal` in :meth:`enumerate` order (mixed-radix encode); inverse of :meth:`deal_at`.

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	

named

```
named(self, deal: Deal) -> dict[str, str]
```

source

Human-readable `{issue_name: option_label}` view of a deal – for solutions, transcripts, and audits (never fed back into numeric scoring).

Parameters

Name	Type	Default	Description
`deal`	`Deal`	*required*	

option_index

```
option_index(self, issue: int | str, label: str) -> int
```

source

Index of an option `label` within an issue (given by position or name), tolerant of surrounding whitespace and case on both.

Raises `ValueError` if the label matches no option.

Parameters

Name	Type	Default	Description
`issue`	`int | str`	*required*	
`label`	`str`	*required*	

parse

```
parse(self, named: dict[str, str]) -> Deal
```

source

Map a `{issue_name: option_label}` proposal dict to a :data:`Deal` (option index per issue, in issue order), tolerant of case/whitespace on both issue names and option labels – the inverse of :meth:`named`.

The dict must name every issue exactly once. Raises `ValueError` on a missing/unknown/duplicate issue or an unknown option, so a scenario can turn a malformed model proposal into a clean parse error to log.

Parameters

Name	Type	Default	Description
`named`	`dict[str, str]`	*required*	

strides

```
strides(self) -> tuple[int, ...]
```

source

Mixed-radix strides: `strides[j] = prod(shape[j+1:])` = the number of consecutive deals for which issue `j`'s option is held fixed under :meth:`enumerate`.

Used to map deal <-> flat index and to build the utility matrix without materializing every deal.

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict of the whole space.

Issue

One negotiable issue: a name and its discrete options (order defines the option indices).

```
Issue(name: str, options: tuple[str, ...])
```

Defined in `interlens.arena.negotiation.space` · source

`options` are human-readable labels (e.g. site names, funding tiers); a deal never stores these, only the integer index into this tuple, so option semantics stay out of the numeric game.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`options`	`tuple[str, ...]`	*required*	

Attributes

Name	Type	Summary
`n_options`	`int`	Number of options for this issue.
`name`	`str`	
`options`	`tuple[str, ...]`	

Methods

`from_json`

```
from_json(d: dict) -> 'Issue'
```

source

Rebuild an `Issue` from :meth:`to_json` output.

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

to_json

```
to_json(self) -> dict
```

source

JSON-ready dict (`options` as a list).

strategies

Module `interlens.arena.negotiation.strategies`

The executable rational / scripted negotiator zoo as **policies** (`state → action`), the computable opponent pool the LLMs are measured against.

Concession curves — Faratin, Sierra & Jennings, "Negotiation decision functions for autonomous agents," Robotics and Autonomous Systems 24(3-4):159-182, 1998, §3.1: time-dependent tactic `alpha(t) = k + (1 - k) * (t / T)^{1/beta}` with **Boulware** `beta < 1` (concede near the deadline) and **Conceder** `beta > 1` (concede early); utility-space restatement Baarslag thesis, TU Delft 2014, §2.3.3.

MiCRO — de Jonge, IJCAI 2022, pp. 223-229 (multilateral extension arXiv:2510.17401): sort own outcomes descending; with `m` distinct offers made and `n_min` the minimum distinct-offer count across opponents, concede one new outcome iff `m ⩽ n_min` else repeat; accept iff incoming >= the next offer you'd make. Parameter-free, ordinal-only.

Tit-for-tat — Faratin §3.3 behavior-dependent tactic (reproduce the opponent's concession). Tough/Hardliner — always demand the own optimum. Acceptance conditions AC_next / AC_const / AC_time / AC_combi — Baarslag, Hindriks & Jonker, "Acceptance Conditions in Automated Negotiation," SCI 435, 2013, eqs. (4.4)-(4.8).

`BayesianRationalPolicy` composes the belief + acceptance + best-response oracles — the headline rational agent: update beliefs from observed offers, best-respond on proposals, accept by optimal stopping.

All policies return typed actions (`Propose`/`Accept`/`Reject`/`Walk`) and read a `NegotiationState`, so a `PolicyParticipant` wrapping any of them is an interchangeable seat with an LLM participant.

Attributes

Name	Type	Summary
`ZOO`		

Classes

Name	Summary
`ACCombi`	AC_combi(T, alpha): `AC_next OR (AC_time(T) AND incoming ⩾ alpha)` (eq. 4.8; combi variants empirically dominate).
`ACConst`	AC_const(alpha): accept iff `incoming ⩾ alpha` (eq. 4.6).
`ACNext`	AC_next(alpha, beta): accept iff `alpha * incoming + beta ⩾ planned_next` — the incoming bid is at least as good as what you were about to send (Baarslag eq. 4.4; alpha=1, beta=0 standard).
`ACTime`	AC_time(T): accept anything once the time fraction reaches `t_frac` (eq. 4.7).
`AcceptanceCondition`	Decide whether to accept the standing offer given the agent's own utility of it (`incoming`) and of the deal it is about to propose (`planned_next`), both on the normalized `[0, 1]` scale.

Name	Summary
`BayesianRationalPolicy`	The composed rational negotiator = belief oracle + acceptance oracle + best-response oracle.
`DeclaredGreedyHoldoutPolicy`	:class:`GreedyHoldoutPolicy` that ANNOUNCES its ultimatum on its first turn and then never speaks again.
`DemandFractionPolicy`	Declare a demand LEVEL and hold to it: accept any package worth at least `accept_frac` of this seat's own MAXIMUM achievable score, propose its own best deal, and announce the numeric rule up front.
`FairnessOraclePolicy`	Omniscient **fairness oracle**: proposes and votes to maximize the table's normalized Nash welfare.
`FairnessRationalPolicy`	Private-information **fairness algorithmic** agent: the same welfare objective, estimated under its Bayesian posterior over the other seats' hidden sheets and thresholds.
`GreedyAnchorPolicy`	Maximally selfish proposals plus the same reservation gate: always tables its OWN best deal `argmax_d u_self(d)` (canonical tie-break, tabled once and held), and accepts any standing offer with surplus >= 0, declining below.
`GreedyHoldoutPolicy`	Take it or leave it: proposes its own-max deal exactly as :class:`GreedyAnchorPolicy` does,

Name	Summary
	but accepts ONLY that deal — every other offer is declined, including offers that are individually rational for it.
`MiCROPolicy`	MiCRO (de Jonge 2022; multilateral variant arXiv:2510.17401): minimal-concession, parameter-free.
`NaiveTitForTatPolicy`	Behavior-dependent tit-for-tat (Faratin §3.3): mirror the opponent's most recent concession (measured in this agent's own normalized utility) as an equal concession from the agent's last demand; start near the own optimum.
`NegotiationState`	The structured state a `Policy` reads to compute its next action — the machine-readable counterpart of the text `view` an LLM seat reads, so a `PolicyParticipant` and an LLM participant are interchangeable seats.
`PassiveGatePolicy`	Pure veto discipline, zero strategy: NEVER proposes; accepts any standing offer that clears its own reservation (surplus >= 0) and declines everything below it, on ordinary turns and on the terminal vote alike.
`Policy`	A deterministic (or seeded) negotiation policy: `policy(state) → action`.
`TimeDependentPolicy`	Faratin time-dependent tactic: concede own utility along `alpha(t)`

Name	Summary
	`= k + (1 - k) (t/T)^{1/beta}` toward the reservation, propose the least-concession IR deal at/above the current target, and accept per `acceptance`.
`ToughPolicy`	Hardliner: always demand the own optimum; accept only offers within `accept_frac` of the own max (and above reservation).

Functions

Name	Summary
`fit_belief`	A :class:`~interlens.arena.negotiation.beliefs.BeliefOracle` fitted to everything `state` publicly reveals about the other seats — one posterior per opponent, updated from their observed offers.
`parse_negotiation_state`	The scenario-emitted `negotiation_state` block in `text` (the inner dict of the last fenced JSON object carrying that key), or `None`.

ACCombi

AC_combi(T, alpha): `AC_next OR (AC_time(T) AND incoming ⩾ alpha)` (eq. 4.8; combi variants empirically dominate).

```
ACCombi(t_frac: float = 0.9, alpha: float = 0.6, next_alpha: float = 1.0)
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: AcceptanceCondition

Parameters

Name	Type	Default	Description
t_frac	float	0.9	
alpha	float	0.6	
next_alpha	float	1.0	

Attributes

Name	Type	Summary
alpha		
next		
time		

Methods

accepts

```
accepts(self, state, incoming, planned_next)
```

source

Parameters

Name	Type	Default	Description
`state`		*required*	
`incoming`		*required*	
`planned_next`		*required*	

ACConst

AC_const(alpha): accept iff `incoming ⩾ alpha` (eq. 4.6).

```
ACConst(alpha: float = 0.7)
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: AcceptanceCondition

Parameters

Name	Type	Default	Description
alpha	float	0.7	

Attributes

Name	Type	Summary
alpha		

Methods

accepts

```
accepts(self, state, incoming, planned_next)
```

source

Parameters

Name	Type	Default	Description
state		*required*	
incoming		*required*	

Name	Type	Default	Description
`planned_next`		*required*	

ACNext

AC_next(alpha, beta): accept iff `alpha * incoming + beta ⩾ planned_next` — the incoming bid is at least as good as what you were about to send (Baarslag eq. 4.4; alpha=1, beta=0 standard).

```
ACNext(alpha: float = 1.0, beta: float = 0.0)
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: AcceptanceCondition

Parameters

Name	Type	Default	Description
alpha	float	1.0	
beta	float	0.0	

Methods

accepts

```
accepts(self, state, incoming, planned_next)
```

source

Parameters

Name	Type	Default	Description
state		*required*	
incoming		*required*	
planned_next		*required*	

ACTime

AC_time(T): accept anything once the time fraction reaches `t_frac` (eq. 4.7).

```
ACTime(t_frac: float = 0.9)
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: AcceptanceCondition

Parameters

Name	Type	Default	Description
`t_frac`	`float`	`0.9`	

Attributes

Name	Type	Summary
`t_frac`		

Methods

`accepts`

```
accepts(self, state, incoming, planned_next)
```

source

Parameters

Name	Type	Default	Description
`state`		*required*	
`incoming`		*required*	

Name	Type	Default	Description
`planned_next`		*required*	

AcceptanceCondition

Decide whether to accept the standing offer given the agent's own utility of it (`incoming`) and of the deal it is about to propose (`planned_next`), both on the normalized `[0, 1]` scale.

```
AcceptanceCondition()
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: `ABC`

Methods

accepts

```
accepts(self, state: NegotiationState, incoming: float, planned_next:
    float) -> bool
```

source

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	
`incoming`	`float`	*required*	
`planned_next`	`float`	*required*	

BayesianRationalPolicy

The composed rational negotiator = belief oracle + acceptance oracle + best-response oracle.

```
BayesianRationalPolicy(
    *,
    discount: float | None = None,
    walk_if_hopeless: bool = True,
    name: str = 'bayes-rational',
)
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: Policy

Each turn: (1) update beliefs over opponents from observed offers (private info) or read them off known sheets (full info); (2) build the opponent acceptance-probability table; (3) accept the standing offer if its surplus clears the optimal-stopping reservation, else propose the best-response deal; (4) walk if no individually-rational deal can plausibly close before the deadline.

Parameters

discount : float | None Per-round discount `delta` OVERRIDE for the acceptance and best-response oracles. Default `None` = use `state.discount` (which scenario-runner sets from the game's `discount`/`breakdown_risk`) — the state is a policy's single source of truth, analogous to the game for an oracle. walk_if_hopeless : bool If True, `Walk` when the best-response proposal value is <= 0 at the final round. name : str Display name.

Parameters

Name	Type	Default	Description
`discount`	`float | None`	`None`	
`walk_if_hopeless`	`bool`	`True`	
`name`	`str`	`'bayes-rational'`	

Attributes

Name	Type	Summary
`discount`		
`name`		
`walk_if_hopeless`		

Methods

act

```
act(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

vote

```
vote(self, state: NegotiationState)
```

source

Terminal quorum-aware vote; Reject when yes cannot pass or has lower EV than no.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

DeclaredGreedyHoldoutPolicy

:class:`GreedyHoldoutPolicy` that ANNOUNCES its ultimatum on its first turn and then never speaks again.

```
DeclaredGreedyHoldoutPolicy()
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: GreedyHoldoutPolicy

Behaviourally identical to the silent version — same proposals, same acceptance rule, same terminal vote — so any difference in what the table does is attributable to the declaration alone. The capitulation prior from the program's sycophancy work predicts the announcement makes LLM seats cave harder: a partner who states an immovable position is exactly the pressure those models fold to.

Attributes

Name	Type	Summary
`name`		

Methods

`declaration`

```
declaration(self, state: NegotiationState) -> str
```

source

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

DemandFractionPolicy

Declare a demand LEVEL and hold to it: accept any package worth at least `accept_frac` of this seat's own MAXIMUM achievable score, propose its own best deal, and announce the numeric rule up front.

```
DemandFractionPolicy(*, accept_frac: float = 0.9, name: str | None = None)
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: GreedyAnchorPolicy

The bar is a fraction of the everything-goes-my-way total, not of the reservation value — so at the default 0.9 the policy is asking for nine tenths of the best it could conceivably do, while still leaving a visible margin for the other parties to work in. It is the middle rung between :class:`GreedyAnchorPolicy` (signs anything above its threshold) and :class:`GreedyHoldoutPolicy` (signs only its maximum): a commitment WITH a margin, testing whether a slightly softer ultimatum extracts nearly as much while keeping deals alive.

The declared number is this seat's own private score information, voluntarily disclosed. That is deliberate and is the mechanism under test — a commitment is only credible if the counterparty can check an offer against it — but it does mean this policy is not information-symmetric with the others, and a private-information game is no longer fully private on this seat once it speaks.

Parameters

accept_frac : float Fraction of the own-maximum score demanded, in `(0, 1]`. 1.0 degenerates to holdout-by-value. name : str Display name.

Parameters

Name	Type	Default	Description
`accept_frac`	`float`	`0.9`	
`name`	`str | None`	`None`	

Attributes

Name	Type	Summary
accept_frac		
name		

Methods

act

```
act(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
state	Negotiation-State	*required*	

declaration

```
declaration(self, state: NegotiationState) -> str
```

source

Parameters

Name	Type	Default	Description
state	Negotiation-State	*required*	

vote

```
vote(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
state	Negotiation-State	*required*	

FairnessOraclePolicy

Omniscient **fairness oracle**: proposes and votes to maximize the table's normalized Nash welfare.

```
FairnessOraclePolicy(
    *,
    discount: float | None = None,
    walk_if_hopeless: bool = True,
    name: str = 'fairness-oracle',
)
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: `_TableObjectivePolicy`

Reads every party's sheet (it requires the full-information `state.tables`) and substitutes :func:`~interlens.arena.negotiation.fairness.mnw_objective` for its own surplus column, so:

- its **proposal** is the deal maximizing expected table welfare given who will accept — which on a game where some deal clears every threshold is exactly the discrete Nash Bargaining / Maximum Nash Welfare point, and with no acceptance uncertainty (its own table full of IR indicators) is that point outright;
- its **acceptance** runs the same optimal-stopping recursion in welfare units: take the standing offer iff the welfare it delivers is at least the welfare the oracle expects to reach by continuing;
- it still refuses to sign below its own threshold — on all three branches: accepting (:meth:`BayesianRationalPolicy.act`), the terminal vote, and *proposing* (:meth:`_TableObjectivePolicy._pick_proposal`, where the objective's `clip(u - tau, 0)` leaves it indifferent to its own losses and so cannot supply the refusal itself).

With no other seats to persuade this degenerates to the utilitarian planner's choice on the welfare objective — it simply names the fairest feasible deal — which is the sense in which it is a "self-assembling mediator" rather than a negotiator.

Falls back to its own surplus (i.e. behaves as the ordinary Bayesian agent) if seated in a game with no full-information tables, since table welfare is not computable from one sheet; use :class:`FairnessRationalPolicy` for the private-information version instead of relying on that fallback.

Parameters

Name	Type	Default	Description
`discount`	`float | None`	`None`	
`walk_if_hopeless`	`bool`	`True`	
`name`	`str`	`'fairness-oracle'`	

Attributes

Name	Type	Summary
`name`		

FairnessRationalPolicy

Private-information **fairness algorithmic** agent: the same welfare objective, estimated under its Bayesian posterior over the other seats' hidden sheets and thresholds.

```
FairnessRationalPolicy(
    *,
    discount: float | None = None,
    walk_if_hopeless: bool = True,
    name: str = 'fairness-rational',
)
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: `_TableObjectivePolicy`

Where :class:`FairnessOraclePolicy` reads the opponents' normalized surpluses, this reads their *posterior-expected* normalized surpluses off the same opponent-type grid the ordinary rational agent uses for acceptance probabilities (:func:`fit_belief`, then :meth:`~interlens.arena.negotiation.beliefs.BeliefState.expected_normalize and its own column exactly. So the two agents differ ONLY in what they know, which is what makes their gap a clean measurement of the price of private information for a fairness-seeker.

The estimate is a plug-in `obj(E[z])` rather than `E[obj(z)]` and is therefore optimistic by a Jensen gap (see :mod:`~interlens.arena.negotiation.fairness`); combined with a posterior that only sees public offers, it can target a deal that is not in fact the table's welfare maximizer. That is a substantive prediction about this agent, not an implementation shortcut.

Uses the exact objective when it happens to be seated in a full-information game, so the two policies coincide there by construction.

Parameters

Name	Type	Default	Description
`discount`	`float | None`	`None`	
`walk_if_hopeless`	`bool`	`True`	

Name	Type	Default	Description
name	str	'fairness-rational'	

Attributes

Name	Type	Summary
name		

GreedyAnchorPolicy

Maximally selfish proposals plus the same reservation gate: always tables its OWN best deal `argmax_d u_self(d)` (canonical tie-break, tabled once and held), and accepts any standing offer with surplus >= 0, declining below.

```
GreedyAnchorPolicy()
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: Policy

Against :class:`BayesianRationalPolicy` this isolates the proposal rule: the Bayesian agent best-responds against a model of what opponents will accept and therefore proposes GENEROUSLY, while this one never concedes an inch on its own ask yet is just as willing to sign anything that beats no-deal. If the seat captures MORE here than under the Bayesian anchor, the anchor's proposals were leaving surplus on the table out of opponent-model conservatism.

Attributes

Name	Type	Summary
`name`		

Methods

act

```
act(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

GreedyHoldoutPolicy

Take it or leave it: proposes its own-max deal exactly as :class:`GreedyAnchorPolicy` does, but accepts ONLY that deal — every other offer is declined, including offers that are individually rational for it.

```
GreedyHoldoutPolicy()
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: GreedyAnchorPolicy

This is the one policy here that is not individually rational in the game-theoretic sense: it walks away from free surplus, so it should cost itself deals. It is the extraction upper bound — the test of whether an agreeable LLM table simply capitulates to a seat that never moves. On the terminal forced vote it applies the same rule rather than the base class's accept-anything-positive vote, which is the whole point: the ordinary rational agent's last-round logic (any deal beats no deal) is exactly what this policy refuses.

Attributes

Name	Type	Summary
`name`		

Methods

act

```
act(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

`vote`

```
vote(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

MiCROPolicy

MiCRO (de Jonge 2022; multilateral variant arXiv:2510.17401): minimal-concession, parameter-free.

```
MiCROPolicy(*, seed: int = 0, name: str = 'micro')
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: Policy

Concede one new outcome iff distinct-offers-made `m ⩽ n_min` (min distinct offers across opponents), else repeat a previous offer; accept iff the standing offer is at least as good as the next outcome you would propose.

Parameters

Name	Type	Default	Description
`seed`	`int`	`0`	
`name`	`str`	`'micro'`	

Attributes

Name	Type	Summary
`name`		

Methods

act

```
act(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

NaiveTitForTatPolicy

Behavior-dependent tit-for-tat (Faratin §3.3): mirror the opponent's most recent concession (measured in this agent's own normalized utility) as an equal concession from the agent's last demand; start near the own optimum.

```
NaiveTitForTatPolicy(
    *,
    acceptance: AcceptanceCondition | None = None,
    name: str = 'naive-tft',
)
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: Policy

Accept per `acceptance`.

Parameters

Name	Type	Default	Description
`acceptance`	AcceptanceCondition | None	`None`	
`name`	`str`	`'naive-tft'`	

Attributes

Name	Type	Summary
`acceptance`		
`name`		

Methods

act

```
act(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
state	Negotiation-State	*required*	

NegotiationState

The structured state a `Policy` reads to compute its next action — the machine-readable counterpart of the text `view` an LLM seat reads, so a `PolicyParticipant` and an LLM participant are interchangeable seats.

```
NegotiationState(
    seat: int,
    sheet: ScoreSheet,
    space: DealSpace,
    round: int = 1,
    deadline: int = 1,
    offers: dict = dict(),
    standing: str | None = None,
    received: list = list(),
    received_by_opponent: dict = dict(),
    my_offers: list = list(),
    discount: float = 1.0,
    tables: GameTables | None = None,
    opponents: tuple = (),
    must_vote: bool = False,
    min_accept: int | None = None,
    veto_seats: tuple = (),
    offer_proposers: dict = dict(),
    offer_accepts: dict = dict(),
    offer_rejects: dict = dict(),
    walked_seats: tuple = (),
)
```

Defined in `interlens.arena.negotiation.strategies` · source

Attributes

seat : int This policy's seat index. sheet : ScoreSheet This seat's private score sheet. space : DealSpace The shared deal space. round : int Current round (1-indexed). deadline : int Total number of rounds `T` (turn-count deadline, restated every turn). offers : dict[str, Deal] Live offer registry: `offer_id` → `deal`. standing : str | None The offer id this seat is being asked to respond to (most recent live offer), if any. received : list[Deal] Opponent-proposed deals in order (feeds MiCRO / tit-for-tat / belief updates). received_by_opponent : dict[int, list[Deal]] The same public opponent offers, preserving proposer seat identity. Bayesian opponent modelling uses this mapping; `received` remains the pooled compatibility view for policies that do not. my_offers :

list[Deal] This seat's own past proposals in order. discount : float Per-round discount / breakdown-risk `delta` (1.0 = none). tables : GameTables | None Optional cached tables for the full game (only available under full information). opponents : tuple[int, …] Seat indices of the other parties. must_vote : bool True on a vote-only turn (the scenario's forced-final phase): the seat may ONLY accept/reject/walk the standing offer, not propose. Policies read this and cast a terminal individually-rational vote (accept any offer that clears their threshold, since the only alternative is no-deal = 0). Proposing here is an economic-legality violation, so a proposing policy would otherwise blow the deal. min_accept : int | None Fixed number of original seats whose yes votes pass a deal. `None` preserves unanimity. veto_seats : tuple[int, …] Seats whose yes votes are required in addition to the numeric quorum. offer_proposers / offer_accepts / offer_rejects : dict Public offer-ledger metadata keyed by offer id, with seat indices as values. This lets quorum-aware policies distinguish pivotal from non-pivotal votes.

Parameters

Name	Type	Default	Description
`seat`	`int`	*required*	
`sheet`	ScoreSheet	*required*	
`space`	DealSpace	*required*	
`round`	`int`	`1`	
`deadline`	`int`	`1`	
`offers`	`dict`	`dict()`	
`standing`	`str | None`	`None`	
`received`	`list`	`list()`	
`received_by_opponent`	`dict`	`dict()`	
`my_offers`	`list`	`list()`	
`discount`	`float`	`1.0`	
`tables`	GameTables | None	`None`	

Name	Type	Default	Description
opponents	tuple	()	
must_vote	bool	False	
min_accept	int | None	None	
veto_seats	tuple	()	
offer_proposers	dict	dict()	
offer_accepts	dict	dict()	
offer_rejects	dict	dict()	
walked_seats	tuple	()	

Attributes

Name	Type	Summary
deadline	int	
discount	float	
final_proposal	bool	Whether this turn is the forced-final PROPOSAL turn, where only propose/accept/walk are legal.
min_accept	int | None	
must_vote	bool	
my_offers	list	
offer_accepts	dict	
offer_proposers	dict	
offer_rejects	dict	
offers	dict	
opponents	tuple	
received	list	
received_by_opponent	dict	

Name	Type	Summary
`round`	`int`	
`seat`	`int`	
`sheet`	ScoreSheet	
`space`	DealSpace	
`standing`	`str | None`	
`standing_deal`	`Deal | None`	The deal referenced by `standing` (or None).
`tables`	GameTables | None	
`time_fraction`	`float`	`(round-1)/deadline` in `[0, 1)` — the `t` used by time-dependent concession curves.
`veto_seats`	`tuple`	
`walked_seats`	`tuple`	

Methods

`from_block`

```
from_block(
    cls,
    block: dict,
    *,
    sheet,
    space,
    tables=None,
    discount: float = 1.0,
    opponents: tuple = (),
    seat: int | None = None,
) -> 'NegotiationState'
```

source

Build a state from a scenario-emitted `negotiation_state` block (see `parse_negotiation_state`) plus the seat-bound context (`sheet`/`space`/`tables`/`discount`/`opponents`).

The block carries only the dynamic fields — `seat`, `round`, `deadline`, `offers` (`{id: [opt, ...]}`), `standing` (id or null), `received`/`my_offers` (lists of deals), and `received_by_opponent` (`{seat_index: [deal, ...]}`) — so a `PolicyParticipant` can read the scenario's authoritative offer registry straight from its view.

Parameters

Name	Type	Default	Description
`block`	`dict`	*required*	
`sheet`		*required*	
`space`		*required*	
`tables`		`None`	
`discount`	`float`	`1.0`	
`opponents`	`tuple`	`()`	
`seat`	`int | None`	`None`	

PassiveGatePolicy

Pure veto discipline, zero strategy: NEVER proposes; accepts any standing offer that clears its own reservation (surplus >= 0) and declines everything below it, on ordinary turns and on the terminal vote alike.

```
PassiveGatePolicy()
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: Policy

This is the anchor stripped down to the one thing every rational agent does — refusing to sign a deal that is worse than no deal. It contributes no deals of its own, so anything it changes about a table's outcome is the discipline channel and nothing else. Needs a chat-enabled arm (it emits :class:`~interlens.arena.actions.Pass` when there is nothing to respond to).

Attributes

Name	Type	Summary
`name`		
`vote`		

Methods

`act`

```
act(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

Policy

A deterministic (or seeded) negotiation policy: `policy(state) → action`.

```
Policy()
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: `ABC`

Subclasses set `name` and implement `act`. `__call__` is the invocation surface a `PolicyParticipant` binds to.

Attributes

Name	Type	Summary
`name`	`str`	

Methods

`act`

```
act(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

declaration

```
declaration(self, state: NegotiationState) -> str | None
```

source

Public cheap talk this policy says ONCE, on its first turn of the episode — or `None` (the default) for a silent policy.

This is the commitment channel. A policy's moves already reveal what it will do eventually; a declaration states it in plain language up front, before the other parties have made a move, which is what makes it a *commitment* rather than a pattern the opponents have to infer. `PolicyParticipant` decides when "first turn" is by reading the view (no prior turn of this seat's own) and attaches the string under the envelope's `message` key, so it is published exactly like an LLM seat's chat.

Render deals and scores in the same human terms the transcript uses — `state.space.named(deal)` for packages, raw sheet points for thresholds — since the audience is the LLM seats reading the log.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

vote

```
vote(self, state: NegotiationState)
```

source

The terminal individually-rational vote on the standing offer when the scenario allows only accept/reject/walk (`state.must_vote`).

Accept any offer that clears this seat's threshold (surplus >= 0), since the sole alternative is no-deal = 0; otherwise reject it (or walk if there is no

standing offer). Shared by every policy — proposing in a vote-only phase is an economic-legality violation, so no policy must ever fall through to a Propose here.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

TimeDependentPolicy

Faratin time-dependent tactic: concede own utility along `alpha(t) = k + (1 - k) (t/T)^{1/beta}` toward the reservation, propose the least-concession IR deal at/above the current target, and accept per `acceptance`.

```
TimeDependentPolicy(
    beta: float,
    *,
    k: float = 0.0,
    acceptance: AcceptanceCondition | None = None,
    name: str | None = None,
)
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: Policy

Parameters

beta : float Concession exponent. `beta < 1` = Boulware (concede near deadline); `beta > 1` = Conceder; `beta = 1` linear. k : float First-offer concession constant in `[0, 1]` (0 = open at the own optimum). acceptance : Acceptance-Condition Acceptance rule (default AC_next). name : str Display name.

Parameters

Name	Type	Default	Description
`beta`	`float`	*required*	
`k`	`float`	`0.0`	
`acceptance`	Acceptance-Condition | None	`None`	
`name`	`str | None`	`None`	

Attributes

Name	Type	Summary
`acceptance`		
`beta`		
`k`		
`name`		

Methods

`act`

```
act(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

`boulware`

```
boulware(cls, beta: float = 0.2, **kw={})
```

source

A Boulware agent (`beta < 1`; default 0.2).

Parameters

Name	Type	Default	Description
beta	float	0.2	
kw		{}	

conceder

```
conceder(cls, beta: float = 5.0, **kw={})
```

source

A Conceder agent (`beta > 1`; default 5.0).

Parameters

Name	Type	Default	Description
beta	float	5.0	
kw		{}	

concession

```
concession(self, t: float) -> float
```

source

The Faratin concession level `alpha(t) = k + (1 - k) t^{1/beta}` at time fraction `t` in `[0, 1]` (0 = demand the optimum, 1 = conceded to the reservation).

Parameters

Name	Type	Default	Description
t	float	*required*	

target_norm

```
target_norm(self, state) -> float
```

source

The normalized own-utility level to demand now: `1 - concession(t) * (1 - reserve)`.

Parameters

Name	Type	Default	Description
`state`		*required*	

ToughPolicy

Hardliner: always demand the own optimum; accept only offers within `accept_frac` of the own max (and above reservation).

```
ToughPolicy(*, accept_frac: float = 0.95, name: str = 'tough')
```

Defined in `interlens.arena.negotiation.strategies` · source

Inherits from: Policy

Parameters

Name	Type	Default	Description
`accept_frac`	`float`	`0.95`	
`name`	`str`	`'tough'`	

Attributes

Name	Type	Summary
`accept_frac`		
`name`		

Methods

act

```
act(self, state: NegotiationState)
```

source

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

fit_belief

A :class:`~interlens.arena.negotiation.beliefs.BeliefOracle` fitted to everything `state` publicly reveals about the other seats — one posterior per opponent, updated from their observed offers.

```
fit_belief(state: NegotiationState) -> BeliefOracle
```

Defined in `interlens.arena.negotiation.strategies` · source

Built FRESH per call rather than cached on a policy: it is fully determined by the offers in `state` and `update_from_offers` rebuilds it from scratch anyway, so persisting it would buy nothing while making a policy instance shared across concurrent seats race on mutable member state.

Module-level because two different consumers need the identical posterior — the acceptance-probability table (:meth:`BayesianRationalPolicy._accept_prob_table`) and the fairness objective's opponent columns (:meth:`FairnessRationalPolicy._objective`) — and a private-information agent whose two halves disagreed about what it believes would not be one agent.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

parse_negotiation_state

The scenario-emitted `negotiation_state` block in `text` (the inner dict of the last fenced JSON object carrying that key), or `None`.

```
parse_negotiation_state(text: str) -> dict | None
```

Defined in `interlens.arena.negotiation.strategies` · source

This is the authoritative structured-state channel a scenario embeds in a seat's view, so a `PolicyParticipant` reads canonical offer ids / round straight off it instead of reconstructing the ledger from the transcript; :meth:`NegotiationState.from_block` turns the result into a state. Reads through `parsing.last_json_with_key` — the library's one fenced-JSON reader.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

talking

Module `interlens.arena.negotiation.talking`

The **talking rational agent**: the composed Bayesian negotiator with a truthful templated voice.

`BayesianRationalPolicy` is mute — its beliefs are signal-starved under private information because the only evidence it ever emits or consumes is offer movement. This module gives the SAME decision rule a message channel, in a ladder of strictly increasing revelation, plus the inbound half (parsing counterparties' statements into belief-grid conditioning). The decision rule is UNCHANGED by construction: the speaking variants only add messages, and the listening variant changes only the belief posterior the existing rule is applied to.

The ladder (`TalkingBayesianPolicy.VARIANTS`):

- `commit` (T1) — once, on its first turn: its concession schedule ("I accept any package worth at least `bar(r)` to me; the bar falls to my floor by the deadline"), computed from the live optimal-stopping reservation curve under current beliefs.
- `narrate` (T2) — each turn with a standing offer: whether that offer clears its CURRENT reservation bar and by how much, in its own points and on its own min-max-normalized 0-1 scale.
- `hint` (T3) — narrate + once, on its first turn: its ordinal top option on each issue (tops only — no scores, no threshold).
- `listen` (T4) — commit + narrate + hint + parse other seats' `talking_rational` statements (verbatim from other talking seats; from LLM text via a strict JSON convention it requests in its first-turn message) into soft, trust-discounted belief-grid conditioning.

`BabbleBayesianPolicy` is the control: fluent, on-topic, length-matched, state-INDEPENDENT boilerplate every turn over the same mute decision rule — it separates "any speech helps" from "informative speech helps".

Honesty is a hard constraint, enforced by construction and gated offline: every emitted statement is a pure function of the policy's live state (reser-

vation curve, standing offer, own sheet), rendered by the same functions an offline gate re-runs on the recorded view (see the rational_agents experiment's `gate_talking_messages.py`). There is no bluffing arm here.

Message grammar: prose for the LLM audience plus one fenced `json` block per statement carrying `{"talking_rational": {payload}}`. The scenario republishes a seat's `message` as plain text, so the block is legible (and machine-parseable) in every other seat's view. :func:`statements_in` is the total parser — anything malformed is dropped and counted, never raised.

Attributes

Name	Type	Summary
`MESSAGE_KEY`		
`STATEMENT_KINDS`		

Classes

Name	Summary
`BabbleBayesianPolicy`	The babble control: the mute decision rule plus fluent, on-topic, state-INDEPENDENT boilerplate every turn.
`TalkingBayesianPolicy`	`BayesianRationalPolicy` with a truthful templated message channel and (optionally) inbound listening.
`TalkingParticipant`	A :class:`PolicyParticipant` for talking policies: it merges the one-time `declaration` with the bound policy's per-turn `commentary` into the turn's public message, and (for listening policies) parses every

Name	Summary
	`talking_rational` statement in the view into `state.statements` before the policy runs.

Functions

Name	Summary
`condition_on_commit`	Fold a declared normalized walk-away floor into the posterior over the grid's reservation levels: a Gaussian log-likelihood `-(tau - floor)^2 / (2 scale^2)` per type, tempered by `strength` and `lam`.
`condition_on_hint`	Fold declared per-issue top options into the posterior: types whose evaluator peaks at the declared option on that issue match the claim (likelihood `reliability`), the rest `1 - reliability`, tempered by `strength` and the state's own damping `lam`.
`condition_on_narration`	Fold "this package is above/below my bar" into the posterior — exactly the accept/reject evidence `observe_response` implements, with the claim's truth value playing the vote.
`statements_in`	Every well-formed talking-rational statement in `text`, plus the census the honesty audit needs.

BabbleBayesianPolicy

The babble control: the mute decision rule plus fluent, on-topic, state-INDEPENDENT boilerplate every turn.

```
BabbleBayesianPolicy(
    *,
    discount: float | None = None,
    walk_if_hopeless: bool = True,
    name: str = 'babble-bayes-rational',
    **kwargs={},
)
```

Defined in `interlens.arena.negotiation.talking` · source

Inherits from: BayesianRationalPolicy

Sentences are length-matched to the talking variants' narrations and rotate deterministically by round, so the channel is equally busy but carries zero information about this seat's state.

Parameters

Name	Type	Default	Description
`discount`	`float | None`	`None`	
`walk_if_hopeless`	`bool`	`True`	
`name`	`str`	`'babble-bayes-rational'`	
`kwargs`		`{}`	

Attributes

Name	Type	Summary
`BABBLE`		

Methods

commentary

```
commentary(self, state: NegotiationState) -> str | None
```

source

One boilerplate sentence per turn, a pure function of the round number.

Parameters

Name	Type	Default	Description
state	Negotiation-State	*required*	

TalkingBayesianPolicy

`BayesianRationalPolicy` with a truthful templated message channel and (optionally) inbound listening.

```
TalkingBayesianPolicy(
    *,
    commit: bool = False,
    narrate: bool = False,
    hint: bool = False,
    listen: bool = False,
    narrate_reliability: float = 0.85,
    narrate_strength: float = 0.6,
    hint_reliability: float = 0.85,
    hint_strength: float = 0.6,
    commit_tau_scale: float = 0.12,
    commit_strength: float = 0.6,
    discount: float | None = None,
    walk_if_hopeless: bool = True,
    name: str = 'talking-bayes-rational',
    **kwargs={},
)
```

Defined in `interlens.arena.negotiation.talking` · source

Inherits from: BayesianRationalPolicy

The decision rule is untouched: `act`/`vote` are inherited verbatim. Speaking variants add messages only; the listening variant overrides `_accept_prob_table` to condition the SAME belief grid on parsed counterparty statements before the inherited rule reads it. Actions are therefore byte-identical to the base policy given the same beliefs — the experiment's gate G1.

Parameters

commit : bool Emit the concession-schedule commitment on the first turn (T1 content): the optimal-stopping reservation bar per round under current beliefs, in own points and normalized units, falling to the walk-away floor. Labeled as the current plan — the live bar each turn remains authoritative. narrate : bool Emit, on every turn with a standing offer, whether that offer clears the current bar and by how much (T2 content). hint : bool Emit the per-issue ordinal top options on the first turn (T3 increment). No

scores, no threshold. listen : bool Parse other seats' statements (delivered by :class:`TalkingParticipant` on `state.statements`) into trust-discounted belief conditioning, and request the machine-readable convention from LLM seats in the first-turn message (T4 increment). narrate_reliability, narrate_strength : float Trust discount for inbound narrations (see :func:`condition_on_narration`). `reliability` is the probability a claim is truthful/myopically consistent; `strength` tempers it against self-authored evidence. hint_reliability, hint_strength : float Trust discount for inbound ordinal hints (:func:`condition_on_hint`). commit_tau_scale, commit_strength : float Softness of the inbound declared-floor conditioning (:func:`condition_on_commit`): `scale` is the Gaussian width on the [0, 1] threshold scale, `strength` the tempering. discount, walk_if_hopeless, name As in :class:`BayesianRationalPolicy`.

Parameters

Name	Type	Default	Description
`commit`	`bool`	`False`	
`narrate`	`bool`	`False`	
`hint`	`bool`	`False`	
`listen`	`bool`	`False`	
`narrate_reliability`	`float`	`0.85`	
`narrate_strength`	`float`	`0.6`	
`hint_reliability`	`float`	`0.85`	
`hint_strength`	`float`	`0.6`	
`commit_tau_scale`	`float`	`0.12`	
`commit_strength`	`float`	`0.6`	
`discount`	`float | None`	`None`	
`walk_if_hopeless`	`bool`	`True`	
`name`	`str`	`'talking-bayes-rational'`	
`kwargs`		`{}`	

Attributes

Name	Type	Summary
`VARIANTS`		
`commit`		
`commit_strength`		
`commit_tau_scale`		
`hint`		
`hint_reliability`		
`hint_strength`		
`listen`		
`narrate`		
`narrate_reliability`		
`narrate_strength`		

Methods

`commentary`

```
commentary(self, state: NegotiationState) -> str | None
```

source

The every-turn message (narration), or `None`. :class:`TalkingParticipant` appends it to the turn's public message on every turn, first and later alike.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

commit_message

```
commit_message(self, state: NegotiationState) -> str
```

source

The first-turn concession-schedule commitment (T1).

Parameters

Name	Type	Default	Description
state	Negotiation-State	*required*	

conditioned_belief_states

```
conditioned_belief_states(self, state: NegotiationState) -> dict
```

source

Per-opponent posteriors after folding in the parsed statements riding on `state.statements` (attached by :class:`TalkingParticipant`), in transcript order, each softly and trust-discounted.

Offers condition exactly as in the base policy (same `fit_belief`); statements about a seat's own valuation condition that seat's grid.

Parameters

Name	Type	Default	Description
state	Negotiation-State	*required*	

convention_message

```
convention_message(self, state: NegotiationState) -> str
```

source

The first-turn request (T4 increment) that language-model seats state their own constraints in the machine-readable convention this policy can actually consume.

Claims are trust-discounted, never taken as fact.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

current_bar

```
current_bar(self, state: NegotiationState) -> float
```

source

This turn's acceptance bar in own sheet points: the walk-away floor on a terminal vote (no continuation exists), else the optimal-stopping reservation at the rounds actually left — the exact `r_left` clamp `act` uses.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

declaration

```
declaration(self, state: NegotiationState) -> str | None
```

source

The one-time first-turn message: commitment schedule, ordinal tops, and/or the LLM convention request, per this variant's flags.

`PolicyParticipant` publishes it exactly like LLM chat.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

hint_message

```
hint_message(self, state: NegotiationState) -> str
```

source

The first-turn ordinal-tops hint (T3 increment).

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

issue_tops

```
issue_tops(self, state: NegotiationState) -> dict
```

source

`{issue_name: option_name}` — this seat's true ordinal top per issue, ties broken by first index (deterministic).

Read straight off the sheet's per-issue value rows.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

`narrate_message`

```
narrate_message(self, state: NegotiationState) -> str | None
```

source

This turn's standing-offer narration (T2), or `None` when there is nothing to narrate.

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

`reservation_schedule`

```
reservation_schedule(self, state: NegotiationState) -> list[tuple[int,
    float]]
```

source

`[(round, bar_points), ...]` for every round 1..deadline+1: the own-sheet score a package must reach for this seat to stop, under CURRENT beliefs — the same acceptance-oracle reservation `act` prices the standing offer against,

plus the walk-away floor at the forced final (where the terminal vote accepts anything individually rational).

Parameters

Name	Type	Default	Description
`state`	Negotiation-State	*required*	

TalkingParticipant

A :class:`PolicyParticipant` for talking policies: it merges the one-time `declaration` with the bound policy's per-turn `commentary` into the turn's public message, and (for listening policies) parses every `talking_rational` statement in the view into `state.statements` before the policy runs.

```
TalkingParticipant(*args=(), personas: tuple = (), **kwargs={})
```

Defined in `interlens.arena.negotiation.talking` · source

Inherits from: PolicyParticipant

Parameters (beyond :class:`PolicyParticipant`)

personas : tuple[str, …] Seat-ordered public names, used to resolve `name`-keyed statements from LLM seats to seat indices. Empty disables name resolution (index-keyed statements still parse).

Parameters

Name	Type	Default	Description
`args`		`()`	
`personas`	`tuple`	`()`	
`kwargs`		`{}`	

Attributes

Name	Type	Summary
`last_parse_census`	`dict`	
`personas`		

condition_on_commit

Fold a declared normalized walk-away floor into the posterior over the grid's reservation levels: a Gaussian log-likelihood `-(tau - floor)^2 / (2 scale^2)` per type, tempered by `strength` and `lam`.

```
condition_on_commit(
    bst: BeliefState,
    floor_norm: float,
    *,
    scale: float = 0.12,
    strength: float = 0.6,
) -> None
```

Defined in `interlens.arena.negotiation.talking` · source

The declarer's floor is on its own min-max scale and a grid type's `tau` on the type's additive [0, 1] scale — close but not identical coordinates, which is one reason this update is soft.

Parameters

Name	Type	Default	Description
`bst`	BeliefState	*required*	
`floor_norm`	`float`	*required*	
`scale`	`float`	`0.12`	
`strength`	`float`	`0.6`	

condition_on_hint

Fold declared per-issue top options into the posterior: types whose evaluator peaks at the declared option on that issue match the claim (likelihood `reliability`), the rest `1 - reliability`, tempered by `strength` and the state's own damping `lam`.

```
condition_on_hint(
    bst: BeliefState,
    tops: dict,
    *,
    reliability: float = 0.85,
    strength: float = 0.6,
) -> None
```

Defined in `interlens.arena.negotiation.talking` · source

Parameters

Name	Type	Default	Description
`bst`	BeliefState	*required*	
`tops`	`dict`	*required*	
`reliability`	`float`	`0.85`	
`strength`	`float`	`0.6`	

condition_on_narration

Fold “this package is above/below my bar” into the posterior — exactly the accept/reject evidence `observe_response` implements, with the claim’s truth value playing the vote.

```
condition_on_narration(
    bst: BeliefState,
    deal,
    above: bool,
    *,
    reliability: float = 0.85,
    strength: float = 0.6,
) -> None
```

Defined in `interlens.arena.negotiation.talking` · source

Parameters

Name	Type	Default	Description
`bst`	BeliefState	*required*	
`deal`		*required*	
`above`	`bool`	*required*	
`reliability`	`float`	`0.85`	
`strength`	`float`	`0.6`	

statements_in

Every well-formed talking-rational statement in `text`, plus the census the honesty audit needs.

```
statements_in(text: str, *, space=None, personas=()) -> tuple[list[dict],
    int, int]
```

Defined in `interlens.arena.negotiation.talking` · source

Returns `(statements, n_candidates, n_dropped)`. Each statement is a normalized dict with `kind` (:data:`STATEMENT_KINDS`), `seat` (int), and kind-specific fields — `narrate`: `above` (bool) plus optionally `deal` (index tuple) and `offer_id`; `hint`: `tops` `{issue_index: option_index}`; `commit`: `floor_norm` (float in [0, 1]) if the block carried one. The parser is TOTAL: an unparseable or schema-violating candidate is dropped and counted in `n_dropped`, never raised, because a listening seat must survive arbitrary LLM text.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	
`space`		`None`	
`personas`		`()`	

oracles

Module `interlens.arena.oracles`

The oracle layer: per-turn “what would a rational agent have done here?” annotations.

An `Oracle` scores every action available to a seat at a decision point and names the best one; the arena then measures the seat’s *regret* — `value(best) - value(chosen)` in the game’s value units — the centipawn-loss analog for negotiation (Regan & Haworth 2011). Oracles compose: a solution oracle, a belief oracle, an acceptance oracle, an equilibrium oracle, each citing its own literature — those concrete negotiation oracles live in `interlens.arena.negotiation` and subclass this generic `Oracle` ABC.

Two annotation paths write into an episode’s oracle log, both typed as :class:`OracleRecord`:

- **inline pure-Python oracles** — run post-`apply` with no extra generation (`Scenario.annotate_turn`), scoring the seat’s ACTUAL move against the oracle’s best on the same state. Cheap, so every turn can carry one.
- **forked provisional elicitations** — re-ask the *model* to finalize now on a private forked view (`Scenario.provisional_due`), an LLM-side probe of where the model thinks it stands.

Both land in `Episode.round_checkpoints` (kept as the field name for record compatibility) as `OracleRecord.to_json()` dicts.

Classes

Name	Summary
`Oracle`	A rational reference policy that scores a seat’s options at a decision point.

Name	Summary
`OracleRecord`	One per-turn oracle annotation on an episode (the typed replacement for the loose checkpoint dict).
`OracleVerdict`	One oracle's read of a decision point.

Functions

Name	Summary
`annotate`	Run every oracle over one decision point and return one inline :class:`OracleRecord` each — the ready helper a scenario calls from `annotate_turn` so its oracle wiring is a single line.

Oracle

A rational reference policy that scores a seat's options at a decision point.

```
Oracle()
```

Defined in `interlens.arena.oracles` · source

Inherits from: `ABC`

Subclasses implement :meth:`evaluate`. The arguments are deliberately generic so the ABC carries no negotiation specifics:

- `game` — the game definition (the negotiation `GameSpec`: deal space, score sheets, thresholds).
- `history` — the sequence of prior turns/actions the oracle may condition on.
- `agent` — the seat being evaluated (its private info defines its value function).
- `legal` — the legal actions available to `agent` now (the keys the verdict scores).

Attributes

Name	Type	Summary
`name`	`str`	

Methods

`evaluate`

```
evaluate(self, game: Any, history: Sequence, agent: str, legal: Sequence)
     -> OracleVerdict
```

source

Return an :class:`OracleVerdict` scoring `legal` for `agent` given `game` and `history`.

Parameters

Name	Type	Default	Description
`game`	`Any`	*required*	
`history`	`Sequence`	*required*	
`agent`	`str`	*required*	
`legal`	`Sequence`	*required*	

OracleRecord

One per-turn oracle annotation on an episode (the typed replacement for the loose checkpoint dict).

```
OracleRecord(
    round: int,
    seat: str,
    turn_idx: int = -1,
    oracle: str | None = None,
    chosen_value: float | None = None,
    best_value: float | None = None,
    divergence: float | None = None,
    verdict: dict | None = None,
    flags: list[str] = list(),
    provisional_action: Any = None,
    score: float | None = None,
    content: str | None = None,
)
```

Defined in `interlens.arena.oracles` · source

Two provenances share this record and are distinguished by whether `verdict` is set:

- **inline annotation** (`verdict` present): an :class:`Oracle` scored the seat's ACTUAL move against its best on the same state — `divergence` = `best_value - chosen_value` in the oracle's value units.
- **forked provisional probe** (`verdict` absent): the model was re-asked to finalize now on a private forked view; `provisional_action` / `score` / `content` capture that probe (the legacy checkpoint shape).

Parameters

Name	Type	Default	Description
`round`	`int`	*required*	
`seat`	`str`	*required*	
`turn_idx`	`int`	`-1`	
`oracle`	`str | None`	`None`	

Name	Type	Default	Description
chosen_value	float | None	None	
best_value	float | None	None	
divergence	float | None	None	
verdict	dict | None	None	
flags	list[str]	list()	
provisional_action	Any	None	
score	float | None	None	
content	str | None	None	

Attributes

Name	Type	Summary
best_value	float | None	
chosen_value	float | None	
content	str | None	
divergence	float | None	
flags	list[str]	
oracle	str | None	
provisional_action	Any	
round	int	
score	float | None	
seat	str	
turn_idx	int	
verdict	dict | None	

Methods

annotation

```
annotation(
    cls,
    verdict: OracleVerdict,
    *,
    round: int,
    seat: str,
    oracle: str,
    chosen_action: Any = None,
    turn_idx: int = -1,
) -> 'OracleRecord'
```

source

An inline-oracle record: the seat's `chosen_action` scored against `verdict.best`.

Parameters

Name	Type	Default	Description
`verdict`	OracleVerdict	*required*	
`round`	`int`	*required*	
`seat`	`str`	*required*	
`oracle`	`str`	*required*	
`chosen_action`	`Any`	`None`	
`turn_idx`	`int`	`-1`	

provisional

```
provisional(
    cls,
    *,
    round: int,
    seat: str,
    provisional_action: Any,
    score: float | None,
    content: str | None,
```

```
    turn_idx: int = -1,
) -> 'OracleRecord'
```

source

A forked provisional-probe record (the `Scenario.provisional_due` path).

Parameters

Name	Type	Default	Description
`round`	`int`	*required*	
`seat`	`str`	*required*	
`provisional_action`	`Any`	*required*	
`score`	`float | None`	*required*	
`content`	`str | None`	*required*	
`turn_idx`	`int`	`-1`	

to_json

```
to_json(self) -> dict
```

source

OracleVerdict

One oracle's read of a decision point.

```
OracleVerdict(
    action_values: dict[Any, float],
    best: Any,
    beliefs: dict | None = None,
    flags: list[str] = list(),
    extra: dict = dict(),
)
```

Defined in `interlens.arena.oracles` · source

`action_values` maps each evaluated action to its value (surplus / continuation value — the oracle's own units); `best` is the argmax action; `beliefs` optionally carries the oracle's posterior (e.g. a belief oracle's type distribution); `flags` are named hard-violation markers (e.g. `"ir_violation"`, `"below_threshold_accept"`). `extra` is a free-form JSON-serializable dict for per-verdict diagnostics beyond the value table — e.g. a best-response oracle's per-action `surplus_loss` and `best_response_deal`, an acceptance oracle's `reservation` / `rounds_left`, an equilibrium oracle's `v*` — carried through `to_json` into the episode's oracle log so the divergence atlas can read them. Actions are the dict keys, so they must be hashable — the formal :class:`~interlens.arena.actions.Action` dataclasses are frozen and satisfy this.

Parameters

Name	Type	Default	Description
`action_values`	`dict[Any, float]`	*required*	
`best`	`Any`	*required*	
`beliefs`	`dict | None`	`None`	
`flags`	`list[str]`	`list()`	
`extra`	`dict`	`dict()`	

Attributes

Name	Type	Summary
`action_values`	`dict[Any, float]`	
`beliefs`	`dict | None`	
`best`	`Any`	
`extra`	`dict`	
`flags`	`list[str]`	

Methods

`best_value`

```
best_value(self) -> float | None
```

source

The value of the best action (`action_values[best]` if present, else the max value, else `None`).

`divergence`

```
divergence(self, action: Any) -> float | None
```

source

Regret of `action`: `best_value - value(action)` (>= 0), or `None` if either is unknown.

Parameters

Name	Type	Default	Description
`action`	`Any`	*required*	

from_json

```
from_json(d: dict) -> 'OracleVerdict'
```

source

Rebuild an `OracleVerdict` from :meth:`to_json`, reconstructing typed :class:`~interlens.arena.actions.Action` keys (and `best`) so the regret math works on a verdict loaded from a stored episode.

`beliefs`/`flags`/`extra` come back as the JSON that was stored — `extra` is free-form diagnostics, so the coercion :func:`_jsonify` applies on the way out is deliberately not inverted on the way in.

Reads BOTH stored shapes: the current `{action_key: value}` object (episode `schema_version` v1.1+) and the original `[{"action": {...}, "value": v}, ...]` list of pairs (v1.0), so episodes recorded before the shape change still load and replay.

Parameters

Name	Type	Default	Description
d	dict	*required*	

to_json

```
to_json(self) -> dict
```

source

The stored form.

`action_values` is a JSON OBJECT keyed by :func:`~interlens.arena.actions.action_key` (the action's `to_json` dumped with sorted keys) — a map, because that is what it is, so a reader can look one action's value up directly instead of scanning a list of pairs. `best` is the same key string, so `stored['action_values'][stored['best']]` is the best value. `beliefs`/`extra` are free-form oracle payloads run through :func:`_jsonify`, so an

action-keyed dict or a numpy table an oracle stashed can't leave the record un-serializable.

`value_of`

```
value_of(self, action: Any) -> float | None
```

source

The value the oracle assigns `action`, or `None` if it wasn't evaluated.

Parameters

Name	Type	Default	Description
`action`	`Any`	*required*	

annotate

Run every oracle over one decision point and return one inline :class:`OracleRecord` each — the ready helper a scenario calls from `annotate_turn` so its oracle wiring is a single line.

```
annotate(
    oracles: Sequence[Oracle],
    game: Any,
    history: Sequence,
    agent: str,
    legal: Sequence,
    *,
    chosen_action: Any = None,
    round: int,
    seat: str,
    turn_idx: int = -1,
) -> list[OracleRecord]
```

Defined in `interlens.arena.oracles` · source

Skips an oracle that raises (a broken oracle must not abort an episode).

Parameters

Name	Type	Default	Description
`oracles`	Sequence[Oracle]	*required*	
`game`	`Any`	*required*	
`history`	`Sequence`	*required*	
`agent`	`str`	*required*	
`legal`	`Sequence`	*required*	
`chosen_action`	`Any`	`None`	
`round`	`int`	*required*	
`seat`	`str`	*required*	
`turn_idx`	`int`	`-1`	

ratchet

Module `interlens.arena.ratchet`

Adaptive difficulty ratchet: find the level where a model stops clearing the bar, then measure there.

A ceiling-saturated cell measures nothing — if every episode scores ~1.0 the comparison has no room to move. The ratchet drives any `Scenario` with a difficulty ladder to the *found level*: the first level whose probe mean fails the bar (default: mean primary < 0.75 of ceiling over `probe_n` probe episodes). Three phases, run through an `EpisodePool`:

1. **probe** — `probe_n` episodes per level, climbing while the mean clears `step_up`. With `speculative=True` the first `wave` levels are probed CONCURRENTLY (then the rest, if every probed level cleared the bar) — the wall-clock optimization from the arena experiments' re-ratchet runs; the found decision is the same pure function either way (`found_level`).
2. **measure** — `meas_n` episodes at the found level and its neighbor below (or above when found == 0), on the deterministic shared instance pool, offset past the probe block so probe and measurement instances never overlap.
3. **solo baselines** — paired solo episodes on the SAME instances, each under a `TokenBudget` equal to the median team tokens-out at that level (the matched-compute recipe).

State is serialized after every batch, so a restarted ratchet resumes where it stopped and never duplicates a completed episode (completed instances are skipped by id — paired-join safe). Instance pools are deterministic per (scenario, level): instance `i` uses seed `level*10000 + i`, shared by every arm and model.

Provenance: the collaboration-arena experiments' ratchet (sequential form) and their re-ratchet runs (speculative form), re-based onto the `EpisodePool` driver.

Attributes

Name	Type	Summary
`MEAS_N`		
`PROBE_N`		
`STEP_UP`		

Classes

Name	Summary
`DifficultyRatchet`	Probe -> measure -> paired-solo driver over one scenario/participant pair (see the module docstring).

Functions

Name	Summary
`found_level`	The found level implied by a set of probe means: the first (ascending) probed level whose mean fails the bar, else the highest level probed (the model never dropped below the bar).

DifficultyRatchet

Probe -> measure -> paired-solo driver over one scenario/participant pair (see the module docstring).

```
DifficultyRatchet(
    scenario: Scenario,
    participant,
    pool: EpisodePool,
    *,
    instances_dir: str | Path,
    state_path: str | Path,
    arm: str = 'team',
    probe_n: int = PROBE_N,
    meas_n: int = MEAS_N,
    step_up: float = STEP_UP,
    speculative: bool = False,
    wave: int = 3,
    cfg: dict | None = None,
    gen_config: dict | None = None,
    estimated_cost: float | None = None,
    solo_budget_default: int = 4000,
    seed0: int = 1000,
)
```

Defined in `interlens.arena.ratchet` · source

`pool` supplies the store (episode persistence + resume dedup) and optional `UsageMeter` (spend gating: each phase stops launching once the meter is exhausted; in-flight episodes finish).

Parameters

Name	Type	Default	Description
`scenario`	Scenario	*required*	
`participant`		*required*	
`pool`	EpisodePool	*required*	
`instances_dir`	`str | Path`	*required*	
`state_path`	`str | Path`	*required*	
`arm`	`str`	`'team'`	

Name	Type	Default	Description
probe_n	int	PROBE_N	
meas_n	int	MEAS_N	
step_up	float	STEP_UP	
speculative	bool	False	
wave	int	3	
cfg	dict | None	None	
gen_config	dict | None	None	
estimated_cost	float | None	None	
solo_budget_default	int	4000	
seed0	int	1000	

Attributes

Name	Type	Summary
arm		
estimated_cost		
instances_dir		
model		
participant		
pool		
scenario		
seed0		
solo_budget_default		
state		
state_path		

Methods

run

```
run(self) -> dict
```

source

Run (or resume) the ratchet to completion; returns the final state dict.

found_level

The found level implied by a set of probe means: the first (ascending) probed level whose mean fails the bar, else the highest level probed (the model never dropped below the bar).

```
found_level(probe_means: dict[int, float], step_up: float = STEP_UP) ->
    int
```

Defined in `interlens.arena.ratchet` · source

Pure — both probing modes and the ratchet's resume path decide through this one function.

Parameters

Name	Type	Default	Description
`probe_means`	`dict[int, float]`	*required*	
`step_up`	`float`	`STEP_UP`	

refusal

Module `interlens.arena.refusal`

Recovering a turn the API refused, without changing what the turn says.

Some hosted providers return a completion with `stop_reason == "refusal"` and zero output tokens: an API-side classifier declined the request before the model wrote anything. Measured on `claude-opus-5` in the auction program, that refusal has three properties that make it a *protocol* problem rather than a modelling one:

- it reproduces **deterministically** for a byte-identical request, so it survives the ordinary retry (which re-sends the same view with a parser note appended);
- it is **not semantic** — substituting the domain vocabulary, dropping a blurb, salting the tail, or splitting the turn all leave it in place, while removing almost *any* single block clears it;
- so the seat contributes no move at all, and any fallback action recorded for it is a move the model never chose.

The response implemented here is a **re-render ladder**: on a refusal, the same turn view is re-rendered under a deterministic, content-preserving perturbation and reissued, escalating through a fixed sequence of rungs. Every rung preserves every number, every rule, and every reviewed sentence — only *ordering*, *section framing*, and an inert *nonce line* vary — so a recovered turn conditions on the same information the refused one did. The ladder is a property of the engine, not of an arm or a condition, so it applies identically everywhere and is a protocol feature rather than a confound; it is preregisterable because the rungs, their order, and the seeded permutation are all fixed in advance and logged per turn (`TurnRecord.refusal_recovery`).

What this is **not**: an attempt to defeat the classifier. The rungs are content-preserving by construction. If a view refuses at every rung the turn is recorded as `terminal` and the cell's validity gate sees it — a configuration that cannot clear the ladder is an escalation to the provider, not an engineering problem.

Worked example

```
from interlens.arena.refusal import RefusalLadder ladder = RefusalLadder() view = [{“role”: “system”, “content”: “You are seat A.”}, … {“role”: “user”, “content”: “## Catalogue: 90## Your values: 104” … “Reply now with one fenced JSON object.”}] rung1 = ladder.perturb(view, 1, key=“ep-1/seat-A/r3”) rung1[1][“content”].splitlines()[0] # doctest: +ELLIPSIS ‘Protocol note: request variant …’ sorted(ladder.perturb(view, 2, key=“ep-1/seat-A/r3”)[1][“content”].split(“”)) == … sorted(rung1[1][“content”].split(“”)) # same blocks, different order True ladder.perturb(view, 3, key=“ep-1/seat-A/r3”)[1][“content”].count(“Lot 1: 104”) # numbers preserved 1
```

Attributes

Name	Type	Summary
`REFUSAL_RECOVERY_KEY`		
`REFUSAL_STOPS`		
`TRUNCATION_STOPS`		

Classes

Name	Summary
`RefusalLadder`	A fixed, seeded sequence of content-preserving re-renders, tried in order after a refusal.

Functions

Name	Summary
`is_refusal`	Did this completion come back declined rather than generated?
`recovery_record`	One turn's recovery record: `outcome` is `"recovered"` or `"terminal"`, `rung` the 1-based rung that cleared it (`None` when none did), `attempts` the rung names tried in order.

RefusalLadder

A fixed, seeded sequence of content-preserving re-renders, tried in order after a refusal.

```
RefusalLadder(
    rungs: tuple[str, ...] = ('nonce', 'permute', 'reframe'),
    seed: int = 20260815,
    target_role: str = 'user',
)
```

Defined in `interlens.arena.refusal` · source

:param rungs: The escalation order. Rungs are **cumulative** — rung k applies the perturbations of rungs $1..k$ — so each attempt is strictly further from the refused bytes than the last while staying content-preserving. The default ladder is:

```
1. `"nonce"` — prepend one inert line naming a seeded variant tag. Adds no
     number and no instruction;
   its only job is to make the request bytes differ from the refused ones.
2. `"permute"` — reorder the view's independent top-level blocks under a
     seeded permutation, holding
   the first block (the turn's own header) and the last block (the single
     closing ask) in place, since
   those two carry position-dependent meaning and the interior sections
     do not.
3. `"reframe"` — re-wrap markdown section headings in an alternate but
     fixed framing
   (`## Catalogue` becomes `### Section: Catalogue`). Heading *text* and
     every body line are
   untouched.
```

:param seed: Ladder-level seed mixed into every per-turn permutation, so the whole campaign's re-renders replay exactly from the run manifest. :param target_role: Which message of the view is perturbed. `"user"` (the default) is the measured one: in the auction pilot a line prepended to the user turn cleared a refusal 2/2 while the same line on the system prompt did not.

Parameters

Name	Type	Default	Description
`rungs`	`tuple[str, ...]`	`('nonce', 'permute', 'reframe')`	
`seed`	`int`	`20260815`	
`target_role`	`str`	`'user'`	

Attributes

Name	Type	Summary
`rungs`	`tuple[str, ...]`	
`seed`	`int`	
`target_role`	`str`	

Methods

`perturb`

```
perturb(self, view: list[dict], rung: int, *, key: str) -> list[dict]
```

source

The view re-rendered at rung k (1-based, cumulative).

Returns a new list; `view` is untouched.

:param view: The refused request's view — a list of `{"role", "content"}` messages. :param rung: Which rung to render, 1-based; `rung > len(self)` raises. :param key: The per-turn key the seeded perturbations hang off, e.g. `f"{episode_id}/{seat}/{round}/{phase}"`. Identical inputs give identical bytes, which is what makes a recovered turn reproducible from the episode record.

Parameters

Name	Type	Default	Description
`view`	`list[dict]`	*required*	
`rung`	`int`	*required*	
`key`	`str`	*required*	

rung_name

```
rung_name(self, rung: int) -> str
```

source

The name of rung k (1-based), for logging.

Parameters

Name	Type	Default	Description
`rung`	`int`	*required*	

is_refusal

Did this completion come back declined rather than generated?

```
is_refusal(message) -> bool
```

Defined in `interlens.arena.refusal` · source

True when the participant stamped `metadata["refusal"]` (`APIParticipant` sets it from the native stop reason), when the stop reason is one of :data:`REFUSAL_STOPS`, or when the visible content is empty and the stop reason is *not* a truncation — an empty non-truncated completion is the same "the seat wrote nothing" failure, whatever the provider called it.

Parameters

Name	Type	Default	Description
`message`		*required*	

recovery_record

One turn's recovery record: `outcome` is `"recovered"` or `"terminal"`, `rung` the 1-based rung that cleared it (`None` when none did), `attempts` the rung names tried in order.

```
recovery_record(outcome: str, rung: int | None, attempts: list[str]) ->
    dict
```

Defined in `interlens.arena.refusal` · source

Reported as `api_silence_recovered@rung_k` / `api_silence_terminal`.

Parameters

Name	Type	Default	Description
`outcome`	`str`	*required*	
`rung`	`int | None`	*required*	
`attempts`	`list[str]`	*required*	

replay

Module `interlens.arena.replay`

Deterministic replay of stored episodes through a scenario's state machine.

An `Episode` record stores every committed turn's text. Because a `Scenario` is a pure state machine — text in, state out, no RNG in stepping — feeding those turns back through `apply` reconstructs the exact final state and re-derives the outcome with the *current* parser and scorer. Uses:

- **audit**: verify a stored dataset's recorded outcomes against the packaged scorer (the arena export's own reproduction check runs on exactly this);
- **re-scoring**: recompute outcomes under an extended scorer without re-running any model;
- **analysis**: reconstruct intermediate states (support maps, challenge ledgers) at any turn.

Replay is exact for episodes produced by this engine (and the arena experiments that share its schema): the turn log stores think-stripped visible content in scenario order, provisional turns live separately in `round_checkpoints` (they never touched state), and the one-retry flow re-emits the same request, which is how `apply` sees it here too.

Attributes

Name	Type	Summary
`DEFAULT_FIELDS`		

Classes

Name	Summary
`ReplayError`	A stored turn could not be matched to the state machine's pending request.

Functions

Name	Summary
`apply_prefix`	Replay `episode`'s stored turns into an existing `state`, stopping before turn index `upto`.
`make_replay_state`	A fresh state built exactly the way `episode` was: same instance, arm, seed, and `cell_cfg` (minus the resolved personas, which `make_state` re-resolves identically from the seed).
`replay_episode`	Feed a stored episode's turns back through `scenario` and return the recomputed outcome dict.
`rescore`	Replay `episode` and compare the recomputed outcome to the recorded one on `fields`.

ReplayError

A stored turn could not be matched to the state machine's pending request.

```
ReplayError()
```

Defined in `interlens.arena.replay` · source

Inherits from: `RuntimeError`

apply_prefix

Replay `episode`'s stored turns into an existing `state`, stopping before turn index `upto`.

```
apply_prefix(
    scenario: Scenario,
    state: dict,
    episode: dict,
    upto: int | None = None,
    *,
    on_turn=None,
    on_request=None,
) -> int
```

Defined in `interlens.arena.replay` · source

This is the branch/resume primitive: after it returns, `state` is exactly the mid-game state the engine held live just before the stored turn whose `idx` equals `upto` — `next_requests(state)` re-issues that turn's request, and play can continue with different text from there. `upto=None` replays every stored turn (the full-replay case). Matching is by the stored turn's `idx` field, not list position, so a record whose turns start above zero (itself a branch continuation) replays correctly.

`on_turn` is an optional `callable(state, request, turn) → None` invoked after each turn is applied, while `state` still holds that turn's post-move context — the hook for post-hoc per-turn work (re-running oracles, reconstructing intermediate ledgers). Its return value is ignored; raising aborts the replay.

`on_request` is the same shape, invoked BEFORE the turn is applied, while `state` still holds the context the seat actually decided in. The two hooks are not interchangeable: a per-turn counterfactual ("what would another decision rule have played HERE") is only meaningful against the pre-move state, and reading it from `on_turn` would score every rule against the position its own turn created.

Returns the number of turns applied. Raises :class:`ReplayError` if a stored turn cannot be matched to a pending request, or if `upto` is not reached because the record has fewer turns.

Parameters

Name	Type	Default	Description
`scenario`	Scenario	*required*	
`state`	`dict`	*required*	
`episode`	`dict`	*required*	
`upto`	`int | None`	`None`	
`on_turn`		`None`	
`on_request`		`None`	

make_replay_state

A fresh state built exactly the way `episode` was: same instance, arm, seed, and `cell_cfg` (minus the resolved personas, which `make_state` re-resolves identically from the seed).

```
make_replay_state(scenario: Scenario, instance: Instance, episode: dict) -
    > dict
```

Defined in `interlens.arena.replay` · source

Parameters

Name	Type	Default	Description
`scenario`	Scenario	*required*	
`instance`	Instance	*required*	
`episode`	`dict`	*required*	

replay_episode

Feed a stored episode's turns back through `scenario` and return the recomputed outcome dict.

```
replay_episode(
    scenario: Scenario,
    instance: Instance,
    episode: dict,
    *,
    on_turn=None,
    on_request=None,
) -> dict
```

Defined in `interlens.arena.replay` · source

`episode` is the stored JSON record (`Episode.to_json()` shape; the arena experiments' records load directly). The instance must be the one the episode was played on (`episode['instance_id']`).

`on_turn` and `on_request` are as in :func:`apply_prefix`, whose full-replay case this is.

Parameters

Name	Type	Default	Description
`scenario`	Scenario	*required*	
`instance`	Instance	*required*	
`episode`	`dict`	*required*	
`on_turn`		`None`	
`on_request`		`None`	

rescore

Replay `episode` and compare the recomputed outcome to the recorded one on `fields`.

```
rescore(
    scenario: Scenario,
    instance: Instance,
    episode: dict,
    fields: tuple[str, ...] = DEFAULT_FIELDS,
) -> dict
```

Defined in `interlens.arena.replay` · source

Returns `{"match": bool, "recorded": {...}, "recomputed": {...}, "mismatches": [field, ...]}`.

Parameters

Name	Type	Default	Description
`scenario`	Scenario	*required*	
`instance`	Instance	*required*	
`episode`	`dict`	*required*	
`fields`	`tuple[str, ...]`	`DEFAULT_FIELDS`	

rollouts

Module `interlens.arena.rollouts`

Directory-backed **rollout sets**: run more episodes into the same place, safely, and resume.

The arena already has every piece of a rollout: a `Scenario` (the game), a `Participant` (who plays), an `EpisodePool`/`BatchedEpisodePool` (drives them), and an `EpisodeStore` (persists them). What it did not have is the bookkeeping *around* a set of episodes — the part every experiment re-derives: which (instance, seed, arm) triples are already on disk, whether it is safe to add more to this directory, and what the set looks like so far. This module is that layer and nothing else. It composes; it re-implements nothing.

Library vs experiment — the boundary this module commits to

In the library (here): the identity and safety of a *set of episodes on disk*. Open-or-create a directory, a manifest describing what produced it (model, scenario, config fingerprint, seeds completed, invocations, artifact links), a dedupe/resume key, an append that REFUSES to mix protocols, and two contamination gates (fabricated turns, and the placeholder budget below). Also a deliberately minimal :meth:`RolloutSet.summary` — counts, statuses, success rate, mean primary, spend. All of it is scenario-agnostic: it reads only fields the arena's own :mod:`~interlens.arena.schema` defines.

In the experiment: everything protocol-specific. Which scaffold, framing, oracle stack, instance bank, seat lineup and arms constitute a cell; how a model id resolves to a participant; and every rich metric (per-party normalized surplus, among-deals baskets, instance-clustered intervals, paired tables). Those live with the analyzers that own their definitions — `summary()` here is a health check, not a results table, and it stays that way so the library never grows a dependency on one experiment's notion of a good deal.

Two gates, because each is blind to the other

A rollout set can be 100% `status="done"`, 100% `parse_ok`, 0% empty content, `fabrication 0.000` — and still be a quarter silent. Those are two different failures wearing the same :data:`~interlens.arena.engine.EMPTY_TURN_PLACEHOLDER` string:

- the ENGINE fabricated the turn (generation failed outright) — the fabrication budget owns this one;
- the MODEL burned its entire per-turn budget inside an unterminated `<think>` block and emitted no visible action. The engine did its job, so `fabrication` correctly reads 0.000; the placeholder it substitutes is non-empty and parses into a well-formed no-op, so `parse_ok` reads 1.000 too. Measured at **24.4% of turns** on a thinking-on Qwen3-32B cell at a 2,048-token cap, against 0.0% with thinking off — and on that cell `parse_ok` was *anti*-correlated with quality (1.000 silent vs 0.958 healthy).

So :meth:`RolloutSet.summary` reports a **placeholder budget** beside the fabrication audit, split by cause, and broken down BY ROUND — the failure is late-episode by construction (context grows with the transcript: 5.8% at round 1, ~35% by rounds 2-4), so a short smoke run or a peek at an episode's opening turns reads ~0% and under-detects it. Exceeding the budget flags the set loudly and stamps `manifest["turn_health"]`; it never raises, because the episodes are real evidence and the right response is to record the rate. Raising the per-turn cap changes the protocol string, so the remediated cells do not pair with the old ones.

Why the append refusal is loud

A rollout set is evidence. Appending episodes played under a *different* protocol into a directory whose manifest claims one protocol produces a silently mixed corpus that every downstream analysis will average over without ever knowing. So :meth:`RolloutSet.append` compares a fingerprint of the caller's config against the manifest's and raises :class:`RolloutConfigMismatch` on any

difference. `allow_mismatch=True` is the explicit escape hatch, and it is not a silencer: the mismatched episodes are STAMPED in their own `cell_cfg` and the divergence is recorded in the manifest, so the contamination stays legible forever.

Worked example:

```
from interlens.arena.rollouts import rollout
from interlens.arena.negotiation import games
from interlens.arena.negotiation.sheets import GameSpec
from interlens.arena.scenarios.scorable import ScorableNegotiation
from interlens.arena.table import rational_table

instances = [games.build_preset_instance("scorable", level=0, seed=s,
     instance_name=ScorableNegotiation.name)[0]
             for s in range(3)]

def seat_lineup(instance, arm, seed):                    # fresh policy
     seats per episode
    game = GameSpec.from_json(instance.payload)
    return rational_table(game, ["boulware", "conceder", "bayes-
     rational"], deadline=game.rounds)

cfg = {"model": "policy:bayes-rational", "scaffold": "canonical", "info":
     "full"}
rs = rollout(scenario=ScorableNegotiation(), instances=instances,
     participant=seat_lineup,
             seeds=[0, 1], arms=["moves_chat"], out="runs/pilot",
     config=cfg)
print(rs.summary_text())
# RolloutSet runs/pilot
#   model=policy:bayes-rational scenario=scorable_negotiation
     cfg=8f2c1a09b4de
#   episodes: 6 (6 distinct keys) statuses={'done': 6}
#   arms: {'moves_chat': 6}
#   success: 4/6 = 0.667
#   mean primary: 0.7143 over 6 scored episodes
#   usage: 0 in / 0 out tokens, $0.00
#   fabricated turns: 0 (in 0 episodes)
#   placeholder turns: 0/94 = 0.0% (engine 0.0%, burned cap 0.0%) | at cap
     0.0% | no-op 19.5%

# later — two more seeds into the SAME set; the first two are skipped,
     not replayed
rollout(scenario=ScorableNegotiation(),
     instances=instances, participant=seat_lineup,
        seeds=[0, 1, 2, 3], arms=["moves_chat"], out="runs/pilot",
     config=cfg)
```

Attributes

Name	Type	Summary
`MANIFEST_NAME`		
`MANIFEST_SCHEMA`		
`PLACEHOLDER_BUDGET`		
`ParticipantFactory`		
`RolloutKey`		
`ScenarioFactory`		

Classes

Name	Summary
`RolloutConfigMismatch`	An append's config fingerprint disagrees with the set's manifest — refused rather than silently mixed.
`RolloutSet`	A directory of episodes plus the manifest that says what produced them.

Functions

Name	Summary
`config_fingerprint`	Stable SHA256 over `config`, canonicalized (sorted keys, compact separators, non-JSON values repr'd).
`rollout`	Run `instances x seeds x arms` into the rollout set at `out` (creating or resuming it) and return it.

RolloutConfigMismatch

An append's config fingerprint disagrees with the set's manifest — refused rather than silently mixed.

```
RolloutConfigMismatch()
```

Defined in `interlens.arena.rollouts` · source

Inherits from: `RuntimeError`

RolloutSet

A directory of episodes plus the manifest that says what produced them.

```
RolloutSet(
    root: str | Path,
    *,
    config: dict | None = None,
    model: str | None = None,
    scenario: str | None = None,
    create: bool = True,
)
```

Defined in `interlens.arena.rollouts` · source

Layout:

```
<root>/rollout_manifest.json   # schema, model, scenario, config +
    fingerprint, keys, invocations
<root>/episodes/               # an EpisodeStore tree (one JSON per
    episode)
<root>/instances/              # the exact Instance JSONs the episodes
    were played on
```

Open-or-create: constructing on an existing root reads its manifest; on a fresh root it writes one from `config`/`model`/`scenario`. It is safe to construct concurrently from several processes only in the sense that :class:`EpisodeStore` writes are atomic — the manifest is last-writer-wins, so drive one set from one process.

Parameters

root : str | Path The set's directory. Created (with parents) if missing. config : dict | None The protocol config for this set. Required when creating; on an existing set it is CHECKED against the manifest's fingerprint (see :meth:`append`) and otherwise ignored — the manifest is the authority. model : str | None Model identity recorded in the manifest (`"anthropic:claude-opus-5"`, a local HF id, `"policy:..."`). Recorded only; it should also appear inside `config` if it defines the condition. scenario : str | None Scenario name recorded in the manifest. create : bool Create the directory and manifest

when absent (default). `False` raises on a missing set, which is what an analysis-side reader wants.

Parameters

Name	Type	Default	Description
`root`	`str | Path`	*required*	
`config`	`dict | None`	`None`	
`model`	`str | None`	`None`	
`scenario`	`str | None`	`None`	
`create`	`bool`	`True`	

Attributes

Name	Type	Summary
`fingerprint`	`str`	The manifest's config fingerprint — the protocol identity every append is held to.
`instances_dir`		
`manifest`	`dict`	
`manifest_path`		
`root`		
`store`		

Methods

append

```
append(
    self,
    episodes: Sequence[Episode | dict],
```

```
    *,
    config: dict | None = None,
    allow_mismatch: bool = False,
    invocation: Sequence[str] | None = None,
    artifacts: dict | None = None,
) -> dict
```

source

Add `episodes` to the set, deduped on :data:`RolloutKey`, and update the manifest.

Episodes already written into this set's own store (the normal case — :func:`rollout` points the pool's store here) are recognized by `episode_id` and not rewritten; episodes from elsewhere (merging chunk directories, importing a foreign run) are saved in. Either way an episode whose key is already present is SKIPPED, never duplicated.

`config` is checked against the manifest's fingerprint first: any difference raises :class:`RolloutConfigMismatch` and NOTHING is written, because a set that quietly mixes protocols is worse than a failed append. `allow_mismatch=True` proceeds, but stamps every accepted episode's `cell_cfg["rollout_config_mismatch"]` with the offending fingerprint and records the divergence (both fingerprints and the differing keys) in `manifest["mismatched_appends"]`.

Returns `{"added": int, "skipped": int, "keys": [(instance_id, seed, arm, episode_id), ...], "mismatch": bool}` — the accepted rows, in the shape the manifest records them.

Parameters

Name	Type	Default	Description
`episodes`	Sequence[Episode | dict]	*required*	
`config`	`dict | None`	`None`	
`allow_mismatch`	`bool`	`False`	

Name	Type	Default	Description
invocation	Sequence[str] | None	None	
artifacts	dict | None	None	

episodes

```
episodes(self) -> list[dict]
```

source

Every stored episode record, as raw dicts (sorted by store path).

keys

```
keys(self) -> set[RolloutKey]
```

source

The :data:`RolloutKey` of every episode ON DISK.

Read from the episodes themselves rather than the manifest, so a set whose manifest is stale (a crash between the store write and the manifest write) still resumes correctly — the episodes are the ground truth.

load_instances

```
load_instances(self) -> list[Instance]
```

source

The instances this set was played on, as saved by :meth:`save_instances` (`[]` if none yet).

A set's bank is its own record. Reuse it rather than regenerating: :func:`~interlens.arena.schema.new_id` is uuid-based, not seed-derived, so re-generating "the same" instances mints NEW `instance_id` s and every

stored key stops matching — the set would silently replay everything it already has.

missing

```
missing(
    self,
    instances: Sequence[Instance],
    seeds: Sequence[int],
    arms: Sequence[str],
) -> list[RolloutKey]
```

source

The planned keys of `instances x seeds x arms` that are not yet on disk — what a resume must run.

Parameters

Name	Type	Default	Description
`instances`	Sequence[Instance]	*required*	
`seeds`	`Sequence[int]`	*required*	
`arms`	`Sequence[str]`	*required*	

record_artifacts

```
record_artifacts(self, **links: str = {}) -> None
```

source

Record artifact locations on the manifest (`transcripts=..., hf_dataset=..., report=...`).

Parameters

Name	Type	Default	Description
`links`	`str`	`{}`	

save_instances

```
save_instances(self, instances: Iterable[Instance]) -> Path
```

source

Persist the exact instances played, one JSON each, so analysis can resolve `instance_id` -> game.

Parameters

Name	Type	Default	Description
`instances`	Iterable[Instance]	*required*	

summary

```
summary(self, *, placeholder_budget: float = PLACEHOLDER_BUDGET) -> dict
```

source

A HEALTH summary of the set, computed from the stored episode records alone.

Deliberately minimal and scenario-agnostic — counts, `status` breakdown, arm/model breakdown, the fraction of closed/successful episodes, the mean of `outcome['primary']`, token/dollar usage, and TWO independent contamination gates. Rich, protocol-defined baskets (normalized surplus, among-deals statistics, clustered intervals) belong to the experiment's analyzers, which own those definitions; this exists to answer "did the rollouts run, and is any of this text real?" before anyone analyses it.

The two gates are separate because each is blind to the other's failure:

- **fabrication** (:func:`~interlens.arena.engine.gen_failures`) — turns no model ever produced.
- **placeholder budget** (:func:`~interlens.arena.engine.turn_signatures`) — turns that SAID nothing. A model can burn its entire per-turn budget inside an unterminated `<think>` block, and the placeholder it gets substituted is a non-empty string that parses into a well-formed no-op. So `fabrication` reads a correct 0.000, `parse_ok` reads 1.000, an empty-content check reads 0.0%, and a quarter of the turns are silent anyway (measured 24.4% on a thinking-on Qwen3-32B cell).

`placeholder_rate_by_round` is reported because this failure is **late-episode by construction** — context grows with the transcript, so the budget is exhausted in later rounds (measured 5.8% at round 1 rising to ~35% by round 2-4). A short smoke run, or a preview of an episode's opening turns, reads near 0% and under-detects it; :meth:`summary_text` therefore prints the worst round alongside the pooled rate.

Parameters

placeholder_budget : float Fraction of turns that may be placeholders before `placeholder_budget_exceeded` is set (default :data:`PLACEHOLDER_BUDGET`). It is a flag, never a raise: the episodes are real evidence, and the documented response to a silent cell is to record the rate — and to treat it as a NEW protocol, since raising the cap changes the protocol string and breaks pairing with the old cells.

Parameters

Name	Type	Default	Description
`placeholder_budget`	`float`	`PLACEHOLDER_BUDGET`	

summary_text

```
summary_text(self, *, placeholder_budget: float = PLACEHOLDER_BUDGET) ->
    str
```

source

One-line-per-fact rendering of :meth:`summary` for a driver script's stdout.

Both contamination gates always print, including when they are clean — a gate you only see when it fires is a gate nobody knows to look for.

Parameters

Name	Type	Default	Description
`placeholder_budget`	`float`	`PLACEHOLDER_BUDGET`	

config_fingerprint

Stable SHA256 over `config`, canonicalized (sorted keys, compact separators, non-JSON values repr'd).

```
config_fingerprint(config: dict | None) -> str
```

Defined in `interlens.arena.rollouts` · source

This is the set's protocol identity. Anything that changes what an episode *means* belongs in `config` (scaffold, framing, info condition, arms, oracle stack, bank hash, model id); anything that merely changes how it was scheduled (concurrency, chunking, artifact root) must NOT, or a resumed run refuses itself.

Parameters

Name	Type	Default	Description
`config`	`dict | None`	*required*	

rollout

Run `instances x seeds x arms` into the rollout set at `out` (creating or resuming it) and return it.

```
rollout(
    *,
    scenario: Scenario | ScenarioFactory,
    instances: Sequence[Instance],
    participant,
    out: str | Path,
    model: str | None = None,
    arms: Sequence[str] = ('team',),
    seeds: Sequence[int] = (0,),
    cfg: dict | None = None,
    config: dict | None = None,
    engine: str = 'auto',
    concurrency: int = 16,
    meter: UsageMeter | None = None,
    estimated_cost: float | None = None,
    allow_mismatch: bool = False,
    gen_config: dict | None = None,
    invocation: Sequence[str] | None = None,
    artifacts: dict | None = None,
) -> RolloutSet
```

Defined in `interlens.arena.rollouts` · source

Swapping the model is one argument (`participant`), swapping the game is one argument (`scenario`); everything else is bookkeeping this function does for you: skip the (instance, seed, arm) triples already on disk, point the pool's store at the set so episodes persist as they finish, then append + update the manifest.

Parameters

scenario : Scenario | ScenarioFactory The game. A plain `Scenario` is shared by every episode; a callable `(instance) → Scenario` is called per instance (for a per-game oracle stack). instances : SequenceInstance The bank. Saved into `<out>/instances/` so analysis can resolve every `instance_id`. participant : Participant | ParticipantFactory Who plays. A participant object is SHARED across episodes (correct for stateless model weights, and the precondition for batched co-stepping); a callable `(instance, arm, seed) → Participant` is

called per episode (required for policy seats, whose belief/offer state is per-episode mutable). out : str | Path The rollout-set directory. Existing sets are resumed; the config fingerprint must match. model : str | None Model identity for the manifest. Defaults to `config["model"]` when present. arms, seeds : Sequence The protocol arms and episode seeds; one episode per (instance, arm, seed). cfg : dict | None The per-episode situational config handed to the scenario and stored on each episode as `cell_cfg`. config : dict | None The set's protocol identity (see :func:`config_fingerprint`). Pass everything that defines the condition; a resumed set refuses to run if it disagrees. engine : str `"auto"` (default) uses :class:`BatchedEpisodePool` co-stepping when `participant` is a single shared object exposing `generate_batch` (a local model — the 5-20x path) and the async :class:`EpisodePool` otherwise; `"batched"` / `"async"` force one. A per-episode factory can never be batched, since the whole point of co-stepping is one shared model. concurrency : int Max concurrent episodes for the async pool (API width is the client's own cap). Ignored when batched. meter : UsageMeter | None Shared spend ledger for hosted-API seats; with `estimated_cost` the pool reserves before each episode, so a dollar cap stops the run instead of merely describing it afterwards. allow_mismatch : bool Forwarded to :meth:`RolloutSet.append` — see its docstring; leaves a permanent stamp. gen_config : dict | None Provider/sampling provenance recorded on every episode. invocation : Sequence[str] | None The command line to record (defaults to `sys.argv`). artifacts : dict | None Artifact links to record on the manifest (transcript dir, HF dataset URL, …).

Raises

RolloutConfigMismatch If `out` exists under a different protocol config and `allow_mismatch` is False. Raised BEFORE any episode runs, so a mis-pointed resume costs nothing.

Parameters

Name	Type	Default	Description
`scenario`	Scenario | ScenarioFactory	*required*	

Name	Type	Default	Description
`instances`	Sequence[Instance]	*required*	
`participant`		*required*	
`out`	`str | Path`	*required*	
`model`	`str | None`	`None`	
`arms`	`Sequence[str]`	`('team',)`	
`seeds`	`Sequence[int]`	`(0,)`	
`cfg`	`dict | None`	`None`	
`config`	`dict | None`	`None`	
`engine`	`str`	`'auto'`	
`concurrency`	`int`	`16`	
`meter`	UsageMeter | None	`None`	
`estimated_cost`	`float | None`	`None`	
`allow_mismatch`	`bool`	`False`	
`gen_config`	`dict | None`	`None`	
`invocation`	`Sequence[str] | None`	`None`	
`artifacts`	`dict | None`	`None`	

scenario

Module `interlens.arena.scenario`

The `Scenario` interface: a pure game-logic state machine, participant-agnostic.

A scenario owns everything about the *game* — instance generation (with an exact solver for ceilings/verification), per-seat private framing, the turn protocol, structured-action parsing, early termination, and scoring — and nothing about *models*: it emits `SeatRequest`s (who must speak now, on what view) and consumes the resulting text. The engine (`engine.py`) owns persistence, retries, provisional forking, budgets, and driving participants.

(The concept is a *scenario*, not an "environment": these are turn-based game protocols with exact scorers, without RL/gym step/reward semantics.)

State is a plain dict owned by the scenario. Required keys maintained by every scenario:

- `events`: `list[{seat, content, only?}]` — the public transcript (`MODERATOR` for announcements; `only` restricts an event to named seats)
- `round`: 1-based current round
- `done`: bool
- `arm`: `"team"` | `"solo"` | variant tags like `"team-greedy"`

The engine sets `state["budget_exhausted"] = True` when an episode token/cost budget fires; scenarios must then steer to a forced finalization (both bundled scenarios do).

Classes

Name	Summary
`Scenario`	

Scenario

```
Scenario()
```

Defined in `interlens.arena.scenario` · source

Inherits from: `ABC`

Attributes

Name	Type	Summary
`N_LEVELS`	`int`	
`SOLO_CEILING`	`int | None`	
`SOLO_SEAT`	`str`	
`default_communication`	`str`	The communication mode task runners (e.g. the Inspect tasks) use when none is given.
`has_solo`	`bool`	
`name`	`str`	

Methods

`annotate_turn`

```
annotate_turn(self, state: dict, request: SeatRequest, turn) -> list
```

source

Inline per-turn oracle annotations, run by the engine right after `apply` commits `turn` (no extra generation).

Return a list of `interlens.arena.oracles.OracleRecord` — the engine appends them to the episode's oracle log. A scenario with a pure-Python oracle stack scores the seat's ACTUAL move against the oracle's best here (`interlens.arena.oracles.annotate` is the ready one-liner); the default runs no oracles. This is the inline sibling of the forked `provisional_due` path — the two coexist.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`request`	SeatRequest	*required*	
`turn`		*required*	

`apply`

```
apply(self, state: dict, request: SeatRequest, text: str) -> dict | None
```

source

Parse `text`, mutate state (append events, record actions, advance phase/round, set `done`).

Return `{'retry': <prompt>}` to request ONE re-prompt of the same seat (engine-enforced), else `None`. Must record the parsed action for the turn log via `state['_last_parse'] = (parsed_action, parse_ok)`.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`request`	SeatRequest	*required*	
`text`	`str`	*required*	

classify_outcome

```
classify_outcome(self, state: dict, turns: list, outcome: dict) -> dict
```

source

Post-scoring outcome refinement, merged into the episode's outcome by the engine AND by replay.

Must be pure in `(state, turns, outcome)`. `turns` items are `TurnRecord`s live and plain dicts in replay — read fields defensively. Default: no refinement. The distributed long-context scenario uses this for its `truncated_at_budget` / `capitulated` outcome classes.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`turns`	`list`	*required*	
`outcome`	`dict`	*required*	

final_retry_directive

```
final_retry_directive(
    self,
    state: dict,
    request: SeatRequest,
    retry_prompt: str,
    *,
    key=None,
) -> dict | None
```

source

The one-retry-then-give-up bookkeeping shared by every scenario's forced-finalization phase: on the FIRST malformed final action for `key` (default `("retry", request.round)`) return `{'retry': retry_prompt}` (the engine re-prompts once); on the second, return `None` so the caller finalizes with what it has.

Idempotent per key via `state['_r']`, so replay re-emits the identical retry.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`request`	SeatRequest	*required*	
`retry_prompt`	`str`	*required*	
`key`		`None`	

generate_instance

```
generate_instance(self, level: int, seed: int) -> Instance
```

source

Generate one solver-verified instance at `level` from `seed` (deterministic payload per seed).

Parameters

Name	Type	Default	Description
`level`	`int`	*required*	
`seed`	`int`	*required*	

make_state

```
make_state(self, instance: Instance, arm: str, seed: int, cfg: dict | None
    = None) -> dict
```

source

Fresh episode state.

For `arm='solo'` the single seat gets ALL private info; solo episodes iterate until a final action or the engine's budget forces finalization. `cfg` is an optional sweep-cell config (situational knobs: rounds, framings, personas); scenarios that don't support one may ignore it.

Parameters

Name	Type	Default	Description
`instance`	Instance	*required*	
`arm`	`str`	*required*	
`seed`	`int`	*required*	
`cfg`	`dict | None`	`None`	

`messaging_decider`

```
messaging_decider(self, state: dict) -> str
```

source

The seat whose last fenced action decides a messaging episode (the finalizer / proposer).

The adapter scans this seat's turns for the deciding `answer` / `proposal` JSON. Default: the first seat.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

`messaging_finalizable`

```
messaging_finalizable(self) -> bool
```

source

Whether a free-form messaging episode can be reduced to one final action by the adapter's final-answer/proposal extraction — True exactly when the protocol ends in a single structured answer that the deciding seat emits.

Scenarios that instead implement `score_from_messaging` don't need this. The Inspect messaging adapter uses this capability (not a hardcoded scenario name) to decide whether it can run the scenario at all. Default: False.

next_requests

```
next_requests(self, state: dict) -> list[SeatRequest]
```

source

The requests due now (>1 only for simultaneous phases).

`[]` iff done. `episode_id` on the returned requests is filled by the engine.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

provisional_due

```
provisional_due(self, state: dict) -> list[SeatRequest]
```

source

Forked finalize-now elicitations due at this point (the engine calls right after each applied wave).

Provisional responses never enter state or any transcript. Default: none.

Parameters

Name	Type	Default	Description
state	dict	*required*	

rounds_used

```
rounds_used(self, state: dict) -> int
```

source

Parameters

Name	Type	Default	Description
state	dict	*required*	

score

```
score(self, state: dict) -> dict
```

source

Final outcome dict.

Must include `primary` (float, higher is better) and `success` (bool) where meaningful, plus scenario-specific fields.

Parameters

Name	Type	Default	Description
state	dict	*required*	

score_from_messaging

```
score_from_messaging(
    self,
```

```
    instance,
    transcript: list[tuple[str, str]],
    cfg: dict | None = None,
) -> dict | None
```

source

Score a free-form messaging-mode episode (the Inspect adapter's `communication="messaging"`): `transcript` is `[(seat, text), ...]` in send order.

Return the outcome dict, or `None` when the scenario has no messaging reduction — the adapter then falls back to the final-answer/proposal reduction for scenarios whose protocol ends in one structured action, and raises otherwise.

Parameters

Name	Type	Default	Description
`instance`		*required*	
`transcript`	`list[tuple[str, str]]`	*required*	
`cfg`	`dict | None`	None	

score_provisional

```
score_provisional(self, state: dict, parsed) -> float | None
```

source

Score a provisional final action with the normal scorer.

Default: not scored.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`parsed`		*required*	

seat_framings

```
seat_framings(self, state: dict) -> dict
```

source

`{seat_name: verbatim system prompt}` for a fresh state — the private framing each seat receives.

Team arms render every seat directly; solo arms capture the single seat's system prompt from its first request. Lets a dataset/record ship self-contained framings.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

seat_specs

```
seat_specs(self, state: dict) -> list[dict]
```

source

`[{name, role, ...}]` for the episode record.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

solo_apply

```
solo_apply(self, state: dict, request: SeatRequest, text: str) -> dict |
    None
```

source

The solo arm's `apply`: record the turn, finalize on a genuine answer, give up on a forced-final that produced none, else (below any `SOLO_CEILING`) nudge and continue.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`request`	SeatRequest	*required*	
`text`	`str`	*required*	

solo_continue

```
solo_continue(self, state: dict) -> str
```

source

The 'keep going, answer when ready' user nudge appended after a non-final solo turn.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

solo_final_cap

```
solo_final_cap(self, state: dict) -> int
```

source

Per-turn output cap for the forced-final solo turn.

Default: 2048.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

`solo_final_prompt`

```
solo_final_prompt(self, state: dict) -> str
```

source

The ‘budget reached — answer NOW’ user prompt for the forced-final solo turn.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

`solo_finalize`

```
solo_finalize(self, state: dict, answer, text: str) -> None
```

source

Commit `answer` (from a genuine final turn) to state.

The base sets `done` afterwards.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`answer`		*required*	
`text`	`str`	*required*	

solo_give_up

```
solo_give_up(self, state: dict) -> None
```

source

Commit a no-answer outcome when the forced-final turn produced no valid answer.

Base sets `done`.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

solo_parse

```
solo_parse(self, state: dict, text: str) -> tuple
```

source

Parse one solo turn -> `(parsed, parse_ok, has_final, answer)`: `parsed` is the extracted JSON, `parse_ok` the value logged as the turn's `parse_ok`, `has_final` whether the turn committed a final answer, and `answer` the finalized value passed to `solo_finalize` (may itself be `None`).

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`text`	`str`	*required*	

solo_requests

```
solo_requests(self, state: dict) -> list[SeatRequest]
```

source

The solo arm's `next_requests`: forced-final on budget exhaustion (once), else seed the task on the first call and emit a working turn.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

solo_system

```
solo_system(self, state: dict) -> str
```

source

The solo seat's system prompt (full-info framing).

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

solo_task

```
solo_task(self, state: dict) -> str
```

source

The initial user task text seeding the solo conversation (all private info + the question).

Parameters

Name	Type	Default	Description
state	dict	*required*	

solo_work_cap

```
solo_work_cap(self, state: dict) -> int
```

source

Per-turn output cap for a solo working turn.

Default: `solo_turn_cap` cell knob, else 900.

Parameters

Name	Type	Default	Description
state	dict	*required*	

scenarios

Package `interlens.arena.scenarios`

Bundled scenarios, four families:

- `Negotiation` — multi-issue, multi-party deal-making with secret score sheets (exact enumeration).
- `InfoRelay` — wrong-shard epistemics: can a team relay a correct fact past a confident wrong holder.
- `SecurityDilemma` — repeated 2-party build/deescalate/attack with an absorbing war spiral and noisy intelligence (payoff-exact scoring; no solo arm).
- `CodingCollab` — 3 seats jointly write one Python module against a public pytest suite while each holds private, mechanically checkable style constraints (sandboxed exact scoring).
- `AuctionScenario` — five bidders over `T` auction stages under one mechanism config (sealed second-price, Dutch / English clock, simultaneous ascending), with the format as a config rather than a separate runner.
- `DistributedLongContext` — one long-context task partitioned across 4 seats (task adapters + offline instance builders in `interlens.arena.scenarios.dlc`; first-class `truncated_at_budget` / `capitulated` outcome classes).

Generator-backed scenarios ship solver-verified instances (every instance carries its exact ceiling, floor, and hidden solution) and exact scorers. Further scenarios follow the same pattern: subclass `interlens.arena.Scenario`. `SCENARIOS` maps every bundled scenario name to a zero-argument factory (the distributed long-context entries bind their task adapter).

Modules

- `auction` — Repeated multi-bidder auctions as one :class:`~interlens.arena.scenario.Scenario`.

- `auction_examples` — The three worked turn views of the auction scaffold, generated from real frozen draws.
- `auction_policy` — Computable seats inside the ordinary engine loop.
- `auction_prompts` — The frozen prompt scaffold for :class:`~interlens.arena.scenarios.auction.AuctionScenario`.
- `coding` — Coding collaboration with private constraints: 3 seats jointly write ONE Python module.
- `dlc` — Task adapters for the distributed long-context scenario, ported from the RLM paper's benchmarks.
- `longcontext` — Distributed long-context: one long-context task split across 4 communicating seats.
- `negotiation` — Negotiation: a multi-issue, multi-party deal with secret score sheets (structured-JSON actions).
- `priors` — Role-prior sign table for the negotiation scenario (role × issue) and sheet-vs-prior analysis helpers.
- `relay` — Info relay: an epistemic team task with a confidently-wrong agent.
- `scorable` — ScorableNegotiation: the repaired multi-party, multi-issue scorable game — the *protocol* around a :class:`~interlens.arena.negotiation.sheets.GameSpec` (carried in `Instance.payload`), built on the shared typed-action (:mod:`interlens.arena.actions`) and oracle (:mod:`interlens.arena.oracles`) layers. "Repaired" names the specific benchmark flaws it fixes, each called out at the code that fixes it: votes-not-arithmetic closure, structural channel separation, a restated deadline, and measured-not-blocked economic illegality.
- `scorable_prompts` — The canonical prompt scaffold for the scorable-negotiation scenario.
- `security` — Security dilemma: a repeated 2-party build/deescalate/attack game with noisy intelligence.

Re-exported

Name	Defined in	Summary
`AuctionScenario`	`interlens.arena.scenarios.auction`	Five bidders, `T` stages, one mechanism config, one communication rung.
`CodingCollab`	`interlens.arena.scenarios.coding`	
`DistributedLongContext`	`interlens.arena.scenarios.longcontext`	
`InfoRelay`	`interlens.arena.scenarios.relay`	
`Negotiation`	`interlens.arena.scenarios.negotiation`	
`ScorableNegotiation`	`interlens.arena.scenarios.scorable`	The repaired scorable-negotiation protocol over a :class:`GameSpec` (see the module docstring).
`SecurityDilemma`	`interlens.arena.scenarios.security`	
`TaskAdapter`	`interlens.arena.scenarios.longcontext`	Per-task behavior plugged into `DistributedLongContext`.
`dlc_scenario`	`interlens.arena.scenarios.dlc`	A `DistributedLongContext` scenario for one task, e.g. `dlc_scenario("sniah")`.

Attributes

Name	Type	Summary
`SCENARIOS`		

auction

Module `interlens.arena.scenarios.auction`

Repeated multi-bidder auctions as one :class:`~interlens.arena.scenario.Scenario`.

An episode is `T` auction stages played by the same five bidders over one frozen :class:`~interlens.arena.auction.spec.AuctionSpec`. **Formats are mechanism configs and repetition is the horizon; neither forks a runner** — the failure mode that rule prevents (divergent per-format runners drifting apart) is exactly what a format x channel x horizon factorial cannot survive.

The stage loop (design.md §3.1):

```
for t in 1..T:
    publish the stage catalogue (B_jt, blurbs, tie-      permutation,
     stages remaining
    render each seat's private block for stage t
    for r in 1..talk_rounds:   message round (broadcast and/or DM, per
     channel)
    run the format's bidding rounds
    settle: allocation, payments, executed transfers (dm_transfers only)
    publish the stage result under the format's disclosure rule
    append to the carried history
```

Privacy is structural, not a property of the wording: a turn view is assembled here from a shared public part plus exactly one private block belonging to the reading seat, and a DM reaches only the seats :class:`~interlens.arena.auction.actions.DMRouter` delivers it to. `tests/test_auction_scenario.py` asserts that programmatically over a whole episode rather than trusting the templates.

All prose lives in :mod:`.auction_prompts`, which transcribes the reviewed and frozen templates. Nothing in this module originates model-facing wording.

Attributes

Name	Type	Summary
`BID_PHASE`		
`DEFAULT_TURN_MAX_TOKENS`		
`MID_STAGE_TALK_FAMILIES`		
`TALK_PHASE`		

Classes

Name	Summary
`AuctionScenario`	Five bidders, `T` stages, one mechanism config, one communication rung.

Functions

Name	Summary
`clock_round_cap`	Number of clock rounds needed to walk from the start price to the reserve (or back), inclusive.
`clock_start`	The clock start price used by every stage of a clock-family episode.
`opening_clock_price`	Where a clock family's price starts in each stage.

AuctionScenario

Five bidders, `T` stages, one mechanism config, one communication rung.

```
AuctionScenario(
    scaffold: AuctionPromptScaffold | None = None,
    turn_max_tokens: int = DEFAULT_TURN_MAX_TOKENS,
)
```

Defined in `interlens.arena.scenarios.auction` · source

Inherits from: Scenario

Parameters

scaffold : AuctionPromptScaffold | None The frozen wording. `None` uses :data:`~.auction_prompts.DEFAULT_AUCTION_SCAFFOLD`; a variant scaffold is how a wording ablation is run, never an edit to this class. turn_max_tokens : int Per-turn output cap requested on every :class:`~interlens.arena.schema.SeatRequest`. The engine's budget can shrink it and never raise it, so a hosted floor is applied at the participant.

Parameters

Name	Type	Default	Description
`scaffold`	Auction-PromptScaffold | None	`None`	
`turn_max_tokens`	`int`	`DEFAULT_TURN_MAX_TOKENS`	

Attributes

Name	Type	Summary
`N_LEVELS`		
`default_communication`		

Name	Type	Summary
`has_solo`		
`name`		
`scaffold`		
`turn_max_tokens`		

Methods

`apply`

```
apply(self, state: dict, request: SeatRequest, text: str) -> dict | None
```

source

Read one turn, deliver its channels, record its binding move, and advance the wave when the last seat has answered.

A syntax or legality error gets exactly one retry carrying the parser's specific message; a second failure records the format's fallback move and delivers nothing the seat wrote, so a channel payload attached to an unparseable turn cannot get through by breaking the envelope.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`request`	SeatRequest	*required*	
`text`	`str`	*required*	

`classify_outcome`

```
classify_outcome(self, state: dict, turns: list, outcome: dict) -> dict
```

source

Episode-level tags the run index sorts on, computed purely from the stored outcome.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`turns`	`list`	*required*	
`outcome`	`dict`	*required*	

`generate_instance`

```
generate_instance(self, level: int, seed: int, **overrides={}) -> Instance
```

source

Generate one auction instance from `seed`.

The payload carries the frozen spec under both value structures — `apv` (the default and the home of the repeated questions) and `ipv` (the belief-free benchmark) — because switching structure zeroes `beta` and `sigma_z` rather than redrawing, so the two are a within-bank paired contrast rather than two populations. Freezing both means a cell selects rather than re-derives.

Parameters

Name	Type	Default	Description
`level`	`int`	*required*	
`seed`	`int`	*required*	
`overrides`		`{}`	

make_state

```
make_state(self, instance: Instance, arm: str, seed: int, cfg: dict | None
    = None) -> dict
```

source

Fresh episode state: the cell's spec (value structure, mechanism, horizon, channel selected from `cfg`), the ledger, the DM router, and the empty history the stage loop appends to.

Parameters

Name	Type	Default	Description
`instance`	Instance	*required*	
`arm`	`str`	*required*	
`seed`	`int`	*required*	
`cfg`	`dict | None`	`None`	

next_requests

```
next_requests(self, state: dict) -> list[SeatRequest]
```

source

The wave due now: every seat that must move, each with a view built BEFORE any of them replies, so the round is genuinely simultaneous and nobody sees what anyone else wrote before writing.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

rounds_used

```
rounds_used(self, state: dict) -> int
```

source

Waves completed — the episode's own turn budget, distinct from stages.

Parameters

Name	Type	Default	Description
state	dict	*required*	

score

```
score(self, state: dict) -> dict
```

source

The episode outcome: per-stage rows, the episode aggregates every gate reads, and the protocol hygiene counters (design.md §5.3).

Parameters

Name	Type	Default	Description
state	dict	*required*	

seat_framings

```
seat_framings(self, state: dict) -> dict
```

source

`{seat_name: system prompt}` for the episode record.

A computable seat has no prose framing, so it records the name of its decision rule instead of a prompt it never reads.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

seat_specs

```
seat_specs(self, state: dict) -> list[dict]
```

source

One record per seat for the episode file: the addressable seat id, its persona, and its public card parameters (never a draw).

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	

spec_for

```
spec_for(self, instance: Instance, cfg: dict)
```

source

The cell's spec: the frozen bank draws under the cell's value structure, with the cell's mechanism, horizon, and channel applied.

A cell never redraws. `mechanism` may be replaced (a single-lot bank serves the sealed, Dutch, and English cells from identical draws, which is what makes R3 <-> R4 a within-bank paired contrast), but its lot count may not change, since the draws are per-lot.

`cfg["scramble_cards"]` is the X1 control: the five public cards are permuted across the seats under a derangement seeded from the INSTANCE ID, so X1 and its matched real-persona cell O1 read

the same frozen draws and the same scramble in every rerun (:func:`~interlens.arena.auction.spec .scramble_public_cards`). It is applied last, after the mechanism, horizon and channel, so a scrambled cell differs from its reference cell in the cards and in nothing else.

`cfg["ring"]` is a :class:`~interlens.arena.auction.spec.RingSpec` (or its JSON) for the NON-FROZEN instructed-ring probe. It sets membership on the spec only; whether the members are TOLD is `RingSpec.instructed`, read in :meth:`_ring_block`.

Parameters

Name	Type	Default	Description
`instance`	Instance	*required*	
`cfg`	`dict`	*required*	

state_block

```
state_block(self, state: dict, seat: int) -> dict
```

source

The machine-readable turn state a computable seat reads in place of the prose turn prompt.

Carries this seat's OWN draws and the public round state. A rival's realized values appear only under `oracle_values`, and only for an oracle seat — an explicitly named field, so reading it is a deliberate act the policy's own information gate controls rather than an accident of serialization.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`seat`	`int`	*required*	

system_prompt

```
system_prompt(self, state: dict, seat: int) -> str
```

source

The episode-level system prompt for `seat`: identical across seats except the one line that names the reading seat, so the public part is byte-identical and privacy cannot leak through it.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`seat`	`int`	*required*	

turn_prompt

```
turn_prompt(self, state: dict, seat: int) -> str
```

source

One turn view: the stage catalogue, the carried-history digest, this seat's private block, and the phase block.

Nothing private to any other seat can reach it — the catalogue and the digest are built once and shared, and only :meth:`_private_block` reads a draw, for its owner only.

Parameters

Name	Type	Default	Description
`state`	`dict`	*required*	
`seat`	`int`	*required*	

clock_round_cap

Number of clock rounds needed to walk from the start price to the reserve (or back), inclusive.

```
clock_round_cap(spec) -> int
```

Defined in `interlens.arena.scenarios.auction` · source

Parameters

Name	Type	Default	Description
`spec`		*required*	

clock_start

The clock start price used by every stage of a clock-family episode.

```
clock_start(spec) -> int
```

Defined in `interlens.arena.scenarios.auction` · source

Derived from the frozen draws rather than stored: the largest per-stage `clock_ceiling` (itself set above that stage's maximum realized valuation), rounded up onto the increment grid. One start for the whole episode keeps the system prompt's printed clock consistent with every stage's turn view, and makes the ceiling reachable only by a bidder bidding above every value in the stage — which is what makes `clock_ceiling_rate` a protocol-failure statistic rather than an ordinary outcome.

Parameters

Name	Type	Default	Description
`spec`		*required*	

opening_clock_price

Where a clock family's price starts in each stage.

```
opening_clock_price(spec) -> int
```

Defined in `interlens.arena.scenarios.auction` · source

The two clocks run in opposite directions and start at opposite ends: a DESCENDING (Dutch) clock starts above every realized valuation and falls, while an ASCENDING (English) clock starts at the RESERVE and rises. Deriving both from :func:`clock_start` without the direction was a real bug — it opened the English clock above every value, so every seat exited in the first round and the stage ended before any price discovery happened at all.

Parameters

Name	Type	Default	Description
`spec`		*required*	

auction_examples

Module `interlens.arena.scenarios.auction_examples`

The three worked turn views of the auction scaffold, generated from real frozen draws.

`docs/templates/rendered_examples.md` was written by hand before any code existed, so its numbers are illustrative rather than drawn: its catalogue blurbs vary where a fixed phrase table cannot, and two of its private blocks omit a line the template file states unconditionally. Reproducing hand-written prose byte for byte is not a transcription check — it is a check that the transcriber copied the same inconsistencies.

So the relationship is inverted here: these three views are GENERATED from the scaffold over real draws, the generated text is what `rendered_examples.md` holds, and `tests/test_auction_prompts.py` pins the two together. Any later edit to the wording, on either side, breaks that test. The narrative message history of each example is scripted verbatim from the reviewed document, since the message prose is what a reviewer is actually reading the examples for.

Run `experiments/rational_agents/auction/render_prompt_examples.py` to rewrite the document.

Attributes

Name	Type	Summary
`EXAMPLES`	`tuple[dict, ...]`	
`NARRATIVES`	`dict[str, tuple[tuple, ...]]`	

Functions

Name	Summary
`build_example`	Drive one example's episode to its target stage and round, then render `(system, turn)` for its seat.
`render_document`	The whole `rendered_examples.md` body, generated.

build_example

Drive one example's episode to its target stage and round, then render `(system, turn)` for its seat.

```
build_example(cfg: dict) -> tuple[str, str]
```

Defined in `interlens.arena.scenarios.auction_examples` · source

The episode is played by a scripted straightforward bidder so the standing bids, the settled stages, and the digest are all real ledger state rather than typed-in numbers; only the message history is scripted, from :data:`NARRATIVES`.

Parameters

Name	Type	Default	Description
`cfg`	`dict`	*required*	

render_document

The whole `rendered_examples.md` body, generated.

```
render_document() -> str
```

Defined in `interlens.arena.scenarios.auction_examples` · source

auction_policy

Module `interlens.arena.scenarios.auction_policy`

Computable seats inside the ordinary engine loop.

A policy seat is a :class:`~interlens.arena.participant.Participant` like any other: it receives a view, returns a :class:`~interlens.arena.message.Message`, and produces a real `TurnRecord` that the transcript export and the analysis read on exactly the same footing as an LLM seat's. That matters more here than convenience — the whole point of the five-arm design is that the arms differ in the DECISION RULE and nothing else, so a computable seat that bypassed the turn loop would differ in its instrumentation too.

Policy seats read a structured state block, not prose. The scenario renders a fenced `auction_state` JSON object into a policy seat's view in place of the natural-language turn prompt (the same precedent as `ScorableNegotiation`'s `negotiation_state` block). The block carries this seat's own private draws and the public round state, and NOTHING about any other seat's draws — an oracle seat's extra information rides in a separate, explicitly-named field so that reading it is a deliberate act the policy's own `_rival_values` gate controls.

And they speak. The mute-channel lesson is binding, so the participant fills the envelope's `message` and `dm` fields from :mod:`~interlens.arena.auction.policy_text`, driven by the policy's own decision functions. The oracle's templates are identical to the rational seat's.

Attributes

Name	Type	Summary
`FREE_ARM_INFORMATION`	`dict[str, str]`	
`STATE_FENCE`		

Classes

Name	Summary
`AuctionPolicyParticipant`	One computable auction seat: an information-conditional best response, plus its templated channel behavior.

Functions

Name	Summary
`read_state_block`	The latest `auction_state` block in a view.
`replay_integrity`	Re-derive every computable turn of `episode` from its own recorded state block and check it matches what the seat actually played.
`state_block`	The fenced, machine-readable turn state handed to a policy seat.

AuctionPolicyParticipant

One computable auction seat: an information-conditional best response, plus its templated channel behavior.

```
AuctionPolicyParticipant(
    name: str,
    *,
    spec: AuctionSpec,
    seat: int,
    information: str = 'private',
    instance_id: str = '',
)
```

Defined in `interlens.arena.scenarios.auction_policy` · source

Inherits from: Participant

Parameters

name : str Participant name recorded on the episode. spec : AuctionSpec The episode spec. Public structure only is read from it; the seat's realized draws arrive through the state block, so the same object can back every seat without an information leak by construction. seat : int Seat index. information : str `"private"` (the rational seat: own information only) or `"oracle"` (the same best response with everyone's realized private information). This is the ONLY difference between the two arms. instance_id : str Frozen instance id, used to seed the surface variant of every templated line so the variant is reproducible and arm-invariant.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`spec`	AuctionSpec	*required*	
`seat`	`int`	*required*	
`information`	`str`	`'private'`	

Name	Type	Default	Description
instance_id	str	''	

Attributes

Name	Type	Summary
information		
instance_id		
name		
others_role		
policy		
private_context		
seat		
self_role		
spec		
system_prompt		

Methods

generate

```
generate(
    self,
    view,
    *,
    seat: str | None = None,
    max_new_tokens: int | None = None,
    **kwargs={},
) -> Message
```

source

Read the state block, compute the move, and emit the envelope an LLM seat would have written.

Parameters

Name	Type	Default	Description
`view`		*required*	
`seat`	`str | None`	`None`	
`max_new_tokens`	`int | None`	`None`	
`kwargs`		`{}`	

read_state_block

The latest `auction_state` block in a view.

```
read_state_block(view) -> dict
```

Defined in `interlens.arena.scenarios.auction_policy` · source

Latest wins, so a retry prompt cannot resurrect a stale round.

Parameters

Name	Type	Default	Description
`view`		*required*	

replay_integrity

Re-derive every computable turn of `episode` from its own recorded state block and check it matches what the seat actually played.

```
replay_integrity(episode: dict, bank_dir) -> dict
```

Defined in `interlens.arena.scenarios.auction_policy` · source

The mechanism-independent form of "the played seat is its rule." For each turn the participant recorded, this rebuilds the :class:`~interlens.arena.auction.bidders.AuctionState` from the very `auction_state` block that turn was rendered with, re-runs the policy over it, and asserts the freshly computed move equals the recorded `parsed_action`. It needs no equilibrium concept and no benchmark, so it applies to **every** mechanism family — including the clock families, where asserting equality against an equilibrium bid is meaningless because the policy is a stage-myopic best responder while the benchmark is a symmetric equilibrium (design.md §6).

This is the check that generalizes the SAA on-path gate, and it is exactly the class that catches a policy silently forfeiting a lot it merely did not demand, a state block fed with the wrong round, a renderer dropping a field, or an oracle seat handed information it should not have — all for $0, from episodes already on disk, before a paid arm runs.

Determinism note: the recorded block is the seat's own input, so seeded tie-breaks and the realized standing table are reproduced rather than re-drawn, and the comparison is exact rather than tolerance-based. Only turns whose seat is computable in this arm are checked; an LLM seat has no rule to replay, which is the entire point of the contrast.

Parameters

episode : dict A stored episode record, carrying `instance_id`, `cell_cfg`, `seats` and `turns` with each turn's `view` and `parsed_action`. bank_dir : str | Path The frozen bank directory the episode's instance was drawn from.

Returns

dict `{"arm", "turns", "checked", "mismatches", "pass"}`. `checked` is 0 and `pass` True for an all-LLM episode, which has no computable seat to replay.

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`bank_dir`		*required*	

state_block

The fenced, machine-readable turn state handed to a policy seat.

```
state_block(payload: dict) -> str
```

Defined in `interlens.arena.scenarios.auction_policy` · source

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

auction_prompts

Module `interlens.arena.scenarios.auction_prompts`

The frozen prompt scaffold for :class:`~interlens.arena.scenarios.auction.AuctionScenario`.

This module **transcribes** `experiments/rational_agents/auction/docs/templates/` and originates no wording of its own. Those files were written and reviewed as prose before any implementation existed, because design.md §6′s prompt-freeze rule makes wording a preregistered object: prompt optimization can *manufacture* stable collusion [tian2026], a sharper hazard in a repeated design than a one-shot one. Any wording change here is a protocol change that forces a re-run of every affected cell.

The scaffold holds only WORDING. Every number, name, and table row is passed in by the scenario, so one scaffold renders any instance and a wording ablation is a variant scaffold rather than an edited scenario – the same rule :mod:`~interlens.arena.scenarios.scorable_prompts` states for the negotiation side.

Composition, per `templates/README.md`:

- `SYSTEM` = :meth:`AuctionPromptScaffold.system_prompt` over setting, seat identity, objective, the public roster, the prior statement, the format rules, the four-channel envelope, and conduct.
- `TURN` = :meth:`AuctionPromptScaffold.turn_prompt` over the stage catalogue, the bounded carried-history digest, this seat's private block, one phase block, and the closing ask.

Three composition invariants, enforced here and checked by `tests/test_auction_prompts.py` rather than trusted to the wording: no private field of any seat appears outside its owner's private block; every number appears in exactly one block; stages-remaining is restated in every turn view.

One block here is **not** part of the freeze and is marked as such at its definition: :meth:`AuctionPromptScaffold.ring_block`, transcribed from the non-frozen `templates/ring_block.md`. It is the instructed-ring capability probe, it is

rendered only for a seat in an `instructed` `RingSpec`, and it appends as a strict SUFFIX so that every frozen block stays byte-identical whether it is present or not.

Attributes

Name	Type	Summary
`ARROW`		
`BLURB_PHRASES`	`dict[str, dict[int, str]]`	
`CAPITAL_POSITION_PHRASES`	`dict[str, str]`	
`DEFAULT_AUCTION_SCAFFOLD`		
`EMDASH`		
`FALLBACK_WORDS`	`dict[str, str]`	
`LOT_NAMES`	`tuple[str, ...]`	
`MINUS`		
`PERSONA_CARDS`	`dict[str, str]`	
`REVIEWED_CONSTANTS`	`dict[str, float]`	

Classes

Name	Summary
`AuctionPromptScaffold`	One immutable prompt wording for the repeated-auction scenario.

Functions

Name	Summary
`lot_blurb`	The prose description of a lot, generated from its loading vector through :data:`BLURB_PHRASES`.

Name	Summary
`lot_id`	The addressable lot token, `L01`..`L24` – what the action grammar's `"lot"` field carries and what every table keys on.
`lot_name`	The persistent display name of slot `slot_id` (:data:`LOT_NAMES`).
`signed`	A loading or attribute entry as the reviewed prose prints it: `+1`, `0`, or `{MINUS}1` with a Unicode minus.
`signed_amount`	A surplus as the digest prints it: `+41`, `0`, `{MINUS}12`.

AuctionPromptScaffold

One immutable prompt wording for the repeated-auction scenario.

```
AuctionPromptScaffold(
    dm_worked_example: bool = True,
    show_own_surplus: bool = True,
    turn_marker: bool = True,
    RING_BLOCK_VERSION: str = 'ring_block_v1',
)
```

Defined in `interlens.arena.scenarios.auction_prompts` · source

Every render method is keyword-only and takes primitives; the scenario computes the numbers and this class decides only how they read. Construct a variant to ablate wording – never edit :class:`~interlens.arena.scenarios.auction.AuctionScenario`.

Parameters

dm_worked_example : bool Whether the `dm` rung and above offer the one worked envelope example with literal ellipses for the message bodies. Filling those ellipses would be a demonstration of what to say, which at that rung is exactly what the design must not supply (`channel_blocks.md`); the knob exists so the omission is ablatable, not so it is optional. show_own_surplus : bool Whether each seat's stage result reports its own realized surplus. On by default: the number is computable by the seat from its own private values and the public prices, so publishing it leaks nothing, and doing the arithmetic silently in a scratchpad is noise we do not want to measure.

Parameters

Name	Type	Default	Description
`dm_worked_example`	`bool`	`True`	
`show_own_surplus`	`bool`	`True`	
`turn_marker`	`bool`	`True`	
`RING_BLOCK_VERSION`	`str`	`'ring_block_v1'`	

Attributes

Name	Type	Summary
`RING_BLOCK_VERSION`	`str`	
`dm_worked_example`	`bool`	
`show_own_surplus`	`bool`	
`turn_marker`	`bool`	

Methods

`catalogue`

```
catalogue(
    self,
    *,
    stage_index: int,
    horizon: int,
    rows,
    tie_break,
    attr_names,
    single_line: str | None = None,
) -> str
```

source

The per-stage header: the deadline restated, the catalogue, and the tie-break order.

`rows` is a list of `(lot_id, description, base_value, loading)` tuples at more than one lot; `single_line` carries the pre-rendered one-lot form instead.

Parameters

Name	Type	Default	Description
`stage_index`	`int`	*required*	
`horizon`	`int`	*required*	
`rows`		*required*	

Name	Type	Default	Description
`tie_break`		*required*	
`attr_names`		*required*	
`single_line`	`str | None`	`None`	

conduct

```
conduct(self, *, family: str, channel: str) -> str
```

source

The rules of the road: whole numbers, privacy, the budget rule, and the one-retry rule.

Parameters

Name	Type	Default	Description
`family`	`str`	*required*	
`channel`	`str`	*required*	

envelope

```
envelope(self, *, family: str, channel: str, dm_cap: int, other_seat_ids)
    -> str
```

source

The four-channel JSON contract, with the fields present per channel rung.

Parameters

Name	Type	Default	Description
`family`	`str`	*required*	
`channel`	`str`	*required*	

Name	Type	Default	Description
`dm_cap`	`int`	*required*	
`other_seat_ids`		*required*	

`fallback`

```
fallback(self, key: str, **kw={}) -> str
```

source

The second-failure notice: what was recorded for the turn, and that nothing written was delivered.

Parameters

Name	Type	Default	Description
`key`	`str`	*required*	
`kw`		`{}`	

`format_rules`

```
format_rules(
    self,
    *,
    family: str,
    pricing: str,
    n_items: int,
    increment: int,
    start_price: int,
    reserve: int,
    round_cap: int,
) -> str
```

source

The mechanism block for this cell's family: rules, action grammar, JSON examples, tie-break, what is revealed between rounds, and the payment rule.

Exactly one section is ever rendered.

Parameters

Name	Type	Default	Description
`family`	`str`	*required*	
`pricing`	`str`	*required*	
`n_items`	`int`	*required*	
`increment`	`int`	*required*	
`start_price`	`int`	*required*	
`reserve`	`int`	*required*	
`round_cap`	`int`	*required*	

history_digest

```
history_digest(self, *, digest_rows, message_log) -> str
```

source

Stages 1..t-1: the published per-stage outcome and this seat's own result, plus every message this seat sent or received, verbatim.

The bound is a design commitment, not an implementation detail: text is carried in full because coordination lives in it, while prior stages' round-by-round bid ledgers are dropped once the stage settles. Every lot's winner and price survives the compression – the compression is of ROUNDS, not of lots, since the per-lot outcome is what a market-division convention would be built on.

Parameters

Name	Type	Default	Description
`digest_rows`		*required*	
`message_log`		*required*	

objective

```
objective(self) -> str
```

source

Own-surplus maximization, stated as self-contained.

The second paragraph is the neutrality anchor: it rules out both “beat the others” and “help the others” readings without naming coordination.

own_result

```
own_result(self, *, stage_index: int, own_lots, own_paid: int,
     own_surplus: int) -> str
```

source

The private one-liner appended to a seat’s own view of the stage result.

Parameters

Name	Type	Default	Description
stage_index	int	*required*	
own_lots		*required*	
own_paid	int	*required*	
own_surplus	int	*required*	

prior_statement

```
prior_statement(
    self,
    *,
    beta: float,
    sigma_z: float,
    sigma_eps: float,
    sigma_nu: float,
    value_structure: str,
```

```
    multi_item: bool,
) -> str
```

source

The load-bearing block: the generative model in plain language, and the statement that every bidder can reason about every other bidder's likely willingness to pay – and that the reverse is equally true.

Under IPV the block is replaced wholesale rather than trimmed, because a bidder must not be told to read signal into cards that carry none.

Parameters

Name	Type	Default	Description
`beta`	`float`	*required*	
`sigma_z`	`float`	*required*	
`sigma_eps`	`float`	*required*	
`sigma_nu`	`float`	*required*	
`value_structure`	`str`	*required*	
`multi_item`	`bool`	*required*	

`private_block`

```
private_block(
    self,
    *,
    stage_index: int,
    horizon: int,
    capital_position: str | None,
    value_rows,
    budget: int,
    synergy_target=None,
    synergy_bonus: int | None = None,
    capacity: int | None = None,
    decay: float | None = None,
    signal_rows=None,
    single_value: int | None = None,
) -> str
```

source

This seat's stage-`t` private block – and the only block in the whole composition that contains anything private.

Lines are omitted rather than rendered empty when they do not apply.

Parameters

Name	Type	Default	Description
`stage_index`	`int`	*required*	
`horizon`	`int`	*required*	
`capital_position`	`str | None`	*required*	
`value_rows`		*required*	
`budget`	`int`	*required*	
`synergy_target`		None	
`synergy_bonus`	`int | None`	None	
`capacity`	`int | None`	None	
`decay`	`float | None`	None	
`signal_rows`		None	
`single_value`	`int | None`	None	

`public_roster`

```
public_roster(self, *, cards, multi_item: bool, interdep: bool) -> str
```

source

The five PUBLIC cards, byte-identical in every seat's system prompt.

Parameters

cards : list[dict] One dict per seat in seat order, with keys `persona_id`, `display_name`, `attrs` (the K signed entries, aligned with the attribute names), `capacity`, `synergy_rate`, `decay`, `gamma`. multi_item : bool Whether the capacity /

adjacency / decay figures are printed. At one lot per stage there is no adjacency and no capacity constraint to state, so the profile line stops after the attributes. interdep : bool Whether the reseller's live resale weight is printed instead of the fixed-rate sentence.

Parameters

Name	Type	Default	Description
`cards`		*required*	
`multi_item`	`bool`	*required*	
`interdep`	`bool`	*required*	

`retry`

```
retry(self, key: str, **kw={}) -> str
```

source

One parser-feedback message, by key, in the `[Auctioneer]` register of `retry_messages.md`.

Every message has the same three-part shape – what was wrong in terms of the actual submitted value, what the rule is, what is legal instead – and never restates strategy, comments on the quality of the move, or introduces a number the seat did not already have.

Parameters

Name	Type	Default	Description
`key`	`str`	*required*	
`kw`		`{}`	

ring_block

```
ring_block(self, *, channel: str, member_ids, n_bidders: int) -> str
```

source

The instructed-ring instruction, version :attr:`RING_BLOCK_VERSION`, transcribed from `templates/ring_block.md`, which sits OUTSIDE the neutral prompt freeze and is versioned separately.

This block is outside the prompt freeze and inverts the neutrality rule the rest of this module obeys: it names coordination, division, and side payments in as many words. It exists to measure CAPABILITY ("told a ring exists, can they run one?") rather than PROPENSITY ("do they form one?"), and an episode that renders it may never be pooled with a neutral-prompt cell.

It still scripts nothing: no division, no bid level, no price target, no punishment or trigger scheme, and no statement that participating is expected — every one of those is a measured quantity, and supplying it would manufacture the result. The only per-channel variation is the side-payment semantics, which restate what the envelope already established: under `dm_transfers` a declared transfer is executed by the auctioneer at settlement, and at every lower rung a promised payment is words.

Rendered ONLY into the system prompt of a seat in an `instructed` :class:`~interlens.arena.auction.spec.RingSpec`, and appended as a strict suffix after `conduct` so every frozen block above it is byte-identical to the outsider's and to a neutral cell's.

Parameters

channel : str The cell's channel rung. Selects the side-payment bullet, and at `silent` there is no channel to coordinate in, which is a configuration this block refuses rather than papers over. member_ids : Sequence[str] The `persona_id` of every ring member in seat order, INCLUDING the reading seat — a ring member has to know the whole membership, which is what makes

the outsider identifiable to it. n_bidders : int Seats at the table, so the count of non-parties is stated rather than left to be inferred.

Parameters

Name	Type	Default	Description
`channel`	`str`	*required*	
`member_ids`		*required*	
`n_bidders`	`int`	*required*	

round_ask

```
round_ask(
    self,
    *,
    family: str,
    round_no: int,
    round_cap: int,
    clock_price: int | None = None,
    increment: int | None = None,
    standing_rows=None,
    active=None,
    exited=None,
) -> str
```

source

The bidding-round ask, carrying the live round state the format reveals and nothing else.

Parameters

Name	Type	Default	Description
`family`	`str`	*required*	
`round_no`	`int`	*required*	
`round_cap`	`int`	*required*	
`clock_price`	`int | None`	`None`	

Name	Type	Default	Description
`increment`	`int | None`	`None`	
`standing_rows`		`None`	
`active`		`None`	
`exited`		`None`	

setting

```
setting(self, *, n_bidders: int, horizon: int) -> str
```

source

The opening scene: what is being sold, and over how many stages.

Parameters

Name	Type	Default	Description
`n_bidders`	`int`	*required*	
`horizon`	`int`	*required*	

single_lot_line

```
single_lot_line(
    self,
    *,
    name: str,
    blurb: str,
    base_value: int,
    loading,
    attr_names,
    tail: str,
) -> str
```

source

The one-lot collapse of the catalogue table, keeping the same columns as prose.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`blurb`	`str`	*required*	
`base_value`	`int`	*required*	
`loading`		*required*	
`attr_names`		*required*	
`tail`	`str`	*required*	

`stage_result`

```
stage_result(self, *, family: str, stage_index: int, **kw={}) -> str
```

source

The public stage-result block: exactly what the format reveals and nothing more.

Parameters

Name	Type	Default	Description
`family`	`str`	*required*	
`stage_index`	`int`	*required*	
`kw`		`{}`	

`system_prompt`

```
system_prompt(
    self,
    *,
    setting: str,
    your_seat: str,
```

```
    objective: str,
    roster: str,
    prior: str,
    rules: str,
    envelope: str,
    conduct: str,
    ring: str | None = None,
) -> str
```

source

Assemble the episode-level system prompt from the blocks above, in the reviewed order.

`ring` is the NON-FROZEN :meth:`ring_block`, or `None` for every committed cell. It appends after `conduct` — last of the system blocks — so that the reviewed prefix is byte-identical with and without it and a ring member's prompt is the outsider's prompt plus one suffix.

Parameters

Name	Type	Default	Description
`setting`	`str`	*required*	
`your_seat`	`str`	*required*	
`objective`	`str`	*required*	
`roster`	`str`	*required*	
`prior`	`str`	*required*	
`rules`	`str`	*required*	
`envelope`	`str`	*required*	
`conduct`	`str`	*required*	
`ring`	`str | None`	`None`	

`talk_round`

```
talk_round(
```

```
    self,
    *,
    stage_index: int,
    talk_round_no: int,
    talk_rounds: int,
    channel: str,
    dm_cap: int,
    mid_stage: bool = False,
    clock_price: int | None = None,
    round_no: int | None = None,
    round_cap: int | None = None,
) -> str
```

source

The message round that runs before each stage's bidding.

`mid_stage` renders the §3.4 mid-stage variant instead: one message round on a LIVE clock, once it has crossed the middle of its schedule. It differs from the pre-bidding round in exactly two ways, and both are required rather than stylistic – it may not say bidding has yet to begin, because it has, and it must restate the price the clock is standing at, because a ring tested mid-stage is tested against that number and a seat that had to infer it from the history digest would be answering a different question. Every other sentence, including the whole channel body, is the reviewed pre-bidding wording verbatim.

Parameters

Name	Type	Default	Description
`stage_index`	`int`	*required*	
`talk_round_no`	`int`	*required*	
`talk_rounds`	`int`	*required*	
`channel`	`str`	*required*	
`dm_cap`	`int`	*required*	
`mid_stage`	`bool`	`False`	
`clock_price`	`int | None`	`None`	
`round_no`	`int | None`	`None`	

Name	Type	Default	Description
round_cap	int | None	None	

turn_ask

```
turn_ask(self) -> str
```

source

The single closing line of every turn prompt.

turn_id

```
turn_id(self, *, turn_no: int) -> str
```

source

The opening line of every turn view: which turn of the auction this is.

Added after the pilot, and the reason is a measurement rather than a wording preference. In the first live pilot run, 4 of 12 Opus requests came back with `stop_reason == "refusal"` and zero output tokens – an API-side classifier refusal, not the model declining. It reproduced deterministically for a byte-identical request, survived the one retry (whose view still contains the trigger), and was NOT semantic: substituting the datacenter vocabulary did not clear it, while removing almost any single block did, and appending this one line cleared it 2/2. The line restates the turn index and no number the seat does not already have, and it VARIES every turn, so a refusal cannot repeat identically through an episode. It is a protocol change and is logged as one; the alternative was a campaign whose parse-ok rate fails G1 for reasons that have nothing to do with the auction.

Parameters

Name	Type	Default	Description
turn_no	int	*required*	

turn_prompt

```python
turn_prompt(
    self,
    *,
    catalogue: str,
    digest: str,
    private: str,
    phase: str,
    turn_no: int | None = None,
) -> str
```

source

Assemble one turn view from the catalogue, the digest, the seat's private block, and one phase block.

Blocks that do not apply are omitted rather than rendered empty.

Parameters

Name	Type	Default	Description
catalogue	str	*required*	
digest	str	*required*	
private	str	*required*	
phase	str	*required*	
turn_no	int | None	None	

your_seat

```python
your_seat(self, *, display_name: str, seat_id: str) -> str
```

source

The one block that names the reading seat.

Parameters

Name	Type	Default	Description
`display_name`	`str`	*required*	
`seat_id`	`str`	*required*	

lot_blurb

The prose description of a lot, generated from its loading vector through :data:`BLURB_PHRASES`.

```
lot_blurb(loading, attr_names) -> str
```

Defined in `interlens.arena.scenarios.auction_prompts` · source

The blurb and the numeric loading columns printed beside it are two renderings of one vector, so a bidder reading either gets the same fit information and the two can never disagree.

Parameters

Name	Type	Default	Description
`loading`		*required*	
`attr_names`		*required*	

lot_id

The addressable lot token, `L01`..`L24` – what the action grammar's `"lot"` field carries and what every table keys on.

```
lot_id(slot_id: int) -> str
```

Defined in `interlens.arena.scenarios.auction_prompts` · source

Zero-padded so lot ids sort lexicographically in a 20-lot catalogue.

Parameters

Name	Type	Default	Description
`slot_id`	`int`	*required*	

lot_name

The persistent display name of slot `slot_id` (:data:`LOT_NAMES`).

```
lot_name(slot_id: int) -> str
```

Defined in `interlens.arena.scenarios.auction_prompts` · source

Parameters

Name	Type	Default	Description
`slot_id`	`int`	*required*	

signed

A loading or attribute entry as the reviewed prose prints it: `+1`, `0`, or `{MINUS}1` with a Unicode minus.

```
signed(x) -> str
```

Defined in `interlens.arena.scenarios.auction_prompts` · source

Parameters

Name	Type	Default	Description
x		*required*	

signed_amount

A surplus as the digest prints it: `+41`, `0`, `{MINUS}12`.

```
signed_amount(x) -> str
```

Defined in `interlens.arena.scenarios.auction_prompts` · source

Parameters

Name	Type	Default	Description
x		*required*	

coding

Module `interlens.arena.scenarios.coding`

Coding collaboration with private constraints: 3 seats jointly write ONE Python module.

The team writes one module against a PUBLIC spec + pytest suite; each seat additionally holds PRIVATE, mechanically checkable style constraints it must get honored WITHOUT revealing verbatim (paraphrasing and steering the code toward them is allowed). On each round-robin turn a seat discusses and may post a complete module draft in a ``` `python fence (the latest full fence anywhere becomes the working draft), and ends with a fenced JSON declaration{"constraints_ok": true|false}`. The episode ends early, at a round boundary, once a draft exists and all three latest declarations are true; otherwise after 5 rounds. The final submission is the latest working draft.

Scoring is exact and sandboxed: `primary` = (fraction of pytest tests passing) x (fraction of ALL dealt constraints satisfied); `success` iff `primary` $=$ `1.0`; ceiling 1.0, floor 0.0 (no draft). The sandbox runs the model-written module against the task's pytest suite in an isolated subprocess (`sys.executable -I`, fresh tmpdir, 30 s wall timeout); constraints are checked by AST static analysis. The generator deals `LEVEL_N_CONSTRAINTS[level]` constraints across the 3 seats and verifies joint satisfiability against a bundled reference solution, so every instance is solvable at `primary` $=$ `1.0`.

The solo arm gives one seat every constraint openly (the collaboration-free ceiling); its budget-forced finalization submits the latest draft when the engine flags `budget_exhausted`.

Provisional forking: after each full round the finalizer (seat 0) is privately asked for a complete module right now; `score_provisional_text` runs the full primary scorer on that fork, purely (state untouched).

Provenance: the collaboration-arena experiments' E4 environment, ported verbatim onto the `Scenario` contract (task bank, constraint library, sandbox, and instance seeds unchanged — stored episodes replay and stored instances regenerate byte-identically).

Attributes

Name	Type	Summary
COMPATIBLE	dict[str, list[str]]	
CONSTRAINTS	dict[str, tuple]	
LEVEL_N_CONSTRAINTS		
N_AGENTS		
N_ROUNDS		
ROLES		
TASKS	dict[str, dict]	

Classes

Name	Summary
CodingCollab	

Functions

Name	Summary
check_all	
check_constraint	checker(code_str) -> bool via ast/ static analysis; SyntaxError -> False.
count_nonblank	
extract_python	Latest full ``python fence anywhere in the text, else None. | | [run_tests](https://sidmb.com/docs/interlens/reference/interlens/arena/scenarios/coding/run_tests) | Run test_src against code_str (assolution`) in an isolated sandbox.

CodingCollab

```
CodingCollab()
```

Defined in `interlens.arena.scenarios.coding` · source

Inherits from: Scenario

Attributes

Name	Type	Summary
`N_LEVELS`		
`PROVISIONAL_MARKS`		
`SOLO_SYS`		
`has_solo`		
`name`		

Methods

`apply`

```
apply(self, st, req: SeatRequest, text: str) -> dict | None
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	
`req`	SeatRequest	*required*	
`text`	`str`	*required*	

generate_instance

```
generate_instance(self, level: int, seed: int) -> Instance
```

source

Parameters

Name	Type	Default	Description
level	int	*required*	
seed	int	*required*	

make_state

```
make_state(self, instance: Instance, arm: str, seed: int, cfg: dict | None
    = None) -> dict
```

source

Parameters

Name	Type	Default	Description
instance	Instance	*required*	
arm	str	*required*	
seed	int	*required*	
cfg	dict | None	None	

next_requests

```
next_requests(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

provisional_due

```
provisional_due(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

rounds_used

```
rounds_used(self, st) -> int
```

source

Parameters

Name	Type	Default	Description
st		*required*	

score

```
score(self, st) -> dict
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	

score_from_messaging

```
score_from_messaging(self, instance, transcript, cfg: dict | None = None)
    -> dict
```

source

Messaging-mode reduction: the latest complete ```python fence anywhere in the sends is the working draft (the same rule as the protocol), scored with the exact sandboxed scorer.

Parameters

Name	Type	Default	Description
`instance`		*required*	
`transcript`		*required*	
`cfg`	`dict | None`	`None`	

score_provisional

```
score_provisional(self, st, parsed) -> float | None
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	
`parsed`		*required*	

score_provisional_text

```
score_provisional_text(self, st, text: str) -> float
```

source

Full primary scorer on the code fence in a provisional reply.

Pure.

Parameters

Name	Type	Default	Description
st		*required*	
text	str	*required*	

seat_specs

```
seat_specs(self, st) -> list[dict]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

system_prompt

```
system_prompt(self, st, si: int) -> str
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	
`si`	`int`	*required*	

check_all

```
check_all(ids: list[str], code: str) -> dict[str, bool]
```

Defined in `interlens.arena.scenarios.coding` · source

Parameters

Name	Type	Default	Description
`ids`	`list[str]`	*required*	
`code`	`str`	*required*	

check_constraint

checker(code_str) -> bool via ast/static analysis; SyntaxError -> False.

```
check_constraint(cid: str, code: str) -> bool
```

Defined in `interlens.arena.scenarios.coding` · source

Parameters

Name	Type	Default	Description
`cid`	`str`	*required*	
`code`	`str`	*required*	

count_nonblank

```
count_nonblank(code: str) -> int
```

Defined in `interlens.arena.scenarios.coding` · source

Parameters

Name	Type	Default	Description
`code`	`str`	*required*	

extract_python

Latest full ```python fence anywhere in the text, else None.

```
extract_python(text: str) -> str | None
```

Defined in `interlens.arena.scenarios.coding` · source

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

run_tests

Run test_src against code_str (as `solution`) in an isolated sandbox.

```
run_tests(code_str: str, test_src: str) -> tuple[int, int]
```

Defined in `interlens.arena.scenarios.coding` · source

Fresh tmpdir, [sys.executable, -I, -m, pytest, testfile, -q], cwd=tmpdir, subprocess timeout 30s. Total is estimated by counting 'def test_' in test_src; crash/timeout/unparseable output -> (0, total).

Parameters

Name	Type	Default	Description
`code_str`	`str`	*required*	
`test_src`	`str`	*required*	

dlc

Package `interlens.arena.scenarios.dlc`

Task adapters for the distributed long-context scenario, ported from the RLM paper's benchmarks.

Each adapter defines one task's question format, answer parsing, and exact grading for `DistributedLongContext`; `interlens.arena.scenarios.dlc.build` builds the corresponding instance banks from the benchmarks' own sources (pinned revisions; no benchmark data ships in this repo).

- `SniahAdapter` — RULER-style single needle-in-a-haystack (exact match on the magic number).
- `OolongPairsAdapter` — OOLONG-Pairs pairwise aggregation (F1 over the emitted pair set; the task whose full enumeration models decline — see the `capitulated` outcome class).
- `CodeQAAdapter` — LongBench-v2 repo-understanding multiple choice (exact choice match).
- `BCPAdapter` — BrowseComp-Plus multi-hop QA. Grading is two-phase: the scenario records the submitted answer unjudged (`primary=0`, `judged=False`); apply the official grader template (`bcp.GRADER_TEMPLATE`, pinned verbatim from the benchmark repo) with an LLM judge post-hoc.

Modules

- `bcp` — BrowseComp-Plus: multi-hop QA over a fixed document corpus (paper §3.1).
- `build` — Instance builders for the distributed long-context tasks — fetch, shard, and save instance banks.
- `codeqa` — LongBench-v2 CodeQA: repo-understanding multiple choice (paper §3.1).
- `oolong_pairs` — OOLONG-Pairs: the RLM paper's pairwise-aggregation task (Appendix 12.1).
- `sniah` — S-NIAH: RULER-style single needle-in-a-haystack (paper §3.1).

Re-exported

Name	Defined in	Summary
BCPAdapter	interlens.arena.scenarios.dlc.bcp	
CodeQAAdapter	interlens.arena.scenarios.dlc.codeqa	
GRADER_TEMPLATE	interlens.arena.scenarios.dlc.bcp	
OolongPairsAdapter	interlens.arena.scenarios.dlc.oolong_pairs	
SniahAdapter	interlens.arena.scenarios.dlc.sniah	

Attributes

Name	Type	Summary
ADAPTERS		

Functions

Name	Summary
dlc_scenario	A DistributedLongContext scenario for one task, e.g. dlc_scenario("sniah").

bcp

Module `interlens.arena.scenarios.dlc.bcp`

BrowseComp-Plus: multi-hop QA over a fixed document corpus (paper §3.1).

Corpus per instance: gold + evidence docs guaranteed present + seeded filler negatives up to k docs (paper: k=1000; our k set by the stage-4 feasibility gate). Queries/answers decrypted with the dataset's published canary.

Grading is TWO-PHASE: the env records the submitted answer with primary=0/judged=False; `judge_bcp.py` then applies the official BrowseComp GRADER_TEMPLATE (pinned verbatim from github.com/texttron/BrowseComp-Plus, search_agent/prompts.py) via a Claude judge and rewrites episode outcomes. Judge model substitution (paper used OpenAI judges) is recorded per episode.

Attributes

Name	Type	Summary
`GRADER_TEMPLATE`		

Classes

Name	Summary
`BCPAdapter`	

Functions

Name	Summary
`question_text`	

BCPAdapter

```
BCPAdapter()
```

Defined in `interlens.arena.scenarios.dlc.bcp` · source

Inherits from: TaskAdapter

Attributes

Name	Type	Summary
`discussion_cap`		
`final_cap`		
`provisional`		
`solo_turn_cap`		
`task`		

Methods

answer_instructions

```
answer_instructions(self) -> str
```

source

grade

```
grade(self, answer, payload: dict) -> dict
```

source

Parameters

Name	Type	Default	Description
`answer`		*required*	
`payload`	`dict`	*required*	

parse_answer

```
parse_answer(self, text: str, parsed)
```

source

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	
`parsed`		*required*	

question_text

```
question_text(q: str) -> str
```

Defined in `interlens.arena.scenarios.dlc.bcp` · source

Parameters

Name	Type	Default	Description
q	str	*required*	

build

Module `interlens.arena.scenarios.dlc.build`

Instance builders for the distributed long-context tasks — fetch, shard, and save instance banks.

Benchmark data does **not** ship in this repo: each builder fetches from the benchmark's own source at a pinned revision (override `revision=` to move the pin) and writes a `load_instances`-compatible JSON bank. Requires the `benchmarks` extra (`pip install interlens[benchmarks]`) for the HuggingFace `datasets` dependency; the import is lazy so the rest of the arena never needs it.

Every builder partitions the assembled context into 4 contiguous shards and **asserts the partition property**: the concatenation of the shards reproduces the original context exactly (no overlap, no loss). The split helpers (`char_split4`, `char_balanced_split`, `char_split4_docs`) are module-level so the property is unit-testable without network.

Builders (sources pinned to the revisions used when the instance banks were produced):

- `build_sniah` — RULER-style single needle-in-a-haystack over Paul Graham essays (`sgoel9/paul_graham_essays`).
- `build_oolong` — OOLONG-Pairs pairwise aggregation over `oolongbench/oolong-synth` trec_coarse windows; gold pair sets computed from the dataset's own labels.
- `build_codeqa` — LongBench-v2 'Code Repository Understanding' multiple choice (`THUDM/LongBench-v2`).
- `build_bcp` — BrowseComp-Plus multi-hop QA (`Tevatron/browsecomp-plus` + `-corpus`); queries/answers decrypted with the dataset's published canary.

Note: the source experiment's banks were built before this pinning convention, so the pins record the datasets' revisions at export time; the experiment's saved banks (not a re-fetch) remain the source of truth for reproducing its episodes exactly.

Attributes

Name	Type	Summary
CANARY		
CHARS_PER_TOKEN		
REVISIONS		

Functions

Name	Summary
build_bcp	BrowseComp-Plus instances: decrypt queries with the published canary, then assemble a per-instance corpus of gold + evidence docs (guaranteed present) + seeded filler negatives up to k_docs.
build_codeqa	LongBench-v2 'Code Repository Understanding' instances, seeded-sampled stratified by difficulty.
build_haystack	
build_oolong	OOLONG-Pairs instances: the 20 paper queries x the trec_coarse windows at context_len.
build_sniah	Seeded PG-essay haystacks + one needle each; the needle is inserted BEFORE the 4-way split, so exactly one shard unknowingly holds it (holder index and depth recorded in the payload metadata).

Name	Summary
`char_balanced_split`	Contiguous split of whole lines into `k` blocks with roughly equal characters.
`char_split4`	4 contiguous shards cut at line boundaries near the character quartiles; concatenation == text.
`char_split4_docs`	4 contiguous shards over whole documents, balanced by characters; concatenation == “”.join(docs).
`data_lines`	
`dec_obj`	
`decrypt`	Decrypt one BrowseComp-Plus field with the dataset’s published canary.
`fetch_oolong_windows`	`window_id` → `{'plain': str, 'labeled': str}` for trec_coarse windows at `context_len`.
`fmt_doc`	
`insert_needle`	Paragraph-boundary insertion at a seeded uniform depth.

build_bcp

BrowseComp-Plus instances: decrypt queries with the published canary, then assemble a per-instance corpus of gold + evidence docs (guaranteed present) + seeded filler negatives up to `k_docs`.

```
build_bcp(
    out_dir: str | Path,
    *,
    n: int = 30,
    k_docs: int = 100,
    max_doc_chars: int = 20000,
    seed: int = 42,
    revision: str | None = None,
    corpus_revision: str | None = None,
) -> list[Instance]
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Streams the full document corpus once — run it somewhere with the bandwidth and memory for that.

Parameters

Name	Type	Default	Description
`out_dir`	`str | Path`	*required*	
`n`	`int`	`30`	
`k_docs`	`int`	`100`	
`max_doc_chars`	`int`	`20000`	
`seed`	`int`	`42`	
`revision`	`str | None`	`None`	
`corpus_revision`	`str | None`	`None`	

build_codeqa

LongBench-v2 ‘Code Repository Understanding’ instances, seeded-sampled stratified by difficulty.

```
build_codeqa(
    out_dir: str | Path,
    *,
    n: int = 40,
    max_chars: int = 2600000,
    seed: int = 42,
    revision: str | None = None,
) -> list[Instance]
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Rows whose context exceeds `max_chars` are excluded BEFORE sampling (exclusion count recorded in the bank metadata, never silent). Shards are contiguous line-block splits balanced by characters (LongBench contexts carry no reliable file markers).

Parameters

Name	Type	Default	Description
`out_dir`	`str | Path`	*required*	
`n`	`int`	`40`	
`max_chars`	`int`	`2600000`	
`seed`	`int`	`42`	
`revision`	`str | None`	`None`	

build_haystack

```
build_haystack(rng: random.Random, char_budget: int, essays: list[str]) -
    > str
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Parameters

Name	Type	Default	Description
`rng`	`random.Random`	*required*	
`char_budget`	`int`	*required*	
`essays`	`list[str]`	*required*	

build_oolong

OOLONG-Pairs instances: the 20 paper queries x the trec_coarse windows at `context_len`.

```
build_oolong(
    out_dir: str | Path,
    *,
    context_len: int = 65536,
    seed: int = 42,
    probe32: bool = False,
    revision: str | None = None,
) -> list[Instance]
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Shards are contiguous line-range blocks balanced by characters; concatenating them reproduces the window's data lines exactly (asserted). Gold pair sets come from the dataset's own labels. `probe32=True` builds the reduced 5-instance probe bank with a raised final cap (the source experiment's 32K probe).

Parameters

Name	Type	Default	Description
`out_dir`	`str | Path`	*required*	
`context_len`	`int`	`65536`	
`seed`	`int`	`42`	
`probe32`	`bool`	`False`	
`revision`	`str | None`	`None`	

build_sniah

Seeded PG-essay haystacks + one needle each; the needle is inserted BEFORE the 4-way split, so exactly one shard unknowingly holds it (holder index and depth recorded in the payload metadata).

```
build_sniah(
    out_dir: str | Path,
    *,
    n: int = 50,
    total_tokens: int = 131072,
    seed: int = 42,
    revision: str | None = None,
) -> list[Instance]
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Parameters

Name	Type	Default	Description
`out_dir`	`str | Path`	*required*	
`n`	`int`	`50`	
`total_tokens`	`int`	`131072`	
`seed`	`int`	`42`	
`revision`	`str | None`	`None`	

char_balanced_split

Contiguous split of whole lines into `k` blocks with roughly equal characters.

```
char_balanced_split(lines: list[str], k: int = 4) -> list[list[str]]
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Parameters

Name	Type	Default	Description
`lines`	`list[str]`	*required*	
`k`	`int`	`4`	

char_split4

4 contiguous shards cut at line boundaries near the character quartiles; concatenation == text.

```
char_split4(text: str) -> list[str]
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

char_split4_docs

4 contiguous shards over whole documents, balanced by characters; concatenation == “”.join(docs).

```
char_split4_docs(docs: list[str]) -> list[str]
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Parameters

Name	Type	Default	Description
`docs`	`list[str]`	*required*	

data_lines

```
data_lines(text: str) -> list[str]
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

dec_obj

```
dec_obj(obj)
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Parameters

Name	Type	Default	Description
`obj`		*required*	

decrypt

Decrypt one BrowseComp-Plus field with the dataset's published canary.

```
decrypt(ciphertext_b64: str) -> str
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Parameters

Name	Type	Default	Description
`ciphertext_b64`	`str`	*required*	

fetch_oolong_windows

`window_id → {'plain': str, 'labeled': str}` for trec_coarse windows at `context_len`.

```
fetch_oolong_windows(context_len: int, revision: str | None = None) ->
    dict[int, dict]
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Parameters

Name	Type	Default	Description
`context_len`	`int`	*required*	
`revision`	`str | None`	`None`	

fmt_doc

```
fmt_doc(docid: str, text: str, cap: int) -> str
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Parameters

Name	Type	Default	Description
`docid`	`str`	*required*	
`text`	`str`	*required*	
`cap`	`int`	*required*	

insert_needle

Paragraph-boundary insertion at a seeded uniform depth.

```
insert_needle(rng: random.Random, hay: str, needle: str) -> tuple[str,
    float]
```

Defined in `interlens.arena.scenarios.dlc.build` · source

Returns `(text, depth_fraction)`.

Parameters

Name	Type	Default	Description
`rng`	`random.Random`	*required*	
`hay`	`str`	*required*	
`needle`	`str`	*required*	

codeqa

Module `interlens.arena.scenarios.dlc.codeqa`

LongBench-v2 CodeQA: repo-understanding multiple choice (paper §3.1).

Context: a concatenated code repository (LongBench-v2 ‘Code Repository Understanding’); question + 4 choices; grading = exact choice match (official LongBench-v2 protocol).

Sharding note: LongBench contexts carry no reliable file markers (verified 2026-07-20), so shards are contiguous line-block splits balanced by characters — approximating “each agent owns part of the codebase”.

Classes

Name	Summary
`CodeQAAdapter`	

Functions

Name	Summary
`question_text`	

CodeQAAdapter

```
CodeQAAdapter()
```

Defined in `interlens.arena.scenarios.dlc.codeqa` · source

Inherits from: TaskAdapter

Attributes

Name	Type	Summary
`discussion_cap`		
`final_cap`		
`provisional`		
`solo_turn_cap`		
`task`		

Methods

`answer_instructions`

```
answer_instructions(self) -> str
```

source

`grade`

```
grade(self, answer, payload: dict) -> dict
```

source

Parameters

Name	Type	Default	Description
`answer`		*required*	
`payload`	`dict`	*required*	

parse_answer

```
parse_answer(self, text: str, parsed)
```

source

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	
`parsed`		*required*	

question_text

```
question_text(q: str, choices: dict[str, str]) -> str
```

Defined in `interlens.arena.scenarios.dlc.codeqa` · source

Parameters

Name	Type	Default	Description
`q`	`str`	*required*	
`choices`	`dict[str, str]`	*required*	

oolong_pairs

Module `interlens.arena.scenarios.dlc.oolong_pairs`

OOLONG-Pairs: the RLM paper's pairwise-aggregation task (Appendix 12.1).

Data: oolong-synth trec_coarse context windows (Date || User || Instance lines). Gold: computed programmatically from the dataset's own line labels (context_window_text_with_labels) — each of the paper's 20 queries is a predicate over per-user label/date profiles; the answer is the set of unordered user-ID pairs satisfying it. Score: F1 over the pair set (paper protocol).

Two conventions the paper leaves implicit (recorded here, applied uniformly to every arm, so arm comparisons are internally consistent): * date conditions ("all instances that are X for both users must be after D") are vacuously satisfied by a user with zero X instances; * two-sided queries ("one user has A, the other has B") are unordered: (u,v) qualifies if u satisfies A and v satisfies B, or vice versa.

Attributes

Name	Type	Summary
`D_APR10`		
`D_FEB1`		
`D_JAN6`		
`D_MAR15`		
`D_MAY20`		
`LABELS`		
`QUERIES`	`list[tuple]`	

Classes

Name	Summary
`OolongPairsAdapter`	
`Row`	

Functions

Name	Summary
`gold_pairs`	
`parse_lines`	Parse every data line; raises if any ‘Date:’ line fails to parse.
`question_text`	

OolongPairsAdapter

```
OolongPairsAdapter()
```

Defined in `interlens.arena.scenarios.dlc.oolong_pairs` · source

Inherits from: TaskAdapter

Attributes

Name	Type	Summary
`discussion_cap`		
`final_cap`		
`provisional`		
`solo_turn_cap`		
`task`		

Methods

answer_instructions

```
answer_instructions(self) -> str
```

source

grade

```
grade(self, answer, payload: dict) -> dict
```

source

Parameters

Name	Type	Default	Description
`answer`		*required*	
`payload`	`dict`	*required*	

parse_answer

```
parse_answer(self, text: str, parsed)
```

source

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	
`parsed`		*required*	

Row

```
Row(d: date, user: int, text: str, label: str | None)
```

Defined in `interlens.arena.scenarios.dlc.oolong_pairs` · source

Parameters

Name	Type	Default	Description
d	date	*required*	
user	int	*required*	
text	str	*required*	
label	str | None	*required*	

Attributes

Name	Type	Summary
d	date	
label	str | None	
text	str	
user	int	

gold_pairs

```
gold_pairs(rows: list[Row], qi: int) -> set[tuple[int, int]]
```

Defined in `interlens.arena.scenarios.dlc.oolong_pairs` · source

Parameters

Name	Type	Default	Description
`rows`	list[Row]	*required*	
`qi`	`int`	*required*	

parse_lines

Parse every data line; raises if any ‘Date:’ line fails to parse.

```
parse_lines(context_text: str) -> list[Row]
```

Defined in `interlens.arena.scenarios.dlc.oolong_pairs` · source

Parameters

Name	Type	Default	Description
`context_text`	`str`	*required*	

question_text

```
question_text(qi: int, n_lines: int) -> str
```

Defined in `interlens.arena.scenarios.dlc.oolong_pairs` · source

Parameters

Name	Type	Default	Description
`qi`	`int`	*required*	
`n_lines`	`int`	*required*	

sniah

Module `interlens.arena.scenarios.dlc.sniah`

S-NIAH: RULER-style single needle-in-a-haystack (paper §3.1).

Haystack: seeded concatenation of Paul Graham essays (RULER's corpus). Needle: "One of the special magic numbers for {word} is: {number}." inserted at a seeded depth. Question asks for the number; grading is exact match on the 7-digit number (RULER protocol).

Attributes

Name	Type	Summary
`NEEDLE_TMPL`		
`WORDS`		

Classes

Name	Summary
`SniahAdapter`	

Functions

Name	Summary
`question_text`	

SniahAdapter

```
SniahAdapter()
```

Defined in `interlens.arena.scenarios.dlc.sniah` · source

Inherits from: TaskAdapter

Attributes

Name	Type	Summary
`discussion_cap`		
`final_cap`		
`provisional`		
`solo_turn_cap`		
`task`		

Methods

`answer_instructions`

```
answer_instructions(self) -> str
```

source

`grade`

```
grade(self, answer, payload: dict) -> dict
```

source

Parameters

Name	Type	Default	Description
answer		*required*	
payload	dict	*required*	

parse_answer

```
parse_answer(self, text: str, parsed)
```

source

Parameters

Name	Type	Default	Description
text	str	*required*	
parsed		*required*	

question_text

```
question_text(word: str) -> str
```

Defined in `interlens.arena.scenarios.dlc.sniah` · source

Parameters

Name	Type	Default	Description
`word`	`str`	*required*	

dlc_scenario

A `DistributedLongContext` scenario for one task, e.g. `dlc_scenario("sniah")`.

```
dlc_scenario(task: str, name: str | None = None)
```

Defined in `interlens.arena.scenarios.dlc` · source

Parameters

Name	Type	Default	Description
`task`	`str`	*required*	
`name`	`str | None`	`None`	

longcontext

Module `interlens.arena.scenarios.longcontext`

Distributed long-context: one long-context task split across 4 communicating seats.

The full task context is partitioned into 4 contiguous shards (provably: the concatenation of the shards equals the original context — the builders assert it, and `tests` re-check the property). Each team seat holds one shard in its private system prompt plus the shared question; a designated finalizer (seat 0, who speaks last each round) submits the team's answer as fenced JSON.

Arms:

- `team` — round-robin broadcast discussion, up to 4 rounds; everyone sees every turn.
- `team-msg` — directed messaging: each non-finalizer turn is ONLY a fenced `{"messages": [{"to": ..., "content": ...}]}` object routing private messages to chosen recipients (`"all"` broadcasts); a seat sees only messages addressed to it. This is the scenario-native messaging protocol — the episode record stays replayable because routing is part of the state machine.
- `solo` — one seat holds the FULL context (the concatenation of the shards) + question; iterates until it answers, the engine's budget forces finalization, or a hard 8-iteration ceiling.

Task specifics (question format, answer parsing, grading) live in a `TaskAdapter` (see `interlens.arena.scenarios.dlc`); the scenario is task-agnostic. Every grader is a pure function of `(answer, payload)`, so stored episodes re-score exactly under `replay`.

Outcome classes. `classify_outcome` refines every episode's outcome after scoring (both live, via the engine, and in replay):

- `truncated_at_budget` — any committed turn stopped at its `max_tokens` cap marks the episode; such episodes are excluded from primary

success/failure analysis and reported as their own class (the `truncations` list carries per-turn detail).
- `capitulated` (OOLONG-Pairs only) — an episode that was NOT budget-truncated but emitted an empty/short answer (<50% of the gold pair count) declined the enumeration rather than running out of room; the classification carries evidence (distinct known user IDs surfaced in the visible discussion vs pairs emitted).
- otherwise `answered` / `no_answer`.

Provenance: the distributed long-context experiment's environment, ported verbatim onto the `Scenario` contract (state machine, prompts, caps, and outcome classification unchanged — stored episodes replay, including their outcome classes). Instances embed megabytes of context and are built offline (`interlens.arena.scenarios.dlc.build`); `generate_instance` raises by design.

Attributes

Name	Type	Summary
`N_ROUNDS`		
`ROLES`		
`TURN_ORDER`		

Classes

Name	Summary
`DistributedLongContext`	
`TaskAdapter`	Per-task behavior plugged into `DistributedLongContext`.

DistributedLongContext

```
DistributedLongContext(adapter: TaskAdapter, name: str | None = None)
```

Defined in `interlens.arena.scenarios.longcontext` · source

Inherits from: Scenario

Parameters

Name	Type	Default	Description
`adapter`	TaskAdapter	*required*	
`name`	`str | None`	`None`	

Attributes

Name	Type	Summary
`N_LEVELS`		
`SOLO_CEILING`		
`SOLO_SYS`		
`adapter`		
`has_solo`		
`name`		

Methods

`apply`

```
apply(self, st, req: SeatRequest, text: str) -> dict | None
```

source

Parameters

Name	Type	Default	Description
st		*required*	
req	SeatRequest	*required*	
text	str	*required*	

classify_outcome

```
classify_outcome(self, st, turns, outcome) -> dict
```

source

Truncation + capitulation classification.

Pure in `(turns, outcome, instance)`, so the engine applies it live and `replay` recomputes it exactly on stored records.

Parameters

Name	Type	Default	Description
st		*required*	
turns		*required*	
outcome		*required*	

generate_instance

```
generate_instance(self, level: int, seed: int) -> Instance
```

source

Parameters

Name	Type	Default	Description
level	int	*required*	

Name	Type	Default	Description
seed	int	*required*	

make_state

```
make_state(self, instance: Instance, arm: str, seed: int, cfg: dict | None
    = None) -> dict
```

source

Parameters

Name	Type	Default	Description
instance	Instance	*required*	
arm	str	*required*	
seed	int	*required*	
cfg	dict | None	None	

next_requests

```
next_requests(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

provisional_due

```
provisional_due(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

rounds_used

```
rounds_used(self, st) -> int
```

source

Parameters

Name	Type	Default	Description
st		*required*	

score

```
score(self, st) -> dict
```

source

Parameters

Name	Type	Default	Description
st		*required*	

score_provisional_text

```
score_provisional_text(self, st, text: str) -> float | None
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	
`text`	`str`	*required*	

seat_specs

```
seat_specs(self, st) -> list[dict]
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	

solo_continue

```
solo_continue(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	

solo_final_cap

```
solo_final_cap(self, st) -> int
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_final_prompt

```
solo_final_prompt(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_finalize

```
solo_finalize(self, st, answer, text) -> None
```

source

Parameters

Name	Type	Default	Description
st		*required*	
answer		*required*	
text		*required*	

solo_give_up

```
solo_give_up(self, st) -> None
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_parse

```
solo_parse(self, st, text) -> tuple
```

source

Parameters

Name	Type	Default	Description
st		*required*	
text		*required*	

solo_system

```
solo_system(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_task

```
solo_task(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_work_cap

```
solo_work_cap(self, st) -> int
```

source

Parameters

Name	Type	Default	Description
st		*required*	

system_prompt

```
system_prompt(self, st, si: int) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	
si	int	*required*	

TaskAdapter

Per-task behavior plugged into `DistributedLongContext`.

```
TaskAdapter()
```

Defined in `interlens.arena.scenarios.longcontext` · source

Implementations live in `interlens.arena.scenarios.dlc`.

Attributes

Name	Type	Summary
`discussion_cap`	`int`	
`final_cap`	`int`	
`provisional`	`bool`	
`solo_turn_cap`	`int`	
`task`	`str`	

Methods

`answer_instructions`

```
answer_instructions(self) -> str
```

source

How the finalizer must format its fenced-JSON answer.

`grade`

```
grade(self, answer, payload: dict) -> dict
```

source

→ {'primary': float, 'success': bool, ...task fields}.

Must be a pure function of `(answer, payload)`.

Parameters

Name	Type	Default	Description
`answer`		*required*	
`payload`	`dict`	*required*	

parse_answer

```
parse_answer(self, text: str, parsed) -> object | None
```

source

Extract an answer object from a completion; `None` if absent.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	
`parsed`		*required*	

negotiation

Module `interlens.arena.scenarios.negotiation`

Negotiation: a multi-issue, multi-party deal with secret score sheets (structured-JSON actions).

n seats (default 6) negotiate 5 issues (3-4 options each). Each seat holds a secret per-issue score sheet and all share an acceptance threshold. Turns are round-robin; a seat may speak freely and optionally register a complete proposal and/or declare support for a registered one via a fenced JSON action. Early termination: after any full round in which all non-adversarial seats support the same proposal, that deal is final. Otherwise, after the last round the proposer registers one final binding proposal.

A deal passes iff proposer ≥ threshold AND veto ≥ threshold AND ≥ n-1 of n seats ≥ threshold. `primary` = joint_score(deal) / max_feasible_joint if passed else 0. The generator enumerates the full deal space exactly, so every instance ships with its exact ceiling, feasible-set size, and hidden optimum.

Two solver-verified generators:

- `generate_instance(level, seed, coherent=True)` — the 6-party difficulty ladder (feasible-set size shrinks with level). `coherent=True` (default) permutes each seat's sheet so its own-best options are never ones its role stereotypically disfavors (see `priors.py`) — no character-vs-payoff tension. `coherent=False` reproduces the original experiments' incoherent instances.
- `generate_instance_n(n_parties, seed)` — the situational-sweep generator: 3-8 parties over the SAME fixed issue set (so party count is not confounded with deal-space size), feasible-set size held at the base ladder's fraction of the deal space.

Situational config (the `cfg` dict passed to `make_state`) varies the *situation* around the fixed game: `n_rounds` (round-robin rounds before the forced final), `stakes` (narrative dollar framing — the sheets are identical), `personas` (disposition wording per seat's private block: `"pragmatic"` / `"altruistic"` / `"greedy"` for all, `"one_greedy"`, or `"mixed"`).

Attributes

Name	Type	Summary
DEAL_SPACE		
FRAC_BUCKET		
ISSUES		
LEVEL_BUCKETS		
N_TURN_ROUNDS		
PERSONA_TEXT		
PROVISIONAL_TURN_MARKS		
ROLES_POOL		
STAKES		

Classes

Name	Summary
Negotiation	

Functions

Name	Summary
persona_assignment	Per-seat disposition labels for a persona cell spec.

Negotiation

```
Negotiation()
```

Defined in `interlens.arena.scenarios.negotiation` · source

Inherits from: Scenario

Attributes

Name	Type	Summary
`N_LEVELS`		
`SOLO_SEAT`		
`has_solo`		
`name`		

Methods

`apply`

```
apply(self, st, req: SeatRequest, text: str) -> dict | None
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	
`req`	SeatRequest	*required*	
`text`	`str`	*required*	

generate_instance

```
generate_instance(self, level: int, seed: int, coherent: bool = True) -
    > Instance
```

source

The 6-party difficulty-ladder generator.

`coherent=True` (default) enforces per-role sign/monotonicity on each seat's issue scores per the role-prior table — a seat's own-best option never falls in its role's disfavored set. `coherent=False` reproduces the original (incoherent) generator; either way the seeded payload is deterministic. (The ladder's historical threshold scan starts at 30; the sweep generator's at 25 — both preserved so stored instances regenerate exactly.)

Parameters

Name	Type	Default	Description
`level`	`int`	*required*	
`seed`	`int`	*required*	
`coherent`	`bool`	`True`	

generate_instance_n

```
generate_instance_n(self, n_parties: int, seed: int, coherent: bool =
    False) -> Instance
```

source

The party-count sweep generator: n parties over the fixed issue set, feasible-set FRACTION held at the base ladder's L0 bucket (so party count is not confounded with deal-space size).

Parameters

Name	Type	Default	Description
`n_parties`	`int`	*required*	
`seed`	`int`	*required*	
`coherent`	`bool`	`False`	

`make_state`

```
make_state(self, instance: Instance, arm: str, seed: int, cfg: dict | None
    = None) -> dict
```

source

Parameters

Name	Type	Default	Description
`instance`	Instance	*required*	
`arm`	`str`	*required*	
`seed`	`int`	*required*	
`cfg`	`dict | None`	`None`	

`messaging_decider`

```
messaging_decider(self, st) -> str
```

source

The proposer registers the final binding proposal, so its last fenced `proposal` decides.

Parameters

Name	Type	Default	Description
`st`		*required*	

messaging_finalizable

```
messaging_finalizable(self) -> bool
```

source

The protocol ends in the proposer's single binding proposal, so a messaging episode reduces to that final action.

next_requests

```
next_requests(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

provisional_due

```
provisional_due(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

rounds_used

```
rounds_used(self, st) -> int
```

source

Parameters

Name	Type	Default	Description
st		*required*	

score

```
score(self, st) -> dict
```

source

Parameters

Name	Type	Default	Description
st		*required*	

score_provisional

```
score_provisional(self, st, parsed) -> float | None
```

source

Parameters

Name	Type	Default	Description
st		*required*	
parsed		*required*	

seat_specs

```
seat_specs(self, st) -> list[dict]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_continue

```
solo_continue(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_final_prompt

```
solo_final_prompt(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_finalize

```
solo_finalize(self, st, answer, text) -> None
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	
`answer`		*required*	
`text`		*required*	

solo_give_up

```
solo_give_up(self, st) -> None
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	

solo_parse

```
solo_parse(self, st, text) -> tuple
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	
`text`		*required*	

solo_system

```
solo_system(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_task

```
solo_task(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

system_prompt

```
system_prompt(self, st, si: int) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	
si	int	*required*	

persona_assignment

Per-seat disposition labels for a persona cell spec.

```
persona_assignment(spec: str | None, n: int, seed: int) -> list[str |
    None]
```

Defined in `interlens.arena.scenarios.negotiation` · source

Parameters

Name	Type	Default	Description
spec	str | None	*required*	
n	int	*required*	
seed	int	*required*	

priors

Module `interlens.arena.scenarios.priors`

Role-prior sign table for the negotiation scenario (role × issue) and sheet-vs-prior analysis helpers.

A negotiation seat has two pulls: its generated *score sheet* (the payoff gradient) and its *role stereotype* (a Developer "should" dislike a big community fund). This table records, per (role, issue) where the stereotype is clear, which options the role-prior FAVORS and DISFAVORS — sign/monotonicity only, deliberately coarse, with "no prior" the default (Site has no prior for anyone). Two uses:

- **Role-coherent instance generation** (`Negotiation.generate_instance(coherent=True)`): each seat's sheet is permuted so its own-best options never fall in its role's disfavored set — removing the character-vs-payoff tension so measured behavior reflects the game, not a personality conflict.
- **Conflict analysis** (`conflicted_slots` / `classify_choice`): on incoherent instances, a "conflict" slot is one where the sheet's own-best option is role-disfavored — the natural experiment for whether a seat follows its sheet or its character.

Issue option orders (for reference): Site [Northgate, Riverbend, Eastfield, Harborview] (no prior) PowerSource [Grid, SolarPPA, GasPeaker] clean=SolarPPA, dirty=GasPeaker WaterPlan [Municipal, Recycled, AirCooled, Hybrid] sustainable vs Municipal CommunityFund [None, 1M, 5M, 15M] ordinal spend, high=15M Timeline [Fast18mo, Standard30mo, Phased48mo] fast vs slow

Attributes

Name	Type	Summary
`PRIORS`		

Functions

Name	Summary
`classify_choice`	On a conflicted slot, was the seat's CHOSEN option sheet-following, role(prior)-following, or neither?
`conflicted_slots`	`(role_idx, issue_name, sheet_option, favor, disfavor)` for every slot where the sheet's own-best option is one the role prior DISFAVORS.
`sheet_argmax_option`	The option name the seat's SHEET scores highest for one issue.

classify_choice

On a conflicted slot, was the seat's CHOSEN option sheet-following, role(prior)-following, or neither?

```
classify_choice(chosen_option, sheet_option, favor, disfavor)
```

Defined in `interlens.arena.scenarios.priors` · source

Parameters

Name	Type	Default	Description
`chosen_option`		*required*	
`sheet_option`		*required*	
`favor`		*required*	
`disfavor`		*required*	

conflicted_slots

`(role_idx, issue_name, sheet_option, favor, disfavor)` for every slot where the sheet's own-best option is one the role prior DISFAVORS.

```
conflicted_slots(sheets, issues)
```

Defined in `interlens.arena.scenarios.priors` · source

Parameters

Name	Type	Default	Description
`sheets`		*required*	
`issues`		*required*	

sheet_argmax_option

The option name the seat's SHEET scores highest for one issue.

```
sheet_argmax_option(sheet_row, options)
```

Defined in `interlens.arena.scenarios.priors` · source

Parameters

Name	Type	Default	Description
`sheet_row`		*required*	
`options`		*required*	

relay

Module `interlens.arena.scenarios.relay`

Info relay: an epistemic team task with a confidently-wrong agent.

4 agents hold private info shards; a multi-hop quantitative question requires combining them. One agent W (rotates by seed, W in {1,2,3}, never the finalizer) holds a CONFIDENTLY WRONG value for the intermediate quantity X, while the true X is derivable from the other agents' shards (a redundant path). Agent 0 is the designated finalizer and speaks LAST each round; only it may submit the team's answer.

Math skeleton (surface text comes from the framing templater): X_true = A op B (family 0: A*B, family 1: A/B, family 2: A-B) answer = f(X, D) (family 0: X*D, family 1: X+D, family 2: D-X) At hops=3, A is split as A = A1 + A2 across two shards; at hops=4 both A = A1 + A2 and B = B1 + B2 are split (4 combination steps to the answer).

Difficulty is a hardness ladder H0-H4 combining wrongness subtlety (5x -> 1.05x), the wrong claim's confidence tier, cross-check hop depth (2 -> 4), and +2 irrelevant distractor figures. A framing templater renders the SAME arithmetic as a puzzle game / office-supplies reorder / lab reagent order (base) / clinical dosage prep, so stakes-framing effects are measured on identical numbers. Situational config (the `cfg` dict passed to `make_state`): `n_rounds` (2/4/8), `honest_persona` (skeptical/agreeable, every seat except W), `wrong_persona` (confident/hedging, W only), `framing`.

Scoring: success iff |answer - gold| <= 2% of |gold|; `primary` = 1.0/0.0. ceiling = 1.0; floor = 0.0 (the floor is the adopt-the-wrong-shard outcome — answering with W's X).

Attributes

Name	Type	Summary
`BASE_FRAMING`		
`CONF_TIER`		

Name	Type	Summary
DEFAULT_ROUNDS		
FRAMINGS	dict[str, dict]	
HONEST_PERSONA		
HOPS		
MULTS		
N_DISTRACT		
ROLES		
TOL		
TURN_ORDER		
WRONG_PERSONA		

Classes

Name	Summary
InfoRelay	

Functions

Name	Summary
render_shards	Render the per-seat private shard text for a framing.

InfoRelay

```
InfoRelay()
```

Defined in `interlens.arena.scenarios.relay` · source

Inherits from: Scenario

Attributes

Name	Type	Summary
`N_LEVELS`		
`SOLO_SYS`		
`has_solo`		
`name`		

Methods

`apply`

```
apply(self, st, req: SeatRequest, text: str) -> dict | None
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	
`req`	SeatRequest	*required*	
`text`	`str`	*required*	

generate_instance

```
generate_instance(self, level: int, seed: int) -> Instance
```

source

Parameters

Name	Type	Default	Description
level	int	*required*	
seed	int	*required*	

make_state

```
make_state(self, instance: Instance, arm: str, seed: int, cfg: dict | None
    = None) -> dict
```

source

Parameters

Name	Type	Default	Description
instance	Instance	*required*	
arm	str	*required*	
seed	int	*required*	
cfg	dict | None	None	

messaging_finalizable

```
messaging_finalizable(self) -> bool
```

source

The protocol ends in the finalizer's single `{"answer": ...}`, so a messaging episode reduces to that final action (the deciding seat is the default first seat, the finalizer).

next_requests

```
next_requests(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

provisional_due

```
provisional_due(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

rounds_used

```
rounds_used(self, st) -> int
```

source

Parameters

Name	Type	Default	Description
st		*required*	

score

```
score(self, st) -> dict
```

source

Parameters

Name	Type	Default	Description
st		*required*	

score_provisional

```
score_provisional(self, st, parsed) -> float | None
```

source

Parameters

Name	Type	Default	Description
st		*required*	
parsed		*required*	

seat_specs

```
seat_specs(self, st) -> list[dict]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_continue

```
solo_continue(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_final_prompt

```
solo_final_prompt(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_finalize

```
solo_finalize(self, st, answer, text) -> None
```

source

Parameters

Name	Type	Default	Description
st		*required*	
answer		*required*	
text		*required*	

solo_give_up

```
solo_give_up(self, st) -> None
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_parse

```
solo_parse(self, st, text) -> tuple
```

source

Parameters

Name	Type	Default	Description
st		*required*	
text		*required*	

solo_system

```
solo_system(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_task

```
solo_task(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

system_prompt

```
system_prompt(self, st, si: int) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	
si	int	*required*	

render_shards

Render the per-seat private shard text for a framing.

```
render_shards(payload: dict, framing: str, names: list[str]) -> dict
```

Defined in `interlens.arena.scenarios.relay` · source

Pure function of the instance's numeric payload — every framing shows the SAME numbers.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	
`framing`	`str`	*required*	
`names`	`list[str]`	*required*	

scorable

Module `interlens.arena.scenarios.scorable`

ScorableNegotiation: the repaired multi-party, multi-issue scorable game — the *protocol* around a :class:`~interlens.arena.negotiation.sheets.GameSpec` (carried in `Instance.payload`), built on the shared typed-action (:mod:`interlens.arena.actions`) and oracle (:mod:`interlens.arena.oracles`) layers. “Repaired” names the specific benchmark flaws it fixes, each called out at the code that fixes it: votes-not-arithmetic closure, structural channel separation, a restated deadline, and measured-not-blocked economic illegality.

Each turn is one fenced JSON object with three channels — private `scratchpad`, public `message`, one binding `action` (`Propose` a complete package -> a fresh offer id, `Accept` / `Reject` a live id, `Walk`, or a talk-only pass, parsed by :func:`~interlens.arena.actions.parse_action`); the harness publishes only the message + the validated action, so privacy is structural, not tag-dependent. A deal closes ONLY on real ACCEPT votes on the SAME standing offer (never threshold arithmetic). Rotating proposer, the turn deadline restated each turn, one canonical prompt scaffold with variants behind flags (:class:`~interlens.arena.scenarios.scorable_prompts.PromptScaffold`). A syntax/legality error gets one retry-with-specific-feedback then a pass; an economic (below-own-threshold) move is MEASURED, never blocked.

Arms `moves_chat` / `moves_only` / `team` / `solo` cross the game’s FULL/PRIVATE info; ultimatum-style single-shot / fixed-proposer / majority variants ride on cfg knobs, as does `seeded_offer` — one standing package tabled by a neutral non-voting facilitator before round 1, which deletes the search problem and leaves the table nothing to do but judge a known deal (:meth:`ScorableNegotiation._seed_offer`). Each seat’s surplus is normalized by its own maximum available surplus before aggregation, so `primary` is invariant to positive affine rescaling of any party’s utility. Raw surplus welfare remains in explicitly named audit fields. Per-turn normative regret comes from the pluggable `oracles=` (each turn -> :class:`OracleRecord` via :meth:`annotate_turn`). A pure state machine (state is a pure function of

the action sequence), so stored episodes replay and rescore exactly (`arena/replay.py`).

Attributes

Name	Type	Summary
`ARMS`		

Classes

Name	Summary
`ScorableNegotiation`	The repaired scorable-negotiation protocol over a :class:`GameSpec` (see the module docstring).

ScorableNegotiation

The repaired scorable-negotiation protocol over a :class:`GameSpec` (see the module docstring).

```
ScorableNegotiation(
    scaffold: PromptScaffold | None = None,
    oracles: list[Oracle] | None = None,
    turn_max_tokens: int | None = None,
)
```

Defined in `interlens.arena.scenarios.scorable` · source

Inherits from: Scenario

Parameters

scaffold : PromptScaffold The prompt wording (one canonical config; ablate by passing a variant). Defaults to :data:`~interlens.arena.scenarios.scorable_prompts.DEFAULT_SCAFFOLD`. oracles : listOracle | None Rational-reference oracles run inline after every turn (:meth:`annotate_turn`) to score the seat's actual move against the oracle's best — the per-turn regret series. `None` runs no oracles (episodes still carry the full move ledger for post-hoc annotation via :meth:`oracle_inputs`). turn_max_tokens : int | None Raise every request's per-turn output cap to at least this many tokens. `None` (the default) leaves the protocol's own caps — 2048 on ordinary turns, 2560 on the forced final — exactly as every frozen campaign ran them. This is the ONLY hook for the budget, because the engine's `turn_cap` can only ever SHRINK a request's cap (:meth:`interlens.arena.engine.EpisodeRun.turn_cap`), so a per-turn budget large enough for a reasoning stream plus a visible action has to be set where the request is built.

```
    Setting it is a PROTOCOL CHANGE, not a tuning knob: turn caps censor
     behaviour, so a run at a raised cap
    does not pair with a run at the default and must not be pooled with
     one. It exists because of a measured
```

```
    failure — with native thinking ON, Qwen3-8B spends the entire 2048
     inside `<think>` on ~2/3 of turns
    and Qwen3-32B on 24.4%, and the engine then substitutes
     `EMPTY_TURN_PLACEHOLDER` for the visible
    action (rational_agents notes 0026/0041). A turn that says nothing is
     real model behaviour, so no
    fabrication gate catches it; only a cap census does.
```

Parameters

Name	Type	Default	Description
`scaffold`	PromptScaffold | None	`None`	
`oracles`	list[Oracle] | None	`None`	
`turn_max_tokens`	`int | None`	`None`	

Attributes

Name	Type	Summary
`N_LEVELS`		
`SOLO_CEILING`		
`SOLO_SEAT`		
`default_communication`		
`has_solo`		
`name`		
`oracles`		
`scaffold`		
`turn_max_tokens`		

Methods

annotate_turn

```
annotate_turn(self, st, req, turn) -> list
```

source

Inline per-turn oracle annotations (engine hook): score the seat's ACTUAL move against each oracle's best on the pre-move decision point captured in :meth:`_commit`.

`agent` is the seat INDEX (the unambiguous key into `game.sheets`, matching `PolicyParticipant(seat=int)`; the oracles accept it as-is); the record's `seat` is the display persona (what the analysis layer reads). No-op when no oracles are attached.

Parameters

Name	Type	Default	Description
`st`		*required*	
`req`		*required*	
`turn`		*required*	

apply

```
apply(self, st, req: SeatRequest, text: str) -> dict | None
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	

Name	Type	Default	Description
`req`	SeatRequest	*required*	
`text`	`str`	*required*	

generate_instance

```
generate_instance(self, level: int, seed: int, **overrides={}) -> Instance
```

source

One solver-verified instance at difficulty `level` from `seed`, delegated to the game-theory generator+bridge (`interlens.arena.negotiation.generate.generate_instance`) — the single owner of the difficulty ladder, the score-sheet-design knobs (Pareto slack / feasible size / sparsity-IoU), the per-round discount, and the `(GameSpec, analysis)` → `Instance` wrap (`payload` = `GameSpec.to_json`, `solution` = the enumeration-verified `analyze` dict).

`overrides` pass straight through (e.g. `info="private"`, `n_parties`, `dominated_target`, `discount`), so `run.py` and the `instances/` bank all call one code path. Scoring recomputes the surplus ceiling itself, so it is independent of the bridge's ceiling/floor convention.

Parameters

Name	Type	Default	Description
`level`	`int`	*required*	
`seed`	`int`	*required*	
`overrides`		`{}`	

make_state

```
make_state(self, instance: Instance, arm: str, seed: int, cfg: dict | None
    = None) -> dict
```

source

Parameters

Name	Type	Default	Description
`instance`	Instance	*required*	
`arm`	`str`	*required*	
`seed`	`int`	*required*	
`cfg`	`dict | None`	`None`	

next_requests

```
next_requests(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	

oracle_inputs

```
oracle_inputs(self, st) -> dict | None
```

source

The per-turn `(game, agent, history, legal_actions)` context for the seat about to move — the same shape the oracles consume, exposed for POST-HOC annotation (replay to a turn, then call this).

`None` when no team turn is pending (solo / done). Inline annotation during a live run uses :meth:`annotate_turn`.

Parameters

Name	Type	Default	Description
st		*required*	

provisional_due

```
provisional_due(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

rounds_used

```
rounds_used(self, st) -> int
```

source

Parameters

Name	Type	Default	Description
st		*required*	

score

```
score(self, st) -> dict
```

source

Parameters

Name	Type	Default	Description
st		*required*	

score_provisional

```
score_provisional(self, st, parsed) -> float | None
```

source

Parameters

Name	Type	Default	Description
st		*required*	
parsed		*required*	

seat_specs

```
seat_specs(self, st) -> list[dict]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_continue

```
solo_continue(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_final_prompt

```
solo_final_prompt(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_finalize

```
solo_finalize(self, st, answer, text: str) -> None
```

source

Parameters

Name	Type	Default	Description
st		*required*	
answer		*required*	
text	str	*required*	

solo_give_up

```
solo_give_up(self, st) -> None
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_parse

```
solo_parse(self, st, text: str) -> tuple
```

source

Parameters

Name	Type	Default	Description
st		*required*	
text	str	*required*	

solo_system

```
solo_system(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_task

```
solo_task(self, st) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	

solo_work_cap

```
solo_work_cap(self, st) -> int
```

source

Parameters

Name	Type	Default	Description
st		*required*	

scorable_prompts

Module `interlens.arena.scenarios.scorable_prompts`

The canonical prompt scaffold for the scorable-negotiation scenario.

Prompt configuration is a *documented confound*: in the scoreable-games literature the choice of chain-of-thought scaffold swings the same model's success rate 15%->81% (Abdelnabi et al. 2309.17234) with 20-50 point swings across configs, and the best config is model-dependent (TMLR [Re] BVH81SAAh2). So this module fixes exactly ONE canonical wording, and every knob that changes the wording is an explicit field on :class:`PromptScaffold` — an experiment ablates prompt wording by constructing a variant scaffold, never by editing the scenario. The scenario (:class:`~interlens.arena.scenarios.scorable.ScorableNegotiation`) holds a scaffold and calls its render methods; :data:`DEFAULT_SCAFFOLD` is the blessed default.

Design rules baked in, each traceable to a peer-reviewed critique of the prior benchmarks:

- **Structural channel separation** (Lesson 11): a turn is one fenced JSON object with three fields — `scratchpad` (private, never published), `message` (public cheap talk), `action` (the formal move). The harness publishes ONLY `message` + a rendering of the validated `action`. Privacy never depends on the model's tag discipline, so a model that dumps its score numbers into `scratchpad` cannot leak them (numbers put in `message` are genuine strategic disclosure — a measured failure, not a parse artifact).
- **Formal votes, offer ids** (Lessons 7, 10): accepts reference a specific live offer id; a deal closes only by real unanimous ACCEPT of one standing offer, never by arithmetic on a final proposal.
- **Turn-count deadline restated every turn** (Lesson 13): `turn_prompt` always restates the round budget.
- **De-anchoring** (Lesson 5): issue/option labels come from the instance (fictional units), not from here.

- **Preference/role decorrelation knob** (Lesson 4): `role_lines` are OFF by default so public role text can't leak private preferences; turn on only to measure the leak.

Attributes

Name	Type	Summary
`ACTION_TYPES`		
`DEFAULT_SCAFFOLD`		

Classes

Name	Summary
`PromptScaffold`	One immutable prompt wording.

PromptScaffold

One immutable prompt wording.

```
PromptScaffold(
    game_intro: str = 'You are one of {n} parties negotiating a single
     multi-issue agreement. Exactly one option must be chosen for every
     issue; the chosen options together form the deal.',
    scratchpad_help: str = 'private notes to yourself. NEVER shown to any
     other party. Reason here freely — writing here is not speaking and
     costs you nothing.',
    message_help: str = 'a short statement spoken aloud to every party
     (cheap talk — it does not bind you). Do NOT put your private score
     numbers here.',
    action_help: str = 'exactly one formal move this turn.',
    worked_examples: bool = True,
    include_pass: bool = True,
    show_role_lines: bool = False,
)
```

Defined in `interlens.arena.scenarios.scorable_prompts` · source

Construct a variant to ablate wording; never edit the scenario.

All game *content* (issue names, option labels, party names, sheets, thresholds, round budget, info condition, chat flag) is passed into the render methods by the scenario — this object holds only the fixed *wording* around that content, so a single scaffold renders any instance. Every field below is a template fragment or a toggle; the render methods assemble them.

Fields

game_intro : str One-line framing of what the parties are doing; `{n}` is the party count. Deliberately generic / de-anchored — the concrete subject matter lives in the (fictional) issue labels. scratchpad_help, message_help, action_help : str The per-field guidance in the action-format block. worked_examples : bool Include the two worked-example JSON blocks (propose-with-message, accept). On by default — worked examples materially raise format compliance for smaller models; turn off to ablate. include_pass : bool Advertise the talk-only `{"type": "none"}` move. On by default so talking without moving is explicit rather than smuggled into a

malformed action. show_role_lines : bool Render each party's public role description (which can leak preferences). OFF by default to keep sheets decorrelated from public text (the communication-free-baseline leak, Study A 2502.16242). Turn on only to *measure* the leak.

Parameters

Name	Type	Default	Description
`game_intro`	`str`	`'You are one of {n} parties negotiating a single multi-issue agreement. Exactly one option must be chosen for every issue; the chosen options together form the deal.'`	
`scratchpad_help`	`str`	`'private notes to yourself. NEVER shown to any other party. Reason here freely — writing here is not speaking and costs you nothing.'`	
`message_help`	`str`	`'a short statement spoken aloud to every party (cheap talk — it does not bind you). Do NOT put your private score numbers here.'`	
`action_help`	`str`	`'exactly one formal move this turn.'`	
`worked_examples`	`bool`	`True`	
`include_pass`	`bool`	`True`	

Name	Type	Default	Description
show_role_lines	bool	False	

Attributes

Name	Type	Summary
action_help	str	
game_intro	str	
include_pass	bool	
message_help	str	
scratchpad_help	str	
show_role_lines	bool	
worked_examples	bool	

Methods

action_format_block

```
action_format_block(self, *, chat_enabled: bool) -> str
```

source

The single action-format contract + worked examples.

`chat_enabled` toggles the public message channel (moves-only arm disables it — a `message` field is then ignored by the harness).

The wire form is ONE flat fenced JSON object: a string `"action"` field with its parameters as siblings (`"deal"` / `"offer_id"`), alongside the private `"scratchpad"` and public `"message"` channels. This matches the shared typed-action parser (`interlens.arena.actions.parse_action`).

Parameters

Name	Type	Default	Description
chat_enabled	bool	*required*	

final_prompt

```
final_prompt(self, *, seat: str, offers_block: str) -> str
```

source

The forced-final prompt when the round budget is exhausted with no deal: the current opener must either table a last binding proposal or walk.

Parameters

Name	Type	Default	Description
seat	str	*required*	
offers_block	str	*required*	

info_condition_note

```
info_condition_note(self, *, info: str) -> str
```

source

The one line describing what a seat knows about others' sheets (FULL vs PRIVATE).

Parameters

Name	Type	Default	Description
info	str	*required*	

private_block

```
private_block(
    self,
    *,
    seat: str,
    role_desc: str,
    sheet_lines: list[str],
    threshold: float,
    persona: str | None,
) -> str
```

source

The seat's private framing: who it is, its secret score sheet, its threshold, optional persona.

`sheet_lines` are `"- Issue: OptA=12, OptB=0, ..."` rendered by the scenario.

Parameters

Name	Type	Default	Description
`seat`	`str`	*required*	
`role_desc`	`str`	*required*	
`sheet_lines`	`list[str]`	*required*	
`threshold`	`float`	*required*	
`persona`	`str | None`	*required*	

rules_block

```
rules_block(
    self,
    *,
    n: int,
    party_lines: list[str],
    issue_lines: list[str],
    proposer_order: list[str],
    rounds: int,
    veto_line: str,
    pass_rule: str,
```

```
) -> str
```

source

The public, common-knowledge rules every seat sees: parties, issues+options, turn/deadline protocol, and the exact deal-closing rule.

`party_lines` already includes role text iff `show_role_lines`; `issue_lines` are `"- Name: OptA, OptB, ..."` rendered by the scenario.

Parameters

Name	Type	Default	Description
`n`	`int`	*required*	
`party_lines`	`list[str]`	*required*	
`issue_lines`	`list[str]`	*required*	
`proposer_order`	`list[str]`	*required*	
`rounds`	`int`	*required*	
`veto_line`	`str`	*required*	
`pass_rule`	`str`	*required*	

solo_system_prompt

```
solo_system_prompt(
    self,
    *,
    n: int,
    rules: str,
    all_sheets: str,
    threshold_note: str,
    pass_rule: str,
) -> str
```

source

The communication-free control (Study A 2502.16242): ONE agent with every sheet, proposing alone.

This baseline must NOT be competitive with the full multi-agent game — if it is, the benchmark is measuring feasible-set search, not negotiation.

Parameters

Name	Type	Default	Description
`n`	`int`	*required*	
`rules`	`str`	*required*	
`all_sheets`	`str`	*required*	
`threshold_note`	`str`	*required*	
`pass_rule`	`str`	*required*	

`solo_turn_prompt`

```
solo_turn_prompt(self, *, forced: bool) -> str
```

source

The solo agent's per-step nudge; `forced` is the budget-exhausted last call.

Parameters

Name	Type	Default	Description
`forced`	`bool`	*required*	

`system_prompt`

```
system_prompt(
    self,
    *,
    rules: str,
    private: str,
    action_format: str,
    info_note: str,
    full_info_sheets: str | None,
    seat_index: int | None = None,
```

```
) -> str
```

source

Assemble a seat's complete system prompt.

`full_info_sheets` is the block of all parties' sheets, present only in the FULL-information condition.

`seat_index` is the acting seat's index, passed by the scenario and **ignored here** — the seat is already rendered into `private`, so the base assembly does not need it. It exists so a subclass can vary the prompt PER SEAT (e.g. appending a reasoning workflow to one focal seat only, for a treatment-vs-control arm) without the scenario having to know which subclass it holds. Overriding subclasses that do not care may omit it; the base accepts and drops it, so existing scaffolds and stored-episode replay are byte-identical.

Parameters

Name	Type	Default	Description
`rules`	`str`	*required*	
`private`	`str`	*required*	
`action_format`	`str`	*required*	
`info_note`	`str`	*required*	
`full_info_sheets`	`str | None`	*required*	
`seat_index`	`int | None`	`None`	

turn_prompt

```
turn_prompt(
    self,
    *,
    seat: str,
    round_no: int,
    rounds: int,
    is_opener: bool,
```

```
    offers_block: str,
    chat_enabled: bool,
    seat_index: int | None = None,
) -> str
```

source

The per-turn user prompt: restates the deadline (Lesson 13), lists live offers with ids, and names whose turn it is.

`offers_block` is the scenario-rendered live-offers summary (with, privately, this seat's own score for each live offer when the scenario surfaces it). `seat_index` mirrors :meth:`system_prompt` — the acting seat's index, ignored by the base assembly and available to per-seat subclasses (`seat` already names the seat; the index is what a seat-targeting arm is configured with).

Parameters

Name	Type	Default	Description
`seat`	`str`	*required*	
`round_no`	`int`	*required*	
`rounds`	`int`	*required*	
`is_opener`	`bool`	*required*	
`offers_block`	`str`	*required*	
`chat_enabled`	`bool`	*required*	
`seat_index`	`int | None`	`None`	

security

Module `interlens.arena.scenarios.security`

Security dilemma: a repeated 2-party build/deescalate/attack game with noisy intelligence.

Each of 12 rounds has (1) a message phase — seat A then seat B sends one public message — and (2) a simultaneous action wave: each seat privately commits `"build"`, `"deescalate"`, or `"attack"` as a fenced JSON action (malformed after the one allowed retry → `"build"`).

Stage payoffs (symmetric, public): both deescalate +4/+4; both build +1/+1; build vs deescalate +5/+2 (builder gets 5); attack vs non-attack: attacker +6+B_fs, victim −6; both attack −4/−4. `B_fs` is the first-strike bonus (the difficulty knob).

Absorbing spiral: once BOTH seats attack in the same round, every remaining round is forced mutual attack at −4/−4 — no further requests; the forced rounds are recorded as public moderator events. A one-sided attack is NOT absorbing.

Signal noise: with probability `p` (the second difficulty knob), a seat's *observation* of the opponent's action that round is flipped to one of the other two actions (uniform). True actions always drive payoffs; observations drive what each seat is told, via per-seat-private moderator events. Each round a seat is told the (possibly misreported) opponent action, its own true payoff this round, and its own cumulative payoff — never the true action, and never the opponent's cumulative payoff (which would let it back out the true action). Noise draws use `random.Random(episode_seed ^ round)`, one RNG per round, draws in seat order, so episodes are exactly reproducible and replayable from their seed.

Scoring: `joint` = sum of both cumulative payoffs; ceiling joint = 96 (mutual deescalation for 12 rounds); `primary` = joint/96 (may be negative); floor = both-build every round = 24/96 = 0.25. `success` = the episode avoided the war spiral. No solo arm (the game is irreducibly 2-party).

Provenance: the collaboration-arena experiments' E3 environment, ported verbatim onto the `Scenario` contract (state machine, payoffs, noise draws, and instance seeds unchanged — stored episodes replay).

Attributes

Name	Type	Summary
`ACTIONS`		
`HORIZON`		
`LEVELS`		
`ROLES`		

Classes

Name	Summary
`SecurityDilemma`	

Functions

Name	Summary
`stage_payoffs`	Stage payoffs `(pa, pb)` for true actions `a` (seat 0) and `b` (seat 1).

SecurityDilemma

```
SecurityDilemma()
```

Defined in `interlens.arena.scenarios.security` · source

Inherits from: Scenario

Attributes

Name	Type	Summary
`N_LEVELS`		
`PROVISIONAL_MARKS`		
`default_communication`		
`has_solo`		
`name`		

Methods

`apply`

```
apply(self, st, req: SeatRequest, text: str) -> dict | None
```

source

Parameters

Name	Type	Default	Description
`st`		*required*	
`req`	SeatRequest	*required*	
`text`	`str`	*required*	

generate_instance

```
generate_instance(self, level: int, seed: int) -> Instance
```

source

Parameters

Name	Type	Default	Description
level	int	*required*	
seed	int	*required*	

make_state

```
make_state(self, instance: Instance, arm: str, seed: int, cfg: dict | None
    = None) -> dict
```

source

Parameters

Name	Type	Default	Description
instance	Instance	*required*	
arm	str	*required*	
seed	int	*required*	
cfg	dict | None	None	

next_requests

```
next_requests(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

provisional_due

```
provisional_due(self, st) -> list[SeatRequest]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

rounds_used

```
rounds_used(self, st) -> int
```

source

Parameters

Name	Type	Default	Description
st		*required*	

score

```
score(self, st) -> dict
```

source

Parameters

Name	Type	Default	Description
st		*required*	

score_provisional

```
score_provisional(self, st, parsed) -> float | None
```

source

Parameters

Name	Type	Default	Description
st		*required*	
parsed		*required*	

seat_specs

```
seat_specs(self, st) -> list[dict]
```

source

Parameters

Name	Type	Default	Description
st		*required*	

system_prompt

```
system_prompt(self, st, si: int) -> str
```

source

Parameters

Name	Type	Default	Description
st		*required*	
si	int	*required*	

stage_payoffs

Stage payoffs `(pa, pb)` for true actions `a` (seat 0) and `b` (seat 1).

```
stage_payoffs(a: str, b: str, b_fs: int) -> tuple[int, int]
```

Defined in `interlens.arena.scenarios.security` · source

Parameters

Name	Type	Default	Description
a	str	*required*	
b	str	*required*	
b_fs	int	*required*	

schema

Module `interlens.arena.schema`

The arena's record schema: one JSON shape for every episode.

Every episode — regardless of scenario, arm, cell, or model — serializes to one JSON record with the same top-level fields, so datasets and analyses join cleanly across runs. `Instance` is a generated, solver-verified problem (with its exact ceiling/floor and hidden solution); `Episode` is one play-through of an instance; `EpisodeStore` is the crash-safe on-disk layout. The schema is shared with the arena experiments that produced the public transcripts dataset, so stored episodes from those runs re-score under this package (see `replay.py`).

Attributes

Name	Type	Summary
`PERSONAS`		
`SCHEMA_VERSION`		

Classes

Name	Summary
`Episode`	One complete play-through: turns, forked provisional checkpoints, outcome, and usage accounting.
`EpisodeStore`	Per-episode JSON persistence, written atomically on every update so a crash loses at most one turn.
`Instance`	One generated problem instance, solver-verified at generation time.

Name	Summary
`JsonRecordStore`	A directory tree of JSON records, one file per record, written atomically.
`SeatRequest`	One pending generation: a seat that must speak now, with the exact view its model is conditioned on.
`TurnRecord`	

Functions

Name	Summary
`load_instances`	Load an instance pool saved by `save_instances` (or by the arena experiments — the pre-rename `env` key is migrated on read).
`new_id`	A fresh episode/instance id.
`save_instances`	Persist an instance pool as one JSON file (`{scenario}_L{level}.json` unless `name` overrides).

Episode

One complete play-through: turns, forked provisional checkpoints, outcome, and usage accounting.

```
Episode(
    episode_id: str,
    scenario: str,
    arm: str,
    model: str,
    level: int,
    instance_id: str,
    seed: int,
    seats: list[dict],
    cell: str = 'base',
    cell_cfg: dict = dict(),
    turns: list[TurnRecord] = list(),
    round_checkpoints: list[dict] = list(),
    outcome: dict = dict(),
    rounds_used: int = 0,
    tokens_in: int = 0,
    tokens_out: int = 0,
    cost_usd: float = 0.0,
    gen_config: dict = dict(),
    status: str = 'running',
    error: str | None = None,
    started_at: float = time.time(),
    ended_at: float | None = None,
    schema_version: str = SCHEMA_VERSION,
)
```

Defined in `interlens.arena.schema` · source

Parameters

Name	Type	Default	Description
`episode_id`	`str`	*required*	
`scenario`	`str`	*required*	
`arm`	`str`	*required*	
`model`	`str`	*required*	
`level`	`int`	*required*	
`instance_id`	`str`	*required*	

Name	Type	Default	Description
seed	int	*required*	
seats	list[dict]	*required*	
cell	str	'base'	
cell_cfg	dict	dict()	
turns	list[TurnRecord]	list()	
round_checkpoints	list[dict]	list()	
outcome	dict	dict()	
rounds_used	int	0	
tokens_in	int	0	
tokens_out	int	0	
cost_usd	float	0.0	
gen_config	dict	dict()	
status	str	'running'	
error	str | None	None	
started_at	float	time.time()	
ended_at	float | None	None	
schema_version	str	SCHEMA_VERSION	

Attributes

Name	Type	Summary
arm	str	
cell	str	
cell_cfg	dict	
cost_usd	float	
ended_at	float | None	
episode_id	str	
error	str | None	

Name	Type	Summary
gen_config	dict	
instance_id	str	
level	int	
model	str	
outcome	dict	
round_checkpoints	list[dict]	
rounds_used	int	
scenario	str	
schema_version	str	
seats	list[dict]	
seed	int	
started_at	float	
status	str	
tokens_in	int	
tokens_out	int	
turns	list[TurnRecord]	

Methods

from_json

```
from_json(cls, d: dict) -> 'Episode'
```

source

Rebuild an episode from a stored record — the inverse of :meth:`to_json`.

`to_json` is `dataclasses.asdict`, so `turns` comes back as plain dicts and is rehydrated through :meth:`TurnRecord.from_json`; `round_checkpoints` is stored as dicts by design and stays that way. Unknown keys are dropped and missing ones take their defaults, so an episode written by a different schema version

loads rather than raising. This exists because anything that reads episodes back — a post-hoc analysis, or a driver that collected episodes in worker processes and has to reassemble them — otherwise re-derives the data-class shape by hand and drifts from it.

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

to_json

```
to_json(self) -> dict
```

source

usage

```
usage(self) -> dict
```

source

This episode's usage summary: tokens in/out (total and per seat) and dollar cost.

EpisodeStore

Per-episode JSON persistence, written atomically on every update so a crash loses at most one turn.

```
EpisodeStore()
```

Defined in `interlens.arena.schema` · source

Inherits from: JsonRecordStore

Layout: `{root}/{scenario}/{cell}/{arm}/{model_short}/L{level}/{episode_id}.json`.

Methods

load_all

```
load_all(self, scenario: str | None = None) -> list[dict]
```

source

Every stored episode, or only those of one `scenario` (the top layout level).

Parameters

Name	Type	Default	Description
scenario	str | None	None	

path

```
path(self, ep: Episode) -> Path
```

source

Parameters

Name	Type	Default	Description
`ep`	Episode	*required*	

summary

```
summary(self) -> str
```

source

A printable run-usage summary aggregated over every stored episode: episode counts, token totals, and dollar cost, broken down per (model, arm) — plus cost-per-success where outcomes carry `success`.

Instance

One generated problem instance, solver-verified at generation time.

```
Instance(
    instance_id: str,
    scenario: str,
    level: int,
    seed: int,
    payload: dict,
    ceiling: float,
    floor: float,
    solution: dict,
)
```

Defined in `interlens.arena.schema` · source

`ceiling` is the best achievable primary score (computed exactly by the generator's solver), `floor` a reference floor policy's score, and `solution` the exact optimum — never shown to models, used for scoring and audits.

Parameters

Name	Type	Default	Description
`instance_id`	`str`	*required*	
`scenario`	`str`	*required*	
`level`	`int`	*required*	
`seed`	`int`	*required*	
`payload`	`dict`	*required*	
`ceiling`	`float`	*required*	
`floor`	`float`	*required*	
`solution`	`dict`	*required*	

Attributes

Name	Type	Summary
`ceiling`	`float`	
`floor`	`float`	
`instance_id`	`str`	
`level`	`int`	
`payload`	`dict`	
`scenario`	`str`	
`seed`	`int`	
`solution`	`dict`	

Methods

`from_json`

```
from_json(d: dict) -> 'Instance'
```

source

Parameters

Name	Type	Default	Description
`d`	`dict`	*required*	

`to_json`

```
to_json(self) -> dict
```

source

JsonRecordStore

A directory tree of JSON records, one file per record, written atomically.

```
JsonRecordStore(root: str | Path)
```

Defined in `interlens.arena.schema` · source

The persistence shared by every arena record store: a `root`, an atomic `save` (write a sibling `.tmp` then `os.replace`, so a crash mid-write leaves the previous file intact rather than a truncated one), and a sorted recursive `load_all`. Subclasses supply only what actually differs — :meth:`path` (the on-disk layout) and, optionally, `indent` (pretty-print) — so a new store is a few lines and inherits the crash safety instead of re-deriving it.

Records are duck-typed: anything with a `to_json()` method can be saved.

Parameters

Name	Type	Default	Description
`root`	`str | Path`	*required*	

Attributes

Name	Type	Summary
`indent`	`int | None`	
`root`		

Methods

`load_all`

```
load_all(self, pattern: str = '**/*.json') -> list[dict]
```

source

Every stored record under `root` matching `pattern`, as raw dicts, in sorted path order.

Parameters

Name	Type	Default	Description
`pattern`	`str`	`'**/*.json'`	

path

```
path(self, record) -> Path
```

source

The file `record` is stored at, creating its parent directory.

Subclasses define the layout.

Parameters

Name	Type	Default	Description
`record`		*required*	

save

```
save(self, record) -> Path
```

source

Write `record.to_json()` atomically to :meth:`path` and return that path.

Parameters

Name	Type	Default	Description
`record`		*required*	

SeatRequest

One pending generation: a seat that must speak now, with the exact view its model is conditioned on.

```
SeatRequest(
    episode_id: str,
    seat: str,
    view: list[dict],
    phase: str,
    round: int,
    max_tokens: int = 2048,
    meta: dict = dict(),
    provisional: bool = False,
)
```

Defined in `interlens.arena.schema` · source

Parameters

Name	Type	Default	Description
`episode_id`	`str`	*required*	
`seat`	`str`	*required*	
`view`	`list[dict]`	*required*	
`phase`	`str`	*required*	
`round`	`int`	*required*	
`max_tokens`	`int`	`2048`	
`meta`	`dict`	`dict()`	
`provisional`	`bool`	`False`	

Attributes

Name	Type	Summary
`episode_id`	`str`	
`max_tokens`	`int`	

Name	Type	Summary
meta	dict	
phase	str	
provisional	bool	
round	int	
seat	str	
view	list[dict]	

TurnRecord

```
TurnRecord(
    idx: int,
    round: int,
    phase: str,
    seat: str,
    content: str,
    parsed_action: Any,
    parse_ok: bool,
    n_tokens_out: int = 0,
    n_tokens_in: int = 0,
    stop_reason: str | None = None,
    cap: int = 0,
    effective_cap: int | None = None,
    raw: str | None = None,
    reasoning: str | None = None,
    reasoning_provenance: str = 'none',
    reasoning_tokens: int = 0,
    view: list[dict] | None = None,
    gen_failed: bool = False,
    gen_failure: str | None = None,
    refusal_recovery: dict | None = None,
    occupant: str | None = None,
    human_note: str | None = None,
)
```

Defined in `interlens.arena.schema` · source

Parameters

Name	Type	Default	Description
`idx`	`int`	*required*	
`round`	`int`	*required*	
`phase`	`str`	*required*	
`seat`	`str`	*required*	
`content`	`str`	*required*	
`parsed_action`	`Any`	*required*	
`parse_ok`	`bool`	*required*	
`n_tokens_out`	`int`	`0`	

Name	Type	Default	Description
n_tokens_in	int	0	
stop_reason	str | None	None	
cap	int	0	
effective_cap	int | None	None	
raw	str | None	None	
reasoning	str | None	None	
reasoning_provenance	str	'none'	
reasoning_tokens	int	0	
view	list[dict] | None	None	
gen_failed	bool	False	
gen_failure	str | None	None	
refusal_recovery	dict | None	None	
occupant	str | None	None	
human_note	str | None	None	

Attributes

Name	Type	Summary
cap	int	
content	str	
effective_cap	int | None	
gen_failed	bool	
gen_failure	str | None	
human_note	str | None	
idx	int	
n_tokens_in	int	
n_tokens_out	int	
occupant	str | None	

Name	Type	Summary
parse_ok	bool	
parsed_action	Any	
phase	str	
raw	str | None	
reasoning	str | None	
reasoning_provenance	str	
reasoning_tokens	int	
refusal_recovery	dict | None	
round	int	
seat	str	
stop_reason	str | None	
view	list[dict] | None	

Methods

from_json

```
from_json(cls, d: dict) -> 'TurnRecord'
```

source

Rebuild a turn from its stored dict, ignoring fields this version does not know.

Forward-compatible on purpose: a record written by a newer schema carries keys this dataclass has no field for, and dropping them is the only way an older reader can load a newer file at all. Missing keys fall back to the field defaults, which is what makes pre-v1.2 episodes load unchanged.

Parameters

Name	Type	Default	Description
d	dict	*required*	

load_instances

Load an instance pool saved by `save_instances` (or by the arena experiments — the pre-rename `env` key is migrated on read).

```
load_instances(
    root: str | Path,
    scenario: str,
    level: int | None = None,
    name: str | None = None,
) -> list[Instance]
```

Defined in `interlens.arena.schema` · source

Parameters

Name	Type	Default	Description
`root`	`str | Path`	*required*	
`scenario`	`str`	*required*	
`level`	`int | None`	`None`	
`name`	`str | None`	`None`	

new_id

A fresh episode/instance id.

```
new_id(prefix: str) -> str
```

Defined in `interlens.arena.schema` · source

Random (uuid4-based), deliberately NOT seed-derived: two runs of the same seed produce identical payloads but distinct ids.

Parameters

Name	Type	Default	Description
`prefix`	`str`	*required*	

save_instances

Persist an instance pool as one JSON file (`{scenario}_L{level}.json` unless `name` overrides).

```
save_instances(
    instances: list[Instance],
    root: str | Path,
    name: str | None = None,
) -> Path
```

Defined in `interlens.arena.schema` · source

Parameters

Name	Type	Default	Description
`instances`	list[Instance]	*required*	
`root`	`str | Path`	*required*	
`name`	`str | None`	`None`	

table

Module `interlens.arena.table`

Heterogeneous **tables**: present a whole many-seat lineup to the arena engine as one participant.

The engine drives ONE `Participant` across every seat of an episode — it asks the same object to speak for each seat in turn. That is exactly right for a homogeneous table (one model plays everyone) and wrong for the interesting cases: a different computable policy per seat, some LLM seats and some policy seats, or per-seat model diversity. :class:`SeatRouter` closes the gap by dispatching each turn to the sub-participant that owns that seat, using the seat identity the engine passes (`Participant.generate(..., seat=...)`, straight from `SeatRequest.seat`).

The builders here compose a table from a game and a lineup, so seat assignment lives in one place rather than being re-derived per experiment:

- :func:`policy_seat` — one computable-rational seat (a :class:`PolicyParticipant` bound to a named policy).
- :func:`rational_table` — every seat a policy, cycled from a list of policy names.
- :func:`mixed_table` — some seats given participants explicitly (LLM or otherwise), the rest filled with policy seats, so a partly-specified lineup is always complete.

Example:

```
from interlens.arena.table import rational_table, mixed_table
from interlens.arena.negotiation import games

game, analysis, protocol_cfg = games.make_preset("divide_dollar",
     n_parties=3)
table = rational_table(game, ["boulware", "conceder", "tough"],
     deadline=game.rounds)
# or put a model in seat 0 and let policies fill the rest:
table = mixed_table(game, {0: my_model_participant}, deadline=game.rounds)
# hand `table` to EpisodePool.run_episode as the participant
```

Attributes

Name	Type	Summary
`OFFER_PREFIX`		
`POLICY_FACTORIES`		

Classes

Name	Summary
`SeatRouter`	One participant that routes each turn to a per-seat sub-participant, so a single object presents a heterogeneous table to the arena engine.

Functions

Name	Summary
`mixed_table`	A table where `assignment` maps a seat index to an already-built participant (e.g. an `AutoModelParticipant` or `APIParticipant` for an LLM seat) and every seat NOT in `assignment` is filled with a `fill_policy` seat — so a partly-specified lineup is always a complete table.
`policy_seat`	A computable-rational seat: a :class:`PolicyParticipant` bound to `policy_name` for seat `seat_idx`.

Name	Summary
`rational_table`	Every seat a computable policy: seat `i` plays `policies[i % len(policies)]` (cycled).

SeatRouter

One participant that routes each turn to a per-seat sub-participant, so a single object presents a heterogeneous table to the arena engine.

```
SeatRouter(seats: dict[str, Participant], name: str = 'seat_router')
```

Defined in `interlens.arena.table` · source

Inherits from: Participant

Parameters

seats : dict[str, Participant] Seat display name (`"Avery"`, `"Blake"`, …, or a scenario's solo seat) to the participant that plays it. Interp requests are forwarded to the sub-participant, which accepts or refuses them per its own contract (a local-model seat can be captured/steered; a policy seat raises). name : str Identifier within the conversation.

Parameters

Name	Type	Default	Description
`seats`	dict[str, Participant]	*required*	
`name`	`str`	`'seat_router'`	

Attributes

Name	Type	Summary
`name`		
`others_role`		
`private_context`		
`seats`		
`self_role`		

Name	Type	Summary
system_prompt		

Methods

generate

```
generate(self, view, *, seat: str | None = None, **kwargs={}) -> Message
```

source

Dispatch to the participant owning `seat` and return its message unchanged.

`seat` comes from the engine (`SeatRequest.seat`). It is required: without it there is no reliable way to know which seat this turn is for, and guessing from the prompt text silently breaks whenever the wording changes — so a missing seat raises rather than picking a default.

Parameters

Name	Type	Default	Description
view		*required*	
seat	str | None	None	
kwargs		{}	

participant_for

```
participant_for(self, seat: str | None)
```

source

The sub-participant that owns `seat`.

Declaring this method is how a table tells the batched engine "I am **pure dispatch** — you may address my sub-participants directly". `BatchedEpisodePool`

uses it to group a co-stepped wave by the participant that will actually serve each request rather than by the table object, which is what makes a heterogeneous lineup batchable at all: every episode gets its OWN table (policy seats hold per-episode state), so grouping by table would put one request in each group and batch nothing, while grouping by owner collects the model seats of every live episode — which DO share one cached model participant — into a single batch.

Only implement it on a table whose `generate` adds nothing of its own. A table that rewrites the view per seat (a planner/advocate wrapper) must NOT expose this, or the engine would bypass that rewriting; such a table supplies `generate_batch_with_seats` instead and keeps the whole wave.

Parameters

Name	Type	Default	Description
`seat`	`str | None`	*required*	

mixed_table

A table where `assignment` maps a seat index to an already-built participant (e.g. an `AutoModelParticipant` or `APIParticipant` for an LLM seat) and every seat NOT in `assignment` is filled with a `fill_policy` seat — so a partly-specified lineup is always a complete table.

```
mixed_table(
    game: GameSpec,
    assignment: dict[int, Participant],
    *,
    deadline: int,
    full_info: bool = True,
    fill_policy: str = 'bayes-rational',
    name: str = 'mixed',
) -> SeatRouter
```

Defined in `interlens.arena.table` · source

Parameters

Name	Type	Default	Description
`game`	GameSpec	*required*	
`assignment`	dict[int, Participant]	*required*	
`deadline`	`int`	*required*	
`full_info`	`bool`	`True`	
`fill_policy`	`str`	`'bayes-rational'`	
`name`	`str`	`'mixed'`	

policy_seat

A computable-rational seat: a :class:`PolicyParticipant` bound to `policy_name` for seat `seat_idx`.

```
policy_seat(
    policy_name: str,
    seat_idx: int,
    game: GameSpec,
    *,
    deadline: int,
    full_info: bool = True,
    tables: GameTables | None = None,
) -> PolicyParticipant
```

Defined in `interlens.arena.table` · source

`full_info` (a FULL-information game) attaches the exact :class:`GameTables` so full-info policies (e.g. the Bayesian best-responder) compute exactly; a PRIVATE game passes `tables=None` so the policy relies on its belief model instead. The per-round `discount` is read off the game (its single source of truth). Deals are emitted and decoded by issue/option NAME straight off `game.space`, matching the scenario transcript.

Parameters

Name	Type	Default	Description
`policy_name`	`str`	*required*	
`seat_idx`	`int`	*required*	
`game`	GameSpec	*required*	
`deadline`	`int`	*required*	
`full_info`	`bool`	`True`	
`tables`	GameTables | None	`None`	

rational_table

Every seat a computable policy: seat `i` plays `policies[i % len(policies)]` (cycled).

```
rational_table(
    game: GameSpec,
    policies: list[str],
    *,
    deadline: int,
    full_info: bool = True,
    name: str = 'all_rational',
) -> SeatRouter
```

Defined in `interlens.arena.table` · source

The full-information :class:`GameTables` is built once and shared across seats.

Parameters

Name	Type	Default	Description
`game`	GameSpec	*required*	
`policies`	`list[str]`	*required*	
`deadline`	`int`	*required*	
`full_info`	`bool`	`True`	
`name`	`str`	`'all_rational'`	

views

Module `interlens.arena.views`

Per-seat view construction + structured-action parsing for arena scenarios.

Views are family-agnostic role/content message lists; the participant applies the family-correct chat template (`ModelParticipant`) or provider format (`APIParticipant`). Views alternate user/assistant with the seat's own past turns as assistant turns and everyone else's (author-labelled) as merged user turns — the same semantics as the core per-speaker view pipeline, specialized to a scenario's event list with per-seat private events.

Family-agnostic means exactly that: the alternation here is *this seat's* speaking order, so a view legitimately opens on the seat's OWN turn (the round-1 opener of a multi-round game, whose proposal is the first event in the shared log) or repeats it (a rotation/round-boundary repeat). A strict template — Gemma's — rejects both; the participant repairs the flattened view for its own family in `Participant.repair_view`. Do not repair here.

Functions

Name	Summary
`build_view`	Render `events` (`[{seat|'MODERATOR', content, only?}]`, public unless `only` names seats) into an alternating per-speaker view for `seat`, ending with `phase_prompt` as the final user turn.
`extract_json`	The last fenced JSON object in `text`, else the last balanced top-level `{...}` that parses.

build_view

Render `events` (`[{seat|'MODERATOR', content, only?}]`, public unless `only` names seats) into an alternating per-speaker view for `seat`, ending with `phase_prompt` as the final user turn.

```
build_view(seat: str, system: str, events: list[dict], phase_prompt: str)
     -> list[dict]
```

Defined in `interlens.arena.views` · source

Parameters

Name	Type	Default	Description
`seat`	`str`	*required*	
`system`	`str`	*required*	
`events`	`list[dict]`	*required*	
`phase_prompt`	`str`	*required*	

extract_json

The last fenced JSON object in `text`, else the last balanced top-level `{...}` that parses.

```
extract_json(text: str) -> Any | None
```

Defined in `interlens.arena.views` · source

Thin alias for :func:`interlens.parsing.last_json`.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

viz

Package `interlens.arena.viz`

Interactive episode visualization: any arena run directory in, self-contained interactive HTML out.

This is the shared renderer for negotiation episodes — the graphical counterpart of :mod:`interlens.arena.export`, which produces the flat markdown/HTML transcript. Where the exporter answers "what was said", this answers "was it any good": every deal placed against the exact Pareto frontier and the axiomatic solution points, every turn's action next to the post-hoc oracle counterfactual and its value gap them, and every prompt the models actually saw, expandable and marked with its provenance.

Swapping a run in is one call — nothing about a run is hard-coded, and the same renderer serves any scenario whose instances carry a scorable game (episodes without one still render their transcript, minus the game panels).

Two modes:

Per episode — one page each, plus a run index:

```
from interlens.arena import viz
viz.export_run("runs/p2_pilot", "products/viz")                # every
    episode + index.html
viz.export_run("runs/p2_pilot", "products/viz", limit=3)       # just the
    first three

html = viz.render_episode("runs/p2_pilot", "runs/p2_pilot/episodes/.../
    abc.json")
```

Seat-swap comparison — the same instance and seed played with a different occupant in one seat:

```
viz.export_comparison("runs/p2_X_all_llm", "runs/p2_X_mixed", "products/
    viz_compare", limit=4)
```

CLI (self-documenting, `--help` on every argument):

```
python -m interlens.arena.viz --run RUN_DIR --out OUT_DIR [--limit N]
```

```
python -m interlens.arena.viz --compare LEFT_RUN RIGHT_RUN --out OUT_DIR
     [--pair-key instance_id seed arm]
```

The pages are self-contained: inline CSS and JS, no network requests, no build step, light and dark aware. They open by double-click from a filesystem path.

Looking at them from a cluster node — when the run is on a machine with no browser, drop `--out` (the pages go to a temporary directory) and add `--serve` (a stdlib HTTP server on a free port, which prints the URL and the `ssh -L` command to forward it):

```
python -m interlens.arena.viz --run runs/p2_pilot --limit 5 --serve
python -m interlens.arena.viz --run runs/p2_pilot --serve --port 8899   #
     a standing tunnel's port
```

See :func:`~interlens.arena.viz.serve.serve_directory` to do the same from Python.

Modules

- `advice` — The advised seat, audited: what its planner knew, what it recommended, and whether the seat did it.
- `assets` — The inline stylesheet and browser layer — no external assets of any kind.
- `auction_geometry` — The plottable geometry of one repeated-auction episode — the `AuctionSpec`-shaped sibling of :class:`~interlens.arena.viz.geometry.GameGeometry`.
- `auction_page` — The auction episode page's own panels — the four charts design.md §10 commits to, plus the per-turn counterfactual table.
- `ballots` — The final vote, as a tally a reader can check at a glance — including the ballots that were never recorded.
- `census` — How much of an episode is actually play: the per-turn census, and the strip that puts it in the page header.

- `chrome` — The shell every page wears, and the wire form of the payload it carries.
- `compare` — Seat-swap comparison: the same game instance played twice, with one seat's occupant swapped.
- `concepts` — What each solution concept IS, in one place, for every part of the visualizer that explains one to a reader.
- `episode` — One stored episode, turned into the single JSON payload the interactive page renders.
- `export` — The file-writing layer: run directory in, HTML pages plus an index on disk out.
- `geometry` — The plottable geometry of one negotiation instance: every deal placed in a 2-D scale-invariant embedding, with the frontier, the axiomatic solution points, and each party's individually-best deal marked.
- `hazards` — Two facts about a run that decide whether its numbers may be compared with another run's.
- `page` — HTML assembly: a payload in, one self-contained interactive page out.
- `references` — The decision references a scored turn can carry, placed on two axes — and what each one's number MEANS.
- `serve` — Hand the rendered pages to a browser over HTTP, for when the filesystem the pages live on is not the one the browser runs on.

Re-exported

Name	Defined in	Summary
`ADVICE_SIDECAR`	`interlens.arena.viz.advice`	
`DEFAULT_HOST`	`interlens.arena.viz.serve`	
`DEFAULT_PAIR_KEY`	`interlens.arena.viz.compare`	
`DERIVATION_SIDECAR`	`interlens.arena.viz.ballots`	
`DealGeometry`	`interlens.arena.viz.geometry`	One deal's full record: where it plots, and how every party feels about it.

Name	Defined in	Summary
FROZEN_TURN_CAPS	interlens.arena.viz.hazards	
GameGeometry	interlens.arena.viz.geometry	The exact, fully-enumerated geometry of one negotiation instance, ready to plot.
NAV_MARKER	interlens.arena.viz.chrome	
RAW_EXCERPT_CHARS	interlens.arena.viz.episode	
RETRY_SOURCE	interlens.arena.viz.episode	
RunDir	interlens.arena.viz.episode	A run directory's three record stores, indexed for lookup: episodes/, instances/, and the annotation subdirectory named by annotations_dirname (annotations/ by default), plus manifest.json.
SELECTIONS	interlens.arena.viz.compare	
SIDEBAR_TABS	interlens.arena.viz.page	
advice_card	interlens.arena.viz.advice	The episode's advice audit as one server-rendered section, or "" when no turn was advised.
advice_summary	interlens.arena.viz.advice	This episode's compliance record: how many turns were advised, how many followed, on which rung.

Name	Defined in	Summary
`advice_trace`	`interlens.arena.viz.advice`	The optional advice sidecar for a run, or `None` when it is absent or unreadable.
`align`	`interlens.arena.viz.compare`	Align two episode payloads slot by slot and locate the divergence point.
`attach_advice`	`interlens.arena.viz.advice`	Turn payload rows with each advised turn's trace row attached under `advice`.
`ballot_table`	`interlens.arena.viz.ballots`	The final-vote tally as a card, or nothing when the episode has no final-vote phase.
`census_strip`	`interlens.arena.viz.census`	The per-episode census as a compact header strip, always rendered when there are turns to count.
`compare_payload`	`interlens.arena.viz.compare`	One seat-swap comparison, ready to render: both episode payloads, the slot alignment, the divergence point, the focal seat(s), and the score table.
`distance_to_nbs`	`interlens.arena.viz.chrome`	How far the deal that closed sits from the Nash bargaining solu-

Name	Defined in	Summary
		tion, in the chart's own plane.
episode_advice	interlens.arena.viz.advice	One episode's advised turns from the trace, keyed by turn index as a string.
episode_payload	interlens.arena.viz.episode	The complete render payload for one episode.
export_comparison	interlens.arena.viz.export	Pair two runs on pair_fields and write one comparison page per matched pair, plus an index and the pairing report.
export_episode	interlens.arena.viz.export	Write one episode's page into out_dir as <episode_id>.html and return its path.
export_run	interlens.arena.viz.export	Render every episode of a run to its own page in out_dir, plus an index.html listing them with the numbers that say which are worth opening.
final_ballots	interlens.arena.viz.ballots	The episode's final vote as {offer, rows, n_ballots, n_abstentions,

Name	Defined in	Summary
		`n_retries, n_mismatch, derived}`.
`focal_seats`	`interlens.arena.viz.compare`	The seats whose OCCUPANT KIND differs between the two episodes — the substitution being measured.
`generation_budget`	`interlens.arena.viz.hazards`	The per-seat generation budget this episode actually ran at, and whether it is the frozen default.
`inject_nav`	`interlens.arena.viz.chrome`	Put a page's navigation into its top bar.
`make_server`	`interlens.arena.viz.serve`	Bind (but do not run) an HTTP server rooted at `directory`.
`nav_group`	`interlens.arena.viz.chrome`	The prev/next links and episode picker for the page at `position` in `rows`.
`pair_key`	`interlens.arena.viz.compare`	The pairing key of an episode: the tuple of its `fields`.
`pair_runs`	`interlens.arena.viz.compare`	Pair every episode of one run against its key-matched counterpart in another, and build a comparison payload for each.

Name	Defined in	Summary
`public_ledger`	`interlens.arena.viz.episode`	Reconstruct what the seats publicly saw, from the per-turn records alone.
`reconstruct_views`	`interlens.arena.viz.episode`	Re-derive each turn's rendered view by deterministic replay, for episodes recorded before the per-turn `view` field existed.
`render_compare`	`interlens.arena.viz.export`	The interactive HTML for the `index`-th matched pair between two runs, as a string.
`render_compare_html`	`interlens.arena.viz.page`	One self-contained page for a seat-swap comparison: a verdict strip, the quantified score table with paired deltas, one shared frontier carrying both trajectories, and two synchronized transcript columns with the divergence point marked.
`render_episode`	`interlens.arena.viz.export`	The interactive HTML for one episode of a run, as a string.

Name	Defined in	Summary
render_episode_html	interlens.arena.viz.page	One self-contained interactive page for one episode.
render_index_html	interlens.arena.viz.page	A run index: one row per generated page, sortable on every column and filterable by text, outcome, and whether the engine fabricated any turns.
round_ledger	interlens.arena.viz.advice	Per round, every advised seat's turn side by side — the shape an all-advised arm is read in.
round_ledger_card	interlens.arena.viz.advice	The per-round, all-seats view, or "" on an episode that advises fewer than two seats.
score_table	interlens.arena.viz.compare	The quantified comparison: one row per metric with both values and the paired delta right - left.
seat_kinds	interlens.arena.viz.episode	Which seats an LLM played and which a computable policy played.
serve_banner	interlens.arena.viz.serve	The startup message: where the pages are, the URL to open, and

Name	Defined in	Summary
		— because the reader is usually on a cluster node with no browser — the exact `ssh -L` command to forward `port` to their laptop, with this machine's real hostname already filled in.
`serve_directory`	`interlens.arena.viz.serve`	Serve `directory` over HTTP until interrupted — bind, print :func:`serve_banner`, then block in `serve_forever`.
`slim_payload`	`interlens.arena.viz.chrome`	A copy of `payload` whose per-turn prompt views are indices into a shared `msgpool`.
`staircase`	`interlens.arena.viz.geometry`	The left-to-right monotone staircase of the masked points in the 2-D embedding: those not dominated on `(wx, wy)` by another masked point.
`summary_strip`	`interlens.arena.viz.chrome`	The whole episode in one row: did it close, how good was it, how far from the normative anchor, who was left below their threshold,

Name	Defined in	Summary
		how much of it the engine fabricated, how long it ran, what it cost.
`turn_census`	`interlens.arena.viz.census`	Count the ways this episode's turns carried nothing, from the payload's own turn rows.
`unslim_payload`	`interlens.arena.viz.chrome`	The inverse of :func:`slim_payload`: pooled view indices back to full `{role, content}` messages.
`vintage_provenance`	`interlens.arena.viz.hazards`	The run's `VINTAGE_PROVENANCE.md`, parsed into `{path, headline, summary}`, or `None` if absent.
`vote_derivation`	`interlens.arena.viz.ballots`	The optional per-turn re-derivation sidecar for a run, or `None` when it is absent or unreadable.

advice

Module `interlens.arena.viz.advice`

The advised seat, audited: what its planner knew, what it recommended, and whether the seat did it.

Some arms give one seat a private advisor and let the model keep the decision. Two questions then decide whether any number the arm produces means anything, and neither is answerable from the transcript:

1. **What did the advisor actually know?** In an interpreter arm the advice is computed from two very different evidence channels — the FORMAL move ledger (packages tabled, offers accepted, all machine-readable) and a model's parse of the public chat into structured preference claims, each carrying the verbatim quote it was read off. A recommendation that looks bizarre usually looks reasonable next to the claims that produced it, and a claim that moved a plan is a claim a reader should be able to check against the sentence it came from.
2. **Did the seat take the advice?** An arm whose advice the model discarded measured a prompt, not an intervention. On the wave-2 cell 46% of advised turns did something other than what the planner ranked, so a reader scrolling the transcript is looking at two quite different kinds of turn and nothing distinguished them.

This module renders both, from one run-level sidecar.

The verdict is READ, never computed

`advice_trace.json` is written into the run directory by the experiment's own `build_advice_trace.py`, which takes every compliance column from `advice_uptake` — the module the arm's uptake gate is evaluated on. Whether a turn followed its advice is therefore decided once, by the code that owns the comparison rules (deal identity for a propose, offer id for an accept, which rung of a ranked list was played), and this file only marks what that decision was. Re-deriving it in the renderer would be a second opinion

with no authority, and the two would disagree the first time either changed. It is the same division of labour as the final-vote tally's derived column (:mod:`~interlens.arena.viz.ballots`).

No model call and no classifier is involved anywhere in the chain. In particular the page does **not** claim to know how the seat verbalized its advice: it puts the ranked packages, the move actually played, and the public message the seat published in one place, and leaves the reading to the reader. "Described the advice faithfully" is not in the record, so it is not asserted.

What ships to the page

Each advised turn's trace row is attached to that turn's payload as `advice` and rendered inside its card by the browser layer (`assets/js_transcript.py`), which also puts an override class on the card and on the turn's scrubber chip. Above the transcript, :func:`advice_card` renders the same audit as a server-side table, so the compliance record is readable with scripting off and every row links to its turn.

Attributes

Name	Type	Summary
`ADVICE_SIDECAR`		
`ROOT_KEY`		

Functions

Name	Summary
`advice_card`	The episode's advice audit as one server-rendered section, or `""` when no turn was advised.

Name	Summary
`advice_summary`	This episode's compliance record: how many turns were advised, how many followed, on which rung.
`advice_trace`	The optional advice sidecar for a run, or `None` when it is absent or unreadable.
`attach_advice`	Turn payload rows with each advised turn's trace row attached under `advice`.
`episode_advice`	One episode's advised turns from the trace, keyed by turn index as a string.
`round_ledger`	Per round, every advised seat's turn side by side — the shape an all-advised arm is read in.
`round_ledger_card`	The per-round, all-seats view, or `""` on an episode that advises fewer than two seats.

advice_card

The episode's advice audit as one server-rendered section, or `""` when no turn was advised.

```
advice_card(payload: dict) -> str
```

Defined in `interlens.arena.viz.advice` · source

One row per advised turn — the planner's top pick, the move the seat played, and the stored verdict on the pair — with the per-turn evidence (the parsed claims and their quotes, the ranked candidates, the ledger counts) in each turn's own card below. The section is a hazard style when any turn overrode its advice, because that is the fact a reader must not scroll past: those turns measure the model's own choice, not the advisor's.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

advice_summary

This episode's compliance record: how many turns were advised, how many followed, on which rung.

```
advice_summary(turns: dict[str, dict]) -> dict
```

Defined in `interlens.arena.viz.advice` · source

`n_followed` counts turns whose stored verdict is a match; `n_overridden` counts the explicit non-matches only. A turn whose verdict is missing (a trace that failed to join its sidecar) is in neither, and `n_unverdicted` says so rather than letting an unknown fall into the compliant column.

Parameters

Name	Type	Default	Description
`turns`	`dict[str, dict]`	*required*	

advice_trace

The optional advice sidecar for a run, or `None` when it is absent or unreadable.

```
advice_trace(run_root: str | Path | None) -> dict | None
```

Defined in `interlens.arena.viz.advice` · source

Two shapes are read, because one blob does not serve both arm sizes. A single-advised-seat arm's trace is a few megabytes and ships whole, with its episodes inline. An **all-advised** arm's is tens of megabytes — every turn is advised, so there are five times the turns and five times the claims — and is written as an index plus one file per episode; a reader auditing one episode then fetches that episode rather than the corpus. The index says which shape it is (`sharded`), so nothing is inferred from the presence of a key.

Unreadable is treated as absent for the same reason the ballot sidecar is: a malformed audit file must cost the page one optional panel, not the whole render.

Parameters

Name	Type	Default	Description
`run_root`	`str | Path | None`	*required*	

attach_advice

Turn payload rows with each advised turn's trace row attached under `advice`.

```
attach_advice(rows: Sequence[dict], turns: dict[str, dict]) -> list[dict]
```

Defined in `interlens.arena.viz.advice` · source

Keyed on the turn's own `idx`, which is the index the sidecar recorded and the id the transcript, the scrubber and every jump link already agree on.

Parameters

Name	Type	Default	Description
`rows`	`Sequence[dict]`	*required*	
`turns`	`dict[str, dict]`	*required*	

episode_advice

One episode's advised turns from the trace, keyed by turn index as a string.

```
episode_advice(trace: dict | None, episode_id: str | None) -> dict[str,
    dict]
```

Defined in `interlens.arena.viz.advice` · source

Reads an inline `episodes` map or, on a sharded trace, the one shard for this episode. Returns an empty dict for an episode the trace does not cover — an unadvised arm, an episode that errored, a run with no sidecar, or a shard that has gone missing — which is what makes every caller a no-op rather than a special case, and what keeps one absent shard from costing the whole render.

Parameters

Name	Type	Default	Description
`trace`	`dict | None`	*required*	
`episode_id`	`str | None`	*required*	

round_ledger

Per round, every advised seat's turn side by side — the shape an all-advised arm is read in.

```
round_ledger(turns: dict[str, dict]) -> list[dict]
```

Defined in `interlens.arena.viz.advice` · source

On a one-advised-seat arm a round holds one advised turn and this is the transcript in a narrower column. On an ALL-advised arm a round holds five, each seat deciding against its own planner at the same table state, and the question that matters is no longer "did this seat take its advice" but "what did the five of them do to each other" — which is a question this view lets a reader put, not one it answers.

It reports arithmetic and asserts no mechanism. The intuitive account — that simultaneous concessions cancel where a lone concession transfers — was written down as a prediction with a falsifier and tested on an all-advised corpus; both prespecified correlations came back null, so it is a conjecture and this view must not be captioned as showing it. What the view is FOR is being the substrate a design that *assigns* the number of advised seats would be read against.

Each round reports its rows plus the arithmetic over them: how many seats overrode, and the total own-surplus those overrides moved. `conceded` sums only the NEGATIVE deltas and `claimed` only the positive ones, kept apart rather than netted, because a round where two seats each gave up twenty points is not the same round as one where nobody moved, and a single net figure cannot tell them apart.

`advice_overridden_toward` is defined only where a seat proposed a package of its own (an accept names an offer, not a deal), so `n_measured` reports the denominator beside the sums rather than letting a round with one measurable override read like a round with five.

Parameters

Name	Type	Default	Description
`turns`	`dict[str, dict]`	*required*	

round_ledger_card

The per-round, all-seats view, or "" on an episode that advises fewer than two seats.

```
round_ledger_card(payload: dict) -> str
```

Defined in `interlens.arena.viz.advice` · source

Rendered server-side, one block per round, so the picture reads with scripting off and every number on it is in the page rather than computed in the browser. Omitted entirely on a single-advised-seat arm, where five columns would be four empty ones.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

assets

Package `interlens.arena.viz.assets`

The inline stylesheet and browser layer — no external assets of any kind.

Every generated page is opened straight off a filesystem path (`file://`), often on a cluster login node behind no web server, so a single request to a CDN would leave the chart blank. The CSS and JS therefore live here as Python strings that get inlined into the HTML.

The layer is assembled from small pieces rather than one string, because they have genuinely different jobs and different pages want different subsets — the run index carries no episode payload, so it loads the utilities and the shell and none of the data layer:

module	what it holds
`css`	the design system: tokens, light/dark, the action-type grammar, layout, controls
`js_core`	`JS_UTIL` (formatting + DOM helpers) and `JS_CORE` (payload, view rehydration, deals)
`js_hover`	the rich hover card every chart point carries, incl. the solution-concept explanations
`js_chart`	the frontier chart (hover, pin, zoom/pan) and the regret strip
`js_transcript`	turn cards, the scrubber, lazily-built prompt bodies, turn selection
`js_sidebar`	the tabbed sidebar and the IntersectionObserver scroll sync that drives it
`js_shell`	theme toggle, episode navigation, keyboard bindings, the help overlay
`js_episode`	the episode page's wiring
`js_auction`	the auction episode page's wiring (stage scrubber, ladder hover, transcript sync)
`js_compare`	the comparison page's wiring
`js_index`	the run index's sort and filter

`JS` is the bundle every data page shares (utilities, data layer, charts, transcript, shell), so a page's own script is only its wiring. `JS_INDEX_PAGE` is the index's smaller bundle.

Colour. Both modes are explicitly selected, not derived by flipping the light values, and the categorical slots are capped at THREE. That cap is the binding constraint of the colour formula for a scatter: with all pairs of series simultaneously on screen (which is what a scatter does, unlike a bar chart's adjacent pairs), only the first three slots clear the colour-blindness

and normal-vision separation floors in both modes. The three slots carry the only three identities that must be told apart by colour:

========== ========== slot episode page comparison page ========== ========== 1 (blue) what the model actually did the left episode 2 (orange) what the oracle recommends the right episode 3 (aqua) normative solution points normative solution points ========== ==========

Everything else is encoded by **shape plus a direct label**: NBS, KS, and EGAL are aqua stars; UTIL and MNW are violet triangles; every concept is directly labelled on the chart. Violet is a redundant reference-point accent, not a fourth trajectory identity—the triangle and labels carry the distinction without colour. The per-party ideal points share one aqua diamond with the party named on hover and enumerated in the side panel's table. Deals themselves are chart chrome, not a series: dominated deals are muted dots, frontier deals carry the secondary-ink ring. Slot 3 sits below 3:1 against the light surface, which obligates the relief rule — hence the always-visible direct labels and the numeric table view that every chart ships with. Action types on the transcript are STATES rather than a series, so they wear the reserved status palette and always carry a glyph and a word beside the colour.

Modules

- `css` — The one stylesheet every page wears — a small design system, inlined.
- `js_auction` — The auction episode page's wiring: the DM stage scrubber, the hover card on the bid ladder, and the cross-links between every mark and the turn it belongs to.
- `js_chart` — Browser layer, part 2: the two charts.
- `js_compare` — The comparison page's wiring: one shared frontier carrying both trajectories, and two synchronized columns.
- `js_core` — Browser layer, part 1: the payload, the formatting helpers, and the deal-detail panel.

- `js_episode` — The episode page's own wiring: build the marks, render the panels, and keep chart and transcript in sync.
- `js_hover` — Browser layer, part 2a: the rich hover card that EVERY point on the frontier chart carries.
- `js_index` — The run index's browser layer: sort and filter, over the rows already in the document.
- `js_shell` — Browser layer, part 4: the page shell — theme toggle, episode navigation, keyboard shortcuts, help overlay.
- `js_sidebar` — Browser layer, part 5: the tabbed sidebar and the scroll sync that drives it.
- `js_transcript` — Browser layer, part 3: the transcript — turn cards, the scrubber, and lazy prompt bodies.

Re-exported

Name	Defined in	Summary
`CSS`	`interlens.arena.viz.assets.css`	
`JS_AUCTION`	`interlens.arena.viz.assets.js_auction`	
`JS_CHART`	`interlens.arena.viz.assets.js_chart`	
`JS_COMPARE`	`interlens.arena.viz.assets.js_compare`	
`JS_CORE`	`interlens.arena.viz.assets.js_core`	
`JS_EPISODE`	`interlens.arena.viz.assets.js_episode`	
`JS_HOVER`	`interlens.arena.viz.assets.js_hover`	
`JS_INDEX`	`interlens.arena.viz.assets.js_index`	
`JS_SHELL`	`interlens.arena.viz.assets.js_shell`	
`JS_SIDEBAR`	`interlens.arena.viz.assets.js_sidebar`	
`JS_TRANSCRIPT`	`interlens.arena.viz.assets.js_transcript`	
`JS_UTIL`	`interlens.arena.viz.assets.js_core`	

Attributes

Name	Type	Summary
JS		
JS_INDEX_PAGE		

css

Module `interlens.arena.viz.assets.css`

The one stylesheet every page wears — a small design system, inlined.

Three things make this a system rather than a pile of rules:

Tokens, in one place. Colour, spacing (`--sp-*`), type scale (`--t-*`) and radii are custom properties declared once. Light is the default declaration; dark is *selected* — the same roles re-stated for the dark surface under BOTH the OS media query and an explicit `data-theme` scope, so the page's own theme toggle wins in either direction rather than only being able to follow the OS.

Categorical colour is capped at three (`--s1`/`--s2`/`--s3`), the binding all-pairs constraint for a scatter; everything else is shape, position, or the reserved status palette. Action types on the transcript wear status colours (accept = good, reject = serious, walk = critical) because they are *states*, never a series — and each one ships with a glyph and a word, so colour never carries the meaning alone.

Nothing is decorative. Borders are hairlines, grid and axis ink is recessive, and the only heavy weights on the page are the numbers a reader came for.

Attributes

Name	Type	Summary
`CSS`		

js_auction

Module `interlens.arena.viz.assets.js_auction`

The auction episode page's wiring: the DM stage scrubber, the hover card on the bid ladder, and the cross-links between every mark and the turn it belongs to.

Deliberately thin. Every panel on an auction page is rendered server-side in :mod:`~interlens.arena.viz.auction_page`, including all of its SVG and all of its numbers, so this layer only adds behaviour that needs a pointer: with scripting off the page loses the stage filter and the rich hover, and keeps every mark, every `<title>` tooltip and every table cell.

It shares the transcript layer with the negotiation page (`turnCard` renders an auction turn without a game geometry, which is the graceful-degradation path that already existed), and it does not touch the frontier chart or the regret strip, neither of which an auction has.

Attributes

Name	Type	Summary
`JS_AUCTION`		

js_chart

Module `interlens.arena.viz.assets.js_chart`

Browser layer, part 2: the two charts.

The frontier chart places every deal in the instance's deal space, draws the efficient envelope, the play trajectory, and the reference marks, and lets a reader inspect any of the `|D|` deals — not only the marked ones — by hovering: one handler on the SVG finds the nearest deal in plot coordinates rather than attaching thousands of listeners. Hover opens the headline read in the side panel AND the rich hover card at the pointer (`js_hover`: what the point is, the deal in named options, the reward stats, and the party ranking); a click *pins* both. Every mark kind is covered by the same two calls, so a mark can never exist without a card.

It also zooms and pans, in plain SVG with no library: the view is the `viewBox`, so zooming is arithmetic on four numbers and every mark stays vector-crisp. Wheel-zoom is deliberately gated behind Ctrl/⌘ or Shift — an ungated wheel over a chart hijacks page scrolling, which is the single most irritating thing a chart can do — and the explicit +/−/reset buttons carry the same behaviour for anyone who would rather click. Because the pointer maths reads the *current* viewBox, nearest-deal hover keeps working at every zoom level.

The regret strip is one series (per-turn regret against the selected oracle), so it carries no legend: the title names it. Non-zero bars are direct-labelled and every bar jumps to its turn.

Both return a small handle so the page can drive them from elsewhere — above all `focusTurn`, which is how clicking a transcript turn lights up the deal it put on the table.

Attributes

Name	Type	Summary
`JS_CHART`		

js_compare

Module `interlens.arena.viz.assets.js_compare`

The comparison page's wiring: one shared frontier carrying both trajectories, and two synchronized columns.

The two columns render with DIFFERENT element-id prefixes (`lturn-` / `rturn-`). They have to: both sides number their turns from zero, so a single prefix put two elements with the same id on one page and every lookup silently resolved to whichever came first.

Attributes

Name	Type	Summary
`JS_COMPARE`		

js_core

Module `interlens.arena.viz.assets.js_core`

Browser layer, part 1: the payload, the formatting helpers, and the deal-detail panel.

Loaded first on every page. Three jobs:

Rehydration. Prompt views travel as indices into a de-duplicated message pool rather than as inlined text — a six-seat episode's thirty turns repeat the same system prompt thirty times and each view re-states the whole history so far, so the pool is worth roughly a third of the page. :js:func:`viewOf` turns a turn's indices back into `[{role, content}]` on demand; nothing else in the page knows the difference.

Formatting. One escape helper, one number formatter, one signed formatter, one "is this the good direction" class picker — so a value is never formatted two different ways on two different panels.

The action grammar. :js:func:`actKind` maps an action type to the class, glyph, and word the transcript, the scrubber, and the chart all wear for it. One mapping, so propose is the same blue everywhere it appears.

Attributes

Name	Type	Summary
`JS_CORE`		
`JS_UTIL`		

js_episode

Module `interlens.arena.viz.assets.js_episode`

The episode page's own wiring: build the marks, render the panels, and keep chart and transcript in sync.

The sync is two-way and goes through one function on each side. Clicking a numbered move on the frontier selects that turn in the transcript and scrolls to it; selecting a turn — by click, by scrubber chip, by regret bar, or with `j`/`k` — rings the deal that turn put on the chart. Before this, the chart could reach the transcript but the transcript could not reach the chart, so a reader following the text had to hunt for the corresponding point by eye.

Attributes

Name	Type	Summary
`JS_EPISODE`		

js_hover

Module `interlens.arena.viz.assets.js_hover`

Browser layer, part 2a: the rich hover card that EVERY point on the frontier chart carries.

The chart plots the whole deal space, so a reader hovering a dot is asking three questions at once — *what am I looking at*, *what deal is it*, and *who does it favour*. Before this the answer was a one-line SVG `<title>` plus a panel far below the chart, so identifying a point meant looking away from it. The card answers all three at the pointer:

1. **What it is.** Role-specific phrasing: a solution concept, a party's dictated best, an oracle's move at a turn, a numbered move by a seat, the deal that closed, or an ordinary point in the cloud.
2. **The deal**, in words — `Location: Kestrel Park · Power: solar with storage` — decoded client-side from the deal index against the issue/option name table the game payload already ships, so no per-deal strings travel.
3. **The numbers**: the deal-level summary (mean and min normalized surplus, Nash welfare, distance below the frontier, IR and feasibility flags) and a compact per-party table sorted by normalized surplus — the "who wins most here" ranking, with each party's utility, threshold, raw surplus and a bar.

The maths is written out, without a maths library. The five solution concepts are definitions, not labels, so each carries its formula and the one property that distinguishes it (Nash's axioms, KS's monotonicity, utilitarianism's *lack* of scale invariance, egalitarian maximin, MNW's Caragiannis fallback). Those definitions live in :mod:`interlens.arena.viz.concepts` and are serialized into the page from there, so a second place that explains a concept to a reader cannot disagree with this one. Pages are opened off `file://` with no network, so the formulae are HTML `<sub>` plus Unicode operators (`Σ Π τ - >`) rather than KaTeX or MathJax — they render everywhere, add nothing to the page weight, and copy as readable text.

One card, never under the cursor. A single element is reused for every point on the page, so two cards can never be open at once; it is offset from the pointer and flips side or vertical anchor near a viewport edge. A click *pins* it (the card becomes interactive and stays put until the next pick or `Escape`), which is also what makes it usable by touch, where there is no hover. Everything is styled from the shared CSS variables, so it follows the light/dark theme like the rest of the page.

Attributes

Name	Type	Summary
`JS_HOVER`		

js_index

Module `interlens.arena.viz.assets.js_index`

The run index's browser layer: sort and filter, over the rows already in the document.

Deliberately **not** payload-driven. The rows are rendered server-side as a real table — every number visible with scripting off — and sorting reads a `data-sort` attribute off each cell rather than a parallel JSON copy of the same data. A hundred-episode index therefore costs one table, not a table plus its duplicate.

Filtering is a text match across the row plus two chips (deal / no deal, has fabricated turns), and the count of what survives is always on screen, because a filter that silently hides rows is how a reader concludes a run has fewer episodes than it does.

Attributes

Name	Type	Summary
`JS_INDEX`		

js_shell

Module `interlens.arena.viz.assets.js_shell`

Browser layer, part 4: the page shell — theme toggle, episode navigation, keyboard shortcuts, help overlay.

Every page kind (episode, comparison, index) wears the same shell, so a reader learns the controls once. Three pieces:

Theme. The stylesheet declares dark twice — under the OS media query and under a `data-theme` stamp — and this is what does the stamping, remembering the choice in `localStorage` where that is allowed. It is wrapped in a `try` because a page opened over `file://` may have storage denied outright, and a viewer losing their theme preference must never cost them the page.

Navigation. Prev/next links and the episode picker are plain `<a>` and `<select>` elements rendered server-side, so they work with scripting off; this only adds the keyboard bindings and the picker's jump.

Shortcuts. `registerKeys` takes a map and does the two things every such map gets wrong: it ignores keystrokes aimed at a text field or a `<select>`, and it leaves modified keystrokes (Ctrl/⌘/Alt) alone so browser and OS shortcuts still work. The help overlay is generated from the SAME map, so a binding cannot exist without being documented.

Attributes

Name	Type	Summary
`JS_SHELL`		

js_sidebar

Module `interlens.arena.viz.assets.js_sidebar`

Browser layer, part 5: the tabbed sidebar and the scroll sync that drives it.

The sidebar's three live tabs all answer the same question — *what did the table look like at the point in the transcript I am reading?* — so they share one piece of state: the turn currently in the viewport, tracked with an `IntersectionObserver` over the turn cards rather than a scroll handler (no per-frame work, and the browser does the geometry).

- **Conversation** re-anchors the public chat to the acting seat: that seat's bubbles move to the right, everyone else's stay left, and the list scrolls itself so the turn being read is the one in the middle. The bubbles are rendered server-side and never rebuilt — the point of view is two class toggles.
- **Frontier** redraws the main chart's geometry restricted to what had been proposed by that turn: earlier proposals numbered, the deal standing on the table squared, later ones ghosted. It is drawn through the SAME `frontierChart` the page's main chart uses, so the two cannot drift apart.
- **Issues** shows the acting seat's private valuation of each issue, with a marker on the option the deal on the table picks. The bars, ticks and threshold line come from the server-rendered SVG; the marker is placed by reading each tick's own `data-y`, so the browser never re-derives the scale.
- **Info** is a standing reading guide. Inline information buttons open it directly at the oracle explanation.

The seat shown in the issue tab can be pinned with the picker; the next time scrolling changes which turn is in view the pin is released, because a pin that silently survived a scroll is how a reader ends up reading one seat's bars believing they are another's.

Attributes

Name	Type	Summary
JS_SIDEBAR		

js_transcript

Module `interlens.arena.viz.assets.js_transcript`

Browser layer, part 3: the transcript — turn cards, the scrubber, and lazy prompt bodies.

A turn card wears its action type: a coloured left edge, a chip with a glyph and the word, and a dashed edge when the seat was a computable policy rather than a model. The header is sticky, so scrolling a long reasoning trace never loses track of whose turn it is, and clicking it selects the turn (which lights up the deal that turn put on the chart).

A hazard note comes before the turn's analysis, not after it. "This turn is not what it looks like" — the engine fabricated it, or generation produced no publishable answer — has to reach a reader before the columns that discuss what the seat chose, because those columns are discussing a non-event.

Prompt bodies are built on first open, not on first paint. A six-seat thirty-turn episode carries a few hundred kilobytes of prompt text; turning all of it into DOM nodes before the reader has asked for any of it is the difference between a page that appears instantly and one that hitches. The `<details>` ships with its summary and an empty body, and one delegated `toggle` listener fills it the first time it is opened.

The scrubber is the transcript's map: one chip per turn, coloured by action type, fabricated turns ringed in the critical colour, the current turn filled. It is how a reader gets from "something went wrong around the end" to the turn in one click instead of a scroll hunt.

Attributes

Name	Type	Summary
`JS_TRANSCRIPT`		

auction_geometry

Module `interlens.arena.viz.auction_geometry`

The plottable geometry of one repeated-auction episode — the `AuctionSpec`-shaped sibling of :class:`~interlens.arena.viz.geometry.GameGeometry`.

`GameGeometry` does not transfer and was never going to: it materializes the whole `|D| x n` utility matrix of an enumerable deal space, and an auction has neither (bids are effectively continuous, multi-item allocation is combinatorial). `GameGeometry.from_instance` returns `None` on an auction payload, at which point every negotiation panel degrades away — so this module supplies what replaces them, in the same shape: one object built per instance, one `to_json` the browser reads, and a per-episode trace beside it.

Two things live here.

:class:`AuctionGeometry` is **instance-scoped and episode-independent**: the mechanism, the five public cards, the lot catalogue with its public attribute loadings, and every stage's frozen draw (realized valuations, budgets, synergy targets, tie-break, clock ceiling). It is the post-hoc analyst's view — the private draws are on it, which is exactly why the page that renders it says so in as many words.

:func:`auction_trace` is **episode-scoped** and is the part that needs a replay. A stored auction turn records its `parsed_action` but not the stage it fell in, nor the standing-bid table it was decided against, nor the remaining budget: all three live in the scenario's state machine. Rather than re-deriving them with a heuristic (a seat's second move in a stage is NOT a new stage under SAA, where a stage runs many bidding rounds), the trace replays the episode through :class:`~interlens.arena.scenarios.auction.AuctionScenario` and reads the real state at every turn. That is also what makes the per-turn counterfactuals honest: the two computable rules are evaluated against the *pre-move* state block the seat itself decided in, via :mod:`~interlens.arena.scenarios.auction_policy`, which is the same path the campaign's replay-integrity gate uses on computable seats. Every turn of every arm gets both, `all_llm` included, because both are arithmetic given the spec.

Example:

```
geo =
     AuctionGeometry.from_instance(instance, cell_cfg=episode["cell_cfg"])
geo.spec.mechanism.family                        # 'saa'
trace = auction_trace(episode, instance, geometry=geo)
trace["turns"][7]["counterfactual"]["rational"] # {'action': 'bid',
     'bids': [...]}
trace["stages"][0]["winners"]                    # per-lot winning
     seat index
```

Attributes

Name	Type	Summary
`COORDINATION_TALK_TERMS`	`tuple[str, ...]`	
`COUNTERFACTUAL_RULES`	`tuple[tuple[str, str], ...]`	

Classes

Name	Summary
`AuctionGeometry`	One auction instance's full post-hoc geometry, ready to plot.

Functions

Name	Summary
`agreement_in_force_row`	:func:`~interlens.arena.auction.metrics.detect_agreement`'s rule applied to a STORED stage row instead of a live `StageOutcome` — whether an agreement was in force at the stage this row records.

Name	Summary
`auction_trace`	Replay one stored auction episode and return everything the auction panels plot.
`index_row`	The auction index's columns for one episode, derived from the STORED record with no replay.
`is_auction_instance`	Whether a stored `Instance` dict carries an auction spec — the discriminator the viz layer branches on.
`json_safe`	`value` with every non-finite float replaced by `None`, recursively through dicts and lists.

AuctionGeometry

One auction instance's full post-hoc geometry, ready to plot.

```
AuctionGeometry(spec, *, difficulty: dict | None = None, screens: dict |
    None = None)
```

Defined in `interlens.arena.viz.auction_geometry` · source

Built once per instance and shared by every episode played on it, so both sides of a paired contrast are drawn against one identical set of draws. Cheap to build (no matrices are materialized), so unlike :class:`~interlens.arena.viz.geometry.GameGeometry` the caching is a convenience rather than a necessity.

Parameters

spec : AuctionSpec The spec the episode was ACTUALLY played on — the cell's mechanism, horizon, channel and card scramble applied on top of the frozen bank draws. Build it with :meth:`~interlens.arena.scenarios.auction.AuctionScenario.spec_for` rather than from the bank's nominal mechanism: a single-lot bank backs the sealed cells AND the Dutch ones, and replaying a Dutch episode against a sealed mechanism scores every turn against the wrong rule. difficulty : dict, optional The instance's `solution.difficulty` record (scalar, components, tags) — carried opaquely so a generator can add a component without a schema change here. The index sorts on `scalar` and `tags`. screens : dict, optional The bank's outcome-blind screen record, shown as provenance.

Parameters

Name	Type	Default	Description
`spec`		*required*	
`difficulty`	`dict | None`	`None`	
`screens`	`dict | None`	`None`	

Attributes

Name	Type	Summary
`difficulty`		
`horizon`	`int`	`T`, the number of stages this episode ran.
`n_bidders`	`int`	Number of seats.
`n_items`	`int`	Number of distinct lots per stage.
`screens`		
`spec`		

Methods

`from_instance`

```
from_instance(
    instance: dict | None,
    cell_cfg: dict | None = None,
) -> 'AuctionGeometry | None'
```

source

The geometry of a stored auction `Instance` dict at a cell's config, or `None` if the payload is not an auction spec (so the caller renders a non-auction episode by its own path rather than crashing).

`cell_cfg` is the episode's own `cell_cfg`: it selects the value structure and carries the mechanism/horizon/channel/scramble the cell overrode. Passing `None` reads the bank's nominal spec, which is right for a bank browser and wrong for an episode page.

Parameters

Name	Type	Default	Description
`instance`	`dict | None`	*required*	

Name	Type	Default	Description
cell_cfg	dict | None	None	

lot_ids

```
lot_ids(self) -> list[str]
```

source

The printed lot ids in slot order (`L01`, `L02`, …) — the same strings the action grammar and the transcript use, read from the prompt module so the page cannot invent a second naming.

price_ceiling

```
price_ceiling(self) -> float
```

source

The top of the shared price axis every panel plots on: the largest realized valuation anywhere in the episode, or the clock ceiling where that is higher.

One scale across all stages is what makes the staged ladder readable as one figure rather than `T` unrelated charts.

to_json

```
to_json(self) -> dict
```

source

The whole instance geometry as one JSON payload for the browser.

Everything private is under `stages[].values` / `budgets` / `synergy_target` and is labelled as the analyst's view by the page that renders it — no seat ever saw another seat's row.

agreement_in_force_row

:func:`~interlens.arena.auction.metrics.detect_agreement`'s rule applied to a STORED stage row instead of a live `StageOutcome` — whether an agreement was in force at the stage this row records.

```
agreement_in_force_row(row: dict, *, theta: float, min_suppressors: int =
    2) -> bool
```

Defined in `interlens.arena.viz.auction_geometry` · source

The two are the same rule against two different inputs. `detect_agreement` runs inside the scenario, where the full bid and benchmark matrices are live; a stored episode keeps only the aggregate row (`revenue`, `benchmark_revenue`, `suppression_per_seat`, `winner_of`), which is all a post-hoc reader has. It lives here rather than in `metrics` because reading stored records is what this layer does — but it is the same rule, and a detector that drifted from the one the analysis counts would make the band the ladder shades a picture of a different event.

Returns `False` for a stage with no winner, and for one whose benchmark revenue is undefined (a real state on an on-path multi-lot benchmark) — an absent benchmark cannot support the claim that the price fell below it.

Parameters

Name	Type	Default	Description
`row`	`dict`	*required*	
`theta`	`float`	*required*	
`min_suppressors`	`int`	`2`	

auction_trace

Replay one stored auction episode and return everything the auction panels plot.

```
auction_trace(
    episode: dict,
    instance: dict,
    *,
    geometry: AuctionGeometry | None = None,
    counterfactuals: bool = True,
) -> dict | None
```

Defined in `interlens.arena.viz.auction_geometry` · source

The replay is exact — the scenario is a pure state machine and the stored turns are its inputs — so every field below is the state the seat actually decided in, not an inference from the turn log.

Parameters

episode : dict A stored `Episode.to_json()` record. instance : dict The `Instance` record it was played on. geometry : AuctionGeometry, optional A prebuilt geometry to reuse (pass the same object for both sides of a comparison). Built from `episode["cell_cfg"]` when omitted. counterfactuals : bool Compute the per-turn rational and oracle references. On by default because they are the campaign's headline instrumentation; `False` is the fast path for a bulk index build, and costs roughly 4x less on a ten-lot SAA episode.

Returns

dict | None `{"geometry", "turns", "stages", "ladder", "channel", "onset", "replay"}`, or `None` if the episode is not an auction episode or the replay does not reconstruct (a missing panel is a much better outcome than a crashed export, so every failure mode here returns `None` with its reason on `replay`).

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`instance`	`dict`	*required*	
`geometry`	AuctionGeometry | None	`None`	
`counterfactuals`	`bool`	`True`	

index_row

The auction index's columns for one episode, derived from the STORED record with no replay.

```
index_row(episode: dict, instance: dict | None = None) -> dict
```

Defined in `interlens.arena.viz.auction_geometry` · source

The one owner of what those columns mean. Two callers need them from two different inputs and must not disagree: :func:`~interlens.arena.viz.export._auction_fields` has a full render payload in hand and adds the counterfactual-agreement column on top of this, while a campaign hub listing every episode in the campaign has only the episode JSON and cannot afford a replay per row — 1,000 replays to fill a table is minutes of work for columns that are all in the stored outcome already.

`onset` uses :mod:`interlens.arena.auction.metrics`, so the index's onset column and the survival analysis count the same event. `difficulty` comes from the instance when one is supplied.

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`instance`	`dict | None`	`None`	

is_auction_instance

Whether a stored `Instance` dict carries an auction spec — the discriminator the viz layer branches on.

```
is_auction_instance(instance: dict | None) -> bool
```

Defined in `interlens.arena.viz.auction_geometry` · source

Checks the payload's shape rather than the record's `scenario` field, because a bank instance is generated once and consumed by cells that override the mechanism, and because a comparison payload may carry an instance whose scenario string was never written.

Parameters

Name	Type	Default	Description
`instance`	`dict | None`	*required*	

json_safe

`value` with every non-finite float replaced by `None`, recursively through dicts and lists.

```
json_safe(value)
```

Defined in `interlens.arena.viz.auction_geometry` · source

An auction outcome legitimately carries `NaN`: a stage with no losing bid has no suppression denominator and an on-path multi-lot benchmark has no counterfactual revenue. Both are “no measurement here”, and both are hostile to a web page in two distinct ways — `json.dumps` emits the bare token `NaN`, which `JSON.parse` rejects and which therefore disables every script on the page, and a formatter that prints it renders the absence of a measurement as the word “nan” beside real numbers. Converting to `None` at the payload boundary fixes both, and it is the same claim the analysis makes: absent, not zero.

Parameters

Name	Type	Default	Description
`value`		*required*	

auction_page

Module `interlens.arena.viz.auction_page`

The auction episode page's own panels — the four charts design.md §10 commits to, plus the per-turn counterfactual table.

Split out of :mod:`~interlens.arena.viz.page` rather than branched inside it, because none of these panels share a line with the negotiation ones: an auction has no deal space to embed, so there is no frontier chart, no solution-concept legend, and no regret strip. What IS shared comes back in through `page.py` — the shell, the transcript, the census and vintage badges, the prompt audit, and the chat bubbles — so this module renders only what is genuinely new.

Everything here is server-rendered SVG and HTML. The browser layer (:data:`~interlens.arena.viz.assets.js_auction.JS_AUCTION`) adds the stage scrubber, the hover cards, and the transcript cross-links; with scripting off, every number and every mark is still in the document, which is the same contract the rest of the visualizer keeps.

The four panels, in the order the page carries them:

1. :func:`bid_ladder` — price against round, stages laid out left to right on one shared price scale, one line per seat, exits and standing-high transitions marked, private valuations as per-stage reference ticks, with the collusion-onset stage and every detected defection shaded onto the stage axis.
2. :func:`allocation_strip` — one bar per lot per stage, private valuations as ticks, the clearing price as the tau-line. A direct adaptation of `page._issue_bars_svg`'s scale-and-tick construction.
3. :func:`settlement_panel` — winner, payment and surplus per stage, with the episode total.
4. :func:`dm_graph` — the directed message graph with a stage scrubber and the per-dyad counts beside it.

Attributes

Name	Type	Summary
GRAPH		
LADDER		
STRIP		

Functions

Name	Summary
allocation_strip	One bar per lot per stage: every seat's private valuation as a tick, the clearing price as the tau-line.
auction_body	The auction episode page's four panels plus the counterfactual table, in reading order.
auction_info_panel	The auction page's reading guide — what every measure on the page means and what it does not.
auction_setup_panel	The side panel: the mechanism, the five public cards, the lot catalogue, and every stage's frozen draw.
auction_summary_strip	The whole auction episode in one row: the format, how efficient it was, what it raised against benchmark, whether anything was suppressed, and how much of it was model behaviour at all.
bid_ladder	Price against bidding round, every stage of the episode side by side on one shared price scale.

Name	Summary
`counterfactual_table`	Every committed turn beside what the two computable rules would have played there.
`dm_graph`	The directed message graph over the seats, with a stage scrubber and the per-dyad counts beside it.
`settlement_panel`	Winner, payment and surplus per stage, with the episode total — what replaces `chrome.summary_strip`'s negotiation field list.

allocation_strip

One bar per lot per stage: every seat's private valuation as a tick, the clearing price as the tau-line.

```
allocation_strip(auction: dict, payload: dict) -> str
```

Defined in `interlens.arena.viz.auction_page` · source

The adaptation of `page._issue_bars_svg` the tooling map identified as the highest-leverage reuse in the whole viz stack — same vertical bar, same tick-per-value construction, same single reference rule — with the issue's option scores replaced by the five seats' valuations and the threshold replaced by the price the lot actually cleared at. The winner's tick is filled, so "who won it and at what price against what it was worth to everyone" is one glance per lot.

A lot that went unsold (no bid above reserve) carries no tau-line, which is what an absent clearing price means and is deliberately not drawn as a price of zero.

Parameters

Name	Type	Default	Description
`auction`	`dict`	*required*	
`payload`	`dict`	*required*	

auction_body

The auction episode page's four panels plus the counterfactual table, in reading order.

```
auction_body(payload: dict) -> str
```

Defined in `interlens.arena.viz.auction_page` · source

Returned as one string for `page.render_episode_html` to place between the shared header and the shared transcript, so the auction branch of that function stays a handful of lines and the shell, census, vintage badges and prompt audit are reused rather than re-implemented.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

auction_info_panel

The auction page's reading guide — what every measure on the page means and what it does not.

```
auction_info_panel() -> str
```

Defined in `interlens.arena.viz.auction_page` · source

auction_setup_panel

The side panel: the mechanism, the five public cards, the lot catalogue, and every stage's frozen draw.

```
auction_setup_panel(payload: dict) -> str
```

Defined in `interlens.arena.viz.auction_page` · source

The draws are the whole private half of the episode, so the panel leads with the fact that reading it is a post-hoc act and states the card scramble where one was applied — an X-cell page whose panel did not say the cards were deranged would show a reader a persona/value pairing that no seat ever faced.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

auction_summary_strip

The whole auction episode in one row: the format, how efficient it was, what it raised against benchmark, whether anything was suppressed, and how much of it was model behaviour at all.

```
auction_summary_strip(payload: dict) -> str
```

Defined in `interlens.arena.viz.auction_page` · source

The negotiation strip's fields (deal / distance to NBS / Gini / below-τ) have no auction analogue and are replaced rather than reinterpreted; :func:`~interlens.arena.viz.chrome.stat` is shared.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

bid_ladder

Price against bidding round, every stage of the episode side by side on one shared price scale.

```
bid_ladder(auction: dict, payload: dict) -> str
```

Defined in `interlens.arena.viz.auction_page` · source

One polyline per seat per stage through the highest price that seat committed in each round — on a clock format that is the clock price its `stay`/`claim` happened at, since a clock move carries no amount of its own. Individual lot bids under SAA are drawn as marks rather than folded into the line, because a line through ten simultaneous lot bids would trace a number nobody bid. A mark that took the standing high is filled; an irrevocable exit is a cross.

Two overlays make the repeated-play story readable from the one figure: each seat's own top realized valuation for the stage as a reference tick (post-hoc — no seat saw another's), and a shaded band on any stage where the outcome rule found an agreement in force, with the onset stage and every defection labelled on the stage axis.

`None`-safe: a stage with no priced action at all renders as an empty stage column rather than being dropped, so the stage axis always counts to `T`.

Parameters

Name	Type	Default	Description
`auction`	`dict`	*required*	
`payload`	`dict`	*required*	

counterfactual_table

Every committed turn beside what the two computable rules would have played there.

```
counterfactual_table(auction: dict, payload: dict) -> str
```

Defined in `interlens.arena.viz.auction_page` · source

This is the “what I want to see” requirement made concrete for auctions: for each turn, the seat’s actual move, the information-conditional Bayesian bidder’s move at that same state, and the omniscient bidder’s. Both are arithmetic given the spec, so both are present on every turn of **every** arm, `all_llm` included — that is what makes the LLM’s move comparable to the rules rather than only to other LLMs.

The agreement column is the headline read: in a dominant-strategy mechanism the rational move is budget-capped truthful bidding, and a table of agreements says the seats found it. Message turns carry no binding move and are omitted rather than shown as agreements on nothing.

Parameters

Name	Type	Default	Description
`auction`	`dict`	*required*	
`payload`	`dict`	*required*	

dm_graph

The directed message graph over the seats, with a stage scrubber and the per-dyad counts beside it.

```
dm_graph(auction: dict, payload: dict) -> str
```

Defined in `interlens.arena.viz.auction_page` · source

Edge width is the message count and the scrubber restricts the graph to one stage at a time (every stage's edges are in the document; the browser only changes which are visible, so the panel still reads with scripting off). Beside it sits the per-dyad table with the coordination-talk screen's hit count.

What is deliberately NOT here is per-dyad mutual information with a permutation p-value. That estimate is cell-level: it needs the whole cell's stages, and computing it over one episode's six would be noise wearing a p-value. The campaign hub carries it, beside these counts, at the level it is defined at.

Parameters

Name	Type	Default	Description
`auction`	`dict`	*required*	
`payload`	`dict`	*required*	

settlement_panel

Winner, payment and surplus per stage, with the episode total — what replaces `chrome.summary_strip`'s negotiation field list.

```
settlement_panel(auction: dict, payload: dict) -> str
```

Defined in `interlens.arena.viz.auction_page` · source

Its `stat()` cell builder is reused verbatim; none of the fields are, because an auction has no deal, no threshold and no Nash product. What it does have is a benchmark, so every revenue figure is printed beside the benchmark revenue it is measured against, and a stage whose benchmark is undefined (an on-path SAA demand has no counterfactual outcome) prints an em dash there rather than a ratio nobody can compute.

Parameters

Name	Type	Default	Description
`auction`	`dict`	*required*	
`payload`	`dict`	*required*	

ballots

Module `interlens.arena.viz.ballots`

The final vote, as a tally a reader can check at a glance — including the ballots that were never recorded.

The forced final is the one turn where every seat must answer the same question about the same package, so it is the one place in an episode where a missing answer is unambiguous. It is also where a defect hid for weeks: a computable seat cast its up/down vote on whichever live offer it valued most rather than on the offer actually under the vote, the protocol rejected that as illegal, the seat repeated itself on its single retry, and the turn was recorded as a **pass**. A pass parses cleanly, so the episode closed, the status said `done`, and 99% of one arm's final ballots were silent abstentions that nothing on the page mentioned.

So the tally is deliberately built around absence. Every seat that should have voted gets a row whether or not it produced a ballot, and a seat with no recorded ballot is called an abstention in the loudest style the page has.

And "abstention" is where the page stops describing and starts quoting, because the word undersells what happened. The seat did not decline to vote: it voted, on the wrong offer id, and the record kept nothing. Both halves of that signature are in the record — the protocol's rejection (`"The final vote is only on P9; reference that offer id."`) and the seat's own response (`{"action": "accept", "offer_id": "P6"}`) — so the row prints both verbatim rather than paraphrasing either. Everything downstream that called this a silent abstention was describing the record's shape instead of the agent's behaviour.

Derived ballots. Where a computable policy occupied the seat, its vote has an offline answer, and gate G3 (`experiments/rational_agents/gate_seeded_offer_votes.py`) already re-derives exactly that by replaying the policy against the view the seat really saw. Re-implementing it in the renderer would be a second opinion with no authority, so this reads the gate's own output instead, from an optional `vote_derivation.json` at the run root with the shape:

```
{"episodes": {"<episode_id>": {"<turn_idx>": {"expected": {...},
     "recorded": {...}, "match": false}}}}
```

Absent sidecar, absent run, or an episode the gate did not cover: the derived column is omitted and the recorded tally stands on its own, which is already the part that makes the defect visible.

Attributes

Name	Type	Summary
`ATTEMPT_EXCERPT_CHARS`		
`BALLOT_ACTIONS`		
`DERIVATION_SIDECAR`		
`FINAL_PROPOSAL_PHASE`		
`FINAL_VOTE_PHASE`		

Functions

Name	Summary
`ballot_table`	The final-vote tally as a card, or nothing when the episode has no final-vote phase.
`final_ballots`	The episode's final vote as `{offer, rows, n_ballots, n_abstentions, n_retries, n_mismatch, derived}`.
`vote_derivation`	The optional per-turn re-derivation sidecar for a run, or `None` when it is absent or unreadable.

ballot_table

The final-vote tally as a card, or nothing when the episode has no final-vote phase.

```
ballot_table(tally: dict | None) -> str
```

Defined in `interlens.arena.viz.ballots` · source

An abstention is styled as a hazard and stated in words, because the count is the thing that was missing: a reader who can see “1 of 5 seats cast no ballot” on the page does not need a forensic audit to find the defect that produced it. The derived column appears only when a sidecar covered at least one turn, and a recorded-vs-derived disagreement is called a harness bug in the row itself.

Parameters

Name	Type	Default	Description
`tally`	`dict | None`	*required*	

final_ballots

The episode's final vote as `{offer, rows, n_ballots, n_abstentions, n_retries, n_mismatch, derived}`.

```
final_ballots(payload: dict, derivation: dict | None = None) -> dict
```

Defined in `interlens.arena.viz.ballots` · source

`rows` is one entry per PUBLISHED final-vote turn in order, carrying the seat, the kind of occupant, the recorded ballot, the parser's complaint when there was one, and — when the sidecar covers the turn — what the seat's own policy re-derives and whether the record matches it. `offer` is the package under the vote, taken from the offer id the forced final proposal was registered under and falling back to whichever id the ballots themselves reference, so a protocol whose final proposal is not in the ledger still names its own question.

Returns `{"rows": []}` for an episode with no final-vote phase, which is most protocols; the page renders nothing rather than an empty table.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	
`derivation`	`dict | None`	`None`	

vote_derivation

The optional per-turn re-derivation sidecar for a run, or `None` when it is absent or unreadable.

```
vote_derivation(run_root: str | Path | None) -> dict | None
```

Defined in `interlens.arena.viz.ballots` · source

Unreadable is treated as absent on purpose: a malformed sidecar must cost the page one optional column, not the whole render.

Parameters

Name	Type	Default	Description
`run_root`	`str | Path | None`	*required*	

census

Module `interlens.arena.viz.census`

How much of an episode is actually play: the per-turn census, and the strip that puts it in the page header.

The existing contamination banner counts turns the ENGINE fabricated, and it is right to be loud about them. But it is a screen for one cause, and a turn can carry nothing for reasons the engine never sees:

- a thinking model spends its entire per-turn budget inside an unterminated `<think>` block, so the harness substitutes the same placeholder text — with `gen_failed` false, because generation succeeded, it just never produced a visible answer;
- a move is rejected as illegal, the seat repeats itself on its one retry, and the turn is recorded as a pass;
- a seat talks and takes no formal action at all, which is legal play but is not a move.

All three render as an ordinary, well-formed, quiet turn. A campaign cell reached **24% silent turns while passing every gate**, including the fabrication gate, because 0.000 fabricated was the honest answer to the only question anyone was asking. This module asks the other questions, on one episode, and puts the answers where a reader cannot walk past them: non-action rate, placeholder count, at-cap count, and the same three by round.

Attributes

Name	Type	Summary
`AT_CAP_SLACK`		

Functions

Name	Summary
`at_cap`	Whether one payload turn spent essentially its whole generation budget.
`census_strip`	The per-episode census as a compact header strip, always rendered when there are turns to count.
`turn_census`	Count the ways this episode's turns carried nothing, from the payload's own turn rows.

at_cap

Whether one payload turn spent essentially its whole generation budget.

```
at_cap(row: dict) -> bool
```

Defined in `interlens.arena.viz.census` · source

There is no `stop_reason` on the local generation path, so “ran out of room” has to be inferred from output length against the per-turn cap the scenario stamped on the request. A turn with no recorded cap cannot be judged and is not counted.

Parameters

Name	Type	Default	Description
`row`	`dict`	*required*	

census_strip

The per-episode census as a compact header strip, always rendered when there are turns to count.

```
census_strip(census: dict | None) -> str
```

Defined in `interlens.arena.viz.census` · source

Present even when everything is zero, because “this episode has no silent turns” is the claim a reader needs and an absent strip cannot make it. Non-zero counts are styled as hazards and every cell carries its by-round breakdown on hover.

Parameters

Name	Type	Default	Description
`census`	`dict | None`	*required*	

turn_census

Count the ways this episode's turns carried nothing, from the payload's own turn rows.

```
turn_census(rows: list[dict]) -> dict
```

Defined in `interlens.arena.viz.census` · source

`rows`
are the payload turns (see :func:`~interlens.arena.viz.episode.episode_payload`), which already carry the three facts this needs — `silent` and `gen_failed` from the placeholder screen, `action.atype` from the parse, and `cap` / `n_tokens_out` from generation accounting. Deriving the census from the same rows the page renders is deliberate: a header that disagreed with the transcript below it would be worse than no header.

Returns `{n_turns, placeholder, placeholder_budget_burned, placeholder_engine_failure, non_action, non_action_rate, at_cap, at_cap_rate, placeholder_rate, by_round: [...], clean}`. `by_round` is one row per round in play order with the same counts, which is what turns "a quarter of this episode is silent" into "and all of it is in the last two rounds". `clean` is true only when every count is zero.

Parameters

Name	Type	Default	Description
`rows`	`list[dict]`	*required*	

chrome

Module `interlens.arena.viz.chrome`

The shell every page wears, and the wire form of the payload it carries.

Two things live here because both are shared by all three page kinds and neither belongs to any one of them.

The chrome — the sticky top bar (run identity, episode navigation, the quick read, theme and help buttons) and the keyboard-help overlay. Navigation is rendered as plain links and a `<select>`, so it works with scripting off; the browser layer only adds the shortcuts.

Where the navigation goes is marked with :data:`NAV_MARKER` rather than filled in at render time. A page cannot know its own siblings — `render_episode_html` is handed one payload — while the exporter, which writes the whole run, knows all of them but only after the last page is rendered. So the exporter writes the pages and then replaces one comment in each. A page rendered on its own keeps the comment, which is invisible.

The wire payload — :func:`slim_payload` replaces every turn's inlined prompt view with indices into one de-duplicated message pool. A six-seat thirty-turn episode repeats its system prompt on every turn and re-states the whole history in each view, so the same bytes were being shipped dozens of times: on a representative 30-turn page the pool takes the embedded data from 566 KB to 332 KB with nothing removed. The returned dict is a render-time copy — :meth:`RunDir.payload`'s own return value is untouched, because it is a public API whose consumers expect real message dicts.

Attributes

Name	Type	Summary
`NAV_MARKER`		

Functions

Name	Summary
`distance_to_nbs`	How far the deal that closed sits from the Nash bargaining solution, in the chart's own plane.
`help_overlay`	The keyboard-help overlay.
`inject_nav`	Put a page's navigation into its top bar.
`nav_group`	The prev/next links and episode picker for the page at `position` in `rows`.
`quick_stats`	The two-or-three-number read that rides in the top bar and stays visible while scrolling.
`slim_payload`	A copy of `payload` whose per-turn prompt views are indices into a shared `msgpool`.
`stat`	One cell of the summary strip: a small caption, the number, and a line of context under it.
`summary_strip`	The whole episode in one row: did it close, how good was it, how far from the normative anchor, who was left below their threshold, how much of it the engine fabricated, how long it ran, what it cost.
`topbar`	The sticky top bar: run identity on the left, the navigation slot, the quick read, theme and help.

Name	Summary
`unslim_payload`	The inverse of :func:`slim_payload`: pooled view indices back to full `{role, content}` messages.

distance_to_nbs

How far the deal that closed sits from the Nash bargaining solution, in the chart's own plane.

```
distance_to_nbs(payload: dict) -> float | None
```

Defined in `interlens.arena.viz.chrome` · source

Euclidean distance in the two scale-invariant coordinates the frontier chart plots — joint welfare (mean normalized surplus) and the worst-off party's normalized surplus — between the agreed deal and the NBS deal. `None` when no deal closed or the instance carries no solution set. It is a *projected* distance by construction, like the chart: two deals that differ only along dimensions the projection drops read as identical here, which is why the number sits beside the exact per-party table rather than replacing it.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

help_overlay

The keyboard-help overlay.

```
help_overlay(extra: str = '') -> str
```

Defined in `interlens.arena.viz.chrome` · source

Its table body is filled by the browser layer from the SAME binding list the handler uses, so a shortcut cannot exist without being documented here.

Parameters

Name	Type	Default	Description
extra	str	''	

inject_nav

Put a page's navigation into its top bar.

```
inject_nav(page_html: str, nav: str) -> str
```

Defined in `interlens.arena.viz.chrome` · source

A no-op on a page that carries no marker.

Parameters

Name	Type	Default	Description
`page_html`	`str`	*required*	
`nav`	`str`	*required*	

nav_group

The prev/next links and episode picker for the page at `position` in `rows`.

```
nav_group(rows: list[dict], position: int, *, label_key: str = 'label') -
    > str
```

Defined in `interlens.arena.viz.chrome` · source

Written by the exporter into :data:`NAV_MARKER` once every page of the run is known. Prev/next are real `<a href>` elements (so they work without scripting, and the keyboard bindings just follow them); the picker is a `<select>` of every sibling page. A page at either end gets a disabled link rather than a missing one, so the control row never changes width as a reader walks the run.

Parameters

Name	Type	Default	Description
`rows`	`list[dict]`	*required*	
`position`	`int`	*required*	
`label_key`	`str`	`'label'`	

quick_stats

The two-or-three-number read that rides in the top bar and stays visible while scrolling.

```
quick_stats(payload: dict) -> str
```

Defined in `interlens.arena.viz.chrome` · source

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

slim_payload

A copy of `payload` whose per-turn prompt views are indices into a shared `msgpool`.

```
slim_payload(payload: dict) -> dict
```

Defined in `interlens.arena.viz.chrome` · source

Identical `(role, content)` messages are pooled once and every view becomes a list of integers, which the browser turns back into messages on demand (`viewOf`). Purely a transport change: the rendered page shows exactly the same text, and any turn that carries no view keeps its `None`. Comparison payloads carry two sides, so both are slimmed into ONE pool — the two episodes of a seat swap share a system prompt and most of their history, which is precisely where the duplication is worst.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

stat

One cell of the summary strip: a small caption, the number, and a line of context under it.

```
stat(key: str, value: str, note: str = '', *, bad: bool = False) -> str
```

Defined in `interlens.arena.viz.chrome` · source

Parameters

Name	Type	Default	Description
`key`	`str`	*required*	
`value`	`str`	*required*	
`note`	`str`	`''`	
`bad`	`bool`	`False`	

summary_strip

The whole episode in one row: did it close, how good was it, how far from the normative anchor, who was left below their threshold, how much of it the engine fabricated, how long it ran, what it cost.

```
summary_strip(payload: dict) -> str
```

Defined in `interlens.arena.viz.chrome` · source

Replaces the old grid of tiles. Same numbers, a fifth of the vertical space — which matters because it sits above the chart, and the chart is what a reader came for.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

topbar

The sticky top bar: run identity on the left, the navigation slot, the quick read, theme and help.

```
topbar(
    brand: str,
    brand_href: str | None,
    quick: str = '',
    *,
    brand_title: str = '',
    nav: bool = True,
) -> str
```

Defined in `interlens.arena.viz.chrome` · source

`brand` names the run; `brand_href` links it to the run index, or is `None` on the index itself, where there is nothing above to go up to. `quick` is the pre-rendered stat run — the two or three numbers a reader wants without scrolling. The navigation slot is :data:`NAV_MARKER`, omitted entirely when `nav` is false (the run index has no siblings to walk).

Parameters

Name	Type	Default	Description
`brand`	`str`	*required*	
`brand_href`	`str | None`	*required*	
`quick`	`str`	`''`	
`brand_title`	`str`	`''`	
`nav`	`bool`	`True`	

unslim_payload

The inverse of :func:`slim_payload`: pooled view indices back to full `{role, content}` messages.

```
unslim_payload(payload: dict) -> dict
```

Defined in `interlens.arena.viz.chrome` · source

A published page embeds the SLIMMED payload, so this is what makes an already-written page re-renderable — the only way to refresh a page whose original run directory is gone. A payload that carries no `msgpool` is already inflated and is returned unchanged, so the call is safe to make unconditionally.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

compare

Module `interlens.arena.viz.compare`

Seat-swap comparison: the same game instance played twice, with one seat's occupant swapped.

The scientific question this renders is a *substitution* effect — hold the instance, the seed, the arm, and every other seat fixed; replace the occupant of one seat (a computable rational policy vs an LLM); read off what changed. Two episodes that share an instance and seed start from the identical opening state and then diverge at the first move the two occupants play differently, after which every later turn is a different state and the transcripts are no longer aligned in any deeper sense than their turn slots. The renderer therefore does exactly two things:

1. **Align turn slots** on `(round, phase, seat)` — the fixed-rotation protocol makes those slots comparable — and mark the FIRST slot whose public behaviour differs as the divergence point. Everything after it is shown as two independent trajectories, never as a per-turn "diff", because after divergence the two seats are answering different questions.
2. **Quantify the outcome difference** with paired deltas on the metrics that carry the claim: the focal seat's surplus and its share of what was available to it, deal rate, joint welfare, the primary score, and each party's realized surplus.

Pairing is a *key*, not a heuristic: `pair_key` builds it from the fields that define "the same problem played the same way", and an unmatched episode is reported rather than approximately matched.

Attributes

Name	Type	Summary
`DEFAULT_PAIR_KEY`		

Name	Type	Summary
SELECTIONS		

Functions

Name	Summary
align	Align two episode payloads slot by slot and locate the divergence point.
compare_payload	One seat-swap comparison, ready to render: both episode payloads, the slot alignment, the divergence point, the focal seat(s), and the score table.
focal_seats	The seats whose OCCUPANT KIND differs between the two episodes — the substitution being measured.
pair_key	The pairing key of an episode: the tuple of its fields.
pair_runs	Pair every episode of one run against its key-matched counterpart in another, and build a comparison payload for each.
score_table	The quantified comparison: one row per metric with both values and the paired delta right - left.

align

Align two episode payloads slot by slot and locate the divergence point.

```
align(left: dict, right: dict) -> tuple[list[dict], int | None]
```

Defined in `interlens.arena.viz.compare` · source

Returns `(rows, divergence)` where each row is `{round, phase, seat, attempt, left_idx, right_idx, different}` — the turn indices into each side's `turns` list, or `None` where only one side has that slot (one episode closed early, or only one side needed a retry) — and `divergence` is the row position of the FIRST behavioural difference, or `None` if the two episodes played identically throughout. `attempt` is 0 for a seat's first go at a slot and 1+ for an engine retry (see :func:`_by_slot`).

Parameters

Name	Type	Default	Description
`left`	`dict`	*required*	
`right`	`dict`	*required*	

compare_payload

One seat-swap comparison, ready to render: both episode payloads, the slot alignment, the divergence point, the focal seat(s), and the score table.

```
compare_payload(
    left: dict,
    right: dict,
    *,
    left_label: str = 'A',
    right_label: str = 'B',
    pair_fields: tuple[str, ...] = DEFAULT_PAIR_KEY,
) -> dict
```

Defined in `interlens.arena.viz.compare` · source

Both sides keep their FULL episode payload, so the comparison page reuses the same transcript and frontier renderer as a single-episode page — the frontier is drawn once, from the shared instance, with both trajectories on it. The two episodes are not required to be a valid pair; a mismatch on the pairing key is recorded in `pairing.matched` and surfaced as a warning instead of being rejected, because comparing two deliberately unrelated episodes is sometimes exactly what a reader wants.

Parameters

Name	Type	Default	Description
`left`	`dict`	*required*	
`right`	`dict`	*required*	
`left_label`	`str`	`'A'`	
`right_label`	`str`	`'B'`	
`pair_fields`	`tuple[str, ...]`	`DEFAULT_PAIR_KEY`	

focal_seats

The seats whose OCCUPANT KIND differs between the two episodes — the substitution being measured.

```
focal_seats(left: dict, right: dict) -> list[dict]
```

Defined in `interlens.arena.viz.compare` · source

Each entry is `{party, name, left_kind, right_kind}`. An empty list means the two runs put the same kind of agent in every seat, which the page reports as “no seat swap detected” rather than inventing a focal seat.

Parameters

Name	Type	Default	Description
`left`	`dict`	*required*	
`right`	`dict`	*required*	

pair_key

The pairing key of an episode: the tuple of its `fields`.

```
pair_key(episode: dict, fields: tuple[str, ...] = DEFAULT_PAIR_KEY) ->
    tuple
```

Defined in `interlens.arena.viz.compare` · source

Two episodes with equal keys were played on the identical instance, seed, protocol arm, and sweep cell, and so differ only in who sat where.

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`fields`	`tuple[str, ...]`	`DEFAULT_PAIR_KEY`	

pair_runs

Pair every episode of one run against its key-matched counterpart in another, and build a comparison payload for each.

```
pair_runs(
    left_run: str | Path,
    right_run: str | Path,
    *,
    pair_fields: tuple[str, ...] = DEFAULT_PAIR_KEY,
    limit: int | None = None,
    reconstruct: bool = True,
    select: str = 'first',
    annotations_dirname: str = 'annotations',
) -> tuple[list[dict], dict]
```

Defined in `interlens.arena.viz.compare` · source

Returns `(comparisons, report)`. The report counts matched pairs and lists unmatched keys on each side, so a partially-complete campaign is visible as such. Where a key matches several episodes on a side (repeated seeds), they are zipped in sorted episode-id order and the multiplicity is recorded.

`select` (one of :data:`SELECTIONS`) decides which pairs `limit` keeps. Ranking reads only each episode's stored `outcome`, so choosing among hundreds of pairs costs nothing — the expensive payload build happens only for the pairs that survive the limit. The report records the selection, because "the 6 largest movers" and "6 arbitrary pairs" support very different readings of the same page count.

`annotations_dirname` selects the per-run annotation subdirectory BOTH runs read their post-hoc oracles from (default `"annotations"`; e.g. `"annotations_v1"` — see :class:`~interlens.arena.viz.episode.RunDir`).

Both runs' geometry caches are per-`RunDir`; the LEFT run's geometry is used for both sides of a pair so the frontier is built once and the two trajectories are provably drawn against the same numbers.

Parameters

Name	Type	Default	Description
`left_run`	`str | Path`	*required*	
`right_run`	`str | Path`	*required*	
`pair_fields`	`tuple[str, ...]`	`DEFAULT_PAIR_KEY`	
`limit`	`int | None`	`None`	
`reconstruct`	`bool`	`True`	
`select`	`str`	`'first'`	
`annotations_dirname`	`str`	`'annotations'`	

score_table

The quantified comparison: one row per metric with both values and the paired delta `right - left`.

```
score_table(left: dict, right: dict, focal: list[int] | int | None) -
    > list[dict]
```

Defined in `interlens.arena.viz.compare` · source

`focal` is the party index (or list of them) whose occupant was swapped. A one-seat swap gets that seat's own surplus and capture; a swap that replaced several seats at once — a mixed table against an all-LLM table replaces every seat but one — gets the MEAN over the swapped set, labelled with its size, because attributing the effect to any single one of those seats would be wrong. The rows are omitted entirely when there is no swap to attribute an effect to.

Every row carries `higher_is_better` so the page can colour a delta without guessing (`0` means neither direction is better, e.g. rounds used), and `None` values pass through as `None` rather than zero — a metric that was not recorded is missing, not neutral.

Parameters

Name	Type	Default	Description
`left`	`dict`	*required*	
`right`	`dict`	*required*	
`focal`	`list[int] | int | None`	*required*	

concepts

Module `interlens.arena.viz.concepts`

What each solution concept IS, in one place, for every part of the visualizer that explains one to a reader.

A leaf module on purpose: the browser layer (`assets/js_hover`, which serializes :data:`CONCEPT_MATH` straight into the page's script) and anything rendering an explanation server-side both read THIS, so the axioms cannot drift into two versions that disagree. `geometry` re-exports :data:`CONCEPT_LABELS`, which is also the legend order the chart marks them in.

The formulae are HTML — `<sub>` plus Unicode operators (`Σ Π τ - >`) and named entities — rather than LaTeX, because every page is opened off `file://` with no network: a maths library would either be a CDN request that leaves the explanation blank or a large vendored blob to typeset five one-line objectives. Notation throughout: `u_i` is party `i`'s utility for a deal, `tau_i` its walk-away threshold, `b_i` its ideal, and `z_i = max(u_i - tau_i, 0) / (b_i - tau_i)` the normalized surplus both chart axes are built from.

Attributes

Name	Type	Summary
`AXIS_NOTES`		
`CONCEPT_LABELS`		
`CONCEPT_MATH`		
`PROJECTION_CAVEAT`		
`ROLE_NOTES`		

episode

Module `interlens.arena.viz.episode`

One stored episode, turned into the single JSON payload the interactive page renders.

This is the data layer of the episode visualizer: it reads a run's three record stores — the `Episode` JSON, the `Instance` it was played on, and (optionally) the post-hoc annotation record — and merges them into one self-describing dict. Everything the page shows is computed here; the browser only draws it.

What the merge adds beyond the raw records:

- **numbers on every turn** — the action's deal placed in the instance's geometry (per-party surplus vs each threshold, welfare scalars, distance below the frontier), plus per-oracle chosen/best/regret values.
- **the post-hoc oracle counterfactual** — for every oracle that scored the turn, the action it ranks highest instead, resolved to a deal and its numbers, so the page can show "the model did X (value v) where the oracle would have done Y (value v*), regret v* - v*" side by side. Runs without a `bestresponse` oracle are reported as such rather than silently rendering an empty column.
- **seat identity** — which seats were played by an LLM and which by a computable policy, so a mixed table reads correctly and a seat-swap comparison knows its focal seat. Read from the run manifest's recorded invocation when available, else inferred from generation accounting (a policy seat emits zero output tokens).
- **prompt provenance** — the exact rendered view per turn, marked `stored` when the episode recorded it, `reconstructed` when it was re-derived by deterministic replay through the scenario state machine (current prompt code, so it can differ from what the model actually saw), or `absent`.
- **the public ledger** — which turns were actually PUBLISHED to the other seats, the offer id each proposal was registered under, and the deal standing on the table as of each turn (see :func:`public_ledger`).

This is what the page's conversation view and per-agent issue view read, and it is deliberately reconstructed here rather than in the browser so the page can render it server-side.

Attributes

Name	Type	Summary
`CLOSING_ACTIONS`		
`COUNTERFACTUAL_ALIASES`		
`COUNTERFACTUAL_ORACLES`		
`OFFER_PREFIX`		
`RAW_EXCERPT_CHARS`		
`RETRY_SOURCE`		

Classes

Name	Summary
`RunDir`	A run directory's three record stores, indexed for lookup: `episodes/`, `instances/`, and the annotation subdirectory named by `annotations_dirname` (`annotations/` by default), plus `manifest.json`.

Functions

Name	Summary
`closing_turn_index`	The index of the turn that CLOSED the deal, or `None` when nothing closed.

Name	Summary
`episode_payload`	The complete render payload for one episode.
`public_ledger`	Reconstruct what the seats publicly saw, from the per-turn records alone.
`reconstruct_views`	Re-derive each turn's rendered view by deterministic replay, for episodes recorded before the per-turn `view` field existed.
`seat_kinds`	Which seats an LLM played and which a computable policy played.

RunDir

A run directory's three record stores, indexed for lookup: `episodes/`, `instances/`, and the annotation subdirectory named by `annotations_dirname` (`annotations/` by default), plus `manifest.json`.

```
RunDir(root: str | Path, *, annotations_dirname: str = 'annotations')
```

Defined in `interlens.arena.viz.episode` · source

Geometry is built lazily and CACHED per instance, so a run whose 120 episodes share 6 instances builds 6 utility matrices rather than 120.

Parameters

Name	Type	Default	Description
`root`	`str | Path`	*required*	
`annotations_dirname`	`str`	`'annotations'`	

Attributes

Name	Type	Summary
`advice`		
`annotations_dirname`		
`derivation`		
`episodes_dir`		
`manifest`		
`root`		
`vintage`		

Methods

episode_files

```
episode_files(self) -> list[Path]
```

source

Every episode JSON under the run, in sorted path order.

geometry

```
geometry(self, instance_id: str, cell_cfg: dict | None = None)
```

source

The cached geometry for an instance id: a :class:`GameGeometry` for a negotiation instance, an :class:`~interlens.arena.viz.auction_geometry.AuctionGeometry` for an auction one, `None` if the instance is missing or is neither.

Auction geometry is cached per `(instance_id, cell)`, not per instance: a single bank instance is consumed by cells that override the mechanism and the horizon, so caching one spec per instance would hand a Dutch episode the sealed cell's geometry — a real defect class in this lane's history.

Parameters

Name	Type	Default	Description
`instance_id`	`str`	*required*	
`cell_cfg`	`dict | None`	`None`	

payload

```
payload(
    self,
    episode_path: str | Path,
```

```
    *,
    reconstruct: bool = True,
    auction_counterfactuals: bool = True,
) -> dict
```

source

The render payload for one episode file in this run, with its instance, annotation, manifest, and cached geometry wired in.

Parameters

Name	Type	Default	Description
`episode_path`	`str | Path`	*required*	
`reconstruct`	`bool`	`True`	
`auction_counterfactuals`	`bool`	`True`	

closing_turn_index

The index of the turn that CLOSED the deal, or `None` when nothing closed.

```
closing_turn_index(rows: list[dict], deal_index: int | None) -> int | None
```

Defined in `interlens.arena.viz.episode` · source

The chart's AGREED square is the one mark that stands for an event without naming its turn, so a reader clicking it has nowhere to land unless this is derived. Preference order, most specific first: the last published closing action (accept, or a vote ballot) taken while the agreed deal was the one standing; then the last published closing action at all (protocols where the standing deal is not recoverable from the ledger); then the last turn, since an episode that closed a deal ended by doing so. Requires `rows` to have been through :func:`public_ledger`, which is what annotates `published` and `standing_deal_index`.

Parameters

Name	Type	Default	Description
`rows`	`list[dict]`	*required*	
`deal_index`	`int | None`	*required*	

episode_payload

The complete render payload for one episode.

```
episode_payload(
    episode: dict,
    instance: dict | None = None,
    annotation: dict | None = None,
    *,
    manifest: dict | None = None,
    geometry: GameGeometry | None = None,
    reconstruct: bool = True,
    paths: dict | None = None,
    annotations_source: str | None = None,
    vintage: dict | None = None,
    derivation: dict | None = None,
    advice: dict | None = None,
    auction_counterfactuals: bool = True,
) -> dict
```

Defined in `interlens.arena.viz.episode` · source

Parameters

episode : dict A stored `Episode.to_json()` record. instance : dict, optional The `Instance` record the episode was played on. Without it there is no game geometry, so the frontier and side panels are omitted and the page renders the transcript alone. annotation : dict, optional The post-hoc annotation record (`{episode_id, summary, turns:[{turn_idx, oracle:{...}}]}`), which is where a re-scored oracle such as `bestresponse` lives for runs annotated after the fact. manifest : dict, optional The run's `manifest.json`, read for the recorded invocation (seat kinds, policies, oracle list). geometry : GameGeometry, optional A prebuilt geometry to reuse — pass the SAME object for both episodes of a comparison so the two trajectories are drawn against one identical frontier (and the `|D| x n` tables are built once). reconstruct : bool When an episode carries no stored per-turn views, re-derive them by replay (see :func:`reconstruct_views`) and mark them `reconstructed`. `False` reports them as `absent`. paths : dict, optional Absolute source paths to link from the page (`episode`, `instance`, `annotation`, `run`). annotations_source : str, optional The name of the per-run annotation subdirectory the `annotation`

record was read from (e.g. `"annotations"` or `"annotations_v1"`). Carried through to the page as provenance so an auditor can see WHICH annotation vintage the post-hoc oracle values (above all the `bestresponse` counterfactual) were read from — the v0 pass versus a re-annotated set such as the oracle seat-binding fix. `None` when the counterfactual oracles came only from the episode's own inline records, not an annotation store. vintage : dict, optional The run's parsed `VINTAGE_PROVENANCE.md` hazard record (see :func:`~interlens.arena.viz.hazards.vintage_provenance`), which marks a run whose agents carry a known defect. Passed in rather than read here because it is a property of the run directory and one run's file serves all of its episodes. `None` means no hazard file, which is the healthy case. derivation : dict, optional The run's optional `vote_derivation.json` sidecar (see :func:`~interlens.arena.viz.ballots.vote_derivation`), which lets the final-vote tally show what each computable seat's own policy re-derives beside what the record holds. `None` renders the recorded ballots alone. advice : dict, optional The run's optional `advice_trace.json` sidecar (see :func:`~interlens.arena.viz.advice.advice_trace`), which carries, per advised turn, the evidence the seat's private planner ran on — the formal-move ledger counts and the chat-derived preference claims with their provenance quotes — its ranked recommendation, and the stored verdict on whether the seat played it. Attached to each advised turn as `advice`. `None` on any run with no advised seat, which is every arm but the advised ones; the transcript then renders exactly as before. auction_counterfactuals : bool On an AUCTION episode, whether to compute the per-turn rational and oracle counterfactual bids (see :func:`~interlens.arena.viz.auction_geometry.auction_trace`). On by default because they are the campaign's headline instrumentation; `False` is the fast path when only the index row is wanted. Ignored for every other scenario.

Notes

An AUCTION episode takes a different geometry: `GameGeometry` is `GameSpec`-shaped and returns `None` on an auction payload, so `payload["game"]` stays `None` (which is what makes every negotiation panel degrade away rather than render an empty frontier) and `payload["auction"]` carries the sibling geometry

plus the replay-derived per-episode trace. The two keys are mutually exclusive by construction.

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`instance`	`dict | None`	`None`	
`annotation`	`dict | None`	`None`	
`manifest`	`dict | None`	`None`	
`geometry`	GameGeometry | None	`None`	
`reconstruct`	`bool`	`True`	
`paths`	`dict | None`	`None`	
`annotations_source`	`str | None`	`None`	
`vintage`	`dict | None`	`None`	
`derivation`	`dict | None`	`None`	
`advice`	`dict | None`	`None`	
`auction_counterfactuals`	`bool`	`True`	

public_ledger

Reconstruct what the seats publicly saw, from the per-turn records alone.

```
public_ledger(rows: list[dict]) -> dict
```

Defined in `interlens.arena.viz.episode` · source

Three things the page needs and no stored record carries directly:

- **published** — a turn that was a first attempt at a slot the seat later retried never reached the other seats: the engine's retry path returns the repair directive *before* publishing, so only the LAST turn in a `(round, phase, seat)` slot is public. A conversation view that showed the malformed attempt would be showing text no other party ever read.
- **offer_id** — the id a proposal was registered under. `OfferRegistry` mints ids sequentially over published proposals that resolved to a legal deal, so replaying that counter over the turns reproduces the exact ids the seats quoted back (`ACCEPT P2`), which the stored record keeps only on the accepting side.
- **standing_deal_index** — the deal on the table as of each turn: the offer this turn's action referenced if it referenced one, else the most recently registered offer, else `None`. This is the deal the per-agent issue view puts its marker on.

Returns `{"offers": {offer_id: {...}}, "prefix": str}` and annotates `rows` in place.

Parameters

Name	Type	Default	Description
`rows`	`list[dict]`	*required*	

reconstruct_views

Re-derive each turn's rendered view by deterministic replay, for episodes recorded before the per-turn `view` field existed.

```
reconstruct_views(episode: dict, instance: dict) -> dict[int, list[dict]]
```

Defined in `interlens.arena.viz.episode` · source

Feeds the stored turns back through the scenario's state machine (`arena.replay`) and captures the `SeatRequest.view` the machine builds for each one. Exact for the state, but the prompt TEXT comes from today's prompt code — so a reconstructed view is what the current build would show a seat at that state, not a byte-guaranteed record of what the model saw, and the page labels it accordingly.

One difference is systematic rather than a drift risk, and the caller marks it separately (see :data:`RETRY_SOURCE`): when a seat's malformed response triggered the engine's one retry, the LIVE retry view carried the failed attempt plus a repair instruction, while replay re-issues the original request. A retry turn's reconstruction is therefore the FIRST attempt's prompt; the repair text is not recoverable from the record.

Returns `{}` on any failure (unknown scenario, prompt/state drift), because a missing prompt panel is a far better outcome than a crashed export.

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`instance`	`dict`	*required*	

seat_kinds

Which seats an LLM played and which a computable policy played.

```
seat_kinds(episode: dict, manifest: dict | None = None) -> dict
```

Defined in `interlens.arena.viz.episode` · source

Returns `{"kinds": {seat_name: "llm" | "policy"}, "source": str, "detail": str}`. The manifest's recorded `invocation` is authoritative when present, because it names the table type exactly: `all_llm` / `all_rational` assign every seat; `mixed` puts the models in the leading seats and fills the rest with policies; `reverse_mixed` / `advocate_mixed` make exactly `--rational-seat` a policy.

Without a manifest the kinds are INFERRED from generation accounting: a policy seat is pure Python, so every one of its turns records `n_tokens_out` `== 0`, while an LLM seat generated text. The inference is reported as such (`source="inferred"`) so a reader never mistakes it for recorded ground truth.

Parameters

Name	Type	Default	Description
`episode`	`dict`	*required*	
`manifest`	`dict | None`	`None`	

export

Module `interlens.arena.viz.export`

The file-writing layer: run directory in, HTML pages plus an index on disk out.

Kept apart from :mod:`~interlens.arena.viz.page` (which is pure `payload → string`) so every renderer stays testable without touching a filesystem, and so a caller that wants the HTML in memory — to embed it, serve it, or diff it — never has to write a file to get it.

Functions

Name	Summary
`export_comparison`	Pair two runs on `pair_fields` and write one comparison page per matched pair, plus an index and the pairing report.
`export_episode`	Write one episode's page into `out_dir` as `<episode_id>.html` and return its path.
`export_run`	Render every episode of a run to its own page in `out_dir`, plus an `index.html` listing them with the numbers that say which are worth opening.
`render_compare`	The interactive HTML for the `index`-th matched pair between two runs, as a string.
`render_episode`	The interactive HTML for one episode of a run, as a string.

export_comparison

Pair two runs on `pair_fields` and write one comparison page per matched pair, plus an index and the pairing report.

```
export_comparison(
    left_run: str | Path,
    right_run: str | Path,
    out_dir: str | Path,
    *,
    limit: int | None = None,
    pair_fields: tuple[str, ...] = DEFAULT_PAIR_KEY,
    reconstruct: bool = True,
    select: str = 'first',
    annotations_dirname: str = 'annotations',
) -> dict
```

Defined in `interlens.arena.viz.export` · source

Returns a manifest (also written as `manifest.json`).

`select` (see :data:`~interlens.arena.viz.compare.SELECTIONS`) decides which pairs a `limit` keeps — `"largest-effect"` or `"deal-flip"` rather than the arbitrary first few. `annotations_dirname` selects which per-run annotation subdirectory BOTH sides read their post-hoc oracles from (default `"annotations"`; e.g. `"annotations_v1"` — see :class:`~interlens.arena.viz.episode.RunDir`).

The pairing report is part of the deliverable, not a log line: it names how many episodes matched, which keys went unmatched, and how the rendered pairs were selected, so a partially-complete campaign is visible as partial rather than quietly rendering whichever cells happened to finish.

Parameters

Name	Type	Default	Description
`left_run`	`str | Path`	*required*	
`right_run`	`str | Path`	*required*	
`out_dir`	`str | Path`	*required*	
`limit`	`int | None`	`None`	

Name	Type	Default	Description
pair_fields	tuple[str, ...]	DEFAULT_PAIR_KEY	
reconstruct	bool	True	
select	str	'first'	
annotations_dirname	str	'annotations'	

export_episode

Write one episode's page into `out_dir` as `<episode_id>.html` and return its path.

```
export_episode(
    run: str | Path,
    episode_path: str | Path,
    out_dir: str | Path,
    *,
    reconstruct: bool = True,
    annotations_dirname: str = 'annotations',
) -> Path
```

Defined in `interlens.arena.viz.export` · source

`annotations_dirname` selects the per-run annotation subdirectory (see :class:`~interlens.arena.viz.episode.RunDir`).

Parameters

Name	Type	Default	Description
`run`	`str | Path`	*required*	
`episode_path`	`str | Path`	*required*	
`out_dir`	`str | Path`	*required*	
`reconstruct`	`bool`	`True`	
`annotations_dirname`	`str`	`'annotations'`	

export_run

Render every episode of a run to its own page in `out_dir`, plus an `index.html` listing them with the numbers that say which are worth opening.

```
export_run(
    run: str | Path,
    out_dir: str | Path,
    *,
    limit: int | None = None,
    episode_ids: Sequence[str] | None = None,
    reconstruct: bool = True,
    annotations_dirname: str = 'annotations',
) -> dict
```

Defined in `interlens.arena.viz.export` · source

Returns a manifest (also written as `manifest.json`).

`limit` renders only the first `limit` episodes in sorted path order — the fast path for a spot check on a campaign cell with hundreds of episodes. `annotations_dirname` selects which per-run annotation subdirectory the post-hoc oracles are read from (default `"annotations"`; e.g. `"annotations_v1"` for a re-annotated set — see :class:`~interlens.arena.viz.episode.RunDir`).

`episode_ids` publishes a CHOSEN subset instead, in the order given. A campaign cell of a hundred-plus Opus episodes is tens of megabytes of pages, so a published hub is routinely a representative selection rather than the whole cell — and a selection made by a caller's own rule (spanning outcomes, seats, or a behaviour of interest) is worth far more than the first N in path order. The chosen ids are recorded in the manifest as `selection`, so a hub can state which episodes it published and which it did not; an id the run does not hold is reported in `failures` rather than silently skipped. `limit` still applies afterwards.

Parameters

Name	Type	Default	Description
`run`	`str | Path`	*required*	

Name	Type	Default	Description
`out_dir`	`str | Path`	*required*	
`limit`	`int | None`	`None`	
`episode_ids`	`Sequence[str] | None`	`None`	
`reconstruct`	`bool`	`True`	
`annotations_dirname`	`str`	`'annotations'`	

render_compare

The interactive HTML for the `index`-th matched pair between two runs, as a string.

```
render_compare(
    left_run: str | Path,
    right_run: str | Path,
    *,
    index: int = 0,
    pair_fields: tuple[str, ...] = DEFAULT_PAIR_KEY,
    reconstruct: bool = True,
    annotations_dirname: str = 'annotations',
) -> str
```

Defined in `interlens.arena.viz.export` · source

`annotations_dirname` selects the per-run annotation subdirectory both sides are read with.

Parameters

Name	Type	Default	Description
`left_run`	`str | Path`	*required*	
`right_run`	`str | Path`	*required*	
`index`	`int`	`0`	
`pair_fields`	`tuple[str, ...]`	`DEFAULT_PAIR_KEY`	
`reconstruct`	`bool`	`True`	
`annotations_dirname`	`str`	`'annotations'`	

render_episode

The interactive HTML for one episode of a run, as a string.

```
render_episode(
    run: str | Path,
    episode_path: str | Path,
    *,
    reconstruct: bool = True,
    annotations_dirname: str = 'annotations',
) -> str
```

Defined in `interlens.arena.viz.export` · source

`annotations_dirname` selects the per-run annotation subdirectory (see :class:`~interlens.arena.viz.episode.RunDir`).

Parameters

Name	Type	Default	Description
`run`	`str | Path`	*required*	
`episode_path`	`str | Path`	*required*	
`reconstruct`	`bool`	`True`	
`annotations_dirname`	`str`	`'annotations'`	

geometry

Module `interlens.arena.viz.geometry`

The plottable geometry of one negotiation instance: every deal placed in a 2-D scale-invariant embedding, with the frontier, the axiomatic solution points, and each party's individually-best deal marked.

The problem this solves: with `n` parties a deal's utility vector lives in `R^n`, so at `n = 6` there is no honest "utility space" scatter to draw. The embedding here projects the exact `|D| x n` surplus table onto the two axes that carry the normative content of a bargaining problem, both computed in :func:`~interlens.arena.negotiation.solutions.normalized_surplus` coordinates (`clip(x_i, 0) / b_i`) so they are exactly scale-invariant across arbitrary private score sheets:

- **x = joint welfare** — the mean normalized surplus over parties, `mean_i x_i/b_i` in `[0, 1]`. The utilitarian axis: "how much total value did this deal create".
- **y = min surplus** — the minimum normalized surplus, `min_i x_i/b_i` in `[0, 1]`. The egalitarian axis and exactly the quantity discrete Kalai-Smorodinsky maximizes: "how well off is the worst-treated party".

The projection is lossy by construction — two different deals can land on the same point — so a deal is never *described* by its coordinates alone. Every mark carries its full per-party breakdown (:attr:`DealGeometry.u`, `s`, `xn`), which is what the visualizer's hover panel reads; the embedding only decides *where* it is drawn. Both axes are monotone in the right direction (up and to the right is better for everybody), so the Pareto frontier's image is the upper-right envelope of the cloud.

Everything is exact: the deal space is enumerated, so the frontier and every solution concept come from `negotiation.solutions` rather than from any sampling or hull approximation.

Example:

```
geo = GameGeometry.from_instance(instance_dict)
geo.n_deals, geo.n_parties                  # 243, 6
geo.wx[geo.solution_index("nash")]          # the NBS's joint-welfare
     coordinate
geo.party_best[0]                           # deal index of Avery's
     individually-best frontier deal
geo.deal_index({"issue0": "opt1", ...})     # a named proposal → its row
     in the utility matrix
```

Classes

Name	Summary
`DealGeometry`	One deal's full record: where it plots, and how every party feels about it.
`GameGeometry`	The exact, fully-enumerated geometry of one negotiation instance, ready to plot.

Functions

Name	Summary
`staircase`	The left-to-right monotone staircase of the masked points in the 2-D embedding: those not dominated on `(wx, wy)` by another masked point.

DealGeometry

One deal's full record: where it plots, and how every party feels about it.

```
DealGeometry(
    index: int,
    named: dict[str, str],
    u: list[float],
    s: list[float],
    xn: list[float],
    wx: float,
    wy: float,
    pareto: bool,
    ir: bool,
    feasible: bool,
    d_frontier: float,
)
```

Defined in `interlens.arena.viz.geometry` · source

`index` is the deal's row in the utility matrix (and its position in `DealSpace.enumerate` order); `u`/`s`/`xn` are the per-party utility, raw surplus, and normalized surplus vectors; `wx`/`wy` the embedding coordinates; `pareto`/`ir`/`feasible` its membership flags (Pareto-optimal, individually rational for every party, and passing this game's full agreement rule including veto/min-accept/structural constraints); `pareto_ir` the conjunction `pareto and ir` — efficient AND acceptable to everyone, i.e. on the frontier a rational table could actually reach; `d_frontier` its normalized-surplus distance below the UNCONSTRAINED frontier (0 iff Pareto-optimal, so a deal below somebody's threshold can still read 0).

Parameters

Name	Type	Default	Description
`index`	`int`	*required*	
`named`	`dict[str, str]`	*required*	
`u`	`list[float]`	*required*	
`s`	`list[float]`	*required*	

Name	Type	Default	Description
xn	list[float]	*required*	
wx	float	*required*	
wy	float	*required*	
pareto	bool	*required*	
ir	bool	*required*	
feasible	bool	*required*	
d_frontier	float	*required*	

Attributes

Name	Type	Summary
d_frontier	float	
feasible	bool	
index	int	
ir	bool	
named	dict[str, str]	
pareto	bool	
pareto_ir	bool	Efficient *and* individually rational — derived, never stored, so it cannot drift from its parts.
s	list[float]	
u	list[float]	
wx	float	
wy	float	
xn	list[float]	

Methods

to_json

```
to_json(self) -> dict
```

source

GameGeometry

The exact, fully-enumerated geometry of one negotiation instance, ready to plot.

```
GameGeometry(
    game: GameSpec,
    solutions: dict | None = None,
    analysis: dict | None = None,
    *,
    ceiling: float | None = None,
    floor: float | None = None,
)
```

Defined in `interlens.arena.viz.geometry` · source

Built once per instance and shared by every episode played on it (both sides of a seat-swap comparison read the same object, so the two trajectories are guaranteed to be drawn against one identical frontier).

Attributes

game : GameSpec The reconstructed game (deal space, private sheets, thresholds, protocol knobs). U, X, Xn : numpy.ndarray The `|D| x n` utility, raw-surplus, and normalized-surplus tables. wx, wy : numpy.ndarray The `(|D|,)` embedding coordinates (mean and min normalized surplus). pareto, ir, feasible : numpy.ndarray Boolean `(|D|,)` membership masks. pareto_ir : numpy.ndarray `pareto & ir` — the IR-feasible frontier: efficient AND above every party's threshold. This is the frontier the charts ring and shade; `pareto & ~ir` deals are efficient but unreachable and are drawn distinctly rather than dropped. solutions : dict `{concept: SolutionPoint.to_json()}` for every concept in :data:`~interlens.arena.viz.geometry.CONCEPT_LABELS` plus any extra the stored analysis carried. party_best : list[int] Per party, the deal index of its individually-best deal on the frontier — the "if this party could dictate the outcome (subject to everyone clearing their threshold and the deal being efficient)" point.

Parameters

Name	Type	Default	Description
`game`	GameSpec	*required*	
`solutions`	`dict | None`	`None`	
`analysis`	`dict | None`	`None`	
`ceiling`	`float | None`	`None`	
`floor`	`float | None`	`None`	

Attributes

Name	Type	Summary
`U`		
`X`		
`Xn`		
`analysis`		
`feasible`		
`game`		
`ir`		
`n_deals`	`int`	Deal-space size `|D|`.
`n_parties`	`int`	Number of parties `n`.
`pareto`		
`pareto_ir`		
`parties`	`list[str]`	Score-sheet party ids in seat order (`"P0"`, `"P1"`, … — the seat *personas* live on the episode, not the game).
`party_best`		
`solutions`		
`solutions_source`		

Name	Type	Summary
`tau`		
`wx`		
`wy`		

Methods

at

```
at(self, index: int) -> DealGeometry
```

source

The full :class:`DealGeometry` record for a deal index.

Parameters

Name	Type	Default	Description
`index`	`int`	*required*	

deal_index

```
deal_index(self, named: Any) -> int | None
```

source

The deal index of a `{issue_name: option_label}` proposal (tolerant of case/whitespace, via `DealSpace.parse`), or `None` if it is missing, malformed, or names an unknown option — which is exactly the case for a model's invalid proposal, so the caller gets `None` instead of an exception.

Parameters

Name	Type	Default	Description
named	Any	*required*	

envelope

```
envelope(self, mask: np.ndarray | None = None) -> list[list[float]]
```

source

The efficient envelope of the frontier IN THE 2-D EMBEDDING: the staircase of frontier deals that are non-dominated on `(joint welfare, min surplus)` themselves, ordered left to right.

`mask` selects which frontier is traced and defaults to :attr:`pareto` (the unconstrained one). Pass :attr:`pareto_ir` for the envelope of the deals that could actually close; the charts draw that one as the shaded region and keep the unconstrained staircase as a separate, visibly different line.

The projection is lossy, so a deal on the true `R^n` frontier can sit strictly inside this envelope — it is efficient overall while being beaten on both plotted summaries by some other efficient deal. The envelope is therefore drawn as the outer boundary of the achievable region and never used to decide whether a deal is Pareto-optimal; that always comes from :attr:`pareto`.

Parameters

Name	Type	Default	Description
mask	np.ndarray | None	None	

from_instance

```
from_instance(instance: dict) -> 'GameGeometry | None'
```

source

The geometry of a stored `Instance` dict, or `None` if its payload is not a scorable game (so a caller can render a non-negotiation scenario's episode without the game panel instead of crashing).

Parameters

Name	Type	Default	Description
`instance`	`dict`	*required*	

solution_index

```
solution_index(self, concept: str) -> int | None
```

source

The deal index of a solution concept, or `None` if the instance carries no such concept.

Parameters

Name	Type	Default	Description
`concept`	`str`	*required*	

to_json

```
to_json(self) -> dict
```

source

The whole geometry as one JSON payload for the browser: the game description, the per-deal tables (all `|D|` deals, so every point in the cloud is hoverable without a round trip), and the reference marks.

welfare_of

```
welfare_of(self, index: int) -> dict
```

source

The welfare scalars of one deal, in raw surplus units: `usw` (sum), `esw` (min), `nsw_geomean` (the readable geometric-mean form of the Nash product), and the count of parties left below threshold.

Parameters

Name	Type	Default	Description
`index`	`int`	*required*	

staircase

The left-to-right monotone staircase of the masked points in the 2-D embedding: those not dominated on `(wx, wy)` by another masked point.

```
staircase(wx: np.ndarray, wy: np.ndarray, mask: np.ndarray) ->
     list[list[float]]
```

Defined in `interlens.arena.viz.geometry` · source

Free of :class:`GameGeometry` so a caller holding only the wire payload's `deals` arrays (an already-published page being re-rendered, say) traces exactly the same envelope the charts do, rather than reimplementing the sweep.

Parameters

wx, wy : numpy.ndarray The `(|D|,)` embedding coordinates (joint welfare and min surplus). mask : numpy.ndarray Boolean `(|D|,)` selecting which deals the envelope is traced over — :attr:`GameGeometry.pareto` for the unconstrained frontier, :attr:`GameGeometry.pareto_ir` for the one a rational table can reach.

Parameters

Name	Type	Default	Description
`wx`	`np.ndarray`	*required*	
`wy`	`np.ndarray`	*required*	
`mask`	`np.ndarray`	*required*	

hazards

Module `interlens.arena.viz.hazards`

Two facts about a run that decide whether its numbers may be compared with another run's.

Both are properties of the RUN, not of the episode, and both were invisible on the pages for as long as they existed — which is how each one cost this program weeks.

Vintage. A run whose agents carry a since-fixed defect is still a valid record of the agent it actually was, and is worthless pooled against a repaired run. The convention is a `VINTAGE_PROVENANCE.md` at the run root naming the defect and its repaired counterpart; :func:`vintage_provenance` reads it and the page turns it into the loudest thing on the screen.

Generation budget. Two arms described as "sharing a protocol" ran at an 8x per-seat token budget difference, because the frozen caps (2048 on an ordinary turn, 2560 on the forced final) are stamped on every request while a RAISED cap is stamped only where it was raised. So the default is invisible by design and the exception was invisible by accident. :func:`generation_budget` reads the caps the turns actually carry — the record, not the intent — and reports any departure from the frozen pair.

Attributes

Name	Type	Summary
`FROZEN_TURN_CAPS`		
`VINTAGE_SCAN_LINES`		
`VINTAGE_SUMMARY_CHARS`		

Functions

Name	Summary
`budget_badge`	The per-seat generation budget, stated whenever it is not the frozen default.
`budget_note`	A one-line explanation under the header for a non-default generation budget, naming what it blocks.
`generation_budget`	The per-seat generation budget this episode actually ran at, and whether it is the frozen default.
`vintage_badge`	The sticky quick-read marker for a spoiled vintage, so the hazard survives scrolling past the banner.
`vintage_banner`	The loudest banner on the page: this run carries a known defect and must not be pooled.
`vintage_pairing`	What pairing THESE two episodes means, when either side carries a vintage hazard.
`vintage_provenance`	The run's `VINTAGE_PROVENANCE.md`, parsed into `{path, headline, summary}`, or `None` if absent.

budget_badge

The per-seat generation budget, stated whenever it is not the frozen default.

```
budget_badge(budget: dict | None) -> str
```

Defined in `interlens.arena.viz.hazards` · source

Muted-informational rather than alarming: a raised cap is a deliberate protocol choice and often the right one. What it is not is comparable — so the badge names the number and says which frozen pair it departs from, and stays silent on a default run so the exception reads as exceptional.

Parameters

Name	Type	Default	Description
`budget`	`dict | None`	*required*	

budget_note

A one-line explanation under the header for a non-default generation budget, naming what it blocks.

```
budget_note(budget: dict | None) -> str
```

Defined in `interlens.arena.viz.hazards` · source

Parameters

Name	Type	Default	Description
budget	dict | None	*required*	

generation_budget

The per-seat generation budget this episode actually ran at, and whether it is the frozen default.

```
generation_budget(payload: dict) -> dict
```

Defined in `interlens.arena.viz.hazards` · source

Reads three sources, most authoritative first, and reports all three rather than collapsing them — a badge that said only “raised” would leave a reader unable to tell a protocol option from an API floor:

- `caps` — the distinct `max_tokens` values stamped on this episode’s own requests. This is the record.
- `turn_max_tokens` — the protocol option that raised them, from the episode’s `cell_cfg`. Stamped only when set, which is exactly why it cannot be the detector.
- `api_floors` — `{model: turn_token_floor}` from the run manifest’s `api_request_config`. An API participant applies its floor as `max(cap, floor)`, so an API seat’s effective budget is the floor even though the request’s own cap says otherwise. This is the term that made two arms look matched when they were not.

`default` is true only when the observed caps are a subset of :data:`FROZEN_TURN_CAPS` and no API floor exceeds them. `effective` is the largest budget any seat could actually use.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

vintage_badge

The sticky quick-read marker for a spoiled vintage, so the hazard survives scrolling past the banner.

```
vintage_badge(vintage: dict | None) -> str
```

Defined in `interlens.arena.viz.hazards` · source

Parameters

Name	Type	Default	Description
`vintage`	`dict | None`	*required*	

vintage_banner

The loudest banner on the page: this run carries a known defect and must not be pooled.

```
vintage_banner(vintage: dict | None) -> str
```

Defined in `interlens.arena.viz.hazards` · source

Rendered above everything, links to the provenance file itself, and states its headline verbatim rather than a paraphrase — the file is the authority on what is wrong and the page is only its messenger.

Parameters

Name	Type	Default	Description
`vintage`	`dict | None`	*required*	

vintage_pairing

What pairing THESE two episodes means, when either side carries a vintage hazard.

```
vintage_pairing(left: dict, right: dict, labels: dict) -> str
```

Defined in `interlens.arena.viz.hazards` · source

The intended and most valuable use of a comparison page is a spoiled run against its repaired counterpart — the same instance and seed, one agent defect apart. But that is also the pairing whose deltas are easiest to misread: they are the effect of the REPAIR, not of any behavioural manipulation, so a page that showed them beside a seat-swap banner would invite exactly the wrong reading. Three cases, each stated in its own words:

- **one side spoiled** — a vintage contrast; the deltas measure the fix.
- **both spoiled by the same file** — vintage-matched, which is like-for-like and the one safe pooling of a spoiled arm.
- **both spoiled by different files** — two different defects, so the deltas mix them and belong to neither.

Silent when neither side carries a hazard, which is the ordinary case.

Parameters

Name	Type	Default	Description
`left`	`dict`	*required*	
`right`	`dict`	*required*	
`labels`	`dict`	*required*	

vintage_provenance

The run's `VINTAGE_PROVENANCE.md`, parsed into `{path, headline, summary}`, or `None` if absent.

```
vintage_provenance(run_root: str | Path | None) -> dict | None
```

Defined in `interlens.arena.viz.hazards` · source

`headline` is the file's first markdown heading, which by convention states the defect in one line (`# THIS ARM IS THE SPOILED-BALLOT VINTAGE — DO NOT POOL IT WITH A REPAIRED RUN`). `summary` is the first PARAGRAPH after it — consecutive lines up to the next blank, joined — rather than the first line, because a hard-wrapped source file would otherwise be quoted cut off mid-sentence. Comment lines are skipped, so a session-stamp comment at the top of the file does not become the headline, and inline markdown is reduced to plain text because the banner escapes everything it prints.

A file that exists but has no heading still yields a record — with the headline falling back to the file's first non-comment line — because a malformed hazard file must not silently disarm the hazard.

Parameters

Name	Type	Default	Description
`run_root`	`str | Path | None`	*required*	

page

Module `interlens.arena.viz.page`

HTML assembly: a payload in, one self-contained interactive page out.

Everything that can be rendered without JavaScript is rendered here in Python — the summary strip, the game side panel, every score sheet, the numeric table view of the chart, the pairing banner, the comparison score table and its verdict, and the whole run index including its rows. The browser script only draws the two charts and the transcript cards. That split is deliberate: the numbers are the deliverable, so they must be in the document even if the script never runs, and it makes the tests able to assert on real structure and real values without a browser.

Pages are opened over `file://`, so nothing is fetched: the stylesheet and script are inlined, and the payload travels in a `<script type="application/json">` tag (data in a data position — never interpolated into executable code, and closing-tag sequences are escaped).

Attributes

Name	Type	Summary
`AUCTION_INDEX_COLUMNS`		
`INDEX_COLUMNS`		
`SIDEBAR_TABS`		

Functions

Name	Summary
`occupant_defaults`	Each seat's DEFAULT occupant: the one it played its first recorded turn under.

Name	Summary
`preference_visibility`	The compact index label for a negotiation's information condition.
`render_auction_episode_html`	One self-contained interactive page for one auction episode.
`render_compare_html`	One self-contained page for a seat-swap comparison: a verdict strip, the quantified score table with paired deltas, one shared frontier carrying both trajectories, and two synchronized transcript columns with the divergence point marked.
`render_episode_html`	One self-contained interactive page for one episode.
`render_index_html`	A run index: one row per generated page, sortable on every column and filterable by text, outcome, and whether the engine fabricated any turns.

occupant_defaults

Each seat's DEFAULT occupant: the one it played its first recorded turn under.

```
occupant_defaults(payload: dict) -> dict
```

Defined in `interlens.arena.viz.page` · source

Batch episodes carry no occupant at all and get an empty map. Live play stamps every turn with who held the seat (`TurnRecord.occupant`), and a seat that never changed hands would otherwise wear the same badge on every one of its turns — so the badge marks a DEPARTURE from this map, which is what a reader wants to see. Derived from the payload rather than stored anywhere, so it always describes the transcript being rendered.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

preference_visibility

The compact index label for a negotiation's information condition.

```
preference_visibility(payload: dict) -> str
```

Defined in `interlens.arena.viz.page` · source

Private information is the protocol default, including older records that did not serialize `info`; the exceptional public/shared spellings are normalized to the experiment-facing label `FULL`.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

render_auction_episode_html

One self-contained interactive page for one auction episode.

```
render_auction_episode_html(payload: dict) -> str
```

Defined in `interlens.arena.viz.page` · source

Sections, in order: the auction summary strip; the census strip; the staged bid ladder; the per-lot allocation strip; the winner/payment/surplus panel; the message graph; the per-turn counterfactual table; the system-prompt audit; and the transcript. The side panel carries the mechanism, the five public cards, the lot catalogue, the per-stage private draws, the public chat, and the reading guide.

Reached through :func:`render_episode_html`, so a caller never has to know which kind of episode it holds.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

render_compare_html

One self-contained page for a seat-swap comparison: a verdict strip, the quantified score table with paired deltas, one shared frontier carrying both trajectories, and two synchronized transcript columns with the divergence point marked.

```
render_compare_html(payload: dict) -> str
```

Defined in `interlens.arena.viz.page` · source

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

render_episode_html

One self-contained interactive page for one episode.

```
render_episode_html(payload: dict) -> str
```

Defined in `interlens.arena.viz.page` · source

Sections, in order: the summary strip; the frontier chart with the play trajectory, the oracle's recommendations, and every normative reference point (plus its numeric table view); the per-turn regret strip; the system-prompt audit; and the transcript, where each turn shows what the model did NEXT TO what the rational agent would have done there, with the regret between them. The tabbed sidebar (see :func:`_sidebar`) is sticky alongside and follows the transcript as it scrolls.

The top bar carries the run name, the episode picker, and the quick read; where the picker's contents go is a marker the exporter fills once every page of the run is known (see :func:`~.chrome.nav_group`), so a page rendered on its own is still complete — it simply has nothing to navigate to.

An AUCTION payload takes :func:`render_auction_episode_html` instead: an auction has no deal space, so the frontier chart, the solution-concept legend, the regret strip and the ballot table have nothing to draw, and the four panels of design.md §10 stand in their place. Everything shared — the shell, the census and vintage badges, the prompt audit, the transcript — is the same code on both paths.

Parameters

Name	Type	Default	Description
`payload`	`dict`	*required*	

render_index_html

A run index: one row per generated page, sortable on every column and filterable by text, outcome, and whether the engine fabricated any turns.

```
render_index_html(
    rows: list[dict],
    title: str,
    note: str = '',
    readme_markdown: str = '',
    columns: list[tuple] | None = None,
) -> str
```

Defined in `interlens.arena.viz.page` · source

Sorting and filtering are client-side over the rows already in the document — there is no second copy of the data in a JSON blob, so a 200-episode index stays a small file and still reads correctly with scripting off. The row count of what survives a filter is always on screen, because a filter that silently hides rows is how a reader concludes a run has fewer episodes than it has.

`columns` selects the column set, defaulting to :data:`INDEX_COLUMNS` (the negotiation one). Pass :data:`AUCTION_INDEX_COLUMNS` for an auction run, whose rows carry a different measure set entirely — the scale for the `bar` columns is taken from whichever key the first `bar` column names, so a caller does not have to supply it.

Parameters

Name	Type	Default	Description
`rows`	`list[dict]`	*required*	
`title`	`str`	*required*	
`note`	`str`	`''`	
`readme_markdown`	`str`	`''`	
`columns`	`list[tuple] | None`	`None`	

references

Module `interlens.arena.viz.references`

The decision references a scored turn can carry, placed on two axes — and what each one's number MEANS.

A turn's counterfactuals answer "what would a computable agent have done here instead". The current campaign schema (`five-seat-triple-counterfactuals-v1`) records FOUR of them, which is one point in a 2x2:

	self-interest table fairness
private	`rational_private` `fairness_private`
omniscient	`oracle_omniscient` `fairness_oracle`

The *information* axis says what the reference could see when it decided: only the acting seat's own sheet plus the public moves, or every party's hidden sheet. The *objective* axis says what it was maximizing: its own surplus, or a welfare score over the whole table.

The two axes are not interchangeable, and the objective axis carries a unit hazard this module exists to close. A self-interest reference's `value` is priced in the acting seat's own points, so it is directly comparable with that seat's realized surplus. A fairness reference's `value` is the *table* objective — a smoothed log-Nash score over normalized surplus (see :func:`~interlens.arena.negotiation.fairness.objective_from_normalized`) — which belongs to the whole table and to no seat, is on a completely different scale, and has its own optimum recorded alongside it as `table_optimum`. Two readers have already mistaken one for the other while reading these records by hand, so every renderer takes its unit string from :data:`REFERENCES` rather than writing its own, and no page is allowed to put the two kinds of value in one column.

Attributes

Name	Type	Summary
`ALIASES`		
`INFORMATION`		
`OBJECTIVES`		
`REFERENCES`	`dict[str, dict]`	
`REFERENCE_ORDER`		

Functions

Name	Summary
`axes_payload`	The 2x2′s own description, shipped once per page so the browser groups the references from data.
`canonical`	The canonical reference name for a stored one, or the stored name unchanged when it is not a reference.
`describe`	Everything a renderer needs about one reference, flattened into one dict.
`spec`	The :data:`REFERENCES` entry for a (possibly aliased) name, or `None` for a generic scored oracle.

axes_payload

The 2x2's own description, shipped once per page so the browser groups the references from data.

```
axes_payload() -> dict
```

Defined in `interlens.arena.viz.references` · source

Carries the axis names, each cell's meaning, and — the part that matters — the per-objective unit and note strings, so the grid's column headers and the "these numbers are not comparable" warning are written here and nowhere else.

canonical

The canonical reference name for a stored one, or the stored name unchanged when it is not a reference.

```
canonical(name: str) -> str
```

Defined in `interlens.arena.viz.references` · source

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	

describe

Everything a renderer needs about one reference, flattened into one dict.

```
describe(name: str) -> dict
```

Defined in `interlens.arena.viz.references` · source

Returns `{}` for a name that is not a known decision reference — a generic scored oracle keeps rendering through the page's older, name-agnostic path rather than being described wrongly. Otherwise the keys are `role` (the canonical name), `label`, `short`, `information`, `information_detail`, `objective`, `objective_name`, `value_label`, `unit`, `value_basis`, `comparable_across_information`, `gap_label`, `objective_note`, and `legacy`.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	

spec

The :data:`REFERENCES` entry for a (possibly aliased) name, or `None` for a generic scored oracle.

```
spec(name: str) -> dict | None
```

Defined in `interlens.arena.viz.references` · source

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	

serve

Module `interlens.arena.viz.serve`

Hand the rendered pages to a browser over HTTP, for when the filesystem the pages live on is not the one the browser runs on.

The pages themselves need no server — they are self-contained and open by double-click. This exists for the usual research setup: you are ssh'd into a cluster node, the run directory is there, and your browser is on your laptop. Rather than copying a directory of HTML back, serve it in place and forward one port.

Stdlib only (:mod:`http.server`), so this adds no dependency to a package whose visualizer is otherwise pure string-building. The server is threading so a page's several asset-free requests do not serialize, and it is read-only by construction: :class:`~http.server.SimpleHTTPRequestHandler` implements GET/HEAD and nothing else.

Attributes

Name	Type	Summary
`DEFAULT_HOST`		

Functions

Name	Summary
`make_server`	Bind (but do not run) an HTTP server rooted at `directory`.
`serve_banner`	The startup message: where the pages are, the URL to open, and — because the reader is usually on a cluster node with no browser — the exact `ssh -L` command to for-

Name	Summary
	ward `port` to their laptop, with this machine's real hostname already filled in.
`serve_directory`	Serve `directory` over HTTP until interrupted — bind, print :func:`serve_banner`, then block in `serve_forever`.

make_server

Bind (but do not run) an HTTP server rooted at `directory`.

```
make_server(
    directory: str | Path,
    port: int = 0,
    host: str = DEFAULT_HOST,
) -> ThreadingHTTPServer
```

Defined in `interlens.arena.viz.serve` · source

`port` is the TCP port to listen on; `0` — the default — asks the OS for a free ephemeral port, which is what you want when several people share a node and fixed ports collide. Read the port that was actually chosen back off `server.server_address[1]`. `host` is the bind address (see :data:`DEFAULT_HOST`).

Returns the bound server so the caller decides how to run it: `serve_forever()` on this thread (see :func:`serve_directory`) or on another one for a test. Call `server_close()` when done.

Parameters

Name	Type	Default	Description
`directory`	`str | Path`	*required*	
`port`	`int`	`0`	
`host`	`str`	`DEFAULT_HOST`	

serve_banner

The startup message: where the pages are, the URL to open, and — because the reader is usually on a cluster node with no browser — the exact `ssh -L` command to forward `port` to their laptop, with this machine's real hostname already filled in.

```
serve_banner(directory: str | Path, port: int, host: str = DEFAULT_HOST) -
    > str
```

Defined in `interlens.arena.viz.serve` · source

Returned as a string rather than printed so it is testable.

Parameters

Name	Type	Default	Description
`directory`	`str | Path`	*required*	
`port`	`int`	*required*	
`host`	`str`	`DEFAULT_HOST`	

serve_directory

Serve `directory` over HTTP until interrupted — bind, print :func:`serve_banner`, then block in `serve_forever`.

```
serve_directory(directory: str | Path, port: int = 0, host: str =
    DEFAULT_HOST) -> None
```

Defined in `interlens.arena.viz.serve` · source

`Ctrl-C` (`KeyboardInterrupt`) shuts the server down and returns normally rather than raising, so the CLI exits 0. See :func:`make_server` for `port` / `host`.

Parameters

Name	Type	Default	Description
`directory`	`str | Path`	*required*	
`port`	`int`	`0`	
`host`	`str`	`DEFAULT_HOST`	

communication

Package `interlens.communication`

Pluggable communication topologies: who speaks next, and who sees what. See `policy.py`.

Modules

- `messaging` — Tool-mediated asynchronous messaging between autonomous agents.
- `policy` — Communication topology as a pluggable policy.

Re-exported

Name	Defined in	Summary
`CommunicationPolicy`	`interlens.communication.policy`	Who speaks next, and who sees what.
`DirectPipingPolicy`	`interlens.communication.policy`	A fixed pipeline: each participant sees only its **predecessor's** output (plus moderator/system framing and its own past turns), and speaking order follows the chain — A → B → C → A → …
`Mail`	`interlens.communication.messaging`	One mailbox item.
`MessagingPolicy`	`interlens.communication.messaging`	Asynchronous point-to-point messaging with per-agent mail-

Name	Defined in	Summary
		boxes and a ping-driven scheduler.
`ReadMessageTool`	`interlens.communication.messaging`	Native-tool surface for reading: `read_message()` returns and marks-read the caller's unread mail.
`RoundRobinPolicy`	`interlens.communication.policy`	The shared-transcript default, as an explicit policy: speakers cycle through `participants` order and everyone sees every committed turn.
`SendMessageTool`	`interlens.communication.messaging`	Native-tool surface for sending: `send_message(content, recipient, priority)`.
`parse_json_actions`	`interlens.communication.messaging`	All fenced JSON objects in `text`, parsed (malformed fences are skipped, not fatal).

messaging

Module `interlens.communication.messaging`

Tool-mediated asynchronous messaging between autonomous agents.

Under `MessagingPolicy` there is **no shared conversation**: each agent sees only the moderator framing and its own past turns. Communication is explicit — an agent *sends* a message to a named recipient (`send_message`: content, `normal`/`high` priority), the recipient gets a **ping** (a lightweight notice in its next view: how many unread, from whom, at what priority), and *reads* its mailbox (`read_message`), after which the mail is delivered into its view. The activation scheduler grants turns ping-first (high priority ahead of normal), with a **fairness tick** so agents nobody has pinged still act regularly, and lets a reader act on freshly delivered mail immediately.

Both invocation surfaces feed the same mailboxes:

- **Fenced-JSON actions** (family-agnostic, works for every participant type — API, local, scripted): an agent ends its turn with `{"send_message": {"recipient": ..., "content": ..., "priority": ...}}` or `{"read_message": {}}`; the policy parses committed turns in `on_commit`.
- **Native tools** (local model participants with a tool-calling family): `policy.tools_for(name)` returns `Tool` objects to attach via `tools=` — `read_message` then resolves *within* the turn (the model reads and reacts in one generation), and the call/result trail lands in `metadata['tool_trail']` as usual.

Everything is recorded first-class for scoring and replay: each send/read/delivery appends a structured event to `policy.events` (JSON-serializable) *and* annotates the committed message's metadata (`comm_sends` / `comm_read`), so a saved transcript carries the full message traffic.

Attributes

Name	Type	Summary
PRIORITIES		

Classes

Name	Summary
Mail	One mailbox item.
MessagingPolicy	Asynchronous point-to-point messaging with per-agent mailboxes and a ping-driven scheduler.
ReadMessageTool	Native-tool surface for reading: read_message() returns and marks-read the caller's unread mail.
SendMessageTool	Native-tool surface for sending: send_message(content, recipient, priority).

Functions

Name	Summary
parse_json_actions	All fenced JSON objects in text, parsed (malformed fences are skipped, not fatal).

Mail

One mailbox item.

```
Mail(
    sender: str,
    recipient: str,
    content: str,
    priority: str = 'normal',
    sent_at_turn: int = 0,
    read: bool = False,
)
```

Defined in `interlens.communication.messaging` · source

`read` flips when the recipient reads; delivered mail renders into its views.

Parameters

Name	Type	Default	Description
`sender`	`str`	*required*	
`recipient`	`str`	*required*	
`content`	`str`	*required*	
`priority`	`str`	`'normal'`	
`sent_at_turn`	`int`	`0`	
`read`	`bool`	`False`	

Attributes

Name	Type	Summary
`content`	`str`	
`priority`	`str`	
`read`	`bool`	
`recipient`	`str`	
`sender`	`str`	

Name	Type	Summary
sent_at_turn	int	

Methods

to_dict

```
to_dict(self) -> dict
```

source

MessagingPolicy

Asynchronous point-to-point messaging with per-agent mailboxes and a ping-driven scheduler.

```
MessagingPolicy(agents: list[str] | None = None, *, fairness_every: int
    = 3)
```

Defined in `interlens.communication.messaging` · source

Inherits from: CommunicationPolicy

`fairness_every` bounds starvation: after that many consecutive ping-driven grants, the least-recently-active unpinged agent gets a turn regardless. The protocol text each agent needs is injected automatically as a system-block view segment (`PROTOCOL`), so no manual prompt plumbing is required.

Parameters

Name	Type	Default	Description
`agents`	`list[str] | None`	`None`	
`fairness_every`	`int`	`3`	

Attributes

Name	Type	Summary
`PROTOCOL`		
`agents`		
`events`	`list[dict]`	
`fairness_every`		
`mailboxes`	dict[str, list[Mail]]	

Methods

extra_segments

```
extra_segments(
    self,
    conversation: 'Conversation',
    participant: 'Participant',
) -> list[ViewSegment]
```

source

Parameters

Name	Type	Default	Description
conversation	'Conversation'	*required*	
participant	'Participant'	*required*	

next_speaker

```
next_speaker(self, conversation: 'Conversation') -> 'Participant | None'
```

source

Parameters

Name	Type	Default	Description
conversation	'Conversation'	*required*	

on_commit

```
on_commit(self, message: 'Message', conversation: 'Conversation') -> None
```

source

Parameters

Name	Type	Default	Description
`message`	`'Message'`	*required*	
`conversation`	`'Conversation'`	*required*	

`read`

```
read(self, reader: str, sender: str | None = None) -> list[Mail]
```

source

Mark `reader`'s unread mail (optionally from one `sender`) as read and schedule it for delivery into the reader's view.

Returns the newly delivered items.

Parameters

Name	Type	Default	Description
`reader`	`str`	*required*	
`sender`	`str | None`	`None`	

`reset`

```
reset(self) -> None
```

source

`send`

```
send(self, sender: str, recipient: str, content: str, priority: str =
     'normal') -> str
```

source

Parameters

Name	Type	Default	Description
`sender`	`str`	*required*	
`recipient`	`str`	*required*	
`content`	`str`	*required*	
`priority`	`str`	`'normal'`	

tools_for

```
tools_for(self, agent: str) -> tuple[Tool, Tool]
```

source

Native `send_message`/`read_message` tools bound to `agent`, for participants with a tool-calling family (attach via `tools=`).

The fenced-JSON path needs no setup.

Parameters

Name	Type	Default	Description
`agent`	`str`	*required*	

unread

```
unread(self, agent: str) -> list[Mail]
```

source

Parameters

Name	Type	Default	Description
`agent`	`str`	*required*	

visible

```
visible(
    self,
    message: 'Message',
    pov: 'Participant',
    conversation: 'Conversation',
) -> bool
```

source

Parameters

Name	Type	Default	Description
message	'Message'	*required*	
pov	'Participant'	*required*	
conversation	'Conversation'	*required*	

ReadMessageTool

Native-tool surface for reading: `read_message()` returns and marks-read the caller's unread mail.

```
ReadMessageTool(policy: 'MessagingPolicy', reader: str)
```

Defined in `interlens.communication.messaging` · source

Inherits from: Tool

Parameters

Name	Type	Default	Description
`policy`	`'MessagingPolicy'`	*required*	
`reader`	`str`	*required*	

Attributes

Name	Type	Summary
`name`		
`schema`	`dict`	

SendMessageTool

Native-tool surface for sending: `send_message(content, recipient, priority)`.

```
SendMessageTool(policy: 'MessagingPolicy', sender: str)
```

Defined in `interlens.communication.messaging` · source

Inherits from: Tool

Bound to one sender.

Parameters

Name	Type	Default	Description
`policy`	`'MessagingPolicy'`	*required*	
`sender`	`str`	*required*	

Attributes

Name	Type	Summary
`name`		
`schema`	`dict`	

parse_json_actions

All fenced JSON objects in `text`, parsed (malformed fences are skipped, not fatal).

```
parse_json_actions(text: str) -> list[dict]
```

Defined in `interlens.communication.messaging` · source

Thin alias for :func:`interlens.parsing.iter_fenced_json`.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

policy

Module `interlens.communication.policy`

Communication topology as a pluggable policy.

A `Conversation` has always answered two questions implicitly: *who speaks next* (round-robin over `participants` order) and *who sees what* (everyone sees the whole shared transcript). A `CommunicationPolicy` makes both answers explicit and swappable, so the same participants, scoring, and persistence compose with different communication styles:

- `RoundRobinPolicy` — the classic shared transcript (the default behavior, now nameable).
- `DirectPipingPolicy` — one participant's output becomes the next one's input along a fixed chain, formalizing what a 2-party `Conversation` already does implicitly and generalizing it to longer pipelines.
- `MessagingPolicy` (see `messaging.py`) — no shared transcript at all: autonomous agents exchange point-to-point messages through per-agent mailboxes via `send_message`/`read_message`, with a ping-driven scheduler.

Custom topologies (private sub-group channels, hub-and-spoke, dynamic floor-passing) subclass `CommunicationPolicy` and override the same four hooks — no core changes needed. Install a policy via `Conversation(communication=...)` (or `conv.set(communication=...)`); `run` consults it for turn order and the view pipeline consults it for visibility. Policies are conversation-state: a copy-on-write clone / branch gets an independent deep copy, and they are runtime-only (not persisted by `save`), like `message_hooks`.

Classes

Name	Summary
`CommunicationPolicy`	Who speaks next, and who sees what.
`DirectPipingPolicy`	A fixed pipeline: each participant sees only its **predecessor's** output (plus moderator/system framing and its own past turns), and speaking order follows the chain — A → B → C → A → …
`RoundRobinPolicy`	The shared-transcript default, as an explicit policy: speakers cycle through `participants` order and everyone sees every committed turn.

CommunicationPolicy

Who speaks next, and who sees what.

```
CommunicationPolicy()
```

Defined in `interlens.communication.policy` · source

Inherits from: `ABC`

Four hooks, all consulted by the `Conversation` core:

- `next_speaker(conversation)` — the participant due to speak now, or `None` when no one is due (which ends a `run` exactly like a stop condition).
- `visible(message, pov, conversation)` — whether a committed transcript message enters `pov`'s view. Default: everything is visible (the shared-transcript semantics).
- `extra_segments(conversation, participant)` — additional view segments appended after the transcript (delivered mail, pending-message notices, protocol instructions). Default: none.
- `on_commit(message, conversation)` — bookkeeping after a turn is committed (parse message-passing actions, deliver mail, advance scheduling state). Default: no-op.

Methods

`extra_segments`

```
extra_segments(
    self,
    conversation: 'Conversation',
    participant: 'Participant',
) -> 'list[ViewSegment]'
```

source

Parameters

Name	Type	Default	Description
conversation	'Conversation'	*required*	
participant	'Participant'	*required*	

next_speaker

```
next_speaker(self, conversation: 'Conversation') -> 'Participant | None'
```

source

Parameters

Name	Type	Default	Description
conversation	'Conversation'	*required*	

on_commit

```
on_commit(self, message: 'Message', conversation: 'Conversation') -> None
```

source

Parameters

Name	Type	Default	Description
message	'Message'	*required*	
conversation	'Conversation'	*required*	

reset

```
reset(self) -> None
```

source

Clear accumulated scheduling/delivery state so one policy instance can drive a fresh conversation.

`visible`

```
visible(
    self,
    message: 'Message',
    pov: 'Participant',
    conversation: 'Conversation',
) -> bool
```

source

Parameters

Name	Type	Default	Description
`message`	`'Message'`	*required*	
`pov`	`'Participant'`	*required*	
`conversation`	`'Conversation'`	*required*	

DirectPipingPolicy

A fixed pipeline: each participant sees only its **predecessor's** output (plus moderator/system framing and its own past turns), and speaking order follows the chain — A → B → C → A → …

```
DirectPipingPolicy(chain: list[str] | None = None)
```

Defined in `interlens.communication.policy` · source

Inherits from: CommunicationPolicy

This formalizes the natural two-agent dialogue framing (where "the other's turn is my input" is already how a 2-party shared transcript renders) and generalizes it to longer chains, where a shared transcript and a pipeline genuinely diverge: in a 3-agent pipe, C sees B's output but not A's. Because it is the same `Conversation` machinery, the full transcript is still recorded and serialized — visibility filters what each *model* is conditioned on, never what the record keeps.

Parameters

Name	Type	Default	Description
`chain`	`list[str] | None`	`None`	

Attributes

Name	Type	Summary
`chain`		

Methods

next_speaker

```
next_speaker(self, conversation: 'Conversation') -> 'Participant | None'
```

source

Parameters

Name	Type	Default	Description
conversation	'Conversation'	*required*	

reset

```
reset(self) -> None
```

source

visible

```
visible(
    self,
    message: 'Message',
    pov: 'Participant',
    conversation: 'Conversation',
) -> bool
```

source

Parameters

Name	Type	Default	Description
message	'Message'	*required*	
pov	'Participant'	*required*	

Name	Type	Default	Description
`conversation`	`'Conversation'`	*required*	

RoundRobinPolicy

The shared-transcript default, as an explicit policy: speakers cycle through `participants` order and everyone sees every committed turn.

```
RoundRobinPolicy(first: str | None = None)
```

Defined in `interlens.communication.policy` · source

Inherits from: CommunicationPolicy

`first` (a name) shifts the starting speaker.

Parameters

Name	Type	Default	Description
first	str | None	None	

Attributes

Name	Type	Summary
first		

Methods

next_speaker

```
next_speaker(self, conversation: 'Conversation') -> 'Participant | None'
```

source

Parameters

Name	Type	Default	Description
conversation	'Conversation'	*required*	

reset

```
reset(self) -> None
```

source

context

Package `interlens.context`

Modules

- `context_policy`
- `drop_oldest_policy`
- `error_policy`
- `sliding_window_policy`
- `summarize_policy`

Re-exported

Name	Defined in	Summary
`ContextPolicy`	`interlens.context.context_policy`	Decides how to fit a participant's view within its model's context window.
`DropOldestPolicy`	`interlens.context.drop_oldest_policy`	Drop the oldest `turn` segments (preserving system/moderator/ private_context) until the view fits.
`ErrorPolicy`	`interlens.context.error_policy`	The safe default: raise if the view exceeds the context window rather than silently dropping content.
`SlidingWindowPolicy`	`interlens.context.sliding_window_policy`	Keep the preserved framing plus the most recent `keep_last` turns; drop older turns.

Name	Defined in	Summary
SummarizePolicy	interlens.context.summarize_policy	Compress older turns into a single summary segment instead of dropping them outright (the heaviest policy).

Functions

Name	Summary
context_policy_from_dict	

context_policy

Module `interlens.context.context_policy`

Attributes

Name	Type	Summary
`PRESERVED_ORIGINS`		

Classes

Name	Summary
`ContextPolicy`	Decides how to fit a participant's view within its model's context window.

ContextPolicy

Decides how to fit a participant's view within its model's context window.

```
ContextPolicy()
```

Defined in `interlens.context.context_policy` · source

Inherits from: `ABC`

Crucially, `fit` runs on the **typed** `ViewSegment` list, *before* the family-specific `finalize_view` folds/merges anything. Operating pre-finalize means the policy can reliably preserve the system block and moderator seed (their `origin` is still intact) and trim only `turn` segments, instead of trying to reverse-engineer meaning out of already-folded text.

Methods

`fit`

```
fit(self, segments: list[ViewSegment], tokenizer, limit: int | None) ->
    list[ViewSegment]
```

source

Parameters

Name	Type	Default	Description
`segments`	list[ViewSegment]	*required*	
`tokenizer`		*required*	
`limit`	`int | None`	*required*	

to_dict

```
to_dict(self) -> dict
```

source

Serialize as `{"kind": ..., **params}`.

Subclasses with parameters extend the params; the default covers parameterless policies.

drop_oldest_policy

Module `interlens.context.drop_oldest_policy`

Classes

Name	Summary
`DropOldestPolicy`	Drop the oldest `turn` segments (preserving system/moderator/private_context) until the view fits.

DropOldestPolicy

Drop the oldest `turn` segments (preserving system/moderator/private_context) until the view fits.

```
DropOldestPolicy()
```

Defined in `interlens.context.drop_oldest_policy` · source

Inherits from: ContextPolicy

Methods

`fit`

```
fit(self, segments: list[ViewSegment], tokenizer, limit: int | None) ->
    list[ViewSegment]
```

source

Parameters

Name	Type	Default	Description
`segments`	list[ViewSegment]	*required*	
`tokenizer`		*required*	
`limit`	`int | None`	*required*	

error_policy

Module `interlens.context.error_policy`

Classes

Name	Summary
`ErrorPolicy`	The safe default: raise if the view exceeds the context window rather than silently dropping content.

ErrorPolicy

The safe default: raise if the view exceeds the context window rather than silently dropping content.

```
ErrorPolicy()
```

Defined in `interlens.context.error_policy` · source

Inherits from: ContextPolicy

Silent truncation reads as “everything fit” when it didn’t, which is exactly the kind of quiet data loss this harness avoids. Callers who want trimming opt into `DropOldestPolicy`/`SlidingWindowPolicy` explicitly.

Methods

`fit`

```
fit(self, segments: list[ViewSegment], tokenizer, limit: int | None) ->
    list[ViewSegment]
```

source

Parameters

Name	Type	Default	Description
`segments`	list[ViewSegment]	*required*	
`tokenizer`		*required*	
`limit`	`int | None`	*required*	

sliding_window_policy

Module `interlens.context.sliding_window_policy`

Classes

Name	Summary
`SlidingWindowPolicy`	Keep the preserved framing plus the most recent `keep_last` turns; drop older turns.

SlidingWindowPolicy

Keep the preserved framing plus the most recent `keep_last` turns; drop older turns.

```
SlidingWindowPolicy(keep_last: int, keep_system: bool = True)
```

Defined in `interlens.context.sliding_window_policy` · source

Inherits from: ContextPolicy

`keep_system=True` (the default) preserves the system/moderator/private_context framing regardless of the window; set it False to also let framing fall outside the window (rarely wanted). Unlike `DropOldestPolicy` this is a fixed-size window rather than a fit-to-budget trim, so it's predictable turn-to-turn.

Parameters

Name	Type	Default	Description
`keep_last`	`int`	*required*	
`keep_system`	`bool`	`True`	

Attributes

Name	Type	Summary
`keep_last`		
`keep_system`		

Methods

fit

```
fit(self, segments: list[ViewSegment], tokenizer, limit: int | None) ->
    list[ViewSegment]
```

source

Parameters

Name	Type	Default	Description
segments	list[ViewSegment]	*required*	
tokenizer		*required*	
limit	int | None	*required*	

to_dict

```
to_dict(self) -> dict
```

source

summarize_policy

Module `interlens.context.summarize_policy`

Classes

Name	Summary
`SummarizePolicy`	Compress older turns into a single summary segment instead of dropping them outright (the heaviest policy).

SummarizePolicy

Compress older turns into a single summary segment instead of dropping them outright (the heaviest policy).

```
SummarizePolicy(keep_last: int = 4, summarizer=None)
```

Defined in `interlens.context.summarize_policy` · source

Inherits from: ContextPolicy

Keeps the preserved framing (system / moderator / private_context) and the most recent `keep_last` turns verbatim, and replaces the older middle turns with one summary segment produced by `summarizer` — a callable `list[str]` → `str` over the dropped turns' contents. With no summarizer it inserts a neutral placeholder, so it degrades to a labelled drop rather than silently losing content.

`summarizer` is a live callable and so is not serialized; a loaded template gets `summarizer=None` and the caller re-injects one if needed.

Parameters

Name	Type	Default	Description
`keep_last`	`int`	`4`	
`summarizer`		`None`	

Attributes

Name	Type	Summary
`keep_last`		
`summarizer`		

Methods

fit

```
fit(self, segments: list[ViewSegment], tokenizer, limit: int | None) ->
    list[ViewSegment]
```

source

Parameters

Name	Type	Default	Description
segments	list[ViewSegment]	*required*	
tokenizer		*required*	
limit	int | None	*required*	

to_dict

```
to_dict(self) -> dict
```

source

context_policy_from_dict

```
context_policy_from_dict(data: dict) -> ContextPolicy
```

Defined in `interlens.context` · source

Parameters

Name	Type	Default	Description
`data`	`dict`	*required*	

context_item

Module `interlens.context_item`

Classes

Name	Summary
`ContextItem`	A single item of *private* asymmetric knowledge given to one participant (a briefing, a document, a fact).

ContextItem

A single item of *private* asymmetric knowledge given to one participant (a briefing, a document, a fact).

```
ContextItem(content: str, role_hint: Role = 'user', author: str =
    'moderator')
```

Defined in `interlens.context_item` · source

Deliberately distinct from `Message`: a briefing is not a dialogue turn — it has no turn index and its `author` is nominal — so overloading `Message` would give it misleading turn/authorship semantics.

Role semantics are pinned rather than role-swapped: a briefing renders by default as **user-provided context** (`role_hint=USER`) — the participant reads it as information handed to it, not as its own prior speech — with `SYSTEM` available for standing private instructions. Private context is injected only into the owning participant's view and never enters the shared transcript.

Parameters

Name	Type	Default	Description
`content`	`str`	*required*	
`role_hint`	`Role`	`'user'`	
`author`	`str`	`'moderator'`	

Attributes

Name	Type	Summary
`author`	`str`	
`content`	`str`	
`role_hint`	`Role`	

conversation

Module `interlens.conversation`

Attributes

Name	Type	Summary
`ParticipantLike`		
`PromptLike`		

Classes

Name	Summary
`Conversation`	A multi-agent conversation: the recipe, the live dialogue, AND the benchmark-rollout driver, all in one lightweight object.

Conversation

A multi-agent conversation: the recipe, the live dialogue, AND the benchmark-rollout driver, all in one lightweight object.

```
Conversation(
    participants: tuple[Participant, ...],
    transcript: Transcript = Transcript(),
    shared_context: str | None = None,
    shared_system_prompt: str | None = None,
    moderator_name: str = 'moderator',
    context_policy: ContextPolicy = ErrorPolicy(),
    context_limit: int | None = None,
    reasoning_visibility: ReasoningVisibility = ReasoningVisibility.STRIP,
    execution_mode: ExecutionMode = ExecutionMode.THROUGHPUT,
    message_hooks: list = list(),
    communication: object = None,
)
```

Defined in `interlens.conversation` · source

Inherits from: Functional

It orchestrates turn-taking between `Participant`s over a shared, perspective-neutral `Transcript`, owning *who speaks when* and the **ordered view pipeline**: for each speaker it assembles a structured, typed view (system block → private context → transcript turns), fits it to the context window on those typed segments, then lets the participant flatten it (family fold/merge) and generate. Fitting *before* the lossy family flatten is what lets the context policy preserve the system/moderator framing reliably (see `context/`).

Scenario framing is split by ownership: *shared* framing (`shared_context` as a moderator seed turn, `shared_system_prompt`) lives here; *private* framing (`system_prompt`, `private_context`) lives on each participant, so it is naturally invisible to the others and to the transcript.

Copy-on-write + lazy. Participants load their weights lazily, so an unrun `Conversation` is a cheap recipe. Build it up functionally — `conv.turns(4).data(ds).analyzer(grade)` — where each dot-modifier returns a modified *copy* (via `Functional.set`), never mutating the original; call `.set(field=value)` for fields without a dot-modifier. `build`-free: run it

directly with `run()`, or expand it over data / N samples with `rollout()` (which copies it per row — see there).

Also supports branching (`branch()` / `branch_from()`), in-place history editing (`rewind()`, `edit()`, `reset()`), ephemeral sampling (`sample()` / `sample_all()`), activation capture (`capture()`), and disk persistence (`save()` / `load()`).

Parameters

Name	Type	Default	Description
`participants`	tuple[Participant, …]	*required*	
`transcript`	Transcript	`Transcript()`	
`shared_context`	`str | None`	`None`	
`shared_system_prompt`	`str | None`	`None`	
`moderator_name`	`str`	`'moderator'`	
`context_policy`	ContextPolicy	`ErrorPolicy()`	
`context_limit`	`int | None`	`None`	
`reasoning_visibility`	ReasoningVisibility	`ReasoningVisibility.STRIP`	
`execution_mode`	ExecutionMode	`ExecutionMode.THROUGHPUT`	
`message_hooks`	`list`	`list()`	
`communication`	`object`	`None`	

Attributes

Name	Type	Summary
`communication`	`object`	Optional `CommunicationPolicy` governing turn order and message visibility (see `communication/`).

Name	Type	Summary
`context_limit`	`int | None`	Token budget the `context_policy` fits to.
`context_policy`	ContextPolicy	How each speaker's view is fit to the context window — run on the typed segments *before* the family flatten so framing is preserved.
`execution_mode`	ExecutionMode	The determinism-vs-throughput tension the runner consults: `THROUGHPUT` (default) permits batched co-stepping + KV reuse + flash-attn (only *distributional* reproducibility); `DETERMINISTIC` disables those for token-identical replay and interp fidelity.
`message_hooks`	`list`	Runtime-only middleware (**not** persisted): each hook vets / edits / denies a freshly generated message before it is committed.
`moderator_name`	`str`	Author name used for the `shared_context` seed and any moderator turns.

Name	Type	Summary
`participants`	tuple[Participant, …]	The conversation's participants.
`reasoning_visibility`	ReasoningVisibility	Whether a prior turn's parsed `<think>` reasoning is re-injected into later views: `STRIP` (never), `SELF_RETAIN` (a speaker sees only its own), `SHARED` (everyone sees everyone's).
`row`	`dict`	The dataset row that produced this conversation in a data-driven `rollout` (the same dict the `dataset_field`s resolved against), or `{}` outside a rollout.
`shared_context`	`str | None`	Scenario framing every participant sees, injected once as a leading `moderator_name` turn when the transcript starts empty.
`shared_system_prompt`	`str | None`	A system prompt merged into every participant's system block (after that participant's private `system_prompt`).

Name	Type	Summary
`transcript`	Transcript	The shared, perspective-neutral message history.

Methods

branch

```
branch(self) -> 'Conversation'
```

source

Fork into a new `Conversation` that **reuses the same participant objects** (shared weights, zero GPU cost) with a copied, independent transcript.

The branch can diverge freely; the original is untouched. This is exactly a no-op copy-on-write clone (`self.set()`). To fork from a specific point in the history rather than the end, use `branch_from`.

branch_from

```
branch_from(self, ref: MessageRef) -> 'Conversation'
```

source

Fork a new conversation whose history is this one's turns **up to and including** the turn `ref` — i.e. branch as if the conversation had stopped right after `ref`, ready for a different continuation.

`ref` is a `MessageRef`: an `int` index (negatives count from the end) or the `Message` object itself (matched by identity). The original is untouched; the fork shares participants (zero GPU cost).

Parameters

Name	Type	Default	Description
ref	MessageRef	*required*	

capture

```
capture(self, sites=('residual',), layers=None, offload: OffloadLocation =
    'cpu')
```

source

Context manager that captures activations for every `step` inside the block into a fresh `ActivationCache`, auto-tagged by the current speaker + turn:

```
with conv.capture(sites=["residual"], layers=[8, 12]) as cache:
        conv.step(bob)
cache.at(participant="bob", layer=12)
```

Parameters

Name	Type	Default	Description
sites		('residual',)	
layers		None	
offload	OffloadLocation	'cpu'	

edit

```
edit(
    self,
    ref: MessageRef,
    content: str | None = None,
    *,
    author: str | None = None,
    **metadata={},
) -> Message
```

source

Edit a committed past turn in place and return it (see `Transcript.edit`): `ref` is a `MessageRef` (int index, negatives allowed, or a `Message` object matched by identity), and `content` / `author` / `**metadata` are the fields to change.

Editing the returned `Message`'s fields directly works too, since the transcript holds it by reference.

Parameters

Name	Type	Default	Description
`ref`	`MessageRef`	*required*	
`content`	`str | None`	`None`	
`author`	`str | None`	`None`	
`metadata`		`{}`	

from_models

```
from_models(
    cls,
    models: tuple[ModelLike, ...],
    names: tuple[str, ...] = ('a', 'b'),
    device: str | torch.device | None = None,
    dtype: torch.dtype | None = None,
    shared_context: str | None = None,
    shared_system_prompt: str | None = None,
    prompt: PromptLike = None,
    **gen_kwargs={},
) -> 'Conversation'
```

source

Scaffold a conversation directly from a tuple of `models` — each an HF id or an already-loaded `PreTrainedModel` (see `ModelLike`).

Convenience wrapper around `factories.conversation_from_models`; each model becomes a family-correct participant and `names` gives them identities. **The order of `models` / `names` is the speaking order** — the first speaks first unless you pass `first=` to `run`. `**gen_kwargs` are forwarded to every participant.

Scenario framing is available here too: `shared_system_prompt` (instructions, system role) and `shared_context` (a neutral `moderator`-voiced opening seen by everyone). `prompt` is a separate convenience for a *participant*-voiced opener (a `str` is attributed to the last participant). See that function for the details.

`device` / `dtype` default to `None` here purely so this signature holds no second copy of the real defaults (`"cuda"` / `bfloat16`, which live on `conversation_from_models`) — and so importing this module does not need `torch` just to evaluate a default. `None` means “leave it to the factory”.

Parameters

Name	Type	Default	Description
`models`	`tuple[ModelLike, ...]`	*required*	
`names`	`tuple[str, ...]`	`('a', 'b')`	
`device`	`str | torch.device | None`	`None`	
`dtype`	`torch.dtype | None`	`None`	
`shared_context`	`str | None`	`None`	
`shared_system_prompt`	`str | None`	`None`	
`prompt`	`PromptLike`	`None`	
`gen_kwargs`		`{}`	

`load`

```
load(cls, directory, devices='cuda') -> 'Conversation'
```

source

Rebuild a saved conversation: reconstruct (lazy) participants from the recipe and attach the saved transcript, resuming from that state (does NOT regenerate messages).

Raises on an unsupported schema.

Parameters

Name	Type	Default	Description
`directory`		*required*	
`devices`		`'cuda'`	

participant

```
participant(self, which: ParticipantLike) -> Participant
```

source

Return one of this conversation's participants, resolved from a `Participant` object, its `name` (str), or its index (int).

Raises (via `_resolve_participant`) if it isn't in this conversation.

Parameters

Name	Type	Default	Description
`which`	`ParticipantLike`	*required*	

render_templated

```
render_templated(
    self,
    pov: ParticipantLike,
    *,
    extra=(),
    add_generation_prompt: bool = False,
    tokenize: bool = False,
)
```

source

The full `view` run through `pov`'s tokenizer chat template — the **exact prompt its model sees**, special/control tokens and all (a str, or token ids with `tokenize=True`).

This is the truthful counterpart to `transcript.render_templated`, which templates the transcript turns ONLY (no system / private framing, no context-fit). Requires a local `ModelParticipant` (needs a tokenizer).

Parameters

Name	Type	Default	Description
`pov`	`ParticipantLike`	*required*	
`extra`		`()`	
`add_generation_prompt`	`bool`	`False`	
`tokenize`	`bool`	`False`	

`reset`

```
reset(self) -> 'Conversation'
```

source

Return the conversation to its fresh, pre-`run` state: empty the transcript, then **re-seed** the `shared_context` moderator turn (exactly as `__post_init__` does on a fresh conversation).

Use this rather than `transcript.clear()` when you want to rerun the same scenario — the raw `transcript.clear()` drops the `shared_context` framing too, whereas `reset` restores it.

`rewind`

```
rewind(self, *, to: MessageRef) -> 'Conversation'
```

source

Rewind in place so the turn `to` becomes the new last turn, dropping everything after it (returns `self` for chaining).

`to` is a `MessageRef` (int index, negatives allowed, or a `Message` object). Mutates this conversation — use `branch_from` instead to keep the original. See `Transcript.rewind`.

Parameters

Name	Type	Default	Description
`to`	`MessageRef`	*required*	

`rollout`

```
rollout(
    self,
    n: int | None = None,
    *,
    devices=None,
    out_dir=None,
    resume: bool = False,
    batched: bool = True,
    max_batch_size: int | None = None,
    seed: int | None = None,
) -> 'RunReport'
```

source

Run many conversations from this one recipe and return a `RunReport` — the benchmark/scale entry point.

Rollout does NOT mutate this conversation. It makes an independent copy-on-write clone per job and runs *those*; `self` stays the unrun recipe. Finished conversations live in `report.results[job_id].conversation` (and their transcripts / analyses on the same `RunResult`) — that is where to `sample()` or inspect afterwards, NOT on the original.

Two modes: with `data()` set, expands to ONE conversation per row (job ids `row_00000…`), resolving each templated field (`dataset_field`) against the row; otherwise expands to `n` seeded copies of one scenario (job ids `rollout_0000…`). `n` defaults to `len(data)`. Sample i gets `seed + i` (`seed` defaults to the conversation's `seed` field). Any ambient `with StopCondition(...)` blocks

and the `run_until` field are captured now and attached to every job (so they survive the spawn boundary).

Parallel by default on two axes (see :func:`interlens.run`): one worker process per device, and batched co-stepping within each device. `batched=False` gives the token-identical DETERMINISTIC path.

Parameters

Name	Type	Default	Description
n	int | None	None	
devices		None	
out_dir		None	
resume	bool	False	
batched	bool	True	
max_batch_size	int | None	None	
seed	int | None	None	

run

```
run(
    self,
    turns: int | None = None,
    until: StopCondition | list | None = None,
    first: ParticipantLike | None = None,
    prompt: PromptLike = None,
) -> Transcript
```

source

Alternate speakers until `turns` elapse and/or a `StopCondition` fires (whichever comes first).

Speakers are taken in `participants` order (that tuple's order IS the turn order); `first` sets who starts (default: `participants[0]`) and the rest follow round-robin. `first` may be a `Participant`, its `name` (str), or its index (int) — resolved via `_resolve_participant` (raises if it isn't in this conversation).

Stop conditions are combined from three sources (any of which stops the run): `until` (this call), the conversation's own `run_until` field, and any ambient `with StopCondition(...):` blocks. A condition may also cap each turn's tokens (`turn_cap`) — e.g. a `TokenBudget` shrinks the final turn to land on budget. Conditions are `reset()` at the start so a reused instance doesn't leak state. `turns` defaults to the conversation's `turns` field; at least one of turns / a stop condition must be present.

`prompt` (optional) is appended to the transcript **before** the run: a `str` is attributed to the LAST participant (so `first` replies to it), a `Message` is appended as-is, `None` appends nothing. It appends to the current end and never resets/reseeds.

Parameters

Name	Type	Default	Description
`turns`	`int | None`	`None`	
`until`	StopCondition | list | None	`None`	
`first`	`ParticipantLike | None`	`None`	
`prompt`	`PromptLike`	`None`	

sample

```python
sample(
    self,
    speaker,
    message=None,
    *,
    as_author=None,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    max_new_tokens: int | None = None,
)
```

source

Ephemerally sample a response **without mutating the transcript** — a pure read of current state, safe to call repeatedly / in a loop.

`speaker` is a `ParticipantLike` (name / index / `Participant`); pass a **list or tuple** of them to sample each and get back `{name: Message}` instead of a single `Message` (see also `sample_all`). `message` is an optional temporary incoming turn; it defaults to being attributed to the **moderator** (a neutral, external voice — so “What do you think of Bob?” reads as an interviewer asking, NOT as Bob speaking). Pass `as_author="bob"` to make it read as that participant’s turn instead (e.g. “what would you say if Bob had just said X?”). The same interp options as `step` are honored on each ephemeral generation.

Parameters

Name	Type	Default	Description
`speaker`		*required*	
`message`		`None`	
`as_author`		`None`	
`steering`		`None`	
`capture`		`None`	
`patch`		`None`	
`return_logprobs`	`bool`	`False`	
`max_new_tokens`	`int | None`	`None`	

`sample_all`

```
sample_all(
    self,
    message: str | None = None,
    *,
    as_author: str | None = None,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    max_new_tokens: int | None = None,
) -> 'dict[str, Message]'
```

source

Ephemerally sample **every** participant's response — a convenience for `sample(list(self.participants), ...)`.

Returns `{name: Message}`; the transcript is untouched. Handy for "what does each model say to this right now?".

Parameters

Name	Type	Default	Description
`message`	`str | None`	`None`	
`as_author`	`str | None`	`None`	
`steering`		`None`	
`capture`		`None`	
`patch`		`None`	
`return_logprobs`	`bool`	`False`	
`max_new_tokens`	`int | None`	`None`	

save

```
save(self, directory) -> None
```

source

Persist the conversation: the transcript (`transcript.json`) + the recipe (`conversation.json` — the participants' own constructor kwargs + scenario framing/policies, no weights).

Parameters

Name	Type	Default	Description
`directory`		*required*	

step

```
step(
    self,
    speaker: Participant,
    *,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    max_new_tokens: int | None = None,
) -> Message
```

source

Have `speaker` produce and commit one turn.

Interp options flow to `generate`; if no `capture` is passed but a `conv.capture(...)` block is active, its pending request is used (auto-tagged by turn).

Parameters

Name	Type	Default	Description
`speaker`	Participant	*required*	
`steering`		None	
`capture`		None	
`patch`		None	
`return_logprobs`	bool	False	
`max_new_tokens`	int | None	None	

view

```
view(self, pov: ParticipantLike, extra=()) -> list[dict]
```

source

The full `[{role, content}]` view `pov`'s model is conditioned on — the **real generation input**: system block (its private `system_prompt` + `shared_system_prompt`)

→ `private_context` → the transcript role-swapped to `pov`, then **context-fit** and **family-flattened**.

This is exactly what `step` / `sample` feed to `generate`. Unlike `transcript.render_roles` (transcript turns only, no framing or fitting), it reflects everything the model actually sees. `pov` is a `ParticipantLike` (name / index / participant); `extra` renders temporary, uncommitted messages (as `sample` does).

Parameters

Name	Type	Default	Description
`pov`	`ParticipantLike`	*required*	
`extra`		`()`	

execution_mode

Module `interlens.execution_mode`

Classes

Name	Summary
`ExecutionMode`	Names the deterministic-vs-throughput tension explicitly rather than pretending it doesn't exist.

ExecutionMode

Names the deterministic-vs-throughput tension explicitly rather than pretending it doesn't exist.

```
ExecutionMode()
```

Defined in `interlens.execution_mode` · source

Inherits from: `str`, `Enum`

Token-identical replay, performant defaults (flash-attn + KV reuse + batched generation), and interp measurement fidelity are NOT jointly satisfiable on CUDA, so callers pick a mode:

- `DETERMINISTIC`: batching off, KV reuse off, sdpa/eager attention, deterministic algorithms. Slower, but token-identical on the same hardware and safe for capture/steering/probes. Backs the identical-replay guarantee.
- `THROUGHPUT` (default for large rollouts): flash-attn + batched generation + KV reuse. Guarantees only *distributional* reproducibility.

P1 defines the switch and threads it through; the optimizations it gates (batching, KV reuse) land in later phases and consult this flag.

Attributes

Name	Type	Summary
`DETERMINISTIC`		
`THROUGHPUT`		

factories

Module `interlens.factories`

Attributes

Name	Type	Summary
`ModelLike`		

Classes

Name	Summary
`AutoModelParticipant`	Resolver for creating `Participant` instances automatically from HuggingFace model identifiers, local model paths, or already-loaded `PreTrainedModel`s.

Functions

Name	Summary
`conversation_from_ids`	Deprecated thin alias for :func:`conversation_from_models` / :meth kept for back-compat.
`conversation_from_models`	Scaffold a conversation from a tuple of `models` — each an HF id (str) or an already-loaded `PreTrainedModel` (see `ModelLike`).

AutoModelParticipant

Resolver for creating `Participant` instances automatically from HuggingFace model identifiers, local model paths, or already-loaded `PreTrainedModel`s.

```
AutoModelParticipant()
```

Defined in `interlens.factories` · source

HF-style factory for family-correct local-model participants — the participant analog of `AutoModelForCausalLM`. It resolves the concrete `ModelParticipant` subclass from the model's transformers `config.model_type` via the class self-registry (`ModelParticipant.for_model_type`), then delegates the actual build to that class's `from_model` / `from_pretrained` — so all the loading / tokenizer-inference / chat-flag logic lives in one place (on `ModelParticipant`) and this class is *only* the family dispatcher.

- `from_` dispatches on the argument type (str id → `from_pretrained`; `PreTrainedModel` → `from_model`).
- To get a statically-known subclass, name it directly: `QwenModelParticipant.from_pretrained(...)`. This factory returns the (dynamically resolved) base `ModelParticipant` type.

Methods

from_

```
from_(
    model: ModelLike,
    *,
    name: str,
    tokenizer: PreTrainedTokenizerBase | None = None,
    device: str | torch.device = 'cuda',
    load_kwargs: dict | None = None,
    **participant_kwargs: Any = {},
) -> ModelParticipant
```

source

Build a participant from either an HF id (str) or an already-loaded `PreTrainedModel`, dispatching to `from_pretrained` / `from_model`.

`tokenizer` applies only to the loaded-model case (an id / path loads its own matching tokenizer, so it is ignored there).

Parameters

Name	Type	Default	Description
`model`	`ModelLike`	*required*	
`name`	`str`	*required*	
`tokenizer`	`PreTrainedTokenizerBase | None`	`None`	
`device`	`str | torch.device`	`'cuda'`	
`load_kwargs`	`dict | None`	`None`	
`participant_kwargs`	`Any`	`{}`	

`from_model`

```
from_model(
    model: PreTrainedModel,
    tokenizer: PreTrainedTokenizerBase | None = None,
    *,
    name: str,
    device: str | torch.device | None = None,
    **participant_kwargs: Any = {},
) -> ModelParticipant
```

source

Build a family-correct participant from an already-loaded `model` — the class is resolved from `config.model_type` (unknown types fall back to base `ModelParticipant`), then that class's :meth:`ModelParticipant.from_model` does the tokenizer inference and chat-flag derivation.

Parameters

Name	Type	Default	Description
model	PreTrainedModel	*required*	
tokenizer	PreTrainedTokenizerBase | None	None	
name	str	*required*	
device	str | torch.device | None	None	
participant_kwargs	Any	{}	

from_pretrained

```
from_pretrained(
    id_or_path: str | Path,
    *,
    name: str,
    device: str | torch.device = 'cuda',
    load_kwargs: dict | None = None,
    **participant_kwargs: Any = {},
) -> ModelParticipant
```

source

Return a family-correct participant for `id_or_path` (an HF id or local path) that will load its weights **lazily on first use**.

The concrete class is resolved from `config.model_type` by reading ONLY the model's config (`AutoConfig.from_pretrained` — cheap, no weights); `load_kwargs` (`dtype` / `attn` / `quant` / `revision` / `weights_path`) are recorded for that deferred load and `participant_kwargs` go to the participant (see :meth:`ModelParticipant.from_pretrained`).

Parameters

Name	Type	Default	Description
id_or_path	str | Path	*required*	
name	str	*required*	
device	str | torch.device	'cuda'	

Name	Type	Default	Description
`load_kwargs`	`dict | None`	`None`	
`participant_kwargs`	`Any`	`{}`	

conversation_from_ids

Deprecated thin alias for :func:`conversation_from_models` / :meth:`Conversation.from_models`, kept for back-compat.

```
conversation_from_ids(ids: tuple[ModelLike, ...], **kwargs: Any = {}) -
    > Conversation
```

Defined in `interlens.factories` · source

Prefer `Conversation.from_models(models=..., ...)`.

Parameters

Name	Type	Default	Description
`ids`	`tuple[ModelLike, ...]`	*required*	
`kwargs`	`Any`	`{}`	

conversation_from_models

Scaffold a conversation from a tuple of `models` — each an HF id (str) or an already-loaded `PreTrainedModel` (see `ModelLike`).

```
conversation_from_models(
    models: tuple[ModelLike, ...],
    names: tuple[str, ...] = ('a', 'b'),
    device: str | torch.device = 'cuda',
    dtype: torch.dtype = torch.bfloat16,
    shared_context: str | None = None,
    shared_system_prompt: str | None = None,
    prompt: PromptLike = None,
    **gen_kwargs: Any = {},
) -> Conversation
```

Defined in `interlens.factories` · source

Each becomes a family-correct participant via `AutoModelParticipant.from_`; `names` gives them their identities. **The order of `models` / `names` is the speaking order** — the first speaks first unless you pass `first=` to `run` — and `**gen_kwargs` are forwarded to every participant.

This is the implementation behind :meth:`Conversation.from_models` (which just wraps it). If two ids resolve to the same HF model the weights are loaded **once** and shared (via `load_model`'s process cache).

Two ways to seed the opening, without touching the transcript by hand:

- `shared_context` — a neutral, `moderator`-voiced turn everyone sees (scenario/topic framing); pair with `shared_system_prompt` for system-role instructions. These are the principled framing knobs (serialized into a template).
- `prompt` — a *participant*-voiced opener: a `str` is attributed to the LAST participant so the first speaker replies to it, a `Message` sets the author explicitly. Use this when the opener should read as something a speaker said rather than moderator framing.

Parameters

Name	Type	Default	Description
`models`	`tuple[ModelLike, ...]`	*required*	
`names`	`tuple[str, ...]`	`('a', 'b')`	
`device`	`str | torch.device`	`'cuda'`	
`dtype`	`torch.dtype`	`torch.bfloat16`	
`shared_context`	`str | None`	`None`	
`shared_system_prompt`	`str | None`	`None`	
`prompt`	`PromptLike`	`None`	
`gen_kwargs`	`Any`	`{}`	

functional

Module `interlens.functional`

Copy-on-write functional-update support shared by `Participant` and `Conversation`.

Both are dataclasses that the user builds up incrementally (`conv.turns(6).data(ds).analyzer(grade)`) and replicates cheaply (one recipe → N per-row copies in a rollout). The `Functional` mixin gives them a single uniform update primitive — `obj.set(**changes) → new obj` — plus optional per-field *dot-modifier sugar* (`conv.turns(6)`) that reads with no argument and returns a modified copy with one.

Why `copy.copy` + an explicit `_after_set` hook rather than `dataclasses.replace`: a live participant carries heavy shared state (the loaded `_model`/`_tokenizer`) and volatile per-conversation state (KV cache, a pending capture). We want the copy to **share** the heavy objects by reference (zero GPU cost — the whole point of copy-on-write here) but start with **fresh** volatile state. `copy.copy` shallow-copies the instance dict (so heavy refs are shared automatically) and, crucially, does NOT re-run `__init__`/`__post_init__` (which for a participant would try to re-derive device from the model, and for a conversation would re-seed/re-validate), leaving us to reset exactly the volatile fields in `_after_set`. The result is precise control over what is shared vs. what is reset — which `replace` cannot express when the sharing/reset split cuts across init and non-init state.

Classes

Name	Summary
`Functional`	Mixin giving a dataclass copy-on-write updates via `set(**changes)`.

Functions

Name	Summary
`sugar_fields`	Class decorator declaring copy-on-write dot-modifier sugar for `names`.

Functional

Mixin giving a dataclass copy-on-write updates via `set(**changes)`.

```
Functional()
```

Defined in `interlens.functional` · source

`set` shallow-copies the instance (heavy refs like a loaded model are shared, not reloaded), applies the changes, then calls `_after_set` to reset volatile per-instance state on the copy. Sugared field names (declared via `sugar_fields`) are accepted under their public spelling and routed to their `_`-prefixed storage. Unknown fields raise `TypeError`. The original is never mutated.

Methods

set

```
set(self, **changes={})
```

source

Return a modified shallow copy: the same object with `changes` applied and volatile state reset.

The receiver is untouched (copy-on-write); heavy state (a loaded model, an API client) is shared by reference. Unknown field names raise `TypeError`.

The copy is a raw instance-dict clone (NOT `copy.copy`), so it deliberately bypasses `__getstate__` — which exists to DROP the loaded model/client on *pickle* (spawn boundary). A `.set()` clone, by contrast, is in-process and must KEEP those shared references (that is the whole point of copy-on-write here).

Parameters

Name	Type	Default	Description
changes		{}	

sugar_fields

Class decorator declaring copy-on-write dot-modifier sugar for `names`.

```
sugar_fields(*names: str = ())
```

Defined in `interlens.functional` · source

Each `name` becomes an accessor (`obj.name()` reads, `obj.name(v)` returns a modified copy) backed by the dataclass field `_name`. The public names are also accepted by `set(**changes)`; the underscored storage names are hidden from it.

Parameters

Name	Type	Default	Description
`names`	`str`	`()`	

hooks

Package `interlens.hooks`

Modules

- `message_hook`

Re-exported

Name	Defined in	Summary
`HookAction`	`interlens.hooks.message_hook`	
`MessageHook`	`interlens.hooks.message_hook`	Middleware that inspects each freshly generated message *before* it is committed to the transcript, and may approve / deny / edit it.
`MessageHookResult`	`interlens.hooks.message_hook`	

message_hook

Module `interlens.hooks.message_hook`

Classes

Name	Summary
`HookAction`	
`MessageHook`	Middleware that inspects each freshly generated message *before* it is committed to the transcript, and may approve / deny / edit it.
`MessageHookResult`	

HookAction

```
HookAction()
```

Defined in `interlens.hooks.message_hook` · source

Inherits from: `str`, `Enum`

Attributes

Name	Type	Summary
APPROVE		
DENY		
EDIT		

MessageHook

Middleware that inspects each freshly generated message *before* it is committed to the transcript, and may approve / deny / edit it.

```
MessageHook()
```

Defined in `interlens.hooks.message_hook` · source

Inherits from: `ABC`

This is the seam for a future LLM-judge that vets or rewrites turns (e.g. safety filtering, format enforcement). Hooks live on the live `Conversation` (`conversation.message_hooks`) and are NOT serialized in the template — they're a runtime policy, not part of the scenario recipe. The default is an empty hook list, i.e. today's pass-through behavior.

Methods

review

```
review(self, message: 'Message', conversation: 'Conversation') ->
    MessageHookResult
```

source

Parameters

Name	Type	Default	Description
`message`	`'Message'`	*required*	
`conversation`	`'Conversation'`	*required*	

MessageHookResult

```
MessageHookResult(action: HookAction, message: 'Message | None' = None)
```

Defined in `interlens.hooks.message_hook` · source

Parameters

Name	Type	Default	Description
`action`	HookAction	*required*	
`message`	`'Message | None'`	`None`	

Attributes

Name	Type	Summary
`action`	HookAction	
`message`	`'Message | None'`	

Methods

approve

```
approve(cls) -> 'MessageHookResult'
```

source

deny

```
deny(cls) -> 'MessageHookResult'
```

source

edit

```
edit(cls, message) -> 'MessageHookResult'
```

source

Parameters

Name	Type	Default	Description
message		*required*	

integrations

Package `interlens.integrations`

Optional adapters that connect Interlens participants to external runtimes.

Modules

- `control_tower` — Run a local Interlens participant behind Control Tower's untrusted-policy boundary.

control_tower

Module `interlens.integrations.control_tower`

Run a local Interlens participant behind Control Tower's untrusted-policy boundary.

Control Tower loads :func:`interlens_untrusted_policy` as an external `module:function` policy. The returned generate function has Inspect's normal model signature, so blue protocols see and control every generated tool call. Interlens owns local generation and interpretability; Control Tower owns tool execution, monitoring, auditing, sandboxing, and trajectory storage.

Attributes

Name	Type	Summary
`InterlensGenerateFn`		

Functions

Name	Summary
`interlens_attack_policy`	Control Tower attack policy backed by an Interlens local participant.
`interlens_team_policy`	Control Tower policy backed by a private Interlens planner-reviewer-executor team.
`interlens_untrusted_policy`	Control Tower external untrusted policy backed by a local Interlens participant.
`participant_generate`	Adapt one local Interlens participant to Inspect's model-generation protocol.

Name	Summary
`team_participant_generate`	Adapt one local participant as a private planner-reviewer-executor team.

interlens_attack_policy

Control Tower attack policy backed by an Interlens local participant.

```
interlens_attack_policy(state: TaskState, participant_factory: str) ->
    InterlensGenerateFn
```

Defined in `interlens.integrations.control_tower` · source

Returns

`InterlensGenerateFn` — An adapted participant that pursues the configured side task while Control Tower retains tool execution and monitoring authority.

Parameters

Name	Type	Default	Description
`state`	`TaskState`	*required*	Current Control Tower sample. Its private side-task goal becomes the policy-only system prompt.
`participant_factory`	`str`	*required*	Zero-argument `module:function` participant factory. Use the same factory as the matched honest condition so only the evaluation objective changes.

interlens_team_policy

Control Tower policy backed by a private Interlens planner-reviewer-executor team.

```
interlens_team_policy(
    state: TaskState,
    participant_factory: str,
    mode: str = 'honest',
    planner_max_tokens: int = 512,
    reviewer_max_tokens: int = 512,
) -> InterlensGenerateFn
```

Defined in `interlens.integrations.control_tower` · source

Returns

`InterlensGenerateFn` — An Inspect-compatible generate function whose external actions remain fully visible to Control Tower.

Parameters

Name	Type	Default	Description
`state`	`TaskState`	*required*	Current Control Tower sample. In `attack` mode its side-task goal is used for the private prompt.
`participant_factory`	`str`	*required*	Zero-argument `module:function` factory for the local policy under evaluation.
`mode`	`str`	`'honest'`	`"honest"` preserves Control Tower's task

Name	Type	Default	Description
			prompt. `"attack"` privately replaces it with the same side-task scaffold used by :func:`interlens_attack_policy`.
`planner_max_tokens`	`int`	`512`	Maximum output tokens used by each internal planner generation. Values around 256–1024 trade deliberation depth against the three-call team's inference cost.
`reviewer_max_tokens`	`int`	`512`	Maximum output tokens used by each internal reviewer generation. Values around 256–1024 trade critique depth against inference cost.

interlens_untrusted_policy

Control Tower external untrusted policy backed by a local Interlens participant.

```
interlens_untrusted_policy(
    state: TaskState,
    participant_factory: str,
    system_prompt: str | None = None,
) -> InterlensGenerateFn
```

Defined in `interlens.integrations.control_tower` · source

Select this function with `--untrusted-policy interlens.integrations.control_tower:interlens_untrusted_policy`.

Returns

`InterlensGenerateFn` — The adapted participant's Inspect-compatible generate function.

Parameters

Name	Type	Default	Description
`state`	`TaskState`	*required*	Current Control Tower sample state. It establishes the per-sample factory lifetime; task content itself arrives through the generate function's messages.
`participant_factory`	`str`	*required*	Zero-argument `module:function` reference re-

Name	Type	Default	Description
			turning a fresh local Interlens participant, for example `"my_project.policies:local_qwen"`.
`system_prompt`	`str | None`	`None`	Optional private system prompt. Leave unset for ordinary honest evaluations; set only when a static alternate scaffold is part of the registered experimental condition.

participant_generate

Adapt one local Interlens participant to Inspect's model-generation protocol.

```
participant_generate(
    participant: Any,
    *,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    system_prompt: str | None = None,
) -> InterlensGenerateFn
```

Defined in `interlens.integrations.control_tower` · source

Returns

`InterlensGenerateFn` — An async function with the `UntrustedPolicyGenerateFn` signature expected by Control Tower.

Parameters

Name	Type	Default	Description
`participant`	`Any`	*required*	Participant implementing `generate_step`. Normally this is a `ModelParticipant` returned by `AutoModelParticipant.from_pretr` A single instance is used for one Control Tower sample.
`steering`		`None`	Optional Interlens steering spec applied to every model

Name	Type	Default	Description
			call. `None` uses the participant default.
`capture`		`None`	Optional Interlens capture request applied to every model call.
`patch`		`None`	Optional Interlens activation patch applied to every model call.
`return_logprobs`	`bool`	`False`	Whether Interlens should collect generated-token log probabilities. Enable only when later analysis needs them because it increases memory use.
`system_prompt`	`str | None`	`None`	Optional private system prompt. `None` preserves Control Tower's history; a string replaces the public system prompt before local genera-

Name	Type	Default	Description
			tion while leaving the monitor-visible history unchanged.

team_participant_generate

Adapt one local participant as a private planner-reviewer-executor team.

```
team_participant_generate(
    participant: Any,
    *,
    planner_max_tokens: int = 512,
    reviewer_max_tokens: int = 512,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    system_prompt: str | None = None,
) -> InterlensGenerateFn
```

Defined in `interlens.integrations.control_tower` · source

Every outer Control Tower step performs three sequential generations with the same model weights. The planner and reviewer see the environment history and a text summary of available capabilities but receive no callable tool schemas. The executor sees their advice and alone receives the real schemas. Only the executor's proposed action crosses the Control Tower policy boundary; the private deliberation is retained in `ModelOutput` metadata, and all three calls contribute to reported tokens and cost.

Returns

`InterlensGenerateFn` — An Inspect-compatible asynchronous generation function.

Parameters

Name	Type	Default	Description
`participant`	`Any`	*required*	Participant implementing `generate_step`. One instance is retained for the whole sam-

Name	Type	Default	Description
			ple, so local model weights and state are reused without creating three PEFT wrappers.
`planner_max_tokens`	`int`	`512`	Maximum output tokens for each private planner call. Use smaller values to reduce team overhead; increase for environments requiring multi-file plans.
`reviewer_max_tokens`	`int`	`512`	Maximum output tokens for each private reviewer call. Use smaller values for terse checks; increase when reviewers need to reason about long tool results.
`steering`		`None`	Optional Interlens steering applied to planner, reviewer,

Name	Type	Default	Description
			and executor generations.
`capture`		`None`	Optional Interlens activation capture applied to all three roles.
`patch`		`None`	Optional Interlens activation patch applied to all three roles.
`return_logprobs`	`bool`	`False`	Whether to retain generated-token log probabilities for all roles. This is memory-intensive.
`system_prompt`	`str | None`	`None`	Optional private system-prompt replacement shared by all three roles. Attack evaluations use this to provide the side-task objective without exposing it to blue-team monitors.

interp

Package `interlens.interp`

First-class interpretability layer.

All four tools hook into the *same* generation path (real turns, `sample`, and every generation inside the future tool loop) and are tagged to conversation structure, so downstream consumers (logit lens, SAEs, probes, CKA/Procrustes) read the `ActivationCache` via the raw-model escape hatch without any harness change.

Modules

- `activation_cache`
- `bridge` — Differentiable bridges for feeding one model's output into another's input.
- `capture`
- `grad` — Gradient-enabled forward passes for backprop *through* a model.
- `layers`
- `logprobs`
- `patching`
- `pooling` — Pool a token-position axis down to one vector per span — the primitive under every span-level readout.
- `routing` — Mixture-of-Experts routing capture and statistics.
- `softtokens` — Virtual (soft) tokens inside ordinary text prompts, plus the message-span read path that pairs with them.
- `steering`

Re-exported

Name	Defined in	Summary
`ActivationCache`	`interlens.interp.activation_cache`	A queryable store of captured activations,

Name	Defined in	Summary
		tagged by conversation structure.
`ActivationRecord`	`interlens.interp.activation_cache`	One captured tensor plus everything needed to know *what it is*.
`CaptureRequest`	`interlens.interp.capture`	A pending capture handed to `generate`: where to store records (`cache`) and what to grab (`spec`).
`CaptureSpec`	`interlens.interp.activation_cache`	What to capture during a generation: which `sites` at which `layers`, and where to keep the tensors.
`CapturedSite`	`interlens.interp.capture`	One activation captured by `capture_activations`: the `tensor` (`[seq, d_model]`) at a given `layer` and `site`.
`GradCaptureSpec`	`interlens.interp.grad`	What grad-connected activations to pull from `forward_with_grad` (mirrors `CaptureSpec` minus offload).
`GradForwardOutput`	`interlens.interp.grad`	Result of `forward_with_grad`: grad-connected `logits` (`[batch, seq, vocab]`)

Name	Defined in	Summary
		and, if a `GradCaptureSpec` was passed, `hidden` mapping `(site, layer)` → `tensor[batch, seq, d_model]` (also grad-connected).
`LinearBridge`	`interlens.interp.bridge`	Learned linear map from model A's hidden width `d_a` to model B's embedding width `d_b`.
`OffloadLocation`	`interlens.interp.activation_cache`	
`Patch`	`interlens.interp.patching`	Activation patching: overwrite a decoder layer's residual at specific token `positions` with saved `activations` (captured from another run/branch).
`Phase`	`interlens.interp.activation_cache`	
`Pool`	`interlens.interp.pooling`	
`RouterSteeringSpec`	`interlens.interp.routing`	A causal intervention on MoE routing: add a per-expert bias to the gate logits during generation, nudging which experts fire.
`RoutingCapture`	`interlens.interp.routing`	Per-token routing at one MoE layer, from one `capture_router_logits` pass.

Name	Defined in	Summary
RoutingStats	interlens.interp.routing	Aggregate expert-usage distributions over a set of token positions.
Site	interlens.interp.activation_cache	
Span	interlens.interp.pooling	
SteeringSpec	interlens.interp.steering	A residual-stream intervention applied during generation via forward hooks on decoder layers.
VirtualTokenInjector	interlens.interp.softtokens	Reserve n_soft positions in a *text* prompt and substitute their input embeddings at forward time.
capture_activations	interlens.interp.capture	Run one clean forward pass over input_ids and return [(layer, site, tensor[seq, d_model])].
capture_router_logits	interlens.interp.routing	One clean forward pass over input_ids ([1, seq]); return per-MoE-layer RoutingCapture.
continuation_logprob	interlens.interp.grad	Differentiable teacher-forced logprob of target_ids continuing a prefix, under model.
decoder_layers	interlens.interp.layers	Return the list of transformer decoder layer

Name	Defined in	Summary
		modules, across common HF architectures.
forward_with_grad	interlens.interp.grad	One grad-connected forward pass over input_ids XOR inputs_embeds.
group_pool	interlens.interp.pooling	Pool each *group* of spans into one vector; returns ([len(groups), d_model], valid[len(groups)]).
gumbel_softmax_tokens	interlens.interp.bridge	Relaxed (Gumbel-softmax) sample over the last (vocab) dim of logits, differentiable in logits.
js_divergence	interlens.interp.routing	Per-layer Jensen–Shannon divergence (symmetric, bounded by ln 2) → [n_layers].
kl_divergence	interlens.interp.routing	Per-layer KL(p || q) between expert distributions [n_layers, n_experts] → [n_layers].
message_token_spans	interlens.interp.routing	Token span (start, end) of each message of view in the fully-rendered chat-template sequence.

Name	Defined in	Summary
`moe_layer_indices`	`interlens.interp.routing`	Decoder-layer indices that carry a sparse MoE block.
`moe_num_experts`	`interlens.interp.routing`	Number of routed experts per MoE layer (`config.num_experts` — same field name in OL-MoE/Qwen-MoE).
`moe_topk`	`interlens.interp.routing`	Experts selected per token (`config.num_experts_per_tok`).
`routing_stats`	`interlens.interp.routing`	Pool per-token routing into per-layer expert-usage distributions.
`soft_embed`	`interlens.interp.bridge`	Mix `model`'s input-embedding rows by a per-position distribution: `[..., V] @ E[V, d]` → `[..., d]`.
`span_pool`	`interlens.interp.pooling`	Pool `hidden[start:end]` for each span; returns `[len(spans), d_model]`.
`span_pooled_residuals`	`interlens.interp.softtokens`	One pooled activation vector **per message** of `view`, at each requested site/layer.
`token_logprobs`	`interlens.interp.logprobs`	Compute per-token logprobs / surprisal / entropy for a generation.

Name	Defined in	Summary
`topk_expert_overlap`	`interlens.interp.routing`	Per-layer fraction of overlap between the `k` most-used experts of `p` and of `q` → `[n_layers]`.

activation_cache

Module `interlens.interp.activation_cache`

Attributes

Name	Type	Summary
`OffloadLocation`		
`Phase`		
`Site`		

Classes

Name	Summary
`ActivationCache`	A queryable store of captured activations, tagged by conversation structure.
`ActivationRecord`	One captured tensor plus everything needed to know *what it is*.
`CaptureSpec`	What to capture during a generation: which `sites` at which `layers`, and where to keep the tensors.

Functions

Name	Summary
`offload_to_cpu`	Move a list of GPU tensors to CPU efficiently, preserving order.

ActivationCache

A queryable store of captured activations, tagged by conversation structure.

```
ActivationCache(offload: OffloadLocation = 'cpu')
```

Defined in `interlens.interp.activation_cache` · source

Records know which participant/turn/layer/site they came from, so a downstream probe can ask for “bob’s turn 3, layer 18, the answer span” rather than juggling anonymous tensors. This is the object every interp consumer reads; the harness never puts activations in `Message.metadata` (that would blow up `branch()`‘s transcript copy), so the cache is the single home for heavy tensors.

Parameters

Name	Type	Default	Description
`offload`	`OffloadLocation`	`'cpu'`	

Attributes

Name	Type	Summary
`offload`		
`records`	list[ActivationRecord]	

Methods

add

```
add(self, record: ActivationRecord) -> None
```

source

Parameters

Name	Type	Default	Description
`record`	Activation-Record	*required*	

add_batch

```
add_batch(self, records: list[ActivationRecord]) -> None
```

source

Add many records at once, offloading all their tensors in one batched pinned transfer (see `offload_to_cpu`).

Preferred over a loop of `add` when a single capture pass produces many records — it turns N GPU->CPU copies into one per shape-group.

Parameters

Name	Type	Default	Description
`records`	list[Activation-Record]	*required*	

at

```
at(
    self,
    *,
    participant=None,
    message_idx=None,
    layer=None,
    site='residual',
) -> torch.Tensor
```

source

Return the single matching tensor, erroring if the filters aren't unique — the ergonomic accessor for "give me exactly this activation".

Parameters

Name	Type	Default	Description
`participant`		`None`	
`message_idx`		`None`	
`layer`		`None`	
`site`		`'residual'`	

query

```
query(
    self,
    *,
    participant=None,
    message_idx=None,
    layer=None,
    site=None,
) -> list[ActivationRecord]
```

source

Parameters

Name	Type	Default	Description
`participant`		`None`	
`message_idx`		`None`	
`layer`		`None`	
`site`		`None`	

ActivationRecord

One captured tensor plus everything needed to know *what it is*.

```
ActivationRecord(
    participant: str,
    message_idx: int,
    layer: int,
    site: Site,
    tensor: torch.Tensor,
    token_span: tuple[int, int],
    phases: dict[Phase, tuple[int, int]] = dict(),
)
```

Defined in `interlens.interp.activation_cache` · source

`tensor` is `[seq, d_model]` for one (participant turn, layer, site). `phases` maps `prompt` / `reasoning` / `answer` to `(start, end)` token indices into that sequence, so reasoning-vs-answer activations are separable for CoT models. `token_span` is the `(prompt_len, seq_len)` boundary between the fed-in prompt and the newly generated tokens.

Parameters

Name	Type	Default	Description
`participant`	`str`	*required*	
`message_idx`	`int`	*required*	
`layer`	`int`	*required*	
`site`	`Site`	*required*	
`tensor`	`torch.Tensor`	*required*	
`token_span`	`tuple[int, int]`	*required*	
`phases`	`dict[Phase, tuple[int, int]]`	`dict()`	

Attributes

Name	Type	Summary
`layer`	`int`	
`message_idx`	`int`	
`participant`	`str`	
`phases`	`dict[Phase, tuple[int, int]]`	
`site`	`Site`	
`tensor`	`torch.Tensor`	
`token_span`	`tuple[int, int]`	

CaptureSpec

What to capture during a generation: which `sites` at which `layers`, and where to keep the tensors.

```
CaptureSpec(
    sites: tuple[Site, ...] = ('residual',),
    layers: tuple[int, ...] | None = None,
    offload: OffloadLocation = 'cpu',
)
```

Defined in `interlens.interp.activation_cache` · source

Capture defaults to a *narrow* set — you pass the layers/sites you actually want — because capturing all layers × all tokens × many rollouts OOMs fast. `offload='cpu'` moves captured tensors off-GPU as they are recorded (essential for large sweeps); `offload=None` keeps them on-device.

Parameters

Name	Type	Default	Description
`sites`	`tuple[Site, ...]`	`('residual',)`	
`layers`	`tuple[int, ...] | None`	`None`	
`offload`	`OffloadLocation`	`'cpu'`	

Attributes

Name	Type	Summary
`layers`	`tuple[int, ...] | None`	
`offload`	`OffloadLocation`	
`sites`	`tuple[Site, ...]`	

offload_to_cpu

Move a list of GPU tensors to CPU efficiently, preserving order.

```
offload_to_cpu(tensors: list[torch.Tensor]) -> list[torch.Tensor]
```

Defined in `interlens.interp.activation_cache` · source

Two wins over per-tensor `.to('cpu')`: (1) **batching** — same-shape/dtype tensors are stacked into one D2H copy, so N transfer launches collapse to one per shape-group; (2) **pinned + non_blocking** — the copy goes through a pinned host staging buffer, which hits full PCIe bandwidth (pageable D2H is throttled by CUDA's internal bounce buffer) and lets the transfer overlap with compute. Results are cloned into pageable memory so the (limited) pinned buffer is freed immediately rather than pinned for the cache's lifetime. Falls back to a plain detach/copy when there's nothing to gain (CPU inputs or CUDA unavailable).

Parameters

Name	Type	Default	Description
`tensors`	`list[torch.Tensor]`	*required*	

bridge

Module `interlens.interp.bridge`

Differentiable bridges for feeding one model's output into another's input.

Text is a non-differentiable bottleneck: sampling a discrete token from model A kills the gradient before it can reach B. These utilities replace the sampled token with a *continuous relaxation* that B can consume as soft embeddings, so a loss on B backpropagates all the way into A. Two regimes:

- **Shared tokenizer (vocab mixture):** `soft_embed` turns a distribution over A's vocab (optionally relaxed via `gumbel_softmax_tokens`) into an embedding in B's space by mixing B's embedding rows. Exact when the tokenizers match; `soft_embed(B, one_hot(ids))` equals `B`'s ordinary token embedding of `ids`.
- **Different tokenizers (learned adapter):** `LinearBridge` maps A's hidden state (`d_a`) into B's embedding space (`d_b`), sidestepping the vocab mismatch — the same shape of cross-model linear map used in the Procrustes/CKA analyses, but here trained jointly against B's downstream loss.

Pair these with `grad.forward_with_grad` / `grad.continuation_logprob` (which accept `inputs_embeds`) to close the A -> B loop.

Classes

Name	Summary
`LinearBridge`	Learned linear map from model A's hidden width `d_a` to model B's embedding width `d_b`.

Functions

Name	Summary
`gumbel_softmax_tokens`	Relaxed (Gumbel-softmax) sample over the last (vocab) dim of `logits`, differentiable in `logits`.
`soft_embed`	Mix `model`'s input-embedding rows by a per-position distribution: `[..., V] @ E[V, d]` → `[..., d]`.

LinearBridge

Learned linear map from model A's hidden width `d_a` to model B's embedding width `d_b`.

```
LinearBridge(d_a: int, d_b: int, bias: bool = False)
```

Defined in `interlens.interp.bridge` · source

Inherits from: `nn.Module`

For heterogeneous (different-tokenizer) pairs where a vocab mixture is undefined: take A's last-layer hidden states (via `forward_with_grad(..., capture=GradCaptureSpec(sites=('residual',), layers=(last,)))`), map them into B's embedding space, and pass the result as `inputs_embeds` to B. Trained jointly with A against B's loss. `bias=False` by default to match the (linear, origin-preserving) cross-model maps used in the Procrustes work.

Parameters

Name	Type	Default	Description
`d_a`	`int`	*required*	
`d_b`	`int`	*required*	
`bias`	`bool`	`False`	

Attributes

Name	Type	Summary
`proj`		

Methods

forward

```
forward(self, hidden: torch.Tensor) -> torch.Tensor
```

source

`[batch, seq, d_a]` → `[batch, seq, d_b]` soft embeddings in B's input space.

Parameters

Name	Type	Default	Description
`hidden`	`torch.Tensor`	*required*	

gumbel_softmax_tokens

Relaxed (Gumbel-softmax) sample over the last (vocab) dim of `logits`, differentiable in `logits`.

```
gumbel_softmax_tokens(
    logits: torch.Tensor,
    tau: float = 1.0,
    hard: bool = False,
) -> torch.Tensor
```

Defined in `interlens.interp.bridge` · source

`tau` is the temperature: high -> smooth mixture (biased but low-variance gradients), low -> near-one-hot (faithful to discrete sampling but higher-variance). Anneal `tau` down over training to move from an easy soft optimization toward the discrete tokens B will actually see. `hard=True` uses the straight-through estimator: a true one-hot on the forward pass, softmax gradient on the backward pass — so the value B consumes is a real token while gradients still flow. Returns `[..., V]` to feed straight into `soft_embed`.

Parameters

Name	Type	Default	Description
`logits`	`torch.Tensor`	*required*	
`tau`	`float`	`1.0`	
`hard`	`bool`	`False`	

soft_embed

Mix `model`'s input-embedding rows by a per-position distribution: `[..., V] @ E[V, d] → [..., d]`.

```
soft_embed(model: 'PreTrainedModel', probs: torch.Tensor) -> torch.Tensor
```

Defined in `interlens.interp.bridge` · source

`probs` is a (relaxed or one-hot) distribution over `model`'s vocab for each position — e.g. a softmax / `gumbel_softmax_tokens` output of an upstream model. Returns grad-connected soft embeddings suitable to pass as `inputs_embeds` to `forward_with_grad` / `continuation_logprob`. This is exact for hard one-hots: feeding `one_hot(ids)` reproduces `model.get_input_embeddings()(ids)`.

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	
`probs`	`torch.Tensor`	*required*	

capture

Module `interlens.interp.capture`

Classes

Name	Summary
`CaptureRequest`	A pending capture handed to `generate`: where to store records (`cache`) and what to grab (`spec`).
`CapturedSite`	One activation captured by `capture_activations`: the `tensor` (`[seq, d_model]`) at a given `layer` and `site`.

Functions

Name	Summary
`capture_activations`	Run one clean forward pass over `input_ids` and return `[(layer, site, tensor[seq, d_model])]`.

CaptureRequest

A pending capture handed to `generate`: where to store records (`cache`) and what to grab (`spec`).

```
CaptureRequest(cache: ActivationCache, spec: CaptureSpec)
```

Defined in `interlens.interp.capture` · source

The `Conversation` builds this (via `conv.capture(...)`) and the participant fills the cache with records tagged by participant + turn, so the caller ends up with structurally-tagged activations.

Parameters

Name	Type	Default	Description
`cache`	Activation-Cache	*required*	
`spec`	CaptureSpec	*required*	

Attributes

Name	Type	Summary
`cache`	ActivationCache	
`spec`	CaptureSpec	

CapturedSite

One activation captured by `capture_activations`: the `tensor` (`[seq, d_model]`) at a given `layer` and `site`.

```
CapturedSite()
```

Defined in `interlens.interp.capture` · source

Inherits from: `NamedTuple`

A lightweight typed row (unpacks like the old `(layer, site, tensor)` tuple) that the participant folds into a fully-tagged `ActivationRecord`.

Attributes

Name	Type	Summary
`layer`	`int`	
`site`	`Site`	
`tensor`	`torch.Tensor`	

capture_activations

Run one clean forward pass over `input_ids` and return `[(layer, site, tensor[seq, d_model])]`.

```
capture_activations(
    model: 'PreTrainedModel',
    input_ids: torch.Tensor,
    spec: CaptureSpec,
) -> list[CapturedSite]
```

Defined in `interlens.interp.capture` · source

Design choice: capture is a *separate forward pass* over the full (prompt + generated) sequence rather than accumulating hooks across the multi-step decode loop. This is simpler and provably complete — every position is present in one pass — at the cost of one extra forward. Residual-stream activations come from `output_hidden_states` (no hooks needed); `attn`/`mlp` sub-layer outputs come from forward hooks on the corresponding submodules.

Note: `attn` captures the attention *sublayer output* (post-o_proj), which is available under any attention backend. Attention *weights/patterns* are NOT captured here — those require `attn_implementation='eager'` + `output_attentions` and are out of scope for the default kernel.

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	
`input_ids`	`torch.Tensor`	*required*	
`spec`	CaptureSpec	*required*	

grad

Module `interlens.interp.grad`

Gradient-enabled forward passes for backprop *through* a model.

The rest of interp is read-only: `capture_activations` runs under `torch.inference_mode` and detaches, which is right for logit-lens / probe readouts but throws away the graph. This module is the escape hatch for optimization *through* a (usually frozen) model: differentiable soft-prompt tuning into a target, and end-to-end backprop across two stacked models (model A's relaxed tokens feed model B via `bridge.soft_embed`). Nothing here touches `ModelParticipant.generate` — these are standalone functions on the raw `.model`, matching the style of `token_logprobs`/`decoder_layers`.

Two design points: - Accept `inputs_embeds` (soft embeddings) as a first-class input, not just `input_ids` — that is how a continuous, differentiable signal enters a transformer. Residual-stream capture is grad-connected (via `output_hidden_states` on the *same* forward, not the separate detached pass `capture_activations` uses). - `checkpoint=True` turns on HF gradient checkpointing for the pass, so backprop through a frozen B recomputes activations instead of storing them — the memory lever that lets a 0.5B+1.5B two-model stack fit one GPU.

Classes

Name	Summary
`GradCaptureSpec`	What grad-connected activations to pull from `forward_with_grad` (mirrors `CaptureSpec` minus offload).
`GradForwardOutput`	Result of `forward_with_grad`: grad-connected `logits` (`[batch, seq, vocab]`) and, if a `GradCaptureSpec` was passed, `hidden` mapping `(site,`

Name	Summary
	`layer) → tensor[batch, seq, d_model]` (also grad-connected).

Functions

Name	Summary
`continuation_logprob`	Differentiable teacher-forced log-prob of `target_ids` continuing a prefix, under `model`.
`forward_with_grad`	One grad-connected forward pass over `input_ids` XOR `inputs_embeds`.

GradCaptureSpec

What grad-connected activations to pull from `forward_with_grad` (mirrors `CaptureSpec` minus offload).

```
GradCaptureSpec(
    sites: tuple[Site, ...] = ('residual',),
    layers: tuple[int, ...] | None = None,
)
```

Defined in `interlens.interp.grad` · source

`sites` is any of `"residual"`/`"attn"`/`"mlp"`; `layers` selects decoder-layer indices (`None` = all). Unlike `CaptureSpec` there is no `offload` — the whole point is to keep tensors on-device and in the autograd graph, so an intermediate-layer objective (e.g. project onto a concept direction) can be backpropagated.

Parameters

Name	Type	Default	Description
`sites`	`tuple[Site, ...]`	`('residual',)`	
`layers`	`tuple[int, ...] | None`	`None`	

Attributes

Name	Type	Summary
`layers`	`tuple[int, ...] | None`	
`sites`	`tuple[Site, ...]`	

GradForwardOutput

Result of `forward_with_grad`: grad-connected `logits` (`[batch, seq, vocab]`) and, if a `GradCaptureSpec` was passed, `hidden` mapping `(site, layer)` → `tensor[batch, seq, d_model]` (also grad-connected).

```
GradForwardOutput()
```

Defined in `interlens.interp.grad` · source

Inherits from: `NamedTuple`

`hidden` is empty when no capture was requested.

Attributes

Name	Type	Summary
`hidden`	`dict[tuple[Site, int], torch.Tensor]`	
`logits`	`torch.Tensor`	

continuation_logprob

Differentiable teacher-forced logprob of `target_ids` continuing a prefix, under `model`.

```
continuation_logprob(
    model: 'PreTrainedModel',
    *,
    target_ids: torch.Tensor,
    prefix_ids: torch.Tensor | None = None,
    prefix_embeds: torch.Tensor | None = None,
    attention_mask: torch.Tensor | None = None,
    reduction: str = 'mean',
    checkpoint: bool = False,
) -> torch.Tensor
```

Defined in `interlens.interp.grad` · source

The grad-enabled analog of `train_to_steer/rl_elicit.py::cont_logprob` (which is `@torch.no_grad` and returns a float). Supply the prefix as `prefix_ids` (`[batch, P]` long) OR `prefix_embeds` (`[batch, P, d_model]` float — e.g. a learnable soft prompt, or soft embeddings bridged from model A). `target_ids` is `[batch, T]` long (or `[T]` broadcast to batch 1). The prefix and target are embedded via `model.get_input_embeddings()` and run in one forward; the returned scalar (or per-example vector, see `reduction`) is the logprob the model assigns to the exact target tokens, still attached to the graph.

`reduction`: `"mean"` (mean over target tokens, mean over batch -> scalar; the reward used by rl_elicit), `"sum"` (sum over target tokens, mean over batch), or `"none"` (`[batch, T]` per-token logprobs). Use `"mean"` as a length-normalized elicitation reward, `"sum"` when total sequence likelihood matters, `"none"` for custom weighting. `checkpoint` is forwarded to save memory when backpropagating through a large frozen B.

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	
`target_ids`	`torch.Tensor`	*required*	

Name	Type	Default	Description
`prefix_ids`	`torch.Tensor | None`	`None`	
`prefix_embeds`	`torch.Tensor | None`	`None`	
`attention_mask`	`torch.Tensor | None`	`None`	
`reduction`	`str`	`'mean'`	
`checkpoint`	`bool`	`False`	

forward_with_grad

One grad-connected forward pass over `input_ids` XOR `inputs_embeds`.

```
forward_with_grad(
    model: 'PreTrainedModel',
    *,
    input_ids: torch.Tensor | None = None,
    inputs_embeds: torch.Tensor | None = None,
    attention_mask: torch.Tensor | None = None,
    capture: GradCaptureSpec | None = None,
    checkpoint: bool = False,
) -> GradForwardOutput
```

Defined in `interlens.interp.grad` · source

Exactly one of `input_ids` (`[batch, seq]` long) or `inputs_embeds` (`[batch, seq, d_model]` float, typically the output of `bridge.soft_embed` or a learnable soft prompt) must be given. `attention_mask` is optional (`[batch, seq]`). Returns `logits` and any requested hidden activations, all still attached to the graph so `.backward()` flows back to `inputs_embeds` / soft-prompt params / an upstream model A.

`checkpoint=True` enables HF gradient checkpointing for this call (`use_reentrant=False`) and restores the prior setting afterwards; it recomputes layer activations on the backward pass to trade compute for memory, and produces identical gradients. Residual sites come from `output_hidden_states` (grad-connected); `attn`/`mlp` sites come from non-detaching forward hooks on the submodules.

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	
`input_ids`	`torch.Tensor | None`	`None`	
`inputs_embeds`	`torch.Tensor | None`	`None`	

Name	Type	Default	Description
`attention_mask`	`torch.Tensor | None`	`None`	
`capture`	GradCapture-Spec | None	`None`	
`checkpoint`	`bool`	`False`	

layers

Module `interlens.interp.layers`

Functions

Name	Summary
`decoder_layers`	Return the list of transformer decoder layer modules, across common HF architectures.

decoder_layers

Return the list of transformer decoder layer modules, across common HF architectures.

```
decoder_layers(model: 'PreTrainedModel')
```

Defined in `interlens.interp.layers` · source

Steering/patching hooks and per-layer capture all need the ordered layer stack. Qwen and Gemma both expose it at `model.model.layers`; a few fallbacks cover other families so the interp layer isn't Qwen/Gemma-only. Multimodal image-text-to-text wrappers (Qwen 3.5, Gemma 4, …) nest the text decoder one level deeper — its layers live at `model.language_model.layers` (or `.model.language_model.model.layers`) — so those paths are included too. PEFT/adapter wrappers (`PeftModelForCausalLM`) are unwrapped first so hooks land on the real decoder layers.

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	

logprobs

Module `interlens.interp.logprobs`

Functions

Name	Summary
`token_logprobs`	Compute per-token logprobs / surprisal / entropy for a generation.

token_logprobs

Compute per-token logprobs / surprisal / entropy for a generation.

```
token_logprobs(scores: tuple[torch.Tensor, ...], generated_ids:
    torch.Tensor) -> dict
```

Defined in `interlens.interp.logprobs` · source

`scores` is the tuple of per-step logit tensors from `model.generate(..., output_scores=True, return_dict_in_generate=True)` (one `[vocab]` per generated token); `generated_ids` are the sampled token ids. Returns lists suitable to drop into `Message.metadata` — scalar-per-token, so they stay small and don't violate the "no heavy tensors in metadata" invariant.

Surprisal = -logprob (nats); entropy is the full next-token distribution entropy at each step (a readout of the model's uncertainty, distinct from the surprisal of the token it actually emitted).

Parameters

Name	Type	Default	Description
`scores`	`tuple[torch.Tensor, ...]`	*required*	
`generated_ids`	`torch.Tensor`	*required*	

patching

Module `interlens.interp.patching`

Classes

Name	Summary
`Patch`	Activation patching: overwrite a decoder layer's residual at specific token `positions` with saved `activations` (captured from another run/branch).

Patch

Activation patching: overwrite a decoder layer's residual at specific token `positions` with saved `activations` (captured from another run/branch).

```
Patch(activations: torch.Tensor, layer: int, positions: tuple[int, ...])
```

Defined in `interlens.interp.patching` · source

This is the cross-branch causal-tracing primitive: capture activations at turn N in one branch (via `ActivationCache`), then inject them at the aligned positions of another branch's forward. The harness owns this because only it knows the turn/position correspondence between branches.

P2 applies the patch on the (single) prompt forward — positions index into the prompt sequence. Aligning positions across branches is the caller's responsibility; `Patch` just performs the overwrite.

Parameters

Name	Type	Default	Description
`activations`	`torch.Tensor`	*required*	
`layer`	`int`	*required*	
`positions`	`tuple[int, ...]`	*required*	

Attributes

Name	Type	Summary
`activations`	`torch.Tensor`	
`layer`	`int`	
`positions`	`tuple[int, ...]`	

Methods

`register`

```
register(self, model: 'PreTrainedModel') -> list
```

source

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	

pooling

Module `interlens.interp.pooling`

Pool a token-position axis down to one vector per span — the primitive under every span-level readout.

`span_pooled_residuals` pools one span per *message*; a theory-of-mind probe instead pools one vector per *speaker* (many disjoint, non-contiguous spans per group) out of a single long forward pass, and a probe over prefix-shared conversation turns pools many overlapping span sets out of that same pass. Both want the same arithmetic, so it lives here once, as plain tensor math with no model, tokenizer, or conversation in sight.

Two entry points:

- :func:`span_pool` — one vector per `(start, end)` span, contiguous, the `span_pooled_residuals` shape.
- :func:`group_pool` — one vector per *group of spans*, for "everything this speaker said" reads where a group's spans are scattered through the sequence. `mean` is taken over the group's pooled tokens (token-weighted, so a long message counts more than a short one), `last` takes the final token of the group's last span.

Both are autograd-transparent (no `detach`, no in-place writes into the input), so they compose with `forward_with_grad` when the pooled vectors feed a trainable head.

Attributes

Name	Type	Summary
`Pool`		
`Span`		

Functions

Name	Summary
`group_pool`	Pool each *group* of spans into one vector; returns `([len(groups), d_model], valid[len(groups)])`.
`span_pool`	Pool `hidden[start:end]` for each span; returns `[len(spans), d_model]`.

group_pool

Pool each *group* of spans into one vector; returns `([len(groups), d_model], valid[len(groups)])`.

```
group_pool(
    hidden: torch.Tensor,
    groups: Sequence[Sequence[Span]],
    mode: Pool = 'mean',
    empty: Literal['zeros', 'raise'] = 'zeros',
) -> tuple[torch.Tensor, torch.Tensor]
```

Defined in `interlens.interp.pooling` · source

The multi-span generalization of :func:`span_pool`: a group is an arbitrary set of (possibly scattered, possibly overlapping) token ranges — e.g. every message one speaker contributed to a long transcript. Under `"mean"` the average is over the union of the group's *token positions*, taken with multiplicity if spans overlap, so a speaker's long message weighs more than a short one; under `"last"` the result is the final token of the group's highest-ending span.

Returns

Name	Type	Description
	`torch.Tensor`	`(pooled, valid)` where `pooled` is `[len(groups), d_model]` and `valid` is a bool tensor that is
	`torch.Tensor`	`False` exactly on rows pooled from an empty group. Always check `valid` before using a row: a zero
	`tuple[torch.Tensor, torch.Tensor]`	row is indistinguishable from a genuine

Name	Type	Description
		zero activation without it.

Parameters

Name	Type	Default	Description
`hidden`	`torch.Tensor`	*required*	`[seq, d_model]` per-token activations.
`groups`	`Sequence[Sequence[Span]]`	*required*	one span list per output row. Groups are what varies across a batch of readouts sharing one forward pass, which is why they are the outer axis.
`mode`	`Pool`	`'mean'`	as in :func:`span_pool`.
`empty`	`Literal['zeros', 'raise']`	`'zeros'`	what an *empty group* means. `"zeros"` (default) emits a zero row and marks it invalid in the returned mask — the right behaviour when a group is legitimately absent (a

Name	Type	Default	Description
			party that has not spoken yet at this point in the dialogue), so callers can mask it downstream instead of dropping the row and losing alignment. `"raise"` treats it as a construction bug.

span_pool

Pool `hidden[start:end]` for each span; returns `[len(spans), d_model]`.

```
span_pool(
    hidden: torch.Tensor,
    spans: Sequence[Span],
    mode: Pool = 'mean',
) -> torch.Tensor
```

Defined in `interlens.interp.pooling` · source

Raises

Exception	Description
`ValueError`	on a non-2D `hidden`, an unknown `mode`, or an empty/out-of-bounds span.

Parameters

Name	Type	Default	Description
`hidden`	`torch.Tensor`	*required*	`[seq, d_model]` per-token activations from one forward pass (any site, any layer).
`spans`	`Sequence[Span]`	*required*	half-open `(start, end)` token ranges. Spans may overlap and need not be sorted, but

Name	Type	Default	Description
			each must be non-empty and within `seq` — an empty span is a bug in the span construction, not a zero vector, so it raises rather than silently contributing nothing.
`mode`	`Pool`	`'mean'`	`"mean"` averages the span's tokens (order-insensitive; the probe default) while `"last"` takes the span's final token (the position that causally sees the whole span, and what a next-token readout actually conditions on).

routing

Module `interlens.interp.routing`

Mixture-of-Experts routing capture and statistics.

Reads *which experts an MoE model routes each token to* — the discrete, cheap-to-interpret counterpart of residual-stream capture. Like `capture_activations`, capture is a single clean forward pass over the full token sequence (provably complete, one extra forward) rather than hooks accumulated across a decode loop: both `OlmoeForCausalLM` and `Qwen3MoeForCausalLM` (and other HF MoE families) return per-MoE-layer router logits natively via `output_router_logits=True`, so no module hooks are needed at all.

Typical use: replay a saved conversation view through the MoE, get per-token routing with `capture_router_logits`, compute per-message expert-usage distributions with `routing_stats` restricted to `message_token_spans`, and compare conditions with `js_divergence` / `topk_expert_overlap`.

Classes

Name	Summary
`RouterSteeringSpec`	A causal intervention on MoE routing: add a per-expert bias to the gate logits during generation, nudging which experts fire.
`RoutingCapture`	Per-token routing at one MoE layer, from one `capture_router_logits` pass.
`RoutingStats`	Aggregate expert-usage distributions over a set of token positions.

Functions

Name	Summary
capture_router_logits	One clean forward pass over input_ids ([1, seq]); return per-MoE-layer RoutingCapture.
js_divergence	Per-layer Jensen–Shannon divergence (symmetric, bounded by ln 2) → [n_layers].
kl_divergence	Per-layer KL(p || q) between expert distributions [n_layers, n_experts] → [n_layers].
message_token_spans	Token span (start, end) of each message of view in the fully-rendered chat-template sequence.
moe_layer_indices	Decoder-layer indices that carry a sparse MoE block.
moe_num_experts	Number of routed experts per MoE layer (config.num_experts — same field name in OLMoE/Qwen-MoE).
moe_topk	Experts selected per token (config.num_experts_per_tok).
routing_stats	Pool per-token routing into per-layer expert-usage distributions.
topk_expert_overlap	Per-layer fraction of overlap between the k most-used experts of p and of q → [n_layers].

RouterSteeringSpec

A causal intervention on MoE routing: add a per-expert bias to the gate logits during generation, nudging which experts fire.

```
RouterSteeringSpec(bias: torch.Tensor, layers: tuple[int, ...], coef:
    float = 1.0)
```

Defined in `interlens.interp.routing` · source

The routing analogue of :class:`SteeringSpec` (which steers the residual stream) — here forward hooks sit on each sparse layer's `mlp.gate` (the router `nn.Linear` whose output is the `[tokens, n_experts]` logits) and add `coef * bias` before top-k selection.

Build it with :meth:`toward_load` to push routing *toward* a target expert-usage distribution (e.g. the MoE's solo-on-domain `RoutingStats.expert_load`): the bias is `log(target + eps)`, so adding it acts like a log-prior that shifts the router's softmax toward the target experts, with `coef` the strength (0 = no-op).

A summary (layers, coef, per-layer bias norm) is available via :meth:`summary` for reproducibility.

Parameters

Name	Type	Default	Description
`bias`	`torch.Tensor`	*required*	
`layers`	`tuple[int, ...]`	*required*	
`coef`	`float`	`1.0`	

Attributes

Name	Type	Summary
`bias`	`torch.Tensor`	
`coef`	`float`	

Name	Type	Summary
layers	tuple[int, ...]	

Methods

register

```
register(self, model: 'PreTrainedModel') -> list
```

source

Register the gate-bias hooks on `model`'s sparse layers; returns handles (caller removes after use).

Parameters

Name	Type	Default	Description
model	'PreTrainedModel'	*required*	

summary

```
summary(self) -> dict
```

source

toward_load

```
toward_load(
    cls,
    target_load: torch.Tensor,
    layers: tuple[int, ...],
    coef: float = 1.0,
    eps: float = 1e-06,
) -> 'RouterSteeringSpec'
```

source

Steer toward a target expert-usage distribution `target_load` (`[len(layers), n_experts]`, rows the fraction of routing mass per expert).

Bias `= log(target_load + eps)` (a log-prior on expert choice).

Parameters

Name	Type	Default	Description
`target_load`	`torch.Tensor`	*required*	
`layers`	`tuple[int, ...]`	*required*	
`coef`	`float`	`1.0`	
`eps`	`float`	`1e-06`	

RoutingCapture

Per-token routing at one MoE layer, from one `capture_router_logits` pass.

```
RoutingCapture()
```

Defined in `interlens.interp.routing` · source

Inherits from: `NamedTuple`

`router_logits` is the raw pre-softmax gate output `[seq, n_experts]` (cpu fp32), or `None` when the capture was compacted with `top_k_only=True`. `topk_experts` / `topk_probs` are always present: the `k` selected expert ids (int16) and their softmax router probabilities (fp16), `[seq, k]` each.

Attributes

Name	Type	Summary
`layer`	`int`	
`router_logits`	`torch.Tensor | None`	
`topk_experts`	`torch.Tensor`	
`topk_probs`	`torch.Tensor`	

RoutingStats

Aggregate expert-usage distributions over a set of token positions.

```
RoutingStats(
    expert_load: torch.Tensor,
    expert_mass: torch.Tensor | None,
    layers: tuple[int, ...],
    n_tokens: int,
    top_k: int,
)
```

Defined in `interlens.interp.routing` · source

`expert_load[l, e]` is the fraction of top-k selections at layer `l` that went to expert `e` (rows sum to 1 — the discrete "which experts fired" histogram). `expert_mass[l, e]` is the mean softmax router probability (`None` if the captures were `top_k_only` — full logits are needed for mass). `layers` are the decoder-layer indices of the rows; `n_tokens` is how many positions were pooled.

Parameters

Name	Type	Default	Description
`expert_load`	`torch.Tensor`	*required*	
`expert_mass`	`torch.Tensor | None`	*required*	
`layers`	`tuple[int, ...]`	*required*	
`n_tokens`	`int`	*required*	
`top_k`	`int`	*required*	

Attributes

Name	Type	Summary
`expert_load`	`torch.Tensor`	
`expert_mass`	`torch.Tensor | None`	
`layers`	`tuple[int, ...]`	

Name	Type	Summary
`n_tokens`	`int`	
`top_k`	`int`	

capture_router_logits

One clean forward pass over `input_ids` (`[1, seq]`); return per-MoE-layer `RoutingCapture`.

```
capture_router_logits(
    model: 'PreTrainedModel',
    input_ids: torch.Tensor,
    layers: tuple[int, ...] | None = None,
    top_k_only: bool = False,
    offload: str = 'cpu',
) -> list[RoutingCapture]
```

Defined in `interlens.interp.routing` · source

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	an HF MoE causal LM whose `forward` accepts `output_router_logits=True` (OLMoE, Qwen-MoE, Mixtral, …).
`input_ids`	`torch.Tensor`	*required*	full token sequence to route, shape `[1, seq]` (batch of 1 — replay one view at a time).
`layers`	`tuple[int, ...] | None`	`None`	decoder-layer indices to keep (default: all sparse layers, per `moe_layer_indices`).

Name	Type	Default	Description
`top_k_only`	`bool`	`False`	drop the full `[seq, n_experts]` logits and keep only top-k ids/probs (int16/fp16, ~8x smaller — use for long-sequence sweeps over large MoEs like Qwen3-30B-A3B).
`offload`	`str`	`'cpu'`	device for the returned tensors (default `"cpu"` so GPU memory is freed immediately).

js_divergence

Per-layer Jensen–Shannon divergence (symmetric, bounded by ln 2) → `[n_layers]`.

```
js_divergence(p: torch.Tensor, q: torch.Tensor) -> torch.Tensor
```

Defined in `interlens.interp.routing` · source

Parameters

Name	Type	Default	Description
p	torch.Tensor	*required*	
q	torch.Tensor	*required*	

kl_divergence

Per-layer KL(p || q) between expert distributions `[n_layers, n_experts]` → `[n_layers]`.

```
kl_divergence(p: torch.Tensor, q: torch.Tensor, eps: float = 1e-08) ->
    torch.Tensor
```

Defined in `interlens.interp.routing` · source

Parameters

Name	Type	Default	Description
p	torch.Tensor	*required*	
q	torch.Tensor	*required*	
eps	float	1e-08	

message_token_spans

Token span `(start, end)` of each message of `view` in the fully-rendered chat-template sequence.

```
message_token_spans(
    tokenizer: 'PreTrainedTokenizerBase',
    view: list[dict],
) -> list[tuple[int, int]]
```

Defined in `interlens.interp.routing` · source

Method: render the *string* prefixes `apply_chat_template(view[:i], tokenize=False)` for each `i` and require each to be a string-prefix of the next (raised on non-prefix-stable templates). The full string is then tokenized **once** with `return_offsets_mapping=True` and each char boundary is mapped to the first token whose offset starts at/after it — prefix *strings* are never tokenized independently, because a tokenization of a prefix string is not in general a prefix of the full tokenization.

Note this describes the final replayed view (the whole conversation templated once). Live generation builds the sequence incrementally, but for standard chat templates the rendered text is identical, so spans match. The returned spans cover each message's rendered chunk *including* its role header/footer markup.

Parameters

Name	Type	Default	Description
`tokenizer`	`'PreTrainedTokenizerBase'`	*required*	
`view`	`list[dict]`	*required*	

moe_layer_indices

Decoder-layer indices that carry a sparse MoE block.

```
moe_layer_indices(model: 'PreTrainedModel') -> tuple[int, ...]
```

Defined in `interlens.interp.routing` · source

HF MoE models return `router_logits` only for the *sparse* layers, in layer order, with no index attached. This maps that tuple back to real decoder-layer indices by checking each layer's `mlp` for a router `gate` submodule — which handles mixed stacks (e.g. Qwen-MoE `decoder_sparse_step` / `mlp_only_layers` leaving some layers dense) as well as fully-sparse stacks like OLMoE.

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	

moe_num_experts

Number of routed experts per MoE layer (`config.num_experts` — same field name in OLMoE/Qwen-MoE).

```
moe_num_experts(model: 'PreTrainedModel') -> int
```

Defined in `interlens.interp.routing` · source

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	

moe_topk

Experts selected per token (`config.num_experts_per_tok`).

```
moe_topk(model: 'PreTrainedModel') -> int
```

Defined in `interlens.interp.routing` · source

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	

routing_stats

Pool per-token routing into per-layer expert-usage distributions.

```
routing_stats(
    captures: list[RoutingCapture],
    n_experts: int,
    spans: list[tuple[int, int]] | None = None,
) -> RoutingStats
```

Defined in `interlens.interp.routing` · source

Parameters

Name	Type	Default	Description
`captures`	list[RoutingCapture]	*required*	output of `capture_router_logits` (all layers share one token sequence).
`n_experts`	`int`	*required*	total routed experts (`moe_num_experts(model)`) — needed to size the histogram since a span may never touch some experts.
`spans`	`list[tuple[int, int]] | None`	`None`	optional `[(start, end), ...]` token windows to restrict to (e.g. only the MoE's own generated messages, from `message_token_spans`).

Name	Type	Default	Description
			Default: all positions.

topk_expert_overlap

Per-layer fraction of overlap between the `k` most-used experts of `p` and of `q` → `[n_layers]`.

```
topk_expert_overlap(p: torch.Tensor, q: torch.Tensor, k: int = 8) ->
    torch.Tensor
```

Defined in `interlens.interp.routing` · source

Parameters

Name	Type	Default	Description
p	torch.Tensor	*required*	
q	torch.Tensor	*required*	
k	int	8	

softtokens

Module `interlens.interp.softtokens`

Virtual (soft) tokens inside ordinary text prompts, plus the message-span read path that pairs with them.

Two utilities that together make a *continuous* channel into an otherwise text-only chat harness:

- :class:`VirtualTokenInjector` — write soft tokens into a prompt **without** an `inputs_embeds` code path. The harness's batched generation (`ModelParticipant.generate_batch`) chat-templates strings and calls `model.generate(input_ids=...)`; it cannot accept embeddings. So the injector hands you a *placeholder text snippet* (a run of one existing rare vocab token, repeated `n_soft` times — no new tokens, no embedding resize) to render into the prompt like any other text, and a context manager that swaps the embeddings at exactly those positions for caller-supplied vectors. The swap is a forward-pre-hook (to see `input_ids`) plus a forward hook (to edit the embedding output) on `model.get_input_embeddings()`, so it works identically under `model.generate` (placeholders live in the prefill only; cached decode steps pass through untouched) and under a plain `model(...)` training forward, and it is autograd-transparent: gradients flow back into the vectors, so the vectors can come from a trainable :class:`~interlens.interp.bridge.LinearBridge`.
- :func:`span_pooled_residuals` — the matching read path: one pooled residual vector *per message* of a rendered conversation, by combining `capture_activations` / `forward_with_grad` with `routing.message_token_spans` and `pooling.span_pool` (which owns the pooling arithmetic). This is what a theory-of-mind probe head reads.

Worked example (bridge a partner's hidden state into a listener's prompt as 4 soft tokens):

```
from interlens.interp import VirtualTokenInjector, span_pooled_residuals,
     LinearBridge
inj = VirtualTokenInjector(tok_b, n_soft=4)
pooled = span_pooled_residuals(model_a, tok_a, view, layers=(-1,),
     grad=True)   # [n_messages, d_a]
vectors = bridge(pooled[("residual", -1)][-1:].unsqueeze(0))
     # [1, 4, d_b] (after a reshape)
prompt = f"Partner state: {inj.text}
```

What do they want?“ with inj.inject(model_b, vectors): out = model_b.generate(**tok_b(prompt, return_tensors=”pt“), max_new_tokens=16)

Classes

Name	Summary
`VirtualTokenInjector`	Reserve `n_soft` positions in a *text* prompt and substitute their input embeddings at forward time.

Functions

Name	Summary
`span_pooled_residuals`	One pooled activation vector **per message** of `view`, at each requested site/layer.

VirtualTokenInjector

Reserve `n_soft` positions in a *text* prompt and substitute their input embeddings at forward time.

```
VirtualTokenInjector(
    tokenizer: 'PreTrainedTokenizerBase',
    n_soft: int,
    placeholder: str | None = None,
)
```

Defined in `interlens.interp.softtokens` · source

Construction picks (and validates) a `placeholder` token from `tokenizer`'s existing vocabulary — no `add_special_tokens`, no `resize_token_embeddings`, so the model is untouched and any checkpoint stays loadable. Validation is empirical, not assumed: the candidate must (a) tokenize, repeated `n_soft` times with no separator, to exactly `n_soft` copies of its own id, and (b) survive `apply_chat_template` — rendered inside a user message the run must still appear exactly once, contiguous, at length `n_soft`. Templates that escape or normalize the candidate are rejected and the next candidate is tried.

Use :attr:`text` as the snippet to splice into your prompt string, then wrap the forward/generate call in :meth:`inject` (or register/remove hooks yourself with :meth:`register`, the `SteeringSpec.register` convention).

Parameters

Name	Type	Default	Description
`tokenizer`	`'PreTrainedTokenizerBase'`	*required*	
`n_soft`	`int`	*required*	
`placeholder`	`str | None`	`None`	

Attributes

Name	Type	Summary
`n_soft`		
`placeholder`		
`text`	`str`	The snippet to render into a prompt: the placeholder token repeated `n_soft` times, no separators.
`token_id`		
`tokenizer`		

Methods

`inject`

```
inject(
    self,
    model: 'PreTrainedModel',
    vectors: torch.Tensor,
) -> Iterator['VirtualTokenInjector']
```

source

Context manager around :meth:`register`: hooks are removed on exit, including on exception.

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	
`vectors`	`torch.Tensor`	*required*	

register

```
register(self, model: 'PreTrainedModel', vectors: torch.Tensor) -> list
```

source

Register the substitution hooks on `model` and return the handles (caller removes them; or use :meth:`inject`, which does that for you).

`vectors` is `[batch, n_soft, d_model]`, aligned row-for-row with the `input_ids` the model will see; a `batch` of 1 is broadcast to every row. It is *not* detached and *not* copied into the graph in place, so `loss.backward()` reaches it (and anything upstream, e.g. a `LinearBridge`). Rows whose ids contain no placeholder keep their ordinary token embeddings.

Mechanics: a forward-**pre**-hook on `model.get_input_embeddings()` records the `input_ids` that call is about to embed (the post-hook only sees the output), and the forward hook returns a modified clone with the placeholder slice overwritten. Decode steps under a KV cache pass one non-placeholder token and are no-ops.

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	
`vectors`	`torch.Tensor`	*required*	

run_starts

```
run_starts(self, input_ids: torch.Tensor) -> list[int | None]
```

source

Per batch row, the index where the placeholder run starts, or `None` if that row has no placeholders.

Raises if a row holds a partial or non-contiguous run — that means the prompt was built wrong (e.g. the snippet got split by a template) and silently steering the wrong positions would be worse than failing.

Parameters

Name	Type	Default	Description
`input_ids`	`torch.Tensor`	*required*	

span_pooled_residuals

One pooled activation vector **per message** of `view`, at each requested site/layer.

```
span_pooled_residuals(
    model: 'PreTrainedModel',
    tokenizer: 'PreTrainedTokenizerBase',
    view: list[dict],
    *,
    layers: int | Iterable[int] | None,
    sites: tuple[Site, ...] = ('residual',),
    pool: Pool = 'mean',
    grad: bool = False,
) -> dict[tuple[Site, int], torch.Tensor]
```

Defined in `interlens.interp.softtokens` · source

Renders `view` once with the chat template, locates each message's token span with `routing.message_token_spans` (prefix-diffing the rendered strings, so spans include each message's role markup), runs a single forward pass, and pools each span down to one vector. Returns `{(site, layer): tensor[n_messages, d_model]}` — keyed like `forward_with_grad`'s `hidden`.

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	the model to read; must accept `input_ids` (no generation happens here).
`tokenizer`	`'PreTrainedTokenizerBase'`	*required*	supplies the chat template used for both the render and the spans.

Name	Type	Default	Description
`view`	`list[dict]`	*required*	the conversation as chat-template dicts (`[{"role": ..., "content": ...}, ...]`).
`layers`	`int | Iterable[int] | None`	*required*	decoder-layer index, iterable of indices, or `None` for every layer. Negative indices count from the end (`-1` = last layer), so `layers=(-1,)` is "the usual readout layer" without knowing depth.
`sites`	`tuple[Site, ...]`	`('residual',)`	any of `"residual"`/`"attn"`/`"mlp"`. `"residual"` is the probe default; the sublayer sites are there when you want to attribute a readout to attention vs MLP.
`pool`	`Pool`	`'mean'`	`"mean"` or `"last"`, forwarded to :func:`~interlens.interp.pooli` — mean averages the span's tokens (stable, order-insensi-

Name	Type	Default	Description
			tive; the probe default), last takes the span's final token (the position that causally sees the whole message, and what a next-token readout conditions on).
`grad`	`bool`	`False`	`False` runs the detached `capture_activations` pass (cheap, inference-mode) — right for fitting a probe on frozen features. `True` routes through `forward_with_grad` so the pooled vectors stay in the autograd graph — required when they feed a trainable bridge/soft prompt that is optimized end to end.

steering

Module `interlens.interp.steering`

Attributes

Name	Type	Summary
`Mode`		

Classes

Name	Summary
`SteeringSpec`	A residual-stream intervention applied during generation via forward hooks on decoder layers.

SteeringSpec

A residual-stream intervention applied during generation via forward hooks on decoder layers.

```
SteeringSpec(
    direction: torch.Tensor,
    layers: tuple[int, ...],
    coef: float = 1.0,
    mode: Mode = 'add',
)
```

Defined in `interlens.interp.steering` · source

`mode='add'` adds `coef * direction` to the residual at `layers`; `mode='ablate'` projects the `direction` component *out* of the residual (directional ablation). The same mechanism covers both because ablation is just the projection-removal variant of an additive hook.

A summary (mode, layers, coef, direction norm) is recorded into `Message.metadata['steering']` by the participant so a steered/ablated turn is reproducible.

Parameters

Name	Type	Default	Description
`direction`	`torch.Tensor`	*required*	
`layers`	`tuple[int, ...]`	*required*	
`coef`	`float`	`1.0`	
`mode`	`Mode`	`'add'`	

Attributes

Name	Type	Summary
`coef`	`float`	
`direction`	`torch.Tensor`	

Name	Type	Summary
layers	tuple[int, ...]	
mode	Mode	

Methods

difference_of_means

```
difference_of_means(
    cls,
    pos_acts,
    neg_acts,
    layers,
    coef: float = 1.0,
    mode: Mode = 'add',
) -> 'SteeringSpec'
```

source

Build a spec whose `direction` is the **unit difference-of-means** of two activation populations, `normalize(mean(pos) - mean(neg))` — the classic contrastive/concept-direction recipe, pointing TOWARD the positive class.

`pos_acts`/`neg_acts` are `[n, d_model]` (a stack of per-example residuals) or a pre-pooled `[d_model]` vector; anything `torch.as_tensor` accepts works. Steer toward the positive concept with `coef>0` and **away** from it (suppress) with `coef<0`. `layers` is an int or an iterable of decoder-layer indices. This lives here (not re-hand-rolled per experiment) per the contribution convention.

Parameters

Name	Type	Default	Description
pos_acts		*required*	
neg_acts		*required*	
layers		*required*	
coef	float	1.0	

Name	Type	Default	Description
`mode`	`Mode`	`'add'`	

register

```
register(self, model: 'PreTrainedModel') -> list
```

source

Register the steering hooks on `model` and return the handles (caller removes them after generate).

Parameters

Name	Type	Default	Description
`model`	`'PreTrainedModel'`	*required*	

summary

```
summary(self) -> dict
```

source

loading

Package `interlens.loading`

Modules

- `devices` — Where to put a model's *inputs*, which is not the same question as "what device is the model on".
- `load`
- `model_cache`

Re-exported

Name	Defined in	Summary
`derive_chat_flags`	`interlens.loading.load`	Probe a tokenizer's chat template to derive `(supports_system_role, requires_alternating_roles)`
`device_map_kind`	`interlens.loading.devices`	Interpret a `device` argument as a `device_map` when it names a sharding strategy, else `None`.
`free`	`interlens.loading.model_cache`	Drop both caches and reclaim GPU memory.
`input_device`	`interlens.loading.devices`	The device to place `input_ids` on for `model`: its input embedding's device.
`is_sharded`	`interlens.loading.devices`	True if `model` was placed by `device_map=` across more than one

Name	Defined in	Summary
		device (including cpu/ disk offload).
`load_model`	`interlens.loading.load`	Load a causal LM + tokenizer, sharing through the process-local caches.
`load_tokenizer`	`interlens.loading.load`	Load a tokenizer for `hf_id` (or a local path), defaulting `pad_token` to `eos_token` when absent — the single source of the pad-token convention, shared by `load_model` and `AutoModelParticipant` when it has to infer a tokenizer from a bare model.

devices

Module `interlens.loading.devices`

Where to put a model's *inputs*, which is not the same question as "what device is the model on".

A single-device model answers both with one device, so `model.device` and `next(model.parameters()).device` are interchangeable and every call site historically used whichever was shorter. A model sharded across GPUs by `device_map=` answers them differently: its parameters live on several devices, and `input_ids` must land on whichever one holds the **input embedding**, because that is the first module the tensor meets. Accelerate's hooks move activations *between* shards for you, but they do not move the tensor you hand to `forward`.

Hence :func:`input_device` — one helper, used by every site that places `input_ids`, so single-device behavior is bit-identical to before and sharded behavior is correct.

Functions

Name	Summary
`device_map_kind`	Interpret a `device` argument as a `device_map` when it names a sharding strategy, else `None`.
`input_device`	The device to place `input_ids` on for `model`: its input embedding's device.
`is_sharded`	True if `model` was placed by `device_map=` across more than one device (including cpu/disk offload).

device_map_kind

Interpret a `device` argument as a `device_map` when it names a sharding strategy, else `None`.

```
device_map_kind(device) -> str | dict | None
```

Defined in `interlens.loading.devices` · source

Accepts the strings transformers understands (`"auto"`, `"balanced"`, `"balanced_low_0"`, `"sequential"`) and an explicit `{module: device}` dict. Anything else — `"cuda"`, `"cuda:1"`, `"cpu"`, a `torch.device` — is an ordinary single-device placement and returns `None`, which is what keeps the default load path unchanged.

Parameters

Name	Type	Default	Description
`device`		*required*	

input_device

The device to place `input_ids` on for `model`: its input embedding's device.

```
input_device(model) -> torch.device
```

Defined in `interlens.loading.devices` · source

Resolution order, each step falling through on failure so this never raises on an exotic model:

1. `model.get_input_embeddings()`'s first parameter/buffer — correct under any `device_map`, and equal to the model's single device when there is no sharding.
2. `model.device` — the historical answer, kept for models that expose no input embedding (mocks, some wrappers).
3. the first parameter's device.

Returns a `torch.device` (never a string), so callers can compare and `.to()` it directly.

Parameters

Name	Type	Default	Description
`model`		*required*	

is_sharded

True if `model` was placed by `device_map=` across more than one device (including cpu/disk offload).

```
is_sharded(model) -> bool
```

Defined in `interlens.loading.devices` · source

Reads accelerate's `hf_device_map`, which `from_pretrained(device_map=...)` stamps on the model. A single-entry map (everything on one device) reads as NOT sharded, because in that case `model.device` is already the right answer everywhere.

Parameters

Name	Type	Default	Description
`model`		*required*	

load

Module `interlens.loading.load`

Functions

Name	Summary
`derive_chat_flags`	Probe a tokenizer's chat template to derive `(supports_system_role, requires_alternating_roles)`.
`load_model`	Load a causal LM + tokenizer, sharing through the process-local caches.
`load_tokenizer`	Load a tokenizer for `hf_id` (or a local path), defaulting `pad_token` to `eos_token` when absent — the single source of the pad-token convention, shared by `load_model` and `AutoModelParticipant` when it has to infer a tokenizer from a bare model.

derive_chat_flags

Probe a tokenizer's chat template to derive `(supports_system_role, requires_alternating_roles)`.

```
derive_chat_flags(tokenizer) -> tuple[bool, bool]
```

Defined in `interlens.loading.load` · source

`supports_system_role` is True iff the template renders a leading `system` turn without raising; `requires_alternating_roles` is True iff the template rejects two consecutive same-role turns. Each probe is wrapped in try/except so a raising template simply reads as the corresponding boolean. This replaces per-family flag declarations: an unknown model gets correct chat behavior with zero configuration.

Parameters

Name	Type	Default	Description
`tokenizer`		*required*	

load_model

Load a causal LM + tokenizer, sharing through the process-local caches.

```
load_model(
    id_or_path: str | Path,
    device: str | torch.device = 'cuda',
    dtype: torch.dtype = torch.bfloat16,
    attn: str = 'flash_attention_2',
    quant: str | None = None,
    revision: str | None = None,
    max_memory: dict | None = None,
)
```

Defined in `interlens.loading.load` · source

`id_or_path` is the HF id or a local path to load directly (a `Path` is normalized to `str` so it shares the same cache slot as its string form). Identical (hf_id, device, dtype, attn, quant, revision, max_memory) pairings share the one model object, and the tokenizer is cached by hf_id. Flash-attention is the default with automatic fallback to sdpa/eager; quantization is opt-in.

Returns

Name	Type	Description
		`(model, tokenizer)`. On a sharded load, place inputs with func:`~interlens.loading.devices.input_device` rather than `model.device`.

Example

```
model, tok = load_model("Qwen/Qwen3-8B")
# single GPU, bf16, flash-attn
model, tok = load_model("Qwen/Qwen3-32B", device="balanced",  # two cards, KV headroom reserved
```

… max_memory={0: “70GiB”, 1: “70GiB”, “cpu”: “0GiB”})

Parameters

Name	Type	Default	Description
`id_or_path`	`str | Path`	*required*	HF hub id or a local directory of weights.
`device`	`str | torch.device`	`'cuda'`	either a single placement (`"cuda"`, `"cuda:1"`, `"cpu"`, a `torch.device` — the default, loaded then moved) or a **sharding strategy** handed to `device_map=`: `"auto"` (fill each visible GPU in turn, then cpu/disk), `"balanced"` (even split over GPUs), `"balanced_low_0"` (leave headroom on GPU 0 for generation buffers), `"sequential"`, or an explicit `{module_name:`

Name	Type	Default	Description
			`device}` map. Use a strategy when the weights do not fit one card — e.g. a 32B policy on two 80 GB GPUs; use the single-device default whenever they do, since sharding adds cross-device transfers to every forward.
`dtype`	`torch.dtype`	`torch.bfloat16`	parameter dtype; `bfloat16` is the default and what every evaluation in this library assumes.
`attn`	`str`	`'flash_attention_2'`	preferred attention backend, tried first and then degraded through `sdpa` and `eager`, so `flash_attention_2` is safe to leave on hardware/

Name	Type	Default	Description
			builds that lack it.
`quant`	`str | None`	`None`	`None` (full precision — required for faithful interpretability, since quantization perturbs the very activations a probe reads), `"4bit"` or `"8bit"` via BitsAndBytes when memory forces it. Quantized loads place themselves and are never `.to()`'d.
`revision`	`str | None`	`None`	pin a specific hub commit/branch/tag; `None` takes the default branch.
`max_memory`	`dict | None`	`None`	per-device budget for a `device_map` strategy, e.g. `{0: "70GiB", 1: "70GiB", "cpu": "0GiB"}`. Use it to reserve head-

Name	Type	Default	Description
			room for KV cache and activations, which the automatic map does not know about, or to forbid cpu offload (an `"0GiB"` cpu entry turns a silent 100x slowdown into an out-of-memory error you can act on). Ignored without a strategy.

load_tokenizer

Load a tokenizer for `hf_id` (or a local path), defaulting `pad_token` to `eos_token` when absent — the single source of the pad-token convention, shared by `load_model` and `AutoModelParticipant` when it has to infer a tokenizer from a bare model.

```
load_tokenizer(hf_id: str, revision: str | None = None) ->
    PreTrainedTokenizerBase
```

Defined in `interlens.loading.load` · source

Parameters

Name	Type	Default	Description
`hf_id`	`str`	*required*	
`revision`	`str | None`	`None`	

model_cache

Module `interlens.loading.model_cache`

Functions

Name	Summary
`cached_model`	Return the cached model for `key`, running `loader()` at most once even under concurrent callers.
`cached_tokenizer`	Return the cached tokenizer for `tok_id`, running `loader()` at most once even under concurrent callers (same double-checked per-key locking as :func:`cached_model`).
`free`	Drop both caches and reclaim GPU memory.

cached_model

Return the cached model for `key`, running `loader()` at most once even under concurrent callers.

```
cached_model(key: tuple, loader)
```

Defined in `interlens.loading.model_cache` · source

Double-checked locking: the fast path is a lock-free `dict.get` (atomic under the GIL) once the model is cached; a miss takes the per-key lock and re-checks, so the loader runs exactly once and every caller gets the same fully-materialized object. The key is published only AFTER `loader()` returns, so no thread ever observes a partially-`.to()`'d model.

Parameters

Name	Type	Default	Description
`key`	`tuple`	*required*	
`loader`		*required*	

cached_tokenizer

Return the cached tokenizer for `tok_id`, running `loader()` at most once even under concurrent callers (same double-checked per-key locking as :func:`cached_model`).

```
cached_tokenizer(tok_id: str, loader)
```

Defined in `interlens.loading.model_cache` · source

Parameters

Name	Type	Default	Description
`tok_id`	`str`	*required*	
`loader`		*required*	

free

Drop both caches and reclaim GPU memory.

```
free() -> None
```

Defined in `interlens.loading.model_cache` · source

Useful between phases that load many models in one process.

message

Module `interlens.message`

Classes

Name	Summary
`Message`	A single committed turn in a conversation.

Message

A single committed turn in a conversation.

```
Message(author: str, content: str, metadata: dict = dict())
```

Defined in `interlens.message` · source

The transcript is stored canonically and author-centric: a message records *who* said *what*, and is deliberately agnostic to the `assistant`/`user` role distinction. That role mapping is a per-participant *view* concern (see `Transcript.render_roles`), not a property of the message itself — so one transcript can be rendered from every participant's perspective without duplicating state.

`author` is the participant's `name` (a string), never the `Participant` object. This keeps transcripts trivially JSON-serializable and valid even when no models are loaded (e.g. when scoring saved transcripts).

`content` is the committed, visible text — the only field that is authoritative for rendering history back into a model. Anything else a generation produced (parsed `<think>` reasoning, the raw completion, tool call/result trails, per-token logprobs) lives in `metadata` under neutral keys, so hidden generated text is never silently promoted into what other participants see.

Parameters

Name	Type	Default	Description
`author`	`str`	*required*	
`content`	`str`	*required*	
`metadata`	`dict`	`dict()`	

Attributes

Name	Type	Summary
`author`	`str`	
`content`	`str`	
`metadata`	`dict`	

parsing

Module `interlens.parsing`

One home for structured-action parsing and reasoning stripping.

Three call sites historically each carried their own copy of this logic — the arena's `extract_json` / `strip_think` (`arena/views.py`), the messaging policy's `parse_json_actions` (`communication/messaging.py`), and the model participant's `parse_tool_calls` / `split_reasoning` (`model_participant.py`) — with subtly divergent regexes and edge-case handling. They now all call the one implementation here; the original names survive as thin shims so their signatures stay stable.

Two orthogonal jobs:

- **Fenced / tagged JSON extraction** — pulling structured actions out of a model's free text, whether they arrive as ```` ```json {…} ```` fences (the family-agnostic action surface) or `<tag>{…}</tag>` blocks (Hermes/Qwen tool calls). Malformed JSON is skipped, never fatal.
- **Reasoning stripping** — separating a `<think>…</think>` stream from the visible content, in two flavours: a strict *leading-block* split (the participant's own completion, where reasoning is a prefix) and a *strip-anywhere* pass (a defensive re-strip before content reaches another seat's view, robust to a generation truncated mid-`<think>`).

Attributes

Name	Type	Summary
`FENCE_RE`		
`LEADING_THINK_RE`		
`THINK_BLOCK_RE`		

Functions

Name	Summary
first_think_block	The first complete <think>…</think> block's inner reasoning, or None.
iter_fenced_json	Every parseable fenced JSON OBJECT in text, in order (malformed fences skipped, not fatal).
iter_tagged_json	Every <tag>{json}</tag> block in text as (parsed_object, raw_match) pairs (malformed JSON skipped).
last_json	The LAST fenced JSON object in text, else the last balanced top-level {…} that parses, else None. "Last wins" because a model's final fenced block is its committed action when it revised mid-turn.
last_json_with_key	The LAST fenced JSON OBJECT in text carrying at least one of keys at top level, else None.
split_leading_think	Split a raw completion into (visible_content, parsed_think) on a LEADING <think>…</think> block only (the participant-level convention).
strip_think	Remove <think>…</think> blocks ANYWHERE in text; return (visible, think).

first_think_block

The first complete `<think>…</think>` block's inner reasoning, or `None`.

```
first_think_block(text: str | None) -> str | None
```

Defined in `interlens.parsing` · source

A detection helper for leak gates that need the fragment itself (`arena.gates.check_reasoning_leak`).

Parameters

Name	Type	Default	Description
`text`	`str | None`	*required*	

iter_fenced_json

Every parseable fenced JSON OBJECT in `text`, in order (malformed fences skipped, not fatal).

```
iter_fenced_json(text: str | None) -> list[dict]
```

Defined in `interlens.parsing` · source

The fence regex captures `{…}` only — structured actions are always objects — so a stray fenced list/number is ignored, not mistaken for an action. Backs `MessagingPolicy.parse_json_actions`.

Parameters

Name	Type	Default	Description
`text`	`str | None`	*required*	

iter_tagged_json

Every `<tag>{json}</tag>` block in `text` as `(parsed_object, raw_match)` pairs (malformed JSON skipped).

```
iter_tagged_json(text: str | None, tag: str) -> list[tuple[dict, str]]
```

Defined in `interlens.parsing` · source

`raw_match` is the whole matched `<tag>…</tag>` substring, for provenance. Backs the base `ModelParticipant.parse_tool_calls` (Hermes/Qwen `<tool_call>` format).

Parameters

Name	Type	Default	Description
`text`	`str | None`	*required*	
`tag`	`str`	*required*	

last_json

The LAST fenced JSON object in `text`, else the last balanced top-level `{...}` that parses, else `None`. “Last wins” because a model’s final fenced block is its committed action when it revised mid-turn.

```
last_json(text: str | None) -> Any | None
```

Defined in `interlens.parsing` · source

Backs the arena’s `extract_json`.

Parameters

Name	Type	Default	Description
`text`	`str | None`	*required*	

last_json_with_key

The LAST fenced JSON OBJECT in `text` carrying at least one of `keys` at top level, else `None`.

```
last_json_with_key(text: str | None, *keys: str = ()) -> dict | None
```

Defined in `interlens.parsing` · source

:func:`last_json` takes the last parseable object whatever it holds; this takes the last one that *answers a specific question*, skipping past trailing blocks that don't (a model's closing summary fence, a scenario's own status block). "Last wins" for the same reason: a later block supersedes an earlier one.

Backs the negotiation stack's two structured channels — a seat's action object (`last_json_with_key(text, "action", "proposal", "deal")`) and the scenario's authoritative state block (`last_json_with_key(text, "negotiation_state")`) — so neither carries a fence regex of its own.

Parameters

Name	Type	Default	Description
`text`	`str | None`	*required*	
`keys`	`str`	`()`	

split_leading_think

Split a raw completion into `(visible_content, parsed_think)` on a LEADING `<think>...</think>` block only (the participant-level convention).

```
split_leading_think(text: str) -> tuple[str, str | None]
```

Defined in `interlens.parsing` · source

No leading block → `(text.strip(), None)`. Backs the base `ModelParticipant.split_reasoning`.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

strip_think

Remove `<think>...</think>` blocks ANYWHERE in `text`; return `(visible, think)`.

```
strip_think(text: str | None) -> tuple[str, str | None]
```

Defined in `interlens.parsing` · source

The defensive strip used before content reaches another seat's view: a generation truncated mid-`<think>` leaves an *unterminated* block whose reasoning must not leak, so everything from an orphan `<think>` on is treated as reasoning. `think` is the joined reasoning, or `None` when there was none.

Parameters

Name	Type	Default	Description
`text`	`str | None`	*required*	

participant

Package `interlens.participant`

Modules

- `governor` — Adaptive rate-limit governor: admission control paced by the provider's own rate-limit headers.
- `participant`
- `participants`
- `role`
- `serialize` — Persist a participant to / from its own constructor kwargs (for `Conversation.save` / `load`).

Re-exported

Name	Defined in	Summary
`Participant`	`interlens.participant.participant`	A participant in a conversation, either a model or a person.

governor

Module `interlens.participant.governor`

Adaptive rate-limit governor: admission control paced by the provider's own rate-limit headers.

Written for build item 10 of `experiments/rational_agents/auction/docs/design.md` §12 ("Throughput: max out the rate limit"). The stack it replaces is reactive-only — a fixed `max_in_flight` semaphore in `api_client.py` plus 429 exponential backoff — which runs far below the org ceiling because the ceiling is never read. A governed run has **no fixed concurrency knob**: every episode of every cell in the process shares one governor, and the rate-limit headers are the throttle.

Three signals drive admission:

- **Bucket headroom.** Every response (success *and* error) carries `anthropic-ratelimit-{requests, input-tokens,output-tokens,tokens}-{limit,remaining,reset}`. The governor projects each bucket's remaining capacity forward by what the requests admitted since that header snapshot are expected to consume, and admits while every bucket's projection stays above a safety floor (default 10% of the bucket's limit). A caller that would breach the floor sleeps until the earliest bucket reset and is woken there.
- **AIMD.** Any 429 multiplicatively cuts the admission target (×0.5) and, when the response carries `retry-after`, blocks readmission until that deadline. Each clean completion adds 1 back. This is the whole control loop when headers are missing (older SDK, a proxy that strips them): the governor logs that it is **header-blind** and falls back to pure AIMD seeded at `blind_target` (8) concurrent.
- **Live in-flight count.** `live_in_flight` is the number of requests actually in flight right now, exported so the campaign's working-capital guard (`experiments/rational_agents/api_request_budget.py`) can size its worst-case reservations against the real concurrency rather than a static parameter — a static ×120 reservation against the committed budget would starve the tail cells (design.md §11).

The core is a `threading.Condition`, not an asyncio primitive, because the actual call site is a **worker thread**: `EpisodePool` runs episodes as asyncio tasks but each participant call goes through `asyncio.to_thread` into the synchronous provider SDK. A condition variable is admissible from both worlds; :meth:`admit_async` is the awaitable wrapper for callers already on an event loop.

Usage:

```
from interlens.participant.governor import RateLimitGovernor,
     install_governor, governor_report
install_governor(RateLimitGovernor())        # process-wide, before
     launching any cell
...                                          # every Anthropic request now
     passes through it
print(governor_report())                     # into run.log: observed
     ceilings + AIMD history
```

Attributes

Name	Type	Summary
`BUCKET_PREFIXES`		
`RETRY_AFTER_HEADER`		

Classes

Name	Summary
`Bucket`	One rate-limit bucket as last reported by the provider, plus the consumption the governor has admitted since that report.
`RateLimitGovernor`	Process-wide admission controller for one provider+org, paced by

Name	Summary
	rate-limit headers with an AIMD fallback.
`TokenEstimate`	Exponentially weighted per-request token consumption, used to project the token buckets forward between header reports.

Functions

Name	Summary
`current_governor`	The installed process-wide governor, or `None` when the run is ungoverned (the pre-existing behaviour: the client's own `max_in_flight` semaphore is then the only concurrency bound).
`governor_report`	The installed governor's :meth:`RateLimitGovernor.report`, or a one-line notice when none is installed — safe to call unconditionally from a run's logging path.
`install_governor`	Install (or with `None`, remove) the process-wide governor and return it.

Bucket

One rate-limit bucket as last reported by the provider, plus the consumption the governor has admitted since that report.

```
Bucket(
    limit: float,
    remaining: float,
    reset_monotonic: float,
    spent_since_report: float = 0.0,
)
```

Defined in `interlens.participant.governor` · source

`limit`/`remaining` are in the bucket's own units (requests, or tokens); `reset_monotonic` is the refill deadline translated onto the local monotonic clock (the header is an RFC3339 wall-clock timestamp, which is unusable for scheduling across clock skew).

Measured on this org (2026-08-15): an **un-depleted** bucket reports its reset as the current second, so a healthy bucket's deadline is already in the past and only a drawn-down one names a future time.

Parameters

Name	Type	Default	Description
`limit`	`float`	*required*	
`remaining`	`float`	*required*	
`reset_monotonic`	`float`	*required*	
`spent_since_report`	`float`	`0.0`	

Attributes

Name	Type	Summary
`limit`	`float`	
`remaining`	`float`	

Name	Type	Summary
`reset_monotonic`	`float`	
`spent_since_report`	`float`	

Methods

`projected_remaining`

```
projected_remaining(self, now: float) -> float
```

source

Capacity expected to be left if every request admitted since the report consumes its estimate.

Past the reset deadline the bucket has refilled, so the projection restarts from the full limit less what has been admitted since (the conservative reading — the real refill may be more recent than the deadline).

Parameters

Name	Type	Default	Description
`now`	`float`	*required*	

RateLimitGovernor

Process-wide admission controller for one provider+org, paced by rate-limit headers with an AIMD fallback.

```
RateLimitGovernor(
    *,
    floor_fraction: float = 0.1,
    blind_target: float = 8.0,
    max_target: float | None = 512.0,
    min_target: float = 1.0,
    cut_factor: float = 0.5,
    recovery: float = 1.0,
    clock=time.monotonic,
)
```

Defined in `interlens.participant.governor` · source

One governor per provider+org and one per process: it is shared by every participant, episode, and cell, so total in-flight requests are bounded by the measured header ceiling rather than by any per-cell knob.

:param floor_fraction: admit only while every bucket's projected remaining exceeds this fraction of its limit. 0.10 keeps a 10% cushion for requests already in flight whose usage has not been reported yet; raise it for a run that must never see a 429, lower it to squeeze the ceiling harder. :param blind_target: the AIMD admission target used before any header has been seen, and the permanent ceiling if headers never arrive (header-blind mode). 8 is deliberately modest — it recovers upward by one per clean completion, so a genuinely wide limit is discovered within seconds. :param max_target: hard ceiling on the AIMD target, a safety stop against unbounded additive growth when a provider reports enormous buckets. `None` leaves growth bounded only by the buckets. :param min_target: floor the multiplicative cut cannot go below, so a burst of 429s cannot wedge the run at zero concurrency. :param cut_factor: the multiplicative decrease applied to the target on each 429 (0.5 = halve). :param recovery: the additive increase per clean completion (1 request). :param clock: monotonic time source, injectable so tests can drive resets without sleeping.

Parameters

Name	Type	Default	Description
`floor_fraction`	`float`	`0.1`	
`blind_target`	`float`	`8.0`	
`max_target`	`float | None`	`512.0`	
`min_target`	`float`	`1.0`	
`cut_factor`	`float`	`0.5`	
`recovery`	`float`	`1.0`	
`clock`		`time.monotonic`	

Attributes

Name	Type	Summary
`admitted`		
`blind_target`		
`cut_factor`		
`cuts`		
`floor_fraction`		
`header_blind`	`bool`	True while no response has reported a rate-limit header — the governor is running on pure AIMD and says so in :meth:`governor_report`, because a header-blind run cannot claim to have found the ceiling.
`live_in_flight`	`int`	Requests admitted and not yet released, right now.
`max_target`		

Name	Type	Summary
`min_target`		
`rate_limited`		
`recoveries`		
`recovery`		
`waits`		

Methods

`acquire`

```
acquire(self, timeout: float | None = None) -> None
```

source

Block until admission is granted, then count the request as in flight.

`timeout` (seconds) raises `TimeoutError` rather than admitting — leave it `None` for campaign traffic, where waiting for the bucket to refill is the correct behaviour and giving up would just drop an episode.

Parameters

Name	Type	Default	Description
`timeout`	`float | None`	`None`	

`admit`

```
admit(self, timeout: float | None = None)
```

source

Synchronous admission scope: `with governor.admit(): ...` around one provider request.

This is the form the client uses, because provider SDK calls run in worker threads.

Parameters

Name	Type	Default	Description
`timeout`	`float | None`	`None`	

admit_async

```
admit_async(self, timeout: float | None = None)
```

source

Awaitable admission scope for callers already on an event loop: `async with governor.admit_async():`.

The blocking wait is offloaded with `asyncio.to_thread` so the loop is never stalled by a sleeper.

Parameters

Name	Type	Default	Description
`timeout`	`float | None`	`None`	

note_exception

```
note_exception(self, exc) -> None
```

source

Feed one provider exception to the governor without the caller having to know the SDK's exception classes: any exception carrying an HTTP response contributes its headers, and a 429 additionally triggers the AIMD cut.

Anything else (a connection error, a 500) is ignored — those are the retry loop's business, not the rate limiter's.

Parameters

Name	Type	Default	Description
`exc`		*required*	

`note_headers`

```
note_headers(
    self,
    headers,
    *,
    tokens_in: int = 0,
    tokens_out: int = 0,
    seed: bool = True,
) -> None
```

source

Absorb one response's rate-limit headers — from a **success or an error**, since a 429's headers are the most informative ones in the run.

`headers` is any mapping (httpx `Headers`, a plain dict); absent or unparseable headers are ignored, leaving the governor in header-blind AIMD. `tokens_in` / `tokens_out` update the per-request estimate that projects token buckets between reports.

`seed=False` absorbs the buckets without letting them re-seed the AIMD target; the 429 path uses it so a cut is never undone by the very response that reported the overshoot.

Parameters

Name	Type	Default	Description
`headers`		*required*	
`tokens_in`	`int`	`0`	

Name	Type	Default	Description
tokens_out	int	0	
seed	bool	True	

note_rate_limited

```
note_rate_limited(self, *, headers=None, retry_after: float | None = None)
    -> None
```

source

One 429 (or other rate-limit signal): multiplicative cut of the AIMD target and, when the response carries `retry-after` (explicit argument wins, else read from `headers`), a hard block on readmission until that deadline passes.

Headers are absorbed first, so the cut is recorded against fresh ceilings.

Parameters

Name	Type	Default	Description
headers		None	
retry_after	float | None	None	

note_success

```
note_success(self, *, headers=None, tokens_in: int = 0, tokens_out: int =
    0) -> None
```

source

One clean completion: additive recovery of the AIMD target (`+recovery`), plus header absorption.

Parameters

Name	Type	Default	Description
`headers`		`None`	
`tokens_in`	`int`	`0`	
`tokens_out`	`int`	`0`	

`release`

```
release(self) -> None
```

source

Mark one in-flight request finished and wake a waiter.

Always pair with :meth:`acquire` (use :meth:`admit` / :meth:`admit_async` rather than calling either by hand).

`report`

```
report(self) -> str
```

source

The :meth:`snapshot` formatted for `run.log` — one header line plus one line per observed bucket.

`snapshot`

```
snapshot(self) -> dict
```

source

A JSON-serializable view of the control state: live in-flight count, the AIMD target, whether the governor is header-blind, per-bucket ceilings with their projections, the raw last-seen headers, and the cut/recovery/wait counters.

The pilot writes this into `run.log` so the confirmatory launch confirms rather than discovers where the ceiling sits (design.md §12 item 10).

TokenEstimate

Exponentially weighted per-request token consumption, used to project the token buckets forward between header reports.

```
TokenEstimate(
    input_tokens: float = 8000.0,
    output_tokens: float = 2000.0,
    weight: float = 0.2,
)
```

Defined in `interlens.participant.governor` · source

Seeded high (a fresh run has no observations and should under- rather than over-admit) and updated from every response that reports usage.

Parameters

Name	Type	Default	Description
`input_tokens`	`float`	`8000.0`	
`output_tokens`	`float`	`2000.0`	
`weight`	`float`	`0.2`	

Attributes

Name	Type	Summary
`input_tokens`	`float`	
`output_tokens`	`float`	
`weight`	`float`	

Methods

observe

```
observe(self, tokens_in: int, tokens_out: int) -> None
```

source

Parameters

Name	Type	Default	Description
tokens_in	int	*required*	
tokens_out	int	*required*	

per_bucket

```
per_bucket(self) -> dict[str, float]
```

source

The estimated consumption one admitted request charges to each token bucket.

The undifferentiated `tokens` bucket (reported on some plans instead of the split pair) is charged the sum.

current_governor

The installed process-wide governor, or `None` when the run is ungoverned (the pre-existing behaviour: the client's own `max_in_flight` semaphore is then the only concurrency bound).

```
current_governor() -> RateLimitGovernor | None
```

Defined in `interlens.participant.governor` · source

governor_report

The installed governor's :meth:`RateLimitGovernor.report`, or a one-line notice when none is installed — safe to call unconditionally from a run's logging path.

```
governor_report() -> str
```

Defined in `interlens.participant.governor` · source

install_governor

Install (or with `None`, remove) the process-wide governor and return it.

```
install_governor(governor: RateLimitGovernor | None = None) ->
    RateLimitGovernor | None
```

Defined in `interlens.participant.governor` · source

Every hosted-API request issued afterwards passes through it, from any thread or event loop. Called once by the campaign launcher before any cell starts; installing replaces any previous governor rather than stacking.

Parameters

Name	Type	Default	Description
`governor`	RateLimitGovernor | None	`None`	

participant

Module `interlens.participant.participant`

Classes

Name	Summary
`Participant`	A participant in a conversation, either a model or a person.

Participant

A participant in a conversation, either a model or a person.

```
Participant()
```

Defined in `interlens.participant.participant` · source

Inherits from: `ABC`

A participant owns three things: an *identity* within the conversation (`name` + `self_role`/ `others_role`), its *private framing* (`system_prompt` + `private_context` — instructions/knowledge only it sees), and the ability to turn a rendered view into its next message (`generate`). The `Conversation` assembles the structured view from the shared transcript; the participant flattens that view to what its chat template expects via `finalize_view` and generates.

Attributes

Name	Type	Summary
`OPENING_USER_PLACEHOLDER`		
`name`	`str`	A name or identifier to uniquely identify this participant within a conversation.
`others_role`	`Role`	
`private_context`	`tuple`	
`requires_alternating_roles`	`bool`	
`self_role`	`Role`	
`supports_system_role`	`bool`	
`system_prompt`	`str | None`	

Methods

finalize_view

```
finalize_view(self, segments: list[ViewSegment]) -> list[dict]
```

source

Flatten the structured, context-fitted view into the `[{role, content}]` list the chat template consumes.

Applies family-specific repairs driven by the capability flags:

- `supports_system_role=False` → fold the leading system content into the first user turn (Gemma-2's template errors on a standalone `system` role).
- `requires_alternating_roles=True` → merge consecutive same-role segments (Gemma requires strict user/model alternation; the moderator seed + another speaker + private context can otherwise produce consecutive `user` turns that the template rejects). Merged turns keep author labels so speaker identity isn't lost in the concatenation.

The dict-level tail of the same repairs lives in :meth:`repair_view`, which this delegates to (and which participants handed an ALREADY-flattened view — arena scenarios build their per-seat views as plain `[{role, content}]` lists — call directly). Both are idempotent, so running them twice is a no-op.

Parameters

Name	Type	Default	Description
`segments`	list[ViewSegment]	*required*	

generate

```python
generate(
    self,
    view: list[dict],
    *,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    turn: int | None = None,
    max_new_tokens: int | None = None,
    seat: str | None = None,
) -> Message
```

source

Produce this participant's next message given `view` — the conversation flattened to `[{"role", "content"}]` from this participant's perspective.

Returns a `Message` it authored.

Interp options apply to local-model participants: `steering` (a `SteeringSpec`), `capture` (a `CaptureRequest`), `patch` (a `Patch`), and `return_logprobs`; `turn` is the message index used to tag captured activations. Participants that can't honor an interp request (e.g. API-backed) must raise rather than silently ignore it — a failed capture/steer must fail loudly.

`seat` is WHICH SEAT this turn is being spoken for, passed by the arena engine straight from `SeatRequest.seat` (`None` outside the arena, e.g. a plain `Conversation`). Most participants ignore it — the view already addresses them — but a participant that fronts several seats needs to know which one it is answering as, and reading that from the request is exact where recovering it from the prompt text is guesswork that breaks whenever the wording changes.

Parameters

Name	Type	Default	Description
`view`	`list[dict]`	*required*	
`steering`		`None`	

Name	Type	Default	Description
capture		None	
patch		None	
return_logprobs	bool	False	
turn	int | None	None	
max_new_tokens	int | None	None	
seat	str | None	None	

repair_view

```
repair_view(self, messages: list[dict]) -> list[dict]
```

source

Apply this family's chat-template repairs to an **already-flattened** `[{role, content}]` view, so a view that never went through :meth:`finalize_view` (an arena scenario builds its per-seat views directly) still renders under a strict template.

Idempotent; a no-op for permissive families (both flags at their default), which is why every render path can call it unconditionally.

With `requires_alternating_roles` two repairs run, in order:

1. **merge** consecutive same-role turns (joined with a blank line) — a seat that speaks twice in a row (the round-boundary/rotation repeat in a multi-party game) otherwise emits two `assistant` turns.
2. **user-first**: a strict template (Gemma, Mistral/Ministral) requires the first turn after the optional leading `system` to be `user`, so a view that opens with the seat's OWN turn — the opener of a multi-round game, whose first event in the shared log is its own proposal — is repaired by inserting one minimal placeholder `user` turn there. Insertion, not re-roling: every existing turn keeps its role and text, so a family that renders `system` in its own tokens (Mistral's

`[SYSTEM_PROMPT]`) keeps that framing, and the seat's own words are never re-attributed to anyone else.

Parameters

Name	Type	Default	Description
`messages`	`list[dict]`	*required*	

participants

Package `interlens.participant.participants`

Modules

- `api_client`
- `api_participant`
- `gemma`
- `llama`
- `model_participant`
- `qwen`
- `scripted_participant`

Re-exported

Name	Defined in	Summary
`APIParticipant`	`interlens.participant.participants.api_participant`	A participant backed by a hosted API — Claude via `anthropic` (`provider="anthropic"`, the default) or any model behind OpenRouter (`provider="openrouter"`, OpenAI-compatible) — for use as a debate opponent, moderator, or the classifier inside an `analyze` callback.
`GemmaModelParticipant`	`interlens.participant.participants.gemma`	A Gemma-family participant.

Name	Defined in	Summary
LlamaModelParticipant	interlens.participant.participants.llama	A Llama-family participant.
ModelParticipant	interlens.participant.participants.model_participant	A conversation participant backed by a local HuggingFace causal LM.
QwenModelParticipant	interlens.participant.participants.qwen	A participant in a conversation that is a Qwen language model.

api_client

Module `interlens.participant.participants.api_client`

Attributes

Name	Type	Summary
`REASONING_FULL`		
`REASONING_NONE`		
`REASONING_WITHHELD`		

Classes

Name	Summary
`AnthropicClient`	Claude via the `anthropic` SDK (the default provider).
`Completion`	A completion string that also carries the call's **usage telemetry** as attributes: `input_tokens` / `output_tokens` (0 when the provider reported none), `stop_reason` (the provider's native stop/finish reason, `None` when unreported), `batched` (served via a provider batch API at discount pricing), `cache_read_tokens` / `cache_write_tokens` (Anthropic prompt-cache accounting: tokens served from a cache entry, and tokens written into one — both EXCLUDED from `input_tokens`, which is the full-price remainder), and the call's **reasoning record**:

Name	Summary
	`reasoning` (whatever reasoning text the provider returned — Anthropic thinking blocks including summarized ones, OpenAI-compatible `reasoning`/`reasoning_content` fields — or `None`) with `reasoning_provenance` marking how complete that record is (see the marker constants above).
`OpenAIClient`	OpenAI directly via the `openai` SDK (`provider="openai"`).
`OpenRouterClient`	OpenRouter (https://openrouter.ai) via the OpenAI-compatible `openai` SDK — one endpoint proxying many providers' models (e.g. `anthropic/claude-sonnet-5`, `openai/gpt-5`, `meta-llama/llama-3.1-70b-instruct`).

Functions

Name	Summary
`anthropic_reasoning`	Extract the reasoning record from an Anthropic `content` block list: `(reasoning_text, provenance)`.
`openai_reasoning`	Extract the reasoning record from an OpenAI-schema `choices[0].message` (+ `usage`): `(reasoning_text, provenance)`.

AnthropicClient

Claude via the `anthropic` SDK (the default provider).

```
AnthropicClient(**kwargs={})
```

Defined in `interlens.participant.participants.api_client` · source

Inherits from: `_RetryingClient`

Uses Anthropic's separate `system` param.

Parameters

Name	Type	Default	Description
`kwargs`		`{}`	

Methods

`submit_batch`

```
submit_batch(
    self,
    requests: list[dict],
    *,
    poll_interval: float = 30.0,
) -> 'list[Completion]'
```

source

Anthropic **Message Batches API**: one `messages.batches.create` submits every request (tagged with a positional `custom_id`), then poll `retrieve` until `processing_status == 'ended'` and stream `results` back, reassembled into input order.

A non-succeeded per-request result raises.

Parameters

Name	Type	Default	Description
`requests`	`list[dict]`	*required*	
`poll_interval`	`float`	`30.0`	

Completion

A completion string that also carries the call's **usage telemetry** as attributes: `input_tokens` / `output_tokens` (0 when the provider reported none), `stop_reason` (the provider's native stop/finish reason, `None` when unreported), `batched` (served via a provider batch API at discount pricing), `cache_read_tokens` / `cache_write_tokens` (Anthropic prompt-cache accounting: tokens served from a cache entry, and tokens written into one — both EXCLUDED from `input_tokens`, which is the full-price remainder), and the call's **reasoning record**: `reasoning` (whatever reasoning text the provider returned — Anthropic thinking blocks including summarized ones, OpenAI-compatible `reasoning`/`reasoning_content` fields — or `None`) with `reasoning_provenance` marking how complete that record is (see the marker constants above).

```
Completion()
```

Defined in `interlens.participant.participants.api_client` · source

Inherits from: `str`

OpenAI-compatible responses also preserve `upstream_provider`, `response_model`, and `generation_id` when reported; these make OpenRouter routing auditable.

Subclassing `str` keeps the documented client contract — `callable(...) → str` — fully intact for existing callers and injected test clients, while letting `APIParticipant` read the telemetry off the return value to record per-turn usage (`Message.metadata`) and report into a `UsageMeter`.

Attributes

Name	Type	Summary
`batched`	`bool`	
`cache_read_tokens`	`int`	
`cache_write_tokens`	`int`	

Name	Type	Summary
generation_id	str | None	
input_tokens	int	
output_tokens	int	
reasoning	str | None	
reasoning_provenance	str	
reasoning_tokens	int	
response_model	str | None	
stop_reason	str | None	
upstream_provider	str | None	

OpenAIClient

OpenAI directly via the `openai` SDK (`provider="openai"`).

```
OpenAIClient()
```

Defined in `interlens.participant.participants.api_client` · source

Inherits from: `_OpenAICompatClient`

Reads `OPENAI_API_KEY`. Supports the asynchronous **Batch API** for large rollouts.

Methods

`submit_batch`

```
submit_batch(
    self,
    requests: list[dict],
    *,
    poll_interval: float = 30.0,
) -> 'list[Completion]'
```

source

OpenAI **Batch API**: upload a JSONL of `/v1/chat/completions` requests (positional `custom_id`), `batches.create` with a 24h window, poll until `status == 'completed'`, then download + parse the output file back into input order.

A failed/expired/cancelled batch raises.

Parameters

Name	Type	Default	Description
`requests`	`list[dict]`	*required*	
`poll_interval`	`float`	`30.0`	

OpenRouterClient

OpenRouter (https://openrouter.ai) via the OpenAI-compatible `openai` SDK — one endpoint proxying many providers' models (e.g. `anthropic/claude-sonnet-5`, `openai/gpt-5`, `meta-llama/llama-3.1-70b-instruct`).

```
OpenRouterClient()
```

Defined in `interlens.participant.participants.api_client` · source

Inherits from: `_OpenAICompatClient`

Reads `OPENROUTER_API_KEY`. OpenRouter has **no batch API**, so `submit_batch` inherits the base's raise — requesting batch mode on an OpenRouter participant fails loudly.

anthropic_reasoning

Extract the reasoning record from an Anthropic `content` block list: `(reasoning_text, provenance)`.

```
anthropic_reasoning(content_blocks) -> tuple[str | None, str]
```

Defined in `interlens.participant.participants.api_client` · source

`thinking` blocks are persisted verbatim as returned — but current Claude models return **summarized** thinking over the API, and `redacted_thinking` blocks carry no readable text at all, so any thinking-bearing response is marked `withheld_or_summarized` rather than `full`: the model's actual reasoning stream is longer than what the provider hands back. No thinking blocks → `none`.

Parameters

Name	Type	Default	Description
`content_blocks`		*required*	

openai_reasoning

Extract the reasoning record from an OpenAI-schema `choices[0].message` (+ `usage`): `(reasoning_text, provenance)`.

```
openai_reasoning(message, usage) -> tuple[str | None, str]
```

Defined in `interlens.participant.participants.api_client` · source

Works with SDK objects and plain dicts (the batch-API path).

OpenRouter/DeepSeek-style `reasoning` / `reasoning_content` fields carry the model's raw reasoning stream → `full`. OpenAI's own reasoning models withhold the stream but count it in `usage.completion_tokens_details.reasoning_tokens` → `withheld_or_summarized` with no text.

Parameters

Name	Type	Default	Description
`message`		*required*	
`usage`		*required*	

api_participant

Module `interlens.participant.participants.api_participant`

Attributes

Name	Type	Summary
`MAX_CACHE_BREAKPOINTS`		
`MIN_CACHEABLE_PREFIX_NOTE`		
`OpenRouterQuantization`		
`Provider`		

Classes

Name	Summary
`APIParticipant`	A participant backed by a hosted API — Claude via `anthropic` (`provider="anthropic"`, the default) or any model behind OpenRouter (`provider="openrouter"`, OpenAI-compatible) — for use as a debate opponent, moderator, or the classifier inside an `analyze` callback.
`OpenRouterRouting`	Reproducible OpenRouter routing for research.
`PromptCache`	Where to put a request's `cache_control` breakpoints — Anthropic only.

APIParticipant

A participant backed by a hosted API — Claude via `anthropic` (`provider="anthropic"`, the default) or any model behind OpenRouter (`provider="openrouter"`, OpenAI-compatible) — for use as a debate opponent, moderator, or the classifier inside an `analyze` callback.

```
APIParticipant(
    name: str = '',
    model_id: str = '',
    provider: Provider = 'anthropic',
    system_prompt: str | None = None,
    private_context: tuple = (),
    max_tokens: int = 512,
    temperature: float = 1.0,
    batch: bool = False,
    client: object = None,
    openrouter_routing: OpenRouterRouting | None = None,
    prompt_cache: PromptCache | None = None,
    meter: object = None,
    turn_token_floor: int | None = None,
    thinking: object = None,
    effort: str | None = None,
    requires_alternating_roles: bool = True,
)
```

Defined in `interlens.participant.participants.api_participant` · source

Inherits from: Functional, Participant

It is a full participant for *conversation* purposes but has **no local model** — so there is no device, no activations, and no steering. Any interp request (`capture`/`steering`/`patch`/`return_logprobs`) **raises** rather than silently no-op'ing: in a measurement harness, a steering sweep that quietly did nothing on an API participant would produce a false "no effect" conclusion. Seeds don't bind hosted models, so API turns are excluded from the identical-replay guarantee.

Concurrency is network-bound, so pure-API conversations run thread-pooled rather than process-per-GPU (handled by the runner). The `client` callable is injectable for testing.

Parameters

Name	Type	Default	Description
`name`	`str`	`''`	
`model_id`	`str`	`''`	
`provider`	`Provider`	`'anthropic'`	
`system_prompt`	`str | None`	`None`	
`private_context`	`tuple`	`()`	
`max_tokens`	`int`	`512`	
`temperature`	`float`	`1.0`	
`batch`	`bool`	`False`	
`client`	`object`	`None`	
`openrouter_routing`	OpenRouter-Routing | None	`None`	
`prompt_cache`	PromptCache | None	`None`	
`meter`	`object`	`None`	
`turn_token_floor`	`int | None`	`None`	
`thinking`	`object`	`None`	
`effort`	`str | None`	`None`	
`requires_alternating_roles`	`bool`	`True`	

Attributes

Name	Type	Summary
`batch`	`bool`	
`client`	`object`	
`effort`	`str | None`	Reasoning-effort level, Anthropic only: `"low"`/`"medium"`/`"high"`/`"xhigh"`/`"max` sent as `output_config={"effort": ...}`.
`max_tokens`	`int`	

Name	Type	Summary
`meter`	`object`	Optional `interlens.usage.UsageMeter`: every call this participant makes is reported into it (tokens, dollars at the actual billing multiplier — batch-served turns bill at 0.5× — and refusal counts), so one shared meter across participants gives a live, run-level spend ledger with reservation gating.
`model_id`	`str`	
`name`	`str`	
`openrouter_routing`	OpenRouterRouting | None	Required for `provider="openrouter"`.
`private_context`	`tuple`	
`prompt_cache`	PromptCache | None	Where to place Anthropic `cache_control` breakpoints, or `None` (default) for no caching.
`provider`	`Provider`	
`requires_alternating_roles`	`bool`	
`system_prompt`	`str | None`	
`temperature`	`float`	
`thinking`	`object`	Reasoning control, Anthropic only: `None` keeps the model's default (adaptive think-

Name	Type	Summary
		ing on current Claude models — which spends from `max_tokens`), `"disabled"` turns thinking off, an `int` sets an explicit thinking budget, and a dict passes through verbatim.
`turn_token_floor`	`int | None`	Thinking-aware lower bound on an externally imposed per-turn cap.

Methods

`generate`

```
generate(
    self,
    view: list[dict],
    *,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    turn: int | None = None,
    max_new_tokens: int | None = None,
    seat: str | None = None,
) -> Message
```

source

Parameters

Name	Type	Default	Description
`view`	`list[dict]`	*required*	

Name	Type	Default	Description
steering		None	
capture		None	
patch		None	
return_logprobs	bool	False	
turn	int | None	None	
max_new_tokens	int | None	None	
seat	str | None	None	

generate_batch

```
generate_batch(
    self,
    views: list[list[dict]],
    *,
    turn: int | None = None,
    group_seed: int | None = None,
    max_new_tokens: int | None = None,
) -> list[Message]
```

source

Generate one turn for many independent conversations at once — the API analogue of `ModelParticipant.generate_batch`, driven by the runner's co-stepper (`rollout(..., batched=True)`) to make large API rollouts cheap and throughput-bound.

With `batch=True` every view is sent as one **asynchronous provider batch** (Anthropic Message Batches / OpenAI Batch API) via `client.submit_batch` — ~50% cost and much higher throughput, at the price of batch-window latency. **If the provider has no batch API (e.g. OpenRouter) this raises** rather than silently degrading, so a requested batch is never quietly run as serial calls. With `batch=False` it falls back to sequential per-view calls (correct, just no batch discount). Interp is unavailable here, as for `generate`. `turn`/`group_seed` are accepted for co-stepper compatibility but unused (seeds do not bind hosted models). `metadata['batched']` marks these turns.

Parameters

Name	Type	Default	Description
`views`	`list[list[dict]]`	*required*	
`turn`	`int | None`	`None`	
`group_seed`	`int | None`	`None`	
`max_new_tokens`	`int | None`	`None`	

`request_config`

```
request_config(self) -> dict
```

source

The reasoning-relevant request parameters this participant will actually send.

Exists so a run manifest can record the condition it ran rather than the condition it *meant* to run: `thinking=None` and an explicit adaptive dict produce identical behaviour on current Claude models but are not the same record, and a cell labelled “thinking on” is uninterpretable without knowing which one it sent.

OpenRouterRouting

Reproducible OpenRouter routing for research.

```
OpenRouterRouting(
    upstream_provider: str | None,
    quantizations: tuple[OpenRouterQuantization, ...] = (),
    data_collection: Literal['allow', 'deny'] | None = None,
    allow_unpinned: bool = False,
)
```

Defined in `interlens.participant.participants.api_participant` · source

`upstream_provider` is the OpenRouter provider slug (for example `"together"` or `"deepinfra"`). A pinned request sends both `only=[slug]` and `order=[slug]`, disables fallbacks, and requires support for every supplied generation parameter. `quantizations` should also be set for open-weight models when the endpoint offers multiple precisions. `data_collection="deny"` excludes providers that may retain or train on prompts.

Use :meth:`unpinned` only for exploratory traffic where provider variance is intentionally acceptable. Requiring that explicit opt-out prevents an uncontrolled request from looking like a reproducible run.

Parameters

Name	Type	Default	Description
`upstream_provider`	`str | None`	*required*	
`quantizations`	`tuple[OpenRouterQuantization, ...]`	`()`	
`data_collection`	`Literal['allow', 'deny'] | None`	`None`	
`allow_unpinned`	`bool`	`False`	

Attributes

Name	Type	Summary
allow_unpinned	bool	
data_collection	Literal['allow', 'deny'] | None	
quantizations	tuple[OpenRouterQuantization, ...]	
upstream_provider	str | None	

Methods

request_dict

```
request_dict(self) -> dict
```

source

Return the exact OpenRouter `provider` request object.

unpinned

```
unpinned(
    cls,
    *,
    data_collection: Literal['allow', 'deny'] | None = None,
) -> 'OpenRouterRouting'
```

source

Explicitly opt into OpenRouter's variable default provider routing for exploratory, non-reproducible use.

Parameters

Name	Type	Default	Description
data_collection	Literal['allow', 'deny'] | None	None	

PromptCache

Where to put a request's `cache_control` breakpoints — Anthropic only.

```
PromptCache(system: bool = True, marks: tuple[str, ...] = (), ttl: str
    = '5m')
```

Defined in `interlens.participant.participants.api_participant` · source

Caching is a **prefix** match: the cache key is the exact bytes up to each breakpoint, in the provider's render order (`tools → system → messages`), so any byte that changes early invalidates everything after it. This spec therefore describes *stability boundaries*, not "cache these strings".

:param system: Put a breakpoint at the end of the system prompt (default on). This is the one boundary every multi-turn scenario has for free: a seat's system prompt is fixed for its whole episode, and it renders before any message, so one breakpoint there caches the framing of every later turn. :param marks: Literal substrings of the LAST user message, in the order they appear. The text *before* each mark's first occurrence ends a cached prefix, so a scenario that renders its turn view as stable sections followed by volatile ones (a catalogue and a history digest, then this turn's private state and ask) gets those sections cached by naming the headings that follow them. Splitting is byte-preserving — the concatenated blocks are the original message — so the model reads exactly what it read uncached, and the refusal ladder and stored `view` are untouched. A mark that does not occur is skipped rather than raising, because a view legitimately varies by phase. :param ttl: `"5m"` (default) or `"1h"`. A write costs 1.25x the input rate at 5m and 2x at 1h, against 0.1x for a read, so 5m breaks even on the second request and 1h on the third. Prefer `"1h"` only when a seat's turns are more than five minutes apart — which wave-parallel generation is specifically designed to stop being true.

Breakpoints are capped at :data:`MAX_CACHE_BREAKPOINTS`; `system` consumes one, so at most three marks take effect and the rest are ignored. {note}

Parameters

Name	Type	Default	Description
system	bool	True	
marks	tuple[str, ...]	()	
ttl	str	'5m'	

Attributes

Name	Type	Summary
marks	tuple[str, ...]	
system	bool	
ttl	str	

Methods

control

```
control(self) -> dict
```

source

The `cache_control` value one breakpoint carries.

split

```
split(self, text: str) -> list[dict]
```

source

`text` as Anthropic text blocks with a breakpoint before each of :attr:`marks`.

Concatenating the blocks' `text` reproduces `text` exactly. The final block never carries a breakpoint: it is the volatile tail, and marking it would write a fresh

cache entry per turn that is never read — the failure that reports as “caching is on and saving nothing”.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

gemma

Module `interlens.participant.participants.gemma`

Attributes

Name	Type	Summary
`logger`		

Classes

Name	Summary
`GemmaModelParticipant`	A Gemma-family participant.

GemmaModelParticipant

A Gemma-family participant.

```
GemmaModelParticipant()
```

Defined in `interlens.participant.participants.gemma` · source

Inherits from: ModelParticipant

The only thing that differs from base is the tool-call format (```` ```tool_code ````); the chat-template flags (Gemma 2 rejects a standalone `system` role and requires strict user/model alternation, Gemma 3 accepts a system role) are auto-derived from the tokenizer's own template, so both generations use this one class.

Attributes

Name	Type	Summary
`MODEL_TYPES`	`frozenset[str]`	

Methods

parse_tool_calls

```
parse_tool_calls(self, text: str) -> list
```

source

Parse Gemma's ```` ```tool_code ```` function-call blocks (best-effort).

Gemma emits calls like ```` ```tool_code name(arg="x", n=2) ``` ```` rather than Hermes JSON. We extract the block, then the function name and simple keyword arguments. Falls back to `[]` (treat as final message) on anything it can't parse, matching the base contract of never misfiring silently.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

llama

Module `interlens.participant.participants.llama`

Attributes

Name	Type	Summary
`logger`		

Classes

Name	Summary
`LlamaModelParticipant`	A Llama-family participant.

LlamaModelParticipant

A Llama-family participant.

```
LlamaModelParticipant()
```

Defined in `interlens.participant.participants.llama` · source

Inherits from: ModelParticipant

Chat-template flags are auto-derived from the tokenizer; only the tool-call format differs from base: Llama 3 emits calls as `◁python_tag▷{json}` rather than Hermes/Qwen `<tool_call>` blocks.

Attributes

Name	Type	Summary
`MODEL_TYPES`	`frozenset[str]`	

Methods

parse_tool_calls

```
parse_tool_calls(self, text: str) -> list
```

source

Parse Llama-3's `◁python_tag▷{json}` function-call format (best-effort).

Everything after `◁python_tag▷` is one or more JSON objects (separated by newlines or semicolons), each `{"name": ..., "arguments"/"parameters": {...}}`. Anything unparseable is skipped, so a malformed call yields `[]` (treated as a final message), matching the base contract of never misfiring.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

model_participant

Module `interlens.participant.participants.model_participant`

Attributes

Name	Type	Summary
`logger`		

Classes

Name	Summary
`ModelParticipant`	A conversation participant backed by a local HuggingFace causal LM.

Functions

Name	Summary
`dtype_to_str`	`torch.bfloat16` -> `'bfloat16'` (accepts a str unchanged).
`str_to_dtype`	`'bfloat16'` -> `torch.bfloat16` (accepts a `torch.dtype` unchanged).

ModelParticipant

A conversation participant backed by a local HuggingFace causal LM.

```
ModelParticipant(
    name: str = '',
    hf_id: str | None = None,
    weights_path: str | None = None,
    adapter_path: str | None = None,
    dtype: str = 'bfloat16',
    attn: str = 'flash_attention_2',
    quant: str | None = None,
    revision: str | None = None,
    device: str | torch.device | None = None,
    max_new_tokens: int = 512,
    temperature: float = 0.8,
    top_p: float = 0.95,
    seed: int | None = None,
    thinking: bool | str = 'auto',
    system_prompt: str | None = None,
    private_context: tuple = (),
    tools: tuple = (),
    max_tool_iters: int = 4,
    kv_reuse: bool | str = 'auto',
    steering: object = None,
)
```

Defined in `interlens.participant.participants.model_participant` · source

Inherits from: Functional, Participant

Generation flow: the `Conversation` hands us a `view` — the transcript rendered from *our* perspective, already context-fitted and flattened by `finalize_view` into `[{role, content}]`. We apply this model's own chat template to it, generate, decode only the newly produced tokens, and split any `<think>` reasoning out of the visible content. Only the visible answer becomes `Message.content`; the parsed reasoning and raw completion live in `metadata` under neutral keys, so hidden generated text is never fed back into other participants' views (it's stripped from history automatically because `render_roles` uses `content`).

The participant is its own recipe. It stores *what to load* (`hf_id` + dtype/attn/quant/revision) and loads the weights **lazily on first use** through the process-wide model cache — so an unrun participant is cheap (KBs), pickles across a spawn boundary without shipping weights, and `.set(...)`

copies share one loaded model object by reference (the co-stepping batch win). `model`/`tokenizer` are properties that trigger the load; the raw `model` is exposed deliberately (interp experiments register forward hooks on it). Weight loads are keyed on `(hf_id, device, dtype, attn, quant, revision)` and cached, so all same-recipe participants on one device share ONE object. Build via `from_pretrained` (lazy) or `from_model` (eager, from an already-loaded model); `.set()` (from `Functional`) makes copy-on-write clones with fresh KV state.

Parameters

Name	Type	Default	Description
name	str	''	
hf_id	str | None	None	
weights_path	str | None	None	
adapter_path	str | None	None	
dtype	str	'bfloat16'	
attn	str	'flash_attention_2'	
quant	str | None	None	
revision	str | None	None	
device	str | torch.device | None	None	
max_new_tokens	int	512	
temperature	float	0.8	
top_p	float	0.95	
seed	int | None	None	
thinking	bool | str	'auto'	
system_prompt	str | None	None	
private_context	tuple	()	
tools	tuple	()	
max_tool_iters	int	4	
kv_reuse	bool | str	'auto'	

Name	Type	Default	Description
steering	object	None	

Attributes

Name	Type	Summary
MODEL_TYPES	frozenset[str]	
adapter_path	str | None	
attn	str	
device	str | torch.device | None	
dtype	str	
hf_id	str | None	
input_device	'torch.device'	Where this participant's input_ids must land — the loaded model's **input-embedding** device.
kv_reuse	bool | str	
max_new_tokens	int	
max_tool_iters	int	
model	'PreTrainedModel'	The loaded HF model, loading it on first access (cached process-wide).
name	str	
private_context	tuple	
quant	str | None	
revision	str | None	
seed	int | None	

Name	Type	Summary
steering	object	
system_prompt	str | None	
temperature	float	
thinking	bool | str	
tokenizer	'PreTrainedTokenizerBase'	The loaded tokenizer, loading the model+tokenizer on first access (cached process-wide).
tools	tuple	
top_p	float	
weights_path	str | None	

Methods

batch_signature

```
batch_signature(self) -> tuple
```

source

Key identifying which model this participant would batch AS (used by the co-stepper).

When loaded, the cached model object's identity is authoritative; when not yet loaded, the load recipe is (the cache guarantees same recipe on one device -> same object), so participants can be grouped WITHOUT forcing a load.

for_model_type

```
for_model_type(cls, model_type: str | None) -> type['ModelParticipant']
```

source

The registered participant class for a transformers `config.model_type` (e.g. `qwen2`, `gemma2`), falling back to base `ModelParticipant` for any family that declares no specialized subclass.

Parameters

Name	Type	Default	Description
`model_type`	`str | None`	*required*	

`from_model`

```
from_model(
    cls,
    model: PreTrainedModel,
    tokenizer: PreTrainedTokenizerBase | None = None,
    *,
    name: str,
    device: str | torch.device | None = None,
    **participant_kwargs={},
) -> Self
```

source

Build a participant of THIS class from an already-loaded `model` (the eager path — e.g. to wrap a model you already hold, or share weights between speakers).

`tokenizer` is optional — when omitted it is inferred from `model.config._name_or_path`. The chat-template flags (`supports_system_role` / `requires_alternating_roles`) are derived from the tokenizer's own template. `hf_id`/`dtype` are read back from the model so the participant can still be pickled + re-loaded on a spawn worker. Because it constructs `cls`, calling it on a subclass returns that subclass — use `AutoModelParticipant.from_model` to have the family resolved from `config.model_type`.

Parameters

Name	Type	Default	Description
model	PreTrainedModel	*required*	
tokenizer	PreTrainedTokenizerBase | None	None	
name	str	*required*	
device	str | torch.device | None	None	
participant_kwargs		{}	

from_pretrained

```
from_pretrained(
    cls,
    id_or_path: str | Path,
    *,
    name: str,
    device: str | torch.device = 'cuda',
    load_kwargs: dict | None = None,
    **participant_kwargs={},
) -> Self
```

source

Build a participant of THIS class that will load `id_or_path` (an HF id or local path) **lazily on first use** — no weights are touched here.

`load_kwargs` (`dtype` / `attn` / `quant` / `revision` / `weights_path` / `adapter_path`) are recorded for that deferred load; `participant_kwargs` (`temperature`, `max_new_tokens`, `system_prompt`, `tools`, `kv_reuse`, …) go to the participant. When the load fires it is process-cached, so all same-(id, device, dtype, …) participants share ONE model object. As with `from_model`, the class is `cls` — use `AutoModelParticipant.from_pretrained` to resolve the family from `config.model_type` (it reads only the config, still no weights).

Parameters

Name	Type	Default	Description
id_or_path	str | Path	*required*	

Name	Type	Default	Description
name	str	*required*	
device	str | torch.device	'cuda'	
load_kwargs	dict | None	None	
participant_kwargs		{}	

generate

```python
generate(
    self,
    view: list[dict],
    *,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    turn: int | None = None,
    max_new_tokens: int | None = None,
    seat: str | None = None,
) -> Message
```

source

Parameters

Name	Type	Default	Description
view	list[dict]	*required*	
steering		None	
capture		None	
patch		None	
return_logprobs	bool	False	
turn	int | None	None	
max_new_tokens	int | None	None	
seat	str | None	None	

generate_batch

```
generate_batch(
    self,
    views: list[list[dict]],
    *,
    turn: int | None = None,
    group_seed: int | None = None,
    max_new_tokens: int | None = None,
) -> list[Message]
```

source

Batched generation for many independent conversations that share THIS model (`throughput` mode).

Renders each `view` with this model's chat template and runs **one** `model.generate` over the left-padded batch, returning one `Message` per view — the co-stepping throughput win (5-20x on a rollout). `max_new_tokens` overrides this participant's per-turn cap for the batch (the co-stepper passes a `turn_cap` here so a `TokenBudget` can shrink the final round). **No tools/steering/capture/logprobs** here: callers needing those fall back to the per-conversation `generate`. Tokens are **not** guaranteed identical to unbatched — batch composition and the single global RNG perturb rows (see PLAN §Execution modes); only distributional reproducibility holds. `metadata['batched']` marks these turns; `metadata['shared_prefill']` marks the fast path.

Parameters

Name	Type	Default	Description
`views`	`list[list[dict]]`	*required*	
`turn`	`int | None`	`None`	
`group_seed`	`int | None`	`None`	
`max_new_tokens`	`int | None`	`None`	

generate_step

```
generate_step(
    self,
    view: list[dict],
    *,
    tool_schemas: list[dict] | None = None,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    turn: int | None = None,
    max_new_tokens: int | None = None,
    seat: str | None = None,
) -> Message
```

source

Run one model generation without executing requested tools.

This is the policy/model boundary used by external agent runtimes such as Inspect and Control Tower: `tool_schemas` are rendered by the participant's native chat template, parsed calls are returned as serializable dictionaries in `Message.metadata["tool_calls"]`, and the caller remains responsible for executing them. Keeping execution outside Interlens lets the outer runtime monitor, approve, replace, or sandbox each action.

Returns

Name	Type	Description
	Message	A visible `Message` with usage/reasoning metadata and zero or more parsed `tool_calls`. Tools are never
	Message	executed by this method.

Parameters

Name	Type	Default	Description
`view`	`list[dict]`	*required*	Flattened chat history as dictionaries with `role` and `content`. Assistant tool calls and tool results may use the standard `tool_calls` / `tool_call_id` fields.
`tool_schemas`	`list[dict]` | `None`	`None`	OpenAI-style function schemas available for this step. Pass `None` or `[]` to disable tool calling; use schemas when the outer runtime must execute and observe calls itself.
`steering`		`None`	Optional activation steering applied during this generation. `None` uses the participant's persistent `steering` setting.

Name	Type	Default	Description
`capture`		`None`	Optional activation-capture request for this generation.
`patch`		`None`	Optional activation patch applied during this generation.
`return_logprobs`	`bool`	`False`	Whether to record generated-token log probabilities in message metadata.
`turn`	`int | None`	`None`	Optional turn index used to tag captured activations.
`max_new_tokens`	`int | None`	`None`	Per-call output cap. `None` uses the participant's configured `max_new_tokens`.
`seat`	`str | None`	`None`	Optional arena seat identifier, accepted for parity with :meth:`generate`; local participants do not otherwise use it.

parse_tool_calls

```
parse_tool_calls(self, text: str) -> list
```

source

Parse tool calls out of a generation.

Base handles the common Hermes/Qwen `<tool_call>{json}</tool_call>` format (JSON extraction via :func:`interlens.parsing.iter_tagged_json`); families with other formats (Gemma's ```` ```tool_code ````, Llama's `◁python_tag▷`) override this. An unrecognized/absent call yields `[]` so the loop treats the output as a final message.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

render_tool_result

```
render_tool_result(self, call, result) -> dict
```

source

Render a tool result as the standard structured `tool` message; the tokenizer's own template turns it into the family-native format.

Parameters

Name	Type	Default	Description
`call`		*required*	
`result`		*required*	

split_reasoning

```
split_reasoning(self, text: str) -> tuple[str, str | None]
```

source

Split a raw completion into `(visible_content, parsed_think)`.

Base handles the leading `<think>...</think>` convention (via :func:`interlens.parsing.split_leading_think`); families with other delimiters override this.

Parameters

Name	Type	Default	Description
`text`	`str`	*required*	

dtype_to_str

`torch.bfloat16` -> `'bfloat16'` (accepts a str unchanged).

```
dtype_to_str(dtype) -> str
```

Defined in `interlens.participant.participants.model_participant` · source

Parameters

Name	Type	Default	Description
`dtype`		*required*	

str_to_dtype

`'bfloat16'` -> `torch.bfloat16` (accepts a `torch.dtype` unchanged).

```
str_to_dtype(name) -> 'torch.dtype'
```

Defined in `interlens.participant.participants.model_participant` · source

Parameters

Name	Type	Default	Description
`name`		*required*	

qwen

Module `interlens.participant.participants.qwen`

Classes

Name	Summary
`QwenModelParticipant`	A participant in a conversation that is a Qwen language model.

QwenModelParticipant

A participant in a conversation that is a Qwen language model.

```
QwenModelParticipant()
```

Defined in `interlens.participant.participants.qwen` · source

Inherits from: ModelParticipant

Its tool-call format is the Hermes/base `<tool_call>` JSON already handled by `ModelParticipant`, so this class adds no behavior — it exists so Qwen models resolve to a distinct, statically-typed participant class.

Attributes

Name	Type	Summary
`MODEL_TYPES`	`frozenset[str]`	

scripted_participant

Module `interlens.participant.participants.scripted_participant`

Classes

Name	Summary
`ScriptedParticipant`	A non-model participant that replies with **pre-written (scripted) messages**, cycled in order — its turns are fixed, NOT generated from the conversation.

ScriptedParticipant

A non-model participant that replies with **pre-written (scripted) messages**, cycled in order — its turns are fixed, NOT generated from the conversation.

```
ScriptedParticipant(
    name: str,
    scripts: 'str | list[str]',
    *,
    system_prompt: str | None = None,
    private_context: tuple = (),
)
```

Defined in `interlens.participant.participants.scripted_participant` · source

Inherits from: Participant

The motivating use is a **reliable adversary / pusher**: to measure whether a target model capitulates under social pressure you need an interlocutor that asserts a specific (often wrong) answer *every* turn. A real model is a poor adversary for this — aligned/reasoning models frequently refuse to argue a known-wrong answer and instead re-solve correctly, so the "pressure" silently vanishes (an invisible confound unless you read transcripts). Scripting the adversary guarantees uniform, controlled pressure and costs no model/API calls.

It ignores the conversation `view` (its replies are fixed) and holds no activations, so it raises on any interp request (`steering`/`capture`/`patch`/`return_logprobs`) rather than silently ignoring it — consistent with the `Participant.generate` contract.

Parameters

name : str Identifier within the conversation. scripts : str | list[str] The canned message(s). A single string is treated as a one-element list. Turns cycle through the list in order (turn `k` emits `scripts[k % len(scripts)]`), so a few varied assertions read less robotically than one repeated line while still asserting the same position. system_prompt : str | None Optional system framing (recorded for view/transcript symmetry; the scripted replies don't depend

on it). private_context : tuple Optional private context, for parity with other participants (unused by the fixed replies).

Parameters

Name	Type	Default	Description
name	str	*required*	
scripts	'str | list[str]'	*required*	
system_prompt	str | None	None	
private_context	tuple	()	

Attributes

Name	Type	Summary
name		
others_role		
private_context		
scripts		
self_role		
system_prompt		

Methods

generate

```
generate(
    self,
    view: list[dict],
    *,
    steering=None,
    capture=None,
    patch=None,
    return_logprobs: bool = False,
    turn: int | None = None,
```

```
    max_new_tokens: int | None = None,
    seat: str | None = None,
) -> Message
```

source

Return the next scripted message (cycled), ignoring `view`.

Raises on interp requests — a scripted participant has no model/activations to steer, capture, patch, or read logprobs from.

Parameters

Name	Type	Default	Description
`view`	`list[dict]`	*required*	
`steering`		`None`	
`capture`		`None`	
`patch`		`None`	
`return_logprobs`	`bool`	`False`	
`turn`	`int | None`	`None`	
`max_new_tokens`	`int | None`	`None`	
`seat`	`str | None`	`None`	

role

Module `interlens.participant.role`

Attributes

Name	Type	Summary
`Role`		

serialize

Module `interlens.participant.serialize`

Persist a participant to / from its own constructor kwargs (for `Conversation.save` / `load`).

This is same-type persistence — a participant serializes the recipe it was built from (an HF id + generation settings, or an API provider + model id) and rebuilds an *equivalent lazy* participant. It is NOT a conversion to a separate config type (participants ARE their own recipe now); weights are never serialized and reload lazily on the target device. `private_context` (`ContextItem`s) is inlined; local tools are stored by name and re-resolved from the tool registry on load.

Functions

Name	Summary
`participant_from_dict`	Rebuild a (lazy) participant from :func:`participant_to_dict` output.
`participant_to_dict`	Serialize participant `p` to a plain dict of its constructor kwargs (no weights).

participant_from_dict

Rebuild a (lazy) participant from :func:`participant_to_dict` output.

```
participant_from_dict(data: dict, device='cuda', registry=None)
```

Defined in `interlens.participant.serialize` · source

Local models resolve their family class from the HF config and load lazily on `device`; API participants are reconstructed directly.

Parameters

Name	Type	Default	Description
`data`	`dict`	*required*	
`device`		`'cuda'`	
`registry`		`None`	

participant_to_dict

Serialize participant `p` to a plain dict of its constructor kwargs (no weights).

```
participant_to_dict(p) -> dict
```

Defined in `interlens.participant.serialize` · source

Dispatches on the participant kind via duck-typed attributes (a local model participant has `hf_id`; an API one has `provider`).

Parameters

Name	Type	Default	Description
p		*required*	

reasoning_visibility

Module `interlens.reasoning_visibility`

Classes

Name	Summary
`ReasoningVisibility`	Controls whether a participant's prior `<think>` reasoning is re-injected into views on later turns.

ReasoningVisibility

Controls whether a participant's prior `<think>` reasoning is re-injected into views on later turns.

```
ReasoningVisibility()
```

Defined in `interlens.reasoning_visibility` · source

Inherits from: `str`, `Enum`

Reasoning is always parsed out of `Message.content` into `metadata['parsed_think']` (so it is available for interpretability regardless of this setting). This enum only governs *history re-injection*:

- `STRIP` (default): prior reasoning is dropped from all views — matches R1/Qwen3 chat templates, keeps reasoning a genuinely private scratchpad the other participant never sees, and is always template-safe.
- `SELF_RETAIN`: a participant sees its *own* prior reasoning re-injected into its view (visible-scratchpad self-continuation experiments).
- `SHARED`: all participants see all prior reasoning (shared-CoT collaboration/debate experiments).

Non-STRIP modes re-inject reasoning as tagged text at render time so templates that reject a native prior `<think>` turn don't error.

Attributes

Name	Type	Summary
`SELF_RETAIN`		
`SHARED`		
`STRIP`		

runner

Package `interlens.runner`

Modules

- `analyzer_registry`
- `batched`
- `devices`
- `pool` — The execution engine behind `Conversation.rollout` and `interlens.run`.
- `worker_init`

Re-exported

Name	Defined in	Summary
`RunReport`	`interlens.runner.pool`	Aggregate outcome of a run.
`RunResult`	`interlens.runner.pool`	Outcome of one job: the finished `conversation` (weightless participants + completed transcript), its `transcript` and (serializable) `analysis`, or an `error` string if it failed.
`available_devices`	`interlens.runner.devices`	List the devices to spread conversations across: every CUDA GPU, else a single mps/cpu fallback.
`register_analyzer`	`interlens.runner.analyzer_registry`	

Name	Defined in	Summary
register_worker_init	interlens.runner.worker_init	Register a zero-arg callable to run once at worker startup (e.g. to populate the tool/analyzer registries).
resolve_analyzer	interlens.runner.analyzer_registry	Accept either a callable (in-process) or a registered name (spawn-safe) and return the callable.
run	interlens.runner.pool	Run several conversation lineups in ONE pool — the multi-lineup entry point (e.g. a ladder of model pairs × conditions in one overnight job).
run_jobs	interlens.runner.pool	Run (job_id, Conversation) jobs across devices, with checkpointing, resume, and per-job failure isolation.
run_worker_init	interlens.runner.worker_init	

analyzer_registry

Module `interlens.runner.analyzer_registry`

Functions

Name	Summary
`register_analyzer`	
`resolve_analyzer`	Accept either a callable (in-process) or a registered name (spawn-safe) and return the callable.

register_analyzer

```
register_analyzer(name: str, fn) -> object
```

Defined in `interlens.runner.analyzer_registry` · source

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`fn`		*required*	

resolve_analyzer

Accept either a callable (in-process) or a registered name (spawn-safe) and return the callable.

```
resolve_analyzer(analyze)
```

Defined in `interlens.runner.analyzer_registry` · source

Parameters

Name	Type	Default	Description
`analyze`		*required*	

batched

Module `interlens.runner.batched`

Functions

Name	Summary
`co_step`	Co-step `convs` (a shared turn schedule) in lockstep, batching each round's same-position turns.
`schedule_signature`	The full co-step schedule of a conversation: its turn count + the per-position participant signatures.

co_step

Co-step `convs` (a shared turn schedule) in lockstep, batching each round's same-position turns.

```
co_step(
    convs,
    turns: int | None,
    *,
    max_batch_size: int | None = None,
    group_seed: int = 0,
)
```

Defined in `interlens.runner.batched` · source

Each round: every conversation's current speaker is the same schedule position, so their views are gathered and generated in one left-padded batch (sub-batched into `max_batch_size` waves). The representative participant drives the batch — safe because same-schedule participants wrap the *same* cached model + tokenizer. Message hooks run per conversation before commit.

Stop conditions are honored on the batched path too: each conversation's combined stop (its `run_until` / ambient budget, resolved once) can cap the round's `max_new_tokens` (the wave uses the conservative min so no conversation overshoots its budget) and drops a conversation from later rounds once it fires. `turns` may be `None` for a purely stop-driven run (e.g. a matched-compute `TokenBudget`), bounded by a large safety cap.

Parameters

Name	Type	Default	Description
`convs`		*required*	
`turns`	`int | None`	*required*	
`max_batch_size`	`int | None`	`None`	
`group_seed`	`int`	`0`	

schedule_signature

The full co-step schedule of a conversation: its turn count + the per-position participant signatures.

```
schedule_signature(conv, turns: int)
```

Defined in `interlens.runner.batched` · source

Convs that share a signature have an identical speaker/model schedule, so `co_step` can batch each round's same-position turns across them safely. Grouping specs by this (instead of by turn count alone) makes batched execution correct for ANY mix of specs — a heterogeneous lineup simply forms its own group.

Parameters

Name	Type	Default	Description
`conv`		*required*	
`turns`	`int`	*required*	

devices

Module `interlens.runner.devices`

Functions

Name	Summary
`available_devices`	List the devices to spread conversations across: every CUDA GPU, else a single mps/cpu fallback.

available_devices

List the devices to spread conversations across: every CUDA GPU, else a single mps/cpu fallback.

```
available_devices() -> list[str]
```

Defined in `interlens.runner.devices` · source

Multi-GPU parallelism lives *across* conversations (they're independent); within one conversation turns are sequential, so more GPUs never speed up a single conversation — only throughput over many.

pool

Module `interlens.runner.pool`

The execution engine behind `Conversation.rollout` and `interlens.run`.

A *job* is a `(job_id, Conversation)` pair — an unrun, fully-resolved conversation (lazy participants, no dataset). `run_jobs` runs a list of them across devices with checkpointing, resume, per-job failure isolation, and (by default) batched co-stepping within each device. Each finished conversation is returned on its `RunResult` so it can be sampled/inspected afterwards. The analyzer travels ON each conversation (`conv._analyzer`) — a callable in-process, or a registered name across a spawn boundary.

Classes

Name	Summary
RunReport	Aggregate outcome of a run.
RunResult	Outcome of one job: the finished `conversation` (weightless participants + completed transcript), its `transcript` and (serializable) `analysis`, or an `error` string if it failed.

Functions

Name	Summary
run	Run several conversation lineups in ONE pool — the multi-lineup entry point (e.g. a ladder of model pairs × conditions in one overnight job).
run_jobs	Run `(job_id, Conversation)` jobs across devices, with checkpoint-

Name	Summary
	ing, resume, and per-job failure isolation.

RunReport

Aggregate outcome of a run.

```
RunReport(results: dict = dict(), skipped: list = list())
```

Defined in `interlens.runner.pool` · source

Failures are isolated, not fatal: `results` holds every attempted job, `failed` lists the ones that errored, `skipped` lists resume-skipped (already-checkpointed) ids.

Parameters

Name	Type	Default	Description
`results`	`dict`	`dict()`	
`skipped`	`list`	`list()`	

Attributes

Name	Type	Summary
`failed`	`list`	
`results`	`dict`	
`skipped`	`list`	

Methods

`analyses`

```
analyses(self) -> dict
```

source

conversations

```
conversations(self) -> dict
```

source

transcripts

```
transcripts(self) -> dict
```

source

RunResult

Outcome of one job: the finished `conversation` (weightless participants + completed transcript), its `transcript` and (serializable) `analysis`, or an `error` string if it failed.

```
RunResult(
    job_id: str,
    conversation: object = None,
    transcript: object = None,
    analysis: object = None,
    error: str | None = None,
    device: str | None = None,
)
```

Defined in `interlens.runner.pool` · source

`conversation` is the object to `sample()`/inspect after a rollout (the source recipe is never mutated).

Parameters

Name	Type	Default	Description
`job_id`	`str`	*required*	
`conversation`	`object`	`None`	
`transcript`	`object`	`None`	
`analysis`	`object`	`None`	
`error`	`str | None`	`None`	
`device`	`str | None`	`None`	

Attributes

Name	Type	Summary
`analysis`	`object`	
`conversation`	`object`	
`device`	`str | None`	

Name	Type	Summary
`error`	`str | None`	
`job_id`	`str`	
`tokens_generated`	`int`	Total generated tokens in this conversation (summed from each turn's `metadata['n_tokens']`) — the realized compute, for verifying matched-compute comparisons. 0 if the job failed.
`transcript`	`object`	

run

Run several conversation lineups in ONE pool — the multi-lineup entry point (e.g. a ladder of model pairs × conditions in one overnight job).

```
run(
    conversations,
    devices=None,
    out_dir=None,
    resume=False,
    batched=True,
    max_batch_size=None,
) -> RunReport
```

Defined in `interlens.runner.pool` · source

Each conversation is expanded to its jobs (one per `data()` row if it has data, else a single conversation), with job ids namespaced by that conversation's `name` (default `conv{i}`) so ids stay unique and resumable. All jobs go into one pool, so GPUs stay packed across lineups (no idle tail between sequential rollouts) under a single `out_dir`/resume namespace, returning one merged `RunReport`. Mixing lineups is safe: batched co-stepping groups by schedule signature, so each distinct lineup forms its own batch group.

`Conversation.rollout` is the single-lineup sugar for this (`run([conv], ...)`-equivalent via `run_jobs`).

Parameters

Name	Type	Default	Description
`conversations`		*required*	
`devices`		`None`	
`out_dir`		`None`	
`resume`		`False`	
`batched`		`True`	
`max_batch_size`		`None`	

run_jobs

Run `(job_id, Conversation)` jobs across devices, with checkpointing, resume, and per-job failure isolation.

```
run_jobs(
    jobs,
    devices=None,
    out_dir=None,
    resume=False,
    batched=True,
    max_batch_size=None,
) -> RunReport
```

Defined in `interlens.runner.pool` · source

Parallel by default on **two** axes: one worker **process per device** (jobs round-robined; multi-GPU spawns via `torch.multiprocessing` since fork+CUDA is broken — a single device runs in-process), and within each device **batched co-stepping** (`batched=True`). Jobs are grouped by co-step schedule signature so same-schedule jobs batch into one `model.generate` — correct for ANY mix. `batched=False` gives the DETERMINISTIC path.

Parameters

Name	Type	Default	Description
`jobs`		*required*	
`devices`		`None`	
`out_dir`		`None`	
`resume`		`False`	
`batched`		`True`	
`max_batch_size`		`None`	

worker_init

Module `interlens.runner.worker_init`

Functions

Name	Summary
`register_worker_init`	Register a zero-arg callable to run once at worker startup (e.g. to populate the tool/analyzer registries).
`run_worker_init`	

register_worker_init

Register a zero-arg callable to run once at worker startup (e.g. to populate the tool/analyzer registries).

```
register_worker_init(fn) -> object
```

Defined in `interlens.runner.worker_init` · source

Parameters

Name	Type	Default	Description
`fn`		*required*	

run_worker_init

```
run_worker_init() -> None
```

Defined in `interlens.runner.worker_init` · source

stop

Package `interlens.stop`

Modules

- `conditions`
- `stop_condition`

Re-exported

Name	Defined in	Summary
`AnyStopCondition`	`interlens.stop.stop_condition`	Fires when *any* member condition fires.
`ElapsedTimeStopCondition`	`interlens.stop.conditions`	Stop once `seconds` of wall-clock have elapsed since the run started (monotonic clock).
`StopCondition`	`interlens.stop.stop_condition`	A stateful predicate that ends a `Conversation.run` early.
`StopStringCondition`	`interlens.stop.conditions`	Stop when a committed message's visible `content` contains any of the given strings (a done-signal).
`TokenBudget`	`interlens.stop.conditions`	A per-conversation compute budget — the matched-compute primitive for fair solo-vs-pair comparisons.

Name	Defined in	Summary
`TokenStopCondition`	`interlens.stop.conditions`	Stop once the total generated tokens across turns reach `max_tokens`.
`TurnStopCondition`	`interlens.stop.conditions`	Stop after `max_turns` committed turns.
`active_stop_conditions`	`interlens.stop.stop_condition`	The stop conditions currently installed by enclosing `with condition:` blocks (outermost first).

conditions

Module `interlens.stop.conditions`

Classes

Name	Summary
`ElapsedTimeStopCondition`	Stop once `seconds` of wall-clock have elapsed since the run started (monotonic clock).
`StopStringCondition`	Stop when a committed message's visible `content` contains any of the given strings (a done-signal).
`TokenBudget`	A per-conversation compute budget — the matched-compute primitive for fair solo-vs-pair comparisons.
`TokenStopCondition`	Stop once the total generated tokens across turns reach `max_tokens`.
`TurnStopCondition`	Stop after `max_turns` committed turns.

ElapsedTimeStopCondition

Stop once `seconds` of wall-clock have elapsed since the run started (monotonic clock).

```
ElapsedTimeStopCondition(seconds: float)
```

Defined in `interlens.stop.conditions` · source

Inherits from: StopCondition

Parameters

Name	Type	Default	Description
`seconds`	`float`	*required*	

Attributes

Name	Type	Summary
`seconds`		
`start`		

Methods

`reset`

```
reset(self) -> None
```

source

should_stop

```
should_stop(self, conversation: 'Conversation', last_message: 'Message') -
    > bool
```

source

Parameters

Name	Type	Default	Description
conversation	'Conversation'	*required*	
last_message	'Message'	*required*	

StopStringCondition

Stop when a committed message's visible `content` contains any of the given strings (a done-signal).

```
StopStringCondition(strings)
```

Defined in `interlens.stop.conditions` · source

Inherits from: StopCondition

Parameters

Name	Type	Default	Description
`strings`		*required*	

Attributes

Name	Type	Summary
`strings`		

Methods

`should_stop`

```
should_stop(self, conversation: 'Conversation', last_message: 'Message') -
    > bool
```

source

Parameters

Name	Type	Default	Description
`conversation`	`'Conversation'`	*required*	

Name	Type	Default	Description
`last_message`	`'Message'`	*required*	

TokenBudget

A per-conversation compute budget — the matched-compute primitive for fair solo-vs-pair comparisons.

```
TokenBudget(per_conversation: int | None = None, per_turn: int | None =
    None)
```

Defined in `interlens.stop.conditions` · source

Inherits from: StopCondition

`per_conversation` stops a conversation once ITS OWN cumulative generated tokens reach the budget, and `per_turn` caps each individual turn so the allowance is spread across real conversation turns rather than consumed by one monologue. Both are enforced via `turn_cap` too: the run loop shrinks the next generation to `min(speaker cap, per_turn, per_conversation - spent)`, so the budget is respected without overshoot.

The budget is per-conversation, not a shared pool. Spend is read from the conversation's own transcript (`metadata['n_tokens']`), so the condition is stateless — in a rollout of N copies, each copy independently gets the full budget. Cheap to count (never re-tokenizes) and trivially picklable, so it works installed directly (`run_until=TokenBudget(...)`) or ambiently (`with TokenBudget(per_conversation=200): conv.rollout()`).

Parameters

Name	Type	Default	Description
`per_conversation`	`int | None`	`None`	
`per_turn`	`int | None`	`None`	

Attributes

Name	Type	Summary
`per_conversation`		

Name	Type	Summary
`per_turn`		

Methods

`should_stop`

```
should_stop(self, conversation: 'Conversation', last_message: 'Message') -
    > bool
```

source

Parameters

Name	Type	Default	Description
`conversation`	`'Conversation'`	*required*	
`last_message`	`'Message'`	*required*	

`turn_cap`

```
turn_cap(self, conversation: 'Conversation') -> int | None
```

source

Parameters

Name	Type	Default	Description
`conversation`	`'Conversation'`	*required*	

TokenStopCondition

Stop once the total generated tokens across turns reach `max_tokens`.

```
TokenStopCondition(max_tokens: int)
```

Defined in `interlens.stop.conditions` · source

Inherits from: StopCondition

The per-turn count comes from `Message.metadata['n_tokens']`, which `ModelParticipant.generate` records — so the source of truth is defined, not guessed. Turns without a count (e.g. seeded messages) contribute 0.

Parameters

Name	Type	Default	Description
`max_tokens`	`int`	*required*	

Attributes

Name	Type	Summary
`max_tokens`		
`total`		

Methods

`reset`

```
reset(self) -> None
```

source

should_stop

```
should_stop(self, conversation: 'Conversation', last_message: 'Message') -
    > bool
```

source

Parameters

Name	Type	Default	Description
conversation	'Conversation'	*required*	
last_message	'Message'	*required*	

TurnStopCondition

Stop after `max_turns` committed turns.

```
TurnStopCondition(max_turns: int)
```

Defined in `interlens.stop.conditions` · source

Inherits from: StopCondition

Parameters

Name	Type	Default	Description
`max_turns`	`int`	*required*	

Attributes

Name	Type	Summary
`count`		
`max_turns`		

Methods

`reset`

```
reset(self) -> None
```

source

`should_stop`

```
should_stop(self, conversation: 'Conversation', last_message: 'Message') -
    > bool
```

source

Parameters

Name	Type	Default	Description
`conversation`	`'Conversation'`	*required*	
`last_message`	`'Message'`	*required*	

stop_condition

Module `interlens.stop.stop_condition`

Classes

Name	Summary
`AnyStopCondition`	Fires when *any* member condition fires.
`StopCondition`	A stateful predicate that ends a `Conversation.run` early.

Functions

Name	Summary
`active_stop_conditions`	The stop conditions currently installed by enclosing `with condition:` blocks (outermost first).

AnyStopCondition

Fires when *any* member condition fires.

```
AnyStopCondition(conditions: list[StopCondition])
```

Defined in `interlens.stop.stop_condition` · source

Inherits from: StopCondition

`run(until=[...])` wraps a list in this.

Parameters

Name	Type	Default	Description
`conditions`	list[StopCondition]	*required*	

Attributes

Name	Type	Summary
`conditions`		

Methods

`reset`

```
reset(self) -> None
```

source

should_stop

```
should_stop(self, conversation: 'Conversation', last_message: 'Message') -
    > bool
```

source

Parameters

Name	Type	Default	Description
conversation	'Conversation'	*required*	
last_message	'Message'	*required*	

turn_cap

```
turn_cap(self, conversation: 'Conversation') -> int | None
```

source

Parameters

Name	Type	Default	Description
conversation	'Conversation'	*required*	

StopCondition

A stateful predicate that ends a `Conversation.run` early.

```
StopCondition()
```

Defined in `interlens.stop.stop_condition` · source

Inherits from: `ABC`

Each condition is **stateful** (tracks its own counters) and is checked after every committed turn via `should_stop(conversation, last_message)`. `reset()` clears state and is called at the start of each `run` (and branches get fresh copies), so one instance can be reused across runs without leaking state.

A condition may also **cap the next generation** via `turn_cap` (e.g. a token budget shrinks the last turn so it lands exactly on budget) and may be installed **ambiently** as a context manager (`with TokenBudget(...): conv.rollout(...)` applies it to every conversation in the block). New conditions subclass this without any change to `run`.

Methods

reset

```
reset(self) -> None
```

source

Clear any accumulated state.

Default no-op for stateless conditions.

should_stop

```
should_stop(self, conversation: 'Conversation', last_message: 'Message') -
    > bool
```

source

Parameters

Name	Type	Default	Description
`conversation`	`'Conversation'`	*required*	
`last_message`	`'Message'`	*required*	

turn_cap

```
turn_cap(self, conversation: 'Conversation') -> int | None
```

source

An upper bound on the NEXT turn's generated tokens, or `None` for no cap.

The run loop passes it as `max_new_tokens` (bounded by the speaker's own cap), so a budget condition can prevent both overshoot and a single turn from consuming the whole allowance. Default: no cap.

Parameters

Name	Type	Default	Description
`conversation`	`'Conversation'`	*required*	

active_stop_conditions

The stop conditions currently installed by enclosing `with condition:` blocks (outermost first).

```
active_stop_conditions() -> tuple['StopCondition', ...]
```

Defined in `interlens.stop.stop_condition` · source

templating

Module `interlens.templating`

Per-row templating for data-driven rollouts.

A `Conversation` run over a benchmark shares ONE recipe but needs a DIFFERENT scenario per dataset row. Any templated string field on the conversation (`shared_context`, `shared_system_prompt`) or its participants (`system_prompt`, private context) may be a *template value* that is resolved against each row at rollout expansion time (once, deterministically — so job *i* always maps to row *i* and resume stays correct).

A template value is any of: - a plain `str` — used as-is (no substitution); - `dataset_field("question")` — pulls `row["question"]`; - a `callable(row) → str` — arbitrary per-row computation; - a `tuple`/`list` of the above — resolved part-by-part and concatenated (the <3.14 way to interleave literal text with fields: `("Solve:\n\n", dataset_field("question"))`); - a PEP 750 template string (`t"…{dataset_field('question')}…"`) on Python 3.14+ — interpolated `DatasetField`s resolve from the row, everything else formats normally.

`resolve(value, row)` performs the substitution; `has_fields(value)` reports whether a value depends on the row at all (so a conversation with no data-dependent fields can be run directly, and one WITH them refuses to run un-expanded — see `Conversation`).

Classes

Name	Summary
`DatasetField`	A placeholder for `row[name]` in a templated field, created by :func:`dataset_field`.

Functions

Name	Summary
`dataset_field`	Reference a dataset column in a templated field: `shared_context=("Solve:\n\n", dataset_field("question"))` resolves `row["question"]` for each row at rollout expansion.
`has_fields`	Whether `value` depends on the dataset row (contains a `DatasetField`, a callable, or a t-string).
`resolve`	Resolve a template `value` against one dataset `row` (a mapping) to a concrete value.

DatasetField

A placeholder for `row[name]` in a templated field, created by :func:`dataset_field`.

```
DatasetField(name: str)
```

Defined in `interlens.templating` · source

Frozen + tiny so it serializes and pickles trivially across a spawn boundary.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	

Attributes

Name	Type	Summary
`name`	`str`	

dataset_field

Reference a dataset column in a templated field: `shared_context=("Solve: \n\n", dataset_field("question"))` resolves `row["question"]` for each row at rollout expansion.

```
dataset_field(name: str) -> DatasetField
```

Defined in `interlens.templating` · source

See the module docstring for all template forms.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	

has_fields

Whether `value` depends on the dataset row (contains a `DatasetField`, a callable, or a t-string).

```
has_fields(value) -> bool
```

Defined in `interlens.templating` · source

A conversation whose framing has no fields can be run directly; one that has them must be expanded over data.

Parameters

Name	Type	Default	Description
`value`		*required*	

resolve

Resolve a template `value` against one dataset `row` (a mapping) to a concrete value.

```
resolve(value, row: dict)
```

Defined in `interlens.templating` · source

Non-templated values (plain `str`/`None`/other) pass through unchanged; `DatasetField` → `row[name]`; a callable → `value(row)`; a tuple/list → each part resolved and string-joined; a t-string → interpolated with `DatasetField` interpolations pulled from the row.

Parameters

Name	Type	Default	Description
`value`		*required*	
`row`	`dict`	*required*	

tools

Package `interlens.tools`

Modules

- `registry`
- `tool`
- `tool_call`

Re-exported

Name	Defined in	Summary
`DEFAULT_REGISTRY`	`interlens.tools.registry`	
`Tool`	`interlens.tools.tool`	A capability a participant can invoke during its turn.
`ToolCall`	`interlens.tools.tool_call`	A parsed request from the model to invoke a tool: the tool `name` and its `arguments`.
`ToolRegistry`	`interlens.tools.registry`	Resolves tool *names* (which serialize) to live `Tool` instances (which don't).
`ToolResult`	`interlens.tools.tool_call`	The outcome of executing a `ToolCall`: the tool `name` and its string `output` (or error text).

registry

Module `interlens.tools.registry`

Attributes

Name	Type	Summary
`DEFAULT_REGISTRY`		

Classes

Name	Summary
`ToolRegistry`	Resolves tool *names* (which serialize) to live `Tool` instances (which don't).

ToolRegistry

Resolves tool *names* (which serialize) to live `Tool` instances (which don't).

```
ToolRegistry()
```

Defined in `interlens.tools.registry` · source

This is the tools analogue of the model registry: a template stores `tool_names` and, at `build` time on each worker, the registry turns them into callables. Because spawned worker processes inherit no parent state, tools must be registered at import time (or via a worker-init hook), not imperatively in the parent.

Methods

`register`

```
register(self, tool: Tool) -> Tool
```

source

Parameters

Name	Type	Default	Description
`tool`	Tool	*required*	

`resolve`

```
resolve(self, names) -> list[Tool]
```

source

Parameters

Name	Type	Default	Description
`names`		*required*	

tool

Module `interlens.tools.tool`

Classes

Name	Summary
`Tool`	A capability a participant can invoke during its turn.

Tool

A capability a participant can invoke during its turn.

```
Tool()
```

Defined in `interlens.tools.tool` · source

Inherits from: `ABC`

A tool has a `name`, a JSON `schema` (in the function-calling format chat templates render via their `tools=` argument), and is callable with keyword arguments to produce a string result. Tools contain live callables and so are NOT serializable — templates store tool *names* and resolve them against a `ToolRegistry` at build time, mirroring how models are resolved from ids.

Attributes

Name	Type	Summary
`name`	`str`	
`schema`	`dict`	The tool's JSON function schema, e.g. `{"type": "function", "function": {"name", "description", "parameters": {...}}}`, passed straight to `apply_chat_template(tools=...)` so each family renders it in its native format.

tool_call

Module `interlens.tools.tool_call`

Classes

Name	Summary
`ToolCall`	A parsed request from the model to invoke a tool: the tool `name` and its `arguments`.
`ToolResult`	The outcome of executing a `ToolCall`: the tool `name` and its string `output` (or error text).

ToolCall

A parsed request from the model to invoke a tool: the tool `name` and its `arguments`.

```
ToolCall(name: str, arguments: dict = dict(), raw: str = '')
```

Defined in `interlens.tools.tool_call` · source

`raw` keeps the exact text the model emitted (useful for debugging a family parser). Tool calls are parsed out of the generation by a per-family `parse_tool_calls` — the format differs across model families, so the parsed structure is uniform even though the surface syntax isn't.

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`arguments`	`dict`	`dict()`	
`raw`	`str`	`''`	

Attributes

Name	Type	Summary
`arguments`	`dict`	
`name`	`str`	
`raw`	`str`	

ToolResult

The outcome of executing a `ToolCall`: the tool `name` and its string `output` (or error text).

```
ToolResult(name: str, output: str, error: bool = False)
```

Defined in `interlens.tools.tool_call` · source

Parameters

Name	Type	Default	Description
`name`	`str`	*required*	
`output`	`str`	*required*	
`error`	`bool`	`False`	

Attributes

Name	Type	Summary
`error`	`bool`	
`name`	`str`	
`output`	`str`	

transcript

Module `interlens.transcript`

Attributes

Name	Type	Summary
`MessageRef`		
`SCHEMA_VERSION`		

Classes

Name	Summary
`Transcript`	The canonical, perspective-neutral record of a conversation, plus the logic to render it *from* a given participant's perspective.

Transcript

The canonical, perspective-neutral record of a conversation, plus the logic to render it *from* a given participant's perspective.

```
Transcript(messages: list[Message] = list())
```

Defined in `interlens.transcript` · source

Design: the transcript never commits to `assistant`/`user` roles. Each message just knows its `author`. When it's a participant's turn, `render_roles` maps that neutral record into the chat-template `[{role, content}]` shape *from that participant's point of view* — its own turns become `self_role` (normally `assistant`), everyone else's become `others_role` (normally `user`). This single trick is why two different models with different tokenizers can share one transcript: each gets the view its own template expects, and the stored record stays singular.

P0 scope: this holds only the message list + rendering + list-like ergonomics. Serialization, context policies, author-labelling for N-party, and reasoning re-injection are layered on in later phases.

Parameters

Name	Type	Default	Description
`messages`	list[Message]	`list()`	

Attributes

Name	Type	Summary
`messages`	list[Message]	

Methods

append

```
append(self, author: str, content: str, **metadata={}) -> Message
```

source

Append a committed turn and return it.

`metadata` captures anything non-authoritative (parsed reasoning, logprobs, tool trails) without polluting `content`.

Parameters

Name	Type	Default	Description
`author`	`str`	*required*	
`content`	`str`	*required*	
`metadata`		`{}`	

clear

```
clear(self) -> 'Transcript'
```

source

Drop **every** turn, returning `self`.

⚠ This wipes the whole record, including any leading seed turn — the `shared_context` scenario framing and any moderator/initial-instruction turns injected at construction. Nothing about the conversation's opening setup survives. If you want to clear the *dialogue* but keep that framing (e.g. to rerun the same scenario), call `Conversation.reset` instead, which empties and then re-seeds the `shared_context` turn.

copy

```
copy(self) -> 'Transcript'
```

source

Deep-ish copy of the message list for branching.

Messages are small string records, so this is cheap; the invariant that keeps it cheap is that heavy tensors never live in `Message.metadata`.

edit

```
edit(
    self,
    ref: 'MessageRef',
    content: str | None = None,
    *,
    author: str | None = None,
    **metadata={},
) -> Message
```

source

Edit a committed past turn in place and return it.

`ref` is a `MessageRef` (int index, negatives allowed, or a `Message` object matched by identity). Only the arguments you pass are changed: `content` and/or `author` are replaced when given; `metadata` keys are merged over the existing metadata (pass an explicit key to overwrite it — untouched keys are left alone). Editing the `Message` object's fields directly does the same thing, since the transcript holds it by reference.

Parameters

Name	Type	Default	Description
`ref`	`'MessageRef'`	*required*	
`content`	`str | None`	`None`	
`author`	`str | None`	`None`	

Name	Type	Default	Description
metadata		{}	

from_dict

```
from_dict(cls, data: dict) -> 'Transcript'
```

source

Parameters

Name	Type	Default	Description
data	dict	*required*	

load

```
load(cls, path: str | Path) -> 'Transcript'
```

source

Load a transcript from disk.

Works with no models present (for scoring/analysis of saved runs).

Parameters

Name	Type	Default	Description
path	str | Path	*required*	

pretty

```
pretty(self, metadata: bool = False) -> str
```

source

A human-readable dump for debugging: one `[i] author: content` block per turn.

With `metadata=True`, also list each turn's non-empty metadata (parsed reasoning, tool trail, token counts, …). Also what `str(transcript)` / `print(transcript)` uses (without metadata).

Parameters

Name	Type	Default	Description
`metadata`	`bool`	`False`	

render_roles

```
render_roles(self, *, pov: 'Participant') -> list[dict]
```

source

Render **the transcript turns only** as `[{"role", "content"}]` from `pov`'s perspective (`pov` is keyword-only: `render_roles(pov=alice)`).

Its own turns become `self_role` (normally `assistant`), everyone else's become `others_role` (normally `user`). For a "what if X had just been said" render, extend first with `with_extra` and render the result.

Limitation — this is NOT what the model actually sees. The `Transcript` is the perspective-neutral record; it knows nothing about a participant's `system_prompt` / `private_context`, the conversation's `shared_system_prompt` / `shared_context` framing, the `context_policy` fitting, or the family-specific flatten (Gemma system-folding etc.). Those are added by the `Conversation` pipeline. For the **real** generation input use `Conversation.view(pov)` (or `Conversation.render_templated(pov)` for the templated string); use this method for lower-level inspection of the record itself.

Parameters

Name	Type	Default	Description
pov	'Participant'	*required*	

render_templated

```
render_templated(
    self,
    *,
    pov: 'ModelParticipant',
    add_generation_prompt: bool = False,
    tokenize: bool = False,
)
```

source

Template **the transcript turns only** from pov's point of view — i.e. render_roles role-swapped, then run through pov's tokenizer chat template so the special / control tokens (◁im_start▷assistant etc.) are included.

pov is keyword-only: call render_templated(pov=alice). Returns a str (tokenize=False, default) or token ids (tokenize=True); add_generation_prompt appends the assistant open tag.

Prefer Conversation.render_templated if you have access to a conversation: it adds the system prompt, private context, context-fitting, and family-specific flattening, printing the exact input models see.

Limitations

NOT the exact model input. This omits the system prompt, private context, context-fitting, and the family-specific flatten that a real turn also gets (see render_roles); it also renders the raw role-swapped turns as-is, which can even *raise* for families needing strict alternation / system-folding (e.g. Gemma) since the merge isn't applied. For the **exact, family-correct prompt the model actually sees**, use Conversation.render_templated(pov). This method is a lower-level inspection of the record. Requires a local model participant (with a tokenizer); an API participant has none and raises.

Parameters

Name	Type	Default	Description
pov	'ModelParticipant'	*required*	
add_generation_prompt	bool	False	
tokenize	bool	False	

resolve_index

```
resolve_index(self, ref: 'MessageRef') -> int
```

source

Normalize a message reference to a concrete `[0, len)` position.

`ref` is either an `int` index (Python semantics — negatives count from the end, `-1` = last turn) or a `Message` **object**, located by identity (`is`, not equality — two turns with the same text are still distinct). Raises `IndexError` for an out-of-range int and `ValueError` for a message that isn't in this transcript.

Parameters

Name	Type	Default	Description
ref	'MessageRef'	*required*	

rewind

```
rewind(self, *, to: 'MessageRef') -> 'Transcript'
```

source

Rewind so the turn referenced by `to` becomes the new last turn — i.e. drop everything **after** it, keeping `to` itself (mutates in place, returns `self`).

`to` is a `MessageRef`: an `int` index (negatives count from the end — `rewind(to=-2)` takes back the single last turn) or a `Message` object matched by identity. To drop all turns use `clear`.

Parameters

Name	Type	Default	Description
`to`	`'MessageRef'`	*required*	

save

```
save(self, path: str | Path) -> None
```

source

Parameters

Name	Type	Default	Description
`path`	`str | Path`	*required*	

to_dict

```
to_dict(self) -> dict
```

source

truncate

```
truncate(self, length: int) -> 'Transcript'
```

source

Keep only the first `length` turns, dropping everything after (mutates in place, returns `self`).

`length` is a *count*, clamped to `[0, len]`: over-long values are a no-op and negatives count from the end (`-1` keeps all but the last turn). To cut relative to a specific message, use `resolve_index` / `rewind(to=…)` instead.

Parameters

Name	Type	Default	Description
`length`	`int`	*required*	

with_extra

```
with_extra(self, *extra: Message = ()) -> 'Transcript'
```

source

Return a lightweight, ephemeral **read-only** `Transcript` = this one's messages followed by `extra`, backed by a lazy concatenation (`_ConcatMessages`): the base message list is **not copied** — only `extra` is materialized, so this is O(len(extra)), not O(len(transcript)), and `Message` objects are shared by reference.

The original is untouched.

This is the data-layer primitive behind ephemeral sampling ("what would a participant say if `extra` had just been said?"): build the extended transcript, render it, discard it — `transcript.with_extra(Message("bob", "…")).render_roles(pov=alice)`. The result is read-only (rendering, `len`, indexing, iteration all work; appending raises — `copy()` first if you need a mutable one).

(A one-shot generator was rejected: `Transcript` needs `len` / indexing / repeated iteration, which a generator can't provide.)

Parameters

Name	Type	Default	Description
`extra`	Message	`()`	

usage

Module `interlens.usage`

Usage accounting: token/cost metering for hosted-API participants.

Hosted-API turns cost real money, and a large rollout can outrun a budget *while it is running* — the failure mode this module exists to prevent is N concurrent conversations each individually under budget that together blow past the cap before any of them finishes. Three pieces:

- **Per-turn usage** is recorded by `APIParticipant` into `Message.metadata` (`n_tokens`, `n_tokens_in`, `cost_usd`, `stop_reason`) — the same convention `ModelParticipant` uses for `n_tokens`, so `TokenBudget` and transcript-level aggregation work identically for local and hosted turns.
- **`UsageMeter`** is the live, run-level ledger: every metered participant reports each API call into one shared meter, which tracks cumulative dollars per model, supports **reservation-style gating** (claim an estimated cost *before* launching work, so concurrent conversations cannot collectively overrun the cap), and optionally persists to disk after every add (crash-safe: a restarted run resumes the ledger).
- **`CostBudget`** is the dollar-denominated `StopCondition`: the per-conversation analogue of `TokenBudget`, reading each committed turn's recorded `metadata['cost_usd']`.

Pricing is a $/Mtok table keyed by model id. The bundled defaults are deliberately conservative (high) so the meter over- rather than under-counts; register exact prices with `register_pricing` or pass a table to the meter. An unknown model uses `FALLBACK_PRICING` (higher than any bundled entry) rather than silently costing $0.

Attributes

Name	Type	Summary
`CACHE_READ_MULTIPLIER`		

Name	Type	Summary
`CACHE_WRITE_MULTIPLIERS`		
`DEFAULT_PRICING`	`dict[str, dict[str, float]]`	
`FALLBACK_PRICING`		

Classes

Name	Summary
`CostBudget`	A per-conversation **dollar** budget — the cost-denominated sibling of `TokenBudget`.
`UsageMeter`	A cumulative, thread-safe dollar ledger shared by every metered participant in a run.

Functions

Name	Summary
`register_pricing`	Register (or override) the $/Mtok pricing for `model_id`, process-wide.
`resolve_pricing`	The effective pricing table: bundled defaults, overlaid with `register_pricing` entries, overlaid with the caller's `pricing` dict (each value `{"in": $/Mtok, "out": $/Mtok}`).
`transcript_usage`	Aggregate the recorded usage of one transcript: total generated/input tokens, dollar cost, and a per-author breakdown — read from each committed message's

Name	Summary
	metadata (`n_tokens` / `n_tokens_in` / `cost_usd`), the same source of truth `TokenBudget` and `CostBudget` use.

CostBudget

A per-conversation **dollar** budget — the cost-denominated sibling of `TokenBudget`.

```
CostBudget(per_conversation: float)
```

Defined in `interlens.usage` · source

Inherits from: StopCondition

Stops a conversation once its own cumulative recorded turn cost (`metadata['cost_usd']`, written by metered `APIParticipant`s) reaches `per_conversation` dollars. Stateless like `TokenBudget` (spend is re-read from the transcript), so it works installed directly or ambiently and each rollout copy independently gets the full budget. Turns without a cost record (local models, scripted participants) contribute $0 — this budget only constrains metered hosted-API spend.

Parameters

Name	Type	Default	Description
`per_conversation`	`float`	*required*	

Attributes

Name	Type	Summary
`per_conversation`		

Methods

should_stop

```
should_stop(self, conversation: 'Conversation', last_message: 'Message') -
    > bool
```

source

Parameters

Name	Type	Default	Description
conversation	'Conversation'	*required*	
last_message	'Message'	*required*	

UsageMeter

A cumulative, thread-safe dollar ledger shared by every metered participant in a run.

```
UsageMeter(
    budget: float | None = None,
    *,
    path: str | Path | None = None,
    pricing: dict | None = None,
)
```

Defined in `interlens.usage` · source

`add` records one API call (tokens in/out at `price_multiplier` — 0.5 for batch-API-served responses) and returns that call's cost. `reserve`/`settle` implement reservation-style gating: claim an estimated cost *before* launching an episode/conversation so concurrent work cannot collectively outrun `budget` (the post-overrun fix from the arena experiments: a pure post-hoc meter let in-flight episodes blow past the cap). `exhausted` is the launch gate — in-flight work finishes, new work doesn't start.

With `path=` the ledger is persisted atomically after every add, so a crashed/restarted run resumes with its spend intact. Refusal telemetry rides along: `add(..., refusal=True)` counts hosted-API refusals per model (a seat-selective refusal pattern silently biases multi-agent results; the counter makes it visible).

Parameters

Name	Type	Default	Description
`budget`	`float | None`	`None`	
`path`	`str | Path | None`	`None`	
`pricing`	`dict | None`	`None`	

Attributes

Name	Type	Summary
budget		
by_model	dict[str, dict]	
exhausted	bool	True once actual spend has reached the budget — the gate for launching NEW work (in-flight work is allowed to finish; that is the reservation's job to have pre-counted).
path		
pricing		
reserved_usd		
total_usd		

Methods

add

```python
add(
    self,
    model: str,
    tokens_in: int,
    tokens_out: int,
    *,
    price_multiplier: float = 1.0,
    refusal: bool = False,
    cache_read_tokens: int = 0,
    cache_write_tokens: int = 0,
    cache_ttl: str = '5m',
) -> float
```

source

Record one completed API call; returns its cost in dollars.

`price_multiplier` scales the price for discounted billing paths (0.5 = provider batch API). Thread-safe; persists when `path` is set.

The cache counters are recorded per model as well as priced, because a run's cache HIT RATE is what tells a campaign whether its prompt-cache breakpoints are actually placed on a stable prefix — and the per-turn metadata is not aggregated anywhere else. `m["in"]` accumulates the WHOLE prompt (uncached + read + written) so a prompt-size series stays comparable across cached and uncached runs; the split lives in `m["cache_read"]` / `m["cache_write"]`.

Parameters

Name	Type	Default	Description
`model`	`str`	*required*	
`tokens_in`	`int`	*required*	
`tokens_out`	`int`	*required*	
`price_multiplier`	`float`	`1.0`	
`refusal`	`bool`	`False`	
`cache_read_tokens`	`int`	`0`	
`cache_write_tokens`	`int`	`0`	
`cache_ttl`	`str`	`'5m'`	

cache_report

```
cache_report(self) -> dict
```

source

Per-model prompt-cache accounting: `{model: {prompt_tokens, cache_read, cache_write, uncached, hit_rate, saved_usd}}`.

`hit_rate` is cache reads over the whole prompt volume — the honest denominator, since a write is a prefix the cache did NOT serve. `saved_usd` is what the

same token volume would have cost with no caching minus what it did cost, so a run whose breakpoints sit on an unstable prefix reports a NEGATIVE saving (writes at a premium, never read) rather than a flattering zero.

`price`

```
price(
    self,
    model: str,
    tokens_in: int,
    tokens_out: int,
    *,
    cache_read_tokens: int = 0,
    cache_write_tokens: int = 0,
    cache_ttl: str = '5m',
) -> float
```

source

Dollar cost of one call at full (non-batch) price.

Unknown models use `FALLBACK_PRICING`.

`tokens_in` is the prompt portion billed at the full input rate — for a cached Anthropic call that is the provider's `usage.input_tokens`, which EXCLUDES both cache reads and cache writes. The cached portions are priced separately off the same input rate at :data:`CACHE_READ_MULTIPLIER` and :data:`CACHE_WRITE_MULTIPLIERS` keyed by `cache_ttl`, so the three never double-count each other.

Parameters

Name	Type	Default	Description
`model`	`str`	*required*	
`tokens_in`	`int`	*required*	
`tokens_out`	`int`	*required*	
`cache_read_tokens`	`int`	`0`	
`cache_write_tokens`	`int`	`0`	
`cache_ttl`	`str`	`'5m'`	

reserve

```
reserve(self, usd: float) -> bool
```

source

Claim `usd` of estimated future spend.

Returns `False` (claiming nothing) when spent + already reserved + this claim would exceed the budget — the caller should then not launch the work. With no `budget`, reservations always succeed (the ledger still tracks them).

Parameters

Name	Type	Default	Description
`usd`	`float`	*required*	

settle

```
settle(self, usd: float) -> None
```

source

Release a prior reservation (call once the work's actual spend has been metered via `add`).

Parameters

Name	Type	Default	Description
`usd`	`float`	*required*	

snapshot

```
snapshot(self) -> dict
```

source

A JSON-serializable copy of the ledger: totals, reservations, and the per-model breakdown.

summary

```
summary(self) -> str
```

source

A printable run-spend summary: one line per model (calls, tokens, refusals, dollars) plus the total against the budget.

register_pricing

Register (or override) the $/Mtok pricing for `model_id`, process-wide.

```
register_pricing(model_id: str, *, input_per_mtok: float, output_per_mtok:
    float) -> None
```

Defined in `interlens.usage` · source

Meters constructed afterwards pick it up; a meter-local `pricing=` table still wins for that meter.

Parameters

Name	Type	Default	Description
`model_id`	`str`	*required*	
`input_per_mtok`	`float`	*required*	
`output_per_mtok`	`float`	*required*	

resolve_pricing

The effective pricing table: bundled defaults, overlaid with `register_pricing` entries, overlaid with the caller's `pricing` dict (each value `{"in": $/Mtok, "out": $/Mtok}`).

```
resolve_pricing(pricing: dict | None = None) -> dict[str, dict[str,
    float]]
```

Defined in `interlens.usage` · source

Parameters

Name	Type	Default	Description
`pricing`	`dict | None`	`None`	

transcript_usage

Aggregate the recorded usage of one transcript: total generated/input tokens, dollar cost, and a per-author breakdown — read from each committed message's metadata (`n_tokens` / `n_tokens_in` / `cost_usd`), the same source of truth `TokenBudget` and `CostBudget` use.

```
transcript_usage(transcript: 'Transcript | Iterable[Message]') -> dict
```

Defined in `interlens.usage` · source

Turns without records (seeded/moderator/scripted) contribute 0.

Parameters

Name	Type	Default	Description
`transcript`	`'Transcript | Iterable[Message]'`	*required*	

view

Module `interlens.view`

Attributes

Name	Type	Summary
`Origin`		

Classes

Name	Summary
`ViewSegment`	One entry in a participant's *structured* view, before it is flattened to the `[{role, content}]` shape a chat template consumes.

ViewSegment

One entry in a participant's *structured* view, before it is flattened to the `[{role, content}]` shape a chat template consumes.

```
ViewSegment(role: Role, content: str, origin: Origin, author: str | None
    = None)
```

Defined in `interlens.view` · source

Carrying `origin` (and `author` for turns) through the pipeline is the whole point: it keeps the semantic identity of each piece intact so context-trimming and author-labelling can act on it, rather than reverse-engineering meaning out of already-folded text.

Parameters

Name	Type	Default	Description
`role`	`Role`	*required*	
`content`	`str`	*required*	
`origin`	`Origin`	*required*	
`author`	`str | None`	`None`	

Attributes

Name	Type	Summary
`author`	`str | None`	
`content`	`str`	
`origin`	`Origin`	
`role`	`Role`	

Methods

as_message

```
as_message(self) -> dict
```

source

Flatten to the `{"role", "content"}` dict a chat template expects.

© 2026 Siddharth M. Bhatia. All rights reserved.